

SECURITY ANALYSIS OF PERVASIVE WIRELESS DEVICES

Physical and Protocol Attacks in Practice



DISSERTATION

for the degree of Doktor-Ingenieur
Faculty of Electrical Engineering and Information Technology
Ruhr-University Bochum, Germany

Timo Kasper
Bochum, September 2011

PhD thesis:

**Security Analysis of Pervasive Wireless Devices
Physical and Protocol Attacks in Practice**

published in November, 2012

Copyright © 2011, 2012 by Timo Kasper. All rights reserved.

The author hereby grants to the Ruhr-Universität Bochum permission to reproduce and publicly distribute paper and electronic copies of this thesis document in whole or in part.

To Michael Lev Chrastina.

Author's contact information:

`timo.kasper@rub.de`

`www.timokasper.de`

Thesis Advisor: **Prof. Dr.-Ing. Christof Paar**
Ruhr-University Bochum, Germany

External Referee: **Prof. Dr. Srdjan Čapkun**
ETH Zürich, Switzerland

Thesis submitted: July 13, 2011

Thesis defense: September 12, 2011

*“It’s all about give and take,
what goes around comes around,
and everything happens for a reason.”*

Abstract

Wireless embedded devices are ubiquitous in our everyday life. They include all-round tools such as smartphones or portable computers, which typically feature high-performance processors, graphic displays, and other means for user interaction, consuming high amounts of energy and thus possessing a short battery life. In contrast to these sophisticated tools, various simpler, low-power devices have become widespread that are tailored to serve one particular purpose (e.g., opening a door) at a low cost: Passive Radio Frequency Identification (RFID) technology requires no battery and is used in the supply chain (barcode replacement), in the medical sector (pace makers), as a countermeasure against product piracy (ink cartridges), or against car theft (immobilizers). Contactless cards, such as the electronic passport, represent the most powerful variant of RFID and are widely employed for comfortable ticketing, contactless payments, identification, and access control purposes. Active, battery-powered remote controls enable greater operating ranges and often form the basis of remote keyless entry (RKE) systems that have already replaced conventional mechanical keys in many applications. With respect to security, special attack vectors apply to all systems communicating via a wireless interface, since manipulations are often feasible from a distance, without physical access to the target. These particular security threats include monitoring the communication, i.e., eavesdropping and spying out sensitive information; replay attacks and Man-in-the-Middle (MITM) attacks; modifying the content stored on the devices; and Denial-of-Service (DoS) attacks that aim at disturbing the intended functionality of a wireless system. As a countermeasure, diverse cryptographic schemes are prevalently implemented to secure the radio link and prevent fraud.

Circumventing the protection schemes typically requires extracting a secret cryptographic key by means of cryptanalysis. Classical cryptanalysis, i.e., based on mathematical models, has cleared the way for mathematically secure, peer-reviewed ciphers. Today many commercial products nonetheless still rely on weak, proprietary algorithms. Meanwhile, physical cryptanalysis or implementation attacks have emerged that target the actual realization of a cryptographic primitive or other security concept in electronic circuitry. The very powerful techniques, e.g., side-channel analysis, enable to extract secret keys even from those products that already implement mathematically secure ciphers. The consequences of a key-recovery can be considerably more severe in the context of contactless systems, compared to contact-based systems, as demonstrated in this thesis: After summarizing potential risks and corresponding attacks, the main part describes practical security analyses of a selection of today's wireless applications and illustrates the security flaws implemented by the designers at the example of different commercial products. The often dramatic impact of a key-recovery by means of side-channel analysis, fault injection attacks, or exhaustive key search is demonstrated by security-analyzing a Mifare Classic based payment system, secure cryptographic contactless cards (Mifare DESfire), KEELOQ RKE systems, and the first version of the electronic passport issued in Germany. To rate the feasibility and complexity of attacks and to determine the relevance of the respective security risks, a purely theoretical analysis is not sufficient. Thus, low-cost tools for practical security analyses are developed in this thesis, e.g., custom reader devices and emulators for contactless cards.

The tools are first employed for analyzing a contactless payment application, which is based on Mifare Classic technology, with more than one million issued cards. This thesis illustrates an implementation for extracting the cryptographic keys of the cards in seconds and then, how to (wirelessly) modify the content of any card in the system. An unskilled adversary can in consequence produce virtual currency on her card (or others' cards), without paying real money, and carry out payments with a fraudulently increased credit balance. The main flaw in this system is a fatal lack of key derivation, i.e., the same secret keys are used for all cards in the system.

More secure contactless cards relying on well-established ciphers might at first glance seem ideal as a direct replacement for the Mifare Classic cards, e.g., in the payment system. Mifare DESfire cards are widely used for access control, payment, and identification purposes, since they provide Triple-DES encryption and hence their secret cryptographic keys cannot be deduced by mathematical methods. However, this thesis demonstrates that Mifare DESfire MF3ICD40 cards can be broken by means of a side-channel attack: The secret keys are revealed by analyzing the information leakage contained in the EM (Electromagnetic) emanation during the cryptographic computations. Consequentially, circumventing systems that employ these or similar contactless cards relying on theoretically secure ciphers has become a real threat.

Further highly efficient side-channel attacks target KEELOQ RKE systems that are widespread in cars and garages. The manufacturer key contained in a KEELOQ receiver is recovered by analyzing only a single measurement of the power consumption. This secret key enables an adversary to completely break the security of the whole RKE system: After eavesdropping of a genuine code she can open (and close) doors that are secured by the KEELOQ code-hopping scheme and create a clone of the original remote control, from a distance and without leaving physical traces. In addition to power-analysis attacks on KEELOQ remote controls and receivers, an implementation of exhaustive key-search on the cryptographic hardware COPACOBANA is presented.

Finally, the first generation of electronic passports issued in Germany is examined: The secret keys used for encrypting the communication can be extracted by means of a brute-force attack, as implemented in this thesis. Due to an unfavorable key derivation, the private data of the passport holder can be eavesdropped and decrypted from a distance, despite the highly secure cryptographic primitives integrated into the passport.

This thesis illustrates an evident lack of security in many commercial wireless systems, especially with respect to physical cryptanalysis. Pinpointing the flaws in the realization of several contactless applications shall help developers and designers to prevent similar wrong or naive usage of cryptography and thereby improve future products. The minimal requirements for the adversary, i.e., what skills and funds are required to break the device or system, are clarified for all the presented attacks. The often dramatic implications are illustrated, the privacy protection of individuals using modern contactless technology is highlighted, and other security-related aspects are discussed. Where applicable, best-practice countermeasures are proposed that can effectively hinder the attacks.

Keywords

Security, Cryptography, Wireless, RFID, Tools, Implementations, Real-World Attacks, Side-Channel Analysis, Keeloq, Mifare, DESfire, Electronic Passport, Access Control, Contactless Payments, Identification

Kurzfassung

Sicherheitsanalyse ubiquitärer drahtloser Anwendungen — Physikalische und Protokoll-Angriffe in der Praxis

Portable drahtlose Geräte sind ein allgegenwärtiger Bestandteil des täglichen Lebens. Diese Dissertation untersucht Schwachstellen der Sicherheit verschiedener kommerzieller Produkte im Hinblick auf Protokollangriffe und physikalische Attacken, insbesondere so genannter Seitenkanalanalysen. Dazu wird zunächst spezielle, kostengünstige Hardware zur Schwachstellenanalyse von RFID (Radio Frequenz Identifikation) Geräten entwickelt, einschließlich eines elektronischen Emulators für kontaktlose Smartkarten. Mit Hilfe dieser Hardware wird ein weit verbreitetes kontaktloses Bezahlungssystem untersucht, das auf kryptografisch schwachen Mifare Classic Karten basiert. Dabei werden eklatante Sicherheitsmängel festgestellt: Das Extrahieren der kryptografischen Schlüssel aus einer dieser Bezahlkarten (mit der entwickelten Hardware) ermöglicht das Modifizieren von Geldbeträgen beliebiger Karten im System, da fatalerweise alle Karten denselben geheimen Schlüssel benutzen. Dies ermöglicht eine Reihe praktischer Angriffe. Beispielsweise ist das Bezahlen mit modifizierten Bezahlkarten bzw. dem Emulator ebenso problemlos möglich wie das Umwandeln von gefälschten digitalen Geldbeträgen in echtes Bargeld.

Eine naheliegende Aufrüstung des Bezahlungssystems auf modernere Mifare DESfire Karten bringt ohne Änderungen der weiteren Systemparameter keine spürbare Verbesserung: Diese Arbeit demonstriert, wie die Sicherheit dieser kontaktlosen Karten durch Auswertung der während der kryptografischen Berechnungen entstehenden elektromagnetischen Abstrahlung ausgehebelt werden kann: Mittels spezieller RFID-Seitenkanalanalyse wird der 112-bit Schlüssel der zur Authentifizierung und Verschlüsselung in DESfire Karten eingesetzten Triple-DES Chiffre vollständig bestimmt. Mit dem Angriff können sämtliche Schlüssel einer Mifare DESfire MF3ICD40 Karte extrahiert und infolgedessen der Inhalt der Karte beliebig ausgelesen und modifiziert werden.

Im Gegensatz zu den passiven, kontaktlosen Karten verfügen aktive Handsender über eine eigene Energiequelle in Form einer Batterie. Die entsprechend größere verfügbare Rechenleistung würde den Einsatz starker Kryptografie ermöglichen, was jedoch in der Praxis häufig nicht umgesetzt wird. Anhand der weltweit eingesetzten „KeeLoq“ Funktüröffner wird demonstriert, wie die Sicherheit dieses aktiven Systems vollständig umgangen werden kann. Durch Auswertung des Leistungsverbrauchs der Handsender können in deren Hardware gespeicherte geheime Schlüssel in wenigen Minuten aus ca. 10–30 Messungen ermittelt werden. Eine optimierte Seitenkanalanalyse der Software eines Empfängers offenbart nach nur einer einzigen Stromkurvenmessung den kryptografischen Herstellerschlüssel des Funktüröffnersystems. Mit Kenntnis des Herstellerschlüssels wird ein verheerender Lauschangriff möglich, der aus nur einem abgefangenen Funkcode alle weiteren gültigen Codes zu berechnen erlaubt. Ein Angreifer kann somit aus der Entfernung mit KeeLoq gesicherte Objekte u. A. beliebig öffnen und schließen. Für einen weiteren Angriff durch erschöpfende Schlüsselsuche dient eine Implementierung auf kryptanalytischer Hardware. Als eine mögliche Alternative zu KeeLoq wird eine neu entwickelte, seitenkanalresistente Implementierung eines Funktüröffnersystems vorgestellt.

Der letzte Teil der Dissertation beschäftigt sich mit dem elektronischen Reisepass (ePass). Das per RFID auslesbare Ausweisdokument sichert die privaten Daten des Inhabers u. A. durch das Basic Access Control (BAC) Protokoll ab. BAC verhindert ein unbefugtes Auslesen des Passes, indem es den Zugriff auf die im Pass gespeicherten Informationen nur authentifizierten Lesegeräten (z.B. Grenzkontrolle) gestattet. Die anschließende drahtlose Kommunikation wird zusätzlich verschlüsselt, um die Vertraulichkeit der Daten zu gewährleisten. Jedoch ist auch das BAC Verfahren wegen einer schwachen Ableitung der kryptografischen Schlüssel angreifbar: Zwei Implementierungen auf der kryptanalytischen Hardware COPACOBANA erlauben in praktischen Angriffsszenarien aus abgehörter verschlüsselter Kommunikation mit dem ePass in Sekunden die kryptografischen Schlüssel des BAC zu bestimmen. Mit Kenntnis der Schlüssel können die persönlichen Daten sowie das Foto des Passinhabers entschlüsselt, Bewegungsmuster von Individuen erstellt oder ein Alarm bei Detektierung eines bestimmten ePasses ausgelöst werden.

Die Ergebnisse dieser Arbeit verdeutlichen, dass die Sicherheitsmechanismen kommerzieller drahtloser Systeme oft mangelhaft umgesetzt sind und häufig nicht dem Stand der Wissenschaft entsprechen. Damit wird ein wichtiger Beitrag zur zukünftigen Entwicklung von praktisch sicheren Geräten geleistet. Besonders beim Schutz gegen Seitenkanalangriffe, doch auch bei der Umsetzung des Gesamtsystems, besteht erheblicher Nachbesserungsbedarf in vielen Produkten.

Schlagworte

Sicherheit, Kryptografie, Funksysteme, RFID, Implementierungen, Werkzeuge, Seitenkanal, Praktische Angriffe, kontaktlos Bezahlen, Funktüröffner, Zugangskontrolle, elektronischer Reisepass, Identifikation, Datenschutz, Datensicherheit, Lauschangriff

Acknowledgements

DURING the past more than six wonderful years in the **EMSEC** (formerly **COSY**) group I have benefitted from countless collaborations with brilliant researchers. I always enjoyed how open-minded we shared ideas and work, painful failure and great success, and I improved a lot by learning other persons' viewpoints and strategies of solving problems. Amongst my colleagues I have made many new friends, without whom my **PhD** thesis would not have come into being. Thus, my deep and sincere gratitude goes to all hands and brains that have been accompanying, helping, and supporting me during this marvelous time.

Let's start with Christof Paar, incarnating the true Doktorvater¹, who has always been a very positive, inspiring, and motivating mentor, giving me wise advice even far beyond the scope of our research. He had to my own surprise accepted me as a (chaotic) **PhD** student, in the following provided an excellent training as a scientist and lecturer, introduced me to the community of cryptographers, and enabled my participation in lots of conferences and workshops worldwide — including sudatory Karate training at 7am on the beach of the University of California. Thank you for your great assistance, patience, and the valuable feedback to our papers and this thesis. In short: Super Arbeit, 1000 Dank!

I would like to express my gratitude to my thesis committee, for interesting discussions and useful feedback. Special thanks to my second referee Srdjan Čapkun, who spontaneously agreed to review my thesis and skipped a conference for attending my **PhD** defense. I deeply appreciate the collaboration with my coauthors, who all contributed precious parts to my thesis. In particular, I would like to thank our power-analysis artist Amir Moradi, who is magically performing practical attacks on any target, with astonishing setups and methods. In excess of the impressive joint research on KeeLoq, thank you for introducing the Persian culture to me, serving the best Persian food (and fumigating Axel's flat in Hamburg, in an attempt to have a barbecue), and being a true, reliable friend. Likewise, very special thanks go to my long-term office mate David Oswald, the implementation attack guru and grand master of DESfire, for lots of productive collaborations including our security analyses of contactless smartcards and developing highly effective hardware and software for side-channel attacks. I admire his calmness, despite the pandemonium that I am causing in our office, and greatly appreciate our uncomplicated way of cooperating. Thank you for having lots of fun at various conferences, spectacular business trips with the measurement vehicle, and patiently answering the incoming phone calls of my grandmothers.

Before David joined our group, my dear colleague Kerstin Lemke-Rust sat next door and explained to me, the complete newbie, the art of writing a scientific paper: Thank you for voluntarily advising and mentoring me, taking your time for teaching me how to work accurately, and for later on opening the great opportunity to teach your students in Sankt Augustin. Moreover, I would like to thank Thomas Eisenbarth (who has escaped to the USA) for his contribution to the analysis of KeeLoq and many enjoyable evenings in Bochum's Bermuda Triangle, and Ingo von Maurich for bringing the Chameleon to life. *Díky moc* also to Martin Novotný — working in the “Czech branch of the **HGI**” at **FIT** in Prague — for illustrating me the secrets of **FPGAs**, for coding KeeLoq into his best friend (the **COPACOBANA**), and for introducing Becherovka to our group. I would further like to express my deep gratitude to those colleagues who proofread some parts of this thesis: Tim Güneysu, who can (besides securing hardware) with virtuosity play piano and bass guitar at the same time; Susanne Engels, the

¹German for **PhD** supervisor, literally “father of doctor”

Acknowledgements

lovely voice of **EMSEC** every Tuesday night and on our Christmas parties; Markus Dürmuth, who is bravely tolerating our music during his nights shifts; Oliver Mischke, our rock climbing trainer wearing the most sophisticated shoes in our group; and — last but not least — my sapient brother Markus Kasper, who ultimately explained my thesis to me (after I finished writing it) and coached me, together with Ralf, Ingo, and Susi, during the preparations for my **PhD** defense. Thanks a lot for all your efforts and extremely valuable feedback, hints, and improvements!

Due to my participation in various projects that are not directly related to my thesis I had the chance to enjoy joint work with many other researchers. Especially, I would like to thank my coauthors of respective **COSY/EMSEC** collaborations: Andrej Bogdanov, who brought up the KeeLoq topic in our group and whom I owe my only (shame on me) **CHES** publication; the almighty Dario Carluccio, formerly advisor of my Diploma thesis and one of the few men on earth who can successfully handle the hazardous combination of IT security and punk rock; Alessandro Barengi, who subtly worked out the missing link for breaking the Xilinx bitstream encryption; Timo Gendrullis, who survived my supervision and the unforgettable, awkward lessons of Sean O’Neill about seducing women with chocolate perfume; Ralf Zimmermann, who is very slowly riding on very fast motor bikes; and Andy Rupp, to my knowledge the only passionate fisherman ever at the **HGI** — working, talking, and partying with Andy was always a great pleasure, because we had a similar quality standard and the same wavelength. Special thanks to Andy and Dario for the legendary barbecue competition at the river Ruhr: swing grill, fired with beech wood, vs. Dario’s portable gas grill, a veritable neck-and-neck race. Moreover, I thank my coauthors “Signore Facebook” Roberto Avanzi, the side-channel experts Jörn-Marc Schmidt and Ilya Kizhvatov, and Michael Tunstall, who introduced us to the best bars in Bristol.

I am very grateful to all my former and new colleagues and friends at **COSY**, **EMSEC**, **SHA**, and **FIT**, for the fantastic working atmosphere and sharing their leisure time with me. Thanks to the senior researchers that already had been at **COSY** when I joined the group and left much too early, in particular: Jan Pelzl, teaching me the importance of soft skills and not to forget the fun at work; Kai Schramm, with whom I experienced thrilling outlaw adventures in France and Spain, playing percussions on the beach (and a special type of piano in the office); Sandeep Kumar, who has settled down in the Netherlands; Marko Wolf, meanwhile also living abroad, in Bavaria; and Bodo Möller who phenomenally played the piano at the **RUB** Jazz sessions and at conferences. I would like to thank Axel Poschmann (living in exile in Singapore), the joint inventor of the famous **COPACOBANA** song, for a really funny rump session talk in Santa Barbara and exploring San Francisco with me, while singing the new Barack Obama song. Further sincere thanks to all my (relatively) new colleagues, namely Georg Becker, who assisted me during the Embedded Processors lecture; Benedikt Driessen, probably the most religious person at **EMSEC**, who is secretly producing rap music from old IKV exams; special thanks to Stefan Heyse, whose organism is running solely off coffee and cigarettes, also for exchanging the injection nozzles of the TiMobil; Gesine Hinterwalder, for the wonderful time in Massachusetts, swimming in the Connecticut River, and showing us Boston; Elif Bilge Kavun, the new center forward of our **EMSEC** soccer team, for her super cheerful nature and spreading a joyful air around her; Thomas Pöppelmann, organizing the brewery visit for the Ecrypt summer school; Daehyun Strobel, our former bass player; and Tolga Yalcin, bringing delicious Baklava and turkish honey to **EMSEC**. It is a great pleasure to work with you all.

Behind the curtains, dealing with all complicated organizational and administrative tasks, is our kind-hearted team assistant Irmgard Kühn — the real boss and mother of **EMSEC**, who

is with a steady hand establishing law and order in our group. Thank you, for always having an open ear and helping with all my various requests, for refreshing chats about other topics than electrical engineering (culture!) in the coffee breaks, and for regularly reminding me of tasks and events that I might otherwise have forgotten. Thanks to our technical assistant Horst Edelmann for setting up SVNs on Sunday night, ordering new research toys for us, regularly fixing my computer problems, helping to get rid of all those Trojan horses and viruses, and (every now and then) enjoying an after-work beer with me at the Jazz or Blues session.

Furthermore, I thank Thorsten Schütze for the great collaboration with Bosch and Andreas Hoheisel, who had probably been a bit über supervised by me and Thorsten during his formidable Master's thesis. My sincere gratitude also goes to all the other students who have chosen to be advised by me, in particular to those, whose results were partly assimilated in my thesis: Michael Silbermann, who considerably helped in analyzing payment cards; Yifei Liu, who implemented the key-search on electronic passports; and Vincent Immler for his great research on 125 kHz transponders and making dodgy advertisement for **EMSEC** plus good jokes about me behind my back. Moreover, I would like to thank Falk Schellenberg for taking lots of project-related workload off me, while self-reliantly producing amazing research results, as well as Dennis Rochel and all other chefs who prepared our routine barbecues in the summertime, on the balcony of the IC building.

Thanks to everyone for regularly having extended lunch breaks, waiting for me, always being the last one to finish my meal (some wagging tongues claim that I am talking too much, but I think it is just about savoring the delicious food of the **RUB** Mensa, e.g., “Rinderhacksteak” or chicken with “tomatisierte Sauce”). Thank you for not killing me because of my singing, playing the guitar, and hammering the drums in the archive (while you were trying to focus on the next paper) and for every collective night shift before a paper deadline (despite our strong efforts to finalize it earlier), when all lights of the university are extinguished, except for certain windows at **EMSEC**.

Special thanks to Gerd Kaerlein for patiently and free of charge helping me to catch up with the mathematics I lacked since school, and thereby laying the foundations for me becoming an engineer; to André Kasper for donating my famous old Nokia mobile phone; to Judith Kühn for her delightful cheekiness, outstandingly beating the drums, and lowering the average age of our band to an acceptable level; to Marian Šuch, the enthusiastic scientist, and the other Dvůrákovci and Medvečskýs hosting me in my second home in Prague, for opening my eyes to completely different viewpoints about stuff and many exciting discussions through the night; to Radka Lemmen for teaching me Czech; and to Stephen and Julie Wainwright for spoiling me and my English during my semester abroad, and ever since, in beautiful South Yorkshire. I am deeply grateful to my parents, Eva and Kalle Kasper, for encouraging me to study, for their love and support, financing my student's life, and for continuously supplying me with beer from the tap — until the bitter end! In this context, I also thank Moritz Fiege for many exhilarant moments and for relaxing my mind while writing this thesis. Finally, my deepest gratitude goes to the charming Adéla Chrástínová for our gorgeous son and for being a wonderful, loving mother.

What, already three pages? Oops, it happened again. I assure that I tried to keep it short.

Glück auf!

Dr. Kasper



Table of Contents

Abstract	ix
Kurzfassung	xi
Acknowledgements	xiii

List of Abbreviations	xxv
------------------------------	------------

1 Introduction	1
1.1 Pervasive Wireless Technology	2
1.1.1 Applications of Passive (RFID) Devices	4
1.1.2 Applications of Active Devices	6
1.1.3 Wireless Devices Analyzed in this Thesis	7
1.2 Security Analyses of Wireless Systems	7
1.2.1 Security (only) from Obscurity	8
1.2.2 Cryptanalysis and Cryptography: About Attackers and Designers	9
1.2.3 Towards more Security	10
1.3 Outline of this Thesis	11
2 Background: Wireless Embedded Systems	17
2.1 Classification of Attackers	17
2.2 Attacks on Wireless Embedded Devices and Related Work	18
2.2.1 Sniffing	19
2.2.2 Replay Attacks and Spoofing	19
2.2.3 Cloning and Manipulating Genuine Devices	20
2.2.4 Man-in-the-Middle	20
2.2.5 Denial of Service (DoS)	21
2.2.6 Tracking	22
2.2.7 Detection	23
2.2.8 Malware	24
2.2.9 Implementation Attacks	24
2.3 Authentication Schemes for Wireless Tokens	25
2.3.1 Fixed-Code Protocol	26
2.3.2 Rolling-Code Protocol	26
2.3.3 Synchronization-Code Protocol	29
2.3.4 Challenge-Response Protocol with Symmetric Cryptography	30
2.3.5 Mutual Authentication Protocol with Symmetric Cryptography	31
2.3.6 Cipher Block Chaining Mode of Operation	32
2.3.7 Key Management of Wireless Tokens	33
2.4 Passive RFID: Operation Principle	34
2.4.1 Long-Range RFID Systems	35

Table of Contents

2.4.2	Short-Range RFID Systems	35
2.5	ISO / IEC 14443 Standard for Contactless Smartcards	36
2.5.1	Overview of ISO 14443	36
2.5.2	Unique Identifier	37
2.5.3	Physical Layer Type A	37
2.5.4	Frame Formats	39
2.5.5	Timing Specifications	40
2.5.6	Anticollision and Initialization Protocol	40
2.5.7	Other Standards for 13.56 MHz	41
2.6	Examples of Contactless Smartcards	42
2.6.1	Innovatron Calypso	42
2.6.2	Legic Identsystems Cards	43
2.6.3	Electronic Passport	43
2.6.4	NXP Mifare Cards	44
2.7	Typical Ranges for Activation and Eavesdropping	45
2.7.1	Ranges of Contactless Smartcards	46
2.7.2	Ranges of Active Wireless Devices	46
3	Background: Physical Cryptanalysis	49
3.1	Cryptanalysis: A Classification of Different Techniques	49
3.1.1	Classical Cryptanalysis	49
3.1.2	Implementation Attacks	51
3.1.3	Types of Attacks Executed in this Thesis	52
3.2	Introduction to Side-Channel Analysis	53
3.2.1	Timing Analysis	53
3.2.2	The Concepts behind Power Analysis	55
3.2.3	Countermeasures against Side-Channel Analysis	61
3.2.4	Processing Measurements	62
3.3	Steps of a Black-Box Analysis	63
3.3.1	Collecting Available Information	64
3.3.2	Profiling by Means of Power Measurements	64
3.3.3	Power Analysis with a Known Key	65
4	Tools for Security Analysis	67
4.1	Communicating with Contactless Smartcards	68
4.1.1	Reader	69
4.1.2	Fake Tag	71
4.1.3	Software for the RFID tool	74
4.2	Communicating with other Wireless Devices	75
4.2.1	USRP and GNURadio	75
4.2.2	RFID Reader for 125 kHz	76
4.3	Data Acquisition	76
4.3.1	Picoscope	76
4.3.2	Probes	78
4.3.3	Amplifiers	79

4.4	Contact-Based Analyses and Fault Injection	79
4.4.1	Extensions for Contact-Based Smartcards	80
4.4.2	Extensions for Fault Injection	80
4.5	COPACOBANA	80
4.5.1	Hardware Architecture	81
4.5.2	Computing with COPACOBANA	81
4.5.3	Applications of COPACOBANA	82
5	Chameleon: A Versatile Emulator for Contactless Smartcards	83
5.1	Introduction	83
5.1.1	Previous Work: Existing RFID Emulators	84
5.1.2	Contribution and Organization of this Chapter	84
5.2	Hardware Setup	85
5.3	Mifare Contactless Cards	86
5.3.1	Mifare Ultralight	86
5.3.2	Mifare DESfire EV1 Cards	87
5.4	Software Implementations on Chameleon	89
5.4.1	Mifare Ultralight Emulator	89
5.4.2	Mifare Classic Emulator	90
5.4.3	Mifare DESfire (EV1) Emulator	91
5.5	Practical Results	92
5.5.1	Mifare Ultralight Emulation	92
5.5.2	Mifare Classic Emulation	92
5.5.3	Mifare DESfire (EV1) Emulation	93
5.6	Real-World Attacks with Chameleon	93
5.6.1	Payment System	94
5.6.2	Copy-and-Print System	94
5.6.3	A Vulnerable Access Control System	94
5.7	Conclusion	96
6	Pitfalls of a Real-World Contactless Payment System	99
6.1	Introduction	99
6.1.1	ID-Card	100
6.1.2	Contribution and Outline of this Chapter	101
6.2	Contactless Payment Systems	101
6.2.1	Components	101
6.2.2	Off-Line vs. On-Line Systems	102
6.3	Mifare Classic Cards	102
6.3.1	Memory Structure	103
6.3.2	Command Set	104
6.3.3	The CRYPTO1 Cipher and Communication Protocol	105
6.4	A Survey on Attacking Mifare Classic	108
6.4.1	Keystream Recovery	109
6.4.2	Key Recovery from Genuine Authentications	109
6.4.3	Card-Only Key Recovery	110

Table of Contents

6.5	Attack Hardware and Software	112
6.5.1	Software for Security Analyses of Payment Cards	113
6.6	Tampering with a Real-World System	116
6.6.1	Recovering the Secret Keys	116
6.6.2	Reverse Engineering the Card Content	117
6.6.3	Practical System-Level Tests	118
6.6.4	Summary of the Vulnerabilities	121
6.7	Exploiting the Weaknesses: Attacks and Economic Implications	122
6.7.1	General Scenario	122
6.7.2	Duplicates based on Blank Cards	123
6.7.3	Exact Clones	124
6.7.4	State Restoration and Charging Cards	125
6.7.5	Creation of New Cards and Unlimited Payments	126
6.7.6	Denial-of-Service	126
6.7.7	Cashback Attack	127
6.7.8	Pickpocketing with a Relay Attack	128
6.7.9	Estimating the System Wide Damage	128
6.8	Countermeasures	129
6.8.1	Organizational Measures and Back-End Improvements	130
6.8.2	Key Diversification	130
6.8.3	Securing the Data on the Card	131
6.8.4	Rolling-Code Scheme	131
6.8.5	Summary of the Countermeasures	132
6.9	Conclusion	133
7	EM Side-Channel Attacks on Cryptographic Contactless Smartcards	135
7.1	Introduction	135
7.1.1	Related Work about Side-Channel Analysis of RFIDs	136
7.1.2	Outline of this Chapter	136
7.2	Mifare DESfire	137
7.2.1	Mifare DESfire Authentication Protocol	137
7.3	Methods, Models, and Setup for the EM Analysis of RFIDs	138
7.3.1	Efficiently Implementing a CPA	139
7.3.2	Modeling the Power Consumption of RFID Devices	140
7.3.3	Measurement Setup and Analogue Filters	141
7.3.4	Software for the Power Analysis of RFIDs	147
7.4	A Real-World EM Attack on Mifare DESfire MF3ICD40	149
7.4.1	Communicating with the DUT	150
7.4.2	Basic Profiling	150
7.4.3	Probe Selection and Positioning	151
7.4.4	Digital Processing and Alignment	153
7.4.5	EM-Analysis of Cryptographic Contactless Cards	153
7.5	Conclusion	159
7.5.1	Outlook	159

8	On the Power of Power Analysis: Breaking KeeLoq Remote Keyless Entry Systems	161
8.1	Motivation	161
8.1.1	KeeLoq	162
8.1.2	Contribution and Outline of this Chapter	162
8.2	Background	163
8.2.1	KeeLoq Cipher	163
8.2.2	KeeLoq RKE Systems	165
8.2.3	Key Management	166
8.2.4	History of Attacking KeeLoq	168
8.3	DPA of KeeLoq	169
8.3.1	Overview of the DPA	169
8.3.2	Building a Scalable CPA for KeeLoq	170
8.3.3	CPA of KeeLoq in Hardware	172
8.3.4	CPA of KeeLoq in Software	174
8.4	SPA of KeeLoq	176
8.4.1	Software Implementations of KeeLoq	176
8.4.2	Visual Inspection of Power Traces	178
8.4.3	A Non-Heuristic Method for SPA Attacks	178
8.4.4	Developing an SPA Attack for KeeLoq	181
8.4.5	Comparison of DPA and SPA	184
8.5	Attacks and Implications	185
8.5.1	Cloning a Remote Control	185
8.5.2	Recovering a Manufacturer Key	186
8.5.3	Cloning a Remote Control by Eavesdropping	187
8.5.4	Denial of Service	188
8.5.5	Revealing Device Keys with Brute-Force Attacks	189
8.6	A Power-Analysis Resistant RKE System	192
8.6.1	Introduction and Related Work	192
8.6.2	PRNG-Based Dynamic Code Generator	193
8.6.3	Security Considerations	196
8.6.4	Implementation	198
8.6.5	Summary of the New RKE System	199
8.7	Conclusion	199
9	Electronic Passport: Cracking Basic Access Control Keys with COPACOBANA	203
9.1	Introduction and Related Work	204
9.2	The Basic Access Control Protocol	205
9.2.1	Key Derivation from the MRZ	206
9.2.2	The Mutual Authentication Protocol	206
9.3	The Threat Model	208
9.4	The Key Search	209
9.4.1	The First Approach based on Two-Channel Communication	209
9.4.2	The Second Approach based on Forward-Channel Communication	210
9.4.3	Complexity Analysis of the Key Space	210

Table of Contents

9.5	Practical Implementation on COPACOBANA	215
9.5.1	Details of the Hardware Architecture	215
9.5.2	Practical Results	218
9.6	Other Implementation Options	219
9.6.1	Software Implementation	219
9.6.2	New FPGA Key Search Machines	219
9.7	Conclusion	219
10	Conclusions and Future Research	221
10.1	Lessons Learnt	224
10.2	Future Directions	225
A	Statistical Tests for Timing Analysis	227
A.1	Two Sample T-Test	227
A.2	Fisher's F-Test	228
B	Side-Channel Resistance in the Context of AES	231
B.1	Introduction	231
B.2	The Masking Countermeasure	232
B.2.1	Additive Masking	233
B.2.2	Transformed Masking Method	234
B.2.3	Perfectly Masking AES Inverter Against First-Order Side-Channel Attacks	236
B.2.4	Combined Masking in Tower Fields	238
B.2.5	Masking the MixColumns Transformation	240
B.3	Countermeasure: Randomization	241
B.3.1	Shuffling	241
B.3.2	Execution of Dummy Operations	242
B.3.3	Combining Shuffling and Dummy Operations	242
B.4	Generating Random Numbers for the Countermeasures	243
B.5	Comparison	243
C	125 kHz RFID: A Customized Reader and Details about Commercial Transponders	247
C.1	125 kHz Transponders	247
C.1.1	Physical Layer	247
C.1.2	Research on 125 kHz Devices	248
C.2	A Customized Reader for 125 kHz	249
C.2.1	Outline of the reader	249
C.3	Overview of Common 125 kHz Transponders	250
C.3.1	125 kHz Protocols	254
C.4	The HITAG 2 Stream Cipher and Transponders	255
C.4.1	Modes of Hitag 2	255
C.4.2	Memory Map of HITAG 2 Transponders	256
C.4.3	HITAG 2 Stream Cipher	257
C.4.4	Keystream Generation	260
C.4.5	HITAG 2 Protocol	262

C.5	Practical Results for 125 kHz RFIDs	264
C.5.1	An Exemplary 125 kHz Communication	264
C.5.2	Reverse-Engineering the Sokymat Magic Protocol	265
C.5.3	Analyzing Car Keys	268
C.6	Future Research for 125 kHz Transponders	268
D	Extensions for Implementation Attacks on Smartcards	269
D.1	A Modified Chipcard Reader for Power Analysis	269
D.2	Chipcard Adaptor for Emulating and Side-Channel Analysis	270
E	Extensions for Fault Injection	271
E.1	Optical Fault Injection	271
E.2	Electro-Magnetic Fault Injection with Sparks	272
F	Toolchain for Realizing Software Implementations	273
F.1	Atmel AVR Studio	273
F.1.1	WinAVR	273
F.2	AVR Dragon Programmer	273
F.3	Microsoft Visual Studio	275
F.4	HTerm	275
G	RFID-Tool Software: Detailed Insights	277
G.1	Low-Level Functions	277
G.1.1	EM 4094 Library	277
G.1.2	FT245R Library	280
G.1.3	Timers and Utilities	281
G.2	Operating System and Modes	281
G.2.1	Menu Mode	282
G.2.2	Passive Man-In-The-Middle (MITM) (Relay) Mode	282
G.2.3	Stand-Alone Passive MITM Mode	282
G.2.4	Active MITM Mode	282
G.2.5	Relay Output Mode	282
G.2.6	Generic Reader Mode	283
G.2.7	Generic Card Mode	283
G.2.8	Mifare Classic Card Mode	284
G.2.9	Timing Mode	285
G.3	Software for Manipulating Mifare Classic	285
G.3.1	Embedded Software	285
G.3.2	Notes to the Modes for Manipulating Mifare Classic Cards	288
G.3.3	Details of the Desktop Software	289
G.3.4	Timing of a Straightforward CRYPTO1 Implementation	294
H	Details of Attacking ID-Cards	297
H.1	Data stored on cards	297
H.2	Details of Practical System Level Tests	298
H.2.1	Cards Used	298

Table of Contents

H.2.2	Cloned Cards	298
H.2.3	Generating New Cards	299
H.2.4	Turning bits into Cash	299
H.2.5	Using a Clone After Cashback	299
H.2.6	Blacklisted ID Card	300
I	Details of Tools for SCA Attacks on Contactless Smartcards	301
I.1	Measurements and Practical Results	301
I.1.1	Performance of the Carrier Subtractor	301
I.1.2	Strange Response Times of a Mifare DESfire MF3ICD40	302
I.1.3	DOM Test	302
I.2	Usage of the CPA Software	302
I.3	Further Developed Circuits	303
I.3.1	Instrumentation Amplifier	303
I.3.2	Analog Filters	304
J	Example Code and Wiring Diagrams for RFID Readers and Emulators	305
J.1	Example Code for the Chameleon	305
J.2	Wiring Diagram of the Chameleon	306
J.2.1	Components of Chameleon	308
J.3	Wiring Diagram of the 125 kHz Reader	309
J.4	Wiring Diagram and Layout of the RFID Tool	310
	Bibliography	337
	List of Tables	340
	List of Figures	344
	Curriculum Vitae	345
	Publications	347

List of Abbreviations

3DES	Triple Data Encryption Standard
AA	Active Authentication
ADC	Analog to Digital Converter
ACK	Acknowledgement
AES	Advanced Encryption Standard
AID	Application Identifier
ANF	Algebraic Normal Form
APDU	Application Protocol Data Unit
API	Application Programming Interface
ASCII	American Standard Code for Information Interchange
ASK	Amplitude-Shift Keying
ASIC	Application Specific Integrated Circuit
ATQA	Answer To Request A
ATS	Answer To Select
BAC	Basic Access Control
BSI	Bundesamt für Sicherheit in der Informationstechnik
CBC	Cipher Block Chaining
CHES	Cryptographic Hardware and Embedded Systems
CIA	Combined Implementation Attacks
CL	Cascade Level
CMRR	Common Mode Rejection Ratio
CMTF	Combined Masking in Tower Fields
CMOS	Complementary Metal Oxide Semiconductor
COPACOBANA	Cost-Optimized Parallel Code Breaker and Analyzer
COSY	Communication Security
CPA	Correlation Power Analysis

CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DECT	Digital Enhanced Cordless Telecommunications
DES	Data Encryption Standard
DEMA	Differential Electro-Magnetic Analysis
DC	Direct Current
DCM	Digital Clock Manager
DoS	Denial-of-Service
DIP	Dual Inline Package
DIMM	Dual In Line Memory Modules
DOM	Difference-of-Means
DPA	Differential Power Analysis
DST	Digital Signature Transponder
DUT	Device Under Test
EAC	Extended Access Control
ECB	Electronic Code Book
ECC	Elliptic Curve Cryptography
ECMA	European Computer Manufacturers Association
EDE	Encrypt-Decrypt-Encrypt
EEPROM	Electrically Erasable Programmable Read-Only Memory
EM	Electro-Magnetic
EMC	Electro-Magnetic Compatibility
EMSEC	Embedded Security
EOC	End Of Communication
EOF	End Of File
ePass	Electronic Passport
EPC	Electronic Product Code
EU	European Union
FDT	Frame Delay Time
FIPS	Federal Information Processing Standard
FIR	Finite Impulse Response

FIT	Faculty of Information Technology
FPGA	Field Programmable Gate Array
FSM	Finite-State Machine
FSK	Frequency Shift Keying
GND	Ground
GUI	Graphical User Interface
GPS	Global Positioning System
GPL	General Public License
GSM	Global System for Mobile Communications
HAC	Handbook of Applied Cryptography
HD	Hamming distance
HF	High Frequency
HGI	Horst Görtz Institute for IT Security
HLTA	HaLT type A
HVSP	High Voltage Serial Programming
HVG	High Voltage Generator
HW	Hamming weight
ICAO	International Civil Aviation Organization
IC	Integrated Circuit
ICC	Integrated Circuit Card
ID	Identifier
IDE	Integrated Development Environment
IFD	Interface Device
IGBT	Insulated Gate Bipolar Transistor
ISO	International Organization for Standardization
ISM	Industrial, Scientific, and Medical
IFF	Identify Friend or Foe
IIR	Infinite Impulse Response
IP	Intellectual Property
ISR	Interrupt Service Routine
ISP	In-System Programming

IV Initialization Vector

JTAG Joint Test Action Group

LED Light-Emitting Diode

LF Low Frequency

LFSR Linear Feedback Shift Register

LIW Listening Window

LSB Least Significant Bit

MAC Message Authentication Code

MF Medium Frequency

MIA Mutual Information Analysis

MITM Man-In-The-Middle

MOSFET Metal-Oxide Semiconductor Field-Effect Transistor

MRZ Machine Readable Zone

MRTD Machine Readable Travel Document

MSB Most Significant Bit

nPA New German ID Card

NACK Negative Acknowledgment

NIST National Institute of Standards and Technology

NLFSR Non-Linear Feedback Shift Register

NLF Non-Linear Function

NFC Near Field Communication

NRZ Non-Return-to-Zero

NOP No Operation

OOK On-Off-Keying

opamp operational amplifier

OS Operating System

OTP One-Time Programmable

PA Passive Authentication

PACE Password Authenticated Connection Establishment

PC Personal Computer

PCB Printed Circuit Board

PCD	Proximity Coupling Device
PhD	Patiently hoping for a Degree
PICC	Proximity Integrated Circuit Card
PIT	Programmable Identification Transponder
PLL	Phase Locked Loop
PMS	Perfectly Masked Squaring
PMM	Perfectly Masked Multiplication
PPC	Pulse Pause Coding
PPS	Protocol and Parameter Selection
PPSR	Protocol and Parameter Selection Request
PRNG	Pseudo-Random Number Generator
PSK	Phase Shift Keying
RADAR	Radio Detection And Ranging
RAM	Random Access Memory
RATS	Request for Answer To Select
REQA	REQuest type A
RISC	Reduced Instruction Set Computer
RF	Radio Frequency
RFID	Radio Frequency IDentification
RGT	Request Guard Time
RKE	Remote Keyless Entry
RNG	Random Number Generator
ROM	Read Only Memory
RSA	Rivest Shamir and Adleman
RTF	Reader Talks First
RUB	Ruhr-University Bochum
SAK	Select AcKnowledge
SAM	Square-and-Multiply
SCA	Side Channel Analysis
SDK	Software Development Kit
SNR	Signal to Noise Ratio

SHA	Secure Hash Algorithm
SQL	Structured Query Language
SOIC	Small-Outline Integrated Circuit
SPA	Simple Power Analysis
SPOF	Single Point of Failure
SRAM	Static Random Access Memory
TLU	Table Look Up
TTF	Tag Talks First
TMTO	Time-Memory Tradeoff
TMDTO	Time-Memory-Data Tradeoff
TMM	Transformed Multiplicative Masking
SOC	Start Of Communication
SHF	Superhigh Frequency
μ C	Microcontroller
UID	Unique Identifier
USB	Universal Serial Bus
USRP	Universal Software Radio Peripheral
VCP	Virtual COM Port
VHF	Very High Frequency
VLf	Very Low Frequency
UHF	Ultra High Frequency
UMTS	Universal Mobile Telecommunications System
VHDL	VHSIC (Very High Speed Integrated Circuit) Hardware Description Language
WLAN	Wireless Local Area Network
WUPA	Wake-UP A
XOR	Exclusive OR

Chapter 1

Introduction

WITH affordable technical solutions available on the market, contactless tokens have become ubiquitous in our everyday life. Wireless devices replace many items which possess (implied) security functionality: mechanical keys, identity cards, paper tickets, and contactless smartcards that are used instead of cash. Various physical objects are tagged and interconnected via Radio Frequency (RF) to form the “Internet of things”. Pervasive wireless technology has conquered consumer products, the medical sector, cars, and military equipment. While on the one hand new, convenient applications have become feasible and widely deployed, on the other hand new attack vectors have to be considered. For instance, augmenting a mechanical key with a wireless interface must not result in a weakened security of the access control system, i.e., a strong mechanical lock securing a door must not be easily circumvented via the electronics. This is where cryptography comes into play: Today’s authentication and encryption schemes can provide a very high level of security, if implemented appropriately.

However, wireless tokens are often limited with respect to their energy supply and hence their computational power. Furthermore, security measures often raise the production cost and hence an adequate cost-security ratio must be found according to the demands of each particular application. Designing a practically unbreakable wireless system requires great effort since not only the security of each component but also their interplay in the system as a whole needs to be considered. It is highly difficult, if not infeasible, to model an application such that all relevant threats are taken into account. Accordingly, the security of each real-world system must be practically investigated and evaluated in order to pinpoint vulnerabilities and fix them, if possible.

Example: Anti-Theft and Keyless-Entry Systems for Cars Car theft is one instructive example for the deployment and benefits of cryptography in wireless applications. Before microcontrollers were embedded into cars, a thief had to circumvent a mechanical lock to open the door and then start the engine via short-circuiting the ignition. In the 1980s the first wireless door openers were introduced for comfort reasons. These simple systems were often straightforward to circumvent, e.g., those communicating via infrared light could be spoofed by means of universal remote controls for televisions. Theft rates and the number of joyrides were constantly growing. As a consequence of the pressure put by insurance companies the industry reacted: improved, RF-based Remote Keyless Entry (RKE) and alarm systems became available that employ cryptographic measures to secure the vehicles. In the beginning of the nineties, immobilizers based on Radio Frequency IDentification (RFID) technology became widespread. A vehicle security directive of the European Commission [eu9] issued 1995 made these anti-theft

systems mandatory for new cars sold in Europe [WMMT99]. This resulted in a 16-year lasting decrease of theft rates in Germany, as illustrated in Fig. 1.1.

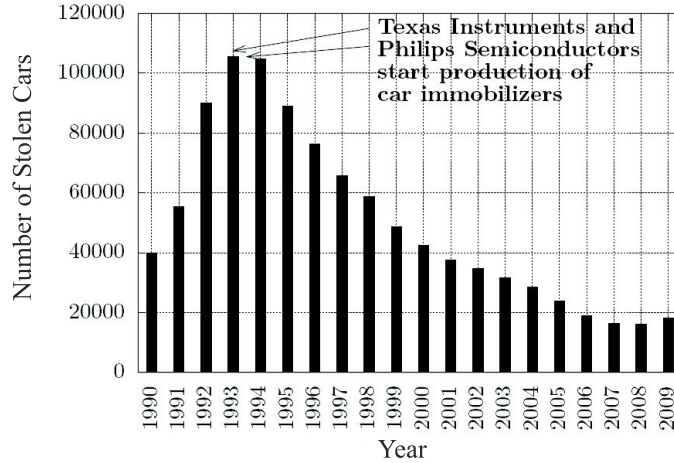


Figure 1.1: Car theft rate in Germany, 1990–2009 (after 1991 including former East Germany). Based on data published by the German Insurance Association (GDV).

Meanwhile, the security concepts became well-known to car thieves, even though the underlying ciphers and most of the cryptographic protocols were kept secret by the manufacturers. Those ciphers that have become known to the public, e.g., HITAG 2 and KEELOQ, turned out to be cryptographically weak and exploitable vulnerabilities were found by means of mathematical cryptanalysis. Due to the large economic benefit for criminals, corresponding attacks evolved and products for comfortably copying keys (or stealing cars) were developed, e.g., Gambit programmer or Tango programmer [sLL]. The majority of the automotive industry — instead of looking ahead and improving the systems — kept integrating the same archaic security mechanisms into new car generations, regardless of new attacks becoming feasible for unskilled adversaries. As a result, the lowest theft rate of 16,136 cars being stolen in 2008 increased 2009 by 12.9% for the first time after the introduction of the immobilizer. The rising trend continues with more than 42,000 vehicles being stolen in 2010 [Bun11]. Even though breaking the electronic protection mechanisms of modern cars demands for a skilled attacker, compared to solely bypassing a mechanical lock, today many widespread systems can be circumvented by thieves.

A similar trend as for car immobilizers can be observed for many other wireless systems employing cryptography: Outdated technology is often employed even in recent commercial products. It seems that the progress of the research in the field of applied cryptography is widely ignored by the industry — as long as (cryptographically weak) products are bought by the customers, they are further marketed.

1.1 Pervasive Wireless Technology

The large variety of wireless embedded applications implies that products come in a lot of distinct flavors, differing amongst others in the dimensions, the maximum achievable range for

a query, and their computational power [Fin02]. The devices typically operate in one of the frequency bands¹

- Very Low Frequency (**VL**F) from 3 kHz to 30 kHz,
- Low Frequency (**L**F) from 30 kHz to 300 kHz,
- Medium Frequency (**M**F) from 300 kHz to 3000 kHz,
- High Frequency (**H**F) from 3 MHz to 30 MHz,
- Very High Frequency (**V**HF) from 30 MHz to 300 MHz,
- Ultra High Frequency (**U**HF) from 300 MHz to 3000 MHz, or
- Superhigh Frequency (**S**HF) from 3 GHz to 30 GHz.

Considering the huge differences in the frequencies, the corresponding wavelengths λ , and antenna sizes, e.g., $\lambda = 10$ km for an **RFID** transponder operating at 30 kHz compared to $\lambda = 10$ cm for another **RFID** transponder operating at 3 GHz, gives an impression how much all other characteristics of the systems can differ in the wireless world (cf. Sect. 2.4).

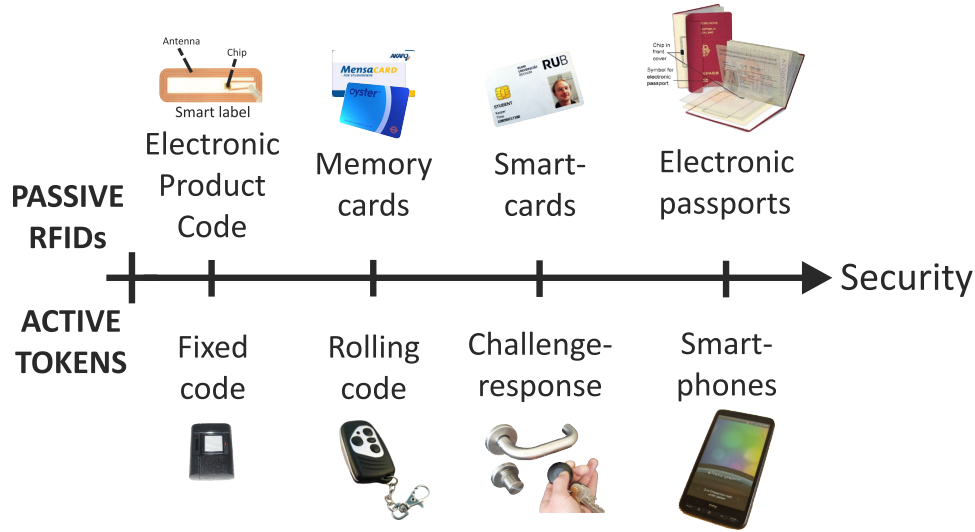


Figure 1.2: Differences in the computational power and the security level of wireless devices

Some examples of wireless tokens are illustrated in Fig. 1.2. Passive **RFID**s as exemplified in the top of the figure are generally restricted with respect to their computational power, since their energy is supplied wirelessly by the **RF** field of a base station. In contrast, the active tokens illustrated in the bottom of the figure possess their own energy source, i.e., a battery, are thus more powerful, and enable greater operating ranges.

However, the computational power and the corresponding achievable level of security differ largely in each of the two groups: while read-only **RFID**s and fixed-code tokens for the purpose

¹Note that the exact boundaries between the frequency ranges vary in the literature, e.g., 2.4 GHz **RFID** is often treated as **SHF**.

of simple object identification and tracking provide no security measures, memory cards such as Mifare Classic and unidirectional rolling-code devices such as KEELOQ remote controls often employ weak, outdated encryption mechanisms. Contactless smartcards, e.g., Mifare DES-fire, can execute state-of-the-art ciphers and hence enable secure challenge-response schemes, as also implemented in corresponding active, bidirectional key fobs. The most powerful passive device is probably the electronic passport: it contains an advanced microcontroller, e.g., SmartMX [NXP04b], that enables computationally demanding cryptographic schemes including public-key ciphers, and in addition features a protection against MITM attacks. Smartphones on the active side contain even more powerful microcontrollers, e.g., from the ARM Cortex family. They can basically perform any desired type of cryptography and provide various communication channels, such as Wi-Fi, Global System for Mobile Communications (GSM), Universal Mobile Telecommunications System (UMTS), and Bluetooth. Some phones possess a Near Field Communication (NFC) interface that is compatible to contactless smartcards and hence constitutes a link between active tokens and passive RFID devices — smartphones equipped with NFC can connect passive devices to the cloud.

For classifying wireless tokens according to their technical capabilities, we distinguish between “active”, “semi-active”, and “passive” devices. Active tokens possess their own energy source and can hence initiate a communication. Passive RFID transponders are powered solely by the RF field generated by a base station and require no battery. They respond to requests of the base station and cannot perform any computations in the absence of the latter. Semi-active devices possess a battery, e.g., for powering sensors, monitor environment data, or keep a time source alive when no base station is present. However, their energy source is not used for the communication which is instead realized according to the passive RFID principle.

Passive tags are limited with respect to their maximum power consumption, i.e., the number of switching transistors during their operation [LSR06], which has a direct impact on their cryptographic capabilities — implementing state-of-the-art cryptography is sometimes infeasible. Another factor is the cost sensitivity of high-volume applications, which often tempts manufacturers to minimize the costs at the expense of the security, e.g., by using outdated but “cheap” cryptographic components. As a consequence, cryptography and other security measures may be very lightweight or not present at all, even when security or privacy issues are relevant.

1.1.1 Applications of Passive (RFID) Devices

Passive RFID transponders have become widely used for identification purposes. The birth of identifying objects via RF technology were Identify Friend or Foe (IFF) systems based on Radio Detection And Ranging (RADAR) [RCT06c] technology used during the second world war for detecting “friendly” airplanes, i.e., those of the own forces. Meanwhile, various passive devices are available that are computationally powerful enough to realize cryptographic schemes and hence enable extended functionality and a broad spectrum of applications. Depending on the use case a designer can choose between different technologies to meet the particular requirements, e.g., the operating range or computational capabilities. Note that the technical characteristics of different RFID systems vary extremely, for instance, depending on the operating frequency and the coupling method between reader and transponder, as detailed in Sect. 2.4. Thus, it is

difficult to make general statements about **RFID** and one has to be very cautious to transfer conclusions drawn from properties of one **RFID** system to another.

Applications of Simple Transponders

The cost-efficient transponders at the low end are used for applications in which no security is required: in the supply chain, **RFID** technology enables the wireless identification of objects that are tagged with transponders, from a distance and without a line-of-sight (“auto-ID”). Electronic Product Code (**EPC**) Gen 2 tags [EPC10] are designed especially for the supply chain management and operate in the **UHF** frequency range. The tags possess a long reading range of several meters but cannot perform any computations — they solely provide a small memory that can be accessed from a distance. The content of the rewritable transponders is updated on their path through the supply chain, e.g., information is added or modified when produced, packaged, and during transportation. The price stored on the tag can be adapted in the shop and the quantities of the products can be acquired by means of automatic inventory control. Finally, after the product is sold to the customer, the tag can be permanently disabled by sending a “kill” command in combination with a password.

Other applications of non-cryptographic **RFID** transponders include baggage handling at airports, automated toll collection (often 915 MHz), and the identification of animals (often 125 kHz), e.g., tracking cattle and tagging pets. **RFID** technology is also used to tag humans: Wristbands identify patients in hospitals to avoid mixing them up and unsecured **RFID** interfaces integrated in pace makers [HHBR⁺08] enable to (re)program them through the human tissue.

Applications of Cryptographic Transponders

More advanced **RFID** devices providing simple encryption schemes are integrated into car keys for anti-theft systems, i.e., immobilizers (often 125 kHz), and are used in medium-security applications for access control purposes. Cryptographic transponders can also help to prevent product forgery: **RFID** chips are integrated into spare parts, such as batteries of mobile phones, ink cartridges of printers, and (mechanical or electrical) components of cars, in a way that they cannot be removed without damaging or destroying the secured spare part. The devices operate only if genuine parts are identified by means of their transponders and otherwise report fraud or malfunction, e.g., a printer will not work if a forged ink cartridge is inserted. Likewise, **RFID** tags are implemented into the package of products, e.g., drugs, to ensure the content is genuine. Public-key cryptography can be applied against counterfeiting: Authentication by means of Elliptic Curve Cryptography (**ECC**) is feasible on passive **RFID**s tags, as shown in [BHM08, BBD⁺08].

Contactless Smartcards Contactless smartcards, constituting the most powerful variant of **RFID**s, are basically smartcards or memory cards², augmented with a wireless interface. They are available in a variety of forms, e.g., plastic cards, key fobs, watches, can be embedded in paper documents as so-called smart labels, or can be sewed into clothes. In contrast to the simple

²Smartcards typically possess a microcontroller, while for the simpler memory cards often a control logic, that allows accessing the memory, suffices.

tags used for goods tracking, contactless smartcards are often used in security-sensitive access control and payment applications. They (optionally) offer a wide range of security features, including cryptographic co-processors for encryption or Message Authentication Code (MAC) generation.

Contactless smartcards are deployed worldwide in a wide range of fields that require fast and secure transactions or to protect personal information, for example in transportation, government or corporate ID cards, and access control applications. Contactless smartcards are integrated in electronic passports [BSI] for the identification of individuals — the cryptographic mechanisms implemented in the wireless chip protect highly sensitive data such as biometric information. Large-scale ticketing systems based on contactless cards, such as public transport in various big cities, SkiPasses for the entrance to skiing areas, and tickets for events in stadiums, accelerate the admission and facilitate controlling the validity of the tickets. Moreover, a growing number of payment systems incorporate RFID-enabled credit cards [Tex]. Various multi-purpose applications have emerged, e.g., the OpenCard³ in Prague (Czech Republic) combines micro-payments, ticketing for public transport, and services of the public library. The computational power of contactless smartcards comes at the cost of a relatively small reading range, compared to other RFIDs. Technical details about the operation of contactless smartcards are given in Sect. 2.5.

1.1.2 Applications of Active Devices

Battery-powered remote controls enable operating ranges in the order of tens or hundreds of meters and can provide strong cryptography. They usually communicate at Industrial, Scientific, and Medical (ISM) frequencies in the UHF range, i.e., in Europe very often at a frequency of 433 MHz or 866 MHz, and serve, e.g., for opening cars or controlling an alarm system. Actively powered solutions are required, if a long reading range or a short interrogation period is demanded and hence passive tags are no option, e.g., objects that are queried while driving past at a high speed. As a drawback, their battery needs to be replaced in certain intervals. Again, different classes of security, ranging from no security to state-of-the-art cryptographic schemes, are available. Simple one-button key fobs or wireless baby phones operate unidirectionally without encryption or other security measures. More advanced bidirectional devices perform authentication by challenge-response schemes before access to a secured area is granted. Various home-automation applications rely on active devices: remote controls open doors, windows, or blinds; electrical power sockets can be switched wirelessly; and fingerprint scanners that are mounted outside the building are connected to the base station inside without a cable. The Zigbee⁴ protocol connects and controls various electronic devices in the household, is used for smart-energy products, and enables to create wireless networks consisting of active sensors. Many wireless phones, certain portable devices for reading credit cards, and even some traffic lights comply to the Digital Enhanced Cordless Telecommunications (DECT) standard. Bluetooth⁵ allows connecting headphones and wireless gadgets to televisions, smartphones, and other consumer devices. Wireless Local Area Networks (WLANs) is omnipresent for connecting devices among each other or to the Internet, and mobile phones encrypt the voice or other data,

³<http://www.opencard.cz>

⁴<http://www.zigbee.org/>

⁵<http://www.bluetooth.com>

transmitted via the **GSM** or **UMTS** network to a base station, to protect the privacy of the owner.

Semi-Active Devices and NFC Semi-active devices are hybrid, i.e., they combine passive communication interfaces with actively powered circuits. Hybrid **RFID** transponders can be equipped with sensors for monitoring and recording various environmental conditions, such as temperature or humidity. A battery or another power source, such as a solar-energy cell, ensures the operation of the sensors in the field when no energy supply from the base station is present. Sometimes special security features, such as detecting an intrusion by means of light or shock sensors, are available [Sav10]: These transponders are mainly used in the supply chain for tagging containers with large assets, e.g., for the transportation of medical supplies, food and other goods, in order to track their route while assuring that certain environmental conditions are met. Sometimes, both an active, long-range **RF** interface and a passive, short-range **RFID** interface are provided — accordingly, some devices cannot be unambiguously classified.

The **NFC** technology provided by various smartphones is used amongst others for payments, e.g., Google wallet⁶, and secure pairing, e.g., in the context of Bluetooth communication. **NFC** is compatible with contactless smartcards: As detailed in Sect. 2.5.7), an **NFC** phone can either act as an **RFID** reader or emulate a contactless smartcard. However, even when emulating cards, the phone needs to be powered by batteries and thus cannot be regarded as a passive **RFID** device. Without using the **NFC** interface, a smartphone or mobile phone is clearly an active device (see above).

1.1.3 Wireless Devices Analyzed in this Thesis

The examples in the previous section illustrate the variety of wireless devices and the ubiquity of wireless applications. In this thesis, only some of the existing applications are analyzed with respect to their security: Simple low-end devices, such as read-only **RFID** transponders and active fixed-code systems, provide no effective security measures — the corresponding threats are obvious and hence no in-depth analysis is required. Smartphones are also excluded from the investigations, since they can be regarded as portable computers and hence different attack vectors apply: for extracting secrets, in most cases computer-related threats such as Trojan horses, root kits, and computer viruses play a more important role, compared to physical attacks. This thesis instead focuses on those portable devices with high security demands and limited computational power, e.g., contactless smartcards and active key fobs that employ cryptography. Applications relying on weak, proprietary cryptographic schemes (Mifare Classic cards and KEELoQ remote controls) are considered as well as those incorporating publicly known, highly secure ciphers (Mifare DESfire cards and electronic passports).

1.2 Security Analyses of Wireless Systems

In the beginning, modern cryptanalysis was solely performed by the military and intelligence services. After academia started researching the topic in the mid to late 1970s, cryptographic primitives and their vulnerabilities became known to the public. Nowadays, due to the large

⁶<http://www.google.com/wallet/>

deployment of cryptographic schemes in consumer products, a third strong force has joined the community breaking cryptographic systems: criminals such as product pirates and thieves. Even in state-of-the-art products that use highly secure cryptographic primitives, they often find and exploit flaws on the system-level, e.g., get hold of secret keys used for the encryption of pay-TV.

Wireless communication implies new, additional attack vectors as compared to contact-based systems: a transponder residing in a pocket or wallet could be read out or modified without the owner taking note of it and the transmission of data via the **RF** field can be monitored or relayed from a distance. Wireless applications, as exemplified in the previous section, thus demand for protecting the over-the-air interface: a private phone call must not be monitored by a neighbor, a door must not be opened by an intruder, and it must be made impossible for a customer to charge his contactless payment card except at a dedicated charging terminal by paying money into the system. To realize security mechanisms that prevent from fraud and unauthorized access, or to establish confidentiality and data protection, cryptography in distinct flavors is widespread in today's wireless tokens. Encryption or authentication schemes incorporating secret cryptographic keys shall guarantee security, data integrity, and ensure the intended functionality of the wireless systems in general.

Assume a strong mechanical lock of a door that is augmented with wireless technology in order to open it with a remote control. Clearly, the level of security established by the electronic parts of the access control system must at least match or outperform the level of security provided prior solely by the mechanical components. Unfortunately, many real-world systems relying on wireless technology have turned out to be weak with respect to their security and privacy. Popular examples are KEELOQ rolling code systems used for access control [EKM⁺08], or Mifare contactless smartcards that are widely used for e-ticketing [Phi06], public transport [Ver], and micropayments [KSP10]. As a result, real privacy and security threats arise for the individuals using contactless technology: movements of individuals can be tracked by eavesdropping at different locations on the wireless communication with their wireless tokens, private data interchanged using a weak encryption can be decrypted, and doors can be opened unauthorizedly. The reasons for the security weaknesses are manifold, e.g., proprietary and weak ciphers, but often (additionally) vulnerable realizations on the system level.

1.2.1 Security (only) from Obscurity

Kerckhoff's principle formulated in 1883 [Ker83] states that a cryptographic system must be secure, even in case it falls into the hands of the enemy. This implies that the cipher must be practically unbreakable, even if all details are known to an attacker, and that all secrecy lies in the secret key. However, many commercial products in the 21st century do not follow this principle. Instead the manufacturers have invented their own cryptographic primitives, often with low-cost properties, and establish "security" by keeping the details secret. The past decade has shown that the vast majority of these schemes is terribly flawed and once the ciphers are reverse-engineered and become public, they can be broken with low to modest efforts. One popular example is the DST40 cipher used in Texas Instrument's Digital Signature Transponder (**DST**) which had been reverse-engineered in 2005 [BGS⁺05]: Its 40-bit secret key can be revealed by means of a brute-force attack. NXP's Mifare Classic and Microchip's KEELOQ have a similar deterrent product history, as detailed in Chapters 6 and 8, respectively. The term "security from

obscurity” became common for describing these and similar products implementing protection schemes that rely only on the secrecy of weak, proprietary algorithms — often providing no security at all, once they are reverse-engineered. Nevertheless some additional obscurity, i.e., secrecy of a sound, secure design or implementation, can be advantageous for most cryptographic systems.

More recently, the trend seems to go towards employing publicly known and secure cryptographic primitives in newly developed products. This is where new, non-mathematical attack vectors come into play: Even if the employed cryptographic primitives are secure on an algorithmic level, a system can be broken in practice, e.g., by means of implementation attacks, due to a flawed derivation of the cryptographic keys, or due to their insecure storage. The potential of physical attacks in the context of commercial, real-world products is analyzed and evaluated in this thesis.

1.2.2 Cryptanalysis and Cryptography: About Attackers and Designers

What is the motivation for a security researcher to practically attack commercial products? One answer to this often posed question is, along with data protection and privacy reasons, that analyzing existing cryptographic systems is indispensable for designing and securing better ones in the future. The relevance of various practical security threats, and the cost of real-world attacks, such as physical attacks, are often not known to the system designers, and thus are not appropriately considered and modeled. Practical security analyses allow to assess the actually provided level of security-related applications and compare it with the advertised statements of the manufacturer.

Of course, a manufacturer whose product turns out to be weak might become unhappy and claim that the researcher has impaired the company’s image and turnover. On the other hand, he and his affected customers must be informed about security weaknesses, so they can be addressed before a criminal finds and exploits them. Furthermore, security analyses in the real-world can even have a direct positive impact on the economy: Flawed products need to be replaced with better, more secure substitutes on the long term, thus the researcher creates an increased consumer demand for appropriately secured products.

Theoretical World In the scientific community of the cryptologists a well-established and indispensable interplay exists between the fields of cryptography and cryptanalysis that relates to Kerckhoff’s principle: New cryptographic primitives are designed and published by cryptographers, and in the following analyzed by their antagonists, the cryptanalysts. If the latter succeed in breaking the new primitive, they in turn publish their findings or report the found vulnerabilities to the designers by other means. The latter, now aware of the revealed weaknesses, can close the security gap and improve their design, which is again analyzed, and so on. A cryptographic primitive is finally regarded as secure, if and only if it has been publicized for a long time and no other researcher has been able to successfully break it. This symbiotic relationship between cryptography and cryptanalysis has in the past years led to highly secure cryptographic schemes and today various peer-reviewed ciphers are regarded as practically unbreakable, e.g., the Advanced Encryption Standard (**AES**) block cipher or the Rivest Shamir and Adleman (**RSA**) or **ECC** public-key schemes (with appropriately chosen parameters).

Real World When realizing such a highly secure cipher in hardware or software and integrating it in a real-world system, new security vulnerabilities arise that are not part of the classical theoretical analyses. Accordingly, a similar interplay as in the theoretical domain is required for analyzing and improving security products in practice. Progress with respect to security is inherently linked to attacking and penetration-testing the products. Some big manufacturers of security-related products have already founded internal security departments that analyze and penetrate inventions of the own company according to the attack-and-learn principle: The aim is to detect vulnerabilities and fix them, if possible before the products are marketed.

However, publications about implementation attacks at the relevant conferences, e.g., the Cryptographic Hardware and Embedded Systems (CHES) workshop, have been mostly academic experiments of researchers attacking and improving their own (or known) implementations for many years. Despite the great developments and improvements of effective countermeasures against most threats, analyses of real-world devices were rarely found. Consequentially, few reliable statements could be given concerning the level of security provided, as well as the real effort and cost required for practical attacks. Recently, a new trend has emerged: The scientific community has started to investigate commercial products with respect to physical attacks. The corresponding research gives essential feedback to the manufacturers by pinpointing weaknesses and flaws, and in consequence enables them to produce improved and more secure devices. This thesis follows this recent trend and the author hopes that the presented findings contribute to the science of building more secure wireless systems.

1.2.3 Towards more Security

Weak, proprietary cryptography is often considered by the industry as “secure enough” for a certain application. As fraud involving wireless devices is becoming more profitable, e.g., contactless smartcards used for payments or car immobilizers, several real-world offences have already emerged. The majority of the users of wireless systems lack understanding of the level of security provided. The RF communication takes places unnoticable and the end-users typically have no means to judge whether a product meets the security demands of the respective application — they have to trust the promises of the manufacturers. The latter mostly focus on keeping the cost low instead of implementing state-of-the-art schemes to provide the expected functionality of their products. It seems that, as long as no major (financial) catastrophe happens, the manufacturers keep selling their vulnerable products, often aware of their weaknesses but not informing the customers for obvious economic reasons.

While many theoretical proposals for attacks exist, the cryptographic literature lacks realistic, practically verified models of the adversary. The cost of (physical) attacks, e.g., side-channel cryptanalysis, is often overestimated and the possible implications are widely underestimated. This thesis performs some of these attacks in practice, with small effort and cost, and examines the vulnerability of real-world systems towards them. Various commercial products advertised as highly secure in consequence turn out to be very weak or provide no security at all, which may have serious implications on the privacy and safety of individuals. Affected customers may unnecessarily lose confidence in using modern technology. Instead, end-users must be informed about the real level of security a product provides, to distinguish between weak and strong solutions, accordingly choose the system for their requirements, and on the long term

(re)establish a trust in those products that are secured with appropriate, modern cryptographic schemes.

Each cryptographic device shows a significantly different behavior, i.e., an **RFID** device is vulnerable to other variants of potential attacks than the microcontroller of an active remote control or an implementation on an Application Specific Integrated Circuit (**ASIC**). Hence, it is necessary to investigate the susceptibility of each new target separately, especially with respect to physical attacks aiming at the specific implementation. For each device, an in-depth profiling is necessary, which should be automated as far as possible. To test and then improve the security of existing devices and systems, this thesis as a first step presents tools that assist penetration-testing of the used platforms and protocols and allow to conduct practical attacks. As the standards differ very much with regard to operating frequency, communication interface and transmission protocol [Fin03], the hardware for a reasonable security analysis must often be custom-made and tailored to the particular system of interest. With the special tools, as developed in this thesis, systems can be investigated in the laboratory or in the field, to identify the relevant threats and counteract them.

1.3 Outline of this Thesis

The remainder of this thesis is structured as follows: The next two chapters introduce wireless systems, the corresponding security threats, and the theory of (physical) cryptanalysis, in particular Side Channel Analysis (**SCA**) attacks. Then, the tools required for finding and exploiting vulnerabilities of wireless systems are introduced. The main part of this thesis is the practical security analysis of commercial applications illustrated in the last four chapters. Additional details and findings that are not directly relevant for the real-world attacks are given in the Appendix, e.g., statistical methods, side-channel countermeasures, additional modules for the measurement setup, as well as software and schematics for the developed hardware.

Background: Wireless Embedded Systems and Physical Security Analysis Chapter 2 classifies attackers, security risks of wireless technology, and the relevant protocols as used by the devices analyzed in the later chapters. Furthermore, the technical background and standards for the contactless communication are detailed for the applications covered in this thesis. Chapter 3 classifies the field of cryptanalysis. Implementation attacks are introduced, the theory of side-channel analysis is described in detail, and the approach of profiling an unknown device is illustrated.

Tools for Security Analysis In order to conduct physical attacks, (customized) hardware for performing security analyses, communicating with the different targets, and assisting the data acquisition need to be developed. In Chap. 4, a unified framework for advanced implementation attacks is introduced that provides the technical basis to conduct automated side-channel analysis, fault injection, and other physical attacks. The target platforms are all kinds of (wireless) embedded cryptographic devices including **RFIDs**. The proposed low-cost setup consists of modular functional units that can be interchanged, depending on the demands of a concrete attack scenario, and enables various types of security evaluation and penetration tests. The developed cost-effective and freely programmable devices comprise a multi-function **RFID** reader

and a card emulator for ISO 14443 compliant smartcards, as widely-used in privacy and security sensitive applications, and a similar reader for **RFIDs** operating at 125 kHz, for example employed in car immobilizers. Furthermore, extension modules for fault injection are sketched and the measurement setup for the side-channel attacks conducted in this thesis is illustrated. The presented cost-effective framework disproves the common belief that highly sophisticated and expensive equipment is required to conduct physical attacks.

Chameleon In Chap. 5, a new, custom-built hardware for emulating contactless smartcards compliant to ISO 14443 is developed. The device, termed Chameleon, is based on a modern low-cost microcontroller and can support all relevant (cryptographic) protocols used by contactless smartcards today, e.g., those based on encryption with AES or Triple-DES, including all computations involved. As a proof of concept, a full emulation of Mifare Classic cards on the basis of a highly optimized implementation of the stream cipher Crypto1 is implemented. The device enables the creation of exact clones of such cards, including the Unique Identifier (**UID**). Furthermore, the protocol of Mifare DESfire EV1 is reverse-engineered and a further implementation realizes the first emulation of DESfire and DESfire EV1 cards in the literature.

The capabilities of our emulator are practically demonstrated by spoofing several real-world systems. For instance, an emulation of a contactless payment card enables an attacker to set the credit balance as desired and hence make an infinite amount of payments. Furthermore, doors secured by a commercial access control system, that is based on identifying the **UID** of Mifare Classic cards, are unauthorizedly opened with the Chameleon.

Pitfalls of a Mifare Classic-based Payment System Despite the publicized vulnerabilities of the Mifare Classic technology, a multitude of applications worldwide still employs tens of millions of these contactless smartcards. Chapter 6 gives a comprehensive summary of the relevant cryptanalytic research and security vulnerabilities related to Mifare Classic. Then, a widespread real-world contactless payment application based on these cards is investigated. In order to analyze the security of the payment system, previous cryptanalytical results are combined and an improved card-only attack is implemented on the customized low-cost tools introduced in Chap. 4, that is to our knowledge the most efficient practical attack to date. It targets the implementation of the Random Number Generator (**RNG**) used in the Mifare Classic cards: The developed reader enables precise control of the communication and the **RF** field such that the **RNG** is bypassed and, exploiting properties of the weak Crypto1 cipher employed, an efficient key-recovery becomes possible.

A subsequent case study demonstrates the effectiveness of the developed attack in practice at hand of an in-depth analysis of the contactless payment system. Various security risks are revealed and precarious vulnerabilities are pinpointed that are likely to be present in other **RFID**-based systems. It is shown in practice how the flaws on the system level allow for devastating attacks including card cloning, recharging and identity theft. Finally, best-practice countermeasures are proposed for raising the overall security to a sufficient level. The findings, illustrating how to minimize losses due to fraud in similar cases with only small effort and cost, may serve as guidelines for integrators to design more secure systems in the future.

The idea of solving the security problems of the analyzed contactless payment system just by upgrading to a more sophisticated class of cryptographic contactless cards, e.g., Mifare DESfire, is not promising: The next chapter illustrates that these cards are vulnerable to implementation

attacks and presents a successful key-recovery attack on Mifare DESfire by means of side-channel analysis.

EM Side-Channel Attacks on Mifare DESfire The attack on Mifare Classic cards in Chap. 6 breaks their security by means of fault injections on the protocol level. The key-recovery has been feasible due to a weak RNG and the usage of an outdated cipher. In contrast, Chap. 7 targets a cryptographic card employing a state-of-the-art cipher, namely Triple Data Encryption Standard (3DES) with a 112-bit secret key. Hence, neither attacks on the protocol level nor mathematical cryptanalysis are promising — instead another class of implementation attacks, i.e., side-channel analysis, is required. Again, the customized hardware introduced in Sect. 4.1 serves as the basis for performing the first non-invasive side-channel attacks on commercial RFIDs in the literature, that rely on extracting the information leakage contained in the Electro-Magnetic (EM) emanations.

The chapter illustrates new techniques for facilitating a Correlation Power Analysis (CPA) in the presence of the strong RF field of an RFID reader. The effectiveness of the developed methods is practically verified by tackling the security of commercial contactless smartcards employing strong cryptography at the example of Mifare DESfire MF3ICD40. The analysis pinpoints weaknesses in the protocol and reveals vulnerability towards side-channel attacks. Despite of integrated countermeasures, the findings result in the first successful key-recovery attack on the cards that are widely used for security-sensitive applications, e.g., access control to governmental and military sites, identification, and payment purposes.

Attacking the secured implementation of the targeted passive RFID requires a high effort: Due to the countermeasures and the noisy EM environment, hundreds of thousands of measurements and sophisticated post-processing of the side-channel information are required to extract the secret keys of the cards. The next chapter of this thesis shows that, depending on the target device, side-channel analysis can be extremely more powerful than in the context of RFIDs: The developed attack techniques enable the recovery of the secret keys of cryptographic implementations, in the best case from processing just a single measurement.

Power Analysis of KeeLoq: Breaking Remote Keyless Entry Systems The previous chapters of this thesis have considered only passive RFIDs that are restricted with regard to their power consumption and thus their computational capabilities. Active, battery-powered systems in contrast possess much more resources and can perform basically any type of cryptographic operation. *Do active wireless systems accordingly provide a higher level of security?*

To answer the question, the potency of power analysis when applied to active wireless tokens and the corresponding receivers is demonstrated in Chap. 8. The developed highly efficient SCA attacks based on Simple Power Analysis (SPA) and CPA techniques can break the wireless access control system KEELOQ with minimal effort. KEELOQ RKE systems are widely used in applications such as garage-door openers or anti-theft solutions in cars, which demand for large operating ranges that cannot be achieved with passive RFID transponders.

In Sect. 8.3 the first successful Differential Power Analysis (DPA) attacks on numerous commercially available products employing KEELOQ code-hopping are presented. The introduced techniques combine side-channel cryptanalysis with specific properties of the KEELOQ algorithm. They efficiently reveal both the secret key of an ASIC implementation in the remote control and the manufacturer key stored in a software implementation in the receiver. As a

result, a practical key recovery is feasible from only ten power traces and a remote control can be cloned in a few minutes. However, a full key-recovery of the manufacturer key by means of **DPA** still requires ten thousands of measurements.

As a remedy, general techniques for effectively realizing an automatic **SPA** are introduced, that can also be applied to other implementations of cryptography. Section 8.4 then presents an **SPA** that is tailored to extracting the manufacturer key from software implementations of KEELOQ. The new attack drastically reduces the effort required for a complete break of **RKE** systems based on KEELOQ. By analyzing implementations of KEELOQ on microcontrollers and exploiting their timing vulnerabilities, a technique is developed that enables a practical recovery of the manufacturer key within seconds of computation time, significantly outperforming all existing attacks: Only one single measurement of a fraction of a KEELOQ decryption is sufficient to extract the 64-bit master key of commercial products, without the prior knowledge of neither plaintext nor ciphertext.

After the one-time extraction of the manufacturer key as a prerequisite, recovering the secret key of a remote control and replicating it from a distance, just by eavesdropping on at most two messages, is demonstrated. This key-cloning without physical access to the remote control has serious real-world security implications, as the attack can be conducted by an unskilled adversary, while the technically challenging part can be outsourced to specialists. Next, a Denial-of-Service (**DoS**) attack is mounted on a KEELOQ access control system that enables an adversary to completely take over control of the system, i.e., lock out the owner of the original remote control while she can still access the secured object. Finally, an instantiation of an exhaustive key-search on cryptanalytic hardware, to evaluate the security of other (seed-based) key-derivation schemes for KEELOQ, is detailed. All the described attacks have been verified on several commercial KEELOQ products. As an option for future unidirectional access control systems a new, secure **RKE** system is developed that is resistant to power analysis.

The chapter on KeeLoq exemplifies the system-wide implications which a key-recovery by means of **SCA** may cause. The key distribution of the **RKE** system, i.e., deriving the keys of the individual remote controls from their serial number and a manufacturer key, enables dramatic real-world attacks once the manufacturer key has been recovered. The secure messaging scheme implemented in electronic passports, as analyzed in the last chapter, relies on a different approach for deriving the secret keys: they are computed from data that is printed on the paper document. Accordingly, the threat scenario is different — a side-channel attack is not adequate, since anyone having physical access to the passport can derive the secret keys from the printed data. Instead, eavesdropping attacks that aim at gathering private data of passport holders from a distance are of particular importance.

Eavesdropping on Electronic Passports The final chapter analyzes the security of the probably most secure wireless system based on passive **RFID**: The e-Pass (electronic passport), as specified by the International Civil Aviation Organization (**ICAO**) is deployed in many countries all over the world. The security and privacy threats have been widely discussed, e.g., in [JMWO5b, KK05, HHJ⁺06, JMV07], and have provoked public debates. The passports provide sophisticated cryptographic mechanisms to protect the private (biometric) data stored on them, including both public-key and symmetric cryptography. Current realizations of the Basic Access Control (**BAC**) that shall prevent unauthorized access to the data stored on electronic passports deploy symmetric cryptography based on SHA-1 and **3DES**. The corresponding en-

encryption and authentication keys are generated from data printed on the passport. As shown in Chapt. 9, a low entropy of the derived keys may enable straightforward attacks with a small complexity compared to an exhaustive key-search attack on 3DES. Using the codebreaker Cost-Optimized Parallel Code Breaker and Analyzer (COPACOBANA) introduced in Sect. 4.5, in realistic scenarios the key for the BAC can be recovered almost in real-time, i.e., the time needed for a person to pass an inspection system at the border control. Despite the secure cryptographic primitives being employed, the private data interchanged during the BAC is hence at risk of getting into the hands of unauthorized persons or organizations — again, the flawed key derivation is the main factor for the found vulnerabilities. After the key-recovery attacks of this chapter had been publicized, the German authorities issued a new generation of passports with an improved scheme for the key derivation that very likely withstands modern cryptanalysis.

Chapter 2

Background: Wireless Embedded Systems

EMBEDDED systems are generally prone to security risks, since large numbers of them are typically in the possession of potential adversaries. In the context of wireless technology some special threats evolve or are of particular importance. For example, copying a mechanical key usually premises physical access to the key or at least to the door lock. In contrast, circumventing a contactless access control system may be possible from a distance and without leaving physical traces. The wireless component causes a paradigm shift with respect to the security of modern access control systems, since in addition to the mechanically established security, today the electronic and cryptographic components need to be considered in order to assess their security.

This chapter gives the background information required for the analyses in the later chapters. It classifies the attackers in Sect. 2.1, compiles the security threats of RF systems in Sect. 2.2 and introduces the authentication schemes that are of relevance for this thesis in Sect. 2.3. The RFID operating principle is generally illustrated in Sect. 2.4 and in particular the relevant standards and protocols for contactless smartcards, as required in the later chapters of this thesis, are studied in Sect. 2.5. Finally, the maximal possible operating range and eavesdropping range are detailed in Sect. 2.7 for all wireless devices that are analyzed in the next chapters of this thesis.

2.1 Classification of Attackers

The feasibility of attacks naturally depends on the know-how and budget of the adversary — we hence start with ranking them and explain the assumptions made about the attacker in the remainder of this thesis. Ross Anderson and Markus Kuhn [AK98] and a paper by IBM [ADDS91] give a suitable classification of adversaries according to their funding and skills. The papers date from 1998 and 1991, respectively, which was before the notion of implementation attacks existed in the research community. When defining the following taxonomy the authors hence did not have implementation attacks (cf. Chap. 3) in mind. Note that even today, the “security requirements for cryptographic modules” Federal Information Processing Standard (FIPS) 140-2, issued by the National Institute of Standards and Technology (NIST), considers protection against implementation attacks optional but not required. However, the taxonomy of the above papers is regarded adequate and the following three classes of attackers are still prevalent:

- **Class I (clever outsiders):** Very intelligent and curious individuals with typically insufficient knowledge of the system. Due to negligible resources they have access to only moderately sophisticated equipment. They often try to take advantage of an existing weakness in the system, rather than try to create one, and exploit it mostly for their

personal use. The findings of such a “hacker” can have a harmful impact if they become widespread, e.g., published on the Internet.

- **Class II (knowledgeable insiders):** They have substantial specialized technical education and experience, e.g., university students or corporate employees. The attackers have varying degrees of understanding of parts of the system, but potential access to most of it. They possess highly sophisticated resources such as specialized tools and instruments for security analyses.
- **Class III (funded organizations):** Teams of specialists with related and complementary skills that are backed by great funding resources. Class II adversaries may be employed as part of the attack team. They perform in-depth analyses of the system, design the most sophisticated attacks, and use the most advanced analysis tools. The intelligence services, e.g., NSA or a comparable well-funded commercial organization, can allegedly break many (cryptographic) systems.

This thesis focuses on those attacks that are realistic for an adversary with a limited budget, i.e., attacks that can be performed by clever outsiders or knowledgeable insiders. The cryptographic algorithm itself is assumed to be known to the attacker (no security only from obscurity, compare with Sect. 1.2.1). The specific implementation of the target device or other confidential information, e.g., the source code or an open sample device, is assumed to be *not available* — with the exception of information that can be publicly accessed, e.g., on the Internet. The adversary has access to equipment such as oscilloscopes etc. that can be found in university labs. The auxiliary tools for physical and protocol attacks, as developed in this thesis, are hence low-cost devices and often self-built from off-the-shelf components, with their design made available to the public.

2.2 Attacks on Wireless Embedded Devices and Related Work

This section guides through the various security and privacy threats of the wireless world and pinpoints common countermeasures. The over-the-air communication channels used by wireless embedded devices can be exploited by an adversary in an unwanted manner. Furthermore, attacks on the actual implementation of security measures (if existent) have to be considered, since the devices are often in the physical possession of potential attackers. The relevant evolving security risks and considerations along with common remedies to prevent deception and protect the (private) data are summarized in the following. Where applicable, pointers are given to the corresponding section or chapter of this thesis in which more related work and the evolving attacks are covered in detail.

The threats can be classified into *non-invasive* attacks, that usually leave no traces on the target device, and *invasive* attacks, that involve a physical manipulation and can cause a non-reversible impact or even destruction of the target. An example of the former is spying out information by means of eavesdropping, while for instance etching of an Integrated Circuit (IC) in order to probe and reverse-engineer its electrical circuit corresponds to the latter. Depending on the type of action an adversary has to carry out for conducting an attack, some threats are further classed as *passive* or *active*, where an active attack usually involves data transmitted by

the adversary. Generally, the actions of an active attacker can be sensed on the physical layer, while a passive threat usually remains undetected.

2.2.1 Sniffing

The data transmitted via the **RF** channel can be easily intercepted by a third party.

Eavesdropping Eavesdropping describes a passive scenario in which an attacker can monitor and record the information exchanged between genuine devices from a distance, e.g., by mounting a disguised antenna in the vicinity.

Active Sniffing An adversary actively initiates the communication with a wireless device by means of a (modified) reader, in order to unauthorizedly obtain some information stored in the target. Actively sniffing the whole information contained in a wireless device in order to produce a duplicate is often termed *skimming* in the literature.

Sniffing allows to electronically spy out information, e.g., private data stored on an ID-card or unnoticed inventory of a competitor's storehouse. The ranges for both the active and passive variants differ extremely, depending on the technology and frequencies used, and are discussed in more detail in Sect. 2.7. Corresponding (self-built) devices for actively communicating and passively monitoring the communication of wireless devices are presented in Chap. 4. Sniffing can be considerably impeded or even completely eliminated by implementing secure cryptographic mechanisms, e.g., enforcing a mutual authentication between the parties and encrypting the transmitted data.

2.2.2 Replay Attacks and Spoofing

By emulating a wireless device, often on the basis of customized hardware and with the help of a Personal Computer (**PC**), previously monitored messages are directly replayed or partly modified and then replayed. Technically, the (re)produced information cannot be distinguished from that being transmitted by a genuine device; hence the attack pretends the presence of a legitimate wireless device. In the context of **RFID**, it is expedient to distinguish between

- replaying data of a transponder and
- replaying data of a reader, while furthermore
- the replayed content can be manipulated to falsify the information (*spoofing*).

The attacks allow to bypass many simple systems for contactless access control or ticketing, as demonstrated by circumventing a widespread door-opener system in Sect. 5.6.3 with the emulating device developed in Chap. 5. To hinder replay attacks, well-known cryptographic measures could be implemented to establish a mutual authentication between the communicating parties, or to detect unauthorized modification of the transmitted information by appending a **MAC**.

2.2.3 Cloning and Manipulating Genuine Devices

Instead of emulating the wireless device with special hardware, a genuine transmitter, receiver or transponder can serve as a basis for an attack. The manipulated device is in the “best” case not only technically but also with respect to its other properties (optical appearance, weight, etc.) indistinguishable from the original. The term *clone* usually corresponds to an exact duplicate obtained on the basis of a genuine platform. Due to the extremely different complexity of conducting the attacks we differentiate between

- wireless devices with a fixed or modifiable content that is stored and transmitted in clear, and
- devices featuring security functions, such as encryption of the communication or a write protection.

The latter approach typically requires — in addition to the data to be (re)produced — further knowledge concerning the security mechanisms, e.g., the employed cipher and its cryptographic keys. In the case of the contactless payment system analyzed in Chap. 6, fraudulent payments can be carried out on the basis of the legitimate cryptographic smartcards that are issued to the customers, by simply reprogramming the content of the cards. The feasibility of a manipulation depends amongst others on the reliability of the inherent security mechanisms of the device, e.g., cryptographic schemes, whether One-Time Programmable (OTP) devices are used, or if a read-out or write protection is present and securely implemented.

2.2.4 Man-in-the-Middle

MITM attacks require an adversary who has access to the wireless communication channel and intervenes in the information exchange, i.e., she possesses the hardware to receive and send data according to the required protocol.

Passive MITM The communication between legitimate wireless devices is redirected in real-time via a separate channel (also termed “relay attack” or “mafia fraud” in the literature), such that both parties assume to be directly interacting with a trustworthy counterpart: The devices are “convinced” that they are in their vicinity, i.e., in the boundaries of their normal operating range, while in fact they can be miles apart.

Active MITM During an active **MITM** attack, the relayed information can be altered by the adversary.

MITM is a very strong type of attack, as it operates on the physical layer and can hence even circumvent systems employing cryptographically secure authentication and strong encryption. The required hardware can often be bought or built from off-the-shelf components (see Sect. 4.1); accordingly **MITM** attacks can be carried out by an unskilled adversary and pose a severe real-world threat for most wireless systems. Relay attacks on **RF** devices, as described in [KW05] and practically realized for example in [Han05] and [KCP07], have recently also been conducted on passive keyless entry and start systems in modern cars [FDv10]. In practice, two adversaries collaborate: one remains in the vicinity of the victim to wirelessly access his key fob or contactless card, while the other one steals the car or carries out a payment on behalf of the victim.

Countermeasures against MITM Attacks

When practically relaying data a time delay is inevitable, as described in Sect. 4.1.1. A countermeasure impeding MITM by detecting this delay, called distance-bounding, has been introduced more than two decades ago in [BC93] and has been proven to be practically feasible [Rv10, HK05]. Properties of the RF field are verified on the physical communication layer to ensure the vicinity of the wireless devices and to detect whether the received information has been relayed. The countermeasure takes into account that EM waves travel approximately at the speed of light. Still, to the knowledge of the author, distance-bounding seems to be too expensive or too complicated to realize and thus has not been implemented in any consumer product, yet. An effective countermeasure that is already available in commercial devices, i.e., the electronic passport, is the BAC protocol detailed in Sect. 9.2. In the BAC scheme, the physical presence of the legitimate passport is ascertained by the protocol requiring some input that is printed on the paper document; hence an optical inspection is mandatory for a successful authentication. Another way to enhance privacy is constructing a Faraday's cage for the tag: experiments of the author confirm that a single layer of aluminum foil wrapped around the smartcard completely protects it from being activated or read out by an RFID reader. Hence, if an RFID tag without protection against MITM is indispensable, metal shielding can prevent unauthorized access. A simple option for future products is the idea that a card should not be able to become active unless the owner has performed an action, e.g., press a button or open the cover of his electronic passport. However, such a mechanism degrades the benefits of the technology with respect to comfort and can contradict the general principle of RFID transponders to be always accessible.

The tools and the corresponding software developed in Chap. 4 enable to practically conduct both relay attacks and active MITM attacks with contactless smartcards at a low cost. As a proof-of-concept for an active MITM, transmitted commands are successfully modified during a running communication in real-time, in order to alter the identifier of a wireless device or other content being transmitted (see Sect. 4.1.3). The author of this thesis conducted practical relay attacks on various contactless cards in his diploma thesis (2006) [Kas06].

2.2.5 Denial of Service (DoS)

DoS attacks aim at disturbing the functionality of a wireless system in order to render it out-of-service or to exploit a resulting malfunctioning behavior.

Jamming An interfering transmitter set up by an adversary can jam selected frequencies and thereby locally hinder or block the communication between legitimate devices. Jamming is widespread amongst thieves: inattentive car owners may leave their car unlocked due to a jamming attack.

Blocker Tags Customized hardware may hinder an intended communication to a particular wireless device by pretending the presence of a large amount of (emulated) devices, for example RFID transponders. The huge amount of virtually present devices overburdens the requesting reader such that it cannot handle all requests [JRS03, RCT05]. As a consequence of the collisions occurring due to the responses of the virtual tags, reading out the targeted transponder fails — it is protected by the blocker tag. Blocker tags are

originally intended to be used in a positive manner, i.e., for protecting the privacy of the carrier.

Destruction The electronic circuitry of a wireless device can be destroyed, for example by means of heat, mechanical force or a strong **EM** field. The latter applies especially in the context of **RFID**, e.g., a photo camera can be transformed into an “RFID Zapper” [MM06] that aims at destroying the circuitry of contactless smartcards, for instance electronic passports. Exposing a wireless device to a microwave oven often has the same destructive effect.

Kill Command Some **RFID** transponders can be permanently deactivated (or the complete content deleted) via the communication protocol by means of a special “kill” command. What is intended as a privacy-preserving feature can be misused for a **DoS** attack, if the attacker gets hold of the respective kill password. For certain **RFID** tags operating in the **UHF** range this password may be extracted remotely by observing the power consumption of the tag (termed remote power analysis) [OS07]: By precisely monitoring the **EM** field and exploiting a side-channel leakage during the response of the device, the password bits can be predicted one after another.

Detaching and Swapping Removing a wireless device from an object is mostly relevant in the context of **RFID** transponders. A tag can be removed at one place and glued to a new, wrong location (e.g., swapped against another tag) for product forgery or to cause malfunctioning, e.g., altering the entry points for trains can cause wrongly debited tickets.

Sleep-deprivation Especially active or semi-passive wireless devices that rely on an battery-powered energy supply are vulnerable to “sleep deprivation” attacks that aim at dissipating the energy available, until the power source runs dry and hence the device malfunctions or its data is erased. A strongly increased energy consumption can be caused for instance by means of a specially prepared **RF** hardware that perpetually issues recurrent requests to the attacked device at a high request rate.

Shielding Placing a wireless device in a Faraday’s cage, i.e. metal shielding, to block any **RF** communication, is a common method for privacy protection. Still, the method can also be employed maliciously, e.g., shielding anti-theft transponders may allow stealing protected products.

DoS attacks usually involve an ill-intended malicious adversary who aims at disturbing or circumventing a system, e.g., that of an opponent. Often, the success of a **DoS** attack depends on the back-up mechanisms provided on the system level for the case of a failure of the electronic component, e.g., a purely optical inspection of the ticket for access control to a stadium. **DoS** attacks on wireless devices are often feasible by completely unskilled attackers since the required tools, e.g., for jamming, are commercially available and countermeasures are often not possible or difficult to implement.

2.2.6 Tracking

Tracking as a security threat refers to the (unauthorized) monitoring and recording of the movements of an individual or object. For conducting the attacks in practice an adversary sets up (covert) eavesdropping devices and antennae at appropriate locations, e.g., airports or

train stations, that automatically record the wireless communication in their surrounding area. Most wireless devices possess an Identifier (**ID**) number that is often transmitted in plain at the beginning of a communication. In most cases the **ID** is factory-programmed and cannot be changed after manufacturing, hence an adversary may track a person holding the wireless device (or a set of wireless devices, e.g., **RFID** transponders) on the basis of the **ID**(s) or other content and properties of the device. Tracking can be realized

- directly, on the basis of the content or other unique properties of a wireless device, or
- indirectly, by collecting the information acquired from a set of devices and statistically evaluating the occurrence of the pattern, e.g., a set of **RFID** transponders carried by an individual or stored inside a container.

Tracking is recently discussed in the context of mobile phones and constitutes mostly a threat for the privacy of individuals, e.g., employees that unawarely wear **RFID** badges, smart tags on clothes, medical implants and other products, or carry contactless cards and other wireless tokens in their pockets. The threat is also relevant the supply chain, e.g., potential thieves tracking containers with expensive goods.

Shielding the wireless device, as detailed above, can be an effective countermeasure, but is often not a practical option. Some of the newer and more sophisticated cryptographically enabled wireless devices, such as the electronic passport, are capable of encrypting their content and generating a random **ID** for every query, thus impeding tracking. Still, due to their high cost these devices are rarely found on the market (especially in the low-cost sector such as in most auto-ID **RFID** applications) and the distribution of the cryptographic keys for the authentication and encryption often poses a problem. Even state-of-the art wireless devices providing some mechanism to hinder tracking often remain vulnerable to the attack: they can often be identified on the basis of their individual physical properties, e.g., the encryption of the latest version of electronic passports is not yet broken but the duration for computing a **MAC** allows to uniquely identify each particular passport [CS10]. The threat of tracking individuals at hand of their electronic passports is covered in detail in Chap. 7 in this thesis.

2.2.7 Detection

In contrast to tracking, for which the wireless device needs to be uniquely identified, the threat of detection requires solely sensing the presence of a particular device by (a tool set up by) an adversary. Since the ranges for active communication and passive eavesdropping often largely differ, we distinguish

- detecting the presence of a device and
- detecting the wireless communication between two devices.

Detecting the presence of a wireless device or its transmissions may allow an attacker to draw conclusions with respect to the presence of objects, individuals or groups. For example, in the context of military operations the existence of certain technical characteristics may identify an enemy from a distance, or enable to distinguish between countries, and trip an alarm accordingly. An extremely dangerous safety risk related to this threat (but also to sniffing and eavesdropping) is triggering explosives upon detection of a (certain) wireless device: Ari Juels et al. publicized the notion of an **RFID** bomb already in 2005 [JMW05b].

2.2.8 Malware

Since small wireless devices are becoming computationally powerful (“computational **RFIDs**”), malware as known from **PCs**, i.e., worms, trojan horses and computer viruses, constitutes a real-world threat also for **RFIDs** [RCT06b]. For example, it has been shown that a virus can be programmed to spread out via **RFID** tags [RCT06a]. Vulnerabilities in the software of the back-end or in the firmware of a wireless device may allow to take over control of the complete system, e.g., by exploiting buffer overflows. Not only consumer products, but also industrial and military applications can be affected. For instance, the recent discovery of details of the STUXNET virus shows the potential implications that embedded malware can have [BMS11]. Another flavor of the malicious manipulation of digital systems was described in a 2005 report by the US Defense Science Board, where the clandestine introduction of hardware Trojans, e.g., [LKG⁺09], was underlined as a serious threat [DSB]: Trojans can spy out confidential information, e.g., by establishing covert channels.

2.2.9 Implementation Attacks

Implementation attacks target the concrete realization of a cryptographic primitive or a security concept in the form of an electronic circuit. The aim is to (partially) recover some secret information contained in the device, e.g., get hold of its secret cryptographic key to compute the plaintext to a given ciphertext. Implementation attacks make use of the information gained from the concrete physical implementation of a cryptosystem, rather than cryptanalyzing the theoretical weaknesses of the cryptographic algorithm. They include

Reverse Engineering Concluding to the content or the functionality of a wireless device, e.g., extracting a proprietary cryptographic algorithm or other Intellectual Property (**IP**). Reverse engineering can be performed by means of etching an **IC** and analyzing its hardware layout [NESP08] or reading out the content of a device and reconstructing its function, e.g., a software implementation of a microcontroller or the bitstream used to program reconfigurable hardware.

Side-Channel Analysis Information about internals of a (secured) device can be obtained by passively monitoring various side channels of the system, such as the execution time of an algorithm, the power consumption during the individual algorithm steps or, particularly in case of the wireless devices, fluctuations in **EM** emanations. The side-channel observations are statistically evaluated and correlated with the cryptographic operation in order to extract secrets.

Fault Injection Actively intervening in the operation of the wireless device, e.g., by generating strong energy pulses or emitting laser pulses, with the aim to cause some malfunctioning or faulty behavior of the device that is related to the cryptographic function executed. An erroneous output can often be exploited to conclude to a secret key [BS97].

The side-channel analysis and fault-injection techniques can be combined — known as Combined Implementation Attacks (**CIA**) [STA⁺10] — for instance to thwart certain countermeasures. In general, every unprotected device can be successfully attacked with relatively small effort by means of implementation attacks, even if state-of-the-art cryptographic primitives are employed.

Physical cryptanalysis and **SCA** of wireless tokens are essential building blocks of this thesis. The theory of implementation attacks, with an in-depth explanation of side-channel attacks, is thus detailed in Chap. 3. Later in this thesis, Chap. 7 illustrates by extracting secret keys from Mifare DESfire cards that many of the cryptographically enabled contactless smartcards currently on the market are susceptible to side-channel cryptanalysis based on measuring the **EM** emanations. Further practical **SCA** attacks targeting active wireless devices are conducted in Chap. 8 to reveal secret keys from **RKE** devices by means of power analysis.

2.3 Authentication Schemes for Wireless Tokens

This section outlines the most relevant protocols for authentication purposes that have been in use since the notion of wireless systems and classifies all schemes of relevance for this thesis. The discussion focuses on simple protocols as often used by commercial wireless tokens and does not cover the elaborate variants. For further reading, Chap. 10 of the Handbook of Applied Cryptography (**HAC**) [MvOV97] details complex identification and authentication schemes including corresponding attacks.

The basic schemes in the following are discussed in the context of passively powered **RFID** systems, but are also implemented likewise in actively (battery-) powered wireless tokens. The participants of the protocols in the following are entitled *base station* and *transponder*. The base station is typically the receiver of information and has a static location, e.g., an **RFID** reader installed at the ignition lock of a car or a base station for **GSM** communication. The transponder is a portable device, e.g., remote control for an active **RKE**, a car key incorporating an **RFID** immobilizer transponder, or a mobile phone. The channel for the wireless communication usually relies on **RF** technology, with an operating range in the order of millimeters for some passive **RFIDs** and in the order of kilometers for active systems.

The protocols demand for different capabilities of the participants. For example, in the first generation of **RKE** systems a read-only transponder controls the access to a secured object by sending a fixed sequence of data to a receiver [Lin]. Other protocols for the mutual authentication of both parties require a two-way communication and computations to be performed in the transponder.

In the following, we distinguish between *unidirectional* and *bidirectional* systems: the transponders for the former provide only the technical means for sending information, but cannot receive any. The corresponding base stations are accordingly equipped only with a receiver and cannot send data. In bidirectional systems, both participants possess a receiver and a transmitter and can hence communicate in both directions. The reason for the existence of unidirectional systems is mostly the cost of the production — remote controls comprising only a transmitter are less complex and hence cheaper than those providing a receiver in addition.

We further differentiate between *read-only* transponders with a fixed content that cannot be altered, *rewritable* transponders with a modifiable content, and transponders providing additional resources, e.g., an **RNG** or cryptographic engines. Due to their performance and compact implementations, symmetric encryption primitives in which both participants share the same secret key K are of special interest for wireless tokens. Symmetric cryptography can attain the security goals authenticity (e.g., challenge-response protocols), confidentiality (e.g., encryption of data), and integrity (e.g., **MACs**).

The prevalent schemes for unidirectional systems, i.e., fixed-code and rolling-code protocols, are illustrated in Sects. 2.3.1 and 2.3.2. Next, the synchronization-code, challenge-response and mutual authentication protocols used for bidirectional systems are detailed in Sects. 2.3.3, 2.3.4, and 2.3.5. Finally, the Cipher Block Chaining (CBC) mode of operation for symmetric ciphers is presented in Sect. 2.3.6 and the very important topic of key management is covered in Sect. 2.3.7.

2.3.1 Fixed-Code Protocol

The first generation of remote controls simply transmitted an analogue signal, e.g., an RF wave at a determined frequency. Upon detection of this signal, the receiver performs the assigned task, e.g., open a garage door. With digital circuitry becoming widespread, the transponders were able to transmit information, e.g., a sequence of binary data, and fixed-code protocols as illustrated in Fig. 2.1 emerged. In the unidirectional scheme a transponder immediately transmits the same fixed data sequence if a button is pressed or another matching condition (for example the presence of a particular RF field) is present. The data is often cyclically transmitted, i.e., after each complete transmission it is starting over again. Only upon reception of the correct fixed code the base station will perform the assigned task.

This type of protocol is also termed *read-only* protocol in the literature, as typically read-only transponders are employed and the data being sent cannot be changed. The transponders operate in a Tag Talks First (TTF) mode, indicating that no start command of the base station is necessary. Eavesdropping the fixed code sequence and replay attacks are straightforward for these systems. In case a spare transponder is required, a “clone” that transmits the same fixed code as the original can be easily created.

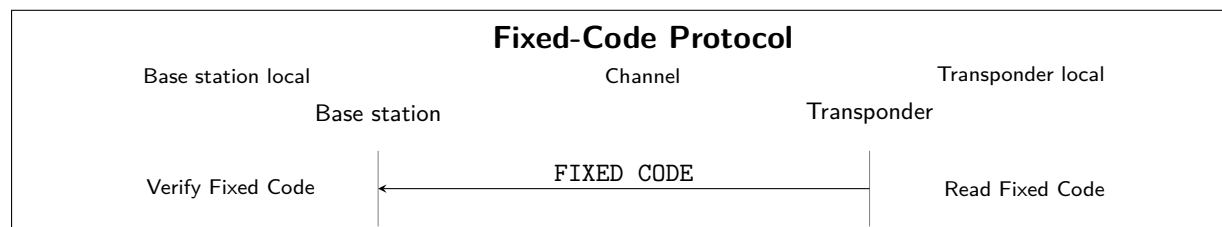


Figure 2.1: A fixed-code protocol relies on read-only transponders and a unidirectional communication channel. The scheme provides no protection against replay attacks and cloning.

Despite their lack of any security measures, fixed-code protocols are still widely employed, e.g., in access control systems. While any electronic hobbyist has the skills to circumvent them, it is often assumed that the invisibility of the wireless communication suffices to prevent from attacks.

2.3.2 Rolling-Code Protocol

The goal of a rolling-code scheme is to generate dynamic transmissions in a transponder in order to defeat replay attacks in a unidirectional system. The rolling-code protocol requires a transponder that possesses rewritable memory and can perform an encryption with a symmetric cipher involving a secret key K .

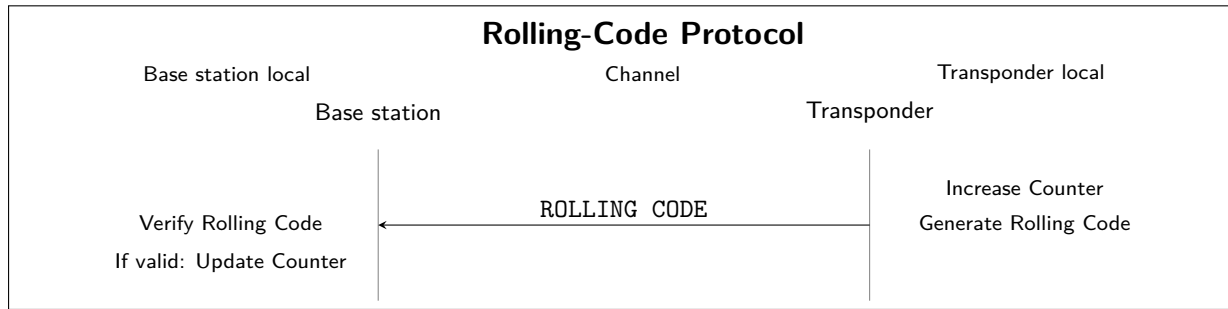


Figure 2.2: A rolling-code protocol protects against replay attacks in a unidirectional setting. For generating rolling codes, the transponders have to store (and increase) a counter value and must be capable of encrypting the counter with a symmetric cipher.

The basic principle is depicted in Fig. 2.2: the communication simply consists of a so-called rolling code being sent by the transponder. Contrary to a fixed-code system, every code is different from its predecessor, which is achieved on the basis of a counter implemented in the transponder. To hinder an attacker from guessing the next correct counter value, its value is encrypted before the transmission, implying that subsequent codes cannot be distinguished from random values.

In a typical rolling code system, the transponder possesses a unique and public “serial number”, a fixed “discrimination value”, an internal “counter”, and a secret key K for the encryption. Initially to each transmission, the counter in the transponder is increased, concatenated with the discrimination value, and then encrypted with a cipher using the secret key K , as shown in Fig. 2.3. The resulting ciphertext, called rolling code, is appended to the unencrypted serial number of the transponder before being sent to the base station. If more than one function shall be triggered with the same transponder, the function to be executed (“Func.” in Fig. 2.3) may also become part of the input for generating the rolling code.

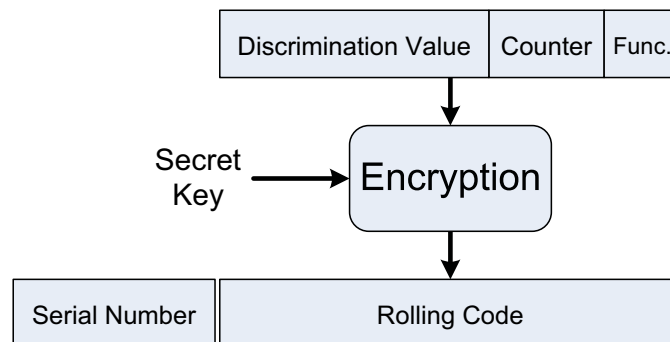


Figure 2.3: Details of a rolling-code scheme as implemented in a transponder. Typically, the transponder’s serial number is transmitted along with the rolling code.

The base station possesses a list of transponders in which it stores the serial number, the discrimination value, the last counter value, and the secret key corresponding to each valid

transponder. When a rolling code is received, the serial number is used to look up the secret key K , the lastly received valid counter value, and the discrimination value of the transponder. Then, the rolling code is decrypted using K , as shown in Fig. 2.4. The base station updates the

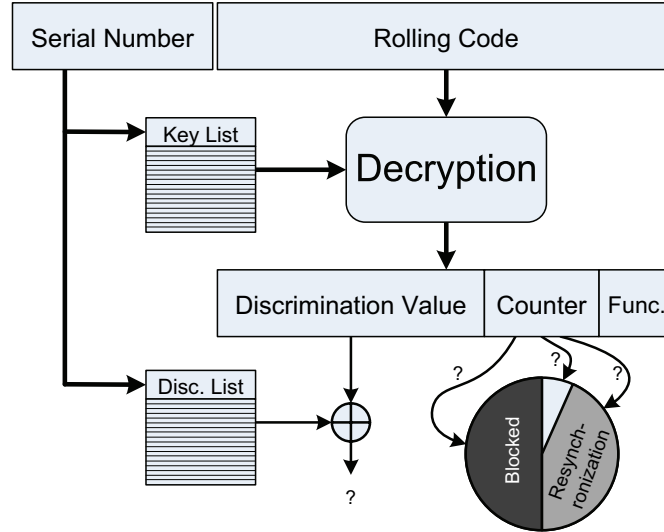


Figure 2.4: Details of a rolling-code scheme in a base station.

respective counter and permits access to the secured object, if and only if the following three criteria are satisfied:

- (1) The serial number of the received rolling code must be valid, i.e., the corresponding remote control including its secret key must be known to the receiver.
- (2) The discrimination value of the decrypted rolling code must be identical to the one stored in the base station.
- (3) The counter value contained in the decrypted rolling code must be moderately increased compared to the counter value of the last valid transmission.

The scheme prevents replay attacks (compare with Sect. 2.2.2) by rejecting previous codes generated from an outdated counter value. Since the rolling-codes are encrypted, an attacker cannot conclude to the current counter value and hence cannot send a valid code. While in an ideal world the value of the counter could be increased infinitely, in a practical realization of the scheme the counter space is finite, e.g., has a length of 16–18 bit. Thus the name rolling code: Once the counter space reaches its end the counter values, and likewise the corresponding dynamic codes, will be repeated from the beginning.

In order to prevent from circumventing the scheme by sending random counter values, as well as from replay attacks, the counter space is split up into three windows, as depicted in the right bottom of Fig. 2.4. The values preceding the last valid counter value correspond to the largest window, e.g., half of the counter space¹, which is designated as the *blocked*

¹The exact size of the windows can be chosen by the system designer and thus can vary between different systems.

window. Any transmission based on a counter value in the block window is rejected in order to preclude the repetition of previously monitored rolling codes by means of an attacker. The window comprising the *valid* codes begins after the lastly received valid value of the counter and is comparably small, i.e., it usually contains no more than 16 values: a transponder will be authenticated instantly, if the decrypted counter is within this window. It may happen that a transponder is activated several times out of the range of the base station and accordingly its counter value is increased beyond these 16 valid values. The so-called *resynchronization* window comprises the next counter values typically up to half of the counter space. To authenticate the transponder and resynchronize with its current counter value, the base station must detect two consecutive valid counter values within this window.

The discrimination value serves as an additional protection against sending random codes: if its decrypted value does not match the one stored in the receiver, the validation of a transmission sent by a transponder is instantly aborted — the likelihood that a decrypted, randomly generated code matches the discrimination value is very small (depending on the chosen size of the discrimination value). A commercial implementation of a rolling-code scheme, i.e., the widespread **RKE** system KEELQ, is analyzed in Chap. 8.

2.3.3 Synchronization-Code Protocol

A synchronization-code scheme is bidirectional and hence premises a receiver and a transmitter (“transceiver”) both in the transponder and the base station. The scheme further requires a rewritable type of transponder. The content of the transponder may sometimes be protected by means of a password, but usually no cryptography is used.

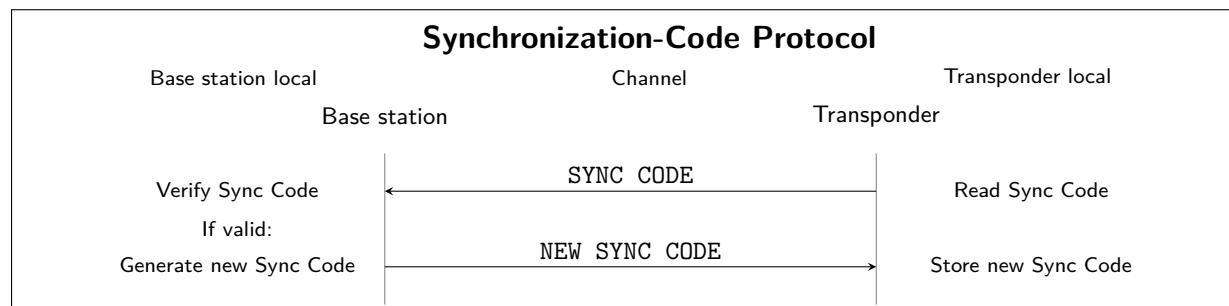


Figure 2.5: A synchronization-code protocol, requiring bidirectional communication and a rewritable transponder, aims at preventing replay attacks without using cryptography.

As illustrated in Fig. 2.5, at the beginning of the protocol the synchronization code (sync code) stored on the transponder is sent to the base station. The latter verifies the received code by comparing it with the sync code that is stored in its internal memory. If the comparison is successful the received code is regarded as valid and the base station generates a new sync code, e.g., a random number. The code is stored in the memory of the base station and then transmitted to the transponder. The latter updates its content with the new sync code and will transmit this new code at the next instantiation of the protocol. On the side of the base-station in the following only the new sync code will be accepted as valid, while an old code is rejected, thus a replay attack based on an outdated sync code is defeated. However, an adversary can

easily eavesdrop on the new sync code that is transmitted unencryptedly and hence directly access the secured object by replaying this new sync code. Note that, after performing such a replay attack, the sync code stored in the original transponder is outdated and will not be accepted as valid by the base station.

In a password-protected scheme the base station sends the password in plain before updating the content of the transponder with the new sync code. Since no encryption is used, eavesdropping on the communication immediately reveals the password as well as the next valid sync code, hence using a password only provides marginal additional protection against attacks.

Combining a synchronization-code scheme with one of the two bidirectional schemes employing cryptography, as illustrated in the following two subsections, can increase their resistance against replay attacks: in case that a new, random secret key is used for encrypting each authentication session, an attacker cannot replay nor generate the correct (encrypted) synchronization code.

2.3.4 Challenge-Response Protocol with Symmetric Cryptography

Challenge-response is a widespread bidirectional scheme involving cryptography. In the context of wireless devices, symmetric primitives are commonly employed, i.e., both participants share an identical secret key K and are able to encrypt or decrypt messages with a cipher, e.g., a block cipher or a stream cipher.

A random value termed “challenge” is generated by the base station and, as illustrated in Fig. 2.6, is transmitted to the transponder. On reception, the transponder encrypts the challenge using K and sends the resulting ciphertext, i.e., the “response”, back to the base station. The latter verifies, whether the decrypted response equals the challenge it had previously sent. In case that challenge and decrypted response match, the base station knows that the transponder possesses the correct secret key and hence assumes it is genuine. Any further communication can now be fully encrypted, e.g., using the key K or on the basis of a *session key* K^* which is newly created in each individual communication session and is valid only for that session. Real-world examples for securely generating K^* can be found in this thesis in Sect. 7.2.1 in the authentication protocol of Mifare DESfire cards or in Sect. 9.2.2 in the BAC protocol of the electronic passport.

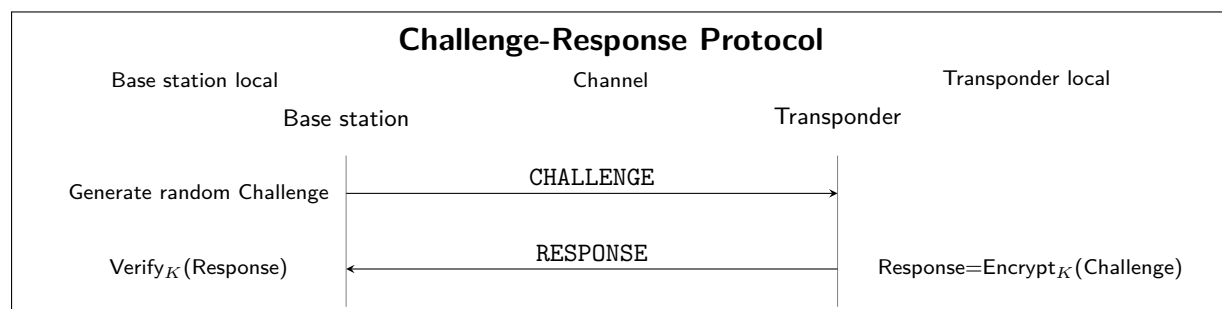


Figure 2.6: A challenge-response protocol ensures the authenticity of a cryptographically-enabled transponder in a bidirectional setting.

Challenge-response protocols are, for example, employed in military (and other) aircrafts for realizing so-called **IFF** systems that often rely on **RADAR** communication [LC01]. Only a friendly airplane possessing the correct secret key can be identified by its response — in case of no response it is not clear whether the airplane is friendly or hostile.

Note that only the genuineness of the transponder is verified by the base station, while the transponder on the other hand has no means to verify the authenticity of the challenging base station. For a mutual authentication, the challenge-response protocol needs to be extended, as follows.

2.3.5 Mutual Authentication Protocol with Symmetric Cryptography

A major disadvantage of a challenge-response protocol is that the transponder cannot verify whether the base station is authentic. One option to overcome this limitation is depicted in Fig. 2.7: the base station generates a challenge and secures it with a **MAC** before transmitting it both challenge and MAC to the transponder. By verifying the latter, the transponder ensures that it is communicating with an authentic base station. If the **MAC** is not valid, e.g., because the base station does not possess the correct secret key, the protocol aborts.

The scheme proceeds as in the challenge-response protocol (see Sect. 2.3.4): The transponder encrypts the challenge and sends the ciphertext as response back to the base station. The latter finally verifies the authenticity of the transponder by decrypting its response and comparing the plaintext with the challenge. Again, the subsequent communication after the authentication is typically encrypted with a session key K^* , e.g., derived from interchanged random numbers, to establish a confidential channel.

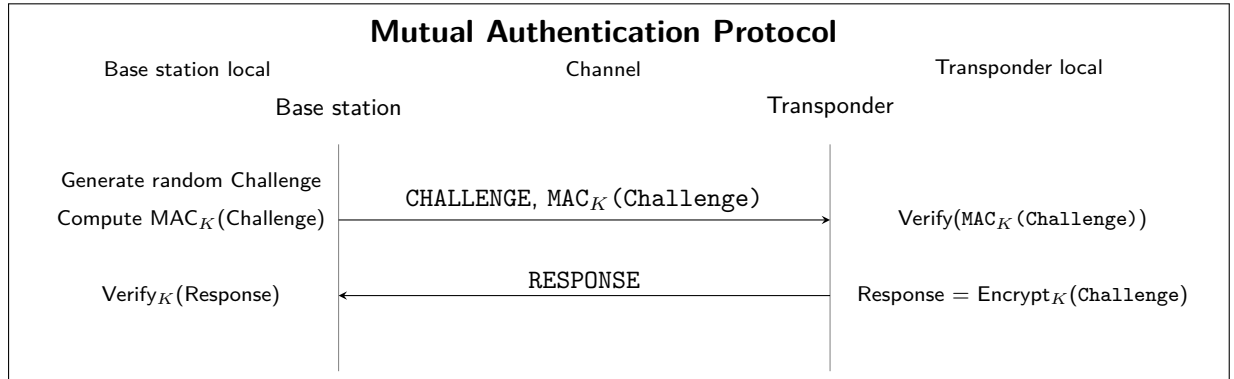


Figure 2.7: A mutual authentication protocol aims at establishing the authenticity of both participants. The scheme requires bidirectional communication and a cryptographic transponder, typically providing symmetric encryption using a key K . During the depicted first protocol steps of this simple variant of the scheme, the authenticity of the base station can be forged by a replay attack.

The illustrated protocol is still vulnerable to a replay attack: the first message sent by the base station, i.e., the challenge appended with the appropriate **MAC**, can be monitored by an adversary and replayed to a transponder. As a remedy, improved variants of mutual authentication protocols comprise a further step at the beginning of the protocol, in which the transponder

generates its own random challenge, to be encrypted or **MAC**ed by the base station. As a prerequisite for this improvement, the transponder must provide a secure **RNG**. Real-world implementations of such secure mutual authentication protocols are given for the Mifare DESfire and the Mifare DESfire EV1 contactless cards in Sect. 7.2.1 and Sect. 5.3.2, respectively. Note that, despite the cryptography involved in the schemes, even these elaborated protocols are vulnerable to **MITM** attacks that are detailed in Sect. 2.2.4. A mutual authentication protocol incorporating a protection against **MITM** as realized in the electronic passport is detailed in Sect. 9.2.2.

2.3.6 Cipher Block Chaining Mode of Operation

A block cipher, such as the **AES** or the Data Encryption Standard (**DES**), encrypts or decrypts messages as data blocks of a fixed length. Usually, the size of a message string M is larger than the block size, hence a long message M is divided into a series of n consecutive message blocks ($M = M_1 || M_2 || \dots || M_n$), which are processed one at a time by the cipher.

For securely encrypting the message blocks, nine different operating modes of block ciphers are currently approved by the **NIST** [Nata]. In wireless devices, often the **CBC** mode is implemented, e.g., in Mifare DESfire cards and electronic passports.

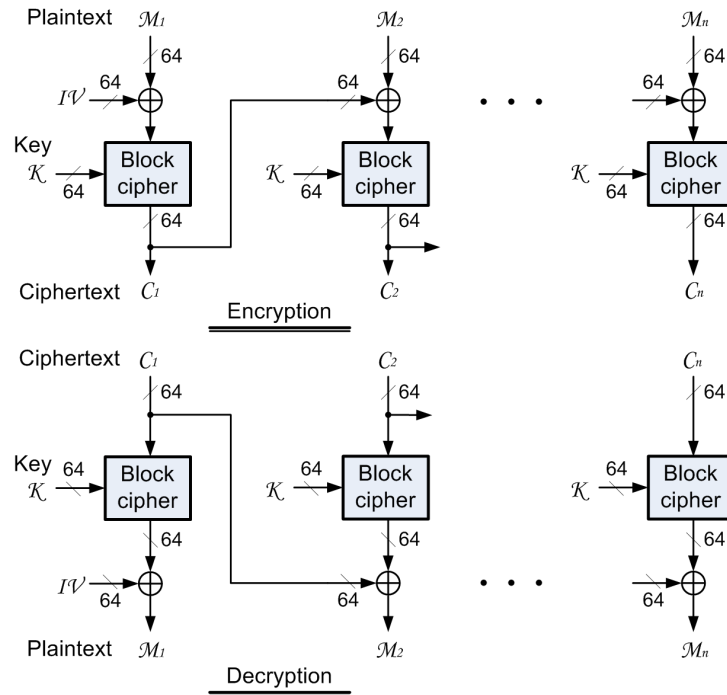


Figure 2.8: Encryption (top) and decryption (bottom) of long messages with a block cipher using the Cipher Block Chaining (CBC) operating mode.

In the **CBC** mode of operation a plaintext is encrypted with a secret key K as illustrated in the top of Fig. 2.8 for a block size of 64 bit: Initially, an *Initialization Vector* (IV) is Exclusive OR (**XOR**)ed with the first plaintext block to form the input of the first encryption. The output is treated as the first ciphertext block C_1 . For processing each successive block i , the previous

ciphertext block C_{i-1} is **XOR**ed with the current plaintext block M_i before the encryption, until all n blocks have been processed. The ciphertext C is thus again a sequence of n blocks which are chained according to $C = C_1 || C_2 || \dots || C_n$. Note that each block of the ciphertext depends on its predecessor. The decryption is accomplished vice versa, as shown in the bottom of Fig. 2.8.

The **IV** has the length of one block and does not have to be secret. For each encryption, a new random value should be chosen as **IV**, in order to prevent the generation of identical ciphertexts when repeatedly encrypting the same plaintext: Due to the randomization of the first plaintext block M_1 by means of the **IV** prior to the encryption, all ciphertext blocks that are output are different, even in case of identical plaintext blocks. This is a major advantage with respect to security, compared to the Electronic Code Book (**ECB**) mode, in which each block is encrypted independently from the others and the same plaintext always results in the same ciphertext.

2.3.7 Key Management of Wireless Tokens

For all the above illustrated (symmetric) authentication schemes we have so far assumed that both participants already share the same secret key K . But how are the keys securely transported into the devices? In the context of practical wireless systems the key distribution, as covered in this section, often poses a major problem and can be crucial for their security.

The secret keys of the transponders can often be generated and programmed during the manufacture. However, in many applications base stations in the field need to be able to learn these secret keys of new transponders, e.g., a spare remote control that shall be used in an access control system or a newly issued contactless card for payments. Transmitting cryptographic keys via the insecure **RF** interface is not an option, hence corresponding key derivation schemes are required. While in theory public-key cryptography and other sophisticated schemes can be employed for the key exchange, these computationally demanding schemes are not yet widely used in wireless tokens, due to the increased cost and the restricted computational power of the devices. Hence, various other (often low-cost) solutions can be found in real-world products. In the following, these basic schemes as used by the commercial products analyzed in this thesis are sketched, to give an impression about the topic of distributing keys. For further reading, an in-depth study of the modern concepts and security requirements of key establishment and key management can be found in [FL93], in Chaps. 7.3 and 10 of [Sta05], and in Chaps. 12 and 13 of the **HAC** [MvOV97].

In the simplest solution for managing keys the *same* secret key (or set of keys) is used in *all* devices in the system, i.e., all transponders and base stations know this key since their production or initialization. Such a “key management” scheme is implemented in various commercial applications, e.g., the flawed contactless payment system analyzed in Chap. 6. If an attacker gets hold of the secret key, she can compromise all devices of the system — clearly, such a Single Point of Failure (**SPOF**) must be avoided. Therefore, it is strongly recommended to generate a unique secret key for each transponder and base station.

In other schemes the secret keys are generated randomly and stored in a large data base, e.g., located on secure servers of the manufacturer that contain all keys of the system. In such schemes the keys usually have to be programmed into the devices (i.e., into the base station and all transponders that shall cooperate with the base station) in a secure environment where no

attacker can monitor the programming process. If a spare transponder is required after-sales, the key transport into this transponder often poses a problem in the field. In this scheme, although it is of course more secure than using identical keys for all devices, the data base containing all secret keys again constitutes a **SPOF**.

Sometimes, the individual key K of each transponder is generated from its serial number using a key-derivation function f , e.g., $K = f(serialnumber)$. For customer retention, often the base stations of one manufacturer shall only cooperate with transponders of the same manufacturer, otherwise a competitor may sell replacement transponders. Thus, f may involve a secret parameter linked to the manufacturer, e.g., a manufacturer key, that is used once in the factory to produce a new transponder and later in the base station when the transponder is registered. This type of key-learning is implemented, e.g., in the KEELOQ access control system analyzed in Chap. 8 (see Sect. 8.2.3 for details). Note that such a scheme comprises a **SPOF**, too: despite the unique keys of the remotes, the overall security of the system collapses, once the master key falls into the hands of an attacker.

One option for a key derivation without a **SPOF** is generating the secret keys of a transponder by means of an advanced key-derivation function f_{SEED} in which (in addition to a manufacturer key) a random seed contributes to the computations. This seed is assigned uniquely to each transponder during manufacture and can be transmitted to a base station for deriving the individual keys of that transponder in the field. The security of such a scheme is analyzed in this thesis (in the context of KEELOQ rolling-code systems) by implementing an exhaustive key-search attack in Sect. 8.5.5: if the size of the seed is chosen sufficiently large, such a key derivation can provide a very high security level. However, the (one-time) communication of the transponder's seed to the base station poses a security risk in practice.

In the key derivation scheme of the Basic Access Control (**BAC**), implemented in electronic passports, the cryptographic keys of an individual passport are derived from data that is printed on the document and can be read by humans or machines. Hence, these keys are not secret, but can be obtained by anyone who has access to the passport. The **BAC** is further described and cryptanalyzed in Chap. 9.

2.4 Passive RFID: Operation Principle

This section illustrates the most relevant characteristics of the different technologies used for passive **RFID**, i.e., for those devices that operate without their own energy source. Further details about these and other **RFID** systems are given in the **RFID** Handbook [Fin03], e.g., a comprehensive overview about all frequency bands used for **RFID** is given in Chap. 10 of [Fin03].

In the **RFID** literature the term “reader” is dominantly used to refer to a base station, hence in the following both terms are utilized synonymously. A minimum **RFID** system as depicted in Fig. 2.9 consists of two main components, namely a reader generating an **EM** field which supplies the second component of the system, termed transponder or tag, with energy and often a clock. Both devices are equipped with a coupling element, i.e., some type of antenna, that enables supplying the energy from reader to tag, and further transferring data in both directions. Note that an **RFID** reader is (despite of its name) also capable of writing data to a transponder. **RFID** transponders appear in various shapes and sizes, e.g., the form factor of a

credit card or coin-shaped, and can be embedded into different materials, e.g., paper, plastic, or tissue².

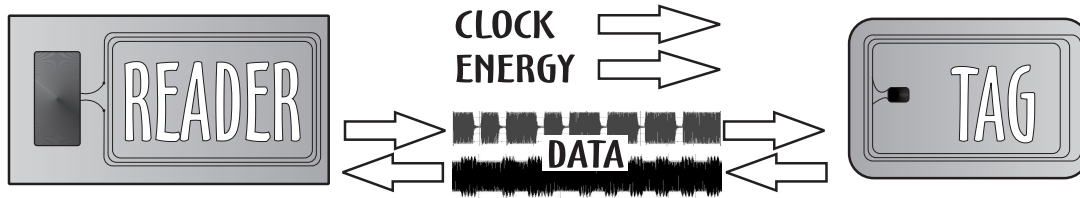


Figure 2.9: Operation principle of passive RFID: A reader provides the energy supply (and often also the clock) of the tag or transponder via the RF link.

The reader usually initiates the communication while the tag responds to its requests. As in many **RFID** applications multiple tags may be present in the field of the reader, an anticollision procedure is often part of the protocol, ensuring that only one card at a time is selected.

2.4.1 Long-Range RFID Systems

Long-range **RFID** systems in the **VHF**, **UHF**, or **SHF** bands are usually coupled using so-called *backscattering* and can operate at a distance of several meters. Similar to the principle of the early **RADAR**-based **IFF** systems, the wavelength $\lambda = c/f_c$ of the **EM** field, where c denotes the speed of light (approx. $3 \cdot 10^8 \text{ m/s}$) and f_c the carrier frequency, is very small compared to the typical operating distance of several meters between reader and tag³. For example at $f_c = 200 \text{ MHz}$ in the **VHF** range the wavelength equals $\lambda = 1.5 \text{ m}$, at $f_c = 2.4 \text{ GHz}$ in the **UHF** range $\lambda = 12.5 \text{ cm}$, and at $f_c = 5.8 \text{ GHz}$ in the **SHF** range $\lambda = 5.17 \text{ cm}$. An **EM** wave transmitted by the reader thus propagates from the antenna of the latter and hits the tag to be identified in the so-called far field, where the signal has already lost a large portion of its initial energy. Some part of the **EM** wave is reflected by the antenna of the tag: The amplitude or other physical characteristics of this reflected wave can be modified by the tag in order to transmit its response. The returning **EM** wave is then detected and interpreted on the side of the reader. An inherent property of (passive) long-range **RFID** systems is that, due to the losses in the far field, the amount of energy that can be transferred to the tag is extremely small and only slow data rates can be achieved.

2.4.2 Short-Range RFID Systems

In contrast, *inductively coupled* **RFIDs** are designed to operate in the **LF** and **HF** bands at smaller reading distances in the order of some centimeters up to a few meters. Their name is derived from their coil-shaped antennae that electrically appear as inductances and are coupled similar to the common transformer principle. The corresponding wavelengths for **HF** and **LF** are approximately $\lambda = 22.1 \text{ m}$ at $f_c = 13.56 \text{ MHz}$ and $\lambda = 2400 \text{ m}$ at $f_c = 125 \text{ kHz}$, respectively, and therefore several times greater than the typical operating distances between reader and tag.

²Embedding **RFID** transponders in metal environments still poses a problem in practice.

³Note that the wavelength λ in general also determines the size of the antenna required for transmitting or receiving data and hence the dimensions of the tags. As a rule of thumb, **EM** waves are reflected by objects that are at least half as big as λ .

Due to the operating range being smaller than λ , no **EM** wave can propagate from the antenna before it reaches the tag, hence the far-field components of the **EM** field are negligible, and only the near-field components need to be considered for technically describing an inductively-coupled system. As a consequence, the field emitted from the coil of the reader can be treated as purely magnetic and is well-suited for establishing a powerful communication and energy link between reader and tag.

Likewise, *capacitively coupled* systems exist in which the coupling elements of tag and reader can be regarded as the plates of a capacitor. The near field used for establishing the communication and power supply is thus purely electric. The operating range is usually small and only little energy can be supplied for practical antenna sizes. However, in practical applications (e.g., passive remote keyless entry systems for cars) the capacitance of the human body can be used to extend the range.

In summary, due to the physical properties of the used wavelengths and coupling elements, a trade-off between operating range, data rate, and computational power (without considering area and cost) has to be made in the world of passive **RFID**. As most implementations of cryptography demand for a significant amount of computational power, backscatter-coupled and capacitively-coupled systems usually cannot provide sufficient energy via the **RF** field and are thus not suitable. Instead, inductively-coupled **RFIDs** are predominant in security-relevant applications. In the remainder of this thesis the term **RFID** hence comprehends short-range **RFID**, i.e., passive, inductively coupled systems. Contactless smartcards, as detailed next, are the prevalent choice for implementing secure **RFID** systems.

2.5 ISO / IEC 14443 Standard for Contactless Smartcards

ISO/IEC 14443 [Int01] is an international standard for contactless smartcards operating in the **HF** frequency band at $f_c = 13.56$ MHz in close proximity with the antenna of the reader. The tags are usually shaped like credit cards and contain a silicon chip with the electrical circuitry that is connected to an antenna coil with typical 3–10 windings. The chip often comprises an embedded smartcard microcontroller or a digital control unit, some sort of internal memory, and an analog front-end for the power supply and communication with an **RFID** reader through the **RF** interface. In the standard, a reader is termed Proximity Coupling Device (**PCD**) and a contactless smartcard is called Proximity Integrated Circuit Card (**PICC**). **PICCs** as specified in ISO 14443 are specified to operate within approximately 8–15 cm of the antenna of a **PCD**. The majority of contactless smartcards on the market is compliant to ISO/IEC 14443.

2.5.1 Overview of ISO 14443

The standard comprises four parts: Part 1 specifies the size and physical characteristics of a card, as well as the environmental conditions it must be able to withstand without permanent damage. Part 2 defines the **RF** power and signal interface for data transmission and is divided into two half-duplex⁴ signaling schemes denoted Type A and Type B. Part 3 specifies the initialization and anticollision protocols and defines commands and responses for the initialization and anticollision scheme, its data frames and timing parameters. The protocol is designed to support multi-protocol readers that are capable of communication with both Type A and Type

⁴Communication only in one direction at a time

B cards. Both communication schemes support high bit rates of up to 848 kBit/s after the initialization (at a bit rate of 106 kBit/s) is completed.

Part 4 is optional and specifies the high-level data transmission protocol. A card reports if it supports Part 4 commands in its response to a polling command defined in Part 3. The protocol defined in Part 4 is also capable of transferring Application Protocol Data Units (**APDUs**) as defined in the standard for contact-based smartcards ISO/IEC 7816-4 and further supports application selection as defined in ISO/IEC 7816-5 [Int04a].

2.5.2 Unique Identifier

Every ISO/IEC 14443 compliant contactless smartcard contains a **UID**, which can be a fixed or random number of single (4 bytes), double (7 bytes) or triple (10 bytes) size. A *fixed* **UID** is usually factory-programmed into a read-only section of the Electrically Erasable Programmable Read-Only Memory (**EEPROM**) or into a one-time programmable memory of the card and, as the name suggests, uniquely identifies the smartcard. A *random* **UID** is randomly generated by the card at the beginning of every transaction with a reader, i.e., on each reset or power-up, and intends to prevent from tracking (see Sect. 2.2.6).

Some security-relevant systems based on contactless smartcards rely on the assumption, that a **UID** is indeed unique for a card and cannot be copied to another card: this is true for genuine cards, e.g., Mifare Classic, but does not hold for faked cards⁵ or card emulators, such as the “Chameleon” developed in Chap. 5 of this thesis.

2.5.3 Physical Layer Type A

All contactless smartcards investigated in this thesis are compliant to Type A of ISO 14443, as detailed in this section. The bit duration during the initialization of the card at a data rate of 106 kBit/s equals

$$\tau_b = \frac{128}{13.56 \text{ MHz}} \approx 9.44 \mu\text{s}.$$

The bit duration after the initialization depends on the mutually agreed data rate which can be set to 106 kBit/s, 212 kBit/s, 424 kBit/s or 848 kBit/s.

Communication Reader → Card

The Type A signaling scheme utilizes 100% Amplitude-Shift Keying (**ASK**) of the **RF** field for communication from the reader to the card, i.e., the field is completely switched off for short periods. Figure 2.11.a depicts the measured waveform of an ISO 14443A-compliant reader sending data to a card using a modified (pulsed) Miller code, as detailed next.

Modified Miller Encoding For generating a modified Miller code, all transitions occurring in a Miller-encoded sequence of bits, as illustrated in Fig. 2.10, are substituted by short pauses with duration of approximately 2.5 μs . The card must hence store sufficient energy in its internal capacitors to keep its circuits alive during these pauses.

⁵The author has obtained faked Mifare Classic cards from a chinese supplier and has verified that all functionality is equivalent to genuine Mifare Classic cards, with the exception that the **UIDs** of the faked cards can be rewritten.

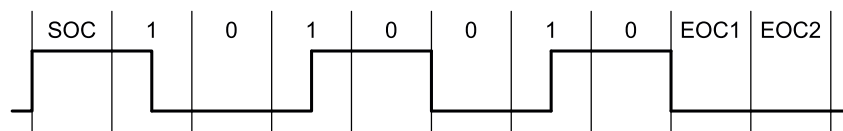


Figure 2.10: A sample data sequence encoded with a Miller code. Substituting the transitions with pauses results in a modified Miller code as utilized for transmitting data from reader to card.

Start Of Communication (**SOC**) and End Of Communication (**EOC**), marking the beginning and the end of a data packet, as well as logical 1 and 0 are thus encoded as follows.

SOC Pause at the beginning of the bit duration

EOC Logic 0 followed by no modulation for a full bit period

Logic 1 Pause after $\tau_b/2$

Logic 0 Pause at the beginning of the bit period, if the previous bit was a Logic 0 or an **SOC**. No modulation for a full bit period, if the previous bit was a Logic 1.



(a) Miller coded on-off-modulation.



(b) Manchester coded load-modulation.

Figure 2.11: Measured waveforms of the modulations used to transmit data according to ISO 14443A. The 100 % amplitude modulation generated by the reader (a) is easier to detect than the response of the card (b).

Communication Card → Reader

Communication from card to reader utilizes On-Off-Keying (**OOK**) modulation of a subcarrier with Manchester encoded data, i.e., the subcarrier is switched on and off depending on the current logic value of the data bit. Due to the inductive coupling, the action of a card drawing more or less energy from the field can be sensed on the side of the reader. Hence, the card transmits data by switching on and off an additional load and thereby deliberately draws more energy from the **RF** field than during normal operation. This process is termed load modulation.

Load Modulation Load modulation works similar to the common electrical transformer principle. If a card is placed within the interrogation zone of a reader, the input impedance of the card is coupled via the antenna coils to the resonant circuit of the reader. Increasing or

decreasing the impedance at the antenna of the card causes a tiny modulation of the amplitude that can be detected at the antenna of the reader. As the coupling between card and reader is very weak, the resulting effect on the amplitude of the field is almost not noticeable. To improve the detectability, a *subcarrier* of the frequency $f_c = 13.56$ MHz of the reader is used for the load modulation, resulting in the transmitted information being placed in the sidebands of the carrier sine wave [Fin03]. This subcarrier at a frequency of $f_c/16 = 847.5$ kHz transmits Manchester-encoded data synchronously to the field of the reader, as illustrated in Fig. 2.11.b.

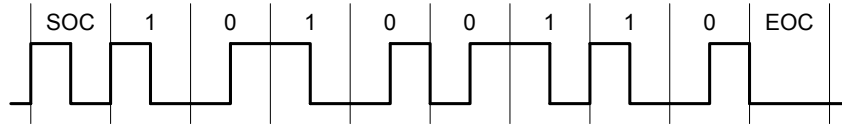


Figure 2.12: Sample Manchester-encoded data sequence transmitted from card to reader.

Manchester Encoding A sample Manchester-encoded bit sequence generated by the card is presented in Fig. 2.12. SOC, EOC as well as logical 1 and 0 are coded as follows:

SOC The carrier is modulated for the first half of the bit period $0 \leq t \leq \tau_b/2$. Note: an SOC is encoded just like a logical 1 (see below).

EOC The carrier is *not modulated* for the duration of one bit.

Logic 1 The carrier is modulated for the first half of the bit period $0 \leq t \leq \tau_b/2$.

Logic 0 The carrier is modulated for the second half of the bit period $\tau_b/2 \leq t \leq \tau_b$.

Note that ISO 14443 requires all cards in the field to reply in a synchronous manner. For instance, one card sending a logical 0 and a second card sending a logical 1 at the same instant results in a not defined “collision” condition, i.e., the carrier being modulated for a full bit duration. Accordingly, the presence of multiple responding cards can be detected by the reader in the anticollision phase of the protocol, as detailed in Sect. 2.5.6.

2.5.4 Frame Formats

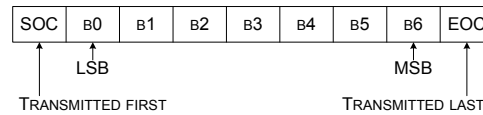


Figure 2.13: Short frame format for transmitting short commands with a length of 7 bits.

There are two types of frame formats for wrapping data packets specified in ISO/IEC 14443-A: the short frame and the standard frame. A *short frame*, as depicted in Fig. 2.13, consists of an SOC and seven data bits followed by an EOC. A *standard frame*, as shown in Fig. 2.14, consists of an SOC, N data bytes, each with an appended parity bit P, and an EOC. The odd parity bit P is computed such that the XOR of all ones in a byte and its appended parity bit equals one. The parity bit enables detecting an odd number of transmission errors. Note that

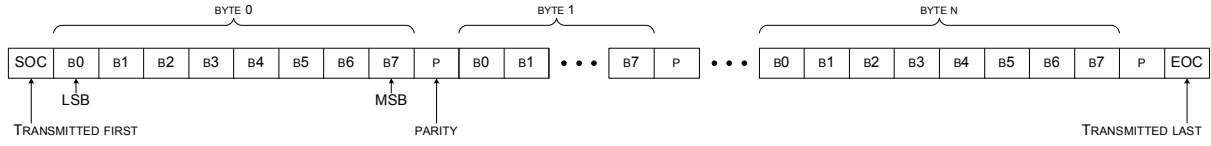


Figure 2.14: Standard frame format with (odd) parity for transmitting longer data blocks.

the Least Significant Bit (**LSB**) is transmitted first. The commands **REQA** type A (**REQA**) and Wake-UP A (**WUPA**) are transmitted according to the short frame format, while most other commands specified in ISO/IEC 14443 are transmitted as standard frames.

2.5.5 Timing Specifications

The Frame Delay Time (**FDT**) is the time that elapses between two transmitted frames. For the commands **REQA**, **WUPA**, **ANTICOLLISION** and **SELECT** the **FDT** between the end of the last pause transmitted by the reader and the first modulation edge of the **SOC** transmitted by the card must be exactly $1172/f_c = 86.43 \mu\text{s}$, if the last bit sent by the reader was logic 0, and $1236/f_c = 91.15 \mu\text{s}$, if the last bit sent by the reader was logic 1. For all other commands the card must ensure that its response is aligned to a bit grid given by $128n+20/f_c$, $n \in \mathbb{N}$, $n \geq 9$, if the last bit sent by the reader was logic 0, and $128n+84/f_c$, $n \in \mathbb{N}$, $n \geq 9$, if the last bit sent by the reader was logic 1. In the opposite direction, i.e., the time elapsed between the last modulation transmitted by the card and the first pause transmitted by the reader, only a minimum **FDT** of at least $1172/f_c = 86.43 \mu\text{s}$ has to be considered. The minimum time between the start bits of two consecutive **REQA** commands, termed Request Guard Time (**RG**T), has the value $7000/f_c \approx 516 \mu\text{s}$.

2.5.6 Anticollision and Initialization Protocol

At the beginning of the initialization process the reader issues a **REQA** command and all cards present in the field synchronously reply with an Answer To Request A (**ATQA**). For selecting one of the cards the reader starts the anticollision procedure consisting of one, two or three Cascade Levels (**CL**s) CL_n , depending on the size of the **UID**. In each **CL** a part of the **UID** is transferred from card to reader.

Initially, the reader issues the **ANTICOLLISION** command, prompting all cards in the field to reply with their respective **UID** for CL_1 and an appended exclusive-or check byte (**BCC**). The presence of more than one smartcard in the field can be sensed by the reader, as described in Sect. 2.5.3. In case of a collision the reader sends a new **ANTICOLLISION** command providing the first bits of the **UID** that are shared between the cards and an appended, chosen bit at the position where the collision has occurred. In the following, all cards that have previously sent the opposite bit-value at the position of the collision will not respond any more. This principle of reducing the number of cards that are replying is repeatedly applied until only those cards whose CL_1 -part of the **UID** is identical remain active.

After acquiring the collision-free **UID** for CL_1 the reader issues a **SELECT** command to select *all* cards with the same **UID** for CL_1 , while cards with a different **UID** for CL_1 do not reply. In case of a single-size **UID** the anticollision process ends here and the selected card responds with a Select Acknowledge (**SAK**). For double- and triple-size **UID**s the selection process is repeated

likewise for the higher cascade levels CL₂ and CL₃, until the complete **UID** is acquired by the reader and exactly one card in the field is selected.

In case the selected card supports the higher-level communication protocol specified in ISO/IEC 14443 Part 4, the reader issues a Request for Answer To Select (**RATS**) command, indicating the maximum frame size supported by the reader. The card replies with the Answer To Select (**ATS**) which contains information about the maximum frame size and the maximal bit rate in both directions, as supported by the card. An optional Protocol and Parameter Selection (**PPS**) command sent by the card, following a Protocol and Parameter Selection Request (**PPSR**) from the side of the reader, allows individually selecting the communication baud rate between reader and card.

In order to communicate with more than one card simultaneously, the reader can issue a HaLT type A (**HLTA**) command to put the currently activated card on hold. The card will remain in the suspended state until it receives the **WUPA** command which reactivates the card for communication. The details of the aforementioned and all further commands are specified in the ISO/IEC 14443 standard [Int01]. Some contactless cards in addition support proprietary commands, depending on the manufacturer.

2.5.7 Other Standards for 13.56 MHz

In addition to ISO 14443, two more standards are popular for wireless devices operating at 13.56 MHz, namely vicinity cards according to the ISO 15693 standard and **NFC** devices.

ISO 15693: Vicinity Cards

An alternative to proximity cards as defined in ISO/IEC 14443 is the ISO 15693 standard for vicinity cards [Inta], i.e., cards which can be read from a greater distance compared to ISO 14443. ISO 15693 systems also operate at $f_c = 13.56$ MHz and offer a maximum reading distance of 100–150 cm.

Due to the larger operating range of vicinity cards, the strength of the magnetic field at the card, required for powering the card and to establish a communication link, is defined smaller (0.15–5 A/m) than that at a proximity card (1.5–7.5 A/m). The lower amount of available energy implies that vicinity cards possess less computational power. Accordingly, the latter is often not sufficient for implementing effective cryptographic schemes. Furthermore, the maximum data rate of vicinity cards is reduced to 26.48 kBit/s. One of the major target markets for ISO 15693-compliant contactless chips is tracking and identifying of consumer products with so-called smart labels, similar to a barcode printed on the product, and other goods-tagging applications.

Near Field Communication (NFC)

NFC is an International Organization for Standardization (**ISO**) and European Computer Manufacturers Association (**ECMA**) standardized interface for short-range wireless communication [Int04b, ECM] that is fully compatible to ISO 14443 proximity contactless smartcards [Int01] and ISO 15693 vicinity contactless smartcards [Inta]. Being integrated into mobile devices for the consumer market, **NFC** is supposed to ease financial transactions and to make

the exchange of information more convenient. The driving force behind **NFC** includes companies from various industrial and service sectors like Vodafone, Deutsche Bahn, Telekom, NXP and BNP Paribas. The number of **NFC** enabled devices is increasing steadily, most of them being smartphones: Today, every large manufacturer of smartphones, e.g., BlackBerry, LG, Motorola, Nokia, Sagem, or Samsung, is vending at least one **NFC**-enabled model. Many different applications and use-cases in areas such as mobile ticketing, electronic payment, identification, car sharing, or access control, are currently being developed or are already in operation.

NFC distinguishes two types of devices: An active device (corresponding to a reader in ISO 14443) generates and modifies its own carrier field, while a passive device (corresponding to the contactless card) only modifies an existing carrier field. The initiator of the communication must be an active device and thus have a power source. Three operating modes are supported: *peer-to-peer* communication, i.e., two **NFC** devices communicating in active mode; *card emulation*, in which the user's device, e.g., a smartphone, operates passively and is read by an active device; and a *reader mode*, in which the **NFC** device functions as a reader, e.g., to read out a contactless card.

Depending on factors like power supply, type of antenna and numerous other conditions, the practical working distance for the latter two modes⁶ is for most smartphones approximately 5 cm with supported data rates of 106, 212 and 424 kBit/s for active and passive communication. As in ISO 14443, modified Miller coding and Manchester coding are used for encoding the communication data (see Sect. 2.5). The security aspects of **NFC**, some of which are identical to those of contactless smartcards, are detailed for example by Haselsteiner et al. [HB], Madlmayr et al. [MLKS08], and Mulliner [Mul09].

2.6 Examples of Contactless Smartcards

Many different types of contactless smartcards are available on the market, produced by various manufacturers. This section introduces some exemplary cards and summarizes their features.

2.6.1 Innovatron Calypso

The ISO/IEC 14443 B compliant Calypso card [Inn05] has been introduced and patented in 1993 by the French company Innovatron, with the first implementation following in 1996. The Calypso technology offers a microprocessor-based contactless interface and is currently deployed in public transit systems of some major European cities, including Paris, Lisbon and Brussels.

The data stored on the card is organized in hierarchical files compliant to ISO/IEC 7816-4 [Int04a]. Calypso employs a proprietary secure session mechanism that allows mutually authenticating reader and card, encrypting all data transmitted, and also deals with the integrity problems occurring when the card accidentally leaves the **RF** field of the reader during an update of its data. Various types of Calypso-compatible cards offer different levels of security. Calypso cards employ the **DES**, **DES-X** and **3DES** algorithms with a key length of 64, 120 and 112 bit, respectively.

⁶The peer-to-peer mode provides an increased operating range

HID iClass

Launched in 2002 by HID Global, the HID iClass contactless smartcard [HID02] is delivered in versatile packages offering compliance to both ISO/IEC 14443 A and B types. The cards are available in 256 Byte, 2 kB and 4 kB memory configurations and can be used for multiple applications. The maximum supported data rate is 212 kBit/s. A mutual authentication between card and reader, as well as encryption of the transmitted data, is based on a non-disclosed proprietary encryption algorithm using 64-bit diversified keys. Additionally, according to manufacturer, higher levels of security may be achieved with the supported DES and 3DES encryption.

2.6.2 Legic Identystems Cards

The Swiss company Legic Identystems AG produces several types of contactless smartcards. Their basic product, Legic Prime, provides no effective cryptographic measures [PN09]. The security of their advanced cards, termed Legic Advant, has to the knowledge of the author not yet been analyzed. Legic Advant transponder chips are offered in five memory configurations: 128 byte, 256 byte, 1 kB, 2 kB and 4 kB. The models up to 1 kB memory are compliant to ISO 15693 and the remaining two models are compliant to ISO/IEC 14443 A. The memory can be subdivided in up to 127 applications. The data rate for communication is 106 kBit/s for the 2 kB card and up to 848 kBit/s for the 4 kB card. The Legic safeID feature enables a secure (authenticated) transfer of the UID of the card, which can be of single (4 byte) or double (7 byte) size. Encrypted data transfer and encrypted data storage can be defined per application. Support for DES, 3DES, AES (only for the 4 kB version), as well as a proprietary LEGIC encryption algorithm, is included for this purpose [LEG]. Recently, we found by investigating some cards that Legic also seems to employ DESfire cards (see below), labeled as Legic cards, in their systems.

2.6.3 Electronic Passport

The Electronic Passport (ePass) has been deployed in the European Union (EU) and many other countries across the world as a governmental identification document, e.g., at border controls. The paper document is combined with an embedded RFID chip in order to reduce fraud, ease identity checks, and enhance security. Other optional services that can be facilitated with the introduction of the ePass technology include registering a change of address, registering a vehicle, filing a tax return (eGovernment), casting a vote in elections (eVoting), and other services provided by the retail, banking, and insurance sectors (eBusiness). Meanwhile, several countries have implemented similar technology into their ID cards, e.g., the New German ID Card (nPA) in Germany which has the form factor of a credit card.

Technical specifications for the new passports, published by the European Commission [Eur06], are binding for the Schengen-agreement countries. The embedded contactless chip of all e-Passports issued in the EU contains at least the same information that is printed on the identity information page of the passport, e.g., the name of the holder, date of birth, and a facial image. Optionally, an e-Passport can contain biometric identifiers, e.g., a fingerprint or an iris scan of the holder of the passport. To ensure interoperability between different countries,

all e-Passports, as well as the **nPA** and various ID cards of other countries, comply to the ISO/IEC 14443 standard.

In Germany, e-Passports have been introduced in November, 2005. The chip of the first generation contained only a photograph as a biometric identifier. The security of this first version is questionable, as detailed in Chap. 9. After the publication of our attacks on the **ePass**, presented in Sect. 9.5, the second generation with improved security mechanisms followed in November 2007, additionally containing two fingerprints of the card holder. Some German e-Passports employ microcontroller-based SmartMX [NXP04b] chips produced by NXP with 72 kB **EEPROM**, while others use similar chips produced by Infineon. The implemented security mechanisms include asymmetric cryptography, e.g., **ECC** enabling short key-lengths, as well as symmetric cryptography, e.g., **3DES** and **SHA-1**. Various (optional) protection mechanisms aim at preventing different types of attacks:

- The contactless chip replies with a random **UID** in each communication to prevent tracking of a person.
- **BAC** provides a means of mutual authentication and encrypting the data exchanged between the **ePass** and the reader. The secret key for the **BAC** is derived from the Machine Readable Zone (**MRZ**) printed on the document.
- Passive Authentication (**PA**) is supposed to prevent modification of the data stored on the passport chip: Hash values and digital signatures of all relevant data are stored on the chip and can be verified by a reader to check the authenticity and integrity of the information.
- Active Authentication (**AA**) can be used to counteract cloning of passport chips. For this purpose a secret key is stored on the chip that can be neither read nor copied. However, its correctness can be proven by means of the **AA** protocol.

The **nPAs** recently issued by the German authorities provide the patent-free Password Authenticated Connection Establishment (**PACE**) [KNS⁺08, FOIS] to protect their content. Despite the weak passwords used, such as a code that is printed on the passport, the inventors of the scheme have proven that it is secure under certain assumptions [BFK09].

A rigorous research has been spurred by the introduction of e-Passports around the globe [CLRPS, HHJ⁺06, JMW05b, LKLRP07]. As a part of this thesis, an implementation of a key-recovery attack targeting the **BAC** scheme is detailed in Chap. 9. The chapter also covers further related work and a detailed description of the protocol and key derivation mechanisms implemented in the **ePass**.

2.6.4 NXP Mifare Cards

The Mifare product line, consisting of Mifare Ultralight (C) [NXP03, NXP08f], Mifare Classic [NXP08b, NXP08a, NXP08c], Mifare DESfire (EV1) [NXP04a, NXP08d], and Mifare Plus [NXP08e], is now maintained by the Philips semiconductor spin-off NXP. All Mifare cards comply with ISO/IEC 14443 A. Mifare cards are basically memory cards, i.e., information can be stored in the internal **EEPROM**, while an integrated digital control handles the communication with a reader. The “MIFARE Type Identification Procedure” published by NXP [NXP09a]

describes how to technically distinguish between the members of the Mifare card **IC** family by evaluating the information obtained from the card during the initialization procedures.

Mifare Ultralight is the smallest member of the Mifare family and has no cryptographic mechanisms implemented. All sectors can hence be directly read out or modified, and most applications using this card rely on the fixed, factory programmed **UID** that allows distinguishing the cards. The protocol of the cards and the corresponding commands are covered in Sect. 5.3.1.

Mifare Classic cards have made their way into many public transportation systems such as the Octopus card in Hong Kong [Oct97], the Oyster card in London [Tra03], the OV-Chipkaart in the Netherlands [OV-05] and the CharlieCard in Boston [Mas06]. The cards are widely used for access control and are also employed for payment applications, e.g., in the real-world contactless payment system analyzed in Chap. 6. Mifare Classic promises to prevent replay attacks, cloning and eavesdropping due to authentication and encryption by means of the proprietary CRYPTO1 stream cipher. Many universities, corporations and even government agencies use Mifare Classic chips for access control and cashless payments. Mifare Classic cards and the CRYPTO1 cipher, as introduced in detail in Sect. 6.3, have turned out to be flawed and thus provide only very little security — the corresponding attacks are illustrated in Sect. 6.4.

The high-end class of NXP's contactless cards is termed *Mifare DESfire* and in the latest version (EV1) provides a special protection against power-analysis attacks (compare with Sect. 3.2.2) that use measurements of the actual implementation of cryptography to reveal secret keys of even mathematically secure ciphers. The cards are used for high security applications, such as access control and identification purposes by the military and police forces, payment systems, and meanwhile serve as a replacement for many outdated Mifare Classic systems. Chapter 7 is dedicated to DESfire and gives detailed information about the cards. Furthermore, a full-key recovery of the secret keys used for their **3DES** encryption and authentication, by means of power analysis, is illustrated. The card emulator developed in Chap. 5 is capable of emulating all types of Mifare cards, including DESfire EV1. Details about the latter card, including the authentication protocol, are given in Sect. 5.3.2.

2.7 Typical Ranges for Activation and Eavesdropping

The practically achievable range for activating a wireless token, detecting it, or eavesdropping on the communication — also taking special equipment operating outside the boundaries of the specifications into account — constitutes a key role for the feasibility of various attacks on wireless devices (see Sect. 2.2), for example:

Eavesdropping From which distance is spying out information possible?

Cloning What is the maximal range for reading out a tag in practice?

MITM attacks How close does an adversary need to get to the victim?

DoS Which minimum distance margin around a tag (e.g., a fence) is required to prevent, e.g., a sleep deprivation attack?

Tracking and detection How distant can the adversary's antennae be placed?

Malware Can a virus be injected from afar?

Implementation attacks What are practical ranges for **EM** analysis and fault injection?

The above questions emphasize that understanding the physical characteristics of the wireless interface and determining the real-world ranges is highly relevant for the majority of the security threats. Since the technologies and **RF** interfaces of wireless tokens differ widely, each type of device has to be analyzed separately. Practical experiments incorporating customized equipment, e.g., a power amplifier [Mel] or a specialized receiver and antenna [Tex04], have to be conducted for each type of token. Especially in the context of **RFID** it is expedient to distinguish between the *operating range*, i.e., the range for powering the token via the **RF** field and actively communicating with it, and the *eavesdropping range* for passively monitoring the communication between a base station and a transponder.

2.7.1 Ranges of Contactless Smartcards

The operating range of contactless smartcards, for example an electronic passport or Mifare cards, is specified with approx. 10 cm. It can be increased up to a range of approximately 30 cm [FK, KW06], depending on the physical environment and also on the type of card, e.g., according to our experiments the maximal range for activating a Mifare Classic card is larger compared to a Mifare DESfire card.

Regarding passive eavesdropping, the asymmetric channels of the forward communication from reader to tag and the backward channel vice versa needs to be considered. While it is straightforward to intercept a reader during a communication by switching its **RF** field completely on and off, it can be far more difficult to acquire the small changes in amplitude caused by the load modulation generated by the tag (see Sect. 2.5.3). Eavesdropping of the complete wireless communication between an **RFID** reader and an ISO 14443A-compliant contactless smartcard is possible from a distance of several meters [FK, Han08], but requires special equipment, including an **RF** receiver and a large antenna, and a relatively skilled attacker. According to a study [BSI08] of the German Federal Office for Information Security (BSI), practical upper limits of an error-free interception of the two-way communication are 2.45 m for **ISO** 14443-systems and 2.65 m for **ISO** 15693 systems. Simulations in a white paper by NXP confirm the theoretical possibility of eavesdropping (without the presence of noise) from a maximum of 50 m [NXP07]. Hancke [Han06] practically demonstrated that the two-way communication between an **RFID** reader and an **RFID** tag can be intercepted from 4 m. Furthermore, the author states that it is feasible to increase this distance, e.g., with application specific antennas and more complex signal processing. Monitoring the forward channel is claimed to be possible from a distance up to about 25 m [Rob].

2.7.2 Ranges of Active Wireless Devices

Active tokens, such as KEELQ remote controls or other active transponders, possess their own transmitter and hence achieve long operating ranges, often in the order of (several) hundred meters. However, in our practical tests using the Universal Software Radio Peripheral (**USRP**) (see Sect. 4.2), the specified range of 122 m of a typical active transponder transmitting at a frequency of 433 MHz can be significantly increased: according to the experimental results, the feasible ranges for actively initiating a successful communication and passive eavesdropping can be increased to 500 m (by a factor of 4) and 1000 m (by a factor of 8), respectively [KOP11c].

Since low-cost equipment and simple analysis methods corresponding to an unskilled attacker have been used for the experiments, the determined ranges have to be regarded as a lower boundary which can be extended by a professional, well-funded adversary.

The achievable ranges imply various real-world threats for both passive **RFIDs** and active wireless tokens. The evolving attack scenarios are manifold, e.g., performing an unauthorized inventory of the warehouse of a competitor, reprogramming transponders attached to containers to change their intended destination, falsify the information recorded by sensors of the tag, and many others. In this thesis, in Chap. 6 contactless payment cards are actively read-out and modified from a distance, in Chap. 9 individuals are tracked by eavesdropping on their electronic passports, and Chap. 8 illustrates how monitoring the communication of active KeeLoq remote controls enables to clone remote controls and open doors unauthorizedly.

Chapter 3

Background: Physical Cryptanalysis

ACCORDING to Kerckhoff’s law [Ker83] the security of a system should rely only on the secrecy of the cryptographic keys, which therefore need to be protected carefully. Consequently, the aim of an attacker is usually to extract these keys of a cryptosystem. Numerous different paths lead to this goal, as illustrated by the classification of the field of cryptanalysis presented in Sect. 3.1. Where well-known, secure cryptography is employed in products, classical cryptanalysis based on mathematical models is very unlikely to be successful. Instead, physical attacks or implementation attacks can tackle the security of embedded devices. These attacks are covered in Sect. 3.2, with a focus on the very powerful, non-invasive class of Side Channel Analysis (SCA) attacks. While SCA attacks are straightforward for known implementations and customized attack platforms in the laboratory, analyzing a so-called “black-box” device, i.e., an unmodified real-world device whose implementation details are unknown to the attacker, is much more demanding. The general approach for analyzing a black-box device, as carried out for the SCA analyses in this thesis, is thus illustrated in Sect. 3.3.

3.1 Cryptanalysis: A Classification of Different Techniques

Figure 3.1 depicts an overview of different approaches for cryptanalysis: *Classical cryptanalysis* comprises all analyses that are feasible by working solely with a model, i.e., a theoretical description, of a cryptosystem. *Implementation attacks* on the other hand require access to the actual device which is executing the targeted cryptographic algorithm. Other attacks, such as social engineering or spying out passwords with video cameras, are not considered in the classification, since they target neither the precise cryptographic algorithm nor its implementation.

3.1.1 Classical Cryptanalysis

In classical cryptanalysis, a cipher is an abstract mathematical object, which turns some plain input data into encrypted output data, parameterized by a key. The security of a cryptographic algorithm is investigated by mathematical analysis. The system as a whole is modeled and proven to be theoretically secure under the assumed model, e.g., assumptions made about the adversary and the allowed inputs. Various models exist for describing the strength of the attacker, e.g., ciphertext-only, known or chosen plaintext, or related-key scenarios [Sti05, Chap. 1].

By analyzing the modeled cryptosystem, vulnerabilities in the design of the *protocol* can be pinpointed, e.g., if the information sent during an authentication allows concluding to the key, or whether a MITM or replay attack is feasible. The efficiency of *exhaustive key-search*, i.e., a

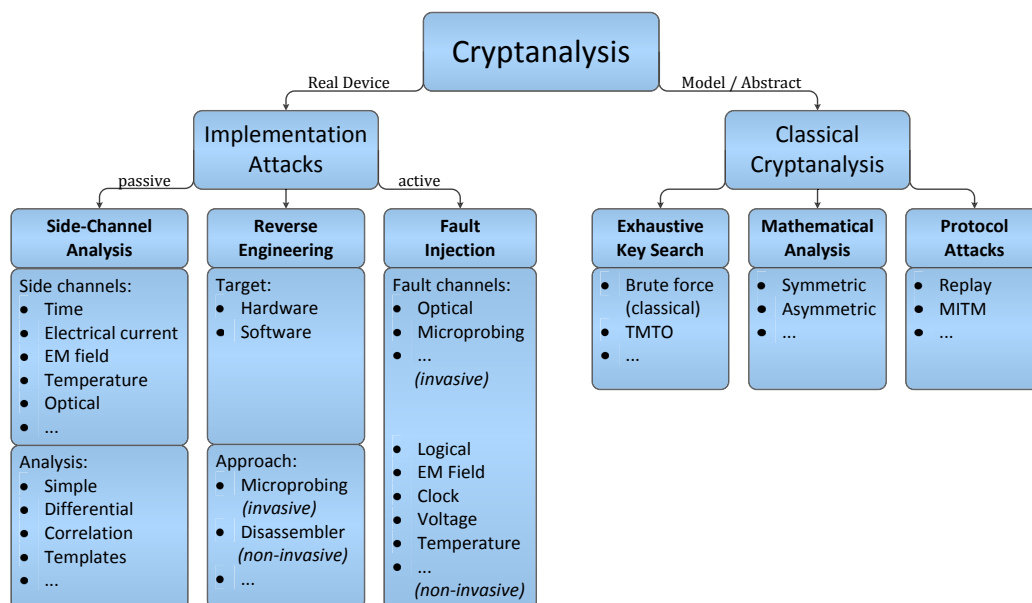


Figure 3.1: Overview of the field of cryptanalysis

classical brute-force attack, can also be evaluated at hand of a mathematical model¹. Often, a trade-off can be made between time, memory (e.g., precomputed values), and data (e.g., amount of known plaintext-ciphertext pairs) required for a successful attack, which is commonly referred to as Time-Memory Tradeoff (TMTO) and Time-Memory-Data Tradeoff (TMDTO). The first TMTO attack was published 1980 by Hellman [Hel80], improved 1982 by Rivest’s distinguished points method [Dor82, Chap. 2], and further improved 2003 with the so-called “rainbow tables” approach by Oechslin [Oec03].

Mathematical attacks exploit weaknesses of an algorithm at hand of its mathematical description. For an ideal cipher, a brute-force attack is the most efficient approach to reveal the secret cryptographic key. Mathematical analyses are hence considered to be successful, if their complexity for computing the secret key is smaller than brute force, e.g., a recent key-recovery attack on the full AES-128 requires $2^{126.1}$ computation steps [BKR11] — of course, performing such a huge amount of computations remains infeasible in practice. Likewise, after the introduction of linear cryptanalysis of DES in 1993 [Mat93], Matsui illustrated that the 56-bit secret key of DES can be revealed faster than exhaustive key search, if 2^{43} plaintext and ciphertext pairs are known to the attacker [Mat94]: Of course, gaining such a large number of known plaintext and ciphertext pairs is again infeasible in practice, while on the other hand a practical brute-force

¹Implementations of brute-force attacks on a certain target platform allow determining the real financial cost and practical run-times of the attacks.

attack on DES [GKN⁺08] takes less than one week². Similar considerations affect “practical” mathematical attacks targeting KEELOQ rolling code systems, as described in Sect. 8.2.4.

The methods for analyzing public-key (asymmetric) and secret-key (symmetric) cryptography differ strongly. Mathematical cryptanalysis of asymmetric cryptography is basically equivalent to solving a mathematically hard problem which forms the basis for the public-key scheme, e.g., factoring integers [Len87] or computing discrete logarithms [Gam83]. Algebraic cryptanalysis is applied to symmetric ciphers and numerous different techniques exist to tackle their security, e.g., linear [JR94] and differential [BS90] cryptanalysis, slide attacks [BW99], or cube attacks [DS09] that target in particular stream ciphers.

3.1.2 Implementation Attacks

Implementation attacks have been known to the scientific community since the late 1990s [BDL97, BS97] and require a real device instantiating a cryptographic algorithm. Since they are well documented in the literature, e.g., in the proceedings of the CHES and FDTC conferences, this thesis does not give a detailed compendium of all possible variants, but rather classifies the methods and details on those techniques that are employed for the analyses in the later chapters. The focus in the following is hence on non-invasive side-channel analyses.

In modern cryptographic systems, a cipher is instantiated in some type of electronic device³. The cipher can for instance be implemented directly in hardware, e.g., in the form of an ASIC or a Field Programmable Gate Array (FPGA). Furthermore, a program realizing the cryptographic algorithm can be running on the processor of an embedded system, such as a smartcard. The implementations contain the cryptographic secrets which are hence subject to new attack vectors. Implementation attacks are usually not considered in a mathematical model of a cryptosystem, since no method exists to fully account for the specific physical behavior of the electronic circuitry: each real-world system has its individual physical characteristics, e.g., the power consumption, the emanated frequency spectrum, or the execution time of the algorithm. If observing these characteristics delivers information about a secret involved in the computations, a passive SCA can be performed, as described in depth later in Sect. 3.2. Due to their non-invasive approach, SCA attacks are usually cost-effective.

Fault Injection While SCA passively observes a device without influencing its normal operation, fault injection on the contrary interferes with the target in a non-intended manner. An attacker actively manipulating the device, e.g., by altering its power supply, can purposely induce faults during the computations, producing an erroneous output. Often, this output can be mathematically evaluated to conclude to the secret key in a fault-injection attack.

A fault injection can be accomplished via the designated inputs of the device or by means of other (unintended) fault channels that directly target the electronic circuitry. For example, a disordered behavior can be triggered by supplying invalid input data that causes a logical error, such as a Structured Query Language (SQL) injection or a buffer overflow. Other *non-invasive approaches* to disturb the cryptographic algorithm inject faults via clock glitches, or by creating

² From the engineer’s point-of-view, the practical relevance of some mathematical definitions and attacks is incomprehensible. One has to be cautious when reading papers claiming that some cipher is “broken” — in the real world the attacks might not compromise the cipher in any practical way.

³ Even early electro-mechanical encryption machines such as the Enigma invented at the end of World War I or Vernam’s cipher telegraph (1926) are susceptible to various implementation attacks [Qui08].

EM pulses. Inducing optical faults, e.g., with a camera flash or a laser pointer [SA02], and fault injection by means of microprobing require baring the silicon die. These *invasive variants* of fault attacks leave physical traces (or even destroy the target) and are often more costly compared to the non-invasive techniques.

A special class of fault injection attacks target **RNGs**. Randomness is of great importance in implementations of many cryptographic schemes. An adversary can succeed in controlling or eliminating the randomness by a physical attack, for instance ring oscillators commonly used to generate random numbers can be attacked by means of frequency injection [MM09]. The basis for the highly efficient key-recovery attacks on Mifare Classic presented in Chap. 6 is a flawed implementation of the **RNG** built into the cards. It can be bypassed by precisely controlling the timing of the **RF** field and thereby faulting the power supply of the **RNG**, as demonstrated in Sect. 6.6.

Reverse Engineering Since the cryptographic secrets are stored and processed in the cryptographic device, another attack option is reverse-engineering the keys (or an undisclosed algorithm) from a *software* or *hardware* implementation. Obtaining and disassembling the program code from a processor is often possible in a non-invasive manner (unless microprobing of the data buses of the silicon chip was necessary). On the contrary, reverse-engineering of hardware chips usually requires an invasive approach in which the circuitry of the Device Under Test (**DUT**) is often destroyed during the analysis: The chip is separated from its plastic packaging, the layers of the silicon die are removed one after another, and the circuitry is optically inspected and reconstructed to conclude to the secret algorithm or the bits of the secret key [NESP08]. Reverse-engineering typically requires expensive equipment, much practical experience and can be very time-consuming. However, reverse-engineering software is generally less costly compared to analyzing a hardware implementation.

3.1.3 Types of Attacks Executed in this Thesis

Considering the type of attacker (i.e., an ordinary attacker using cost-effective equipment, as decided in Sect. 2.1) and the requirements of the different implementation attacks, the analyses performed in this thesis exclude invasive attacks which rely on directly tampering with the silicon wafer, e.g., probing attacks [HPS99] or reverse-engineering hardware. For state-of-the-art implementations of cryptographic algorithms, these attacks generally demand for highly sophisticated equipment and require a very strong and well-funded adversary. Instead, the following non-invasive attacks are practically performed on real-world targets:

- Fault-injection via the **EM** field circumvents the **RNG** of Mifare Classic cards,
- side-channel analyses exploiting the power consumption and **EM** emanations enable extracting keys from **RKE** devices and Mifare DESfire smartcards, and
- a brute-force attack with cryptographic hardware allows to decrypt data stored on a electronic passport.

The effects of the key-recovery attacks on the system security of the targeted real-world systems are analyzed in detail in the respective chapters.

3.2 Introduction to Side-Channel Analysis

SCA attacks have been known for decades in the mechanical world. Suppose a combination lock of a safe that requires the correct combination of digits to open the door. A brute-force attack, i.e., trying all possible combinations, is usually not feasible in practice. In contrast, a thief possessing a stethoscope might be able to open the safe in a realistic time by eavesdropping on the combinational lock while turning it: If the sound of the lock taking the position of a correct digit differs from the invalid positions, the attacker can find the correct digits one after another. By incorporating this unwanted information leakage, which reveals the position of the lock, the complexity of the attack is drastically reduced. Eventually, the protection mechanism is circumvented by monitoring a physical characteristic of the mechanical lock, i.e., the acoustic waves emanated by its mechanics. In this example, the side-channel “sound” is exploited to conclude to the secret combination code of the safe — in a similar manner, an electronic side-channel, e.g., the consumption of electrical current, can be exploited to conclude to secret information processed by a cryptographic implementation. Typical side-channels of cryptographic devices are

- the *time* an algorithm (or a part of it) needs for the execution,
- the *power consumption* of the device, and
- the emanated *electromagnetic field*.

Side-channel attacks in the context of cryptographic schemes usually attempt to recover small fractions of the secret key (which can be only one bit) one after another, in a divide-and-conquer approach. The recovery process is repeated for all subkeys until the full key is found.

3.2.1 Timing Analysis

A key-dependent runtime of a cryptographic implementation can be exploited as a side-channel. The idea was introduced by Kocher in [Koc96] while a formalized description can be found in [HMS00].

As a short example for a timing analysis, the private exponent e of the exponentiation

$$y = x^e \bmod n \tag{3.1}$$

shall be revealed: Assume that the calculation is performed with the Square-and-Multiply (**SAM**) method (left-to-right binary exponentiation) described in Alg. 1. Then, the run-time of the algorithm directly depends on the $p + 1$ bits of the secret exponent $e = (e_p, \dots, e_0)$. If a bit of the exponent is one, an additional multiplication by the base x (and a reduction by n) has to be performed, implying a longer processing time for a logical one compared to a logical zero.

Now assume that the first bits $e_p, \dots, e_{m+1}$ of the exponent are already known to the attacker and bit e_m is to be revealed next. For the attack, the execution times T_i for N modular exponentiations with a random (but known) base x_i are measured:

$$T_i = T(x_i^{e_m} \bmod n), \quad i = 1, \dots, N. \tag{3.2}$$

By modeling the timing behavior of the algorithm, in the following two hypotheses are tested at hand of the measured run-times: Hypothesizing that the unknown bit e_m is zero corresponds to a

Algorithm 1 Left-to-right binary exponentiation (Square-and-Multiply)**Input:** $x, n, e = (e_p, \dots, e_0)_2$ **Output:** $y = x^e \bmod n$

```

1:  $y \leftarrow 1$ 
2: for  $i = p$  downto 0 do
3:    $y \leftarrow y \cdot y \bmod n$ ;
4:   if  $e_i = 1$  then
5:      $y \leftarrow y \cdot x \bmod n$ ;
6:   end if
7: end for

```

simulated execution time $T_i^{m=0}$ until to the processing of bit e_m . The empirical variance, defined for a sample $t_1 \dots t_N$ as $v^2 = 1/N \sum_{i=1}^N (t_i - \bar{t})^2$, where $\bar{t}$ denotes the sample mean $\bar{t} = 1/N \sum_{i=1}^N t_i$, is computed for the zero hypothesis according to $V_0 = \text{Var}(T_i - T_i^{m=0})$. The process is repeated likewise for hypothesizing $e_m = 1$ and building the empirical variance $V_1 = \text{Var}(T_i - T_i^{m=1})$. Finally, the decision whether the exponent bit e_m was zero or one is reached according to

$$e_m = \begin{cases} 0 & \text{if } V_0 < V_1, \\ 1 & \text{else.} \end{cases} \quad (3.3)$$

A wrong guess of e_m leads to an increase of the empirical variance, provided that the simulation of the execution time correctly predicts the timing behavior of the target. Note that the same run-time measurements can be re-used for recovering the other unknown bits of the exponent e .

A timing attack is generally applicable (not only to the **SAM** algorithm but also to various other instances of ciphers), if conditional branches of the algorithm have different execution times depending on the secret key⁴. Statistical tests to verify the vulnerability of an implementation towards timing attacks are detailed in the Appendix A.

As a countermeasure against timing attacks, the algorithm should be implemented in a way that it always has a constant — thus key-independent — execution time, e.g., square-and-always-multiply [Cor99]. Furthermore, the processed data can be masked, for example, masking of the message in a public-key scheme following Chaum's message blinding method [Cha82], which can in addition be applied to the exponent [MOP07]: The exponentiation is performed with a modified (additively masked) exponent $\tilde{e} = \lambda \cdot \Phi(n) + e$, where λ is a new random value in each run of the exponentiation and $\Phi(n)$ denotes Euler's totient function. Due to Euler's theorem (for n prime) $x^{\Phi(n)} \equiv 1 \bmod n$, it follows that $y = x^e \bmod n = x^{\tilde{e}} \bmod n$ and hence the result of the computation with the blinded exponent $\tilde{e}$ remains correct. Since λ is not known to the attacker, she cannot simulate the execution time and thus cannot recover the exponent with the above described timing attack.

Note that, after a successful **SPA** attack (introduced in Sect. 3.2.2) she cannot directly conclude to the exponent e , but she can reveal the blinded exponent $\tilde{e}$: Knowing $\tilde{e}$ allows her to decrypt a message, e.g., in the context of **RSA**, hence securing an exponentiation against **SPA** only by means of exponent blinding is not sufficient. However, exponent blinding can protect

⁴Note, that the duration of a multiplication or a modulo reduction in Alg. 1 also depends on the base x .

against attacks requiring several queries, such as the above timing attacks and **DPA** attacks (introduced in Sect. 3.2.2). This simple example illustrates an important fact for securing cryptographic systems in practice: Protecting an algorithm against one type of (implementation) attack does usually not imply a raised security level against other (implementation) attacks — each practical attack vector needs to be addressed separately.

3.2.2 The Concepts behind Power Analysis

In contrast to a mathematical cryptanalysis which usually requires pairs of plaintexts and ciphertexts, in the context of power analysis knowing either the input or the output of the cipher suffices to mount a key-recovery attack⁵. Power analysis is based on measuring and evaluating the power consumed by a cryptographic device during its operation.

The consumption of electrical current can be acquired by inserting a small resistor in series with the power or ground line of the attacked **DUT**. The current I flowing through the **DUT** is usually derived from measuring the voltage U_R across that resistor and dividing by its resistance R , i.e., $I = U_R/R$ (Ohm's law). The power consumption of the **DUT** $P = U \cdot I$ depends linearly on the current I , as long as the voltage supplying the **DUT** can be considered constant⁶. The analogue signal, i.e., the monitored voltage U_R , is digitized and recorded on a computer as a discrete and quantized time series called a *power trace* in the literature — while the term “current trace” in fact would be more adequate. The length of a power trace is the product of its sampling rate, e.g., 500 MS/s and the duration of one measurement in seconds.

In practice, often several traces with varying input data are collected and processed. From the data-dependent information leakage contained in the traces, e.g., the amplitude of the peaks occurring in every clock cycle of a microcontroller, an attacker can derive intermediate results of the computations, e.g., the value of a bit being processed after the first round of **AES**. Thus, a divide-and-conquer strategy becomes possible, i.e., the secret key can be recovered in small fractions or even bit by bit.

Simple Power Analysis

The power consumption of a device can reveal details about the execution path of an implementation, such as the outcome of conditional branches depending on secret information. Let again serve the **SAM** approach given in Alg. 1 as an example to illustrate an **SPA** attack that exploits observations in a power trace. Since the algorithm sequentially processes the bits of the exponent, the knowledge whether a multiplication takes place or not directly reveals the currently processed bit. Figure 3.2 depicts the power consumption of an implementation of **RSA** on an 8051-based microprocessor: By visual inspection of the trace, an attacker can directly distinguish whether a multiplication occurs or not, and thus recover the secret exponent of the **RSA** signature from the sequence of instructions visible in the measurements.

An **SPA** is typically performed on a single power trace. However, a single measurement often contains unwanted noise. Messerges has observed several sources for noise in [Mes00], for example, the clock edges, quantization effects, and algorithmic noise. To improve the quality of

⁵The power analysis attack developed in Sect. 8.4 even enables a full-key recovery without knowing neither input nor output.

⁶The assumption holds for resistance values much smaller than the impedance of the DUT — for large values of R the voltage drop U_R , lessening the supply voltage of the DUT, has to be taken into account.

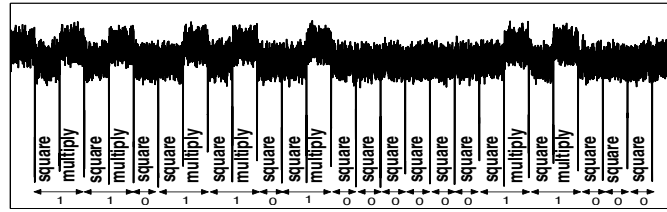


Figure 3.2: Simple power analysis of an implementation of RSA. For the analyzed 8-bit microcontroller, the power consumption of executing a multiplication is higher compared to a squaring: The duration of processing different bits of the secret exponent and thus their values can be visually distinguished.

the power traces, several measurements of the device performing the *same operations* with the *same input data* can be recorded. Computing the mean of those power traces (again yielding one trace) can improve the Signal to Noise Ratio (SNR) and thereby facilitate an SPA attack.

Differential and Correlation Power Analysis

Contrary to SPA, DPA and CPA exploit the relationship between the processed data and the power leakage of *multiple traces* with *varying input data*. The method takes the statistical properties of the input or output, i.e., often uniformly distributed plaintexts or the resulting ciphertexts, into account. A cryptographic device can be attacked by passively observing the power consumption during its normal operation, since no *chosen* plaintexts (or ciphertexts) are required. A *known* input (or output) is sufficient, as long as its statistical properties are known, which holds for the majority of cryptographic schemes. A DPA or CPA does not require a detailed description of the implementation of the algorithm, a fact rendering it a very powerful attack that can be applied to any unprotected black-box implementation. In addition to the secret key, the points in time where information leaks during the execution of the cipher are revealed. Hence, in subsequent analyses, the investigated length of a trace can be narrowed to the relevant time period to accelerate the key recovery.

Differential Power Analysis The DPA technique was first suggested by Kocher et al. [KJJ99] and later formalized by Messerges et al. [MDS99a]. The acquired power traces are divided into *two sets* according to the key hypotheses: An intermediate bit value of the cipher is predicted for each trace and each key hypothesis, at hand of a model of the cipher. The traces are then assigned to the two sets depending on whether the predicted bit is one or zero. For the correct key hypothesis, one set now contains only those power traces in which a one is being processed in a certain time instant, while in the other set the processed bit is always zero. Accordingly, when calculating the mean value of all traces in each set, for each point in time, a difference in the power consumption of the two sets must occur. For a good cipher and uniformly distributed plaintexts, the probability for a zero or a one being processed at any time instant during the computations is also uniformly distributed, i.e., a 50 % chance for a one or zero being processed. Thus the difference of the means of the sets for one and zero will vanish, except for the set belonging to the correct key hypothesis. When plotting the difference-of-means trace, a peak

indicates the time instant of the information leakage. The approach is further illustrated below in this section.

Correlation Power Analysis In a CPA, i.e., an improved variant of DPA, several key bits are hypothesized at the same time. The power consumption caused by the targeted bits is predicted at hand of a model of the implementation. The correct key candidate is then identified by calculating a correlation coefficient. A maximum correlation between the hypothetical power consumption and actually measured power values indicates the correct key hypothesis [BCO04].

Prerequisites A first-order CPA attack requires the following four conditions:

- (1) The attacker has physical access to the device and can measure the power consumption during several encryptions or decryptions. Variations of power analysis attacks, e.g., exploiting the EM side-channel, can be performed without direct physical contact, but the DUT still has to be in the vicinity of the attacker.
- (2) The input or output of the cryptographic algorithm in each run, e.g., the plaintext or the ciphertext of a cipher, is known, i.e., it can be monitored by the attacker. In some cases, even this information can be derived from the power traces, as practically demonstrated in Sect. 8.4. The known data is stored in a vector $\mathbf{d} = (d_1, \dots, d_i, \dots, d_D)$, where d_i denotes the plaintext or ciphertext of the i^{th} decryption or encryption run, and D is the total number of measurements.
- (3) An intermediate result during a de-/encryption must be a function of the processed data d_i and a small part of the key⁷ $\mathbf{k}^* = (k_1^*, \dots, k_j^*, \dots, k_K^*)$, where K is the total amount of key candidates k_j^* that influence the intermediate result.
- (4) The power consumption of the device must depend on the processed data. The trace containing the measured, digitized power consumption that corresponds to the known data d_i is denoted as $\mathbf{t}_i = (t_{i,1}, \dots, t_{i,l}, \dots, t_{i,T})$, where $t_{i,l}$ is the amplitude of the power consumption for the i^{th} de-/encryption run at the time instant l and T denotes the length of the traces. The traces of all runs can be stored in a *power trace matrix* $\mathbf{T}$ with dimension $D \times T$.

Hypothesizing Key-Dependent Intermediate Values The basic idea of a CPA is to hypothesize the influence of one or more bits of the secret key on an intermediate result which depends on the processed data d_i and a key candidate k_j^* . Intermediate values with a non-linear dependency on the key bits, e.g., the output of an S-Box after the key addition, are beneficial for the later analysis. The hypothetical intermediate values $v_{i,j}$, where i denotes the i^{th} encryption (or decryption) and j corresponds to the j^{th} key candidate, are calculated for all D encryption (or decryption) runs and for all K key hypotheses. This results in a *matrix of intermediate values* $\mathbf{V}$ with dimension $D \times K$, where column j contains the hypothesized intermediate results for the key hypothesis k_j^* .

⁷The amount of subkeys is usually sufficiently small when attacking single S-Boxes with few in- and outputs.

Power Models: Mapping Intermediate Values to their Power Consumption The *hypothetical power consumption matrix* $\mathbf{H}$ is derived from the hypothetical intermediate values $\mathbf{V}$ via a power model. Since the dynamic power consumption of a device is caused by the switching activity of the logic gates [Eri07], a proper dynamic consumption model tries to express the power consumption depending on the intensity of the switching activity. This switching activity may be caused, e.g., by the combinational logic computing the result of a Boolean function, or by a latch storing a single bit value. The better the power model matches the actual power consumption characteristics of the analyzed device, the less power traces are needed for a key recovery.

The most common *power models* are

- the bit model⁸ (here: Least Significant Bit (LSB)):

$$h_{i,j} = \text{LSB}(v_{i,j}), \quad (3.4)$$

- the Hamming weight (HW) model (number of ones in a data word):

$$h_{i,j} = \text{HW}(v_{i,j}), \quad (3.5)$$

- the Hamming distance (HD) model (number of toggling bits in a data word):

$$h_{i,j} = \text{HD}(v_{i,j}, d_i), \quad \text{and} \quad (3.6)$$

- the zero-value model (the power consumption for the data value 0 is assumed to be lower than for all other data values):

$$h_{i,j} = \begin{cases} 0 & \text{if } v_{i,j} = 0 \\ 1 & \text{if } v_{i,j} \neq 0. \end{cases} \quad (3.7)$$

A suitable approximation of the information leakage of many circuits, e.g., data buses, is provided by the HW of the intermediate value. Since only the data processed in one time instant has to be predicted, no precise knowledge about the implementation is required. The HW model is often the most appropriate for predicting the power consumption of software implementations running on microcontrollers.

The switching activity of an electronic circuit is often better modeled by the HD, e.g., between the former value stored by a register and its current value. A typical register toggles internally (and thereby consumes noticeable amounts of power) only if the previous value being held is different from the new one to be stored, i.e., when their HD is one. Consequentially, the model requires to predict two intermediate values, i.e., those stored in a register Y before (clock cycle $c - 1$) and after (clock cycle c) the targeted operation. The HD is then computed from the HW according to $\text{HD}(Y^{(c)}, Y^{(c-1)}) = \text{HW}(Y^{(c)} \oplus Y^{(c-1)})$. The HD model thus presumes additional knowledge (or assumptions) about the implementation, compared to the HW model. The HD model is the most natural power model for analyzing hardware implementations.

⁸Any other bit instead of the LSB can be hypothesized likewise.

Evaluation: Comparing Hypothesized and Measured Power Consumption The final step is to compare the measured power traces $\mathbf{T}$ with their hypothesized values $\mathbf{H}$, in order to derive the correct key. In the basic **DPA** [KJJ99] the authors hypothesize only one key bit at a time and partition the measured power traces according to the two key guesses (one and zero). Computing the *difference of means* for each point in time of the two corresponding measurement sets reveals whether the key guess was correct (noticeable difference of means at certain points in time) or wrong (no obvious difference).

In this thesis, most **SCA** attacks are performed using **CPA** hypothesizing several bits n at the same time. The respective $K = 2^n$ key guesses are evaluated by computing a correlation coefficient involving $\mathbf{T}$ and $\mathbf{H}$. Note that, when hypothesizing only one key bit at a time ($n = 1$), the correlation coefficient becomes equivalent to calculating the difference of means, i.e., the **CPA** becomes a (normalized) **DPA**.

In an actual **CPA** attack, the hypothesized power consumption $h_{i,j}$ of the targeted intermediate result of the cipher is computed for each key candidate k_j^* ($1 \leq j \leq K$) and the given input data d_i ($1 \leq i \leq D$). The measured power traces $t_{i,l}$ ($1 \leq l \leq T$) for all executions of the cipher i will match best with the hypothesized power consumption of the correct key candidate (at the point in time where the intermediate result is processed). Calculating the correlation coefficient establishes the missing link between the hypothesized power consumption and the actually measured power traces for each key candidate k_j^* , and allows to assess their resemblance.

The *Pearson correlation coefficient* $r_{j,l}$ between the predicted and the recorded values for a fixed key k_j^* and one point in time l is estimated by the sample correlation coefficient

$$r_{j,l} = \frac{\sum_{i=1}^D (h_{i,j} - \bar{h}_j) \cdot (t_{i,l} - \bar{t}_l)}{\sqrt{\sum_{i=1}^D (h_{i,j} - \bar{h}_j)^2 \cdot \sum_{i=1}^D (t_{i,l} - \bar{t}_l)^2}} \quad (3.8)$$

$$= \frac{\sum_{i=1}^D h_{i,j} \cdot t_{i,l} - D \cdot \bar{h}_j \cdot \bar{t}_l}{(D-1) \cdot \sqrt{S_h \cdot S_t}} \quad j = 1, \dots, K, \quad l = 1, \dots, T. \quad (3.9)$$

Here, $\bar{h}_j$ and $\bar{t}_l$ denote the mean values of the columns h_j in $\mathbf{H}$ and t_l in $\mathbf{T}$. S_h and S_t are the sample variances of the hypothesized power values and the measured traces, respectively. The expected values for $r_{j,l}$ are in the interval $-1 \leq r_{j,l} \leq 1$. The resulting *matrix of correlation coefficients* $\mathbf{R}$ indicates whether a linear relationship exists between the hypothetical power consumption $\mathbf{H}$ and the measured power traces $\mathbf{T}$. A high absolute correlation value $r_{j,l}$ corresponds to a great similarity.

Every row in matrix $\mathbf{R}$ contains to the values of the correlation coefficient over time for a specific key hypothesis. If the key hypothesis is wrong, the hypothetical power consumption and the power trace should be uncorrelated: the absolute values of $r_{j,l}$ lie within a threshold that depends on the number of processed traces and the **SNR**. If the key guess is correct, one or more distinct peaks occur in the correlation curve. These peaks indicate not only the correct key guess, but also pinpoint the time instants where it is processed. This property of the technique is extremely important in the context of analyzing an unknown implementation (see Sect. 3.3).

Eventually, plotting $\mathbf{R}$ for all l yields a curve indicating the correlation over time that features significant peaks, if k_j^* is the correct key guess, and has a random distribution otherwise. Thus, by iterating over all k_j^* and analyzing the resulting $r_{j,1} \dots r_{j,T}$, the cryptographic secret can be

revealed, given that enough traces have been acquired and that the hypothesized link between the side-channel leakage and the processed data exists.

Comparing CPA with Exhaustive Key Search Contrary to a brute-force attack on the whole key space of the cipher, a divide-and-conquer approach becomes possible: the number of hypothesized key candidates K is chosen small enough such that verifying the correct key guess at hand of the power consumption is computationally feasible. After deriving the correct key candidate from $\mathbf{k}^*$, the attack is repeated (re-using the same traces) for the subsequent part of the key, and so on, until the whole secret key is recovered. For example, revealing a 128-bit key by hypothesizing 8 bit at a time results in a computational complexity of $128/8 = 16$ times guessing $2^8 = 256$ candidates, i.e., $16 \cdot 256 = 4096$ keys need to be processed for a full key recovery. In contrast, for a brute-force attack $2^{128} \approx 3.4 \cdot 10^{38}$ keys need to be checked.

A straightforward implementation of a CPA requires storing all traces in the memory of the PC. Hence, in case of processing a large amount of traces, e.g., when attacking modern ASIC implementations [MBKP11], or if the traces are very long, the attack becomes extremely slow (or even infeasible) due to the limited memory of the PC. An efficient version of implementing a CPA with low memory demands for a large amount of data, e.g., more than one million traces, is developed in Sect. 7.3.1.

Advanced Side-Channel Attacks For completeness, advanced SCA attacks are summarized in the following. A second-order CPA allows attacking protected implementations in which the processed values are masked with some random value (see Sect. 3.2.3). It considers multiple points within the same power trace t simultaneously. These points belong to two different intermediate values, e.g., a mask value m and the hidden intermediate value $d \oplus m$. The knowledge of either m or $d \oplus m$ alone is not sufficient to verify a key hypothesis. But if the attacker correlates on a combination of both values, she can gain some information on d . Higher-order CPA attacks targeting two or more values involve a large computational overhead during the evaluation.

Detailed instructions on how to perform a CPA can be found in [MOP07]. An in-depth discussion of higher-order CPA is given in [JPS05], and [LRP07] introduces sophisticated statistical methods for higher-order attacks. Recently, Mutual Information Analysis (MIA) described in detail in [BGP⁺11], became known as a highly effective technique, especially for attacking hardware implementations. New microarchitectural side-channel attacks such as cache and branch predication attacks [AKS07, Ber05a, OST06, NS06] evolved that take the architecture of particular processors or platforms into account. Collision SCA attacks exploit, that identical intermediate values of a cryptographic implementation exhibit very similar power patterns [BBKK07]. Detecting internal collisions, i.e., the same intermediate values occurring for different inputs, enables efficient strategies for recovering secret keys [SWP03].

If the attacker has access to a training device in which she can modify secret parameters, e.g., random masks for a protected implementation or secret keys, she can create profiles of the device and perform template attacks [GLRP06, APSQ06, CRR02]. Template attacks are considered as the most powerful type of side-channel attack, since one measurement during the attack phase suffices to extract a secret key. On the downside, the templates first need to be constructed for each individual target, implying a highly demanding and time-consuming profiling phase. Advanced side-channel attacks are not further considered in this thesis, since

one of its goals is to keep the required effort and skills as low as possible. Recovering the secret keys of all analyzed targets was feasible just by applying the standard CPA methods.

Electro-Magnetic Analysis

For the above discussed power analysis attacks, the side-channel leakage in terms of the electrical current consumption of the device — while executing a cryptographic operation — is measured via a resistor inserted into the ground path of the target IC. The whole circuitry of wireless tokens including the antenna is usually packaged, e.g., in plastic cards, and neither a contact interface nor the chip itself is accessible to an attacker in a non-invasive attack scenario. Hence, it is difficult to perform a direct on-chip measurement of the power consumption. Invasive attacks on commercial RFID tokens, i.e., dissolving the chip from its plastic package and separating it from the antenna, were not successful [CLP05].

For analyzing wireless (and other) devices in a non-invasive manner, a more promising source of information leakage are fluctuations of the emanated EM field. The military is aware of corresponding EM side-channel attacks at least since the second world war [Natb]. The corresponding side-channel information is acquired by means of near-field probes (see Sect. 4.3.2) or far-field antennae that are positioned in the vicinity of the chip. EM attacks thus typically require no physical contact to the device, i.e., they can be performed from a distance and leave no traces. The mathematical methods for evaluating EM emanations are, except for the signal processing, identical to an SCA based on measuring the electric current, as detailed above.

While invasive techniques are in general costly and traces of an attack can be discovered afterwards, the non-invasive approach of analyzing the small information-carrying variations of the EM emanations, e.g., by means of a Differential Electro-Magnetic Analysis (DEMA), is an attractive, cost-efficient option for analyzing various embedded devices. Previous results [AARR03, Han04, Plo08] suggest that this approach is suitable for a wide range of wireless tokens. Especially in the context of RFID devices, analyses based on measuring the EM field are convenient [KOP09]. A customized measurement setup for the analysis of contactless smartcards and details about corresponding filtering techniques are illustrated in Sect. 7.3.3.

3.2.3 Countermeasures against Side-Channel Analysis

Hiding and *Masking* are amongst the most common countermeasures for securing both hardware and software implementations of cryptographic schemes. *Hiding* methods aim at increasing the noise factor [YWV⁺05] or at equalizing the power consumption values [TAV02] independently of the processed data, thereby decreasing the SNR. *Masking* countermeasures rely on randomizing all key-dependent intermediate values processed during the execution of the cipher and can be employed on either algorithmic level [OMPR05, OS05a] or cell level [PM05]. Masking is often combined with the hiding techniques *shuffling* [HOM06b], i.e., randomizing the order of the operations, and inserting *dummy rounds* that appear like genuine rounds of the cipher but process no relevant information.

However, each countermeasure comes along with some costly disadvantages and implementation constraints, e.g., increased power consumption, enlarged chip-area, and increased execution time. As a consequence, countermeasures are not yet widely implemented in wireless tokens — especially in large-scale applications the increased cost is crucial and in the case of RFIDs the required energy may not be available. Despite the substantially raised effort on the side of the

adversary, none of the so far proposed countermeasures perfectly counteracts a key-recovery by means of a (higher-order) **DPA** in the real world, and improved attacks can defeat even sophisticated countermeasures [CCD00a, MPO05, WW04]. A sufficient security level can often be established by additionally supporting the implemented countermeasures on the system level, e.g., by restricting the amounts of encryptions per time window: assume, that one million traces are required for a successful key recovery, then limiting the amounts of requests to one in ten seconds would dramatically complicate the attack. A comprehensive survey about countermeasures against power analysis and respective advanced attacks is given in the context of the **AES** in Appendix B.

3.2.4 Processing Measurements

Building a good measurement setup and adequately preprocessing the traces before their further analysis should not be underrated in side-channel attacks. From an attacker's viewpoint it is important to avoid noise as much as possible. Also, if multiple traces of the same operation are recorded, they have to be aligned exactly in time. The most important preprocessing techniques are sketched in the following, while the details of applying them to particular devices can be found in the respective later chapters of this thesis.

Alignment and Filtering

When analyzing a self-made implementation in the laboratory, an exact trigger signal for starting the oscilloscope is usually provided by the specially prepared target device. For unknown implementations, it is often much more difficult to find an appropriate trigger point, i.e., a special feature in the traces that reoccurs at the same instant in each measurement. Accordingly, the alignment of the measurements typically needs to be improved as a first preprocessing step after the acquisition, e.g., by applying a least-squares method to a reference trace and each measured trace. Filtering the traces, e.g., with a low-pass filter, can reveal special features in the traces for the alignment and improve the **SNR**, i.e., increase the amount of information leakage compared to the noise in the traces. The filter aims at separating the side-channel leakage from the unwanted signals, in order to extract only the relevant information. Digital filters are of crucial importance when dealing with large digital components [BPT10].

Peak Extraction

Power traces can become very large or too noisy for an effective evaluation – thus they may need to be compressed prior to the statistical analysis. The dynamic power consumption is the dominant factor for disclosing the processed data of Complementary Metal Oxide Semiconductor (**CMOS**) circuits. In each clock cycle, the internal capacitors of the electronic circuitry need to be charged and discharged according to their logical state. The corresponding peaks appearing in the measurements on each edge of the clock play a prominent role, since they often contain most of the relevant information. Processing only the amplitudes of these peaks — instead of all acquired data points — enables a great reduction of computations and memory required during the analysis. Moreover, misalignments arising from a clock jitter due to an unstable oscillator in the cryptographic device are eliminated by peak extraction.

Averaging

In case of a questionable quality of the acquired power consumption, e.g., due to a noisy environment, bad measurement setup, or cheap equipment, often *averaging* can be applied: The same operation with identical input data is performed repeatedly, e.g., the same ciphertext is decrypted repeatedly, and the mean of the corresponding traces is computed. This method reduces the noise floor and can enormously increase the SNR of the power traces, if no countermeasures are present. A decreasing amplitude towards the end of the averaged power traces indicates a misalignment, e.g., caused by clock jitter. As exactly the same input data is processed, the measurements (or parts of the measurements) can be accurately aligned by performing a cross-correlation between two traces.

Often, some patterns and special features can be spotted in the averaged measurements, which are not visible from inspecting only one power trace, e.g., the ten rounds of an AES-128 are typically cognizable after the averaging. Likewise, computing the variance of the traces can reveal further interesting characteristics of the measurements. Comparing (averaged) traces for different ciphertexts can help to find the time-window in the traces where a data-dependent behavior occurs and hence the decryption takes place. This technique, further detailed in Sect. 3.3.2, is particularly helpful when analyzing a black-box implementation.

3.3 Steps of a Black-Box Analysis

In practice, a side-channel attack targeting a known or self-made implementation (“white-box”) extremely differs from attacking an unknown, commercial device (“black-box”). Analyzing a device in a black-box scenario typically takes a lot of effort, even for an implementation featuring no countermeasures against side-channel attacks. The DUT first has to be integrated into the measurement setup, i.e., a suitable way of accessing the input (or output) data and acquiring the side-channel leakage needs to be established. If applicable, removing filtering capacitors that are positioned close to the supply pins of the DUT on the Printed Circuit Board (PCB) might improve the measurements.

Finding an adequate power model and locating the time window in which the targeted cryptographic operation takes place is often the most time-consuming step of a practical attack — depending on the device this can take months of trial-and-error. Often in the beginning millions of very long traces are required to pinpoint some information leakage. With gaining more knowledge about the DUT, improved analysis techniques can be developed that require only a small amount of short traces for a successful key recovery. The attacker has to put herself into the position of the designer of the DUT, make assumptions about the internal architecture of the analyzed device, and try to verify them by means of physical observations. Piece by piece, more details of the black box become known, until finally a key-recovery may succeed. Blank training devices containing known or modifiable cryptographic keys can be helpful for the profiling. If older versions of the same or a similar device are available, e.g., with an outdated, larger semiconductor technology, these should be preferred for the first analyses, since they are often easier to attack. With the know-how gained from analyzing an outdated target the next step towards the sophisticated version of the device is often small. The exact procedure of analyzing a black-box strongly depends on the DUT and other parameters, but often comprises the steps as described in the following sections.

3.3.1 Collecting Available Information

All available written information about the target device, e.g., maximal data- and clock rate, maximal supply voltage, input formatting, protocols, which cipher is used, etc., needs to be found from data sheets and other sources. If the protocol is not available it can often be reverse-engineered by monitoring the communication with corresponding base stations or readers. Note that for extracting secret keys it is often sufficient to implement only some part of the protocol, until the **DUT** performs the targeted cryptographic operation, e.g., encryption, which happens often at the beginning of the communication.

By sending different inputs to a blank device with a known key and comparing its outputs with a model of the **DUT** on a **PC** further knowledge is collected: an implementation of the targeted cryptographic algorithm on the **PC** serves to verify which cipher is employed and which operating mode is used, e.g., **ECB** or **CBC** for block ciphers. Furthermore, the exact order in which the bits and bytes of the input are processed by the **DUT** is checked, and if additional checksums or parity bits need to be considered for the communication. Further tests can be carried out to pinpoint how the device reacts if unexpected data is sent, for example, if chosen input data sent to the device is processed unconditionally, or where the authentication aborts.

3.3.2 Profiling by Means of Power Measurements

For the first profiling steps, no assumptions about the cryptographic algorithm being executed on the target platform are required. From plotting and visually inspecting some side-channel measurements, various conclusions can be drawn (depending on the **DUT**) about the actual implementation, e.g., the internal clock frequency can often be derived from the distance between consecutive peaks in the trace. An experienced evaluator can identify the type of a particular device at hand of its power traces, e.g., distinguish between an **ASIC** implementation, an Atmel AVR Reduced Instruction Set Computer (**RISC**) core requiring one clock cycle per instruction, a Microchip PIC Microcontroller (**μC**) that needs four cycles per instruction, or an 8051-compatible chip which requires twelve clock cycles per instruction.

A suitable feature in the power traces that is related to the targeted cryptographic operation needs to be identified in order to align and then statistically evaluate the traces. Computing the *mean* of many traces (averaging, see Sect. 3.2.4) can help to visualize certain characteristics of the implementation that are identical or similar in all measurements, e.g., rounds of a block cipher can often be distinguished. A decreasing amplitude of the mean trace with the time indicates a misalignment that occurs during the computations, e.g., due to an instable internal oscillator.

On the contrary, the *variance* of traces with different input data can reveal the points in time of data-dependent computations, while those parts of the traces in which fixed operations and data are processed feature a low variance. Another option for a simple statistic test, that requires no previous knowledge or assumptions about the implementation, is a difference-of-means test described next.

Difference-of-Means Test

To determine the quality of the traces acquired with a particular measurement setup and to find out if and in which instants the traces contain data-dependent information, a Difference-

of-Means (DOM) test can be conducted as a preparation for the key-recovery. The idea is to alternately send two chosen challenges⁹ that cause a different power consumption or EM emanation according to the assumed power model. On the basis of the corresponding traces $\vec{t}_1$ and $\vec{t}_2$ of length T , four equally sized sets S_i are created, with $|S_i| = D$ denoting the number of traces per set.

Let $S_1 = \{\vec{t}_{1,1}, \dots, \vec{t}_{1,D}\}$ contain the traces of the first challenge and $S_2 = \{\vec{t}_{2,1}, \dots, \vec{t}_{2,D}\}$ those of the second challenge. The average traces $\bar{t}_1(l)$, $\bar{t}_2(l)$ and their DOM timeseries $\delta_{(1-2)}(l)$, where $1 \leq l \leq T$ denotes the current sampling point, are given as:

$$\begin{aligned}\bar{t}_1(l) &= \frac{1}{D} \sum_{d=1}^D \vec{t}_{1,d}(l) \\ \bar{t}_2(l) &= \frac{1}{D} \sum_{d=1}^D \vec{t}_{2,d}(l) \\ \delta_{(1-2)}(l) &= \bar{t}_1(l) - \bar{t}_2(l)\end{aligned}$$

The maxima of $\delta_{(1-2)}$ correspond to the points in time where a data-dependent behavior occurs: the higher the amplitude of the peaks, the more information is contained in the measurements, hence for more traces a higher SNR is expected.

Two further sets S_3 and S_4 containing randomly assigned traces that belong to *either* challenge are created, i.e., all traces are distributed uniformly to the two sets. Accordingly, $\bar{t}_3(l)$, $\bar{t}_4(l)$ and their DOM timeseries $\delta_{(3-4)}(l)$ are computed. They illustrate, whether a uniform distribution of the power consumption can be assumed for the available measurements: The DOM $\delta_{(3-4)}(l)$ would ideally vanish for a large amount of traces. Its amplitude indicates, if sufficient traces are assigned to the sets — for big absolute values of $\delta_{(3-4)}(l)$, the amount of traces needs to be increased. The DOM test requires no previous assumptions about the DUT or implementation, e.g., no power model. It is thus a helpful tool to determine during the initial analysis of a target device, if the measurements capture any information leakage at all, for example when positioning near-field probes for capturing EM emanations (see Sect. 7.4.3).

3.3.3 Power Analysis with a Known Key

When analyzing a black-box implementation, one assumption always holds: the input data of the algorithm (for example the plaintext in case of an encryption) must be processed internally at some time instant. Likewise, the data that is output by the DUT must have been computed before. Hence, correlating on this known data often allows to precisely locate the targeted cryptographic operation in a power trace.

The attacker triggers the oscilloscope, e.g., with the challenge being sent to the DUT, and acquires long traces that with a high probability contain the targeted computations. When correlating on the known data in the following, it is convenient to target only one bit at a time, i.e., follow the difference-of-means approach, since it requires the least assumptions about the implementation. By targeting all bits of the known data one after another, the time instant of each bit being processed is revealed.

⁹The principle could be extended for more challenges

However, even when hypothesizing only one bit at a time, the power model has to be chosen adequately. For example, assuming an ideal **HW** leakage will theoretically not be successful for a device that leaks power according to a **HD** model¹⁰. Furthermore, a power model that enables correlating on the input- and output data may not be appropriate for other intermediate values of the cipher, e.g., a microcontroller supplying the input data to a dedicated cryptographic hardware processing the intermediate values. An example for this scenario are Mifare DESfire cards, as illustrated in Chap. 7.

As a result, conclusions about the internal architecture of the **DUT** can be drawn: correlating on the known input and output values reveals the exact location where the cryptographic algorithm is computed; the duration of the algorithm allows to estimate whether a hardware or a software implementation is present; and correlations for different bits that occur at the same time instant reveal the size of one data word being processed. Knowing the secret key during the profiling, intermediate values of the cipher can be hypothesized and verified by means of their power traces. This allows to optimize the power model for the later analysis of the cipher and to gain further insights into the internal processing, e.g., the order in which operations are carried out.

Finally, after the profiling, the attacker has gained first insights into the black box — the **DUT** can be considered as a “grey box”: She has found an appropriate power model, knows the exact time window in which the targeted computations take place, and can hence acquire shorter measurements. Her preparations for analyzing the side-channel leakage of a device with an unknown key, by means of the methods introduced in Sect. 3.2, are complete.

¹⁰In practice, the real leakage of a device differs from the model, hence attacks are still feasible — with a larger amount of traces — even when using a wrong power model.

Chapter 4

Tools for Security Analysis

THE setup for implementation attacks targeting wireless devices, as detailed in this chapter, is designed on a flexible modular basis so that it can be adapted to test the physical security of basically any cryptographic device. The different functional units are categorized in modules, e.g., for the communication with the DUT and modules for side-channel analysis. These modules may in turn consist of a set of smaller sub-modules that enable arbitrary extensions of the framework according to the requirements of the evaluator. As a DUT, any microcontroller, FPGA, ASIC, or other embedded system is taken into account, with a focus on wireless devices and RFIDs. In case of specialties, e.g., concerning communication via RF, it is straightforward to incorporate additional modules to the ones described in this thesis.

For carrying out side-channel or fault-injection attacks, an adversary must usually have access to the plaintext or ciphertext processed by the DUT, in order to evaluate measurements or determine whether a penetration test was successful. Though it is possible to carry out some cryptanalytical and physical attacks using commercially available equipment, customized hardware for communicating with the DUT is highly advantageous. Commercial readers for (contactless) smartcards or RFIDs often rely on proprietary ICs that carry out certain tasks autonomously. For instance, a built-in RNG will generate the nonces for a challenge-response protocol; correct parity bits and checksums are computed automatically; data will be encrypted, encoded and sent; and this all happens without that the process can be directly influenced.

However, an attacker often has an interest to intentionally send wrong data and thus requires readers or base stations that are specially prepared for executing physical attacks and other security analyses. The tools should ideally enable full control over the communication, e.g., sending arbitrary bits or repeatedly the same chosen plaintext; intentionally computing wrong checksums; and — most important in the context of side-channel attacks — provide precise control of the timing, e.g., to generate reliable trigger signals. For some practical attacks, additionally a device that can serve as a replacement for the original DUT is required, e.g., an emulated smartcard or wireless token that can be fully controlled by the evaluator. For each form of communication covered in this thesis, such an emulation extension is developed, that is compatible with the corresponding reader or receiver. The emulators further allow to monitor the communication data, e.g., for reverse-engineering unknown protocols. The customized embedded system for *communicating with contactless cards* at a frequency of 13.56 MHz, presented in Sect. 4.1, has become a vital part of our lab setup. Further hardware for communicating with various other wireless tokens is introduced in Sect. 4.2.

For *side-channel attacks*, the behavior of the DUT with respect to timing, power consumption, EM emanation, etc. has to be accurately monitored. The corresponding *data acquisition module* described in Sect. 4.3 serves for this purpose, while the recorded data is often post-processed and evaluated by a controlling PC. Special modules for the analogue preprocessing of the

EM information leakage emanated by **RFID**s are detailed in Sect. 7.3.3. Further extensions of the measurement setup with respect to fault injection and analyzing contact-based cards are described in Sect. 4.4.

Brute-force attacks demand for immense computational power, hence a dedicated hardware platform for cryptanalysis can help to verify their practicability and real cost. The **FPGA**-based cryptographic hardware **COPACOBANA** introduced in Sect. 4.5 serves in this thesis as the target platform for implementations of exhaustive key-search targeting the secret keys of electronic passports (see Chap. 9) and a brute-force attack on the KEELOQ cipher (see Sect. 8.5.5).

4.1 Communicating with Contactless Smartcards

The hardware for security-analyzing contactless smartcards introduced in Sect. 4.1.1 and Sect. 4.1.2 has been developed in my Diploma thesis [Kas06] in 2006, back then with the purpose to carry out practical relay attacks. The details of the developed tools have been published in [KCP07]. Meanwhile, as a part of this Patiently hoping for a Degree (**PhD**) thesis, both hardware and software of this embedded system have been largely extended to provide a variety of new operation modes, as detailed in Sect. 4.1.3. A new stand-alone emulator for contactless smartcards is developed in Chap. 5. The devices serve as an essential basis for the real-world attacks on a contactless payment system detailed in Chap. 6, the key-recovery attack on electronic passports in Chap. 9 and the side-channel analysis of **RFID** devices, as exemplified at hand of the Mifare DESfire card in Chap. 7.

The embedded system for analyzing contactless smartcards consists of a multi-purpose reader device which is equipped with a μC , an **RF** interface and some components for signal processing. A second device, termed *Fake Tag*, is designed to appear like an authentic card to an **RFID** reader, e.g., in the context of **MITM** attacks, and furthermore can acquire the information contained in the field. The two units cooperate by means of a communication link while the μC on the reader provides the physical communication layer for both devices.

As depicted in Fig. 4.1, the **RFID** tool is integrated in a measurement setup controlled by a **PC**. The latter starts the communication, triggers the measurements, and later combines and further processes the data acquired from scope and reader. The developed cost-efficient embedded system eases the security analysis of (cryptographically enabled) smartcards with an ISO 14443A compliant **RF** interface. Extensive control of the communication and the energy supply is provided, as well as interoperability with other hardware and measurement equipment. In addition, stand-alone operation for performing practical attacks and mobile data acquisition in the field is supported.

The most relevant tasks that can be carried out with the system are

- communication on the bit layer with a low-level reader,
- emulation of an ISO 14443 compliant tag,
- perform practical **MITM** (relay) attacks,
- assist side-channel analyses (trigger signal, ...),
- manipulate the power supply of the **EM** field to inject faults,

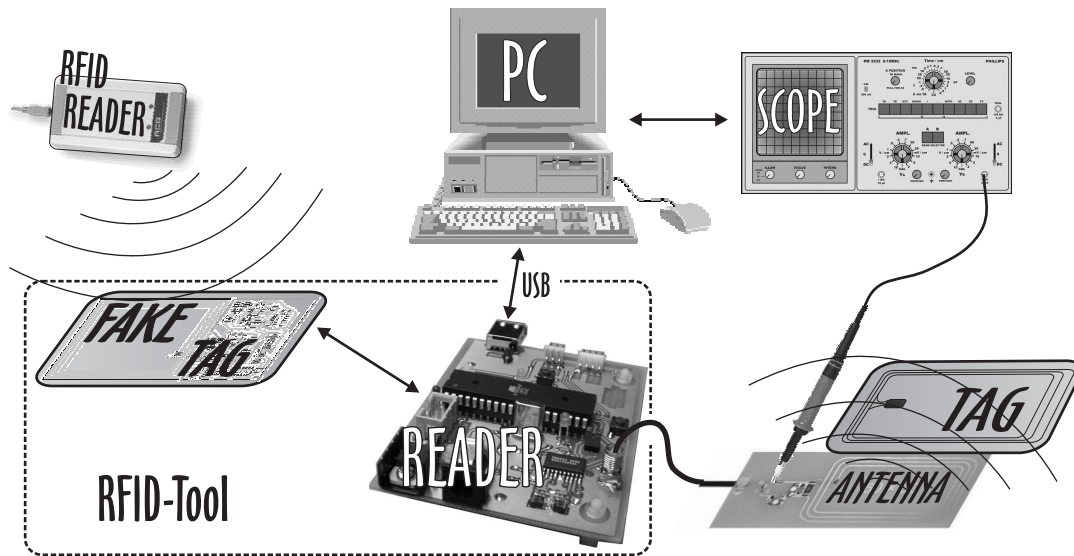


Figure 4.1: System Overview: The developed ISO-14443 reader cooperates with the card emulator “Fake Tag” such that the communication between a genuine RFID reader and a genuine card can be relayed. A PC can communicate with a genuine RFID reader via the Fake Tag, or access an RFID tag or a contactless smartcard via the reader of the RFID tool. An oscilloscope is used to measure side-channel leakage or to record communication data.

- logging of the interchanged data, and
- testing of different types of antennas and power amplifiers.

The following subsections illustrate the relevant technical details of the hardware, as fully described in [Kas06, KCP07].

4.1.1 Reader

The operation principle of the low level reader, that can be built for less than 40€ using off-the-shelf equipment, is depicted in Fig. 4.2. The device has been made available to the public as a General Public License (GPL) project [KCP07]. Its core, an ATmega32 μ C [Atma], is clocked at a frequency of 13.56 MHz and features, amongst others, 32 kB Flash Random Access Memory (RAM), 2 kB internal Static Random Access Memory (SRAM), 1 kB non-volatile EEPROM and an Analog to Digital Converter (ADC). For applications with high timing requirements two 8-bit timers and one 16-bit timer are provided that can be operated in different modes. For flexible operation and testing, the software running on the μ C can be easily updated from a PC using the In-System Programming (ISP) interface, without the need to remove the μ C from the PCB.

Analogue Front-End The main part of the analogue front-end is provided by the EM 4094 RF-transceiver [EM 03] which is capable of various modulation schemes, such as 100% ASK required for ISO 14443 A compliant operation, and costs less than 5€. A high input level on its DIN pin switches off the field instantaneously, while a low level switches it on. Thus,

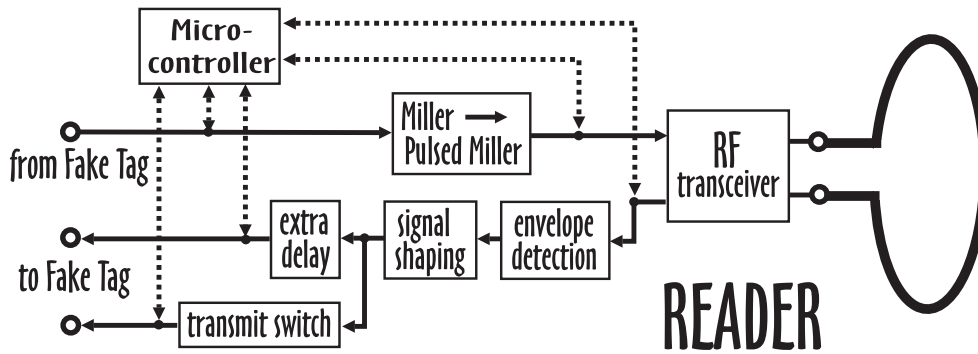


Figure 4.2: Operation Principle of the RFID-Reader

in contrast to commercially available products, the **RFID** reader allows to directly control the **RF** field with a high timing accuracy of approx. 75 ns, corresponding to the duration of one clock cycle of the μC . This function was extensively used for our attacks on Mifare Classic as described in Chap. 6. The received **HF**-Signal can be conditioned by internal filters and adjustable receiver gain. The transceiver is suitable for communication according to both Type A and Type B of ISO 14443 at all data rates. Various antennae and power amplifiers can be connected to the reader via an output socket that is matched to $50\,\Omega$, in order to extend the eavesdropping and activation range out of the bounds of the specification.

To disburden the μC , the generation of pauses in the field required for transmitting Modified Miller encoded data is carried out by a 74123 [Phid] monoflop that creates a pulse on every rising edge on its input, driven by the μC . The output pulses are applied to the DIN input pin of the EM 4094 **RF**-transceiver, resulting in the field being switched off for a pulse duration ($\approx 2.5\,\mu\text{s}$). Alternatively, the pulses can be created by two additional monoflops which are capable of converting Miller-encoded data to Modified Miller code without the μC , as required, e.g., during an **MITM** attack.

The Manchester-coded data modulated with the subcarrier (cf. Sect. 2.5.3) output by the EM 4094 is demodulated using an envelope detector circuit. The signal is rectified by a diode and then fed into a low-pass filter. The presence of the subcarrier is detected by an LM 311 [Nate] voltage comparator, which outputs Manchester encoded data with the appropriate 0 V and 5 V levels so that the information can be further processed by the μC .

As the demodulation of the signal received from the **RF** transceiver costs some time (in this case $\approx 1.5\,\mu\text{s}$) it can happen that the answer of the tag is not well synchronized with the reader when relaying data in the context of a **MITM** attack. For compensation, a circuit has been developed that adds a short, adjustable time delay to the forwarded signal, without altering its waveform. The delaying enables to re-synchronize the response of the card with the bit grid as specified in ISO 14443.

USB Communication Fast communication with a **PC** or other Universal Serial Bus (**USB**)-equipped hardware is handled by an FT 245R [FTD] parallel-to-USB chip. The pins of the chip are connected to Port C (data) and Port A (control signals) to receive or send data in packets of eight bits, by pulling a read or write input pin at Port A high. Two more pins of Port A are required for controlling the data flow. Port C of the μC is used for transferring data byte-wise

from and to the **USB** chip. Using the supplied Virtual COM Port (**VCP**) driver, the **USB** port can be accessed as a standard serial COM port from a **PC**, thereby achieving a maximum possible data rate of 1 MB/s.

Communication with the Fake Tag An interface for serial communication between the developed reader and the Fake Tag is also installed on the **PCB**. The data pins can be driven directly by the peripheral circuitry of the **RFID** tool or steered by the μC , which allows for the emulation of a tag as well as for μC -based processing of the interchanged signals. In order to reverse engineer unknown protocols, the data exchanged between the two entities can be stored in the **EEPROM** on the reader or directly sent to a **PC** via **USB**.

User Interface Besides the reset button, an additional user-definable button integrated on the **PCB** can be used for interaction with the software of the μC . It is particularly useful in case of a stand-alone operation of the **RFID** tool, where no input to the μC via **USB** is possible. Furthermore, two Light-Emitting Diodes (**LEDs**) — green and red — are provided as indicators, e.g., for the status of the reader or for debugging purposes.

Practical Tests with the Reader The low-level reader has been successfully tested with several ISO 14443 compliant tags, by implementing and executing the initialization and anticollision part of the protocol. The exact behavior and timing of the contactless interface can be flexibly steered, even transcending the ISO standard, if desired. Every bit sent via the **RF** interface is fully controlled by the attacker. Various antennae were built to perform tests with regard to the operating range and the influence of the physical environment of the card. Trigger signals for starting measurements can be issued at any desired point during the communication, and the energy supply of the contactless cards, i.e., the **EM** field, can be controlled. The stand-alone operation mode of the tool is a crucial advantage for practical security analyses in the field, as demonstrated in Chap. 6.

4.1.2 Fake Tag

A device termed “Fake Tag” has been developed to work with the reader. The Fake Tag can be utilized in the context of **MITM** attacks for emulating of a contactless smartcard. Unlike a normal (passive) **RFID** tag, the device described here possesses its own power source¹ which may also be used for supplying an optional wireless **RF** module for communicating with the reader.

The functional principle of the Fake Tag is depicted in Fig. 4.3. A coil that is part of the **PCB** establishes the coupling with its counterpart at the antenna of an **RFID** reader (see Sect. 2.4). A capacitor is connected in parallel to this inductance, to form a **parallel resonant circuit**. For an ideal parallel resonant circuit, $f_c = 1/2\pi\sqrt{LC}$ applies [TS01], where f_c denotes the carrier frequency of the reader, C the capacitance and L the inductance of the tuned circuit. To determine the appropriate values for the components in practice, first the approximate value for L is derived from the shape and dimensions of the coil. Afterwards, the optimal C is calculated and then realized as a variable capacitor, so that the circuit can be tuned more precisely later on. As detailed in my Diploma thesis [Kas06], the quality factor of the circuit

¹A small lithium battery is sufficient for the power supply.

and its bandwidth can be tuned with an additional parallel resistor. The induced voltage can become relatively large, thus two antiparallel Zener-diodes limit the maximum possible voltage to 4.7 V and thus protect the remainder of the circuit.

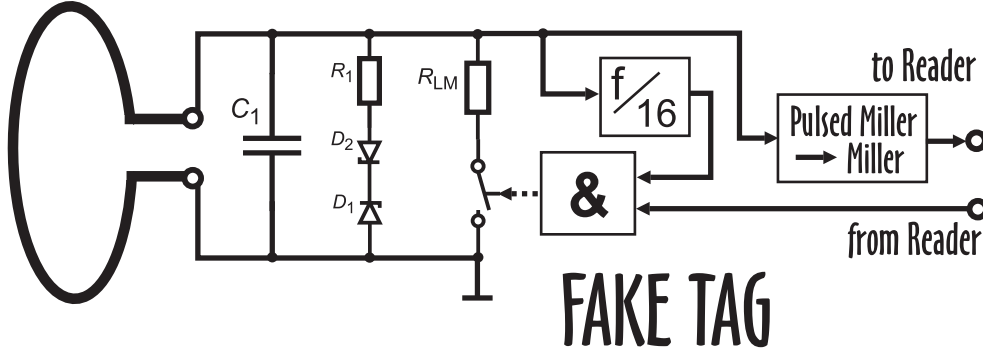


Figure 4.3: Operation Principle of the Fake-Tag

The *subcarrier* required for the load modulation of the **RF** field is derived directly from the field received at the coil by means of a 4-bit binary counter 74393 [Phia]. The fourth output of the counter toggles every $2^3 = 8$ clock cycles, thus delivering the desired frequency of $f_c/16 = 847.5 \text{ kHz}$. A resistor limits the maximum current into the input stage, as proposed in [Fai]. For modulating the incoming Manchester coded signal with the subcarrier, a 7408 AND gate [Phib] combines the former with the output of the binary counter. To generate the load modulation, as described in Sect. 2.5.3, a resistor in parallel to the coil of the tag has to be switched on and off. This is realized with an IRFD 110 [Intb] N-channel Metal-Oxide Semiconductor Field-Effect Transistor (**MOSFET**), enabling fast switching and a maximum drain-source voltage of 100 V. The output of the AND gate is connected to the gate of the transistor. Accordingly, by toggling the resistor, the 848 kHz-modulated Manchester code is modulated onto the 13.56 MHz field of the reader, i.e., the information is transported in the sidebands of the carrier.

To acquire data sent from a reader, an LM311 comparator combined with two envelope detectors (similar to Sect. 4.1.1) are connected in parallel to the resonant circuit. One of the detectors has a fast response time and distinguishes between the field being completely switched off and the load modulation case. The other envelope detector reacts slower and averages the signal at the antenna, for adapting the threshold voltage of the comparator to the momentary field strength. This approach immunizes the circuit to noise caused by the **RF** field and hence extends the operating range.

The output of the comparator is connected to a 7474 D-type flip-flop [Phic], whose inverted output is fed back into its input. Accordingly, a change of the output state occurs on every rising edge at the input. This conversion from pulses into transitions, i.e., converting the received pulsed Miller coded signal into a “standard” Miller coded data signal, is amongst others necessary to reduce the bandwidth required for transferring signals via the communication link of the **RFID** tool.

Practical Tests with the Fake Tag The data transmitted by the Fake Tag is accepted by various **RFID** readers, e.g., an ACG Dual 2.1 Passport Reader [id06], just as if it was a genuine

tag. During our tests, the answer **ATQA** of the Fake Tag to a **REQA** issued by the reader was intentionally delayed by more than $250\ \mu\text{s}$ and the resulting behaviour was analyzed. Compliance of the ACG reader with the strict timing requirements during the initialization phase² could not be observed, i.e., the delayed answer was still accepted, thus the reader eases relay attacks.

Basic Security Analyses with the RFID Tool

The **RFID** tool introduced in this section was built using electronic hobbyist equipment and off-the-shelf components at a cost of less than 40€, and its design is simple enough to be reproduced by a low skilled attacker. Integrated in a measurement system, the proposed tool can help to carry out security analyses, such as a **DEMA** or a remote power analysis, and assist fault injection attacks. The stand-alone operation modes permit for mobile tag emulation, reader operation and logging of the interchanged data in practical scenarios.

Relay Attack For executing a relay attack (compare with Sect. 2.2.4) with the developed tool, the adversary places the antenna of the bit level reader close to a contactless card of the victim. At the same time, the Fake Tag is brought into the field of an **RFID** reader, e.g., at the cash desk, by an accomplice. The data being transferred by this genuine reader is acquired and directly forwarded on the bit layer through a communication link to the reader of the second attacker. There, the data is retransmitted to the card of the victim. The answer of the latter is relayed back to the reader at the cashpoint and so, as the attackers continue relaying the data, both reader and tag will be convinced that they are in close vicinity to each other and thus carry out their task, e.g., authorize a payment.

Such an attack has been successfully carried out using the **RFID** tool with various contactless smartcards, e.g., an RFID-enabled passport of Germany, a student identity card (Ruhr-University Bochum, Germany) containing a Mifare Classic chip, a ticket for the FIFA world cup 2006 based on Mifare Ultralight, as well as Mifare DESfire cryptographic smartcards, until to at least reading out the **UID** of the tags. In the case of the Mifare Classic, after a successful login, encrypted data blocks were read out and modified remotely.

Monitor Communication The tool can also be used for monitoring and logging the data interchanged between a tag and a reader. This can be helpful for debugging purposes, to analyze unknown protocols, as well as for further processing of the eavesdropped data, e.g., key-search with cost effective hardware, as proposed in [KPP⁺06a]. The input data for the brute-force attack presented in Chap. 9, i.e., the communication with electronic passports, was also monitored with the **RFID** tool.

Conclusion of the Basic Security Analyses Employing ISO 14443 compliant contactless smartcards in security sensitive applications should be regarded critically, as the physical interface is proven to be insecure against relay attacks. A smartcard identified by a reader does not have to be in its direct vicinity, as declared by many manufacturers. Instead, the data can be forwarded from large distances with little effort, without permission or even notification of the owner.

²ISO 14443 requires the tag to answer to a **REQA** exactly after $86.9\ \mu\text{s}$, see Sect. 2.5.5

4.1.3 Software for the RFID tool

In general, the **RFID** reader and the Fake Tag support three main modes of operation:

- Reader mode: freely programmable ISO 14443 (and hence Mifare-compliant) reader.
- Card mode: emulation of an ISO 14443 compliant contactless smartcard, e.g., Mifare, by means of the Fake Tag.
- **MITM** mode: both reader and Fake Tag are utilized to relay the data between a genuine reader and a genuine contactless smartcard in real-time, e.g., to pretend the vicinity of a card to a reader, which in reality can be miles apart from each other.

In the course of this **PhD** thesis, various programs and software extensions have been implemented both on the **RFID** tool and the controlling **PC** to realize versatile applications for utilizing the devices in the context of various real-world attacks.

Active MITM

As one example, the active **MITM** mode allows to actively influence the data interchanged during a relay attack, i.e., the information forwarded can be modified in real-time. This is for instance useful to falsify the **UID** of an **RFID** tag, that is usually fixed in the hardware, or to enforce communication at a lower data rate than the original one.

Some smartcards support bit rates higher than 106 kBit/s, e.g., Mifare DESfire [NXP08d] from NXP provides support for bit rates of up to 848 kBit/s. The **RFID** tool can relay communication only at a bit rate 106 kBit/s, hence the command for the negotiation of the bit rate exchanged between card and reader is replaced with a fake one, forcing the bit rate of the communication to 106 kBit/s. The **ATS** command defined in ISO/IEC 14443 (see Sect. 2.5.6) contains the maximum possible data rates in both directions supported by the smartcard. In order to replace it with a fake one encoded for 106 kBit/s, the **RFID** tool decodes and monitors the incoming commands from a genuine reader. Once the **RATS** command is received, the direct relay path is deactivated and the μC controls the subsequent communication. Timer 0, ensuring the correct timing of the communication, is initialized and started. Consequently, the forged **ATS** command is forwarded to the EM 4094 **RF**-transceiver and sent to the genuine reader instead of the original **ATS** with a coded higher bit rate.

The waveform of the Manchester-encoded data in Fig. 4.4, captured by an oscilloscope, illustrates this process for a relay attack with a DESfire card. Finally, the μC gives back control of the communication to the relay circuitry — both parties continue to communicate at 106 kBit/s and the **RFID** tool continues to relay (and monitor) the communication passively.

More Functions and Operating Modes

The low-level and auxiliary functions for realizing the physical layer are described in Appendix G.1, while details of the operating system running on the μC and various operating modes are given in Appendix G.2. The authentication protocols for several contactless smartcards employing **3DES**, **AES** and some proprietary ciphers, as required for side-channel analysis and fault injections, have been fully implemented. Furthermore, an efficient full-key recovery of Mifare Classic cards is implemented (see Sect. 6.5.1). Details about the other parts of the software are given in the corresponding chapters.

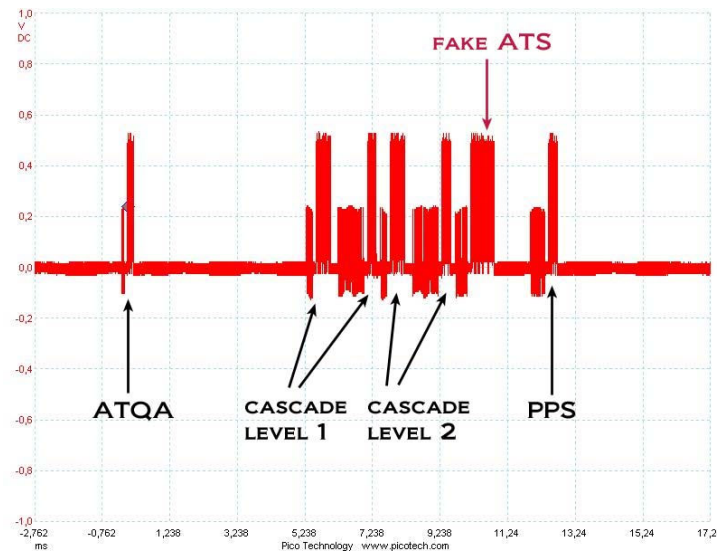


Figure 4.4: Manchester-encoded card responses in a typical card activation sequence with a faked ATS command.

4.2 Communicating with other Wireless Devices

Active wireless devices, such as remote controls, operate at various different frequencies, e.g., 27 MHz, 433 MHz, or 866 MHz. They can often be conveniently emulated by modifying the existing hardware: on a typical **PCB** the **ICs** for receiving or transmitting data at the corresponding frequency are separated from the parts performing computations. Thus, everything on the **PCB** except for the transceiving (transmit and receive) part is deactivated / desoldered and the pins of the transceiver are connected to a **PC**. The latter then prepares all data to be sent and processes all received data — the wireless token is solely used as an **RF** interface. Due to the typically slow data rates, often the parallel port is a straightforward means for both emulating a transmitter and emulating a receiver. All practical attacks (eavesdropping and emulating) on KEELOQ in Chap. 8 were performed on the basis of accordingly modified commercial receivers and remote controls.

4.2.1 USRP and GNURadio

For communicating on any other frequency in the range from Direct Current (**DC**) to more than 4 GHz, the **USRP**³ transceiver (Version 2) is integrated in the setup. Its core is a Xilinx Spartan 3-2000 **FPGA** [Xil05]. The **USRP** can be powered from a battery and various daughterboards enable receiving and transmitting in diverse frequency bands. The device is programmed and controlled via the open-source GNURadio⁴ software that realizes the modulation, demodulation and other signal processing of the signals — the **USRP** hardware solely mixes the signal down (or up) and supplies the digitized base band to the **PC** via a Gigabit Ethernet interface. Performing

³www.ettus.com

⁴www.gnuradio.org

the signal processing completely in software enables great flexibility, for instance the **USRP** can emulate a base station of mobile phones in the **GSM** band, serve as a reader for **RFID** devices, or implement a Global Positioning System (**GPS**) receiver.

4.2.2 RFID Reader for 125 kHz

A similar reader as presented in Sect. 4.1.1 has been developed for communicating with **RFID**s operating on 125 kHz, which are mainly used for car immobilizers and access control. Compared to contactless smartcards, these devices possess less computational power and typically use simple (and often insecure) proprietary ciphers, if cryptography is implemented at all. Details of the reader are given in Appendix C.

4.3 Data Acquisition

A **USB** oscilloscope, controlled by a **PC**, forms the basis of the data acquisition system. The acquired data can be side-channel information (e.g., current, voltage, **EM** emanation or timing information), or communication data such as bitstreams in any format which then later can be evaluated by a **PC**. The software framework follows the modular approach of the setup and allows for straightforward substitution of its parts, e.g., when switching to a new oscilloscope or analyzing a **DUT** that has different requirements with regard to mounting an attack.

4.3.1 Picoscope

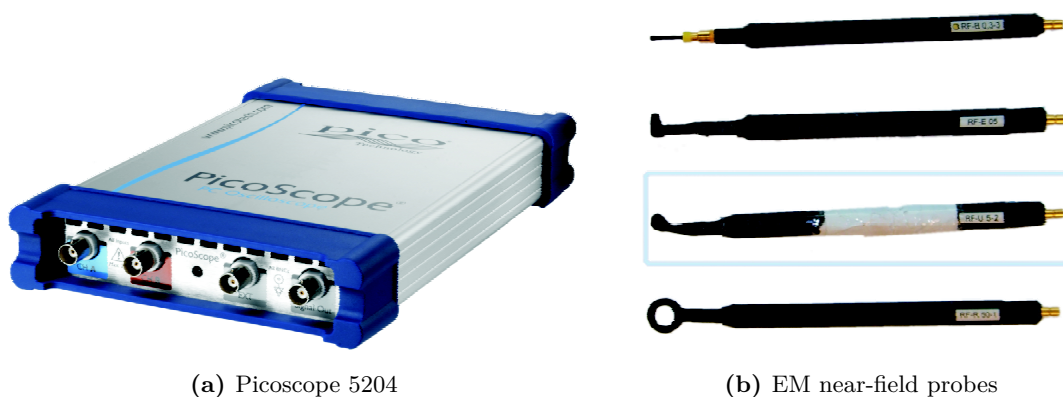


Figure 4.5: USB scope for digitizing data (left) and near-field probes for acquiring EM emanations (right).

The *Picoscope 5204* is a dual-channel storage **USB2.0**-oscilloscope [Pic08], featuring a maximum sample-rate of 1 GHz, an 8 bit **ADC** with an accuracy of $\pm 3\%$ and a huge 128 MSamples waveform memory. The input bandwidth is 250 MHz, with a minimum input range of ± 100 mV. Additionally, the device offers a separate external trigger input and a built-in arbitrary signal generator, e.g., for recording frequency responses of antennae or filters. These characteristics

are well suited for performing power analyses⁵, alone the minimum input range of ± 100 mV might pose a problem, if voltage variations with very small amplitude need to be detected with a high accuracy.

In order to handle the device like a standalone oscilloscope, Picotech provides the software *PicoScope* that enables accessing most features of the scope via a Graphical User Interface (GUI). Besides, all functions can be invoked using a library available for Windows and Linux. Especially when acquiring measurements for power analysis, this permits the creation of one singular application controlling both the communication with the DUT and the oscilloscope.

Programming Interface

When using the C Application Programming Interface (API), the Picoscope is controlled with routines defined in the Software Development Kit (SDK) header `ps5000Api.h`. In the context of a typical side-channel analysis, three main tasks are required: Opening/closing the communication link, setting up the trigger, starting a measurement, and finally reading back the acquired data. For an in-depth description of all functions, refer to the manual [Pic07].

To keep the architecture of our measurement setup modular and flexible with respect to future modifications, the API calls for the data acquisition are wrapped in `scope.c`. Moreover, internal handles and measurement data are stored in the structure `scope_state_t`. This prevents multiple initialization or cleanup, yielding a singleton-like pattern. The following basic commands implemented in `scope.c` give an idea which most relevant functions of the Picoscope API usually need to be called.

unsigned int scope_open() Opens the scope connection with `ps5000OpenUnit()`. Prints error code and returns 0 on failure, 1 else.

unsigned int scope_close() Closes the scope connection with `ps5000CloseUnit()` and frees measurement buffers. Prints error message and returns 0 on failure, 1 else.

unsigned int scope_sample_start(unsigned long sample_count) Selects measurement range for channel A, with channel B disabled (`ps5000SetChannel()`). Then, sets up trigger on external trigger rising edge (`ps5000SetTriggerChannel*` API calls). The data buffer for measurement values is allocated and selected with `ps5000SetDataBuffer()`, sample rate (1 GHz) and count (passed as parameter) are set and the scope is started, waiting for a trigger condition (`ps5000RunBlock()`).

scope_sample_poll(short * buffer) Busy-waits for an internal flag (`measuring`) to be reset by a callback function which is invoked when a trigger event occurred and the specified number of samples has been recorded. After that, the oscilloscope is stopped by `ps5000Stop()`, the recorded data is acquired (`ps5000GetValues()`) and written to the specified memory buffer.

scope_sample_to_file(const char* file) Helper function provided for convenience that invokes `scope_sample_poll()` and writes the results to the specified file in American Standard Code for Information Interchange (ASCII) format, one value per line.

⁵In fact, the provided memory is much bigger than required for a typical side-channel attack.

4.3.2 Probes

Various types of probes can be connected to the oscilloscope for acquiring side-channel leakage and other data. They establish the link between the **DUT** and the oscilloscope, i.e., the probes are required to tap the signals to be measured. Their characteristics, such as attenuation or amplification and bandwidth, are of great importance for the quality of the measurements.

When digitizing data, the Nyquist-Shannon sampling theorem is of relevance. It states that to exactly reconstruct a sampled analogue signal⁶ with a bandwidth B in the digital domain, without losing information, the sampling frequency f_s must at least be twice big as the highest frequency component of the signal: $f_s > 2B$. For example, an oscilloscope sampling at $f_s = 500$ MHz captures all frequency components of a signal up to $B < f_s/2 = 250$ MHz. The bandwidth of the probe should hence be selected accordingly.

Voltage Probes

Voltage probes usually measure the voltage with respect to a common Ground (**GND**), e.g., the voltage drop of a resistor inserted in the **GND** path of a device. By connecting the ground tip of the probe to the **DUT**, a connection between the oscilloscope (and thus the controlling **PC**) and the **GND** of the **DUT** is established. The resulting “ground loop” should be avoided, because it induces additional noise, e.g., a 50 Hz sine that disturbs the measurements. Whenever possible, the **GND** of the **DUT** should thus be floating, so that the only connection to **GND** is via the ground tip of the probe: This can be achieved, e.g., by powering the **DUT** from a battery.

The attenuation ratio of the probes can often be selected by means of a switch, e.g., between 1:1 and 1:10. The latter ratio implies that the measured amplitude is decreased by a factor of ten, but enables the full bandwidth of the probe. The higher amplitude of a direct connection (ratio 1:1) comes at the cost of a bandwidth that is limited by at least a factor of ten⁷. For example, the *MI 145* passive high impedance probes that are supplied with the Picoscope have a bandwidth of $B = 250$ MHz at a 1:10 attenuation ratio, which is lowered to only $B = 10$ MHz for the 1:1 connection. Agilent’s passive probes 1160A have a bandwidth of $B = 500$ MHz at a 1:10 attenuation, while the bandwidth of the corresponding 1162A probe with a 1:1 ratio is limited to $B = 25$ MHz [Agi06].

One outcome of my experiments with respect to the amplitude and bandwidth required for **SCA** is, that limiting the bandwidth worsens the efficiency of an **SCA** attack much more than a limited amplitude. If one has the choice, usually the smaller amplitude (corresponding to the maximal bandwidth) is the better compromise, even if the digitized traces appear to be terribly quantized.

In contrast to passive probes, *active probes* possess an amplifier positioned close to the tip of the probe and thus allow for a higher bandwidth. The Agilent 1156A active probe in our setup has a bandwidth of 1.5 GHz at an attenuation ratio of 1:10. An *active differential probe* is useful if the voltage cannot be measured in the ground path but instead the resistor has to be placed in the voltage supply path: acquiring the voltage difference that drops at the resistor with a standard probe will result in a bad resolution, since the measurement range of the oscilloscope has to be set at least to the level of the supply voltage. A differential probe instead outputs only the differential voltage drop at the resistor (in contrast to the normal

⁶We assume a band limited signal, the highest frequency component of which does not exceed B .

⁷In electronics, generally a trade-off between bandwidth and amplification can be made.

probe which measures the voltage referenced to ground), hence a better resolution is possible. The LeCroy AP033 active differential probe in our setup allows for differential measurements with a bandwidth of $B = 500$ MHz, while the signal can be internally amplified by a factor of ten, and/or attenuated by a factor of ten. In consequence, amplification or attenuation factors of $\times 10$, $\times 1$, and $\div 10$ can be selected. The costly differential probes may be required for highly sophisticated attacks on real-world devices, especially hardware implementations. If no differential probe is available, many oscilloscopes offer a mode in which the signals acquired from two connected probes are instantly subtracted and displayed, the result of which is similar to measuring with a differential probe.

The bandwidth required for SCA attacks is usually low: often 50 MHz bandwidth is sufficient for a successful attack. An SCA attacker in most cases does not require expensive, sophisticated equipment. Rather, cost-efficient solutions for data acquisition are often adequate. All side-channel attacks in this thesis can be conducted with the low-cost Picoscope USB oscilloscope and the supplied standard, passive probes (or the EM probes detailed below).

EM Probes

For measuring EM emanations, near-field probes manufactured by Langer EMV that are originally designed for Electro-Magnetic Compatibility (EMC) tests have proven to be highly suitable for the purpose of physical cryptanalysis. The probes are connected via standard BNC cables with a characteristic impedance of $50\ \Omega$. They are available for different frequency ranges and each probe captures a different type of field, e.g., a capacitive probe acquires the electric field or an inductive probe captures solely the magnetic emanations. For most measurements, the *RF-U 5-2* probe (highlighted in Fig. 4.5b) is used, because it allows capturing the emanated H-field, which is proportional to the current flow in the IC. Note that, if no commercial EM probes are at hand, a self-wound coil can be a suitable replacement [Car05].

4.3.3 Amplifiers

The small signal amplitudes (max. 10 mV) delivered by a near-field probe or other signals with a small amplitude are preamplified with the *PA-303* amplifier [Lan] to meet the dynamic range of the oscilloscope. The amplifier features an almost constant gain of 30 dB ($\hat{=}$ multiplication by ≈ 31.6) over a wide frequency range of 3 GHz. Note that this amplifier was designed for a characteristic impedance of $50\ \Omega$ thus appropriate termination is required for high-impedance scope inputs. Besides, its output amplitude is — according to the vendor — limited to ≈ 1 V. Above this level, non-linear clipping effects occur. Thus, the input must not exceed ≈ 30 mV.

4.4 Contact-Based Analyses and Fault Injection

Other hardware extensions have been developed for analyzing contact-based cards and for injecting faults, as described next. Although these extensions are not directly relevant for attacks targeting wireless devices, as performed in this thesis, they enable other attack options that may become relevant in future security analyses: For example, various secured cards offer both a contactless and a contact-based interface, and fault attacks by means of emitting light or electric pulses are promising future attack vectors for wireless tokens.

4.4.1 Extensions for Contact-Based Smartcards

In the course of this thesis, a special adaptor for *emulating contact-based chipcards* and a *modified chipcard reader* have been realized. They extend the measurement setup in order to support widespread smartcards according to ISO 7816 [Int04a]. The former adaptor can be used for performing relay attacks, for monitoring purposes, and for assisting side-channel analyses. The latter modified smartcard reader enables convenient power analyses of chipcards. It has meanwhile become an essential part of a power-analysis lab course at the Ruhr-University Bochum and is used in various projects, e.g., power-analysis of Java cards and Atmel CryptoMemory cards. The adaptor and the modified reader are further described in Appendix D.

4.4.2 Extensions for Fault Injection

Special laser-based setups for the injection of optical faults are common in the evaluation labs or in the test-departments of large manufacturers. This extremely costly equipment is not considered here, since the scope of this thesis is on determining the security impact of low-cost attacks. Instead, low-cost methods for injecting faults have been developed: A dedicated fault injection module takes care of the active aspects of side-channel analysis. The FPGA-based functional group supports injecting a variety of faults to the DUT and is designed on a flexible basis, such that it can be easily extended with respect to other fault sources.

Fault injections targeting the supply voltage of passively powered wireless tokens can be conducted with the developed customized RFID readers, as shown in Chap. 6, since the readers are in control of the EM field providing the energy supply. Further extension modules, one for injecting optical faults by means of a modified flash unit of a photo camera, and another one for creating EM pulses by means of spark plugs, are sketched in Appendix E. A description of a versatile framework for conducting fault attacks that is incorporating the here developed modules has been published in [KOP10].

4.5 COPACOBANA

Cryptanalysis of ciphers often involves massive computations. The security parameters of cryptographic algorithms are commonly chosen so that attacks are infeasible with available computing resources. Thus, in the absence of mathematical breakthroughs to a cryptanalytical problem, a promising way for tackling the computations involved is to build special-purpose hardware exhibiting a (much) better performance-cost ratio than off-the-shelf computers. This section briefly introduces such a customized, reconfigurable hardware platform termed Cost-Optimized Parallel Code Breaker and Analyzer (COPACOBANA). The architecture of the high-performance, low-cost cluster that can be realized for less than US\$ 10,000 had been publicized in 2006 [KPP⁺06c]. COPACOBANA is the first reconfigurable parallel FPGA machine optimized for code breaking tasks reported in the open literature. Depending on the actual algorithm, the parallel hardware architecture can outperform conventional computers by several orders of magnitude.

4.5.1 Hardware Architecture

The **COPACOBANA** has been developed according to the following design criteria [KPP⁺06c]: First, it is assumed that computationally costly operations are parallelizable. Secondly, parallel instances have only a very limited need to communicate with each other. Thirdly, the demand for data transfers between host and nodes is low due to the fact that computations heavily dominate communication requirements. Ideally, communication between the hardware and a host computer is only required for initialization and the transfer of results. Fourthly, the targeted algorithms and their corresponding hardware nodes demand only little local memory.

A total of 120 **FPGA** devices is installed on the **COPACOBANA** cluster. Its hardware functionality is stripped down to the bare minimum in order to achieve an optimal cost-performance for code breaking. The modular and maintainable architecture consists of small **FPGA** modules which can be dynamically plugged into a backplane providing a shared bus with an aggregate bandwidth of 1.6 Gbps. Each of the modules in Dual In Line Memory Modules (**DIMM**) form factor hosts six low-cost Xilinx Spartan3-XC3S1000 **FPGAs** which are directly connected to a common 64-bit data bus on board. The data bus of the module is interfaced to the global data bus on the backplane. While disconnected from the global bus, the **FPGAs** on the same module can communicate via the local 64-bit data bus. Additionally, control signals are run over a separate 16-bit address bus.

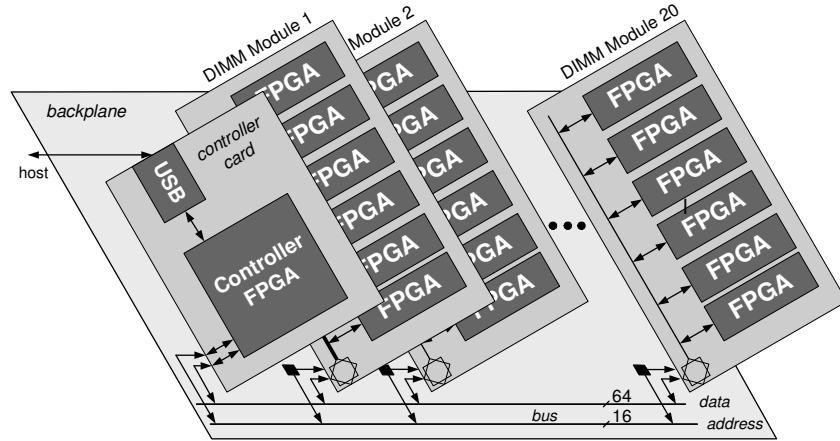


Figure 4.6: Architecture of COPACOBANA

Figure 4.6 gives a detailed overview of the architecture of **COPACOBANA**. For simplicity, a single master bus was selected to avoid interrupt handling. Hence, if the communication scheduling of an application is unknown in advance, the bus master has to poll the **FPGAs**. A 24 MHz clock for the backplane, generated by a clock synthesizer, is used to derive a system clock by means of Digital Clock Managers (**DCMs**) that are part of each **FPGA**.

4.5.2 Computing with COPACOBANA

The top level entity of **COPACOBANA** is a host-**PC** which is used to initialize and control the **FPGAs**, as well as for the accumulation of results. Programming can be done simultaneously for all or a specific subset of **FPGAs**. Data transfer between the **FPGAs** and the host-**PC** is

accomplished by a dedicated control interface. This interface has also been designed as a slot-in module so that **COPACOBANA** can be connected to a computer either via a **USB** or an Ethernet controller card. A software library on the host-**PC** provides low-level functions that allow for device programming, addressing individual **FPGAs**, as well as storing and reading **FPGA**-specific application data. Several **COPACOBANAs** can be combined in order to further increase the performance.

The hardware is optimized for computational problems which are parallelizable onto independent nodes with low communication and memory requirements. **COPACOBANA** is not equipped with dedicated memory modules, but offers a limited number of **RAM** blocks inside each **FPGA**. The bottleneck of the architecture is the communication via the buses and to the **PC**. This has to be taken into account for an efficient implementation, so special care has to be taken to minimize the data traffic.

Meanwhile, various improved **FPGA** clusters have emerged as follow-ups of **COPACOBANA**, e.g., the RIVYERA⁸, which possess increased computational power, due to larger logical and memory resources as well as faster buses and interfaces. For the re-design of the successors, various disadvantages of **COPACOBANA** have been addressed to enable a more flexible usage of the clusters, e.g., in the field of bioinformatics.

4.5.3 Applications of COPACOBANA

Breaking modern ciphers such as **AES** (128/192/256 bit of key), full-size **RSA** (1024 bit or more), or **ECC** (160 bit or more) is out of reach with **COPACOBANA**. However, the machine can be employed to gather data for extrapolating attacks with realistic security parameters in terms of financial cost and attack time. Equally important, there are numerous legacy systems (and not-so-legacy systems such as the electronic passport) which are still operating with key lengths that can be tackled with **COPACOBANA**. Hence, the machine can attack unfavorably used cryptography in real-world applications, as detailed in our article [GKN⁺08].

In this thesis, the **COPACOBANA** serves as the hardware platform for analyzing the security of two wireless embedded systems: Chapter 9 details on extracting secret keys that are used for securing the wireless communication of electronic passports and illustrates respective attack scenarios. Another implementation of exhaustive key-search, as described in Sect. 8.5.5, is used to examine the security margin of KEELOQ **RKE** systems for the special case that the secret keys are generated from random seed values.

⁸www.sciengines.com

Chapter 5

Chameleon: A Versatile Emulator for Contactless Smartcards

AN emulator for contactless smartcards is a convenient tool for circumventing various security-relevant applications. The cost for such a device is often overestimated. This chapter presents a cost-efficient emulator for contactless smartcards that possesses sufficient computational power to implement state-of-the-art cryptography. Thus, the versatile device termed Chameleon can technically appear as any modern contactless smartcard, such as Mifare Classic or Mifare DESfire, and allows conducting practical attacks on commercial systems. The creation of the Chameleon has been a joint work with Ingo von Maurich and David Oswald, as published in the proceedings of ICISC 2010 [KvMOP11], and has meanwhile become a [GPL](#) project¹.

5.1 Introduction

Due to the cost sensitivity of high-volume [RFID](#) applications, card manufacturers and system integrators are tempted to use outdated but “cheap” cryptographic components, e.g., in Mifare Classic products. Since the reverse-engineering of the Crypto1 cipher used in Mifare Classic cards, various attacks have evolved (cf. Sect. 6.4). The cards have to be regarded as insecure, as the secret keys can be extracted in seconds, e.g., by means of the efficient card-only attack implemented in Sect. 6.5.

Accordingly, many recently installed contactless systems, especially those with high security demands, are based on the DESfire variant of the Mifare family, and some system integrators upgrade the old Mifare Classic technology to these newer cards (at least on the long term). While the [3DES](#) cipher employed in these cards is secure from the mathematical point of view, the implementation on the card is vulnerable to side-channel analysis. As a consequence, it is again possible to extract the secret keys of a card², as proven by the power-analysis attack developed in Chap. 7. Hence, emulating these modern cards is also practical and renders various attacks in real-world scenarios possible³.

The resulting security weaknesses can become very costly. One example is the widespread contactless payment system based on Mifare Classic cards analyzed in Chap. 6, where the credit

¹sourceforge.net/projects/chameleon14443/

²Note that the effort for extracting secret keys from Mifare DESfire MF3ICD40 cards by means of side-channel analysis is much higher compared to the Mifare Classic key recovery described in Chap. 6.

³After our side-channel attacks, the production of DESfire cards has been discontinued in June 2010 and a more secure follow-up of DESfire, termed DESfire EV1, has been placed on the market. These improved cards can also be emulated by the Chameleon (see Sect. 5.4.3).

value on the cards can be modified by an adversary with minimal effort. For many of these systems, the read-only **UID** of each card constitutes the last remaining means to detect fraud in the back-end. Various access-control systems even rely solely on reading the **UID** of a card, assuming that this number is fixed, unique, and unforgeable. In this chapter, the possibility of fully emulating and cloning cryptographic smartcards compliant to ISO 14443, including their **UID**, is exhibited. The chapter demonstrates that electronic devices for security-analyzing and practically spoofing **RFID**-enabled systems can be realized at a very low cost.

5.1.1 Previous Work: Existing RFID Emulators

Several researchers have proposed custom devices to emulate and counterfeit **RFID** devices. However, virtually all emulators presented so far suffer from certain drawbacks limiting the threat they pose in the context of attacking real-world systems, e.g., insufficient computational resources, high cost, or impractical dimensions.

A custom **RFID** emulation hardware called Ghost is presented in [Ver]. The Ghost is able to emulate Mifare Ultralight cards which do not use any encryption. Emulating contactless cards employing secure cryptography seems to be impossible using this device due to computational limitations. The OpenPICC project [Ope] is mainly a device for sniffing, i.e., eavesdropping, of **RFID** communication. There was an approach to offer support for ISO 14443A, but the project seems to be discontinued. The Proxmark III [Pro] enables sniffing, reading and cloning of **RFID** tags. Since the device is based on an **FPGA**, it is capable of emulating various cards including Mifare Classic, but at a comparably high cost of \$399. Programming the **FPGA** in VHSIC (Very High Speed Integrated Circuit) Hardware Description Language (**VHDL**) is rather complicated compared to programming a microcontroller. The “HF Demo tag” [IAI] is based on an Atmel ATMega128 microcontroller which is not computationally powerful enough to perform encryptions with state-of-the-art ciphers in the time window given by the relevant protocols.

Our **RFID** tool illustrated in Sect. 4.1 is based on an Atmel ATMega32 [Atma] microcontroller which lacks sufficient computational power for sophisticated cryptography. All important functionality is provided by the **RFID** reader, thus the Fake Tag cannot emulate cards independently from the reader — this can be a major drawback for practical attacks. Similar to the HF Demo tag, the existing implementation of an encryption with Crypto1 runs too slow so that timing constraints of the protocol cannot be met. The Fake Tag constitutes the starting point for the development of a new stand-alone **RFID** emulator developed in this chapter which eliminates the mentioned drawbacks.

5.1.2 Contribution and Organization of this Chapter

A freely programmable low-cost device is presented that is capable of emulating various types of contactless smartcards, including those employing secure cryptography. The device operates autonomously without the need of a **PC**, can be powered from a battery, and possesses an **EEPROM** for storing received bitstreams or other non-volatile information. An attacker using the hardware detailed in Sect. 5.2 has complete control over the contactless interface of the emulated card. The targeted variants of Mifare cards are detailed in Sect. 5.3.

In order to demonstrate the capabilities of the developed tool in the context of real-world attacks, in Sect. 5.4 optimized versions of the Crypto1 stream cipher, the **DES**, **3DES**, and the

AES ciphers are implemented, as required for emulating the widespread Mifare Classic, Mifare DESfire, and Mifare DESfire EV1 cards. Practical results for these cards, as well as for Mifare Ultralight cards, are given in Sect. 5.5. All data stored on an emulated card, e.g., its **UID**, the cryptographic keys, or access conditions, can be adapted. With the developed hardware and software, it is thus possible to simulate the presence of cards with an arbitrarily chosen content and identifier, and spoof real-world systems in various manners, as described in Sect. 5.6. For example, the emulator can behave as a Mifare Classic card that automatically restores its credit value after a payment, or that possesses a new **UID** and card number on each payment, which impedes the detection of fraud.

The protocol of the DESfire EV1 card is reverse-engineered in Sect. 5.3.2 and it is practically shown that the developed tool is capable of emulating more complex, secure devices such as Mifare DESfire and Mifare DESfire EV1. Besides the simulation of cards, the hardware allows for sniffing, e.g., reverse-engineering of protocols, relay attacks, and testing the vulnerability of **RFID** readers towards a behavior of the card that does not conform to the specifications, for instance, with respect to timing, intentionally wrong calculation of parity bits, or buffer overflows.

5.2 Hardware Setup

The freely programmable emulator for contactless smartcards, termed “Chameleon”, is built from off-the-shelf hardware for less than \$25. It is based on an Atmel ATxmega192A3 microcontroller [Atmb, Atme] which provides 192 kB of program memory, 16 kB **SRAM** and 4 kB **EEPROM** memory. Using an FTDI FT245RL chip [Fut], the ATxmega is able to communicate with a **PC** via **USB**. This communication link can be used for debugging purposes and data manipulation at runtime. Figure 5.1 shows the first version of our **RFID** emulator.

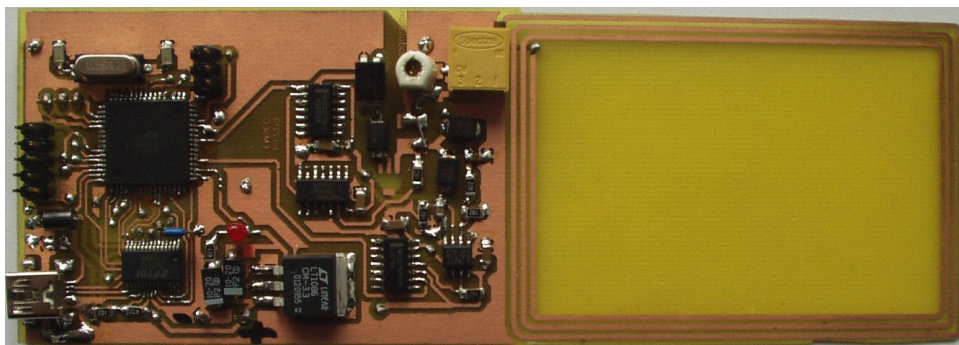


Figure 5.1: The stand-alone RFID emulation device “Chameleon”.

The ATxmega μ C features a hardware acceleration of both **DES** and **AES-128**. After loading the key and the data to the corresponding registers, the ATxmega is able to perform a **DES** en- or decryption in 16 clock cycles, i.e., one **DES** round per clock cycle, whereas the **AES** engine runs concurrently to the Central Processing Unit (**CPU**) and requires 375 clock cycles for an encryption or decryption of one 128-bit block. The microcontroller is clocked by an

external 13.56 MHz crystal, which is internally doubled to 27.12 MHz using a high frequency Phase Locked Loop (PLL).

The coupling to the reader is established by a rectangular coil that is part of the PCB and is dimensioned according to the size of a standard credit card. Variable capacitors are placed in parallel to the inductance of the antenna to form a parallel resonant circuit that is tuned to the carrier frequency. Analog circuitry assists the microcontroller in extracting the encoded data from the EM field and transmitting bitstreams. The design is similar to the Fake Tag illustrated in Sect. 4.1.2 and mainly shapes the signals according to the ISO 14443 standard and converts them to the appropriate voltage levels. Our emulation device can either be powered via the USB interface or run on battery. As all functionality is directly provided by the microcontroller, the Chameleon operates autonomously without the support of a PC. The full schematics of the developed hardware are given in Appendix J.2. Figure 5.2 illustrates the operating principle of the Chameleon.

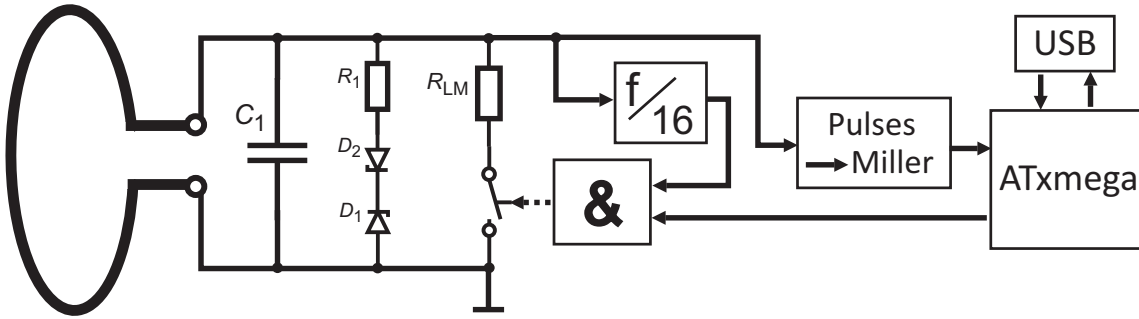


Figure 5.2: Operating Principle of the Chameleon: A parallel resonant circuit is formed by the antenna and C_1 . The Zener diodes D_1 and D_2 together with resistor R_1 protect the remaining circuitry from high voltages, while the resistor R_{LM} generates the load modulation for sending responses to a reader.

5.3 Mifare Contactless Cards

This section gives details about Mifare Ultralight and Mifare DESfire EV1 cards, including the reverse-engineered authentication protocol for the latter. The relevant protocols and other information required for the emulation of Mifare Classic and Mifare DESfire cards, as well as their security, are discussed in Sect. 6.3 and Sect. 7.2, respectively.

5.3.1 Mifare Ultralight

With Mifare Ultralight cards, NXP offers an inexpensive electronic replacement for (disposable) paper tickets such as one-way tickets in public transport or tickets for events, e.g., the Football World Cup 2006. They comply with Parts 1-3 of ISO 14443A and offer a 512 bit (64 byte) EEPROM memory that is organized in 16 pages of four byte each. 384 bit of the memory and an additional 32-bit OTP area can be used for the application. Each page can be locked by setting corresponding lock bits located in Page 2. Locking freezes the content of a page and

permanently restricts it to read-only access. An illustration of the memory structure is given in Table 5.1.

Byte Number	0	1	2	3	Page
UID	UID ₀	UID ₁	UID ₂	BCC ₀	0
UID	UID ₃	UID ₄	UID ₅	UID ₆	1
Internal/Lock	BCC ₁	Internal	Lock ₀	Lock ₁	2
OTP	OTP ₀	OTP ₁	OTP ₂	OTP ₃	3
Data r/w	Data ₀	Data ₁	Data ₂	Data ₃	4
Data r/w	Data ₄	Data ₅	Data ₆	Data ₇	5
Data r/w	Data ₈	Data ₉	Data ₁₀	Data ₁₁	6
Data r/w	Data ₁₂	Data ₁₃	Data ₁₄	Data ₁₅	7
Data r/w	Data ₁₆	Data ₁₇	Data ₁₈	Data ₁₉	8
Data r/w	Data ₂₀	Data ₂₁	Data ₂₂	Data ₂₃	9
Data r/w	Data ₂₄	Data ₂₅	Data ₂₆	Data ₂₇	10
Data r/w	Data ₂₈	Data ₂₉	Data ₃₀	Data ₃₁	11
Data r/w	Data ₃₂	Data ₃₃	Data ₃₄	Data ₃₅	12
Data r/w	Data ₃₆	Data ₃₇	Data ₃₈	Data ₃₉	13
Data r/w	Data ₄₀	Data ₄₁	Data ₄₂	Data ₄₃	14
Data r/w	Data ₄₄	Data ₄₅	Data ₄₆	Data ₄₇	15

Table 5.1: Memory structure of Mifare Ultralight according to [NXP10a].

The cards are equipped with an unchangeable 7-byte **UID** that is programmed into the first few memory bytes. They communicate at a fixed data rate of 106 kbit/s and ensure data integrity by parity bits and 16-bit Cyclic Redundancy Check (**CRC**) checksums. Cryptographic security measures are not existent — all data is stored and transmitted unencryptedly. Hence, confidentiality and sound cryptographic data integrity are not provided by Mifare Ultralight.

In addition to standard ISO 14443A commands, a proprietary high-level command set consisting of one read and two write commands (cf. Table 5.2) is used for exchanging data between reader and card. As there is no cryptography available, no authentication protocol has to be completed before data manipulations are granted: After a card has been successfully selected according to ISO 14443A, the read and write commands can be directly executed.

Command	Description
0x30 ADR	Read 16 bytes starting from ADR
0xA2 ADR D ₀ . . . D ₃	Write bytes D ₀ . . . D ₃ to page ADR
0xA0 ADR D ₀ . . . D ₁₅	Write the four least significant bytes to ADR

Table 5.2: Data manipulation commands of Mifare Ultralight. Command 0xA0 only exists for compatibility reasons.

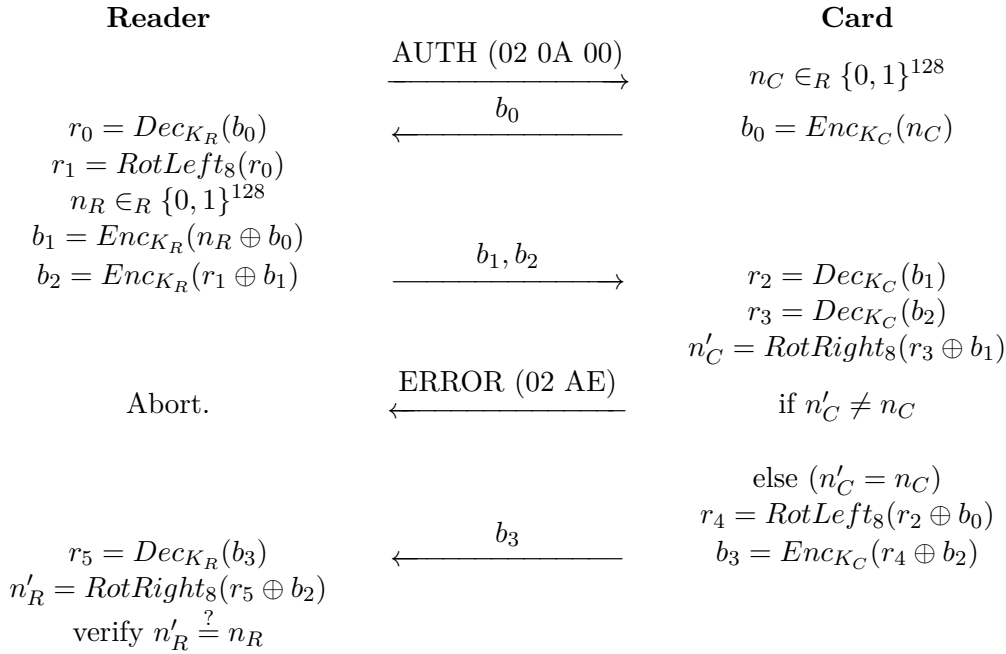
5.3.2 Mifare DESfire EV1 Cards

Mifare DESfire EV1 cards are compliant to Parts 1-4 of ISO 14443A and are sold in three variants with 2 kB, 4 kB and 8 kB of non-volatile memory. Their seven-byte **UID** is unchangeably

programmed into the card at production time. A communication with high baud rates up to 848kBit/s can be performed in plain, with an appended **MAC**, or secured with full data encryption. For the latter, the **DES**, **3DES**, and **AES**-128 ciphers are provided. Each card holds up to 28 different applications with up to 14 different keys per application. Each of application may contain up to 32 files.

Mifare DESfire EV1 Authentication Protocol

Depending on the access rights for each application a mutual authentication protocol, ensuring that the symmetric keys of the card K_C and of the reader K_R are identical, has to be completed before reading or manipulating the content of the card. We reverse-engineered this scheme, presented in Protocol 1, by eavesdropping on genuine protocol runs.



Protocol 1. The Mifare DESfire EV1 authentication protocol.

The DESfire EV1 protocol mutually authenticates both parties upon successful execution. It resembles the protocol of Mifare DESfire cards (see Sect. 7.2.1), with some modifications as follows. While Mifare DESfire cards only perform (3)DES *encryptions*, the DESfire EV1 uses encryption and decryption in the common sense, i.e., data that is to be sent is encrypted and data that was received is decrypted. **AES**-128 encryption and decryption involving the key K are denoted by $Enc_K(\cdot)$ and $Dec_K(\cdot)$, respectively.

The **CBC** mode is modified in a way that all encryptions or decryptions are chained, even though they operate on different cryptograms. The **IV** is not reset when en- or decrypting a new message, but instead depends on the previously en- or deciphered block. The nonces are extended to a length of 128 bit to match the block size of **AES**-128 and the second rotation is

executed in the opposite direction on both sides. The exact binary format of the commands, as required for an ISO 14443-compliant implementation, is given in Table 5.3.

Table 5.3: Messages exchanged during an authentication protocol run between a reader R and a Mifare DESfire EV1 card C.

#	Direction	Protocol Message	Explanation
1	R → C	02 AA, key (1 byte), CRC1 CRC2	Auth key number CRC
2	C → R	02 AF, 16 byte, CRC1 CRC2	Card nonce b_0 CRC
3	R → C	03 AF, 16 byte, 16 byte, CRC1 CRC2	Rdr. response b_1 b_2 CRC
4	C → R	03 00, 16 byte, CRC1 CRC2	Success b_3 CRC

Security of Mifare DESfire EV1

The non-invasive side-channel attack presented in Chap. 7 allows to extract secret information from contactless cards by measuring the electromagnetic emanations of a card while it carries out a cryptographic operation. Note that the power-analysis attack is currently *not* applicable to DESfire EV1 cards, which have been certified according to Common Criteria EAL 4+⁴. However, more efficient attacks might come up in the future or the secret key could be obtained by other means, e.g., by exploiting weaknesses of the back-end system or social engineering.

5.4 Software Implementations on Chameleon

This section describes the software implementations for emulating Mifare Ultralight, Classic, and DESfire(EV1) cards with Chameleon, including a highly optimized variant of the Crypto1 stream cipher for an 8-bit platform.

5.4.1 Mifare Ultralight Emulator

As Mifare Ultralight cards are not cryptographically secured, reading out the full memory content does not pose a problem for an attacker. The only “security feature” of Mifare Ultralight is the unchangeable **UID** which cannot be altered when writing the memory content to another Mifare Ultralight card. With Chameleon we are able to control the full memory content of an emulated card, hence, we also control the memory blocks in which the **UID** is stored. Being able to set the **UID** as desired eliminates the last security measure of Mifare Ultralight and enables an attacker to produce exact clones.

In addition, with our emulator we are able to subvert the effectiveness of lock bits. If a reader freezes certain pages by setting the lock bits, we are still able to modify the corresponding pages since we are in control of the memory and simply do not enforce the locking. For example, invalidating a one-way public transportation ticket can be achieved by setting a page to a specific value and locking this page afterwards. If such a method is used we are able to re-validate one-way tickets when emulated with Chameleon.

⁴<http://www.commoncriteriaportal.org/>

5.4.2 Mifare Classic Emulator

The attacks detailed in Sect. 6.4 prove that an adversary can easily read out the secret keys and all content of a Mifare Classic card. In order to produce a duplicate, the adversary can write all previously read data to a blank Mifare Classic card, as described in Sect. 6.7. This results in an almost perfect clone, differing from the original only in the single block containing the read-only **UID** of the blank card. If the **UID** is verified by a contactless system, this “restricted clone” becomes useless in practice (compare with Sect. 6.8). For emulating exact clones, we implemented the features of Mifare Classic on Chameleon, with complete control over all memory including the previously unchangeable manufacturer block. Most importantly when emulating Mifare Classic, the Crypto1 cipher (see Sect. 6.3.3 for a full description of the cipher) needs to be implemented efficiently, as shown next.

Optimized Crypto1

A first approach to emulate a Mifare Classic card on an AVR ATmega32 microcontroller revealed difficulties in complying to the timing requirements given in ISO 14443, as described in Sect. 2.5.5: After a command being sent by the reader, the card has to reply within 4.8ms, otherwise the reader reaches a timeout and aborts the connection. Compiling the open-source Crypto1 C-library [Cra08] for an 8-bit microcontroller results in inefficient code regarding the underlying platform — the time limit of 4.8ms set in the protocol is exceeded with 11.7ms for an 18-byte encryption, neglecting all other necessary computations, such as encoding the encrypted data. Since an 18-byte encryption is required every time when reading or writing a 16-byte memory block with an appended 2-byte **CRC** checksum, the existing implementation is not suitable for real-world emulations. The (long) durations of the individual parts of a straightforward implementation of a Mifare Classic emulator are summarized in more detail in Appendix G.3.4.

Since the number of clock cycles required for executing single instructions on an ATmega32 is comparable to that required on the ATxmega192A3 of the Chameleon, it became obvious that a significant speedup of Crypto1 is essential for a successful Mifare Classic emulation. Hence, we implemented the cipher from the scratch in AVR assembly. This allows to optimize the code for an 8-bit platform and to make use of special commands that may not be considered by the C compiler. The number of clock cycles required for an encryption was reduced amongst others by replacing inefficient shifting and masking operations to access single bits with instructions that directly access a particular bit in one clock cycle (e.g., SBRC, BST, BLD).

We further implemented the non-linear filter functions f_a , f_b and f_c of Crypto1 (see Fig. 6.2 in Sect. 6.3.3) by means of look-up tables to avoid time-consuming Boolean AND, OR and XOR operations. As described in Sect. 6.3.3, in the first stage f_a is used twice and f_b three times with a 4-bit input of the state Linear Feedback Shift Register (**LFSR**). Their output is used to generate a 5-bit input to f_c , which in turn generates one bit of keystream. For both f_a and f_b , we created a dedicated look-up table that includes the respective shifting of the output. Thereby, the input of f_c — again implemented as a simple 5-bit input, 1-bit output table — is obtained by ORing the five outputs of f_a and f_b . This speed advantage comes at the cost of storing one bit of information in one byte of memory. The look-up tables for the filter functions were computed by iterating over all possible input combinations

and storing each result. The input (a, b, c, d, e) was incremented according to the scheme $(0, 0, 0, 0, 0)$, $(1, 0, 0, 0, 0)$, $(0, 1, 0, 0, 0)$, $\dots$, $(1, 1, 1, 1, 1)$.

The resulting look-up tables for the filter functions are

f_a : 0x712D or $(0111\ 0001\ 0010\ 1101)_2$,

f_b : 0x1979 or $(0001\ 1001\ 0111\ 1001)_2$, and

f_c : 0x5017EA37 or $(0101\ 0000\ 0001\ 0111\ 1110\ 1010\ 0011\ 0111)_2$.

The overall size of the look-up tables sums up to 112 byte, formed by two 16-byte tables for f_a , three 16-byte tables for f_b and one 32-byte table for f_c . With respect to the 192 kB available program memory, the tables are negligibly small.

Furthermore, we precomputed values required in the protocol, where possible. The nonce n_C is fixed before the authentication protocol (see Sect. 6.3.3) is executed, hence the card is able to precompute the corresponding answers a_R and a_C which saves time during the authentication process. Precomputing keystream bits is not possible because of two reasons: First, the sector to be accessed by the reader cannot be predicted, hence it is not clear which key has to be loaded into the LFSR. Second, the random reader nonce n_R that only becomes known during the authentication process is an input to the cipher.

5.4.3 Mifare DESfire (EV1) Emulator

Similar to the Mifare Classic emulator, we implemented the authentication protocols of both Mifare DESfire and Mifare DESfire EV1, as given in Sect. 7.2.1 and Sect. 5.3.2. For encryption, Mifare DESfire cards use DES/3DES, whereas Mifare DESfire EV1 cards can use either DES/3DES or AES-128. For encryptions and decryptions the CBC mode (cf. Sect. 2.3.6) is employed as illustrated in Fig. 5.3: The plaintext is split into blocks P_x and the resulting ciphertext is split into blocks C_x . Depending on the cipher the block size is either 64 bit for DES/3DES or 128 bit for AES. While DESfire cards constantly set the IV to 0, DESfire EV1 cards always use the previous plaintext (or ciphertext) block as IV, even if the previous block is not related to the current encryption or decryption.

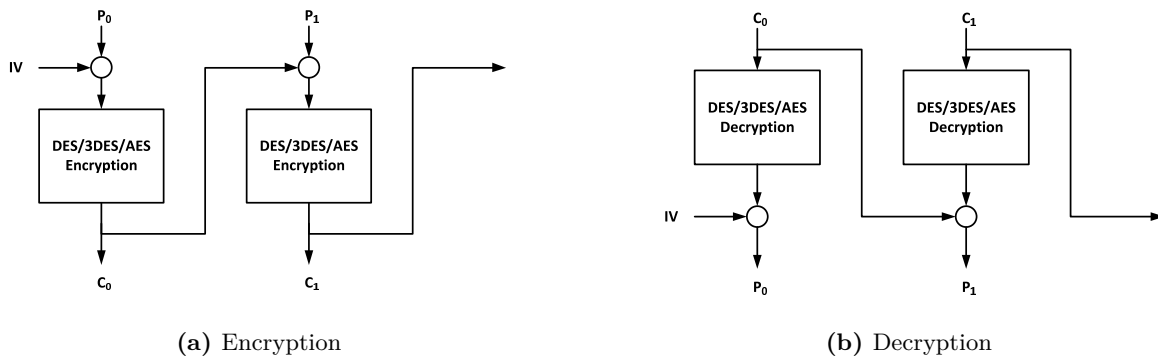


Figure 5.3: Illustration of the CBC-modes used in DESfire encryptions and decryptions.

Since the ATxmega microcontroller offers hardware acceleration for both DES and AES-128, programming the encryption and decryption routines is straightforward. The key and

data has to be loaded into the corresponding registers and the **CBC**-mode has to be configured. Furthermore, as **3DES** is not directly supported by the ATxmega, a **3DES** engine was built using the **DES** accelerator as core building block. An exemplary program code for the Chameleon is given in the Appendix [J.1](#).

In addition to **ISO 14443** anticollision-, select- and authentication-procedures, we implemented example responses for the commands `GetApplicationIDs()`, `SelectApplication(Application Identifier (AID))`, `GetFileIDs()` and `Read()`. Note that implementing the complete logical layer of DESfire cards is more complicated than implementing Mifare Classic, as DESfire cards offer more commands and a small file-system.

5.5 Practical Results

Before carrying out security analyses in the real world (see Sect. [5.6](#) and Sect. [6.6.3](#)), we thoroughly tested our emulator in the laboratory. The reliability and accurate timing behavior was successfully verified with different **RFID** readers, including an ACG passport reader [[id06](#)] and a Touchatag [[Tou](#)] reader.

5.5.1 Mifare Ultralight Emulation

Since Mifare Ultralight cards do not use cryptography, we have no trouble in meeting the timing requirements of **ISO 14443** and the readers in our laboratory successfully communicate with cloned Ultralight cards. The emulated Mifare Ultralight card can be equipped with a **UID** of our choice and its **OTP** area is controlled by us. Furthermore, lock bits that are set for certain pages can be ignored and, if desired, data can be written to locked pages.

5.5.2 Mifare Classic Emulation

With the optimized implementation of **Crypto1** detailed in Sect. [5.4.2](#), we successfully emulated Mifare Classic 1K cards with varying content. Table [5.4](#) summarizes the execution times for the relevant operations which are now all well within the limits specified in **ISO 14443**. All features, e.g., authentication, encrypted read and write of blocks, or specifying an arbitrary **UID**, are fully functional with the tested readers.

Table 5.4: Execution times of crucial **Crypto1** functions.

Command	Execution time	Explanation
<code>setup_crypto1()</code>	98 μ s	Initializes the cipher
<code>auth_crypto1()</code>	542 μ s	Keystream for the authentication
<code>crypto1_1()</code>	8.3 μ s	Generates 1 bit of keystream
<code>crypto1_8()</code>	49 μ s	Generates 8 bits of keystream
<code>crypto1_32()</code>	186 μ s	Generates 32 bits of keystream

5.5.3 Mifare DESfire (EV1) Emulation

Likewise, we verified our DESfire and DESfire EV1 emulations from Sect. 5.4.3. Table 5.5 shows the execution times for the needed cryptographic functions using the hardware accelerators of the ATxmega. Note that the first call of an encryption or decryption function involves some overhead for the initial setup. After that, subsequent blocks can be processed faster. For reference, we included the runtime both for a single block and for ten data blocks in Table 5.5.

Table 5.5: Durations of 3DES and AES-128 encryption and decryption functions implemented on the Chameleon.

Command	Block count	Execution time
TripleDES_CBC_Enc()	1 block	14.1 μ s
TripleDES_CBC_Enc()	10 blocks	85.1 μ s
AES128_CBC_Enc()	1 block	35.9 μ s
AES128_CBC_Enc()	10 blocks	270.2 μ s
AES128_CBC_Dec()	1 block	58.4 μ s
AES128_CBC_Dec()	10 blocks	304.9 μ s

According to [Car05], an original Mifare DESfire card answers 690 μ s (9356 clock cycles at 13.56 MHz) after b_1, b_2 were received during the execution of Protocol 3 (see Sect. 7.2.1). In this time window, two 3DES encryptions are performed (one encryption of two blocks and another encryption of a single block). Our implementation performs about three times faster than a genuine card, with 219 μ s (5932 clock cycles at 27.12 MHz) to produce a valid answer b_3 after b_1, b_2 was received. If the ATxmega would be clocked at 13.56 MHz, which is equal to the field frequency, with about 438 μ s it would still be faster than a genuine card.

A genuine *DESfire EV1* card replies with b_3 approx. 2.2 ms after having received b_1, b_2 . In contrast, our implementation only consumes about 438 μ s and is thus faster by a factor of five. As we are able to encrypt and decrypt faster than both DESfire cards, encrypting or MACing data, which is the most critical part of emulating Mifare Classic, does not pose a problem in the context of emulating DESfire (EV1) cards. For both Mifare DESfire and Mifare DESfire EV1, our implementation was tested successfully with the readers in our laboratory. Equally to the emulation of Mifare Classic cards, we are able to program our emulator with any secret keys and a UID that is free of choice. We conclude that the ATxmega microcontroller on our current hardware revision is powerful enough to handle the computations needed for the emulation of the simple Mifare Ultralight and Mifare Classic cards, and also for more sophisticated contactless smartcards using 3DES or AES.

5.6 Real-World Attacks with Chameleon

We successfully employed the Chameleon to bypass the security mechanisms of several real-world systems, for example, we utilized the Mifare Classic emulation to fake a payment card that is accepted by the widespread payment system analyzed in Chap. 6. Additionally, we were

able to clone an employee identification card which is part of a large scale access control system, and can unauthorizedly open doors and use elevators with the Chameleon.

5.6.1 Payment System

In Sect. 6.6.3 we successfully demonstrate how to

- create an exact clone of a payment card and pay with the Chameleon,
- perform a state-restoration attack, i.e., restore the previous credit value after it has been expended, and
- make the Chameleon appear like a new card with a different UID, a different card number, and a new credit balance on each payment.

Note that the last attack implies that the card cannot be blacklisted and blocked in the back-end, nor can the adversary be tracked.

5.6.2 Copy-and-Print System

In a similar manner, we were able to spoof a copy-and-print service that relies on contactless smartcards. The copy-and-print stations are equipped with RFID readers that decrease the credit stored in a block of the Mifare Classic card according to the number of copies or printings carried out. By repeatedly using the service and comparing the content of the card between the payments, we found the block in which the remaining credit was stored. We hence programmed our card emulator to simulate the original card such that the credit appears to be lowered on each payment. However, the previous state of the card, i.e., charged to a high credit value, was again restored by pressing the state restoration button on our hardware. As a consequence, we gain unlimited copies with our hardware.

5.6.3 A Vulnerable Access Control System

CerPass [Sie04a], SiPass [Sie07] and SIPORT [Sie04b] are scalable access control solutions produced by Siemens. Besides access control of regular employees they offer features such as visitor management, personalized access control cards, time recording, and elevator control. Because of the modularity of the system, the customer can choose from various authentication techniques. It is possible to use pin pads, magnetic stripe readers and several contactless solutions. Among the applicable readers are 125 kHz proximity card readers using EM4102 [EM 05], Hitag1/2 [NXP01] [NXP97] or Cotag [Spe], and also 13.56 MHz readers for contactless smartcards, such as Mifare Ultralight, Mifare Classic and Mifare DESfire.

In our tests we had a closer look at an instantiation of the Siemens SIPORT system in a large international company with more than 8,000 employees, located in Germany. Each employee owns an access-card that must be held in front of a reading device to be granted access to a company building. Not only the doors but also the elevators are equipped with proximity card readers. Changing floors is only possible after a valid access-card is presented in the elevator. Furthermore, areas with extra security needs, such as computer centers, require employees to re-authenticate themselves via their access-card. Access to these areas is generally restricted to a small subgroup of employees.

The access control systems of all branches of the company are interconnected, so new employees get access to all branches and floors for which their security clearance is sufficient and access-cards cannot be used at other branches after revocation. Additionally, the live back-end connection allows for the creation of temporary guest accounts and immediately effective modifications of security clearances for employees. This particular access control system uses Siemens AR6181-MS **RFID** readers, as shown in Fig. 5.4, that are connected to the door- and elevator controllers. They are designed for usage in Mifare Classic 1K & 4K access control applications and use an RS-485 interface for on-line communication to the back-end. Three **LEDs** inform the user about the current status: A blinking yellow **LED** represents the idle mode, during which the reader is waiting for an access-card to enter its **EM** field. If the presented card was positively checked in the back-end the green **LED** flashes and access is granted. The red **LED** is turned on, if the card is unknown or the security clearance is not sufficient. All tested access-cards turned out to be Mifare Classic 1K cards printed with the name and portrait photo of the employee holding the card. However, neither the name nor the photo are checked at any point inside the branches.

In order to gain access to a building, an extra secured area or an elevator, a valid access card has to be held in the vicinity of an **RFID** reader (the card can remain in the wallet). The access control system reads out a personalized ID-number that is stored in the first sector. A back-end check is performed with the **UID** and card number and access is either granted or denied, depending on the security clearance.

Cloning Cards for Access Control

Attacking the access control system described above is fairly straightforward with the Chameleon. Reading out the **UID** and other content of an access-card is trivially possible because the **UID** is sent in clear and the default keys of the underlying Mifare Classic card were not changed. Key A is hence set to 0xA0A1A2A3A4A5 for all sectors on all cards, whereas key B is unused and locked.

Analyzing a memory dump of an access card revealed that all blocks of all sectors are set to zero except for the blocks of the first sector. This sector contains the **UID** and the ID-number of the employee. Since all other sectors are set to zero, it is sufficient to read out the first sector of the card to be cloned. The attack time during which an attacker has to be in the vicinity of the targeted access-card is therefore reduced to a minimum. The attacker does not have to crack any sector keys and only has to read a single sector instead of the complete memory content. Such an attack is feasible within milliseconds and can be performed while passing a victim.

In order to verify, whether the **UID** is checked by the system, we wrote the dumped memory content of a genuine access card to a blank Mifare Classic 1K card. Naturally, the blank card has a different **UID** stored in its manufacturer block and is therefore not an exact clone (compare with Sect. 6.7.3). Trying to overcome the access control system with this clone did not succeed as shown in Fig. 5.4(a). This demonstrates that besides the personalized ID-number also the **UID** is checked in the back-end, as claimed by the vendor. However, with Chameleon we are able to emulate an arbitrary **UID** in addition to the personalized ID-number stored in memory. Accordingly, bypassing the access control system with Chameleon succeeds as shown in Fig. 5.4(b).

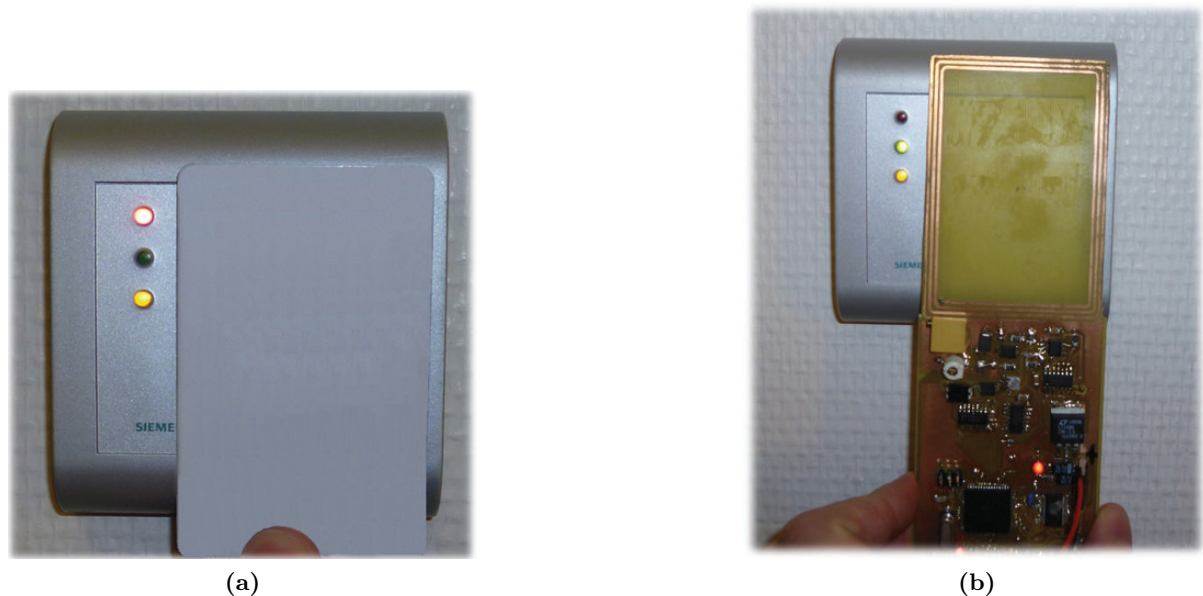


Figure 5.4: Tests with the Siemens AR6181-MS **RFID** reader installed in the access control system: A duplicate of an access-card based on a blank Mifare Classic 1K does not succeed in bypassing the access control system (left). Chameleon instantly opens the door (right).

5.7 Conclusion

A microcontroller-based, freely programmable emulator for ISO 14443 compliant **RFID**s has been developed that enables simulating the presence of various contactless smartcards at a very low cost (less than 25\$). The device works autonomously, operated from a battery, and its card-sized antenna fits into the slots of most readers for contactless smartcards. Due to its small dimensions, the emulator can be used covertly, e.g., hidden in the wallet, and is well-suited for real-world attacks. Our hardware can be connected to a **PC** by means of a **USB** interface and the non-volatile memory of the microcontroller allows amongst others to monitor the communication with an **RFID** reader and store the acquired data in order to reverse-engineer unknown protocols.

The protocol of Mifare DESfire EV1 cards has been exposed, the **DES**, **3DES** and **AES** block ciphers have been implemented, and the first successful emulation of Mifare DESfire and DESfire EV1 cards in the literature was presented. The current software further includes the emulation of Mifare Ultralight and Mifare Classic cards, based on a highly optimized variant of the Cryptol stream cipher for 8-bit μ Cs. The firmware of the “Chameleon” device is not limited to Mifare cards but can be adapted to support other contactless smartcards and their respective protocols, e.g., the electronic passport and cards from other manufacturers.

The card emulations have been tested with different **RFID** readers and it has been shown that the implementations of the ciphers and protocols meet the timing requirements of all protocols — the performance in most cases is even faster than that of original cards. In all tests, the emulator could not be distinguished from a genuine card.

Since secret keys of Mifare Classic cards and Mifare DESfire MF3ICD40 cards can be extracted by means of mathematical cryptanalysis and side-channel analysis, respectively, the emulator poses a severe threat for many commercial applications, if it was used by a criminal. To demonstrate the capabilities of our findings we performed several real-world attacks, amongst others on a contactless payment system. We emulated exact clones (including the **UID**) of Mifare cards, successfully spoofed an access control system, and carried out payments with the Chameleon. The Chameleon has thus proven to be a valuable tool for the security analysis of contactless technology and can be used to practically identify security weaknesses of real-world **RFID** systems. All details of the device have been published on the Internet as a **GPL** project [KvMOP11].

Altogether, considering the various possible attacks sketched in this chapter, the Chameleon sets a new lower limit for the cost and skills required to spoof systems relying on contactless smartcards, such as NXP's Mifare portfolio, in the real-world, and illustrates that many attacks can become profitable for a criminal. With contactless payment, ticketing, and access-control systems being omnipresent today, it is crucial to realize that only strong cryptography, together with sound protocol design and protection against implementation attacks, can ensure long-term security. Bug-fixes for cryptographically broken systems, that are based on false assumptions on certain card characteristics, e.g., **UID**-based protection schemes for Mifare Classic, are a fatal design choice: this chapter demonstrates (amongst others) that exact cloning of cards is feasible at a very low cost.

Chapter 6

Pitfalls of a Real-World Contactless Payment System

DESPITE the publicized vulnerabilities of the Mifare Classic technology, a multitude of applications worldwide employs tens of millions of these contactless smartcards. Recent security analyses of real-world systems demonstrate that many system operators have not implemented measures for improving the end-to-end security of their system to a tolerable level. Rather, weakly configured system parameters, e.g., using default keys set by the manufacturer, key re-use, or lack of consistency checks in the back-end, often ease attacks.

This chapter, evolved from a joint paper with Michael Silbermann at Financial Cryptography 2010 [KSP10], gives a comprehensive summary of the relevant cryptanalytic research and security vulnerabilities related to Mifare Classic. Then, as a main contribution, a real-world contactless payment system based on these cards is analyzed. In order to tackle its security, previous cryptanalytical results are combined and an improved card-only attack is implemented using the customized low-cost tools introduced in Chap. 4. To our knowledge this is the most efficient practical attack to date.

In a subsequent case study the effectiveness of the developed attack is studied in the real-world by an in-depth analysis of the contactless payment system. Various security risks are revealed and precarious vulnerabilities are pinpointed, that are likely to be present in other **RFID**-based systems. The presented devastating, practical attacks include card cloning, recharging, identity theft, and even converting spurious, digital money into real cash. A simple model is introduced for assessing the (financial) implications from the viewpoints of the system operator, users, and attackers. Economic aspects are addressed, with the aim to quantify the losses that may occur for the system operator, and it is investigated whether conducting the attacks in practice pays off for a criminal, taking her effort and the risk to be detected into account.

Finally, an ensemble of best-practice countermeasures for raising the overall security to a sufficient level is illustrated, that may serve as guidelines for integrators to design more secure systems in the future. The improvements aim at minimizing losses due to fraud, with only small effort and cost.

6.1 Introduction

A growing number of payment systems incorporate contactless technology, as it offers additional benefits in terms of flexibility and convenience over its contact-based counterpart. Contactless payment transactions require no physical connection between the card and the reader device at the checkout. Instead of swiping or inserting a card, as it is the case with magnetic stripe cards

and contact-based smartcards, the contactless smartcard is tapped on or held near a contactless reader device at the cash register, resulting in wireless debit or credit of the card balance. The wireless technology implies new threats compared with contact-based systems, for instance, a card could be read out from the pocket or wallet without the owner taking note of it. With the recent trend towards issuing contactless smartcards in large companies, universities and government entities, a number of privacy- and security-related concerns have been raised.

One well-known representative of contactless smartcards is Mifare Classic. These cards have become very widespread in the past two decades. Over the course of the last three years, several vulnerabilities of their security have been discovered. Still, many applications worldwide rely on this technology, often because the cost of upgrading a legacy system to more secure solutions is perceived as too high. Meanwhile, analyses of several real-world systems have revealed the alarming fact that the majority of system integrators have not taken steps to mitigate the weaknesses of the Mifare Classic cards. Adding insult to injury, incorrect system designs often facilitate attacks that eventually have a dramatic impact with respect to privacy, security and financial loss [KSP10].

6.1.1 ID-Card

One instance of such a flawed commercial application, termed ID-card¹, is analyzed in this chapter. According to the manufacturer it is used by more than a million customers in Germany, e.g., employees of large enterprises with typical user populations of 10,000 or more. The multi-purpose electronic ID and payment card is based on a dual-interface smartcard, i.e., both contact-based and contactless interfaces are provided. Besides the use for cashless payments, e.g., for food, copy and printing services, or paying for using a washing machine, the contactless technology handles the access control to buildings, offices, and apartments of employees. We examine the security of the ID-card as a case-study of a widespread payment system and intend to answer the question: how secure are electronic payment systems relying on today's wireless technology really? We thereby focus on an instantiation of the ID-card e-cash system that is operational since 2006 at a large enterprise in Germany.

Before the contactless cards can be used in the system, the issuing organization initializes each card to an individual default state. There exist personalized ID-cards for the employees of the company, but also anonymized ID-cards for externals which can be obtained for a deposit of 10 € at any cash register. The ID-card can be charged with a maximum credit amount of 150 € at dedicated terminals, or directly at a cash register, and allows for fast, wireless payments without the need to give change. The staff encourages the customers to hold their wallets or even their bags close to the reader, which interacts with the card wirelessly and carries out the payment.

In addition to contactless payments, most cash registers in the ID-card system still accept cash payments. According to the issuing institution, since their introduction in 2006, the amount of payments with contactless cards has been above 80 %, with an increasing tendency. Contactless payments were instantly accepted by the employees and have become the most important payment method in the enterprise. In the investigated instantiation of the payment system more than a million customers cards have been issued, spread over a few dozen branches.

¹To prevent damage for the maker of the contactless system, the name has been altered

6.1.2 Contribution and Outline of this Chapter

The weaknesses of the investigated payment system that are exposed in the following and the lack of appropriate usage of cryptography make the ID-Card an excellent educational example to demonstrate which mistakes are actually made in practice. At the same time, the fully as-sailable system provides a well-suited starting point to illustrate how best-practice approaches employing cryptography can help to secure the system. Hence, after security-analyzing and attacking it, we investigate the effect of various technical and non-technical measures for improving the overall security of the ID-card system. We assess if and which practically applicable countermeasures exist for transforming a previously totally insecure implementation to a level that provides sufficient security for most real-world contactless applications. Our findings may help system integrators to judge the security of their own applications and further enable them to fix potential security bugs.

This chapter is structured as follows: Contactless payment systems and their building components are illustrated in Sect. 6.2. Technical details of Mifare Classic cards and the Crypto1 stream cipher used by them are introduced in Sect. 6.3. An extensive survey of the cryptanalysis of Mifare Classic and published attacks on contactless payment systems in Sect. 6.4 completes the fundamentals required for the following practical security analyses. In Sect. 6.5, a highly efficient practical key-recovery attack on Mifare Classic is implemented on the custom-built **RFID** tool (see Chap. 4). The software for the embedded system presented in Sect. 6.5.1 automatically extracts the secret keys in seconds and enables comfortably reading and writing of ID-cards in the field. During the subsequent in-depth analysis of the contactless payment system in Sect. 6.6 severe flaws are highlighted and a variety of real-world attacks are conducted. The possible dramatic implications of the attacks are illustrated in Sect. 6.7. They can be counteracted by means of the security improvements and countermeasures proposed in Sect. 6.8. The countermeasures can be realized with little cost and effort, impede all threats of practical relevance and thus hinder fraud and financial damage for the system operators. They provide guidelines and code of practice for mitigating the security risks of similar systems based on cryptographically weak **RFID** devices.

6.2 Contactless Payment Systems

A typical contactless payment system consists of **RFID** readers at the points of sale and contactless smartcards in the field, that operate as electronic wallets and supposedly safely store the credit balance. The corresponding back-end system contains a database for all cards and can communicate with the points of sale. At each cash register, an ISO 14443-compliant reader initiates the communication once a payment card gets into its vicinity (see Sect. 2.5 for details about the communication).

6.2.1 Components

The main components of the contactless payment system can be categorized in a front-end and a back-end. The *front-end* consists of (i) cash registers that are based on standard desktop computers equipped with a reader for contactless smartcards, (ii) self-service charging terminals

where the credit balance of the ID-cards can be increased by means of cash or credit cards², and (iii) the ID-cards that autonomously store the credit balance.

A database in the *back-end* collects and retains the payment information, e.g., the balance of each cash register and the daily turnover. Optionally, “shadow accounts” for each ID-card may contain the credit that should be stored on it according to the registered incoming payments and outpayments for each particular card. They could be used for detecting functional errors in the system or fraud. Additionally, it would be possible to blacklist cards, for example, to trigger an alarm and confiscate a card on the next payment, if a conspicuous inconsistency is detected between the content of the shadow account and the actual credit value on the card.

6.2.2 Off-Line vs. On-Line Systems

The basic idea of using contactless smartcards instead of simpler memory-only **RFID** transponders or a barcode system is that they store the credit balance secured by means of cryptography, so that the payment function can be performed only by the card, without additional back-end components. A payment is technically carried out by lowering the value of the balance digitally stored on a card. This enables off-line systems, in which the cash registers and charging terminals usually work autonomously without an on-line connection to the database in the back-end. Their data, such as revoked cards or the balance of the cash register, is synchronized only infrequently, e.g., once per day. In an off-line system inconsistencies of the shadow accounts during the daytime are thus normal. An example: assume an empty card that is charged at an off-line terminal just before a payment is carried out. The corresponding shadow account in the back-end is not updated, i.e., has a balance of 0€, or might even not be created yet. However, the customer can at once pay at any cash register, which in consequence cannot directly and autonomously detect fraud. In general the shadow accounts are consistent with the credit value of the card after each synchronization at the end of the day. Hence, fraud can be detected in the back-end and blacklists can be effective at the latest when a manipulated card is used for the next time after a synchronization. This off-line scenario also applies to the investigated ID-card payment system.

On-line systems on the contrary rely on a continuous connection to the back-end. Any inconsistency in the balance of a card can be instantly detected by the database of the back-end, hence these systems should be preferred from the security point-of-view. However, on-line systems are usually not an option for various reasons, to name the reliability and availability of the system, cost, speed of the transactions, but also portable devices accepting contactless payments, e.g., used in the public transport, without a connection to the back-end.

6.3 Mifare Classic Cards

The low-cost Mifare Classic [NXP08a] cards are the most widely spread product out of NXP’s range of contactless smartcards (compare with Sect. 2.6.4). The Mifare Classic chip was introduced in 1995 by Philips and is today the most widespread type of contactless smartcards. Allegedly, more than one billion Mifare **ICs** and ten million reader components have been sold [NXP10b] and are now used in many security and privacy relevant applications worldwide, covering more than 70% of the contactless smartcard market. The cards provide data

²Note, that the cards can also be charged at any cash register.

encryption of their memory content based on the proprietary stream cipher *Crypto1* in order to protect sensitive data and prevent from security risks like eavesdropping, replay attacks, and unauthorized reading or modifying of the data stored on the card.

6.3.1 Memory Structure

Mifare Classic can be regarded as a memory card providing a cryptographic protection: The data is stored in an internal **EEPROM** with an integrated digital control unit. The latter handles the communication with a reader and can secure the transmitted information with cryptography. Three variants of Mifare Classic exist: Mifare Classic 1K with 1024 byte **EEPROM**, the Mifare Classic 4K with 4096 byte **EEPROM** and the Mifare Mini with 320 byte **EEPROM**. In the following we focus on the most widely used 1K version.

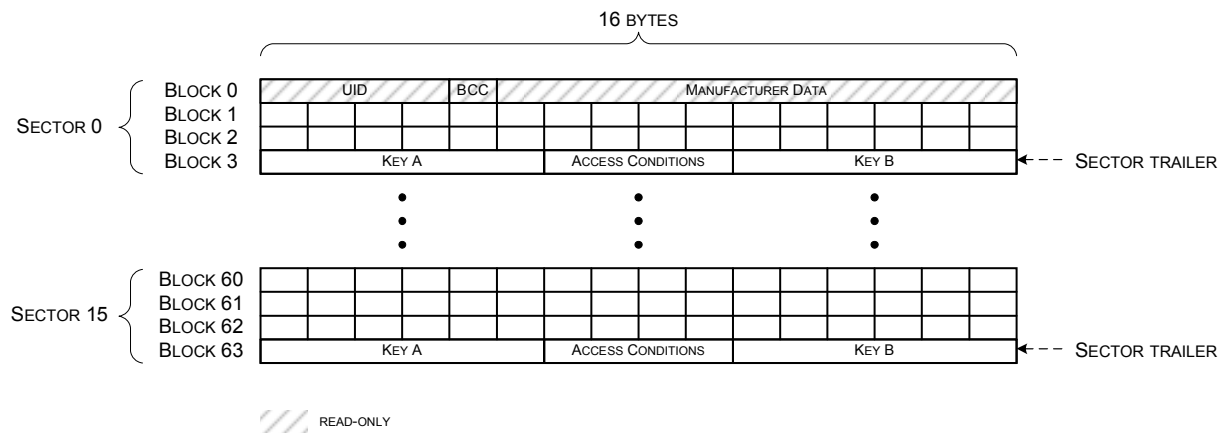


Figure 6.1: Memory structure of a Mifare Classic 1K card.

The memory of a Mifare Classic 1K chip is partitioned into 16 sectors, each consisting of four blocks, as illustrated in Fig. 6.1. The read-only block 0 of the first sector contains the factory-programmed **UID** of the card and other fixed data of the manufacturer. The fourth block of each sector, termed sector trailer, contains two 48-bit keys used for the authentication to that sector and a set of bits defining the access conditions for each individual block. All other memory blocks of the card can be used to store user data according to the respective access conditions.

Prior to executing any read or write operations, the reader must perform an authentication with one of the sector keys (A or B). The authentication is only valid for one sector at a time. Once the reader wants to access blocks of another sector, it has to repeat the authentication process with the appropriate sector key, as detailed in Sect. 6.3.3. The default key A or B on a Mifare Classic card produced by NXP or Philips is `0xFF FF FF FF FF FF`, whereas cards produced by Infineon³ use `0xA0 A1 A2 A3 A4 A5` as the default key A and `0xB0 B1 B2 B3 B4 B5` as the default key B.

Two types of blocks are supported by the Mifare Classic system: data blocks and value blocks. Value blocks are especially well suited for payment applications. While normal data blocks can

³Infineon Technologies has licensed the Mifare technology and is also producing Mifare Classic cards.

be written with arbitrary data, value blocks need to be conform to a special format: each value block contains a 4-byte unsigned value which is stored once inverted and twice non-inverted. NXP further proposes that the last 4-byte segment of the block may be used to store a 1-byte address pointing to a block, for implementing a backup management. Such an address pointer is stored four times: twice inverted and twice non-inverted.

Access Conditions

The sector trailer of each sector contains the access conditions for each individual block of that sector. The access conditions determine the commands that are allowed for each block of the sector, e.g., read, write, increment or decrement, and define the role of the keys. For example, key A can be configured to enable only reading of the block, while key B may give a permission to write. Three bits reserved for each block are stored redundantly once inverted and once non-inverted in bytes 6, 7 and 8 of each sector trailer, making a total of $3 \text{ bit} \cdot 4 \cdot 2 = 24 \text{ bit}$ being used for setting the access conditions on the card⁴. The coding of the access condition bits defines which operations are allowed for that particular block and with which key (A or B) the preceding authentication has to be performed. Each data block can be configured to adopt one of the four states [Tee08]

fluid the block can be arbitrarily modified,

restricted data in a pre-specified data format (i.e., value blocks) can be written,

frozen the block is read-only, or

dead the block cannot be accessed at all.

The sector trailer is treated separately and can be in one of the states

fluid the access condition bits and the keys can be modified,

restricted the access condition bits cannot be changed, but the keys can still be overwritten,
or

frozen both access condition bits and the keys are in an unmodifiable state.

For each block and each of the four operations, namely read, write, increment and decrement, the access can be granted or denied separately. More details on the coding of the access condition bits can be found in [NXP08a].

6.3.2 Command Set

The Mifare Classic high-level transmission protocol diverges from Part 4 of ISO 14443. It consists of a proprietary Mifare command set that has been reverse-engineered in the course of this work.

The commands for the side of the reader are summarized in Table 6.1. Note that all higher-level commands in standard frame format are appended with two CRC bytes according to ISO/IEC 14443 A for data integrity.

The coding of the three short-frame card responses is as follows:

⁴The access conditions are stored twice (inverted and non-inverted) for each of the 2^4 blocks of the card.

Table 6.1: Proprietary command set of a Mifare Classic reader.

Type	#	Command	Description
AUTHENTICATION	1	0x60 0xXX	Authenticate for block 0xXX with key A
	2	0x61 0xXX	Authenticate for block 0xXX with key B
DATA BLOCKS	3	0x30 0xXX	Read block 0xXX
	4	0xA0 0xXX	Write block 0xXX
VALUE BLOCKS	5	0xC0 0xXX	Decrement value block 0xXX
	6	0xC1 0xXX	Increment value block 0xXX
	7	0xC2 0xXX	Restore value block 0xXX
	8	0xB0 0xXX	Transfer to value block 0xXX

- Acknowledged: ACK = 0x0A
- Not acknowledged (not allowed): NACK = 0x04
- Not acknowledged (transmission error): NACK = 0x05

The authentication process with the associated commands is discussed below in Section 6.3.3. After a successful authentication, reading a data or value block is accomplished by sending the read block command followed by a block number to be accessed. The card replies with 16 data bytes or Negative Acknowledgment (**NACK**). In order to write a block the reader issues the write command, waits for the response of the card (Acknowledgement (**ACK**) or **NACK**), sends 16 data bytes to be stored and, again, waits on the response of the card (**ACK** or **NACK**). When writing a value block, the data format is checked by the card and, if it does not comply to the expected format of a value block, the protocol aborts.

Four commands are available for value blocks only: decrement, increment, restore and transfer. To increment or decrement a value stored in the value block, the reader first issues the increment or decrement command, waits for the card response (**ACK** or **NACK**), sends the 4-byte unsigned integer value to increment or decrement by, and finally receives **ACK** or **NACK** from the card. Increment and decrement operations load the value stored in the value block to a temporary register inside the integrated circuit of the card, where all value operations are performed. In order to write the processed result back to the **EEPROM**, the transfer command must be issued after all value operations are completed. The restore command on the contrary discards all previously performed operations, i.e., preserves the original value of the value block.

6.3.3 The CRYPTO1 Cipher and Communication Protocol

All members of the Mifare family comply to Parts 1-3 of ISO/IEC 14443A [Int01]. While ISO 14443 also specifies higher data rates, Mifare Classic cards communicate at a fixed rate of 106 kBit/s and use a higher-level communication protocol that diverges from Part 4 of the standard. This proprietary protocol for authentication and subsequent data encryption is a security measure which was meant to prevent from threats such as replay attacks, cloning, and eavesdropping.

Cipher

The CRYPTO1 stream cipher, illustrated in Fig. 6.2, is based on a 48-bit **LFSR** with the feedback polynomial $x^{48} + x^{43} + x^{39} + x^{38} + x^{36} + x^{34} + x^{33} + x^{31} + x^{29} + x^{24} + x^{23} + x^{21} + x^{19} + x^{13} + x^9 + x^7 + x^6 + x^5 + 1$. Six non-linear filter functions for the feedback, of which there are only three different types f_a , f_b and f_c , are defined by:

$$\begin{aligned} f_a(x_0, x_1, x_2, x_3) &= ((x_0 \vee x_1) \oplus (x_0 \wedge x_3)) \oplus (x_2 \wedge ((x_0 \oplus x_1) \vee x_3)), \\ f_b(x_0, x_1, x_2, x_3) &= ((x_0 \wedge x_1) \vee x_2) \oplus ((x_0 \oplus x_1) \wedge (x_2 \vee x_3)), \\ f_c(x_0, x_1, x_2, x_3, x_4) &= (x_0 \vee ((x_1 \vee x_4) \wedge (x_3 \oplus x_4))) \oplus \\ &\quad ((x_0 \oplus (x_1 \wedge x_3)) \wedge ((x_2 \oplus x_3) \vee (x_1 \wedge x_4))), \end{aligned}$$

where $\vee$, $\wedge$, $\oplus$ denote logical OR, AND and XOR operations, respectively.

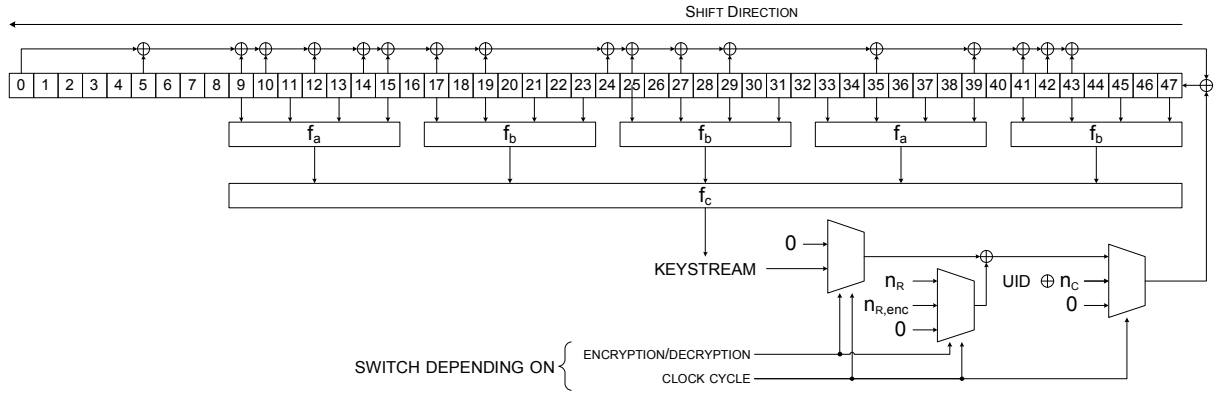


Figure 6.2: Top-level structure of the CRYPTO1 stream cipher. The functions f_a , f_b and f_c introduce some amount of non-linearity into the feedback loop of the 48-bit LFSR.

The cipher is outdated and has to be regarded as cryptographically very weak: even without further cryptanalysis it is possible to recover the small 48-bit state of the **LFSR** by means of an exhaustive search.

Pseudo-Random Number Generator

An integrated Pseudo-Random Number Generator (**PRNG**), clocked simultaneously with the cipher, is utilized to generate nonces for the authentication between reader and card. It is based on a 16-bit **LFSR** with feedback polynomial $x^{16} + x^{14} + x^{13} + x^{11} + 1$. The state of

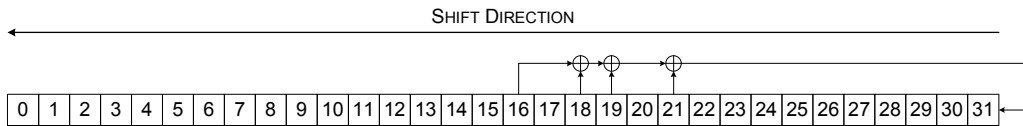


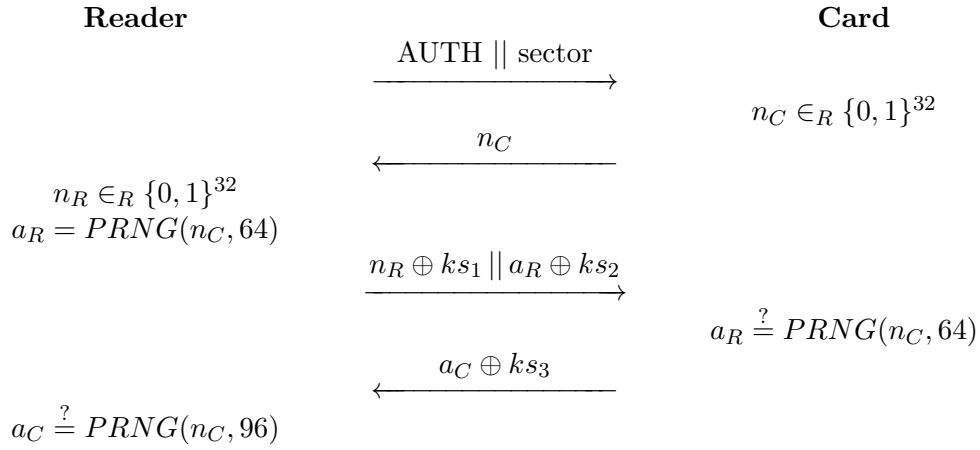
Figure 6.3: Structure of the internal Mifare Classic **PRNG**.

the **LFSR** is expanded to 4-byte values by shifting the output bits of the 16-bit **LFSR** into an

enlarged 32-bit shift-register, as illustrated in Fig. 6.3. Therefore, the resulting 32-bit pseudo-random numbers contain only 16 bit of entropy, i.e., only $2^{16} = 65536$ different numbers are generated. Our experiments confirm that the PRNG is clocked every $2^7 = 128$ clock cycles, synchronously to the carrier frequency of 13.56 MHz, hence the same number is output every $65536 \cdot 9.44 \mu s \approx 619$ ms.

Authentication Scheme and Operation of the Cipher

Protocol 2 illustrates the sequence of actions taken for the three-pass authentication between a reader and a Mifare Classic card, where $\parallel$ denotes a concatenation and AUTH is the authentication command.



Protocol 2. The Mifare Classic authentication protocol.

An example of an eavesdropped authentication between a reader (R) and a Mifare Classic card (C), presented in Tab. 6.2, gives the exact details of the communication.

After the initialization and anticollision part of the ISO 14443 protocol⁵ in Steps 1 to 6, the reader sends the command 0x60 or 0x61 for an authentication with key A or key B, respectively, followed by the number of the sector that it wants to access. The card replies with a random nonce n_C . The LFSR of the cipher is loaded with the respective secret key, and during the clock cycles $i = 0 \dots 31$ the value $UID \oplus n_C$ is clocked in, while no keystream is sent to the output. Starting with the message in Step 9, the communication is encrypted by XORing the bits of the plaintext with the keystream bits for $i > 31$, which are grouped into 32-bit words and denoted as ks_1 for $i = 32 \dots 63$, ks_2 for $i = 64 \dots 95$, ks_3 for $i = 96 \dots 127$, and so on. The reader generates a nonce n_R and concatenates it with its response to the challenge of the card a_R , which is obtained by loading n_C into the PRNG and clocking it 64 times. The card receives the encrypted nonce of the reader $n_{R,enc} = n_R \oplus ks_1$ and the (decrypted) n_R is fed into the cipher during the clock cycles $i = 32 \dots 63$. Note that, due to the feedback structure of the cipher, the later bits of $n_{R,enc}$ depend on the earlier bits of n_R . At this point the initialization

⁵The ISO 14443 commands REQA, ATQA, ... are explained in Sect. 2.5

Table 6.2: A typical authentication protocol run between a reader R and a Mifare Classic C.

#	Direction	Hex Bytes	Description
1	R → C	26	REQA
2	C → R	04 00	ATQA
3	R → C	93 20	ANTICOLLISION
4	C → R	74 82 F9 D7 D8	UID BCC
5	R → C	93 70 74 82 F9 D7 D8 BB AD	SELECT UID BCC CRC
6	C → R	08 B6 DD	SAK CRC
7	R → C	60 00 F5 7B	AUTH sector number CRC
8	C → R	CD 98 4F AE	n_C
9	R → C	73 09 C4 F8 10 2D 97 4E	$n_R \oplus \text{ks}_1 \parallel a_R \oplus \text{ks}_2$
10	C → R	B7 52 8C 02	$a_C \oplus \text{ks}_3$
...			
AUTHENTICATION ESTABLISHED, BLOCK READ/WRITE COMMANDS FOLLOW			
...			

is complete, and the **LFSR** autonomously produces one bit of keystream per clock cycle. The card finally replies with a_C , generated by the **PRNG** 96 clock cycles after n_C was output. The authentication is successful if the received values for a_C and a_R are identical to those generated by the internal **PRNGs**.

Cipher Rollback

If the internal state of the **LFSR** is known at any point in time during the operation of the cipher, it can be easily rolled back by computing the tapped bits in each clock cycle and shifting the **LFSR** in the opposite direction. This approach can be used to obtain the initial value loaded into the **LFSR**, which is the secret key. In addition to the known internal state, the nonces n_C and n_R that are input to the cipher during the first 64 clock cycles are required to perform a full rollback operation.

6.4 A Survey on Attacking Mifare Classic

Since its invention in 1995 by Philips [NXP95], all details of the Mifare Classic chip had been kept secret until 2007, when Nohl et al. [NESP08] succeeded in reverse-engineering the cipher. They manually removed layer by layer of the silicon chip by means of grinding paste, took pictures of each layer with a microscope, and deducted the structure of the cipher from the layout of the chip. The authors instantly discovered numerous weaknesses in CRYPTO1, as well as flaws in the **PRNG**. In the following, we summarize the state-of-the-art of the research on the security of Mifare Classic, which constitutes the basis for our key-recovery attack and the real-world system break presented in Sect. 6.6.

6.4.1 Keystream Recovery

The first discovered weakness [NESP08] is that the random nonce generated by the card is only dependent on the time elapsed between the power-up of the card and the issuing of the authentication command by the reader. Hence, by controlling the timing, the same nonce can be reproduced with a certain probability. The consequence of this weakness is the keystream recovery attack described by de Koning Gans et al. [dKGHG08]. Prerequisite is a recorded authentication session between a genuine reader and a Mifare Classic card. In the second part of the attack, several queries to the card are issued by a specially prepared reader. As for all stream ciphers, a part of the keystream can be recovered from a known pair of plaintext and ciphertext.

Exploiting the fact that key A of each sector is not readable and that trying to read it returns encrypted zeros as a response, the keystream for encrypting the first six bytes of each block of that sector can be recovered. Often, key B is not readable as well, which reveals another six bytes of the keystream. Furthermore, the first five bytes of the manufacturer data (see Fig. 6.1) are known, altogether allowing to recover all sixteen keystream bytes for Sector 0 and up to twelve keystream bytes for higher sectors. The attack does not enable card cloning, as the cryptographic keys remain secret. However, if an attacker is able to eavesdrop on the communication between card and reader, and can afterwards access the card long enough for the second part of the attack, she can read or modify the whole Sector 0 and partially the higher sectors. Amongst others, the authors were able to read out Sector 0 of the OV-Chipkaart, used in the public transit system in the Netherlands, with their attack.

6.4.2 Key Recovery from Genuine Authentications

The possibility to recover the keystream led to an attack proposed by Garcia et al. [GdKGM⁺08], where one can extract the secret key from just two eavesdropped authentications between a card and a genuine reader. The goal is to retrieve the state of the LFSR at some point in time. Once its state is recovered, the cipher can be rolled back to obtain its initial value which is the secret key of a sector (cf. Sect. 6.3.3). The secret key can then be used to decrypt the previously recorded messages.

First, the part of the keystream ks_2 used to encrypt a_R is computed from a recorded authentication session. The filter functions f_a , f_b , and f_c of CRYPTO1 are inverted to obtain 65536 key candidates for the state of the LFSR, of which the correct one is obtained off-line on a PC by rolling back the cipher (cf. Sect. 6.3.3) and checking it against a second authentication. No precomputation is required and, after recording the two authentication sessions, it takes only 0.1 s to recover the secret key of one sector using standard PCs, as the authors claim. Note that eavesdropping can be nontrivial in a real-world scenario.

This attack was the first to exploit another weakness of Mifare Classic cards concerning the parity bits: According to ISO 14443 [Int01], each transmitted byte has to be appended with a parity bit in order to detect transmission errors. However, the parity bits in the Mifare Classic system are computed from the plaintext instead of the actual bits transmitted. As shown in Fig. 6.4, the parity bit of byte N of the plaintext is furthermore encrypted with the same keystream bit KS_j that is used to encrypt the first bit P_j of the next byte $N + 1$. This vulnerability contradicts an important paradigm for stream ciphers stating that the keystream bits should never be reused to encrypt multiple plaintext bits. Taking these weaknesses into

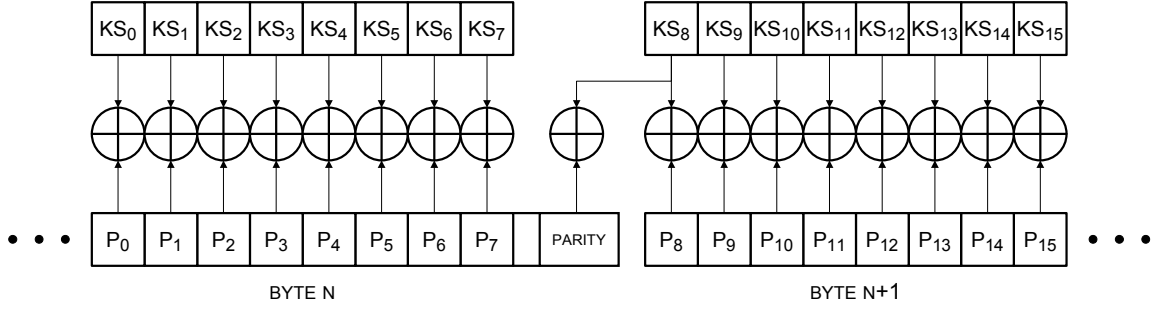


Figure 6.4: Principle of the encryption mechanism of Mifare Classic. For generating the ciphertext, each bit P_i of the plaintext is XORed with a new bit KS_i of the keystream produced by the CRYPTO1 cipher. The parity bit computed from the plaintext is encrypted by using a bit of the keystream twice (KS_8 in the figure).

account, the authors are able to recover the secret keys from the OV-Chipkaart and the London Oyster Card by recording genuine authentication sessions. They propose to use additional system-level security measures of Mifare Classic, such as blocks configured as counters that decrement with each transaction, or authentications to random sectors on the card for verifying its genuineness.

6.4.3 Card-Only Key Recovery

After the previous findings, the first card-only attacks emerged that do not require to eavesdrop on a communication with a genuine reader and are hence more practical in the real world. An attacker only needs to be close enough to the targeted Mifare Classic card to activate it and communicate with it by means of a special-purpose reader, in order to retrieve the state of the **LFSR** and thus recover a secret key.

Garcia et al. [GvRVS09] propose four different attacks to obtain the secret key of one sector. Their first proposal is an off-line brute-force attack on the entire 48-bit key space of the cipher, after carrying out 1500 authentication attempts. For their second attack, the authors need to keep n_C constant and vary n_R to gain some knowledge about the internal cipher state. On average, 28500 queries to the card with a fixed card nonce are required, which takes about 15 min. The respective off-line key search takes approx. 1 min on a standard PC. To perform the third attack, an attacker needs to precompute a table with a size of 384 GB. Then, she keeps n_R constant and varies n_C , ultimately revealing a special internal cipher state. The required 4096 authentication attempts on average can be obtained in about two minutes.

Nested Authentication If one sector key is known, either because the system integrator did not change the default key⁶ for Sector 0 or a key recovery has already been performed for one sector of the card, the fourth attack termed “nested authentication attack” can be performed. First, an authentication to the exploit sector is decrypted to obtain the corresponding n_C . The subsequent n_C that is used for authenticating to another sector can be guessed due to the weak **PRNG**. Since the new n_C is encrypted with the key of the new sector, an attacker

⁶This scenario is often reality in commercial systems.

instantly recovers 32 bits of keystream. This attack is very efficient, as it requires just three authentication attempts to obtain 2^{16} key candidates and to identify the correct one in under a second on a standard computer.

In addition to fixing n_C and exploiting the parity weakness described above, the authors make use of the fact that in Step 10 of the protocol (see Tab. 6.2) the card sometimes replies with four bits instead of four bytes. This happens if and only if the parity bits sent along with the eight byte reader cryptogram $n_R||a_R$ are correct and a_R is incorrect, i.e., not accepted by the card. The four bits sent in this case are an encrypted **NACK** = 0x5 command. This knowledge enables to recover four bits of the keystream for each failed authentication attempt with correct parity bits, occurring with a probability of $1/256$. This corresponds to a recovery speed of (on average) 64 authentications per keystream bit.

Differential Attack The latest and most efficient attack is proposed by Courtois [Cou09]. It requires only 300 queries to the card on average and a few seconds of off-line computation to find the sector key. In addition to the weaknesses described above, Courtois exploits a mathematical vulnerability, i.e., a bias of the filter functions of Crypto1, to mount a differential attack on the cipher: Altering only the last three bits of n_R in each authentication attempt, eight encrypted **NACK** replies from the card are obtained, giving $4 \cdot 8 = 32$ bit of the keystream. Since this attack is the basis for the key extraction implemented in Sect. 6.5, the exact steps of the approach are detailed in the following.

- (1) After receiving n_C the correct combination for the eight bytes of the randomly chosen reader cryptogram $n_{R,enc}||a_{R,enc}$, that is kept fixed in each iteration, needs to be found. The eight parity bits that are variable in each iteration are modified until the card replies with four bits (encrypted **NACK**), indicating that the parity bits were correct. In case the parity bits are not correct the card does not reply at all. For this step on average $\frac{2^8}{2} = 128$ queries to the card are required.
- (2) Then, the first 29 bits of $n_{R,enc}$ and the first three parity bits from Step 1 are fixed. Additionally the card nonce n_C from Step 1 must be fixed (through precise timing control). Only the last three bits of $n_{R,enc}$ ($2^3 = 8$ cases) are altered, while $a_{R,enc}$ and the last five parity bits are chosen randomly in each authentication attempt. On average, every $\frac{2^5}{2} = 16$ queries the card replies with an encrypted **NACK**, thereby revealing 4 bit of keystream. This process is repeated eight times so that finally eight encrypted **NACK** replies from the card are obtained, giving $4 \cdot 8 = 32$ bit of keystream.
- (3) Next, candidates for the state of the **LFSR** prior to the encryption of the **NACK** are computed. The four keystream bits from the first authentication attempt are re-used to reduce the key space by a factor of $2^4 = 16$. Then, for each candidate the corresponding states for the other seven authentications are predicted and compared with the received keystream bits. A match is found, if and only if the corresponding output of the filter functions, ks_1 , does not depend on the last three bits of n_R . Due to weaknesses of the boolean functions f_b and f_c (cf. Section 6.3.3), the output keystream does not depend on the last three bits of the **LFSR** state with a probability of approx. 75%, which also equals the success rate of this attack. If a contradiction is found at any stage, the attack has to be repeated starting from Step 1 with a new cryptogram.

The offline computations for the attack are negligible, as on average only $2^{48-32} = 2^{16}$ key candidates need to be tested for recovering a secret key. Courtois claims to have verified this attack on a number of access control and public transit systems from different countries. Furthermore, the author points out that some cards deployed in practice are even weaker in terms of the time complexity of the attack. For instance, the public transit cards used in Kiev, Ukraine, are based on unlicensed Mifare Classic clones manufactured by the China-based company Fudan Microelectronics. These cards reply with **NACK** even when the parity bits for the reader cryptogram are incorrect, reducing the average number of card queries to 16.

The authors of all the above summarized papers report difficulties in fixing the nonce of the card n_C in practice, which is crucial for the attacks. In contrast, the setup detailed in Sect. 6.5 enables to fix the nonce to one value.

Other Related Work Countermeasures to thwart a state restoration attack (see Sect. 6.7.4) and strategies to conserve the integrity of the data stored on the card are proposed by Teepe [Tee08]: the basic idea is to establish a key infrastructure at the back-end in order to tie the content of each card to a decrementing counter or a **UID** by means of a cryptographic function, e.g., a digital signature algorithm.

Siekerman et al. [SvdS08] provide a system-level overview of the disposable OV-Chipkaart tickets that are based on Mifare Ultralight chips. The authors claim that, although Mifare Ultralight chips do not offer any internal cryptographic mechanisms, there exists a cryptographic binding between the **UID** of the card and the card content that prevents cloning of the cards. This feature, however, does not protect from state restoration attacks, i.e., resetting the content of the card to a previously recorded state. The authors discovered through a series of experiments a severe vulnerability due to the poor implementation on the system level that allows a criminal to travel free-of-charge. This study shows, amongst others, that the back-end implementation often plays a key role for the overall security level of the system.

6.5 Attack Hardware and Software

For the security analyses and practical attacks in the field the freely programmable **RFID** reader developed in Sect. 4.1 is used. The setup for a key-recovery is illustrated in Fig. 6.5. In contrast to commercially available products, the customized **RFID** reader enables to fully control the communication and to inject faults by manipulating the **RF** field with a high timing accuracy of approx. 75 ns, which is a key advantage in the context of key-recovery from Mifare Classic cards. Furthermore, the card emulator “Chameleon” introduced in Chap. 5 is employed in this chapter (amongst others) for a full emulation of Mifare Classic 1K cards, in particular for cloning and emulating ID-Cards with an *arbitrary* **UID**.

Various analyses and attacks targeting the ID-Card and the respective payment system are conducted with the **RFID** reader and the Chameleon in Sect. 6.6. During our real-world security tests in the field, the battery-operated stand-alone modes of the devices, i.e., operation without the need of a laptop, turned out to be advantageous.

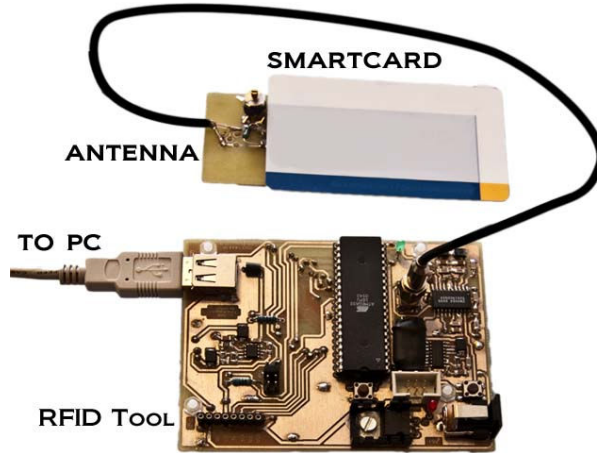


Figure 6.5: Setup for card-only analyses, consisting of our customized reader with a standard antenna and the ID-Card under attack on top of its antenna. The **USB** connection to a **PC** is not required when conducting key-recovery attacks in the field.

6.5.1 Software for Security Analyses of Payment Cards

We extended the existing software for the reader described in Sect. 4.1.3 with the full Mifare Classic authentication protocol described in Sect. 6.3, to support the corresponding data structure and to allow comfortable reading, writing, and cloning of Mifare Classic cards. We furthermore implemented an automatic key-recovery, which is subdivided into two parts corresponding to the two steps of the attack: during an on-line step, the embedded part of the software runs on the μC of the **RFID** reader and emulates a Mifare Classic reader collecting the data needed for the attack. This can be done in the field by powering the reader from a battery. The data collected in the on-line step is at first stored in the internal **EEPROM** of the reader and afterwards sent via **USB** interface to a desktop **PC** where the software computes the sector keys off-line. Once recovered, the revealed keys are programmed into the non-volatile memory of the reader which is then able to read or modify ID-cards in the field.

For the key-recovery it is crucial to exploit the weakness of the **PRNG**, as detailed in Sect. 6.3.3, which generates the same numbers if the time interval between power up and issuing the authentication command is fixed. During a normal protocol run, the response times of a Mifare card vary. Hence, using commercial readers, the same card nonce n_C reoccurs only with a relatively low probability during subsequent authentications. On the contrary, by actively manipulating the energy supply of the card with the customized reader, i.e., by injecting faults via the **RF** field, the nonces output by the **PRNG** are limited to one constant value. In order to precisely fix the timing independently of the behavior of the card, the μC of the reader is programmed to require always the same fixed multiple of 75 ns between the power-up of the card and the authentication command. As a result of the fault injections, we can force the card to generate exactly the same n_C in every attempt, which highly improves the efficiency of the key-recovery attack.

The software framework developed for the security analysis of Mifare cards consists of two main parts: embedded software for the μC and desktop software running on the host PC. Both were used for the subsequent security analysis of the system, and are therefore presented next.

Embedded Software

This section sketches the embedded part of the C software framework for manipulating Mifare Classic cards. All details about the involved functions are given in Appendix G.3.

Cipher and Protocol The embedded part of the software runs on the Atmel ATmega32 μC on the reader. On the basis of the operating system and the low-level functions for accessing the ISO 14443 interface described in Sect. 4.1.3, an API for manipulating Mifare Classic card has been developed. Its core is a software implementation of the Crypto1 stream cipher (see Sect. 6.3.3) used for the authentication and data encryption in Mifare Classic smartcards. The code is based on an open-source project [Cra08] and has been adapted and optimized for the 8-bit runtime environment of the μC . The resulting “light-weight” version of the referenced implementation includes functions for rolling back the cipher (see Sect. 6.3.3) and an implementation of the PRNG.

Further functions for realizing the proprietary protocol enable the authentication to a sector, as well as reading and writing the memory of a Mifare Classic card. All commands of Mifare Classic, as detailed in Sect. 6.3.2, are implemented for both normal memory blocks and value blocks. The advanced modes detailed in the following are compatible to the operating system in Sect. 4.1.3. The embedded code operates in conjunction with the developed desktop command-line tool described in Sect. 6.5.1.

Key Recovery Mode For recovering all sector keys of a Mifare Classic, the

Command (hex : ascii) = 0x66 : 'f'

initiates the combined implementation of the differential attack as proposed by Courtois [Cou09] and the nested authentication attack by Garcia et al. [GvRVS09].

In the first step of the attack the secret key of Sector 0 is recovered using the *differential attack* by issuing multiple authentication requests to the card. In each authentication attempt, the concatenation $n_{R,enc}||a_{R,enc}$ is sent to the card. In stage one (cf. Sect. 6.4.3), the values in $n_{R,enc}||a_{R,enc}$ are randomly generated and each of the eight associated parity bits has a fixed value of 1. In stage two, the 29-bit prefix of $n_{R,enc}$ as well as the first three parity bits are kept fixed, while iterating over all eight cases for the three last bits of $n_{R,enc}$. Four bytes of $a_{R,enc}$ and the five last parity bits are chosen at random.

In order to reliably reset the PRNG and thereby fix the nonce of the card, the EM field is completely switched off in between the authentications for at least 40 ms. The minimal power-off time has been experimentally ascertained — smaller time windows do not allow to fix n_C to one value. This defines a new practical lower bound for the time required for a key-recovery attack and puts some theoretical estimations in the literature into question. For controlling the timing during the authentications, in addition to the fixed running time of the code, a function for creating a delay in the order of microseconds is utilized. The timing of issuing the authentication command is tweaked in order to fit the time grid of the PRNG and hence force it to reproduce the same card nonce n_C in every authentication. The nonces generated by the

PRNG of the card are output via **USB** and can be observed for debugging purposes on a **PC** to adjust the delay accordingly in case of a change in the hardware setup.

Once the trace data containing eight encrypted 4-bit **NACK** replies from the card is collected, as required for the differential attack, it is sent to the desktop software via **USB** for further processing (cf. Sect. 6.5.1). As the attack succeeds only with a certain probability (approx. 75%) the μC waits for the response of the **PC** indicating whether the attack has been successful and the sector key has been found or whether another set of trace data must be collected from the card. If the attack is not successful, the process of collecting data is restarted for Sector 0. In case of a success, the revealed key is saved to **RAM** and the remaining 15 keys are computed with the nested authentication attack as follows.

The second step of the attack is the *nested authentication* that starts with the initial authentication to the so-called exploit sector, i.e., the sector for which the key has been cracked by the differential attack in the first step. The (fixed) nonce of the card is decrypted with the known key of the exploit sector. The following authentication to the target sector delivers a card nonce that is encrypted with the respective sector key of the target sector. Since the **PRNG** of the card keeps running in a deterministic fashion throughout the power-on period, the plaintext of this nonce can be predicted. Hence, 32 bit of keystream are obtained (cf. Sect. 6.4.3) that are used to compute on average $2^{48-32} = 2^{16}$ candidates for the internal state of the **LFSR**. The process of repeatedly authenticating to the exploit sector and to the target sector is repeated 16 times to obtain $4 \cdot 16 = 64$ bit of information about the key from the parity bits transmitted with the card nonces. Using this information, the correct key is finally computed offline from the 2^{16} candidates.

Dump Card Mode The complete memory of a Mifare Classic 1K card is read out (“dumped”) with the

Command (hex) = 0x00,

for example, to produce clones of an ID-card. The corresponding secret keys A and B of the card need to be stored in the Flash memory of the μC beforehand. The desktop software can be used to save the dump (including the secret keys) into a file on the **PC**.

Write Card Mode A previously dumped content of a card can be written to a blank Mifare Classic card with the

Command (hex) = 0x01.

The target card does not necessarily need to be blank, it is however required that the access conditions stored in each sector trailer permit overwriting of the existing data/value blocks. To enable re-writing to the newly created card in the future, the access condition bits have to be modified to FF 07 80.

Increase & Decrease Balance of ID-card The

Command (hex) = 0x02 (increase), 0x03 (decrease)

is used to increase or decrease the credit value blocks stored in Sector 1 of an ID-Card. The unsigned 32-bit integer value by which the balance is to be increased or decreased is received via **USB**, e.g., from the command-line tool (see Section 6.5.1).

Stand-alone ID-Card Manipulation The**Command (hex) = 0x04**

triggers a stand-alone, battery-powered operation of the reader. An infinite loop is implemented which wirelessly sets the credit balance of any ID-Card in the vicinity of the reader antenna to a fixed value of 100 €. A green LED indicates a successful completion. The mode is highly suitable for demonstration purposes, e.g., convincing the system operator that his system is flawed. Since no user input is required, the reader can operate in this mode autonomously in the field.

Desktop Software

The PC-side part of the software framework communicates and cooperates with the embedded software on the μ C. A command-line tool was developed to facilitate the interaction between the RFID tool and a Mifare Classic smartcard, with a main focus on the ID-card deployed in the here analyzed payment system. The program is based on an open-source serial terminal software [Sch01] that makes use of the Windows API to access the serial COM port. Five modes of operation are supported, corresponding to the respective modes of the embedded software running on the μ C:

- c** Crack secret keys of a Mifare Classic card
- r** Dump the content of a Mifare Classic smartcard to a file
- w** Write the dump file to a (blank) Mifare Classic card
- i** Increase the credit balance on the ID-card
- d** Decrease the credit balance on the ID-card

When executed without any parameters a short help screen is displayed on the screen explaining the usage of the program. In the current implementation, dumping of the entire memory of a Mifare Classic card takes approx. 1.2s and writing the entire EEPROM of the card takes approx. 1.5s.

6.6 Tampering with a Real-World System

Analyzing the security of the payment system, we initially observed that an ID-card replies with $ATQA = 00\ 04$ and $SAK = 08$ (see Sect. 2.5 for the ISO 14443 commands), indicating that it contains a Mifare Classic chip [NXP08a]. Hence, we performed a practical key-recovery with the hard- and software introduced in Sect. 6.5.

6.6.1 Recovering the Secret Keys

Figure 6.6 illustrates how the differential attack to obtain the key for the first sector is combined with the nested authentication approach to reveal the remaining keys. Using our setup detailed in Sect. 6.5 it takes less than 30s to perform the authentication runs required for the differential attack and to collect the data for revealing the first sector key of the card. Once the data is collected, it takes less than 3s to recover the key on a standard PC. For the remaining 15 sector

keys the nested authentication attack is utilized which takes on average 30 s per sector. Hence, to extract all 16 sector keys from the card we need less than $30 + 3 + 15 \cdot 30 = 483 \text{ s} \approx 8 \text{ min}$, which is by a factor ten faster than the 80 min for 16 sectors proposed in [Cou09]. No precomputation or other data storage is required for our attack.

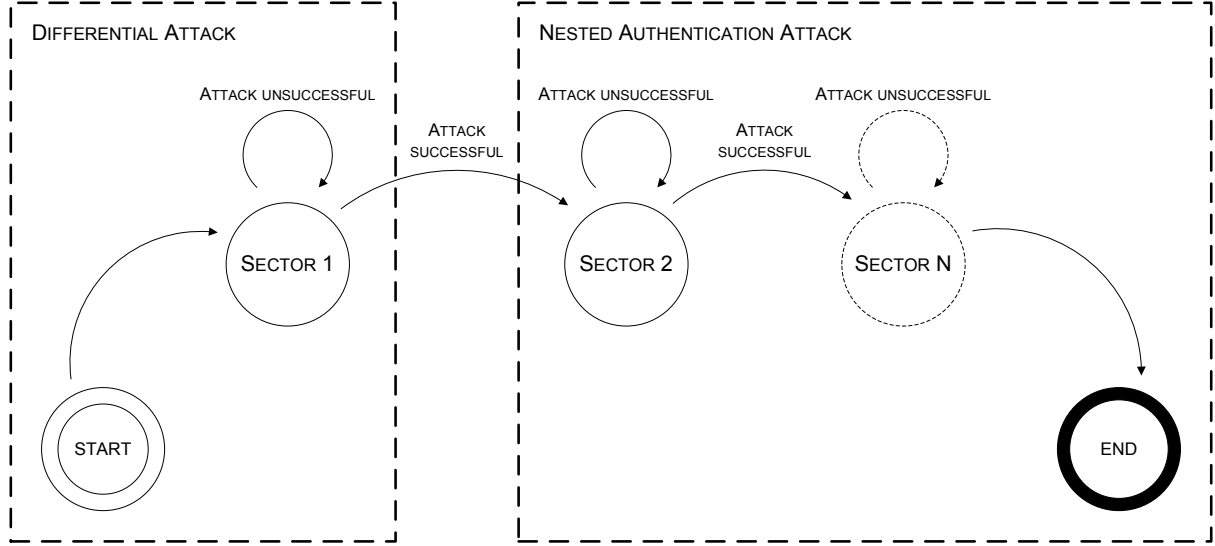


Figure 6.6: Our Implementation: Combining the Differential Attack with Nested Authentication.

We executed our implementation on the ID-card and successfully revealed all of its secret keys, thereby noticing that the revealed keys differ from the factory-programmed default keys of Mifare Classic. When analyzing a second card, we found that identical secret keys are used for all sectors. We conjectured that all ID-cards in the system might have the same keys. By analyzing a dozen more cards we verified the hypothesis: *all* ID-cards in the payment system seem to have an *identical* set of sector keys.

6.6.2 Reverse Engineering the Card Content

From comparing the content of several ID-cards, before and after carrying out charges and payments at different charging terminals and cash registers, we learned which exact data the smartcard contains apart from the factory-programmed **UID** and where it is stored in the memory. The most relevant content for practical attacks is listed next.

Card Number During the initialization, each card obtains an individual card number⁷ that is stored in a block on the card and serves for identifying the card in the back-end.

Credit Balance The credit balance is stored in plain with no additional security measures. Hence, knowing the secret key for the sector, the credit value can be arbitrarily changed.

Timestamps of the last charging and the last payment, as well as identifiers for the lastly used payment and charging terminal, are updated on each transaction.

⁷Note that the user-programmable card number is independent of the factory-programmed, read-only UID.

Checksum A (non-cryptographic) checksum, computed by XORing all bytes of two subsequent blocks, is appended to the sectors containing the card number and the timestamps.

The latter **XOR** checksum was reverse-engineered by a trial-and error approach when trying to produce fake ID-cards. It provides no extra protection — the check byte can simply be adapted to any new **UID** or card number. Interestingly, the most important information on the card, i.e., the credit balance, is not even secured by such a checksum.

6.6.3 Practical System-Level Tests

By prior agreement with the system owner we carried out a number of tests in order to determine which actual vulnerabilities the system exhibits. Note that the aim was *not* to cause the maximal financial damage or other harm, but rather to investigate the spectrum of weaknesses. For some of the transactions detailed in the following, blank Mifare Classic cards were used. Since the imprint of these cards differs from that of genuine ID-cards, payments with the forged cards were carried out without removing them from the wallet, by simply putting the whole wallet on the reader.

Payments with Cloned Cards

The goal of the first experiment is to determine whether payments with clones of an original ID-card are possible. We copied a genuine ID-card to three blank cards — which naturally all have different **UIDs** — without modifying the credit value and the card number. Then, several payments were carried out with these (almost) cloned cards. The possibly existing shadow account associated with the card number of the genuine card was intentionally overdrawn by paying a larger sum of money than the initial value stored on the card. On the next working day another payment with the genuine ID-card was carried out in order to check whether it had been blacklisted or not.

The cloned cards technically appeared exactly as the genuine ID-card, and more than twice the credit balance of the original ID-card was spent with the clones, to overdraw the shadow account (if a shadow account exists). Despite the overdrawn shadow account the card number was not blacklisted on the next working day, since paying with the original card was still possible on the next working day. The existence of several cards with identical card numbers but different **UIDs** and credit amounts has not been detected, implying that seemingly no automated checks based on the **UID** of the card or the shadow account are performed.

Tampering with the Credit Balance

The intent of the next tests one day later was to find out whether payments using a cloned card with a manipulated credit balance are accepted. Since the previous fraud was not detected, a second goal was to further overdraw the corresponding shadow account with a substantial amount of money.

The genuine ID-card from the previous test was copied to another blank Mifare Classic card, and the credit balance was set with our reader to an eye-catching value of 100€. Subsequently the generated 100€-card was used to pay for the meals of four people at the enterprise restaurant, overdrawing the shadow account by 28.45€. The successful test revealed that payments using the cloned card with a faked credit balance is possible without problems.

Generation of New Cards

The next goal was to test whether cards with modified card numbers and fake credit balances are detected. For that purpose four new cards with fake credit balances were generated. Two of the new cards obtained random numbers, while the card numbers of the remaining two cards were generated by increasing the (known to be valid) card number of a genuine ID-card. Our first naive attempt failed, since we did not take the **XOR** check byte into account (see Sect. 6.6.2).

Once we found out the meaning of this byte and modified it accordingly, we were able to produce cards with a fake card number that are recognized as genuine: all the faked cards with a new **UID**, a fake card number, and a fake credit balance are accepted for further payments. In order to check whether the cards have been blacklisted or remained valid, another batch of payments was carried out the next day.

We discovered that paying with all four cards was once again possible without any problem — none of the cards was blacklisted the next day. The money was withdrawn from cards with slightly modified card numbers and even from cards with completely random card numbers. Since payments with random card numbers are accepted, it is likely that no checks are performed whether the card number is known to the system. However, it is conceivable that the shadow accounts of other users of the system, holding ID-cards with the same card numbers as used in the test, might have been charged for our purchases⁸.

Cashback

External visitors or guests can obtain an anonymous ID-card at any payment counter for a deposit of 10€ that can be used for cashless payments. When not needed any more, the cards can be returned and the deposit as well as the current credit balance is paid out in cash at the checkout. The next experiment intends to find out if an anonymous pay card with a modified credit balance can be used to convert fraudulently increased digital money into real cash.

Accordingly, we bought an anonymous card and fraudulently increased its credit value from 4.00€ to 44.00€. The content of the card was saved to a dump file for future tests (compare with Sect. 6.6.3). Two hours later, the cashback was initiated at the checkout. The cashier paid out the full credit balance of 44.00€ stored on the anonymous pay card plus the 10.00€ deposit fee, in cash. The anonymous pay card was retained by the system operator. Thus, we have found and practically tested a means of converting digital money into real cash.

Using a Clone After Cashback

In the next step, we wanted to test if paying with a duplicate of the already returned anonymous pay card is still possible. Note, that the shadow account of this card, if existing, is overdrawn by −40.00€ due to the previous cashback attack.

A new blank Mifare Classic card was generated with the content of the dump file saved in the “cashback” test, containing the credit balance of 44.00€. Several payments were carried out at various points of sale, where contactless payments with the ID-card are supported. A total of 38.12€ was spent, showcasing that paying with the clone of the invalidated anonymous

⁸All transactions during the security analyses were recorded. The protocol was afterwards handed over to the system integrator, who promised to restore the correct credit to the affected payment cards.

pay card was still possible without any problem, while the corresponding shadow account would contain $-(40.00\text{€} + 38.12\text{€}) = -78.12\text{€}$.

Blacklisted ID-card

Two days after the initial test, the system operator sent out a notification that the card number of the genuine ID-card used to generate the 100€ card for the test “Tampering with the Credit Balance”, has been blacklisted. Hence, it was interesting to find out whether the card was actually revoked in all locations accepting ID-cards for cashless payments.

Consequently, payments were carried out at five different locations within the organization that accept ID-cards. It turns out that the ID-card was only revoked at the main restaurant. The fraud was not detected at all other locations where contactless payments are supported. During the experiments, the card has been fraudulently re-charged three times, since the original credit balance had been used up. In the follow-up talk with the system operator it was mentioned that the cause was presumably a delay in the update of blacklisted card numbers in the system, which resulted in the described behavior.

Paying with the Card Emulator Chameleon

As a final test of our analysis, we verified whether paying with our low-cost card emulator Chameleon introduced in Chap. 5, instead of paying with a card, is practical. We dumped the content of a genuine card, including the **UID**, and copied it to the Chameleon, thereby creating an exact clone. Hiding the device in a wallet, we consequently were able to carry out contactless payments. The credit value is stored in the **EEPROM** of the Chameleon and is decreased according to the balance due. As a result, the remaining credit displayed to the cashier appears to be correct and the emulator was accepted as genuine.

The Chameleon automatically recharges the balance to its original value by restoring the initial dump upon pressing a button. Hence, after paying several times with the Chameleon, we restored the original credit and successfully carried out more payments. As a result, unlimited payments could be carried out. We furthermore verified that the Chameleon allows an intruder to open doors secured by an ID-card, by cloning a valid card of an employee.

However, if the fraud occurring due to the state restoration attack would be detected on the long term, the card number and/or the **UID** could be blacklisted and blocked for future payments. For a more powerful attack, we programmed the Chameleon to generate a new random **UID** and card number after each reset, as shown in Fig. 6.7. In our practical tests our emulator now appears like a new genuine card on every payment. Again, we were able to carry out regular payments, but this time, the device cannot be blacklisted and blocked in the back-end.

Since cards of other customers can be read out from a distance (modified **RFID** readers allow for reading distances up to 30 cm, see Sect. 2.7) the Chameleon can also be used to clone their cards in a real-world scenario. Reading out the relevant sectors with our reader takes less than 100 ms. Several cards of other customers can be stored in the Chameleon and hence payments can be carried out with cloned cards that already exist in the payment system. Note that the original card of the customer remains unmodified and thus still contains the original credit value — the financial damage only occurs for the payment institution, while the customer is not affected (unless his card would be blocked).

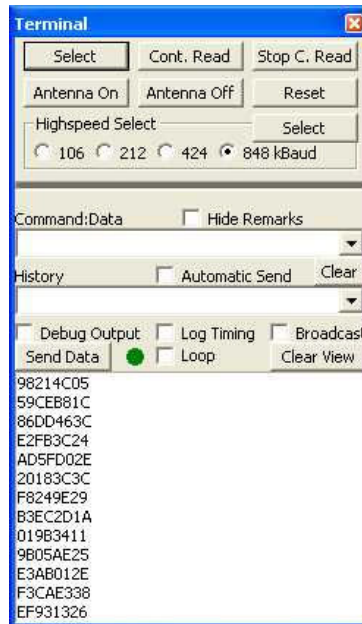


Figure 6.7: UID emulation with Chameleon: On each selection process the device appears as a card with a new UID, displayed in the bottom of the terminal program.

6.6.4 Summary of the Vulnerabilities

From the tests described above, we conclude that the security of the analyzed ID-card payment system is extremely low based on today's cryptanalytical knowledge. The application relies solely on cryptographically weak Mifare Classic cards with *identical* secret keys. In addition, the security mechanisms on the system level are implemented very poorly.

All data on the cards is stored in plaintext and almost no integrity checks are performed. Neither the **UID** is verified, nor are cards with the same card number, but a different credit balance, detected. Even cards with new random card numbers and an arbitrary credit balance can be generated that are accepted for payments. Furthermore, the attack related to the cashback option, enabling an attacker to have an illegally increased credit balance paid out in cash, is possible without a problem.

From the practical experiments we conclude with near certainty that no automated monitoring of the shadow accounts is implemented to detect fraud. However, fraudulently increasing the credit balance of a genuine ID-card to the eye-catching amount of 100€ resulted in the corresponding card number being blacklisted, which suggests that shadow accounts are existing and are maybe inspected manually.

In summary, our tests revealed that small inconsistencies between the credit balance stored on the card on the one side and the shadow account of the corresponding card number on the other side were *not* detected. Seemingly, no additional (automated) checks are performed, neither in the front-end, nor in the back-end of the system, allowing for various attacks as illustrated in the following section.

6.7 Exploiting the Weaknesses: Attacks and Economic Implications

The security vulnerabilities as detailed in Sect. 6.4 put every system relying on Mifare Classic at a great risk. However, system integrators not being capable of implementing security measures that mitigate the known risks often worsen the financial situation, as illustrated in the following for the ID-card system. The implications of the security flaws described in Sect. 6.6 are dramatic. We recall that, fatally, the keys of all ID-cards used in one enterprise are identical. Hence, once the secret keys are recovered from *one* card, devastating attacks can be mounted for *any* card in that enterprise. This non-technical section illustrates the possible implications.

For each revealed security vulnerability, we investigate possible practical attacks and also the financial implications that may emerge in the real-world by estimating both the possible profit of the attacker and the corresponding financial damage on the side of the issuing institution. We focus on those attacks that imply cost for the issuing institution and treat the impact on individual customers only in passing. When estimating the monetary implications, for convenience we often make worst-case assumptions and work on simple numbers, e.g., 50 €. In the real-world, a criminal would charge cards with different, inconspicuous values, to reduce the detectability of the modifications.

6.7.1 General Scenario

The pillars of our analysis for a real-world scenario are defined according to personal talks with the system integrator of the ID-card system. The monetary model and other conditions introduced in the following shall allow to compare the attacks and determine their relative impact.

Practical Parameters for Attacking ID Cards

The attacks, including the one-time key extraction, require no special knowledge, but can be performed by an unskilled attacker, e.g., possessing our public-domain reader and the corresponding software or by means of a commercial reader and open-source program code [Cra08]. According to our practical results, accessing the card for ten minutes at a fixed distance is sufficient to break all its secret keys. Extracting keys is hence straightforward when having physical access to a card but is not considered practical in a pickpocketing scenario. Once the one-time extraction of all keys of the card is performed, any reader can be used with standard software to read or modify a card. Modifying one memory block takes approximately 20 ms [NXP08a], including the authentication to the corresponding sector.

Knowing the secret keys, all cards in the system can be read and written wirelessly from up to 30 cm in less than one second, and valid duplicates of cards can be generated easily. Using the hardware from Sect. 6.5, attacks can be carried out in a stand-alone mode in the field. For increasing the operating range of the device to 30 cm (compare with Sect. 2.7) we estimate a cost of about 50 €, hence we fix the price for a “disguised reader” (explained in Sect. 6.7) to 100 €.

The adversary does not need to be an employee or staff member to gain access to a card, since an anonymous ID-card can be obtained at any cashpoint for a deposit of 10 €. Blank Mifare Classic cards can be purchased for 0.50 €, while cards with a custom imprint equal to that of genuine ID-cards are estimated to cost 0.70 €. Emulating ID-cards is feasible with

the Chameleon from Chap. 5 that costs less than 20€. It is further conceivable that an NFC mobile or a freely programmable contactless smartcard can be used to emulate a Mifare card. We assess that a suitable NFC mobile phone can be obtained for 200€. The author possesses a completely rewriteable Mifare Classic card (including the block storing the UID) that can be purchased from China for 20€.

Turnover per Customer

Assuming 30 holidays per year and five working days per week, a customer will use the contactless payment system roughly 46 weeks, i.e., on 230 days in a year. We further assume that a typical customer has on average 50€ credit on his ID-card and spends on average 10€ per day for food, drinks or other services such as copying and printing, by carrying out on average two payments per day. Consequently, each ID-card is recharged once per week, corresponding to 46 chargings per year or $50\text{€} \cdot 46 = 2300\text{€}$ turnover per year and customer. In case of manipulations of ID-cards, the financial loss per customer and week in the worst case hence amounts 50€.

Inhibition Threshold

For estimating the possible financial damage, it is required to model whether a human might become a criminal. We assume that a customer might illegally charge his card only if (i) he saves at least half of the regular price of the items, implying a cut down of his expenses by a factor of two, and (ii) the crime is very unlikely to be detected. Even if these two conditions are fulfilled, we assume (hopefully rather conservatively) that the majority of customers remains honest and that only 1% of them, i.e., 10,000 customers in all organizations using the system, cannot overcome the temptation of achieving some financial gain by means of cheating.

6.7.2 Duplicates based on Blank Cards

An adversary needs about 40 ms to covertly extract the card number and the credit value from the ID-card of a victim from a distance. The same information can be obtained from eavesdropping on one successful payment. Knowing this data, the adversary can produce duplicates on the basis of anonymous cards or blank Mifare Classic cards (that may even have an imprint that is indistinguishable from the original). Technically, these duplicates are still no exact clones, because the copied Mifare card will have a UID that differs from that of the original. However, this discrepancy is not detected by the ID-card payment system: for a cash register the duplicated cards appear identical to the original card.

Digital Pickpocketing

For *digital pickpocketing*, the attacker decreases the money on the card of the victim (requiring another 40 ms) and produces a new ID-card with the same card number and the “stolen” amount of money on it. The adversary can now pay with an ID-card that is known to the system, and spend the money of the victim. Since the shadow account of the card is not overdrawn, the fraud will *not be detected* in the back-end. This attack would harm only the victim, while no damage is caused for the issuing institution.

Impersonation

In another variant of the pickpocketing attack, the adversary *impersonates* the victim by generating a new card with the extracted card number and the extracted credit value, to obtain a duplicate. This time, the card of the victim remains unchanged. The adversary is able to pay on the behalf of the original owner of the card, while the credit amount on the card of the latter remains untouched.

Both parties can now independently spend the full credit value that was stored on the original card at the instant of reading out the content. Hence, the fraud rises the amount of digital monetary value in the payment scheme without real money being deposited into the system, implying direct financial damage for the issuing institution.

On the long term, the issuing institution could identify a card with an overdrawn shadow account and block the respective card number. The original card then would have to be revoked, implying additional cost with respect to customer service, as a new card has to be issued for the victim. We would like to stress that shadow accounts were sparsely, if at all, checked in the system investigated.

Monetary Implications of Trafficking Duplicates A criminal could sell restricted clones with an average credit balance of 50€ for 25€. Subtracting her investment for either a blank card at a cost of 0.50€ or 0.70€ for a card with a falsified imprint, the gain per marketed restricted clone would be 24.50€ or 24.30€, respectively. We conclude that the estimated profit for the criminal is sufficiently high so that producing restricted clones is likely to take place in the real-world. Neglecting the losses for the customer service in case a card has to be blocked, the financial damage for the issuing institution is equal to the credit amount that is fraudulently written to the cloned cards, i.e., in the example 50€ per card marketed by the criminal. Note, that an ID-card can be charged with up to 150€, hence the criminal could further increase her profit per card.

6.7.3 Exact Clones

Producing exact clones requires that not only the content of the card but also its read-only **UID** is copied to the duplicate. Blank standard Mifare Classic cards contain new **UIDs** that differ from the original **UID**, hence they cannot serve as a basis for an exact clone. Note, that Mifare Classic cards with a (re)writable **UID** are available on the market. Further an attacker aiming to create exact clones can to use special equipment that allows emulating Mifare Classic cards, e.g., the Chameleon or an **NFC**-compliant smartphone. In the following, the term “exact clone” is used for devices that are technically indistinguishable from a genuine card by a reader, regardless of their optics.

The relevant sectors can be read out from a distance in less than 100 ms, hence cloning the cards of victims is feasible in a real-world scenario. Several cards of other customers can be stored in the Chameleon, thus payments can be carried out with different cloned cards that already exist in the payment system. Note that the credit value of the card of the victim remains unmodified. Accordingly, a financial damage will only occur for the payment institution, while the customer is not affected.

Monetary Implications of Trafficking Exact Clones Marketing exact clones based on the basis of the card emulator Chameleon (20€) results in a possible profit of 55€ per clone, assuming a maximum market value⁹ of a restricted clone of 75€. If an NFC mobile is available, offering virtual ID-cards, e.g., via Internet, is also assumed to pay off for a criminal. Despite the higher effort compared to duplicates on the basis of blank cards and the required skills for programming the devices, the possibility of this crime cannot be ruled out. The resulting loss for the issuing institution, analogous to the case of restricted clones, would be in the worst case 150€ per clone sold by the criminal.

6.7.4 State Restoration and Charging Cards

One straightforward option for an attacker is to read out the data of an ID-card, e.g., her own card, and as a first step store its content to a computer. During the next days she keeps paying normally with the original card. In case the credit is used up, the adversary simply copies the original content back to the ID-card, thereby restoring the previous state and hence the full credit balance. For this powerful *state restoration* attack, no knowledge is required about how the memory of the card is organized. An adversary possessing the Chameleon can perform this state-restoration attack easily by pushing one button (compare with Sect. 6.6.3). Note that the attack even works if the content of the card is secured, e.g., by encrypting the credit value using AES with an individual key before storing it on the card.

In the attacks described so far, the criminal has never modified the credit value stored on the cards. In reality, as detailed in Sect. 6.7.1, even an unskilled attacker can write a new value to the respective block in the ID-card. She is hence able to modify the amount of the credit balance on her card or on the cards of others with minimal effort. In the case of emulating cards, modifying the credit value stored on the Chameleon is trivial. This enables new options for the business of the criminal: She can for instance produce clones from any ID-card with a low credit value, charge them with the maximum permitted amount of 150€ and sell the clones. Moreover, she can offer a charging service for the original cards of her clients. This approach is difficult to detect and circumvent for the issuing institution, since the cards are genuine and known to the system, and only their credit value has been modified. In court, the charging crime would be difficult to prove, because no physical traces are left when modifying existing cards.

Monetary Implications of Charging Cards Charging cards is highly profitable for a criminal, amongst others because the attack can be performed with existing cards. The maximum gain is 75€ per card charged with 150€. The best option is charging original cards, since it is difficult to prove whether they have been unauthorizedly modified. For charging exact clones the one-time investment, e.g., for our card emulator or an NFC mobile, can be neglected on the long term. The fraud rises the amount of digital monetary value in the payment system without that real money has been paid into it, implying direct financial damage for the issuing institution. The latter amounts to at most 150€ per illegally charged card. Manipulated cards can be revoked on the long term. However, in this case the customer can purchase a new genuine card which can again be charged by the criminal.

⁹The cards can be charged with a maximum of 150€ — according to our model a criminal can hence sell the cards for 75€.

6.7.5 Creation of New Cards and Unlimited Payments

When paying with an ID-card, the existence of its card number in the back-end is not checked, as has been practically verified in Sect. 6.6. Hence, instead of charging clones, it is even possible to generate new valid cards with an arbitrary card number and an arbitrary credit value. While the risk and financial impact for this attack is comparable to the case of producing clones, this approach becomes interesting in the context of emulating ID-cards.

Emulation of Arbitrary Cards

The Chameleon or a similar card emulator can generate a new **UID**, a new random card number and a random credit on every activation. We have implemented this “arbitrary card” mode on the Chameleon. The device appears as if a new ID-card is used for every payment, which renders detection of the fraud in the back-end and tracking of a person using such a device practically impossible. It is conceivable that mobile phones capable of **NFC** can be programmed in a similar way.

Monetary Implications In order to estimate the value of a tool that allows for unlimited free payments we assume that a customer would buy it, if the purchase pays off after three months. According to our model, this equals $12 \text{ weeks} \cdot 50 \text{ €} = 600 \text{ €}$. Thus, we estimate that an attacker can sell an ID-card emulator that is programmed for unlimited free payments for 600 €. Comparing this with the low cost of 20 € for our card emulator, the criminal would make a profit of 580 € on every device sold.

Despite the fact that the criminal can only sell one device per customer and that the majority of potential customers may not buy the device due to the high price, the gain for the criminal is large, hence this attack seems realistic on a small scale. The devices for unlimited payments could even be programmed such that they switch off after a period, i.e., the business could be run on a subscription basis. The issuing institution faces the worst-case loss of 50 € per dishonest user and week, i.e., 2300 € per customer and year.

6.7.6 Denial-of-Service

A malicious attacker who is not interested in direct financial profit, but instead aims mostly at harming the issuing institution, e.g., a competitor or an extortionist, can execute a **DoS** attack. For the attack, she stealthily sets the credit balances of cards in the field to zero, for instance by means of hidden readers. The manipulating readers are placed at promising locations, e.g., at the entrance or close to a waiting line at the check-out counter, and can reach cards from a distance of up to 30 cm. Modifying the two value blocks storing the credit value takes again just about 40 ms, hence the attack is considered feasible in practice. As a result, the affected customers will complain about their vanished credit, implying image loss and costs due to the customer service for the issuing institution.

Distributed All-You-Can-Eat

An alternative for the classical **DoS** approach is to have all cards in the range of a disguised reader charged to a *high* monetary value. It is very likely that the affected customers either might not

even notice the increased amount on their cards (especially if increased by a moderate amount, e.g., 5 €-10 €) or, if noticed, at least might not complain about having too much money on their cards — the fraud might hence be detected by the system operator very late. If this attack would be conducted on a large scale, the impact can be dramatically more severe compared to that of all other above attacks, in which only dishonest customers participate willingly. In this scenario, *all* cards of customers coming into the vicinity of the disguised reader will be charged, thus enormously increasing the amount of contributors in the attack, and thereby also increasing its financial impact.

Monetary Implications To estimate the impact of the attack we assume that one well located, disguised reader charges one card of a victim passing by in every five minutes, corresponding to twelve cards being charged in one hour, or about 100 charged cards in one day. In theory, dozens of cards could be charged per minute — charging one card takes below 50 milliseconds — the conservative estimation above hence takes times with less customers passing by into account. Assuming that on average 50 € are previously stored on each ID-card and that they are loaded during the attack to an amount of 100 €, the issuing institution loses 50 € per card charged. Accordingly, one appropriately placed, disguised reader could on the first day produce a substantial surplus of about $100 \cdot 50 \text{ €} = 5,000 \text{ €}$ of virtual money in the system. Since many customers with infected cards will not charge their cards with real money, a direct loss for the organization occurs.

If unauthorized chargings of random cards in a large-scale are detected, the only option for the enterprise may be to completely shut down the contactless payment system and replace it, or upgrade it with a more secure one — the “all-you-can-eat” approach is hence an extremely efficient variant of DoS from the attacker’s view point. Again, if the issuing institution would decide to sue all victims for manipulating the content of their cards, in practice it is not possible to prove whether a card has been modified with or without permission (or notification) of the owner.

6.7.7 Cashback Attack

As practically proven in Sect. 6.6.3, an adversary can receive a cash payout of the digital currency stored on his ID-card at the checkout. Virtual money can accordingly be converted into real cash:

- (1) Get an anonymous pay card for a deposit of 10 €.
- (2) Increase the credit balance stored on the card to an arbitrary value.
- (3) Go to the checkout and receive a payout in cash corresponding to the fake balance value.

The attacker in consequence obtains real money out of the cash registers of the system that never “existed” on her pay card, and thus cause direct financial losses to the issuing institution. Multiple accomplices could collaborate and work alternately to conceal the scam. Note that it is not possible to trace back a criminal who would abuse the system in this way, as the anonymous pay card contains no information about the card holder.

Monetary Implications Converting fraudulently increased digital money into real cash corresponds to a maximal gain for the criminal of 150€ per attack. The crime can be conducted at no cost, as the adversary will obtain her deposit of 10€ back in the course of the attack. Likewise, the direct financial loss on the side of the issuing institution for each instant of the attack in the worst-case equals 150€ in cash. Note, that the (daily) repeatability depends on the number of attackers and the amount of cash registers in the system, because the staff would probably notice if the same customer wants to return his card more than once on the same day.

6.7.8 Pickpocketing with a Relay Attack

Performing a classical relay (or MITM) attack requires two adversaries. By relaying the digital information between the card of a victim and the cash register back and forth in real-time, the reader at the cash desk is made believe that the card of the victim is in its direct vicinity, and thus carries out the payment with a distant card (see Sect. 2.2.4 for details about the attack). Compared to the simplicity of the other sketched attacks, it is relatively difficult to perform a relay attack in practice, since the remote communication with the card of the victim has to be established in the exact moment when the adversary at the cash register wants to pay. The relevance of the attack in the context of micropayments is hence marginal.

Monetary Implications After the one-time investment for the required RFID tool (40€) or a similar device, the criminal can repeatedly pay with the cards of varying victims. The maximum gain per attack amounts to the value of one payment, i.e., on average 10€ and maximal (but very unlikely) 150€. Despite the fact that in each instant of the attack a new victim is affected, there is a chance of being detected because the equipment required for the relay attack looks different from a plastic card. The issuing institution faces no losses, since the money is directly withdrawn from the card of a victim. The probability of this attack taking place in the real-world is rather low, due to its difficulties and the small profit for the attacker.

6.7.9 Estimating the System Wide Damage

As detailed above, an unskilled attacker has different options to sell newly created cards or charge the cards of others at a very low cost, with little effort, and a low chance to be detected. The content of a card can be wirelessly altered without permission or notification of the holder. A transaction leaves no physical evidence, rendering a “wireless crime” even more likely to be committed in the real-world.

Based on the scenario sketched in Sect. 6.7.1, we summarize in the following the losses for the enterprises that use the current realization of the ID-card system. We consider the two most devastating practical attacks: (i) *charging cards*, for which the gain for both the criminal and her client is high and the probability to be detected is low, and (ii) *DoS*, involving the most dramatic economic implications. The estimations intend to give a raw idea about the extent of the financial damage, in case that people would become criminal. The damage in the real-world would very likely be a lot smaller. We do not discuss attacking the access-control function of the ID-card here, which could have other severe, especially privacy-related, implications.

System-Wide Damage due to Charging Cards

A card-charging business set up by a criminal would be assumed to run well on a buy-one-get-two basis, i.e., for each Euro paid by the customer, his card will be charged by the criminal with two Euros. The card of a customer as modeled in Sect. 6.7.1 is recharged with 50€ once a week, thus a customer having his card charged illegally saves 25€ a week or $46 \cdot 25\text{€} = 1150\text{€}$ a year. For the criminal, 100 clients imply an illegal income of 2,500€ per week, i.e., approximately 120,000€ a year. Note that the profit scales: If all 10,000 customers prone to become criminals across all enterprises employing the scheme (cf. Sect. 6.7.1) are taken into account, the criminal earns a weekly 250,000€, corresponding to a (fantastic) annual income of 12,000,000€. A face-to-face interaction with the clients is not compulsory, because the business could be based on mailing services or charging the cards via the Internet on a subscription basis.

The financial loss for the issuing company is twice as large as the profit of the criminal, i.e., 50€ per week and tampered card. In case of 100 criminal customers the institution loses 5,000€ per week or 240,000€ per year. If 10,000 customers became criminal, the theoretical yearly financial damage increases to 24,000,000€. However, this is the damage caused in all installed systems, i.e., it would be spread over all branches of the enterprise.

System-Wide Damage due to Denial of Service

In a DoS scenario, contrary to having their cards charged intentionally, the customers walking past a disguised reader become unwillingly part of the attack — hence, the number of contributors is much higher for a DoS attack. According to Sect. 6.7.6, ten disguised readers set up by a criminal, at a cost of 100€ per reader, will unnoticedly charge 1,000 cards per day, i.e., reach one per mill of the ID-card customers daily. Her investment of 1,000€ implies a daily increase of $1,000 \cdot 100\text{€} = 100,000\text{€}$ of digital money in the ID-card system that has never been deposited as real money. After the first two weeks, i.e., ten working days, this already corresponds to harming the system provider by 1,000,000€. On the long term, i.e., periods longer than two weeks, a saturation effect would have to be considered. In summary, the contactless system would have to be shut down completely, shortly after the beginning of the attacks, once major irreparable inconsistencies are noticed.

6.8 Countermeasures

In the following, we propose simple countermeasures that require little effort by the system integrators, prevent most fraudulent uses of ID-cards, and are also applicable to other contactless systems. With more than 1 billion Mifare Classic cards worldwide still in deployment in various kinds of e-payment and access control systems, modernizing the existing cards and hardware would imply a high cost. Hence, the improvements in this section only require modifications of the software in the back-end and the content of the cards, while the hardware and the type of cards, i.e., Mifare Classic, remains unchanged. Our proposals thus cannot provide 100% security, amongst others due to the weak Crypto1 cipher used in the cards, but aim at increasing the security level such that the attacks become impractical or unprofitable for a criminal.

6.8.1 Organizational Measures and Back-End Improvements

The back-end shadow accounts, as currently utilized in the system, do not provide enough protection against cloning attacks or malicious manipulations of the card content. Thus, some improvements of the system security can be attained by introducing new rules and educating the staff. In order to detect fraud and blacklist the affected card number, automated consistency checks of the shadow accounts should be performed on a regular basis and all cash registers should be updated synchronously, preferably several times a day, to restrict the time-slot in which financial losses for the issuing institution could be generated. In addition to the card number, the **UID** should be read by the readers and checked later in the back-end system, thus preventing fraud with illegally generated cards that are not registered in the system. Furthermore, checkouts at all locations where an ID-card is accepted for payments must block the blacklisted card numbers.

Educating the staff can further impede fraud. By optically controlling each ID-card, i.e., prohibiting to pay with a card hidden in the wallet, both a relay attack and the attacks relying on electronic emulation of ID-cards could be made much more difficult to accomplish in practice. For preventing a cashback attack, a manual inspection of the corresponding shadow account has to take place always before the withdrawal of a card and paying out any cash. As a result, an adversary is restricted to using original cards or falsify the imprint of the clones.

6.8.2 Key Diversification

Each Mifare Classic card in the system should obtain its own set of secret keys that is essentially different for each individual card. We recommend to employ a cryptographically secure key derivation mechanism in the cash register. More precisely, the keys should be derived from the individual **UID** of the card, the sector number to be accessed, and one or more master keys. Since the cash registers are based on standard computers, sufficient computational power is available such that any secure cryptographic function can be used for generating the keys, e.g., an **AES**-based **MAC** [PP09]. The master key used for the key derivation has to be kept secret and the key derivation has to be implemented securely to prevent an adversary from replicating it, e.g., based on highly secure hardware modules.

For many systems, depending on how the access condition bits are currently set, the upgrade from the fixed-key to a diversified-key scheme can be realized without any special interaction of the customer, during a normal payment. The content of a sector is read out using the old, weak keys and then written back with new, securely derived keys. Similarly, the keys can be upgraded in case a master key gets compromised. If the access conditions do not permit changing of the keys, the cards have to be exchanged at a cost of 0.50€ per card plus the customer service.

Using different keys for each card renders most of the attacks described in Sect. 6.7 a lot harder. For instance, each **DoS** attack would require to extract at least three sector keys which takes about 2 min — as compared to 40 ms in the current implementation — rendering a **DoS** attack with a high penetration rate practically infeasible. Producing restricted clones and creating new cards on the basis of blank Mifare Classic cards becomes impossible, because the blank cards have new **UIDs** and hence their correct secret keys cannot be guessed by an attacker. Still, exact clones as described in Sect. 6.7.3 would be accepted as a valid ID-card at the cash desk.

6.8.3 Securing the Data on the Card

To prevent an adversary from manipulating the monetary value stored on an ID-card, the authenticity of the credit value can be secured by a **MAC**, e.g., the Poly1305-AES [Ber05b]. The **MAC** could be computed from the credit amount and the card number, with an individual secret key for each card that can be derived similarly to Sect. 6.8.2 as a function of a **MAC** master key and the **UID** of the card. The 16-byte output of the **MAC** fits into one block of the Mifare card.

Before any transaction is carried out, the **MAC** has to be verified by the cash register to ensure the authenticity of the content of the card. If the verification fails, fraud is likely and the transaction is aborted. The **MAC** computation and verification again takes place in the cash registers, implying no additional computations for the card. As a result, an attacker can neither copy the dump of a genuine ID-card to a blank card with another **UID**, nor tamper with the credit value stored on the ID-card, as the **MAC** verification would fail. For further disorienting an adversary, and impede reverse-engineering of the data structure on the card, the content of all utilized blocks can be stored encrypted, e.g., using **AES**. This is mandatory in case the card contains private or other data that has to be kept confidential. The time required for computing or verifying a **MAC** or executing an **AES** on a cash register is less than a millisecond. Hence, the overhead is negligible compared to the time required to access the memory of a Mifare Classic card.

Teepe [Tee08] proposes an asymmetric setup based on digital signatures instead of **MACs**. The author distinguishes between two types of readers in the system, namely

Validation readers that can only validate the state of the card using the public key, and

State-modifying readers that are able to change the state of the card with the private key.

This scheme is more costly and allows for more flexibility. However, in the case of the ID-card system every cash register would have to implement both a state-modifying and a validation reader (since cards must be charged at any cash register), hence no added security is expected.

6.8.4 Rolling-Code Scheme

A so-called *counter sector* proposed in [Tee08] holds multiple copies of an irreversibly decrementing counter stored in two or three value blocks of that sector. The countermeasure relies on a feature of Mifare Classic cards that enables to configure a block with *frozen* access conditions such that it can only be decremented or read (see Sect. 6.3). This block is used to store the decreasing counter that in turn becomes part of a digital signature that shall prevent from cloning and state restoration attacks. While this method is theoretically effective against state restoration attacks (see Sect. 6.7.4) based on blank Mifare Classic cards, it does not protect from card emulation, e.g., with the Chameleon.

However, it is advisable to be independent of the proven to be unreliable security mechanisms of Mifare Classic cards, since further undocumented features or bugs of the Mifare Classic cards might be revealed in the future. We hence propose to implement a mechanism similar to a rolling-code system, as described in Sect. 2.3.2, in which both the cash register and the payment card store the value of a counter that is increased in both devices upon every successful payment. The counter is individual for each ID-card, i.e., each card number is linked to a particular

counter, and stored encrypted in a normally configured block of the card and in the cash registers respectively the back-end. An ID-card is only accepted as valid, if the corresponding decrypted counter value read from the card is greater than the current value stored in the cash register. During the synchronization with the back-end, the highest counter value stored for a particular card number in any of the cash registers of the system is stored as the new counter value and transferred to all cash registers. Hence, after a synchronization any card with an outdated counter value, e.g., a clone of an ID-card, can be automatically rejected independently of the state of its shadow account.

A 32-bit counter enables more than four billion payments before an overflow occurs. According to the specification, a Mifare Classic card guarantees 10^5 writing accesses to its **EEPROM**, hence a 32-bit counter is more than sufficient for the lifetime of a card. For encrypting the counter, again the **AES**-128 block cipher is suitable, since its output fits in one block of the Mifare Classic card. The counter can further be included in the computation of the **MAC** proposed in Sect. 6.8.3. Since the **MAC** cannot be falsified, the authenticity of the counter value is guaranteed and spoofing the system by guessing a random encrypted counter value is impeded.

6.8.5 Summary of the Countermeasures

Before its first usage in the payment system, each card is initialized by the issuing institution. According to the key derivation schemes, each card obtains its individual set of secret keys for the Crypto1 encryption, the **AES** encryption and the generation of **MAC**s. A card number is generated, the credit amount and the counter for the rolling code are initialized, and the whole content is encrypted with **AES**, secured with a **MAC**, and stored in the memory of the cards using the respective Crypto1 keys. Then, a cash register incorporating the proposed technical countermeasures would be operating according to the following scheme¹⁰.

- (1) Read **UID** and compute the card's individual key set for the Crypto1 encryption.
- (2) Read (encrypted) credit balance, (encrypted) card number and **MAC** from the card.
- (3) Compute the card's individual key for the **MAC** and verify it. Success: card is authentic. Otherwise: abort transaction and retain the card.
- (4) Compute the card's **AES** key, decrypt its counter and check against the value stored in the cash register. Abort and retain card, if the counter is outdated.
- (5) Decrypt the card number and check if the card is blacklisted. Store **UID** and card number in the database (to detect fraud and blacklist the **UID** at the next synchronization). Abort and retain card, if blacklisted.
- (6) Carry out the payment, i.e., decrypt, modify, and encrypt the credit balance with **AES**.
- (7) Increment and encrypt the counter. Update the database in the cash register to the new counter value.
- (8) Compute the updated **MAC** for the new content, and

¹⁰To simplify the scheme, in the following we assume that only the credit value is updated during a payment — other data, such as time of payment or payment terminal, is processed likewise.

- (9) write encrypted counter, encrypted credit value and **MAC** back to the card.

Combined with the proposed behavioral and organizational countermeasures, more precisely optical inspection of each payment card and thorough automated checks of the shadow accounts, the security is adequately strengthened.

Impact of the Countermeasures on the System Security

Impersonation and cloning based on blank cards as well as card trafficking are rendered impossible due to the strong cryptographic binding of the card content to the **UID**. **DoS** attacks become infeasible in a practical scenario, because the key diversification requires an adversary to perform a key recovery on the card of every single victim. This would in the first step require minutes of access to each single card at a fixed distance which cannot be achieved by means of a disguised reader or by walking past a victim in a practical scenario. Unauthorizedly altering the credit balance in the second step would not have the desired effect, because the verification of the **MAC** would fail, implying that the card is no longer accepted by a cash register. For the same reason, the **MAC** renders a cashback attack impractical — still a manual inspection of the corresponding shadow account is mandatory before paying out any cash and the subsequent withdrawal of a card. The threat of creating exact clones via a card emulator would require an attacker to extract the individual secret keys from the card of the victim. Both state restoration and usage of exact clones are detected automatically after the next synchronization by means of the proposed Rolling-code scheme, since the counters of the clones (or the original card) will be outdated. The profit a criminal could make in between two synchronizations is so small in the context of micropayments, that her effort will not pay off anymore.

The countermeasures are largely independent of the security and implementation issues of Mifare Classic cards, because the cards basically serve as data storage. All relevant security functions are implemented in the back-end of the system. The only contribution of the weak cryptography of the Mifare Classic cards is to complicate reading or modifying their content by walking past a victim. The conditions and time required for extracting the secret keys of Mifare Classic are sufficient to hinder an attacker accessing the cards “on-the-fly”.

Altogether, the countermeasures proposed in this section strengthen the overall security typically to a level that hinders any real-world fraud in the context of micropayments, even for contactless payment systems based on Mifare Classic chips. The major investment for the system operator is the one-time development of the software realizing the additional (cryptographic) schemes in the back-end.

6.9 Conclusion

This chapter gives a comprehensive survey of the security issues around Mifare Classic contactless smartcards and summarizes the existing attacks on these cards. The most efficient practical card-only attack to date is implemented on the **RFID** tool presented in Chap. 4 by combining two of the attacks.

As an example for flawed **RFID** systems in the real-world, the presented in-depth security analysis of the commercial ID-card contactless payment application reveals severe vulnerabilities that pose a great threat to the overall security of the system. With our hardware set-up it takes less than two minutes to recover the relevant secret keys from the used payment card.

Once the keys are compromised, the security of the whole system collapses instantaneously, as the contactless payment cards have identical secret keys and no additional cryptographic mechanisms or obvious other checks are implemented on the system level. An adversary can, in 40 ms and imperceptibly for the victim, read out a card or write to it, increase or decrease its credit balance, clone his card and impersonate the victim. Furthermore, a criminal can sell counterfeit cards or electronic devices that emulate a new random card, and hence permit an unlimited amount of payments. Chapter 5 has proven that such a card emulation is practicable at a low cost. Another fatal flaw on the organizational level enables converting fraudulently increased virtual money to real cash. Most attacks, including the key recovery, can be carried out by an unskilled adversary using commercially available equipment and open-source software. Various attacks were conducted with the **RFID** tool in a battery-operated mode in the field. Due to the wireless nature of the fraud it is difficult (or in some cases impossible) to prove that a crime has been committed, hence prosecution of the adversary or the victims is not promising.

The implications are drastic: Both the victims of our various attacks and (probably to a much larger extent) the operator of the payment system face severe financial losses due to fraud. To impede financial damage, the security risks of this and similarly flawed applications should be instantly addressed by the system integrators. Therefore, an ensemble of efficient countermeasures that can minimize the harm incurred through the described attacks is proposed. For the security upgrade, neither the type of cards nor the hardware of the payment system have to be changed. The countermeasures mainly rely on authenticating and encrypting the content of a card using secure cryptography and establishing a cryptographic binding between a card and its factory-programmed **UID**. Implementing our proposed rolling-code scheme obviates state restoration and even hinders a strong adversary being able to create perfect clones including the **UID** from making profit. Our countermeasures counteract all attacks of practical relevance and hence diminish the financial losses to a minimum. The established security level is sufficient for most systems in the context of ticketing and micropayments.

Despite the findings and proposals in this chapter, a system based on a weak cipher cannot provide bullet-proof security. A major requirement to achieve a high overall end-to-end system security is a (recommended) upgrade to contactless smartcards using ciphers such as **AES** or **3DES**. However, since cryptographic contactless smartcards are vulnerable to power analysis, as shown in the next chapter, secret keys can be extracted even from highly secure cards. Once the keys of one card are known, all the illustrated attacks could be conducted again for all cards of the analyzed system — the cost and effort for the card upgrade hence would be without any effect. Accordingly, an appropriate key derivation and countermeasures as proposed in this chapter are indispensable.

Chapter 7

EM Side-Channel Attacks on Cryptographic Contactless Smartcards

THE key-recovery attacks on Mifare Classic cards presented in the previous chapter are feasible for a completely unskilled attacker, amongst others due to the weak, proprietary cipher employed. Next, we turn towards a more complex cryptographic device and tackle the security of a commercial contactless smartcard based on **3DES**, i.e., the Mifare DESfire MF3ICD40. The **3DES** cipher is regarded as secure, since it has been analyzed by researchers for several decades and no mathematical key-recovery attacks have been found. In this chapter, the vulnerability of the *implementation* of **3DES** on Mifare DESfire cards, with respect to side-channel analysis, is investigated. We are facing a black-box situation, i.e., we have no previous knowledge about the implementation and existent countermeasures, so extensive profiling is necessary in preparation for a key-recovery attack. In addition, the difficulties in acquiring the information leakage contained in the **EM** emanations of the card in the presence of the strong **RF** field of the **RFID** reader need to be addressed. The content of this chapter, including the first successful key-recovery attack on a cryptographic smartcard by means of **DEMA**, is joint work with David Oswald and is mainly based on a paper presented on the Workshop on Information Security Applications [**KOP09**] and an article in the Transactions of Computational Science [**KOP10**]. Special thanks to Kerstin Lemke-Rust and Dario Carluccio, who initiated the analysis of DESfire cards in our group many years ago.

7.1 Introduction

As described in the introductory chapters, **RFID** technologies rapidly evolved in the past few years and have already become omnipresent. Along with this trend grows the necessity for secure communication and authentication. **RFID**-based applications, such as electronic passport, payment systems, car immobilizers, or access control systems, require strong cryptographic algorithms and protocols, as privacy and authenticity of the transmitted data are crucial for the system as a whole.

Severe weaknesses have been discovered in the “first generation” of **RFID**s (compare with Sect. 1.2.1) that rely on proprietary ciphers [**Plö08**, **EKM⁺08**, **CNO08**, **GdKGM⁺08**], such as Mifare Classic contactless smartcards (successfully attacked in Chap. 6) and KEELOQ rolling code products (fully broken in Chap. 8). Current contactless systems for sensitive areas with very high security and privacy demands tend to employ stronger cryptographic primitives. Likewise, antiquated contactless systems relying on weak products will be upgraded to more secure substitutes in order to prevent fraud and malfunction. In general, new systems are more

and more based on state-of-the-art cryptography. This trend can already be observed, as several contactless products exist and become widespread that provide secure ciphers, e.g., (3)DES or AES.

The previous chapter demonstrated severe attacks on a payment system using weak Mifare Classic cards. As a consequence, the system integrator might believe that replacing the weak Classic cards against the more secure Mifare DESfire cards, while the other parameters of the system (identical keys in all cards) remain unchanged, will result in a sufficient level of security for the payment system. We demonstrate in the following that this is not a good idea, because it is possible to extract secret keys from Mifare DESfire cards.

The aim of this chapter is to evaluate the security of these contactless smartcard solutions that are believed (and advertised) to be highly secure. Since encryption is performed using well-known and carefully reviewed algorithms, cryptanalytical attacks on the algorithmic level are unlikely to be found. Thus, we aim at performing an SCA which exploits the physical characteristics of the actual hard- or software implementation of the cipher. In the course of the analyses of Mifare DESfire cards, the first successful key-recovery attack targeting the 3DES implemented on these RFID devices is accomplished. The attack presented can be regarded as a proof-of-concept and is very likely applicable to other comparable cryptographic RFIDs in a similar fashion.

7.1.1 Related Work about Side-Channel Analysis of RFIDs

Oren and Shamir [OS07, OS] presented a successful side-channel attack against so-called Class 1 EPC tags operating in the UHF frequency range which can be disabled remotely by sending a secret “kill password”. Small fluctuations in the reader field during the communication with the tag allow to consecutively deduce the correct password bits. However, the very limited type of RFID tag does not offer any cryptography.

At the CHES 2007, Hutter et al. [HMF07] performed an EM attack on their own AES implementations on a standard 8-bit microcontroller and an AES co-processor in an RFID-like setting, i.e., the self-made devices are powered passively and brought into the field of a reader. On their prototype devices the antenna and analogue frontend are separated from the digital circuitry, while on a real RFID tag, these components are intrinsically tied together. A trigger signal is generated by the DUT before the attacked S-Box operation to ensure perfect time alignment. Moreover, the clock signal for the digital circuitry is generated independently from the field of the reader using an external oscillator, hence the carrier of the reader is uncorrelated with the power consumption of the AES and can be easily removed. In contrast to their work, we face the real-world situation, i.e., we have no knowledge about the internal implementation details of the unmodified contactless smartcard to be attacked. We analyze a commercial chip with all RFID and cryptographic circuitry closely packed on one silicon die and get no help like artificial triggering for the alignment.

7.1.2 Outline of this Chapter

Section 7.2 introduces Mifare DESfire cards and illustrates their authentication protocol. Details about power analysis in the context of RFIDs, the specialized measurement setup, filtering techniques, and a recursive approach of computing a CPA follow in Sect. 7.3. Next, all relevant steps to analyze an unknown RFID device in practice are described: The extensive profiling

that is required to gain first insights into the operation of the contactless “black box”, the positioning on the probes, and the processing of the digitized side-channel signals are detailed in Sect. 7.4. The results of the full-key recovery by means of EM analysis are finally given in Sect. 7.4.5.

7.2 Mifare DESfire

The DUT in this chapter, termed Mifare DESfire (MF3 IC D40) by its manufacturer NXP, is an ISO 14443-compliant RFID card (cf. Sect. 2.5) operating at 13.56 MHz. It features a challenge-response authentication protocol which relies on 3DES in Encrypt-Decrypt-Encrypt (EDE) mode as described in [Nat99], using the two 56 bit halves of $k_C = k_1 || k_2$ as the symmetric key. The 4 kB of memory on the card can hold up to 28 different so-called applications which can be regarded as folders of a file system. Each of these applications consists of up to 16 files that can be secured with up to 14 different keys. Each card further possesses a master key that defines general access conditions, e.g., whether a list of the applications can be obtained with or without a previous authentication. Note, that some of the information about Mifare DESfire summarized in the following (especially with respect to the cryptography employed) was only available *after* our analyses in Sect. 7.4.

7.2.1 Mifare DESfire Authentication Protocol

An initial rough description of the Mifare DESfire protocol by Carluccio [Car05] has been modified and corrected according to our findings, yielding the mutual authentication scheme described next. Prior to an authentication, an application is selected by its AID.

The reader then starts the authentication in Protocol 3 with an authenticate command specifying the key number to be used. The DESfire card responds with an encrypted 64-bit random nonce n_C . The reader decrypts the received n_C and rotates it eight bits to the left to obtain r_1 . Then, the reader chooses a 64-bit random nonce n_R and *decrypts* n_R to obtain b_1 . The decrypted nonce of the reader¹, b_1 , and the rotated nonce of the card, r_1 , are XORed and then decrypted, resulting in b_2 . Next, the computed values b_1 and b_2 are sent to the card.

The card encrypts b_1 to obtain the nonce of the reader, n_R , rotates the latter eight bits to the right, and stores the result in r_3 . Then, it encrypts b_2 and performs an XOR of the result with b_1 , revealing the received card nonce n'_C after a rotation by eight bits to the right. The card verifies, if the received n'_C equals the initially generated n_C , and aborts the protocol in case of a mismatch.

In case of a successful comparison, the card encrypts r_3 to obtain b_3 which is passed to the reader. The rotated and encrypted nonce is likewise verified by the reader, and both parties are mutually authenticated in case this final step is completed successfully. After a successful authentication, the subsequent communication is encrypted with a session key. Note that Mifare DESfire cards only perform (3)DES encryptions $Enc_K(\cdot)$ employing the secret key K and rotations to the *right*, while DESfire readers always use (3)DES decryptions $Dec_K(\cdot)$ and rotate the values to the *left*.

¹Since the ciphertext of n_R is obtained by means of a decryption, it can be deciphered by means of an encryption.

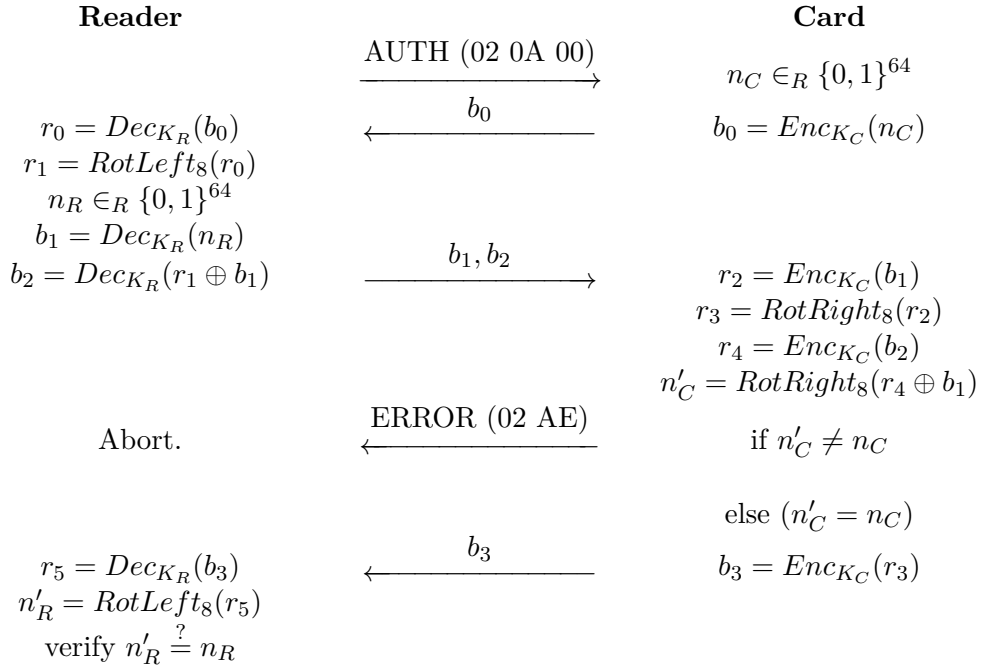
**Protocol 3.** The Mifare DESfire authentication protocol.

Table 7.1 illustrates the exact binary representation of the commands as required for the implementation of the authentication protocol, according to the message format specified in ISO 14443A Part 4 (including the 16-bit CRC).

Table 7.1: Messages exchanged during an authentication protocol run between a reader R and a Mifare DESfire card C.

#	Direction	Protocol Message	Explanation
1	R → C	02 0A, 1 byte, CRC1 CRC2	Auth key number CRC
2	C → R	02 AF, 8 byte, CRC1 CRC2	Card nonce b_0 CRC
3	R → C	03 AF, 8 byte, 8 byte, CRC1 CRC2	Rdr. response b_1 b_2 CRC
4	C → R	03 00, 8 byte, CRC1 CRC2	Success b_3 CRC

7.3 Methods, Models, and Setup for the EM Analysis of RFIDs

A non-invasive SCA of an RFID device cannot be performed by directly measuring its electric current, since the electronic circuitry is isolated in the plastic packaging and hence its power supply cannot be tapped (cf. Sect. 3.2.2). Instead, the information-carrying EM emanations of the DUT are captured by means of near-field probes. Unfortunately, there is another powerful

EM field present that strongly disturbs the measurements, i.e., the field generated by the **RFID** reader. Due to this noise, for an **EM** analysis of **RFIDs** typically a very large amount of measurements is required, compared to acquiring the power consumption in the traditional way via a resistor. A successful key-recovery by means of **CPA** from only ten measurements, as demonstrated later in this thesis in Chap. 8.3, is considered infeasible in the context of **RFIDs**. For a practical, efficient attack, thus special evaluation methods for processing millions of traces are required.

This section presents an efficient method of performing a recursive **CPA**, dedicated for processing a large amount of traces. Next, models describing the power consumption of **RFID** tags are introduced and the measurement setup, including various analogue filters for removing the carrier signal, is illustrated. Finally, the software developed for the processing of the side-channel measurements is detailed.

7.3.1 Efficiently Implementing a CPA

Straightforward implementations of a **CPA** (see Sect. 3.2.2 for a detailed explanation and the used notation) read all D traces, each with a length of T samples, into memory before calculating the correlation coefficient $r_{j,l}$ (see Eq. 3.9 in Sect. 3.2.2). This traditional approach is computationally demanding for long traces and/or a large amount of measurements, e.g., $D = 10,000$ traces with $T = 350,000$ data points (stored as 4 byte single precision values) consume approx. 13 GB of memory. In some practical scenarios, millions of traces need to be processed. Therefore, a *recursive* computation of $r_{j,l}$ becomes attractive. Instead of first reading and then processing all data, existing values of the correlation coefficient can be updated with every new trace. The recursive **SCA** technique detailed in the following is based on an algorithm given in [Knu98], originally proposed by Welford.

The sample mean $\bar{t}_l$ and the sample variance S_t for a finite set of measurements are updated in the i^{th} step of the recursion according to

$$\bar{t}_l^{(i)} = \bar{t}_l^{(i-1)} + \frac{t_{i,l} - \bar{t}_l^{(i-1)}}{i} \quad \text{with } \bar{t}_l^{(0)} = 0 \quad (7.1)$$

$$s_t^{(i)} = s_t^{(i-1)} + \left(t_{i,l} - \bar{t}_l^{(i-1)} \right) \left(t_{i,l} - \bar{t}_l^{(i)} \right) \quad \text{with } s_t^{(0)} = 0, \quad (7.2)$$

where $t_{i,l}$ denotes the i^{th} data point of measurement l and $s_t^{(i)}$ is a helper value to derive the sample variance after i measurements as $S_t^{(i)} = s_t^{(i)} / i - 1$. Applying this idea for recursively computing the correlation coefficient of a key candidate, it suffices to keep track of T trace means $\bar{t}_l$ and helper values s_t .

Analogously, $\bar{h}_j$ and s_h are updated, however, these are independent of the data point l and thus need to be stored only once. For evaluating Eq. 3.9, the helper variable $c_{j,l} = \sum_{i=1}^D h_{i,j} \cdot t_{i,l} / D - 1$ is stored for T points in time and is updated according to

$$c_{j,l}^{(1)} = t_{1,l} \cdot h_{1,j} \quad i = 1 \quad (7.3)$$

$$c_{j,l}^{(i)} = c_{j,l}^{(i-1)} + \frac{h_{i,j} \cdot t_{i,l} - c_{j,l}^{(i-1)}}{i - 1} \quad i \geq 2. \quad (7.4)$$

$$(7.5)$$

Finally, the correlation coefficient $r_{j,l}$ after i measurements, $1 \leq i \leq D$, is computed as

$$r_{j,l}^{(i)} = \frac{(i-1) \cdot c_{j,l}^{(i)} - i \cdot \bar{h}_j \cdot \bar{t}_l}{\sqrt{s_t^{(i)} \cdot s_h^{(i)}}}. \quad (7.6)$$

Advantages of the Recursive CPA Approach

The method requires to store $\mathcal{O}(D)$ values for each key candidate k_j^* . In contrast, the traditional two-pass method, i.e., read all traces before processing them, needs $\mathcal{O}(T \cdot D)$ memory. Thus, for large T , the memory footprint of the above described computations remains *constant*, while a straightforward algorithm becomes infeasible.

The recursive correlation approach further allows to compare the effort required for the **SCA** of different implementations or different measurement setups: As a measure for the efficiency of a key-recovery by means of **SCA**, often the minimal amount of traces, D_{min} , that need to be processed to reach a certain correlation magnitude is considered. To determine D_{min} via the traditional evaluation, all matrix computations detailed in Sect. 3.2.2 have to be carried out repeatedly for each new guess for D_{min} , to find the corresponding magnitude of the correlation. Determining D_{min} is thus extremely slow using the standard methods and is likely to become infeasible for a large amount of traces. Using the recursive approach detailed above, the correlation coefficient is updated with each trace, hence the minimal amount of traces for a successful attack can be precisely determined “for free”, i.e., without the significant computational overhead that occurs by repeatedly applying the traditional method.

7.3.2 Modeling the Power Consumption of RFID Devices

As sketched in Sect. 2.4, the energy is supplied to the **IC** of a contactless card via its (rectangular) antenna coil that is also embedded in the plastic card. In return, the power consumption of the **IC** has a direct effect on the voltage induced in the antenna coil. The latter is (loosely) coupled to the antenna of the **RFID** reader. The side-channel information emanated by the **IC** can hence be measured by means of **EM** probes positioned directly above the surface of the **IC** or positioned close to the windings of the card’s antenna. As a third option the antenna coil of the reader can be tapped directly, which is less promising, since some information leakage will be lost due to the relatively weak coupling between the antennae.

Remote Power Model

For a simple model of the frequencies the **EM** leakage occurs, consider a band-limited power consumption $p(t)$ that directly affects the amplitude of the carrier frequency $\omega_0 = 2\pi \cdot 13.56$ MHz. Similar to communicating by means of load modulation (cf. Sect. 2.5.3) the amplitude of the **RF** field generated by the reader will be different when the chip requires more energy compared to an instant when no energy is consumed.

This may result in detectable frequency components in the side-bands of the carrier, as depicted in Fig. 7.1, that contain data-dependent information. Equation 7.7 describes the corresponding relation between the amplitude-modulated carrier $x(t) = p(t) \cos(\omega_0 t)$ at ω_0

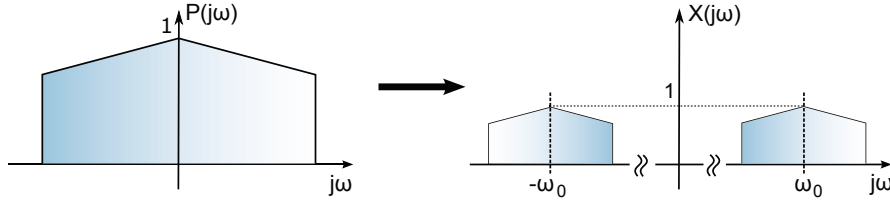


Figure 7.1: Frequency spectrum of the band-limited power consumption of a card (left) and the assumed spectral information leakage for the remote power analysis model (right).

and its spectral information leakage $X(j\omega)$, where $\circ\bullet$ denotes the Fourier transform² and $P(j\omega)$ denotes the frequency spectrum of $p(t)$, i.e., $p(t) \circ\bullet P(j\omega)$.

$$x(t) = p(t) \cos(\omega_0 t) \circ\bullet X(j\omega) = \frac{1}{2} (P(j\omega - j\omega_0) + P(j\omega + j\omega_0)) \quad (7.7)$$

We refer to this approach as *remote power model*, as the fluctuations in the power consumption of the device are modulated via the antennae onto the strong carrier signal of the reader and might thus be detectable even in the far field. Despite its name, the remote power model is also the most adequate model when acquiring the EM leakage directly at the antenna of the card.

Additive Power Model

Another model assumes that the leakage of the internal switching of transistors on the gate-level can be detected by means of near-field probes that are positioned directly above the surface of the chip. In this case, the bits of a secret key might be extracted from signals in frequency bands that are independent of the carrier of the reader.

The respective signal model in Eq. 7.8 is additive. $P(j\omega)$ corresponds again to a band-limited signal with a center frequency ω_s . This time, the leakage is independent of the carrier frequency ω_0 , as illustrated in Fig. 7.2. The model is most likely applicable when measuring directly on the surface of the silicon die. Note, that these on-surface measurements usually require to dismantle the chip from the plastic card and position a probe extremely close to its surface.

$$x(t) = p(t) + \cos(\omega_0 t) \circ\bullet X(j\omega) = P(j\omega) + \frac{1}{2} (\delta(j\omega - j\omega_0) + \delta(j\omega + j\omega_0)) \quad (7.8)$$

7.3.3 Measurement Setup and Analogue Filters

The introduced models for the EM leakage make clear, that the best possible suppression of all other frequency components — in particular the carrier frequency — is desirable. This is taken into account by the measurement setup introduced next.

The core of the employed EM measurement equipment for RFIDs, illustrated in Fig. 7.3, is a standard PC that controls the data acquisition and the communication via the freely

²The Fourier transform is commonly used to transform signals from the time domain into the frequency domain.

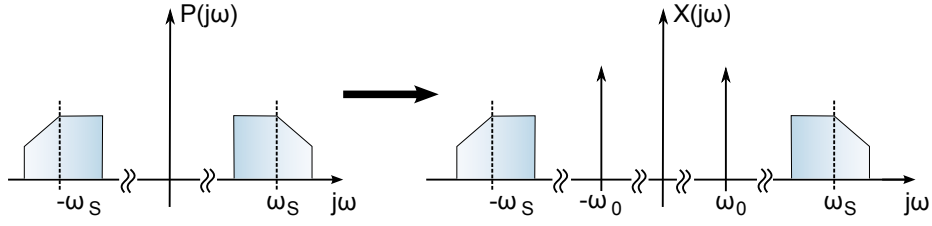


Figure 7.2: Frequency spectrum of the power consumption (left) and the spectrum of the information leakage, with a carrier signal at ω_0 , for the additive model.

programmable reader for contactless smartcards introduced in Sect. 4.1.1. Furthermore, a specially developed circuit for analogue preprocessing of the signal and a suitable EM probe are essential components to obtain the traces for our non-invasive key-recovery attack.

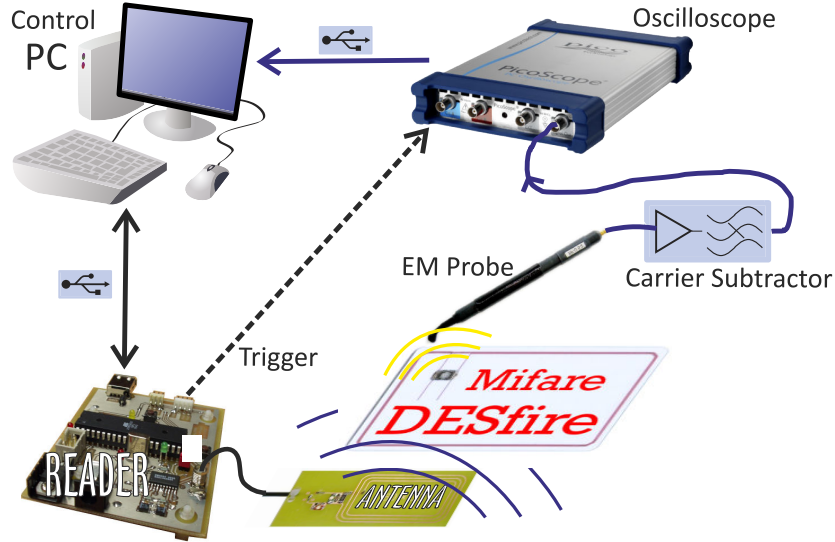


Figure 7.3: Measurement setup for analyzing contactless smartcards (here Mifare DESfire): The communication and timing control is established with the self-built RFID reader (left bottom), while the data acquisition of the EM emanations of the card is accomplished by means of a near-field EM probe and a USB oscilloscope (right top).

For measuring the EM-field emanated by the contactless smartcard, various (self-made and commercial) near-field probes are available (see Sect. 4.3.2). The measured signal is boosted with the *PA-303* amplifier described in Sect. 4.3.3 and processed by the analogue circuitry described next. Finally, the power trace is digitized with the USB-oscilloscope detailed in Sect. 4.3.1.

Analogue Filters

Although to our knowledge there exist no reliable estimations about the exact amplitude of the EM emanations caused by digital circuitry — especially when attacking an unknown implemen-

tation — the unintended emanations of the chip are expected to be orders of magnitude smaller, compared to the strong field generated by the reader for the energy supply of a contactless card.

The quantization error induced by the ADC of the oscilloscope constitutes a minimum boundary for the achievable SNR, depending on the number of bits used for digitizing an analogue value. Each bit improves the SNR by about 6 dB [Hay83]. Thus, for the best SNR the full input scale should be utilized for the signal of interest, implying that a maximum suppression of the carrier frequency and a subsequent amplification of the small side-channel information must already take place in the analogue domain, *before* the digitizing, as a preprocessing step.

For minimizing the disturbing influence of the carrier frequency on the measurements, several prototypes of active and passive analogue filters have been built and evaluated, as described in the following. The PCBs of the filters include custom (non-)inverting amplifiers realized with standard operational amplifier (opamp) circuits which can be combined with the commercial amplifier from Sect. 4.3.3 for higher amplification. The utilized AD8012 opamp is specified for a small signal bandwidth of 350 MHz.

Passive RLC Notch Filter A notch filter suppresses only the signals at its center frequency $\omega_0 = 2\pi f_0$ and ideally does not affect the remaining frequency spectrum [Voc04, Car06]. Hence, this type of filter appears to be suitable for removing the unwanted carrier signal of the reader. Several papers suggest that for high frequencies (> 1 MHz), only passive notch filters consisting of resistors (R), inductances (L) and capacitances (C) provide sufficient performance [Kug03, Car06]. Therefore, a passive 2nd order circuit has been realized on a PCB. The transfer function of this filter is [OKA08]

$$H_{notch}(s) = \frac{s^2 + \omega_0^2}{s^2 + 2\omega_0\zeta s + \omega_0^2},$$

where $s = j\omega = 2\pi f$. The notch *center frequency* ω_0 and the *dampening ratio* ζ (as a function of the *quality factor* Q) can be expressed in terms of the passive components as

$$\omega_0 = 2\pi f_0 = \frac{1}{\sqrt{LC}}, \quad \zeta = \frac{1}{2Q} = \frac{1}{2}R\sqrt{\frac{C}{L}}.$$

The quality factor Q is defined as the ratio of the energy stored and the energy dissipated in a system. It is related to the bandwidth B according to $Q = f_0/B$. In the context of filters a high quality factor hence corresponds to a narrow bandwidth. The -3 dB bandwidth, i.e., the distance between ω_0 and the point where the transfer function has dropped by 3 dB, is $B = \frac{R}{2\pi L}$.

For realizing this filter, the values $L = 0.56 \mu\text{H}$, $C = 226 \cdots 250$ pF, and $R = 39 \Omega$ have been chosen, where ω_0 can be tuned by the adjustable capacitor C . The resulting -3 dB frequencies $f_{0,min}$ and $f_{0,max}$, describing the range of the notch filter, and the bandwidth B are

$$f_{0,min} = 13.45 \text{ MHz}, \quad f_{0,max} = 14.13 \text{ MHz}, \quad B = 11.08 \text{ MHz}.$$

At a first glance, the bandwidth appears rather large for narrow-band filtering. However, B refers to the distance between the -3 dB points and the bandwidth can be reduced by increasing Q , i.e., selecting smaller values for R . Figure 7.4 plots the magnitude response of the circuit in dBV (0 dBV correspond to 1 V), tuned for maximum rejection of the center frequency $f_0 = 13.56$ MHz.

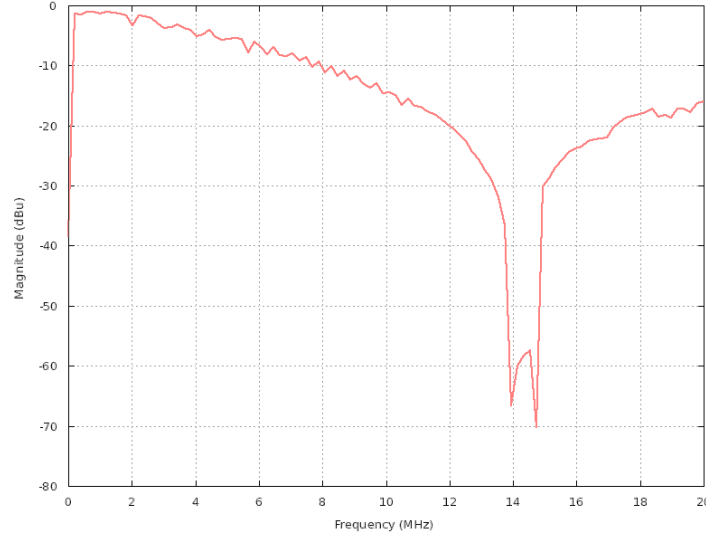


Figure 7.4: Measured frequency (in MHz) vs. magnitude (in dBU) response of the RLC notch filter.

Passive Twin-T Notch Filter As a second passive filter architecture a Twin-T filter, i.e., a second-order notch only consisting of capacitors and resistors, has been realized. Its center frequency is given as $f_0 = 1/2\pi RC$, with a fixed quality factor $Q = 0.25$. Due to this relatively low quality factor, the rejection performance is worse compared to the RLC circuit. Besides, to adjust f_0 , two components values have to be changed simultaneously, making tuning more difficult. Although the filter can be extended to an active variant with modifiable Q , the passband gain cannot be set independently.

Active Wien-Robinson Notch Filter An active Wien-Robinson notch has been adopted from [Kug03]. It consists of a passive, differential-output bandstop filter integrated into the feedback path of an amplifier circuit, allowing for a high quality factor Q . The utilized opamp is an AD8058 325 MHz voltage feedback amplifier [Ana03]. All parameters (center frequency, quality factor, amplification in passband) can be adjusted independently. The theoretical transfer function is

$$H_{WienRobinson}(s) = -\frac{\frac{\beta}{1+\alpha}(s^2 + \omega_0^2)}{s^2 + \frac{3}{1+\alpha}\omega_0 s + \omega_0^2},$$

with $\alpha = \frac{R_2}{R_3}$, $\beta = \frac{R_2}{R_4}$, $\omega_0 = \frac{1}{RC}$, passband gain $A_0 = -\frac{\beta}{1+\alpha}$ and $Q = \frac{1+\alpha}{3}$.

Testing the Filters The active architecture and the passive RLC notch have been realized for testing purposes on a PCB. The schematics are illustrated in Appendix I.3.

Our practical tests with the active filter confirms the statements made in [Car06]: the circuit is unstable and prone to oscillate at a multiple of the center frequency. However, for lower frequency systems (e.g., 125 kHz RFID tags), the Wien-Robinson structure can still be of interest, due to its flexibility with respect to the filter parameters.

The passive filter runs stable, but due to its high bandwidth also dampens frequencies of interest that are close to the carrier frequency. It may be suitable to improve measurements in case of an additive power model, but it is not optimal for a remote power model (see Sect. 7.3.2).

Carrier Subtractor

As a consequence of the stability problems with the active architecture and due to the fact that notch filtering would also remove side-channel information nearby the carrier frequency, another method for removing the carrier is developed in the following. The straightforward, inexpensive technique turns out to be most effective in bypassing the influence of the field of the reader.

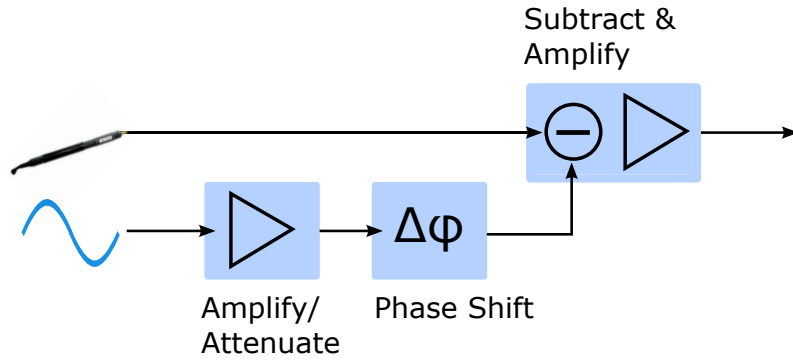


Figure 7.5: Principle of subtracting the unwanted carrier frequency of the reader from the measured side-channel signal.

A part of the analogue front-end of the RFID reader detailed in Sect. 4.1.1 is a crystal oscillator generating an almost pure sine wave with a frequency of 13.56 MHz that serves as the source for the field transmitted to the contactless smartcard. The basic idea of the developed carrier subtractor is to tap the oscillator of the reader and subtract its (sine) signal from the EM emanation acquired by the EM probe.

The sine wave of the reader has a constant amplitude and a constant shift in time with respect to the signal acquired with the EM probe. Hence, the developed analogue circuitry is capable of delaying and scaling the sine wave of the crystal, as shown in Fig. 7.5. After matching the amplitude and phase of the latter to that of the EM measurements, the pure sine wave is subtracted from the EM measurements. This approach, based on a low-cost analogue circuit employing operational amplifiers, allows to significantly suppress the unwanted signal component while keeping the possibly interesting variations. The analogue preprocessing unit can also be used for other types of RFIDs, such as 125 kHz transponders in car immobilizers, when combined with the 125 kHz reader from Sect. 4.2.

Phase Shifter The phase-shift is performed by an adjustable *allpass filter*, which does not alter the amplitude of the processed signal, but only its phase, depending on the center frequency ω_g . Its transfer function is

$$H_{allpass}(s) = \frac{\omega_g - s}{\omega_g + s},$$

with $\omega_g = 1/RC$. The magnitude response $|H_{allpass}(s)| = \sqrt{\omega_g^2 + \omega^2 / \omega_g^2 + \omega^2} = 1$ is constant, while the phase is given as $\varphi(\omega) = \arctan -2\omega\omega_g / \omega_g^2 - \omega^2$. Figure 7.6 illustrates the resulting phase shift for different values of $f_g = \omega_g / 2\pi$.

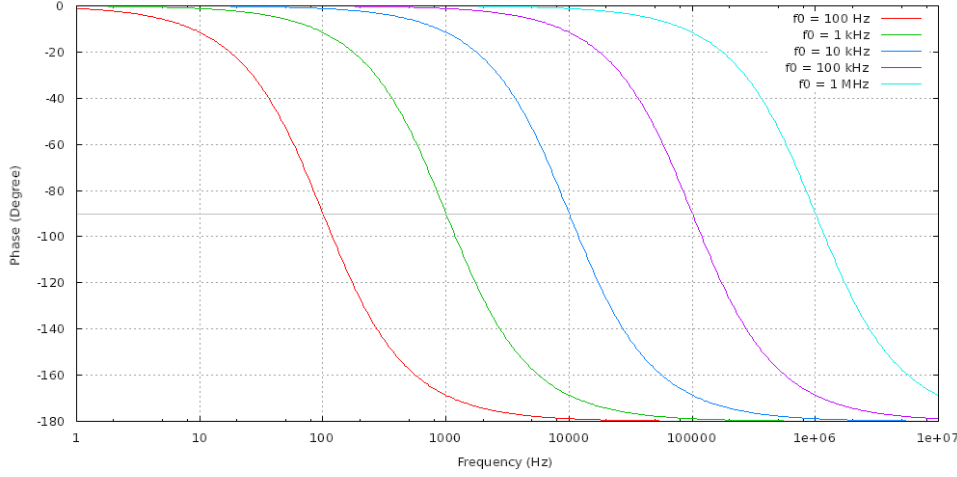


Figure 7.6: Phase response of a first-order allpass filter as a function of the frequency, for varying center frequencies f_g .

By varying the value of C (or R) and thus shifting f_g , a fixed-frequency sine can be delayed by an adjustable amount. For the chosen component values in Fig. 7.7, the minimum and maximum center frequencies are $f_{g,min} = 1.96$ MHz and $f_{g,max} = 9.82$ MHz. The respective phase shifts of the 13.56 MHz carrier are $\varphi_{min} = -24.78^\circ$ and $\varphi_{max} = -95.37^\circ$. Around the center frequency, the phase shift is almost linear, so that frequency components in this range are subject to the same time shift — the *group delay* $\tau_g = -\partial\varphi/\partial\omega$ remains almost constant.

Effects of Phase and Amplitude Mismatch In the case of not perfectly phase- and amplitude-synchronous signals, the subtraction yields a scaled and phase shifted wave

$$v(t) = \sin(\omega_c t) - b \sin(\omega_c t + \Delta\varphi) = c \sin(\omega_c t + \Delta\psi)$$

with

$$c = \sqrt{1 + b^2 - 2b \cos \Delta\varphi} \text{ and } \Delta\psi = \arctan\left(\frac{-b \sin \Delta\varphi}{1 - b \cos \Delta\varphi}\right).$$

For $b \approx 1$, i.e., signals with an almost matched amplitude, one obtains a result of

$$v(t) = -2 \sin\left(\frac{\Delta\varphi}{2}\right) \cos\left(\omega_c t + \frac{\Delta\varphi}{2}\right),$$

which corresponds to a growing attenuation when the phase difference is reduced.

Final Realization of the Carrier Subtractor

The complete circuit of the subtracter, based on an opamp adder with prior inversion, is illustrated in Fig. 7.7. It features a single-supply architecture, enabled by a circuit providing a

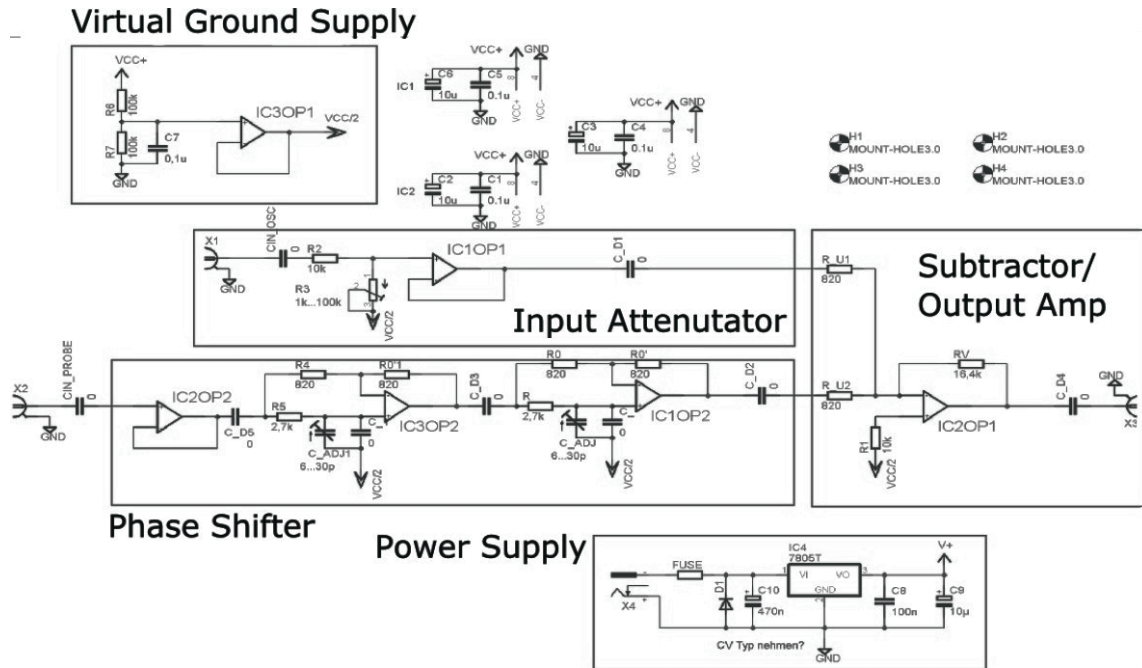


Figure 7.7: Phase shift and addition circuit

virtual ground at half the supply voltage $V_{CC}/2$. A flexible adjustment of amplitude and phase is provided by the input attenuator and the phase shifter, respectively. The gain of the output amp of the subtractor is fixed to a factor of 10, i.e., 20 dB.

During our experiments, the subtractor circuit turned out to be highly suitable for analogue pre-filtering of the carrier. The circuit is used for the measurements and the key recovery in this chapter. Before acquiring traces, phase and amplitude are adjusted according to the respective position of the probe, until the digitized signal has minimal amplitude, i.e., the influence of the carrier is removed optimally.

7.3.4 Software for the Power Analysis of RFIDs

Several PC-side programs have been developed for executing a CPA with pre-recorded traces and for conducting an on-line CPA that makes use of the recursive computation of the correlation coefficients introduced above in Sect. 7.3.1. These and various auxiliary functions for the pre- and post-processing of the acquired signals, programmed in C or C++, are summarized next.

Controlling Communication and Data Acquisition

The control part of the software sets up the oscilloscope and handles the RFID protocol. The program first activates the card, identifies its UID and then initializes the ISO 14443-4 protocol. The oscilloscope is activated and prepared for data collection, before an authentication with a chosen (random) plaintext for b_2 (see Sect. 7.4.1) is performed. The EM emanations during the encryption are acquired and either stored to the harddisk for further processing or used to update the live-CPA, depending on the operation mode.

Trace Alignment

Both during the profiling of the **DUT** and the subsequent key-recovery attack, proper time synchronization of the traces is essential. Measurements that were started by a trigger signal often need to be precisely (re-)aligned in the **PC**. Two general methods have been implemented for aligning the **EM** emanations acquired from an **RFID** device.

One option is a *carrier phase* alignment with respect to the phase of the field generated by the reader. This approach assumes that the internal DESfire clock is frequency- and phase-synchronous to the field of the reader and that an encryption always starts at the same number of clock cycles after receiving the challenge. The alignment is performed using the normalized cross-correlation between the trace data and a short reference sine signal at 13.56 MHz. The offset is then computed as the position of maximum correlation, i.e., the point where the signals are in phase.

As a second option, the measurements can be aligned with respect to the answer of the card, i.e., to the beginning of the *load modulation*. The synchronization is performed by locating the position of the first bit of the load-modulated answer sent by the smartcard. This is implemented using a peak-detection that finds the first time instant in which the amplitude exceeds a threshold.

Signal Processing

Several digital signal processing algorithms have been implemented for the **EM** analysis. The basic routines include the computation of mean, variance, maximum, normalized cross correlation, and (inverse) Fourier transformation (following [FJ07]) of a discrete timeseries. Further digital filters that have been realized for processing the side-channel measurements include

- a *moving average* Finite Impulse Response (**FIR**) filter with an adjustable window width, which realizes a form of low-pass filtering (cf. [Smi97a]) by averaging over the samples contained in the window without output feedback;
- a *Chebyshev low-pass filter*, i.e., an Infinite Impulse Response (**IIR**) filter [KK06], providing a definable cut-off frequency and passband ripple according to [Smi97b] — due to its recursive nature, stability is an issue with this filter, so care has to be taken when selecting the number of poles together with the cut-off frequency;
- an **FIR** low-pass filter [Smi97c] that has been realized to overcome stability problems with Chebyshev filters — since for sharp band separation a relatively high order is required, computational costs are higher than for **IIR** versions, if the filtering is done via time-domain convolution (and not by multiplication in the frequency domain);
- *digital notch filters* allowing to suppress certain frequencies, e.g., a 4th order notch filter with adjustable center frequency and quality factor has been implemented according to [Bou00].

Precomputing Special Plaintexts for Attacking DES

For profiling with a known key, special plaintexts that target the input of a single S-Box in the first round of **DES** can be precomputed. The aim is to generate chosen challenges in such a way,

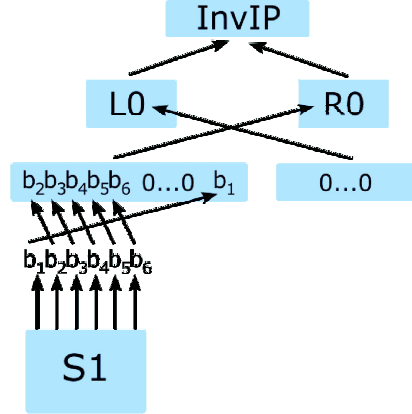


Figure 7.8: Path for deriving a special DES plaintext that produces a given S-Box input pattern.

that all input patterns occur for this S-Box. At the same time, the inputs of the other S-Boxes are held constant where possible, to minimize their influence on the side-channel measurements. Performing power analyses with these special input patterns for different S-Boxes can reveal details about the implementation, e.g., the order in which the S-Boxes are executed.

The basic procedure is outlined in Fig. 7.8 for the first S-Box. A chosen 6-bit pattern for the input of the S-Box is “de-expanded” to 6 bits at the indices 32, 1, 2, 3, 4, 5 before **XOR**ing the (known) round key (omitted in the picture). This yields the **DES** register R_0 in which all other bits that are not dependent on the selected S-Box input are set to zero. The register L_0 is filled with only zeros and both registers are passed through the inverse initial permutation (InvIP) to obtain a plaintext that generates the desired bit pattern. Similarly, other parts of the cipher can be targeted with special, chosen plaintexts.

7.4 A Real-World EM Attack on Mifare DESfire MF3ICD40

Next, we give details on each step of the power analysis, starting from pinpointing the time instant for the attack, filtering the signal in the analogue domain, digitizing the measurements, and finally processing the data in the digital domain. The process of performing an **EM** key-recovery attack on DESfire cards (compare with the general steps of a black-box analysis given in Sect. 3.3) is structured as follows:

- (1) Establish the communication with the card (Sect. 7.4.1) and perform a pre-analysis of the behavior of the card (Sect. 7.4.2).
- (2) Choose a suitable **EM** probe and optimize its position (Sect. 7.4.3).
- (3) (Digitally) filter and align the measurements (Sect. 7.4.4).
- (4) Profile the device and locate the **3DES** encryption (Sect. 7.4.5).
- (5) Perform the **EM** analysis of the **3DES** and recover the secret key (Sect. 7.4.5).

7.4.1 Communicating with the DUT

Rather than implementing high-level functionality directly on the μC of the **RFID** reader, it only executes low-level operations, including demodulation and Manchester decoding, as well as Miller encoding and modulation. All binary data to be sent is formatted on the measurement **PC**. Likewise, responses sent by the card are returned in raw binary format to the **PC**, where the actual interpretation takes place.

We have implemented the entire authentication protocol (see Protocol 3 in Sect. 7.2.1), but for the **CPA** attack focus on the second message sent by the reader, as depicted in Fig. 7.9: Both challenges b_1 and b_2 with a length of 64 bit are encrypted with a function $\text{Enc}_{K_C}(\cdot)$ that is supposedly a (3)**DES**³. The **SCA** targets only the encryption of b_2 .

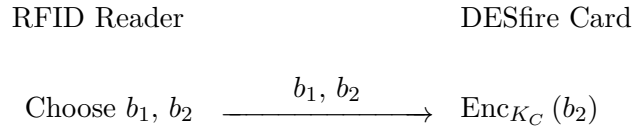


Figure 7.9: Simplified excerpt of the authentication protocol, as required for an EM attack.

During the profiling, the (default) keys of the blank Mifare DESfire serving as the **DUT** are known and can hence be set to any chosen key. A known key is extremely helpful for profiling a black-box implementation, since the intermediate values of the cipher can be precomputed. By correlating on these precomputed values, various details about the implementation can be revealed, as shown in Sect. 7.4.5.

By performing a full authentication, and reproducing the responses of the Mifare DESfire by computing a **3DES** on the **PC**, we verify that a standard (3)**DES** [Nat99] cipher is used for the encryption of the challenge b_2 in Fig. 7.9. The **3DES** encryption involving the key $k_C = k_1 || k_2$ takes place according to $\text{Enc}_{K_C}(\cdot) = \text{DES}_{k_1}(\text{DES}_{k_2}^{-1}(\text{DES}_{k_1}(\cdot)))$. Note that, for choosing $k_1 = k_2$ the **3DES** encryption equals an encryption with standard **DES**.

Some practical experiments reveal, that the protocol aborts always *after* the encryption of b_2 , even in case its verification was not successful. We thus observe that the card unconditionally encrypts any value b_2 and we can freely choose the plaintext. For the **CPA** described in Sect. 7.4.5, we send random, uniformly distributed plaintexts for b_2 .

7.4.2 Basic Profiling

After acquiring some measurements with the setup detailed in Sect. 7.3.3, the profiling of the **DUT** continues, in order to gain more insights of the unknown implementation. The behavior of the DESfire card is analyzed by observing the response time of the card and inspecting the acquired **EM** traces. A **DOM** test introduced in Sect. 7.4.2 allows to estimate if and in which time window information leakage related to the **DES** hardware can be expected.

³The name “DESfire” and the advertisement of the manufacturer suggests that 3DES is implemented.

Measuring the Response Time

The time elapsed between sending the *same* challenge b_2 in the authentication protocol and the first bit of the answer of the card has been measured 80 times, with the key of the DESfire card set to all zeros. Two almost discrete response times of $376\ \mu\text{s}$ and $452\ \mu\text{s}$ can be identified.

One might suspect that this behavior is related to the timing requirements specified in the ISO 14443 (see Sect. 2.5.5), more precisely the **FDT**. However, the expected time variation due to the **FDT** equals $84-20/13.56\text{ MHz} = 4.72\ \mu\text{s}$, while the measured timing exhibits a deviation of $\approx 76\ \mu\text{s}$. Instead, it seems probable that the internal clock of the DESfire is the cause: the difference of $T = 76\ \mu\text{s}$ corresponds to a clock frequency of $f = 1/T \approx 13\text{ kHz}$. The internal clock of the contactless smartcard might accordingly be derived from the carrier frequency of the reader, through dividing the frequency by a factor of approx. 1000.

Besides, the minimal waiting time for *resetting the card* by switching off the carrier has been determined: our experiments indicate that after 20 ms all volatile memories have been cleared. In principle, shorter intervals around 10 ms are possible; however, this sometimes results in an incomplete reset, causing the card not to answer at all⁴. After switching the field back on, a delay of about 50 ms needs to be obeyed before a transmitted **REQA** or **WUPA** command is accepted as valid.

Difference-of-Means Test

The **DOM** test according to Sect. 3.3.2 was performed with 13,000 measurements in each set. The two challenges were selected to target only the six input bits of S-Box 1, as explained in Sect. 7.3.4, and to yield a single bit difference in the output of the S-Box: $0x5$ at the input gives $0x7$ at the output and $0xF$ results in $0x3$. Note that for investigating the supposedly stronger effect of multiple bit changes a **DOM** test can be conducted with challenges producing $0x1$ and $0x12$ at the input of the S-Box, corresponding to the outputs $0x0$ and $0xF$, respectively.

For aligning the traces, two alternatives have been examined: alignment to the first bit of the load modulation and to the phase of the reader field. The latter approach always led to higher amplitudes in the **DOM** test and is hence applied in the following.

Figure 7.10a shows that significant, challenge-dependent behavior can be identified in the time window of the encryption. For reference, Fig. 7.10b illustrates the result of a random, uniform classification of the traces to the two sets rather than according to the challenge. The figures display the squared difference $\delta(l)^2$ in order to highlight peaks. The results suggest a significant correlation between the S-Box input data and the measured side-channel information. As a consequence, the outer boundaries of the time window containing information leakage are identified and it can be assumed that successful **EM** attacks can be executed.

7.4.3 Probe Selection and Positioning

The **DOM** test further turns out to be a useful utility for selecting an appropriate near-field probe for capturing the **EM** emanation and finding its optimal position. For that purpose, we implemented a *Live DOM* application on the controlling **PC** that sends the alternating challenges to the card and records the **EM** traces. After the automatic detection of a reference

⁴This behavior might be worth to be further exploited in the context of fault injections.

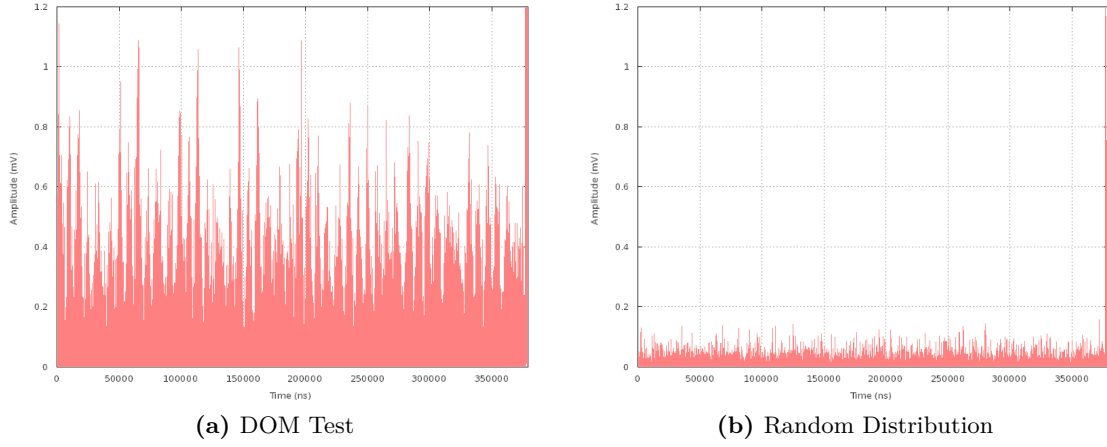


Figure 7.10: Difference-of-Means for the two sets according to the challenges (left) and a random assignment of the traces to the sets (right)

pattern for the alignment⁵, all subsequent traces are aligned accordingly and passed to the **DOM** algorithm. The **DOM** results are displayed instantly and are continuously updated while the probe can be moved and different digital filters can be applied.

By trying different probes and iteratively adjusting the probe position to maximize the amplitude of the **DOM**, we obtained optimal results for a *RF-U 5-2* probe [Lan] (see Sect. 4.3.2). The probe captures the near H-field that is proportional to the flow of the electric current in the horizontal plane.

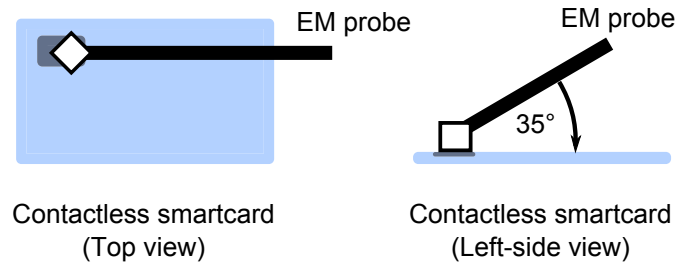


Figure 7.11: Optimal positioning of the EM near field probe.

The optimal position was found to be directly above the **IC** and in parallel to the long side of the card, at a vertical angle of approx. 35° (with respect to the card plane), as shown in Fig. 7.11. It furthermore turned out that placing the smartcard at a vertical distance of about 5 mm to the antenna of the reader considerably improves the amount of side-channel leakage for our setup.

⁵The reference pattern is chosen by finding the first peak whose amplitude exceeds a fixed threshold.

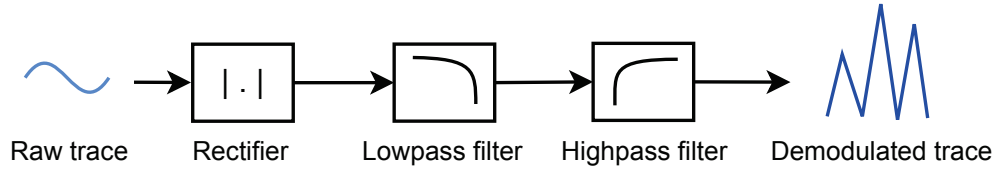


Figure 7.12: Block diagram of incoherent digital amplitude demodulator.

7.4.4 Digital Processing and Alignment

The raw EM measurements recorded between the last bit of the command sent by the reader and the first bit of the answer of the card do not expose any distinctive features, as illustrated in the right of Fig. 7.13. It is not possible to deduce, if and where any computations are performed by the card. Hence, digital preprocessing is applied in order to spot information in the measurements and to identify interesting patterns, e.g., for a more precise alignment.

On the basis of the RFID power model introduced in Sect. 7.3.2 we assume that the power consumption of the smartcard modulates the amplitude of the carrier wave at frequencies much lower than the 13.56 MHz carrier frequency, which is justified by a preliminary spectral analysis and the well-known fact that the on-chip components (such as capacitances, resistors, inductances) on the DUT typically imply a strong low-pass filter characteristic.

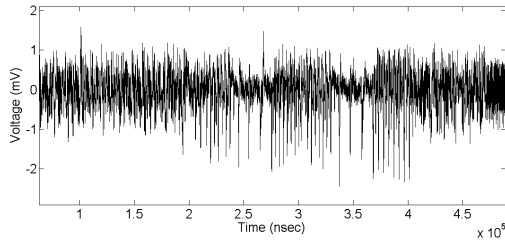
Digital Amplitude Demodulation

In order to obtain the relevant side-channel information, we record raw (not demodulated) traces and perform the amplitude demodulation digitally on a standard PC, using a straightforward incoherent demodulation approach [Sha06]. Figure 7.12 illustrates the process: the raw trace is first rectified and then low-pass filtered using an FIR filter. A subsequent high-pass IIR filter removes the constant component of the demodulated signal and low-frequency noise. The most suitable values for the cutoff frequencies of the filter were determined experimentally as $f_{lowpass} = 2 \text{ MHz}$ and $f_{highpass} = 50 \text{ kHz}$.

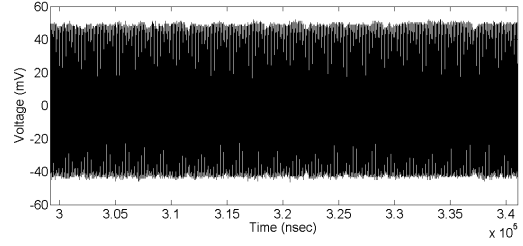
Figure 7.13a displays a demodulated trace in which distinct features are visible, especially two patterns starting at $240 \mu\text{s}$ and $340 \mu\text{s}$, preceded and followed by a number of equally spaced peaks. For comparison, Fig. 7.13b shows a zoomed part of the same trace without demodulation, in which no obvious information is visible. Fig. 7.13c and Fig. 7.13d originate from a trace recorded without the analogue prefilter described in Sect. 7.3.3 and demonstrate that our filter circuit effectively increases the amplitude of the signal of interest and reduces the noise level of the demodulated signal. The traces are now in a sufficient state for trying an EM attack.

7.4.5 EM-Analysis of Cryptographic Contactless Cards

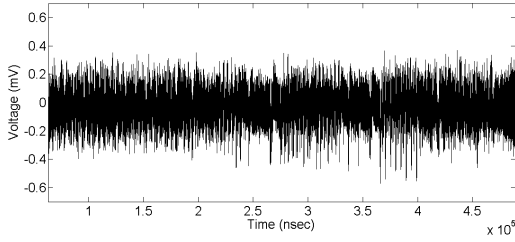
The process of performing a DEMA of the 3DES implemented on the Mifare DESfire smartcard can be split up into the following steps: (i) find a suitable trigger point, (ii) align the traces, (iii) locate the DES encryption, and (iv) recover the secret key with CPA. While the first step has been solved by the low-level reader providing the trigger signal for the oscilloscope at the



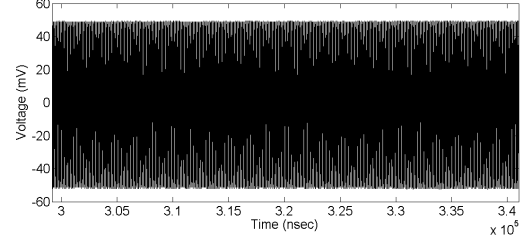
(a) Demodulated trace (50 kHz–2 MHz) with carrier subtractor



(b) Raw trace with carrier subtractor (zoomed)



(c) Demodulated trace (50 kHz–2 MHz) without carrier subtractor



(d) Raw trace without carrier subtractor (zoomed)

Figure 7.13: Raw traces and demodulated traces, with and without carrier subtractor: The shape of the demodulated trace when using the carrier subtractor (a) is the clearest, with the least noise and presumably the most information.

right instant (after the last bit of the command sent by the reader), the remaining three steps are detailed in this section.

Precise Trace Alignment

For precise alignment during the digital processing, we select a short reference pattern in a demodulated *reference trace*. This pattern is then located in all subsequent traces by finding the offset that minimizes the squared difference between the reference and a pattern of the same length of the trace to align, i.e., we apply a least-squares approach.

For devices with a synchronous clock, e.g., a microcontroller, the alignment with respect to one distinct pattern is usually sufficient to align the whole trace. However, in our measurements we found that the analyzed smartcard performs the operations in an asynchronous manner, i.e., the alignment may be wrong in portions not belonging to the reference pattern. A re-alignment has thus to be performed during the attack with respect to the part of the trace we aim to examine by means of CPA.

Data Bus Transfer of Plaintext and Ciphertext

Since performing a power analysis of long measurements is computationally demanding, it is beneficial to precisely localize the cryptographic operation to be attacked and find narrow bounds for the data acquisition. The key of the card used for this final profiling step needs to be known to the attacker. Once the boundaries are known, i.e., the exact location and duration

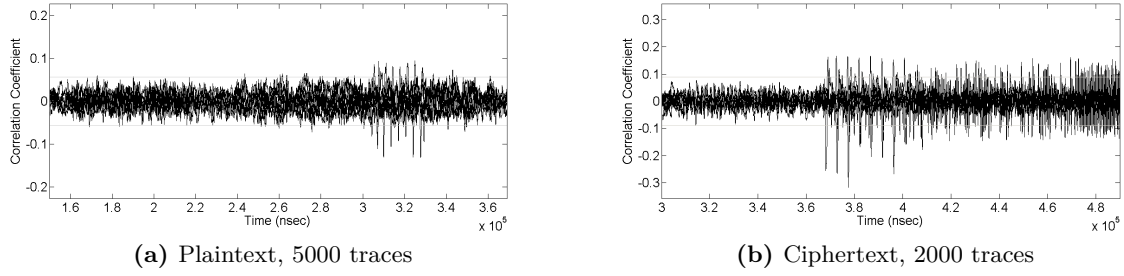


Figure 7.14: Correlation coefficients (Hamming weight) of the plaintext and ciphertext bytes.

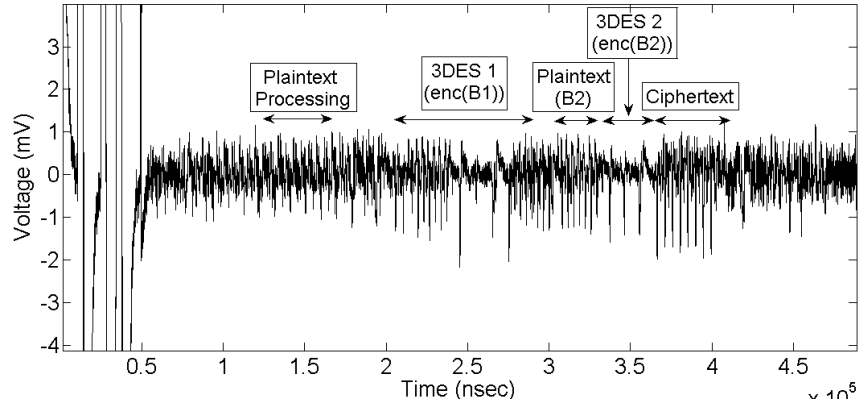


Figure 7.15: Overview of the order of internal operations in an amplitude-demodulated trace of a Mifare DESfire. The time window, in which the targeted encryption of b_2 is executed, is bounded by the processing of plaintext and ciphertext.

of the encryption is found, the side-channel analyses (with an unknown key) can be performed much more efficiently.

For the Mifare DESfire card, the plaintext for the targeted **3DES** operation, i.e., the challenge b_2 sent by our reader, is known and the ciphertext can be computed in a known-key scenario. Hence, the **3DES** encryption can be exactly localized in the power traces by correlating on these values: the encryption obviously must take place after the plaintext has been processed and before the ciphertext is available.

After trying several power models and computing their correlation with the processed data in this known-key scenario, it turns out that the smartcard uses an 8-bit data bus to transfer plaintexts and ciphertexts. The corresponding values can be clearly identified from 2000–5000 traces using an 8-bit Hamming weight model, as depicted in Fig. 7.14a and Fig. 7.14b. Note that the ciphertext is *not* output by the smartcard; however, power analysis pinpoints the exact time instant of it being processed internally.

The short duration of the encryption compared to the processing of the bytes of the plaintext and the ciphertext, in addition to the 8-bit Hamming weight model being the most effective, suggests that the smartcard logic is implemented on a microcontroller communicating with a

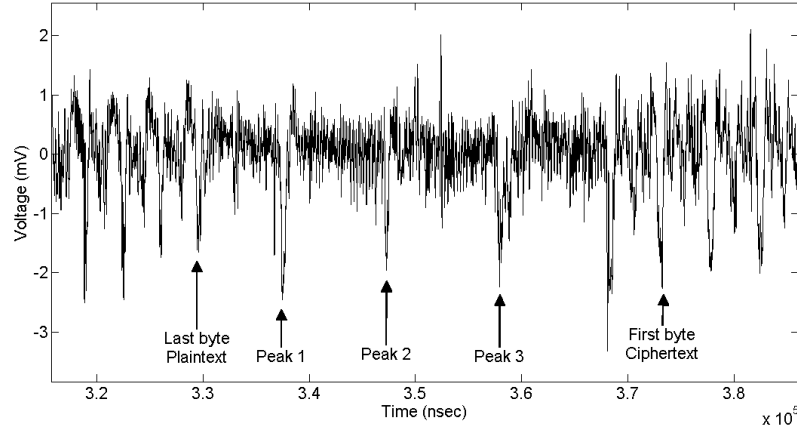


Figure 7.16: Part of a demodulated trace of a 3DES encryption, filtered with $f_{lowpass} = 8$ MHz and $f_{highpass} = 50$ kHz. Three peaks are visible, each of which probably corresponds to one DES execution.

separate 3DES hardware engine over a data bus using pre-charged wires. This assumption is further supported by the fact that correlation with the plaintext bytes can be observed twice, but with reversed byte order. The microcontroller probably first receives the bytes of the plaintext via the RF module, byte-reverses the plaintext and then transmits it via an internal bus to the encryption engine. The ciphertext is later sent back to the μC using the same byte order as for the second appearance of the plaintext.

Fig. 7.15 summarizes the observations made during the profiling: While the first 3DES encryption (labeled 3DES 1) traces back to a prior protocol step, the correlation with the correct ciphertext appears after the execution of the second 3DES (labeled 3DES 2).

Attacking the 3DES Engine

After having localized the $40\mu s$ interval of the 3DES operation from the position of the corresponding plaintexts and ciphertexts, we now focus solely on this part of the trace, thereby enormously accelerating the computations required for the power analysis. Figure 7.16 shows a zoomed view of the targeted 3DES operation, filtered with $f_{lowpass} = 8$ MHz and $f_{highpass} = 50$ kHz.

We acquired millions of measurements from the execution of the cipher and first analyzed the effect of various filters and alignment techniques. We then tried various power models and attack strategies with the goal to extract the secret keys of the 3DES. Assuming that the cipher is implemented in a special hardware module separated from the microcontroller, we expected a Hamming distance model⁶ to result in high correlation peaks for the correct key candidates.

The three marked peaks in Fig. 7.16 appear during the computation of the 3DES, seemingly at the end of one complete (single) DES, and are thus promising candidates as patterns for a (re-)alignment. Consequently, we conduct a CPA on demodulated traces aligned to each of

⁶We also considered several other models, e.g., Hamming weight, however, we did not reach conclusive results for these.

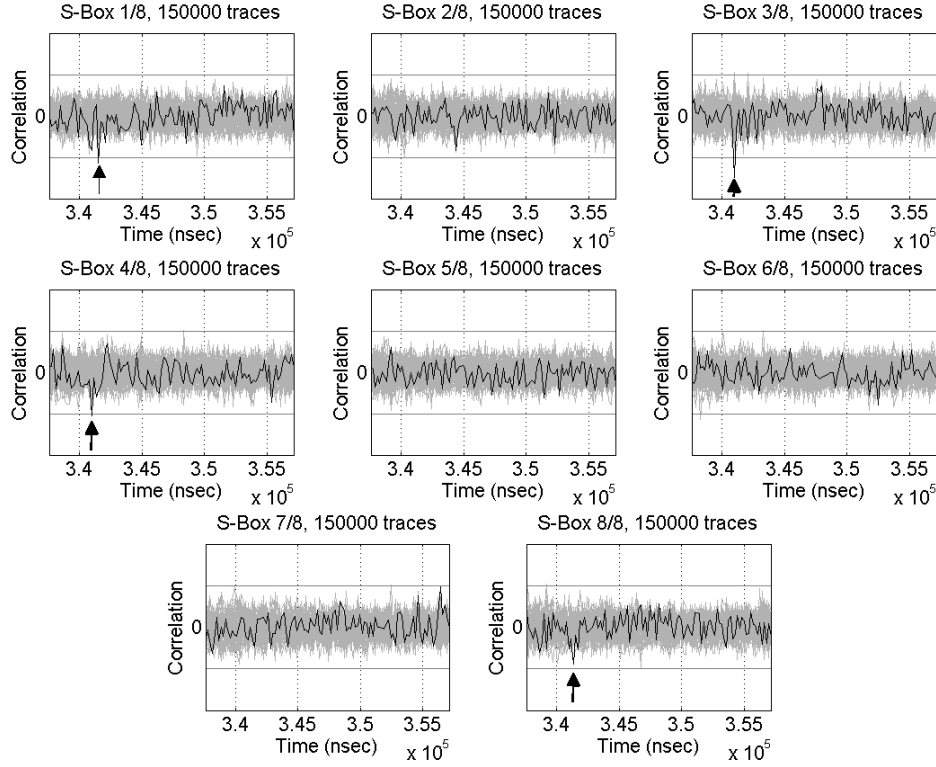


Figure 7.17: First attempt of a bin-wise CPA with peak extraction. The depicted correlation coefficients were computed from 150,000 filtered traces, with $f_{lowpass} = 8$ MHz and $f_{highpass} = 50$ kHz, and reveal the correct key bits for only some of the S-Boxes.

these peaks, where we consider the Hamming distance between the **DES** registers (L_0, R_0) and (L_1, R_1), i.e., the state before and after the first round of the first Single-**DES**. It turns out that, for aligning to the second peak, results are generally most conclusive: When performing a standard **CPA** with $D = 150,000$ traces, correlation peaks with maximum amplitude for the correct key candidate occur for S-Box 1 and 3 at a position which we consider the beginning of the first **DES**.

In order to improve the alignment, we extract local maxima and minima from the trace part belonging to the first **DES** operation. The resulting data points (composed of time position and amplitude) are then grouped on the basis of their time coordinate by dividing the time axis into equal intervals termed *bins* in the following. Thus, minima and maxima occurring at slightly different points in different traces are assigned to the same bin, correcting the timing variations up to a certain extent. The **CPA** is then performed *bin-wise*, i.e., the correlation coefficient for each bin is computed from all extrema occurring within the corresponding time interval. The correlation coefficients for this experiment are given in Fig. 7.17, where the y-axis has been normalized to the theoretical noise level of $4/\sqrt{D}$ (cf. [MOP07]), accounting for the different number of data points per bin. Using this improved re-aligning method, the correct subkey can now be identified for S-Box 1, 3, 4, and 8 from 150,000 traces.

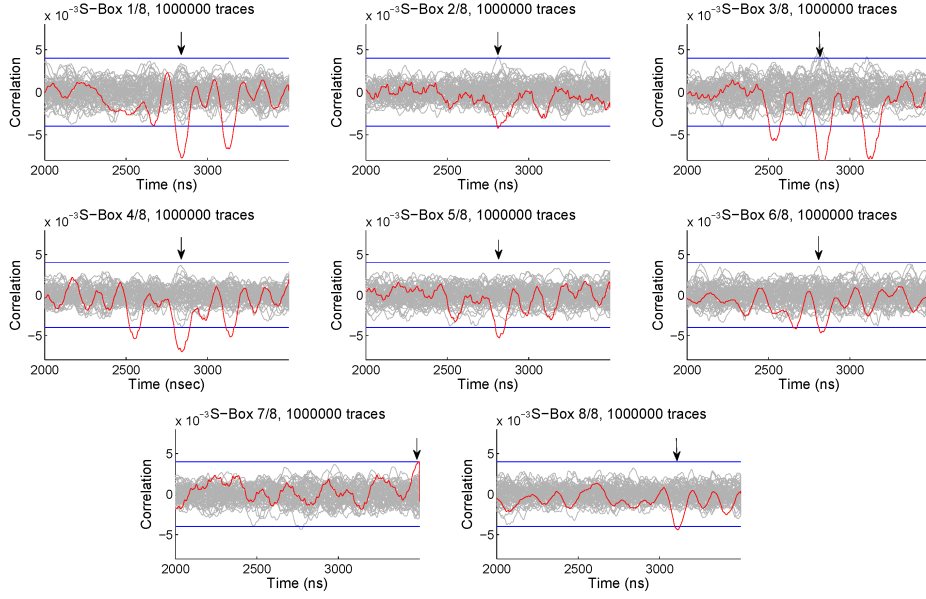


Figure 7.18: Correlation coefficients for a full-key recovery by means of CPA, after processing 1,000,000 traces.

Full-Key Recovery

Finally, when increasing the amount of processed measurements to 1,000,000, a full-key recovery becomes feasible: correlation peaks with maximum amplitude occur for the correct subkey candidate of each S-Box, at a position which we consider as the start point of the first DES. The results are given in Fig. 7.18.

We observe that (i) the correlation for the output of some S-Boxes is significantly stronger than for others, e.g., for S-Box 1 and 3, for which the correct subkey can already be identified after 150,000 traces; (ii) several maxima of the correlation appear at different points in time for one S-Box; and (iii) the instant of the maximum correlation varies depending on the S-Box. Since the input of the next DES execution can now be computed, the remaining bits of the secret 3DES key can now be recovered by attacking the next DES instantiation likewise. We have practically verified the full-key recovery attack on different commercial Mifare DESfire cards, with different unknown keys, and have extracted all keys with a 100% success rate.

As the attack allows for a successful key-recovery (albeit after a large number of traces), we suppose that no masking scheme [MOP07] is implemented to protect the hardware engine. Rather, we conjecture that hiding in the time dimension is used, i.e., dummy cycles with no computation or similar measures, to prevent correct alignment of the traces and thereby hamper a CPA. This assumption is strengthened by the fact that even when repeatedly sending the same plaintext b_2 to the smartcard, the shape and the length of the DES operations, and hence the position of the peaks depicted in Fig. 7.16, vary⁷.

⁷The resulting misalignment when acquiring several traces for the same plaintext also hinders improving the SNR by means of averaging.

With our measurement setup, recording one million traces takes approx. two days, i.e., we achieve a rate of approx. 700 measurements per minute. Due to the extensive profiling of the **DUT**, we focus exactly on the relevant region of the **EM** traces and thus achieve substantial savings both with regard to disk space and processing time. Therefore, our attack is still feasible in a practical scenario, despite the considerable amount of traces⁸.

7.5 Conclusion

As the main result attained in this chapter, it is demonstrated that appropriate countermeasures against power-analysis attacks are also required for **RFIDs**: The strong noise induced by the **EM** field of the reader does not hinder extracting the cryptographic keys from Mifare DESfire MF3ICD40 cards by means of **SCA**. Knowing all keys, an attacker can arbitrarily access the content and functionality of the card. The non-invasive key-recovery attack requires no modification of the card and leaves no physical traces.

A new approach for performing effective analyses, that exploits the side-channel leakage contained in the **EM** emanations of **RFIDs**, is presented and a corresponding analogue hardware and digital filters have been realized. At hand of the introduced low-cost measurement environment and the customized **RFID** reader, the effectiveness of the developed analogue circuitry, that aims at isolating the information leakage contained in the **RF** signal measured during an encryption, is quantified. Furthermore, a general improvement of **CPA** attacks is introduced that enables on-the-fly analysis with a constant-storage, and hence facilitates the analysis of a very large amount of measurements. The steps of analyzing the security of cryptographic **RFIDs** in a black-box scenario, i.e., without previous knowledge about the implementation details, are illustrated.

The security analysis in this chapter pinpoints several weaknesses of the protocol and the actual implementation of the widespread DESfire cards, including a vulnerability to **DEMA**. We investigated the leakage model for the data bus, located the time window of the encryption, and successfully a **CPA** on the **3DES** hardware implementation running on the contactless smartcard. We finally demonstrated the effectiveness of the methods, that are generally applicable for analyzing all kinds of cryptographic **RFIDs**, by performing a full key-recovery.

7.5.1 Outlook

The presented key extraction from Mifare DESfire cards jeopardizes many real-world systems in which these cards are used, such as access control, payments, identification, or public transport. It is conceivable, that DESfire-based real-world systems are subject to attacks in the near future — first analyses of a commercial system have already been published [**KOP11a**]. However, now that the vulnerability of the cards is known to the public⁹, system integrators have a chance (and the need) to react and improve their systems, before a criminal finds and exploits flaws.

It seems very probable that the **EM** key-recovery attack can be improved to require less traces, e.g., by modifications of the measurement setup or advanced attack strategies such as

⁸Compare with ten (!) traces for **DPA**-attacking a **KEELOQ IC** in the next chapter.

⁹Note, that the manufacturer NXP had been informed about our **EM** attacks several years before any publication naming the card has been released by us, and thus had sufficient time to react, e.g., by informing the customers about the security vulnerabilities and having the follow-up product DESfire EV1 ready for the market.

template attacks. First steps into this direction have been taken in [KOP11b] with an improved measurement setup, and as our research continues more sophisticated attacks on Mifare DESfire are likely to appear. Similar attacks will also be feasible on most contactless smartcards of other manufacturers, and other **RFIDs**. After our attacks, NXP announced in June 2010 the discontinuation of DESfire MF3ICD40 and its replacement by DESfire EV1 cards. The successor provides some countermeasures against **SCA** attacks. Still, the card has not yet been thoroughly analyzed by us, hence the possibility of a successful key-recovery by means of implementation attacks cannot be eliminated.

Chapter 8

On the Power of Power Analysis: Breaking KeeLoq Remote Keyless Entry Systems

THIS chapter is dedicated to wireless systems based on the KEELOQ cipher, e.g., used for door openers, alarm systems, and car immobilizers. It summarizes all relevant aspects of several years of joint work with Thomas Eisenbarth [EKPI11], Markus Kasper [KKMP09], Amir Moradi [EKM⁺08, MK09], and Martin Novotný [NK09]. Starting from “straightforward” power-analysis attacks, more sophisticated variants are developed, until a complete break of the KEELOQ rolling-code system, i.e., copying remote controls solely via the air interface, is evident. Furthermore, exhaustive key search of KEELOQ is investigated and a power-analysis resistant replacement for KEELOQ rolling-code systems is proposed.

8.1 Motivation

Since the introduction of power analysis in 1999 [KJJ99], it has become an established method for extracting cryptographic keys from security devices by exploiting measurements of their power consumption. However, when Andrey Bogdanov brought the topic of KEELOQ into our group back in 2006, side-channel attacks still seemed to be a mere academic experiment: There was a surprising discrepancy between the well established theory of power analysis (cf. for instance the CHES proceedings since 1999) and the very few confirmed DPA attacks on real-world security systems¹. The targets considered in the literature were often self-made or known implementations of ciphers on platforms that are familiar to the attacker, and were typically examined in an idealized environment [SLFP04, MPO05, ÖOP03], for example with an artificially generated trigger signal. The practical relevance of such a “white box” cryptanalysis sometimes remains an open question in the context of commercial devices.

This chapter presents a complete break of real-world security products relying on the cipher KEELOQ, by means of power analysis and other attacks, which lay the foundations in the academic world for security-analyzing commercial devices with side-channel attacks. During our investigations we are confronted with a known cipher, but with a black-box implementation, i.e., no knowledge or information about the devices, except for the characterization in the data sheet, and no means to modify the implementation. These circumstances demand for special techniques and extra effort for the analysis and reverse engineering of the unknown targets. Despite these obstructions, we are able to mount highly effective attacks with considerable implications on the security of all commercial KEELOQ systems.

¹Our key-recovery attack targeting Mifare DESfire cards, described earlier in Chap. 7, arose a couple of years after the attacks on KEELOQ presented in this chapter.

8.1.1 KeeLoq

The KEELOQ block cipher is widely used for security relevant applications, e.g., **RKE** and alarm systems for securing the access to a car or a building, as well as passive **RFID** transponders for car immobilizers [Mich]. The cipher was developed in the 1980s in South Africa and has been licensed by Microchip Technology Inc. in the 1990s. Since then, it has been integrated in many products incorporating their secure authentication product family. The cipher had been kept confidential for about two decades.

KEELOQ did not receive much attention until 2006, when the algorithm got known to the public and moved into the focus of the international cryptographic research community. Shortly after the first cryptanalysis of the cipher [Bog07], more mathematical weaknesses and attack vectors were discovered [CBW08, IKD⁺08]. The analytical attacks target the Identify Friend or Foe (**IFF**) mode of KEELOQ, i.e., a challenge-response protocol (see Sect. 2.3.4). In the best case, a transponder can be cloned after monitoring a minimum of 2^{16} plaintext-ciphertext pairs and several days of computations. Despite the impressive contribution to the cryptanalysis of the cipher, the real-world impact of the mathematical attacks is limited, as described in more detail in Sect. 8.2.4. Their main drawback is, that they cannot be applied to the by far more widespread code-hopping mode of KEELOQ: The term “hopping code” used by Microchip refers to a “rolling code” scheme, as detailed in Sect. 2.3.2, in which the plaintext is not available to an attacker. Our aim is thus to find more practical attacks that can also be applied to the yet “unbroken” code-hopping mode.

8.1.2 Contribution and Outline of this Chapter

This chapter provides an in-depth study of the vulnerability of real-world KEELOQ implementations, with respect to **SCA** and brute-force attacks, that is based on several of our publications. The security of both KEELOQ modes, i.e., **IFF** and the code-hopping mode, is analyzed and various known key-derivation schemes are taken into account.

The evolving, highly efficient key-recovery attacks by means of **SCA** can be applied to all implementations of KEELOQ: A **CPA** proposed in the proceedings of CRYPTO 2008 [EKM⁺08] allows extracting the secret device keys from hardware realizations, i.e., the Microchip HC-SXXX family of **ICs**, with minimal effort. A very effective **SPA**, subsequently released in Africacrypt 2009 [KKMP09], reveals the manufacturer key contained in software implementations on Microchip PIC **μCs**. Knowing the manufacturer key enables cloning of KEELOQ remote controls from a distance, and other fatal attacks. To tackle the security of advanced key-derivation schemes, a brute-force attack on KEELOQ is implemented on cryptanalytic hardware, as published in SHARCS 2009 [NK09]. Finally, a new, **SCA**-resistant design of an **RKE** system is developed, as proclaimed in ICICS 2009 [MK09].

Structure

The remainder of this chapter is structured as follows. KEELOQ **RKE** systems, the involved cipher, key management schemes, and the attack history are introduced in Sect. 8.2. Next, various methods for extracting keys from commercial KEELOQ products are illustrated: Section 8.3 elaborates on how the secret key of hardware implementations, employed in most remote controls, can be revealed by means of **CPA** with as few as ten power traces and only minutes

of computation time. Various practical results and figures of power analysis attacks, based on measuring the electric current or the **EM** emanation of different KEELOQ devices, are presented.

Recovering secret keys from software implementations of KEELOQ by means of **CPA** initially turns out to be more demanding. However, in the following it is illustrated that performing a key recovery by **SPA** can be much more efficient than using **CPA**: the powerful **SPA** attack developed in Sect. 8.4 requires neither a plaintext nor a ciphertext and enables recovering the full manufacturer key contained in commercial KEELOQ products, from just a *single* measurement of their power consumption. The **CPA** and **SPA** key-recovery attacks are summarized and compared in Sect. 8.4.5.

Investigating the system-wide security risks arising from a key extraction, in Sect. 8.5 several very practical attacks are developed, and their implications are illustrated. To begin with, Sect. 8.5.1 explains how remote controls which are in the possession of an attacker can be cloned. Section 8.5.2 is dedicated to the direct consequences of extracting manufacturer keys from KEELOQ receivers. A devastating attack illustrated in Sect. 8.5.3 allows recovering the secret key of a transmitter from a distance, just by eavesdropping on the **RF** communication, e.g., when opening a door. Section 8.5.4 details on a **DoS** attack allowing to put an **RKE** system out-of-service, i.e., prevent the legitimate owner from accessing an object secured by KEELOQ, while the adversary can still enter. In Sect. 8.5.5, a brute-force attack for extracting device keys is implemented, and the effort required for a key recovery is evaluated.

In contrast to the hitherto existing attacks, the proposed techniques are also applicable in case of more sophisticated key derivation schemes (cf. Sect. 8.2.3) and are suitable for breaking both the KEELOQ code-hopping mode and the **IFF** mode. To counteract power-analysis attacks, a new, **SCA**-resistant **RKE** system, that can serve as a replacement for KEELOQ, is finally presented in Sect. 8.6.

8.2 Background

In the following, the KEELOQ cipher, KEELOQ **RKE** systems, and the corresponding key management are introduced. Afterwards, the related work on KEELOQ, i.e., the hitherto published mathematical attacks, is detailed.

8.2.1 KeeLoq Cipher

KEELOQ is a block cipher that operates on a 32-bit input using a secret key $K = \{k_0, \dots, k_{63}\}$ with a length of 64 bit. One encryption or decryption takes 528 rounds, during each of which one bit of the 32-bit state $Y = \{y_0, \dots, y_{31}\}$ is computed and shifted into the state, as illustrated in Fig. 8.1.

The cipher consists of a Non-Linear Feedback Shift Register (**NLFSR**), the feedback of which depends linearly on two register bits, one key bit, and the output of a Non-Linear Function (**NLF**) that maps five other register bits to a single bit. The key schedule is a simple rotation of the key, i.e., in each clock cycle the key register is rotated to the right for an encryption or to the left for a decryption. Accordingly, each bit of the key is reused at least eight times, i.e., every 64 rounds [Bog07, CBW08, IKD⁺08] — this fact is exploited by the **SPA** attack on the receiver detailed in Sect. 8.4.

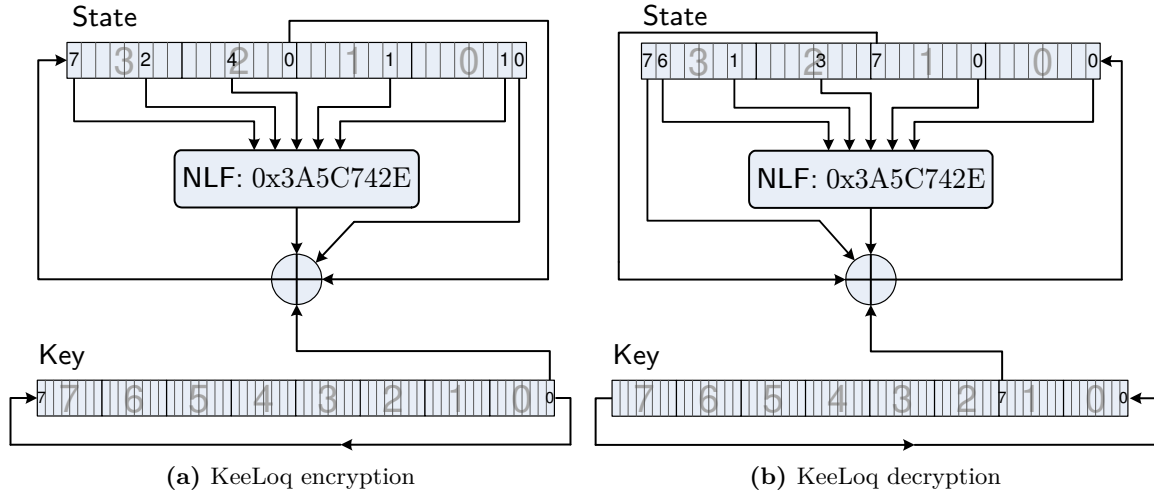


Figure 8.1: Block diagram of the KeeLoq encryption (a) and decryption (b).

The Non-Linear Function

While the **NLF** could also be realized by means of Boolean functions, performing Table Look Ups (**TLUs**) as described in the following is common practice. Defining a look-up table by the hexadecimal constant $LUT = 0x3A5C742E$, the j -th bit of LUT is equivalent to one output bit of the non-linear function $NLF(x_4, x_3, x_2, x_1, x_0)$. The index $j \in \{0, 1, \dots, 31\}$ is obtained from the decimal representation of the input bits x_4 to x_0 , i.e., $j = 2^4 \cdot x_4 + 2^3 \cdot x_3 + 2^2 \cdot x_2 + 2^1 \cdot x_1 + 2^0 \cdot x_0$.

Encryption

Prior to an encryption, depicted in Fig. 8.1a, the secret key K and a block of the plaintext P are loaded into the key register and the state register, respectively. In each round i of the cipher five bits of the state Y are fed into the **NLF**, whose output bit $NLF(y_{31}, y_{26}, y_{20}, y_9, y_1)$ is combined with the state bits y_{17} , y_0 , and a bit of the secret key $k_{i \bmod 64}$ by means of an **XOR**. The fresh bit prepared by the **XOR** function becomes the Most Significant Bit (**MSB**) y_{31} of the state register, after shifting the latter to the right. After repeating the above for 528 rounds, the state register contains the ciphertext C .

Decryption

The decryption illustrated in Fig. 8.1b is used both for deciphering hopping codes and during the key-learning phase (cf. Sect. 8.2.3). The process is similar to the encryption, except for the direction of the shifts and the position of the taps for the **NLF** and the **XOR** function. After initializing the state with a ciphertext C , in each round i the key bit $k_{(15-i) \bmod 64}$ is **XORed** with the two bits y_{31} and y_{15} of the state and the output bit of $NLF(y_{30}, y_{25}, y_{19}, y_8, y_0)$. Then, the state is shifted to the left and the output of the **XOR** becomes the **LSB** y_0 of the state. After clocking the cipher 528 times, Y contains the plaintext P . The pseudo-code for the decryption

given in Alg. 2, where $\oplus$ denotes a bitwise XOR and FB the output bit of the feedback function, illustrates the program flow of a typical implementation of KEELOQ in software.

Algorithm 2 KEELOQ Decryption (Pseudo Code)

Input: ciphertext $C = \{c_0, \dots, c_{31}\}$, key $K = \{k_0, \dots, k_{63}\}$

Output: plaintext $P = Dec_K(C)$, where Dec_K denotes KEELOQ decryption with K

- (1) Load ciphertext: $Y = C$
 - (2) For $i = 0$ to 527 do
 - 2.1. Compute bit of NLF: $OUT = NLF(y_{30}, y_{25}, y_{19}, y_8, y_0)$
 - 2.2. Compute feedback bit: $FB = k_{(15-i) \bmod 64} \oplus y_{31} \oplus y_{15} \oplus OUT$
 - 2.3. Update state
 - 2.3.1. left-shift state: $Y = (Y \ll 1)$
 - 2.3.2. assign LSB: $y_0 = FB$
 - (3) RETURN Y
-

8.2.2 KeeLoq RKE Systems

In contrast to KEELOQ IFF systems which provide authentication of a transponder to the main system using a bidirectional challenge-response protocol (compare with Sect. 2.3.4), KEELOQ is commonly used in the unidirectional code-hopping (in general termed rolling code, compare with Sect. 2.3.2) applications [Mica]. The code-hopping scheme appears to be dominant for authenticating a remote control to a receiver in commercial RKE applications of KEELOQ, e.g., in car anti-theft systems and garage door openers. Such a unidirectional RKE system consists of one receiver in the secured object and one or more remote controls that can send transmissions to the receiver and thereby control the access to the object. The early fixed-code or multi-code systems² were developed soon after digital circuitry became available. As detailed in Sect. 2.3.1 they rely on sending a fixed sequence of binary data when pressing the button of the remote control, and permit access only in case the code is correctly identified.

Hopping Code Scheme

The obvious need for a protection against replay attacks, with only a unidirectional channel available, brought the invention and wide deployment of hopping code systems. KEELOQ RKE systems typically employ hardware implementations of the cipher, such as HCSXXX [Micf], for generating hopping codes in the remote controls. A software implementation, typically running in the receiver on an 8-bit PIC microcontroller [Mici], serves amongst others for decrypting the transmissions. Each remote control and the receiver share a device key K_{dev} and a fixed discrimination value $disc$ with a length of 10 to 12 bits. In addition, they are synchronized with a 16-bit to 18-bit synchronization counter cnt . The latter is incremented in a remote control

²Note that even these outdated systems are still available on the market.

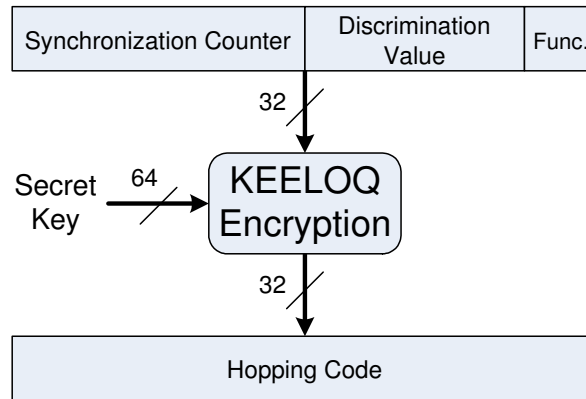


Figure 8.2: Generation of KEELOQ hopping codes

each time one of its function buttons is pressed. For each remote, the receiver stores the *cnt* contained in the last valid transmission and updates the counter only upon decryption of a valid hopping code. Figure 8.2 depicts, how such a hopping code is constructed by encrypting a 32-bit message formed of *disc*, *cnt* and a 4-bit function information. The latter selects the exact task to be triggered, for instance, it enables to open or close different doors. One message finally sent via the RF interface consists of the (encrypted) hopping code preceded by the (unencrypted) serial number of the remote control.

The receiver looks up the K_{dev} that corresponds to the serial number and decrypts the hopping code with K_{dev} to obtain *disc* and the current *cnt*. The remote control is authenticated if *disc* is identical to the shared one and the decoded *cnt* is moderately increased compared to the counter value cnt^* stored in the receiver for that remote control.

Three windows are defined for verifying the counter cnt ³: If a received *cnt* is within the **open window** with a length of typically 16 values, starting after the last stored value cnt^* , the intended function will be executed instantly after a single button press on the remote control. Otherwise, the **resynchronization window** containing the next 2^{15} counter values is examined. If a *cnt* is received that lies in this window, the desired function is carried out only if two consecutive, valid counter values are received, i.e., after pressing the button again for sending $cnt + 1$. The **block window** contains the remainder of the counter space, including the last valid counter cnt^* . Any transmission with a *cnt* value within this window will be ignored. Following this scheme, the receiver is capable of rejecting repetitive codes and can thereby prevent replay attacks (except if combined with jamming, see Sect. 8.2.4). More details about the principle of a rolling code scheme are given in Sect. 2.3.2.

8.2.3 Key Management

Extra remotes can usually be made known to the receiver by putting it into a learning mode in which the key of the extra remote is derived and stored, as detailed next. Microchip suggests several key derivation schemes for generating a unique device key K_{dev} for each remote

³The listed window sizes are recommended by Microchip, but they can be altered to fit the needs of a particular system.

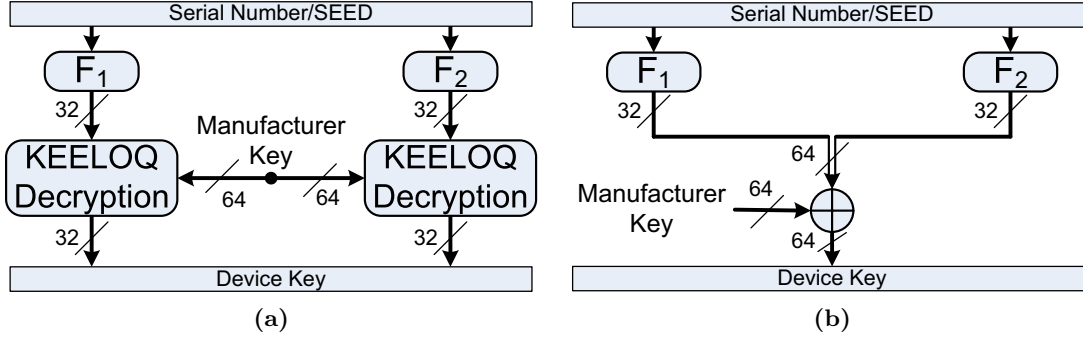


Figure 8.3: Key derivation schemes, as proposed by Microchip, use a manufacturer key for deriving unique device keys for the remote controls of the RKE system.

control [Micb]. All of these schemes involve a secret manufacturer key K_{man} that is to our knowledge identical for all receivers of a given manufacturer. K_{man} is used once in the factory for a freshly produced remote control, and later in the receiver during a learning phase in which K_{dev} is derived. In particular, the master key K_{man} of the RKE system ensures that a receiver can compute the K_{dev} of a new remote control. Thus, K_{man} is contained in every receiver and has to be stored securely in the memory of the microcontroller.

For each manufacturer, the following key-derivation schemes incorporating K_{man} allow to establish a unique, manufacturer-specific binding between the serial number of a remote control and its secret key K_{dev} . Thus, each manufacturer can produce remote controls that cannot be cloned by competitors (unless they get hold of K_{man}).

- (a) In the most widespread key-derivation scheme in practice, as illustrated in Fig. 8.3.a, the device key K_{dev} is computed from two KEELOQ decryptions employing K_{man} as the secret key. The two functions F_1 and F_2 — which are usually simple paddings — are applied to the serial number of the transmitter to form the plaintexts for the decryptions:

$$K_{dev} = Dec_{K_{man}}(PAD1||Serial)||Dec_{K_{man}}(PAD2||Serial)$$

Likewise, a randomly generated seed value stored individually in each remote control can be used instead of the serial number, or can be combined with it, in order to generate the K_{dev} .

- (b) As presented in Fig. 8.3.b, K_{dev} can also be generated from simply XORing K_{man} with a (padding) function of the serial number.

$$K_{dev} = K_{man} \oplus (PAD1||Serial||PAD2||Serial)$$

Again, the seed value of the remote control can be involved in the key derivation.

Note that a manufacturer may develop his own, proprietary key-derivation scheme. In practice we have even encountered real-world KEELOQ systems without any key derivation, i.e., with the same fixed K_{dev} in all remote controls of that manufacturer.

The seed value of length 32–60 bit is not mandatory and seems to be used very rarely in practice, hence in the following we focus on the common key derivation from the serial number. Details and consequences for the security of using a seed during the key derivation are covered in Sect. 8.5.5. Without using a seed, the security of the schemes relies on the confidentiality of K_{man} : Any party knowing the latter, e.g., the receiver, is capable of calculating K_{dev} .

The above schemes require different efforts with respect to extracting K_{man} . For the first key-derivation function relying on KEELOQ decryptions, a straightforward inversion of the latter is impossible. Even if a pair of ciphertext and plaintext is known, i.e., a device key K_{dev} and the corresponding serial number of a remote control, the manufacturer key K_{man} remains secret⁴. In case of the second scheme, assuming that an attacker by any means reveals K_{dev} from one remote control, the corresponding manufacturer key K_{man} can be computed easily by reversing the XOR function.

8.2.4 History of Attacking KeeLoq

A common method for electronically breaking into cars secured with hopping code systems, e.g., KEELOQ, is a combined eavesdropping-and-jamming attack: while the legitimate owner tries to lock his car with a remote control, the transmission is monitored and at the same time the frequency of the transmission is jammed, with the effect that the car won't be locked and the attacker possesses a temporarily valid (one-time) hopping code. There are devices that automatically perform the described process, but in practice they are rather unreliable. Furthermore, one successful transmission of a new hopping code from the original remote to the car invalidates all previously eavesdropped hopping codes.

Recently, several cryptanalytic attacks on the KEELOQ cipher have been published [CBB08, CBW08, IKD⁺08]. The first cryptanalysis of the KEELOQ cipher was presented by Bogdanov in 2007 [Bog07]. Two attacks were proposed: a combined slide and guess-and-determine attack that requires $2^{50.6}$ KEELOQ encryptions and an additional cycle-structure-analysis technique revealing a secret key from 2^{37} encryptions. As a major drawback, both attacks require the entire codebook, i.e., all 2^{32} plaintext-ciphertext pairs need to be known to an attacker.

Courtois et al. [CBW08] proposed two further attacks in 2008. The first is a slide-algebraic attack requiring $2^{51.4}$ KEELOQ encryptions but only 2^{16} known plaintext-ciphertext pairs. The second approach is a slide attack for which again almost the entire codebook needs to be known. It reveals the secret key with a complexity of approximately 2^{27} KEELOQ encryptions.

The most practical mathematical attacks on the KEELOQ cipher were proposed by Indesteege et al. in [IKD⁺08, ABDM⁺10]. The authors combine slide and meet-in-the-middle attacks. Their strongest attack needs 2^{16} known plaintext-ciphertext pairs and has a complexity of $2^{44.5}$ KeeLoq encryptions. The attack can recover the secret key of a KEELOQ transponder in IFF mode in two days using 50 dual-core PCs.

As a consequence of the mathematical weaknesses, KEELOQ has to be regarded as insecure from the cryptographic point-of-view. Still, the mathematical attacks rely on known pairs of plaintexts and ciphertexts and are hence only practical for KEELOQ challenge-and-response protocols, e.g., [Micg]. For a practical RKE system using hopping codes, mathematical attacks are impractical: the required minimum of 2^{16} plaintext-ciphertext pairs cannot be obtained in

⁴The mathematical attacks require thousands of plaintext and ciphertext pairs, corresponding to thousands of successfully revealed keys of remote controls

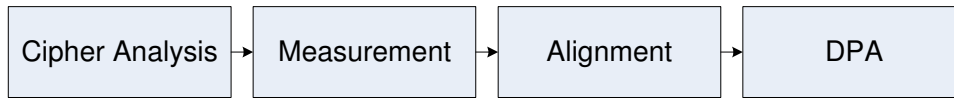


Figure 8.4: Steps taken for a key recovery by means of DPA.

case of a code-hopping system, because an adversary has only access to the ciphertexts that are transmitted by a remote control, while the corresponding plaintexts remain unknown. In theory, hence hopping-code systems remain secure. The following sections pinpoint new attack vectors based on power analysis, that are applicable to *all* operation modes of KEELOQ.

8.3 DPA of KeeLoq

When starting to analyze the targets using KEELOQ, we were exposed to a “classical” situation for physical attacks: Even though the algorithm was known, hardly anything was known about the implementation. We found that the transmitters usually employ HCSXXX ICs of Microchip, featuring a hardware implementation of the cipher. The receivers we looked at are all equipped with a read-protected PIC microcontroller on which a KEELOQ decryption routine is implemented in software. This section explains the details of DPA-attacking transmitters and receivers, starting with a general approach that is appropriate for both types of realizations.

8.3.1 Overview of the DPA

We first give a short outline of the single steps for performing the key recovery, as illustrated in Fig. 8.4, before taking a closer look at DPA-attacking the different targets. The theory behind power-analysis attacks is detailed in Sect. 3.2.2.

Step 1: Cipher Analysis

For performing a DPA of a particular implementation of a cipher, initially one needs to make certain assumptions about the leakage produced by it. Then, a prediction function for exploiting that leakage must be developed, which depends on the cipher structure as well as on the particularities of the given implementation.

Even though no details of the implementation are known in advance, some assumptions about the side-channel leakage can be made, e.g., software executed on a PIC microcontroller often exhibits a HW leakage, while a hardware realization is more likely to leak the HD. Since hopping codes employ short messages, we assume straightforward implementations without pipelining or other measures for accelerating the algorithm.

Step 2: Measurement

Measuring the power consumption can expose the clock frequency of the device and often allows for visually distinguishing between different parts of the computations. For a DPA attack, several measurements of the targeted operation are required.

The power traces are gathered by measuring the current via a shunt resistor connected to the ground pin of the target chip. For comparison, we also acquire the EM emanation of

the device by means of near-field probes (see Sect. 4.3.2). We built a PCB for controlling a transmitter from a PC and emulate KEELOQ chips; hence a measurement sequence can be executed automatically. The power traces were acquired using an Agilent Infiniium 54832D digital oscilloscope that provides a maximum sampling rate of 4 GS/s.

Step 3: Alignment and Pre-Processing

One problem of aligning the power traces of an unknown implementation is the absence of a suitable trigger signal. The solution for this is target-specific and detailed in Sect. 8.3.3 and Sect. 8.3.4 for transmitters and receivers, respectively.

Another problem is that the target devices are clocked by a noisy internal oscillator. Hence we had to find a way to remove the clock jitter. We know that most of the data-dependent leakage occurs in the instant when the registers are clocked, producing peaks with varying amplitudes in each clock period. The amplitudes of these peaks directly correspond to the dynamic power consumption of the target circuit and thus contain most of the relevant information. Accordingly, we extract the peaks from the power consumption after the data acquisition and perform the power analysis solely on the amplitudes of the peaks. This peak extraction step has two advantages for the subsequent analysis:

- The amount of data points per trace is greatly reduced, which facilitates the post-processing and storage of the acquired measurements.
- The disturbing effect of a clock jitter is removed and thus the traces are accurately aligned.

Other methods for removing the clock jitter, such as Fourier transform, filtering, etc., turned out to be more complicated and less effective.

Step 4: Developing and Performing the DPA

After the peak extraction and alignment steps, the traces are processed by the DPA algorithm. When attacking the ICs of remote controls, only the ciphertext, i.e., the hopping code, is available to the adversary. Hence, the power analysis start from the *last* round of the encryption. For the software implementation of the PICs, the plaintexts are known and the attack initially targets the *first* round of the decryption.

8.3.2 Building a Scalable CPA for KeeLoq

For successfully performing a CPA attack, some intermediate value of the cipher has to be identified that (i) depends on known data (such as the plaintext or the ciphertext), (ii) depends on the key bits, and (iii) is easy to predict (compare with Sect. 3.2.2). It is advisable to choose a value that has a high degree of non-linearity with respect to the key, to avoid so-called ghost peaks for similar keys [BCO04].

Compared to the two shift registers, the power consumption of the combinational part, i.e., a few XORs and the NLF, is small and can be neglected. Note that the HD of the key register does not change, since the key is simply rotated. The identical HD of the key register in each clock cycle implies — in theory — a constant power consumption per clock cycle.

Hence, for a known-ciphertext attack on the encryption we focus solely on the state register Y . Starting from the last (528th) round, 32 bits of the final state $Y^{(528)} = (y_0^{(528)}, \dots, y_{31}^{(528)})$, are known. Furthermore, 31 bits of the previous state, $Y^{(527)}$, i.e., the bits $(y_1^{(527)}, \dots, y_{31}^{(527)})$, are identical to $(y_0^{(528)}, \dots, y_{30}^{(528)})$. Therefore, just one bit $y_0^{(527)}$ is unknown in round 527 of the cipher.

According to the description of the cipher in Sect. 8.2.1 we can express the update function of the state register as

$$y_{31}^{(i+1)} = k_0^{(i)} \oplus y_{16}^{(i)} \oplus y_0^{(i)} \oplus \text{NLF} \left(y_{31}^{(i)}, y_{26}^{(i)}, y_{20}^{(i)}, y_9^{(i)}, y_1^{(i)} \right), \quad (8.1)$$

where $k_0^{(i)}$ is the rightmost bit of the key register in the i^{th} round.

Exploiting that $k_j^{(i)} = k_{(i+j) \bmod 64}$ we can rewrite Eq. 8.1 as

$$y_0^{(527)} = k_{15} \oplus y_{16}^{(527)} \oplus y_{31}^{(528)} \oplus \text{NLF} \left(y_{31}^{(527)}, y_{26}^{(527)}, y_{20}^{(527)}, y_9^{(527)}, y_1^{(527)} \right). \quad (8.2)$$

Recovering $y_0^{(527)}$ by means of power analysis thus directly reveals one bit k_{15} of the key register. This process can be repeated for recovering the **LSBs** of the state register of the previous rounds, i.e., $y_0^{(i)}$, $i = (526, 525, \dots)$, and the corresponding key bits, until the full key is extracted.

However, Eq. 8.2 depends linearly on the key bit k_{15} . Above we stated that nonlinearity helps distinguishing correct key hypotheses from wrong ones. Fortunately, as depicted in Fig. 8.1a, the new bit of the round state, $y_{31}^{(i)}$, directly enters the **NLF** leading to a nonlinear relation between the key bit k_{15} and the state $Y^{(526)}$. Accordingly, the nonlinearity for one key bit increases in each round after it was clocked into the state. Taking this increased nonlinearity in the successive rounds into account, we developed a scalable **CPA** that allows for hypothesizing m bits of the key and delivers the n most probable key candidates. Using the **HD** power model introduced in Sect. 3.2.2, the **CPA** is performed on round $(528 - m)$, thereby taking advantage of the increased nonlinearity induced by a key bit passing the **NLF** m times.

After the first execution of the scalable **CPA**, we iterate the process for all partial keys, i.e., at least $\lceil 64/m \rceil$ times. The length of the hypothesized partial key in each iteration increments by m bits until consequently all bits of the key are processed. Since some of the bits of a previous key guess might be wrong, in each iteration a number n of the most probable partial key candidates is kept as “survivors”. These n key candidates and the respective states Y originating from each key candidate serve as a known input for the next execution of the **CPA**.

Wrong surviving candidates of the previous iteration result in a misleading initial state Y for the following execution of the **CPA** and hence strongly decrease the correlation of subsequent key guesses. Accordingly, wrong key candidates are identified by the decreased correlation, while the correlation for the correct key must further increase in each iteration of the **CPA**. If all key guesses of one round show a low correlation, indicating that the correct key is not amongst the surviving key candidates any more, we go one step back and broaden the number of surviving key guesses n . Thus, this procedure, which is similar to the extend-and-prune strategy described by Chari et al. [CRR02], has an error-correcting property. In the last iteration $i = \lceil 64/m \rceil$ the key with the highest correlation coefficient is selected out of the n surviving keys.

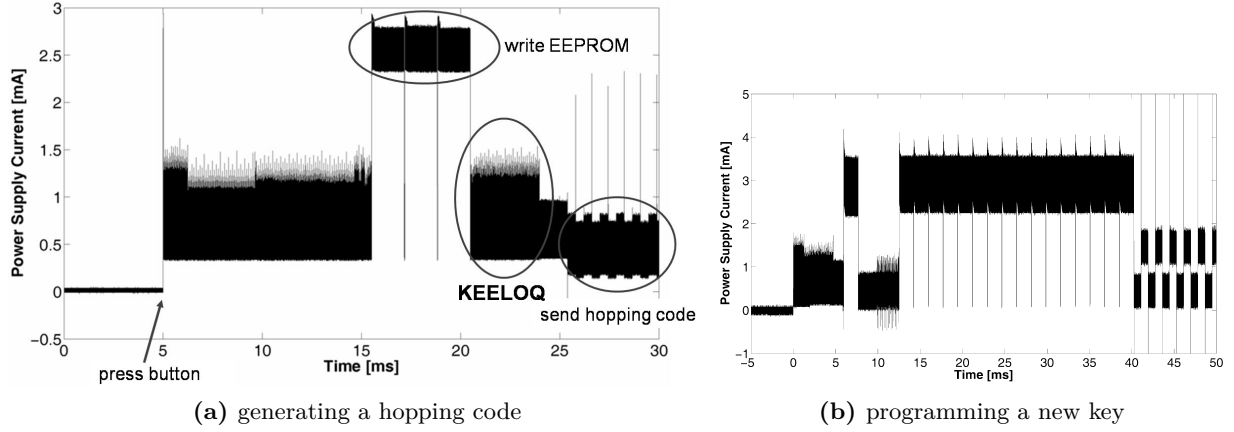


Figure 8.5: Power consumption of a HCS IC contained in a remote control, measured during the generation of a hopping code (left) and the programming of a new key (right).

8.3.3 CPA of KeeLoq in Hardware

For attacking commercial KEELOQ code hopping encoders, such as HCS200 [Micc], HCS201 [Micd], HCS300 [Mice] and HCS301 [Micf], we first had to identify the time window in the power traces that corresponds to the encryption function. Note, that this step, comparable to looking for a needle in a haystack, has been the most time-consuming, as in most real-world attacks⁵.

Locating the Encryption With CPA

The targeted encryption must be located somewhere in between the power-up of the remote control, i.e., pressing its button, and the transmission of the hopping code (compare with Fig. 8.5.a). The highest amplitude during the operation of a remote control occurs for the period between 15 ms to 20 ms. Comparing this period with the high amplitude periods of Fig. 8.5.b, recorded while programming a new key into the HCS module, indicates that this part corresponds to writing into the internal EEPROM cells. We verified that this time window corresponds to storing the new counter value for generating the next hopping code into the EEPROM by switching off the power supply of the transmitter, once exactly before the high amplitude occurs, and next exactly after the end of the high amplitude. As a result, we observed that the counter is only increased in the latter case.

We developed a CPA on the full trace based on predicting the HD of the state register Y between two subsequent clock cycles:

$$P_{HD}^{(i)} = \text{HD}(Y^{(i)}, Y^{(i-1)}) = \text{HW}(Y^{(i)} \oplus Y^{(i-1)}) \quad (8.3)$$

⁵In fact, we spent several months acquiring power traces and trying to correlate on intermediate values of the cipher with different power models, until the time window of the encryption was found. After spotting the first correlation peak in the power traces, the further analysis was fairly easy — a typical experience with real-world targets.

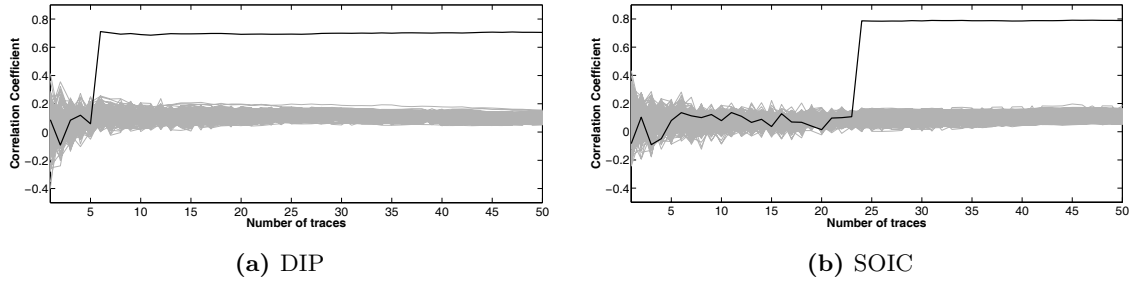


Figure 8.6: Power analysis of KEELOQ in hardware: Correlation coefficients of key hypotheses of an exemplary HCS201 IC, as a function of the number of measured traces. The secret key is found after 6 or 23 traces for a **DIP** or an **SOIC** package, respectively.

where $P_{HD}^{(i)}$ denotes the hypothetical power consumption in the i^{th} round, HD and HW are Hamming distance and Hamming weight, respectively, $Y^{(i)}$ indicates the content of the state register in the i^{th} round, and $\oplus$ is a 32-bit **XOR** function.

Applying the **CPA** to several thousand measurements we found — by trial and error — that the encryption happens after writing to the **EEPROM**, namely between 20.5 ms to 24 ms (compare with Fig. 8.5.a), and focus only on this time window. The distance between two peaks in the measurements furthermore suggests that the frequency of the internal oscillator in the chip is approximately 1.25 MHz.

Exploiting the Leakage of all Rounds per Trace

After further profiling, the time instant of the leakage of each round is exactly located. Based on this knowledge, we extend the attack described in Sect. 8.3.2 to correlate all known and predicted rounds to the corresponding power peaks, i.e., we exploit the **HDs** occurring in *all* rounds up to the latest predictable one. All available bits of the key are considered, i.e., those that are already recovered and the m new hypothetical bits of the key. These predictions (for each round) are then correlated to the measurement values *within* each trace, i.e., the correlation coefficient is evaluated in a trace-wise fashion, trace by trace. The averaged correlation of all traces is then used as the key distinguisher.

Results of Power Analysis (Hardware)

The extended attack exploiting all rounds was performed on HCS200, HCS201, HCS300, HCS301, HCS361, HCS362, and HCS410 ICs in both **DIP** and the surface-mounted **SOIC** packages. Figure 8.6 compares the (averaged) correlation coefficients of the correct key of HCS201 chips in **DIP** and **SOIC** packages as a function of the number of traces. The sudden increase of the averaged correlation is due to evaluating all 528 rounds of the algorithm in each step of the **CPA**, as described above, and the error-correcting property of our attack.

In the best case we were able to recover the secret key of **DIP** packages from only six power traces when sampling at a rate of 200 MS/s . At most 30 power traces are sufficient to reveal the

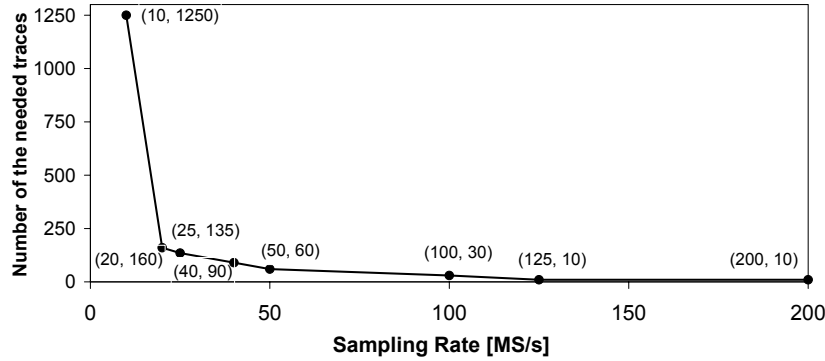


Figure 8.7: Number of measurements required for revealing the secret key of a HCS201 IC in a DIP package as a function of the sampling rate. The numbers in parentheses give the exact coordinates of the points.

secret key of an HCS chip in an **SOIC** package: its lower power consumption results in a worse **SNR** of the measurements.

Results of EM Analysis (Hardware) We repeated the above experiments with an **EM** near-field probe RF-U 5-2 (see Sect. 4.3.2) to monitor the electromagnetic emanation, instead of measuring the electric current via a shunt resistor. The probe was positioned close to the ground pin of a HCS201 **IC** in a **DIP** package, in order to acquire the peaks of the **EM** field that are caused by the change of electric current. Compared to inserting a resistor in series to the device, this **DEMA** (compare with Chap. 7) is truly non-invasive, as no modification of the **PCB** is necessary. Contrary to our assumption that the **SNR** would suffer from environmental noise and thus much more traces would be required to recover the key, the results obtained and the number of traces needed for a successful key recovery are comparable to the case of power traces acquired by means of a resistor. In the best case, we succeeded in recovering the key from only 10 **EM** measurements.

Technical Requirements To estimate the minimum technical requirements for the **SCA**, we performed experiments with varying sampling rates and evaluated the number of power traces required for recovering the correct key. Figure 8.7 shows the results of attacking a HCS201 chip in a **DIP** package in the case of power measurements via a resistor. We conclude that our attack can be carried out effectively even with low-cost equipment, e.g., a very cheap type of oscilloscope with a maximum sample rate as low as 50 MS/s suffices to find the secret key from 60 power traces.

8.3.4 CPA of KeeLoq in Software

The next target to be investigated in the context of power analysis is the code-hopping decoder implemented in the receiver. We recall that the latter contains the manufacturer key K_{man} , which is an attractive target for a complete break of the system. Inside the receiver, typically a

PIC microcontroller performs the KEELOQ decryption in software, handles the key management, and controls for instance the motor of the garage door or the locking system of a car.

Receivers usually offer a so-called “learning mode”. In this learning mode the user can register new remote controls to cooperate with the receiver, according to the implemented key management. We identified the key derivation scheme of the target receiver as scheme (a) in Fig. 8.3, in which the device keys are computed on the basis of the serial number. We aim at recovering the manufacturer key K_{man} by performing a CPA on the KEELOQ decryptions that are performed during the learning phase. The targeted PIC16F687 microcontroller on the PCB of the commercial receiver runs at a clock rate of 4 MHz.

Before executing the CPA, we adapt the power model of the attack to a software implementation on an 8-bit μC . Typically, these μC s leak the HW of the processed data [Eri07]. Furthermore, one can assume that the state is stored in the 8-bit registers of the PIC microcontroller. Hence, predicting the HW of the least significant byte of the KEELOQ state register Y in round i is appropriate:

$$P_{HW}^{(i)} = \text{HW} \left(Y_{\text{LSByte}}^{(i)} \right) = \sum_{j=0}^7 y_j^{(i)}$$

Communicating with a Receiver

For establishing the RF communication to the receiver in a low-cost manner we directly connected the parallel port of a PC to the communication pin of a remote control with the HCS module removed, i.e., only the RF part remains active on its PCB. To verify the functionality we sent various valid messages using a known serial number, key pair and discrimination value, to register the PC to our receiver. Despite the known time jitter when accessing the parallel port from Microsoft Windows the simple device works excellently.

The receiver has to be forced to enter the learning mode by manually pressing a button — a circuit consisting of a standard bipolar transistor and two resistors soldered to the PCB of the reader allows performing this action automatically by the PC via the parallel port. The setup now allows emulating a remote control and perform all actions required for triggering the learning mode and registering virtual remote controls in the receiver, by means of a PC.

Measuring the Key Derivation

In order to measure the power consumption during the key-learning phase, we disconnected the GND-pin of the targeted microcontroller from the PCB and inserted a shunt resistor in series. We acquired the measurements for the attack by repeatedly putting the receiver into the learning mode and sending hopping-code messages with randomly generated serial numbers to the receiver, in order to mount a known-plaintext attack on the decryption. Lacking any special features in the power consumption of the PIC that could have been used as a trigger, we started the measurements directly after transmitting the last bit via the RF interface. This results in our traces not being well-aligned — one possible reason for hindering an effective DPA attack.

Results of the CPA (Software)

While performing the CPA we noticed that the correlation coefficient of the correct key becomes continuously worse with an increasing number of rounds, as shown in Fig. 8.8. For revealing the

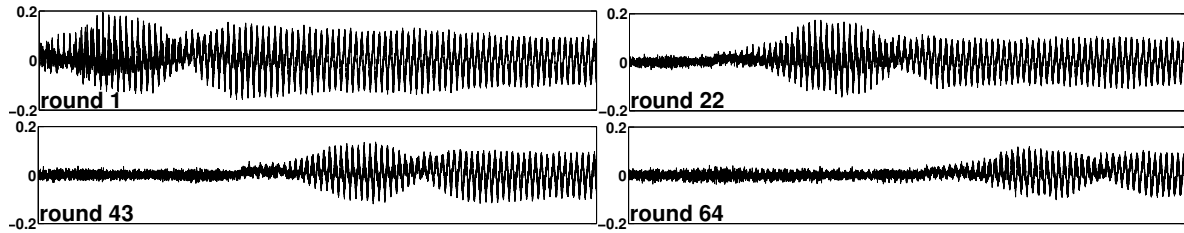


Figure 8.8: Correlation coefficient of the correct key in a CPA attack on the software implementation of a KEELOQ decryption running on a PIC microcontroller. The amplitude decreases with the attacked round number, here depicted for Rounds 1, 22, 43, and 64.

first few key bits, 1000 traces sampled at 125 MS/s are sufficient. Surprisingly, we need roughly ten times as many traces for recovering the full 64 bit K_{man} — a huge effort compared to 5–30 traces for extracting K_{dev} from hardware implementations.

This gradual decrease of the correlation is due to a misalignment that occurs during the execution of the KEELOQ algorithm. The cause for this effect is not a bad trigger condition, since the trigger affects all time instances in the same way. We assumed that either the oscillator of the PIC is very unstable or the program code has a data-dependent execution time for each round of KEELOQ that causes the misalignment with an increasing number of rounds.

8.4 SPA of KeeLoq

This section demonstrates that SPA constitutes a remedy for the open problem of efficiently extracting the manufacturer key from KEELOQ software implementations. A typical software implementation of the algorithm, as recommended by Microchip, is presented in Sect. 8.4.1. On the basis of this KEELOQ instantiation we develop a highly effective SPA attack. Usually, an SPA requires tedious visual inspection, as illustrated in Sect. 8.4.2, or massive profiling of a similar device, which takes a lot of effort and time. A non-heuristic method to avoid the visual inspection for most SPA attacks is introduced in Sect. 8.4.3. Using this method, we then practically attack various commercial KEELOQ implementations on PIC 8-bit microcontrollers in Sect. 8.4.4 and thereby prove the effectiveness of our methods.

8.4.1 Software Implementations of KeeLoq

Meanwhile, source code as proposed by Microchip for a PIC 8-bit microcontroller has become available on the Internet [Web]. Figure 8.9 shows an excerpt of the program code, revealing that actually the execution time of each round in the code example varies, depending on the processed data. While most of the program code takes the same amount of clock cycles, in particular the specific implementation of the look-up table to build the NLF (compare with Sect. 8.2.1) is realized in a time-dependent manner. As a result, the execution time of a decryption varies for different ciphertexts — a typical indicator for a susceptibility towards SPA that is exploited in the following.

```

; DECRYPT using [Key7 . . . Key0]
; | HOP4 | HOP3 | HOP2 | HOP1 |<-- Feed

DECRYPT
00: MOVLW 11+1 ; OUTLOOP COUNTER
01: MOVWF CNT1 ; 11+1 TIMES

OUTLOOP
02: MOVLW 48 ; INLOOP COUNTER
03: MOVWF CNT0 ; 48 TIMES

INLOOP
04: CLRWDI ;
05: MOVWF CNT1 ;
06: XORWF 1 ;
07: SKPNZ ; LAST 48 LOOPS
08: GOTO ROT_KEY ; RESTORE THE KEY

09: CLRC ; CLEAR CARRY
0A: MOVLW 1 ; MASK = 1
0B: BTFSC HOP3,3 ; SHIFT MASK 4X
0C: MOVLW 10000B ; IF BIT 2 SET
0D: MOVWF MASK ;

0E: BTFSS HOP2,0 ; SHIFT MASK
0F: GOTO $+3 ; ANOTHER 2X
10: RLF MASK ; IF BIT 1 SET
11: RLF MASK ;

12: BTFSC HOP1,0 ; SHIFT MASK
13: RLF MASK ; 1X MORE IF BIT 0

14: MOVLW 0 ; TABLE INDEX = 0
15: BTFSC HOP4,1 ; IF BIT 3 SET
16: IORLW 2 ; TABLE INDEX += 2
17: BTFSC HOP4,6 ; IF BIT 4 SET
18: IORLW 4 ; TABLE INDEX += 4

19: ADDWF PC ; PC += TABLE INDEX

TABLE
1A: MOVLW 02EH ; BITS 4:3 WERE 00
1B: GOTO T_END ; END OF TABLE

1C: MOVLW 074H ; BITS 4:3 WERE 01
1D: GOTO T_END ; END OF TABLE

1E: MOVLW 05CH ; BITS 4:3 WERE 10
1F: GOTO T_END ; END OF TABLE

20: MOVLW 03AH ; BITS 4:3 WERE 11
T_END
21: ANDWF MASK ; ISOLATE THE
22: MOVLW 0 ; CORRECT BIT
23: SKPZ ;
24: MOVLW 80H ; W = NLF OUTPUT

25: XORWF HOP2,W ; W XOR= HOP2,7
26: XORWF HOP4,W ; W XOR= HOP4,7
27: XORWF KEY1,W ; W XOR= KEYREG1,7

28: MOVWF MASK ; FEEDBACK = BIT 7
29: RLF MASK ; CARRY = BIT 7

2A: RLF HOP1 ; SHIFT IN
2B: RLF HOP2 ; THE NEW BIT
2C: RLF HOP3 ;
2D: RLF HOP4 ;

ROT_KEY
2E: CLRC ; CLEAR CARRY
2F: BTFSC KEY7,7 ; IF BIT 7 SET
30: SETC ; SET CARRY

31: RLF KEY0 ; LEFT-ROTATE
32: RLF KEY1 ; THE 64-BIT KEY
33: RLF KEY2 ;
34: RLF KEY3 ;
35: RLF KEY4 ;
36: RLF KEY5 ;
37: RLF KEY6 ;
38: RLF KEY7 ;

39: DECFSZ CNT0 ;
3A: GOTO INLOOP ; INLOOP 48 TIMES

3B: DECFSZ CNT1 ;
3C: GOTO OUTLOOP ; OUTLOOP 12 TIMES
3D: RETLW 0 ; RETURN

```

Figure 8.9: A source code for a KEELOQ decryption as proposed by Microchip

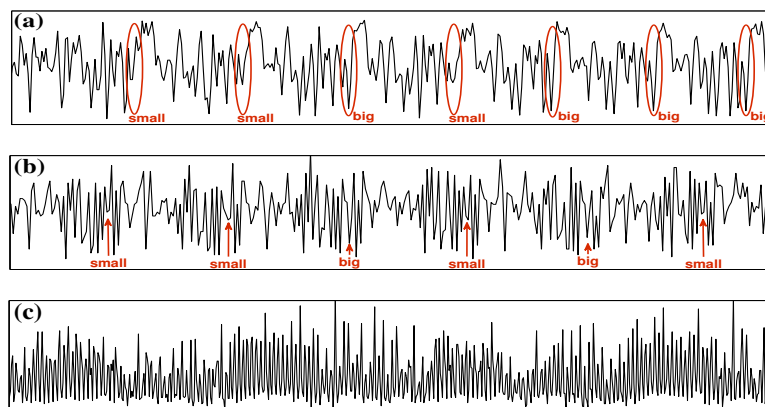


Figure 8.10: Visual inspection of power traces of the KEELOQ cipher.

8.4.2 Visual Inspection of Power Traces

Visual inspection and its utilization in an SPA attack on RSA, as described in Sect. 3.2.2, is well-known in the crypto community. However, the KEELOQ block cipher is very different from public-key schemes such as RSA and there are no distinguishable functions called during the encryption or decryption routines. As illustrated in Sect. 8.4.1, the conditional branches depend on the values of the state register of a software implementation of the KEELOQ decryption recommended by Microchip. Typically, no difference in the power patterns for taking or not taking these branches can be observed for a PIC microcontroller, as it mostly leaks the operands being processed, not the operations. Hence, pinpointing the small variations of two or three instructions between two rounds of the algorithm by visual inspection is a very challenging and sometimes impossible task.

Figure 8.10 shows power traces measured from commercial implementations of the KEELOQ decryption on different PIC microcontrollers. Manually analyzing the details of the power consumption, in few cases a distinguishable behavior can be spotted in the periodic power patterns of the microcontroller, as illustrated in Fig. 8.10a and Fig. 8.10b. If the difference in these patterns would furthermore directly depend on the values of the status register, a key recovery became possible. However, Fig. 8.10c illustrates that in some cases no difference between the periodic patterns can be detected by means of heuristic methods, even by averaging the traces as detailed in Sect. 3.2.4. Note that Fig. 8.10 shows pure measurements without averaging, directly sampled by the oscilloscope.

8.4.3 A Non-Heuristic Method for SPA Attacks

In the following, we develop a non-heuristic method that allows for automatically identifying differences in power traces — as required for an SPA — even for those implementations in which a visual inspection is not effective. First we examine the time-variations occurring during a KEELOQ decryption more precisely, using the example of a program code proposed by

Table 8.1: Counting clock cycles required by selected conditional branches, as a function of their input parameters.

CNT0=1	HOP4, 1=1 and HOP4, 6=1	HOP2, 0=1	No. of clock cycles	
			#	# mod 2
X	X	X	2	0
X	X	✓	3	1
X	✓	X	0	0
X	✓	✓	1	1
✓	X	X	6	0
✓	X	✓	7	1
✓	✓	X	4	0
✓	✓	✓	5	1

Microchip. The conclusions drawn from analyzing this code can be applied to many different, unknown implementations of KEELOQ decryptions⁶ on PIC microcontrollers.

Scrutinizing the Program Code of KeeLoq

The duration of all conditional branches that take place during the decryption is examined in Fig. 8.11 by comparing the number of clock cycles required for each instruction of a PIC microcontroller [Mici]. While the decisions taken in the code excerpts (a), (c), (d), (e), (g), and (h) shown in Fig. 8.11 do not affect the execution time, the number of cycles required for (b), (f), and (i) varies with the respective condition being fulfilled or not. Hence, the time variations in different rounds are due to the lastly mentioned three conditional branches.

Pinpointing the Time Variances Table 8.1 summarizes the effect of the conditional branches on the clock cycles required for the execution of one KEELOQ round. The duration of the program code in (f) and (i) of Fig. 8.11 can increase by a multiple of two cycles, depending on the state of the checked register, while (b) can increase the length of a round by one cycle. As a consequence, taking the *execution time of each round modulo 2* can reveal the result of the decision taken in (b), where the value of HOP2,0, i.e., the 9th bit of the status register, y_8 , is tested. Counting the amount of clock cycles required for one round and deciding whether the number is even or odd allows deducing one bit of the status register from the duration of each round.

Automated Inspection of the Power Traces

As shown in Sect. 8.4.2, visual inspection is not feasible for some implementations — even less can the length of each round be precisely detected. In the following, we thus introduce a non-heuristic technique for determining the number of cycles in each round.

⁶In fact, the described method allows for extracting K_{man} from all implementations we are aware of.

1	0	= HOP3,3
1	2	0B: BTFSC HOP3,3
1	0	0C: MOVLW 10000B
2	2	sum

(a) HOP3,3

1	0	= HOP2,0
2	1	0E: BTFSS HOP2,0
0	2	0F: GOTO \$+3
1	0	10: RLF MASK
1	0	11: RLF MASK
4	3	sum

(b) HOP2,0

1	0	= HOP1,0
1	2	12: BTFSC HOP1,0
1	0	13: RLF MASK
2	2	sum

(c) HOP1,0

1	0	= HOP4,1
1	2	15: BTFSC HOP4,1
1	0	16: IORLW 2
2	2	sum

(d) HOP4,1

1	0	= HOP4,6
1	2	17: BTFSC HOP4,6
1	0	18: IORLW 4
2	2	sum

(e) HOP4,6

6	4	2	0	= W Reg
2	2	2	2	19: ADDWF PC
0	0	0	1	1A: MOVLW 02EH
0	0	0	2	1B: GOTO T_END
0	0	1	0	1C: MOVLW 074H
0	0	2	0	1D: GOTO T_END
0	1	0	0	1E: MOVLW 05CH
0	2	0	0	1F: GOTO T_END
1	0	0	0	20: MOVLW 03AH
3	5	5	5	sum

(f) W Reg

1	0	= MASK
1	2	23: SKPZ
1	0	24: MOVLW 80H
2	2	sum

(g) MASK

1	0	= KEY7,7
1	2	2F: BTFSC KEY7,7
1	0	30: SETC
2	2	sum

(h) KEY7,7

≠1	1	= CNT0
1	2	39: DECFSZ CNT0
2	0	3A: GOTO INLOOP
0	1	3B: DECFSZ CNT1
0	2	3C: GOTO OUTLOOP
0	1	02: MOVLW 48
0	1	03: MOVWF CNT0
3	7	sum

(i) CNT0

Figure 8.11: Number of cycles required for the execution of an exemplary implementation of the KEELOQ decryption, depending on the conditional branches.

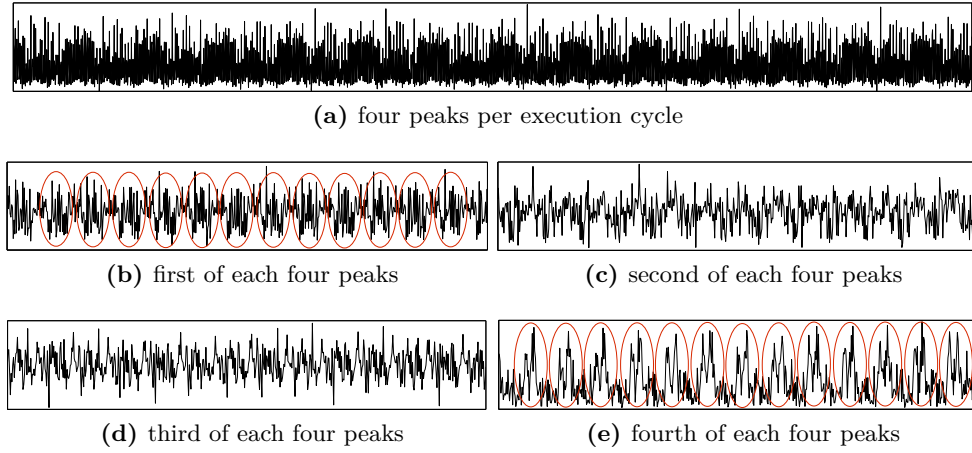


Figure 8.12: Extracting peaks of the power consumption trace of a PIC microcontroller executing KeeLoq.

Power Leakage of PIC microcontrollers Each execution cycle of a PIC microcontroller lasts four clock cycles [Mici], hence four peaks of a power trace relate to one execution cycle. Figure 8.12 shows extracted peaks of power traces of a PIC microcontroller executing a KEELOQ decryption. Extracting all four peaks of an execution cycle, as illustrated in Fig. 8.12a, does not allow to locate the rounds of the decryption algorithm. In an attempt to facilitate the round detection, only the first, second, third or fourth peak of each execution cycle are taken into account to yield Fig. 8.12b, (c), (d) and (e), respectively. While focusing on the second or third peak does not improve the detectability of the KEELOQ rounds, confining the analysis to the first or the fourth peak of each execution cycle, as shown in Fig. 8.12b and Fig. 8.12e, allows for accurately distinguishing successive rounds.

Scrutinizing the Timing In order to pinpoint the duration of each round of the algorithm, the cross-correlation between periodic patterns in the traces and a reference pattern is computed similarly to [MDS99b]. A reference pattern $\mathcal{R} = (r_1, r_2, \dots, r_l)$ with a length of l is generated such that it contains the power-peaks of one particular round of the KEELOQ decryption. Furthermore, a vector containing the power-peaks of a whole KEELOQ decryption, with a length of n , is denoted by $\mathcal{P} = (p_1, p_2, \dots, p_n)$. Then, a vector $\mathcal{C} = (c_1, c_2, \dots, c_{n-l+1})$ can be computed, where the c_i are obtained by correlating $\mathcal{R}$ with each section of $\mathcal{P}$, i.e., $(p_i, p_{i+1}, \dots, p_{i+l-1})$.

As illustrated in Fig. 8.13, the rounds can be clearly identified as consecutive maxima of $\mathcal{C}$. The locations of these maxima reveal the exact length of each round. Taking the length of each round modulo 2 discloses the value of the bit processed by the algorithm in that time instant. Hence, one bit after another the content of the state register and finally the bits of the secret key are revealed, as described next in Sect. 8.4.4.

8.4.4 Developing an SPA Attack for KeeLoq

Let us denote the content of the state register of all rounds during a KEELOQ decryption by a bitstream $S = \{s_j ; -31 \leq j \leq 528\}$. After initializing the first 32 bits $\{s_{-31}, s_{-30}, \dots, s_0\}$

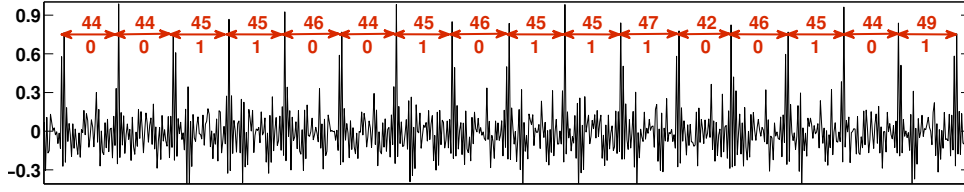


Figure 8.13: An example for the correlation coefficients in vector $\mathcal{C}$. The exact duration of each round of KeeLoq, and thus whether the amount of clock cycles per round is even or odd, can be clearly identified.

of the bitstream with the corresponding ciphertext bits $\{c_{31}, c_{30}, \dots, c_0\}$, the bits with indices $1 \leq j \leq 528$ can be computed according to Step 2 of Alg. 2 using the recursive equation

$$s_{j+1} = k_{(15-j) \bmod 64} \oplus s_{j-31} \oplus s_{j-15} \oplus \text{NLF}(s_{j-30}, s_{j-25}, s_{j-19}, s_{j-8}, s_j). \quad (8.4)$$

According to Eq. 8.4, one bit of the secret key $k_{(15-j) \bmod 64}$ can be revealed from the knowledge of eight bits of the stream S . For extracting all 64 bits of the key, a consecutive section of the stream with $32 + 64 = 96$ bits is sufficient. Note that in a typical known-plaintext or known-ciphertext scenario up to 32 bit of the required stream might be known a priori.

The whole 64-bit secret key is recovered by means of SPA from analyzing the execution time of 64 consecutive rounds. The value of one key bit being processed is simply deduced from the observation whether the amount of clock cycles per round is even or odd. Even if neither the ciphertext nor the plaintext is known prior to the attack, extracting the secret key succeeds by analyzing 96 consecutive rounds of one measurement — the key-recovery attack is feasible with unknown input and unknown output of the cipher.

However, there are still three remaining problems:

- (1) Due to noise in the traces of the power-consumption, the detection of the length of individual rounds may fail, leading to an incorrect detection of bits. Thus, a method to verify the recovered key bits would be convenient.
- (2) The efficiency of the illustrated method depends strongly on the accuracy and correctness of the reference pattern $\mathcal{R}$. This demands for a method for choosing an accurate reference pattern.
- (3) Suppose that all 64 bits of the key are recovered correctly. Since the key bits are used periodically during the KEELQ decryption and the attack can be started at any point in time with respect to the beginning of the decryption, the correct position of the recovered key bits in the secret key is not clear. Thus, the correct order of the bits needs to be found out of 64 different alternatives.

Solutions for these problems are elaborated next.

Error Correction

Suppose all bits of the secret key are deduced with the described attack and let $\hat{S} = \{\hat{s}_j ; 1 \leq j \leq 528\}$ be the resulting bitstream, containing a part of the stream S of a decryption. The

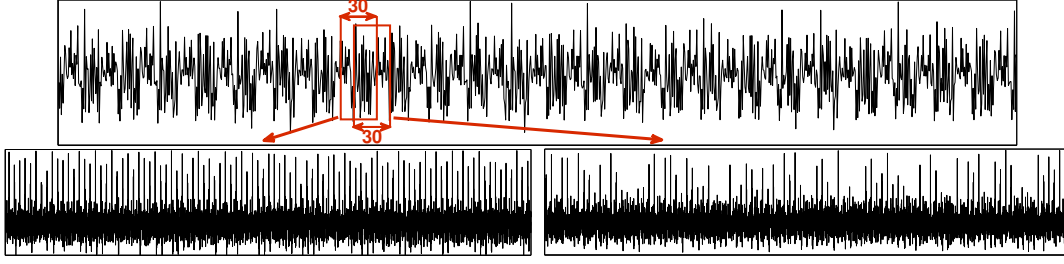


Figure 8.14: Two exemplary reference patterns resulting in different correlation vectors c . The pattern generating the left bottom plot is more appropriate, due to periodical peaks.

corresponding $\widehat{K} = \{\widehat{k}_i ; 0 \leq i \leq 527\}$ contains the key bits computed from $\widehat{S}$. As each key bit is used at least eight times — every 64 rounds of the decryption — the correct key bits reappear in $\widehat{K}$. Let $\widetilde{K}_l = (\widehat{k}_l, \widehat{k}_{l+1}, \dots, \widehat{k}_{l+63})$, $0 \leq l \leq 64$, be a part of $\widehat{K}$ with a length of 64 bits, then

$$\exists i, l ; i \neq l, i = l \bmod 64, \widetilde{K}_i = \widetilde{K}_l.$$

Accordingly, errors in the detection of the correct key-bits due to noise can be corrected by a majority decision.

Generating an Appropriate Reference Pattern

Since the characteristics of the power consumption strongly depend on the **DUT**, the best basis for the reference pattern is a part of the power peaks produced by the **DUT** itself. As Fig. 8.12b illustrates, the durations of the rounds can be estimated by visual inspection of a decryption. Comparing with the source code described in Sect. 8.4.1 one can estimate that each round takes between 42 and 49 execution cycles — a reference pattern with a length of approximately 30 cycles is hence adequate. However, as the beginning and the end of a round can only be guessed, the best position of the reference pattern in the power-peaks has to be found by moving the window until the correlation vector c contains regular maximums with a similar amplitude. In Fig. 8.14, two c vectors of the same power-peaks are plotted for two different reference patterns of length 30 clock cycles — the vector on the left-hand side is more appropriate.

Order of the Key Bits As detailed in Sect. 8.2.3, a device key K_{dev} is obtained by a KEELOQ decryption of the corresponding serial number of a remote control. Suppose that K_{dev} is already known from performing a **CPA** attack on the remote control (see Sect. 8.3) and that $\widetilde{K}_k$ contains the correct bits of the secret manufacturer key. With $\widetilde{K}_k^{(i)}$ denoting a rotation of the bits⁷ of $\widetilde{K}_k$ the correct secret key is found by rotating i times, where $0 \leq i < 64$, if

$$\exists i ; f(\widetilde{K}_k^{(i)}, serial) = K_{dev},$$

where f denotes the key derivation function as detailed in Sect. 8.2.3. Hence, a known device key K_{dev} can be used to verify the correctness of the revealed key bits and furthermore simplifies to find the correct number of rotations i of $\widetilde{K}_k$.

⁷The direction of the rotations is not important, as long as it remains the same.

Results of Power Analysis (Software)

The power traces of several PIC microcontrollers, such as PIC16C56 and PIC16F84A, were acquired with an Agilent Infiniium 54832D digital oscilloscope at a sampling rate of 125 MS/s by measuring the differential voltage of a $100\,\Omega$ resistor inserted in the ground path. Using the presented techniques we are able to extract the secret manufacturer key K_{man} of commercial KEELOQ code hopping receivers from only one single power trace, without knowing any input or output of the attacked KEELOQ decryption. The efficiency of our attack is due to a software implementation containing key-dependent conditional branches and due to the nature of the KEELOQ cipher, i.e., using the key bits more than once.

Handling an Interrupt-based Countermeasure In most real-world applications of KEELOQ, the microcontroller computing the decryption is performing various others tasks simultaneously, e.g., involving calls of Interrupt Service Routines (ISRs). Due to the interrupts, the run time of a decryption will be very different for each execution of KEELOQ, i.e., the program flow becomes unpredictable, which hinders a straightforward DPA or SPA. Still, as shown in [KKMP09], the disturbing influence of these parasitic effects can be removed with special pre-processing techniques, so that the attacks described in this chapter finally become applicable again. Consequently, calls of ISRs that not depend on intermediate values during an execution of the cipher constitute no countermeasure in the context of power analysis.

8.4.5 Comparison of DPA and SPA

As shown in Sect. 8.3.3, hardware implementations of KEELOQ, such as HCS301 [Micf] ASICs, are an ideal platform for conducting DPA or CPA attacks that naturally process multiple traces. The timing behavior of the chip can be foreseen very precisely, as it always performs exactly the same digital operations independent of the secret key. This implies that the power consumption at each point in time of the acquired traces is always related to the same step of the KEELOQ cipher, and extracting device keys K_{dev} with DPA techniques is relatively straightforward. In contrast, the data-dependent variations of the execution time of KEELOQ implementations in software imply misaligned traces. This misalignment complicates a CPA and can hence be regarded as a countermeasure. On the other hand, the data-dependency enables an SPA.

Generally, the effort for performing an SPA is significantly smaller than that for a DPA, because the latter naturally requires acquiring many traces and a lot of memory for storing and evaluating them. Analyzing commercial black-box implementations with DPA moreover poses the in practice sometimes difficult tasks of triggering the oscilloscope and aligning the measurements accurately. The SPA described in Sect. 8.4 requires neither alignment nor a lot of memory, as one measurement starting at an almost arbitrary point⁸ during a decryption is sufficient for a full key recovery. Furthermore, the presented SPA requires no knowledge about neither the plaintext nor the ciphertext of the attacked decryption, as all necessary parameters for the SPA can be derived solely from the power measurements. A DPA is clearly impossible under these premises.

Summarizing, the outcome that conducting a DPA is difficult for an unknown implementation does not imply that the implementation is more secure. On the contrary, it may turn out —

⁸Any starting point that captures ≥ 96 rounds of KEELOQ is appropriate

as demonstrated in this chapter — that an even simpler and much more effective attack is applicable, due to data-dependent execution times in the algorithm.

Implementing the cipher such that the duration of each conditional branch takes equally long for any input will most likely prevent from a key recovery with the SPA illustrated in Sect. 8.4. This will on the other hand facilitate an extraction of the secret key via DPA of the — now well aligned — measurements. As a consequence, the constant execution time is only the first step towards an SCA-resistant implementation: the algorithm further needs to be protected with DPA countermeasures. One example of such a protected implementation is given in Sect. 8.6.

8.5 Attacks and Implications

In the previous section we have shown how the keys of hardware and software implementations of KEELOQ can be recovered by a moderately skilled and equipped adversary using SCA. Next, we evaluate the vulnerability of commercial systems to our attacks and illustrate the implications of recovering secret keys in the real-world. We detail four different attack scenarios, which allow for breaking basically any RKE system using KEELOQ with little effort. We focus on code-hopping applications, since they are more commonly used and, due to the lack of known plaintexts, harder to cryptanalyze than IFF systems.

8.5.1 Cloning a Remote Control

For cloning a KEELOQ transmitter using power analysis, an adversary needs physical access to it, in order to acquire at least 10 to 30 power traces: The button of the remote control has to be pressed several times, while measuring the power consumption and monitoring the transmitted hopping-code messages to obtain the ciphertexts. After recovering the device key K_{dev} with the side-channel attack described in Sect. 8.3.3, the recorded messages are decrypted. Hence, the discrimination value *disc* and the counter value *cnt* of the original transmitter at the time of the attack are disclosed⁹. Now, the HCS IC of a spare remote control can be programmed with the serial number, *cnt*, and *disc* of the master remote control. Consequently, the freshly produced clone of the transmitter appears to be genuine to a receiver and allows for accessing the same target as the original.

Implications This attack applies to scenarios in which a remote control is handed over to an untrustworthy person for some minutes, e.g., car rental or cleaning personnel. Possessing the transmitter for some minutes, an attacker can clone it. Depending on the time interval between producing the clone and using the reproduced remote control, the attacker will have to press the button of the remote control several times for catching up with the counter value in the receiver, which might have been increased meanwhile by the legitimate operator.

Given that a technically demanding SCA has to be carried out in order to clone just one remote control, and physical access to it is required, the impact of this attack is somewhat limited. The cost-benefit ratio is low, except for the case that very high monetary values are involved, e.g., rental of expensive cars. In most cases it is easier, for instance, to smash a window in order to gain access to a secured object. Nevertheless, the attack is interesting for an

⁹Refer to Sect. 2.3.2 for an explanation of *disc*, *cnt*, and other details of a rolling-code scheme.

adversary intending to remain unnoticed or who needs to gain access to an object repeatedly, e.g., intelligence agencies.

8.5.2 Recovering a Manufacturer Key

The key recovery of the manufacturer key K_{man} depends on the particular key derivation scheme. If scheme (b) in Fig. 8.3 is used, i.e., an XOR of a known input and the manufacturer key K_{man} , disclosing the latter is trivial as discussed in Sect. 8.2.3. After a successful key recovery attack on any remote control (see above) of the same brand, K_{man} is found by reversing the XOR function. The known input is either part of each hopping code message in case of the serial number, or can be obtained by pressing a special button on the remote control in case of a seed being involved in the key derivation¹⁰. The derived K_{man} , and whether scheme (b) is employed, can be verified by applying the same process to a second remote control. A receiver is not required to obtain K_{man} for scheme (b).

On the contrary, an adversary targeting the manufacturer key K_{man} for scheme (a) of Sect. 8.2.3 requires physical access to one receiver of that manufacturer. The secret key, recovered by means of the SPA described in Sect. 8.4, can be verified with two monitored code-hopping messages: First, the device key K_{dev} is computed from the serial number and the extracted K_{man} . Then, the two hopping codes are decrypted. If their *disc* values are identical and *cnt* is increased by one, K_{man} is correct. Note that all the prior cryptanalyses [IKD⁺08, Bog07, CBW08] of KEELOQ practically fail to recover K_{man} for scheme (a), unless they would recover the secret keys K_{dev} of at least 2^{16} different remote controls.

Knowing the manufacturer key K_{man} , valid device keys for producing new remote controls with arbitrary serial numbers can be generated, just by applying the respective key-derivation scheme. The counterfeit remote controls will be recognized as genuine by the receivers of the targeted manufacturer.

Implications The described key recovery requires access to a transmitter (key derivation by XOR) or a receiver (key derivation by KEELOQ decryption) of the manufacturer to be attacked and, in case of a key derivation employing KEELOQ decryptions, a skilled adversary for performing the SCA. An RKE system of the target manufacturer can be purchased, e.g., from a hardware store, and even could be returned after extracting K_{man} from the receiver. The recovered K_{man} does not directly permit to unauthorizedly open a door secured by KEELOQ, because the newly produced transmitters need to be registered at the receiver first, which requires physical access to the latter. Still, the described key recovery is highly relevant in the context of product piracy: the economic function of K_{man} is customer retention, e.g., a business model could rely on making the main profit on selling spare remote controls that cooperate only with the receivers of that manufacturer. However, a competitor possessing K_{man} can produce arbitrary remote controls compatible to those receivers, or even produce whole RKE systems being compatible to those of the original manufacturer, and hence severely affect the business of the latter. Due to the depicted significant economic impact, it is highly conceivable that manufacturer keys of competitors will be extracted by criminals sooner or later. In the worst

¹⁰The seed is stored in a KEELOQ remote control and is sent in plain to a receiver, upon pressing a dedicated button on the remote control.

case, this might result in publicly available lists of manufacturer keys on the Internet that would render the RKE systems of all these manufacturers instantly insecure.

8.5.3 Cloning a Remote Control by Eavesdropping

Possessing the manufacturer key K_{man} and knowing the key derivation method of a target RKE device family, a remote control can be cloned by eavesdropping from a distance: The adversary has to intercept at most two hopping code messages, c_1 and c_2 , sent by the target transmitter of the respective manufacturer¹¹. As shown in Sect. 2.7.2, such attacks are practically feasible from several hundred meters. The process of finding the secret key of the eavesdropped transmitter depends on the key derivation schemes detailed in Sect. 8.2.3.

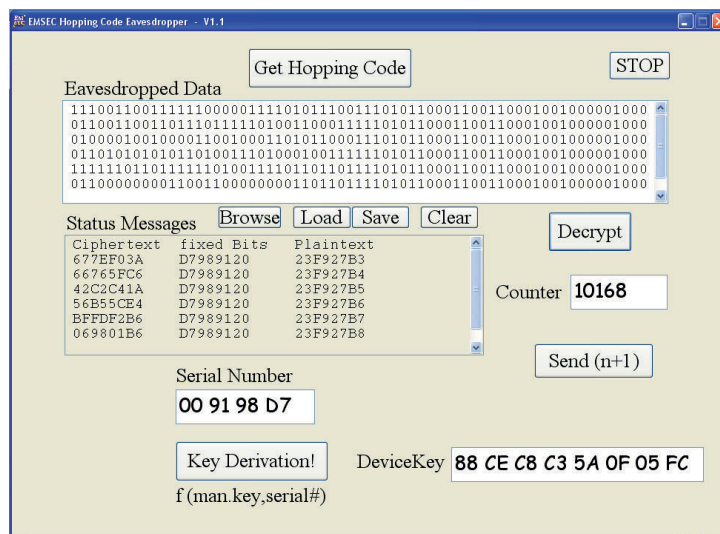


Figure 8.15: Screenshot of a GUI for eavesdropping, deriving K_{dev} , decrypting, and sending new valid KeeLoq hopping codes, with exemplary eavesdropped codes. The decrypted counter (Plaintext) increases with each code.

If the key is derived from the serial number of the remote control, finding its device key K_{dev} is straightforward, since the intercepted messages contain the serial number in plain. The adversary can simply perform the key derivation using the known K_{man} to obtain the device key. After decrypting one message c_i with K_{dev} and thereby disclosing the current counter value cnt , the adversary can generate the next valid hopping code messages by increasing cnt and encrypting it accordingly. For practical attacks in the field, we have developed a software with a GUI as illustrated in Fig. 8.15 that performs all required computations. The attack laptop executing the software is connected to a modified receiver for eavesdropping data and to a modified transmitter for sending hopping code generated by the laptop.

The eavesdropping attack allows for spoofing the receiver and gaining access to a protected site with the computational complexity of a single KEELQ decryption. However, if a seed value is used during the key derivation, recovering the secret key of a remote control only by

¹¹The second message is only required for verifying off-line that the correct key has been derived from the first message.

eavesdropping is more demanding: an exhaustive search as detailed in Sect. 8.5.5 needs to be performed to find the seed value.

Implications For monitoring hopping codes from a range of up to several hundred meters, depending on the targeted RKE system, it is sufficient to connect the RF part of a legitimate receiver to the parallel port of a (laptop) PC (see Sect. 4.2). This attack has the most devastating impact and scales very well. It affects all KEELOQ RKE systems of a manufacturer, as soon as the respective K_{man} is known. Extracting K_{man} turns out to be even less demanding than extracting a K_{dev} from a remote control, as described in Sect. 8.4. Possessing K_{man} , a remote control can be cloned from successfully eavesdropping on one hopping code sent by the remote, using hobbyist equipment and know-how. The one-time key recovery of K_{man} can be outsourced to criminal cryptographers who may construct a machine that eavesdrops on two messages, automatically extracts the device key, and opens the target. Such a machine enables a completely unskilled person to gain illicit access to objects secured with KEELOQ, at little cost and without leaving any traces. Note that the manufacturer himself (knowing his own key K_{man}) can access any object secured with one of his products, by monitoring a single hopping code sent by a genuine remote — a further unwanted threat for the privacy of the customers.

8.5.4 Denial of Service

As detailed in Sect. 8.2.2 the counters of a receiver and a remote control are synchronized with every received, valid hopping-code message. This behavior can be exploited for putting an RKE system out of operation: We assume that the adversary has recovered the device key K_{dev} of a target transmitter by performing one of the attacks described above and is thus able to generate valid hopping code messages. She sets the counter value to the maximum value inside the resynchronization window and sends two consecutive valid hopping codes. As a result, the receiver resynchronizes its counter to the new value. Consequently, the counter of the original remote control is at the beginning of the block window and considered to be outdated, i.e., its respective hopping codes are ignored by the receiver. The owner of the original transmitter would have to press the button very often, typically $2^{15} = 32768$ times, to increase the counter value forward into the first counter window, where the remote control produces valid hopping code messages. While the legitimate owner of the remote control is now locked out, the adversary still has full control of the system, as she can send hopping codes with the appropriate counter values.

Implications This attack allows for locking out a legitimate user, leaving the impression that the KEELOQ RKE device is out-of-service. The attack can be performed by an unskilled adversary: As described in Sect. 8.3.4, connecting a modified remote control enables a standard PC to transmit self-generated hopping codes. Accordingly, a victim can be locked out with little effort, while the secured object is still accessible for the attacker. The software for generating valid hopping codes could be provided by a skilled criminal, e.g., via the Internet. The attack can have dramatic consequences, especially for the automotive industry, where reliability is of paramount importance. Apart from compromising the corporate image, the additional costs for increased customer support, e.g., fixing the spoofed devices, have to be considered.

8.5.5 Revealing Device Keys with Brute-Force Attacks

The previous sections have illustrated how to instantly override the security of all KEELOQ code-hopping schemes in which the secret key of a remote control is derived from its serial number. In contrast, if a unique random seed is involved in the derivation of the secret key of each remote control (compare with Sect. 8.2.3) the cryptanalysis demands for higher computational power and becomes practically infeasible with a standard PC. In this section, a hardware architecture for the cryptanalysis of KEELOQ is developed, in order to estimate the security margin for the case that a seed is used during the key derivation. For compatibility with the eavesdropping attacks described in Sect. 8.5.3, we again assume a scenario in which an attacker can monitor two valid hopping codes transmitted by the remote control to be attacked. Our brute-force attack described in the following is implemented on the COPACOBANA (see Sect. 4.5). The goal is to find the correct secret key K_{dev} of a remote control from two intercepted hopping codes by means of exhaustive key-search, when a random seed is involved in the generation of the device key.

Key Derivation with Random Seeds

As an alternative for obtaining the device key on the side of the receiver, a randomly generated seed value may be combined with the serial number for the key derivation (see Sect. 8.2.3). The seed is unique for each remote, stored in the IC of the latter, and can be transmitted to the receiver upon a particular action, e.g., pressing a special button on the remote control.

Three different seed sizes, i.e., 32-bit, 48-bit or 60-bit, can be used in KEELOQ systems, depending on the chip family. Microchip accordingly proposes three patterns to form the input of the key derivation on the basis of the scheme depicted in Fig. 8.3.a):

- (1) 28 bits of the serial number (N) are combined with 32 bits of the random seed (S) according to the pattern 0x0N NN NN NN SS SS SS SS,
- (2) 12 bits of the serial number are combined with 48 bits of the seed corresponding to the pattern 0x0N NN SS SS SS SS SS SS, or
- (3) 60 bits of the seed are used to form the pattern 0x0S SS SS SS SS SS SS SS.

The 32 MSBs of the device key K_{dev} are derived by a KEELOQ decryption of the higher 32 bits of the input value, and the 32 LSBs are likewise generated from the lower 32 bits of the input. Depending on the size of the random seed, the degree of freedom of the input varies: the lower 32 bits of the device key are always random, while the upper 32 bits may have either a fixed value, one out of 2^{16} potential values, or one out of 2^{28} potential values. Consequently, when implementing a brute-force attack each combination of the 32 MSBs of the device key may be precomputed in software and then combined with all 2^{32} combinations of the 32 LSBs that are generated in hardware by means of a counter, until the correct seed and the corresponding device key K_{dev} is identified.

Implementing an Exhaustive Device Key Search

For verifying whether a correct key candidate is found, two hopping codes c_1 and c_2 of the same remote control are required. For example, the codes can be intercepted by means of

eavesdropping as described in Sect. 8.5.3. The eavesdropped codes are generated with the same device key, have identical discrimination and function values, but naturally originate from different counter values. The difference between the counter values must be sufficiently small, i.e., in the best case two consecutive (or almost consecutive) transmissions of the remote are monitored.

To recover K_{dev} , the adversary decrypts the two intercepted messages using the known manufacturer key K_{man} and a hypothesized seed value, for all possible values of the seed. Accordingly, our implementation computes $K_{dev}^{seed} = \text{KeyDerivation}(K_{man}, seed)$ for each possible value of $seed$ and decrypts the two intercepted messages c_1 and c_2 using K_{dev}^{seed} :

$$(cnt_1, disc_1) = \text{KEELOQ}^{-1}(c_1, K_{dev}^{seed}) \quad (cnt_2, disc_2) = \text{KEELOQ}^{-1}(c_2, K_{dev}^{seed})$$

If both decrypted messages have the same discrimination value, i.e., $disc_1 = disc_2$, and similar counter values¹² cnt_1 and cnt_2 , a candidate for the device key is found. Note that with a small probability false positives are found for K_{dev} . These can be easily identified by sending one hopping code to the target receiver.

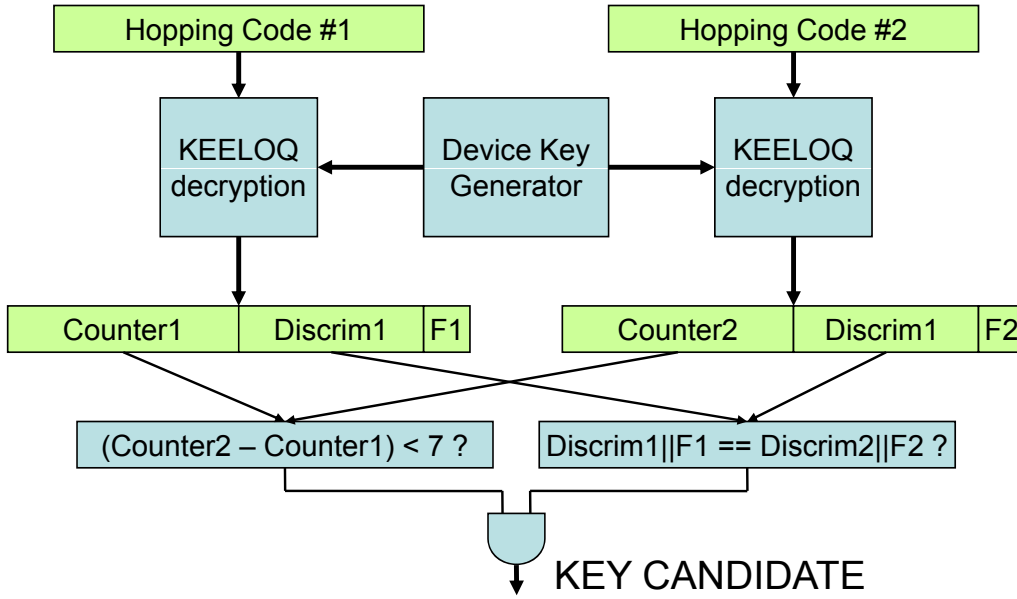


Figure 8.16: Principle of the hardware architecture for an exhaustive key search on the basis of two intercepted KEELoQ hopping codes.

We implemented the described brute-force attack of KEELoQ on the parallel computation cluster **COPACOBANA** detailed in Sect. 4.5. The diagram of the circuit implemented in each **FPGA** is shown in Fig. 8.16. The core of the implementation is a key generator consisting of a 32-bit register and a 32-bit counter. The register holds 32 **MSBs** of the device key (being precomputed in software and assigned by the host computer), while the counter is repeatedly

¹²“Similar” counters means that the difference $cnt_2 - cnt_1$, depending on the time period between intercepting the two hopping codes, is less than an implementation-dependent boundary, e.g., 16.

increased in hardware to generate all possible values for the lower 32 bits of the device key. Two KEELOQ decryptions are implemented in parallel. If the discrimination values of the decrypted hopping codes are identical and the difference between the counters is smaller than seven, the (correct) key candidate is output. If all counter-values have been generated and no key candidate has been found, a new value for the upper 32 bits of the key is assigned to the **FPGA**.

In a straightforward implementation, a KEELOQ decryption is executed in 528 rounds. For our highly efficient realization on the **COPACOBANA**, we unrolled both decryption units into a pipeline structure and optimized the number of pipeline stages according to the available resources on the **FPGAs**. As a result, each path of the pipeline consists of 132 stages, i.e., each stage contains four rounds of the cipher. In sum, the KEELOQ breaker occupies 6423 out of 7680 slices (83%) of each Xilinx Spartan 3-1000 **FPGA**. According to the critical path, the maximum achievable clock frequency for the **COPACOBANA** is 110 MHz, i.e., each **FPGA** can test up to 110 million keys per second.

Results of the Exhaustive Key Search

Given two eavesdropped KEELOQ hopping codes, our brute-force attack is able to find the secret key of a remote control. If a 32 bit seed value is used, for example in HCS200, HCS201, HCS300, HCS301 and HCS320 KEELOQ code hopping encoders, the adversary has to run on average 2^{32} KEELOQ decryptions to find the correct seed. This takes $2^{32}/120 \cdot 110 \cdot 10^6 \approx 0.33$ seconds on one **COPACOBANA** in the worst case. Finding the correct device key in case of a 48-bit seed, e.g., provided by HCS360 and HCS361 chips, takes up to $2^{48}/120 \cdot 110 \cdot 10^6$ seconds ≈ 5.9 hours on one **COPACOBANA**. For the 60-bit seed, such as used in a HCS410, the duration of the key search is up to $2^{60}/120 \cdot 110 \cdot 10^6$ seconds ≈ 1011 days on one **COPACOBANA**. Note that the attack is arbitrarily parallelizable and could thus be run on multiple **COPACOBANAs** to decrease the attack time. Worst-case durations for all possible seed lengths are summarized for one **FPGA**, one **COPACOBANA** and 100 **COPACOBANAs** in Table 8.2.

Table 8.2: Worst case times for the brute force attack on KEELOQ

SEED length (bits)	1 FPGA (< 80 \$)	1 COPACOBANA (< 10000 \$)	100 COPACOBANAs (< 1000000 \$)
32	39 secs	0.33 secs	3.3 msecs
48	29.6 days	5.9 hours	213 secs
60	332 years	1011 days	10.1 days

We conclude that using a 32-bit seed provides no security, since a key can be found in real-time. While a seed with 48 bits can be broken in less than 6 hours by one **COPACOBANA**, employing a 60-bit seed can provide an acceptable security margin: running 2^{60} KEELOQ decryptions is not feasible in a realistic time with currently existing computers or hardware. Though rarely found in commercial applications, the key derivation from random seeds can according to our analyses improve the security of KEELOQ rolling code systems to a reasonable level. We would like to stress that *none* of the real-world KEELOQ systems we analyzed used a seed. Moreover, if physical access to the remote control is given, even 60-bit seed values are obtained by pressing the respective button on the remote control.

8.6 A Power-Analysis Resistant RKE System

In the previous sections it has been shown that power-analysis attacks allow to completely break real-world **RKE** systems based on the KEELOQ code-hopping scheme. A successful key-recovery involves a couple of security and privacy risks for the consumers as well as for the manufacturers. In this section we introduce a new **RKE** system that is inherently resistant against most side-channel attacks, independent of the implementation platform, and hence could be used as a replacement for remote controls relying on the insecure KEELOQ scheme.

8.6.1 Introduction and Related Work

Several schemes have been proposed to protect cryptographic implementations with respect to side-channel analysis, particularly to **DPA**. The goal of **DPA** countermeasures is to avoid a dependency between the power consumption of a cryptographic device and intermediate values of the executed algorithm [MOP07] — however, none of the so far proposed practical countermeasures can provide bullet-proof security.

A new approach to thwart side-channel analysis has been presented in ASIACCS 2008 [PSP⁺08]. The authors propose a re-keying strategy via a **PRNG** that is based on a block cipher and theoretically investigate the resulting side-channel leakage of the implementation. The basic idea is that each secret key is used only once, hence those side-channel attacks requiring to combine several queries in order to reach high success rates, such as **DPA**, become ineffective. Still, this method remains susceptible to the most powerful key-recovery attacks based on side-channel analysis, namely template attacks [CRR02], that — after a profiling phase in which the attacker can set keys and other parameters of the cipher — require just a single leakage query. The **PRNG** should therefore be combined with other countermeasures such as hiding to counteract template attacks more efficiently.

Our Contribution

The vulnerability of KEELOQ **RKE** systems and the aforementioned secure **PRNG** inspired us to develop a new **RKE** scheme that is inherently resistant to power-analysis attacks and can provide reliable security and privacy to the customers. For the new approach, the design of the **PRNG** is modified, to achieve a reasonable resistance against template attacks, and embedded into a secure remote-control application. The **PRNG** serves as a **DPA**-resistant core of our proposal. The scheme is implemented on a general-purpose 8-bit μC , as used in many commercial **RKE** systems. Analyzing and evaluating the final realization, we ensure that its performance is suitable for commercial systems and demonstrate that a practical key-recovery by means of power-analysis is not feasible.

Outline

The basic idea of our new **RKE** system is presented in Sect. 8.6.2, including a description of its functionality and details about distributing its secret parameters. Section 8.6.3 then analyzes the security of the basic idea. Considering the remaining vulnerabilities pinpointed during the analysis, the system is modified to withstand advanced security analyses. As a proof-of-concept,

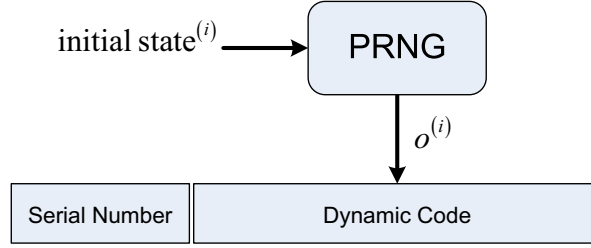


Figure 8.17: Principle of generating Dynamic Codes and message format of the proposed PRNG-based RKE system.

the developed, final **RKE** application is implemented and tested on a μC in Sect. 8.6.4. Finally, difficulties and limitations in the context of real-world applications are addressed.

8.6.2 PRNG-Based Dynamic Code Generator

The new **RKE** scheme, the core of which is a **PRNG** generating dynamic codes, is depicted in Fig. 8.17. Each time a message is to be sent by a remote control the **PRNG** is clocked. Thereby, a pseudo-random sequence of data is generated from a previously stored internal state, that serves as dynamic code. After computing the code, the internal state of the remote control is updated. The generated pseudo random sequence is — similar to a rolling-code scheme — appended with the unique serial number of the remote before being transmitted.

Figure 8.18 shows the basic idea of the **PRNG** construction that is adapted from [PSP⁺08]. Two identical block ciphers E_1 and E_2 are connected in series, using two distinct secret keys k_1 and k_2 , respectively. The input x of the first cipher is derived by XORing the previous output y of the second cipher and a seed value *Input SEED*¹³. Both secret keys, k_1 and k_2 , are updated in each clock cycle by XORing their previous value with an intermediate value m , i.e., the output of E_1 (and also the input of E_2). The output o , which is used as the dynamic code, is computed by an XOR of y and another seed value termed *Output SEED*. The variables of the **PRNG** in clock cycle i can be recursively computed according to the following equations:

$$\begin{aligned}
 x^{(i)} &= y^{(i-1)} \oplus \text{Input SEED} \\
 m^{(i)} &= E_{k_1}^{(i)}(x^{(i)}) \\
 y^{(i)} &= E_{k_2}^{(i)}(m^{(i)}) \\
 o^{(i)} &= y^{(i)} \oplus \text{Output SEED} \\
 k_1^{(i)} &= k_1^{(i-1)} \oplus m^{(i-1)} \\
 k_2^{(i)} &= k_2^{(i-1)} \oplus m^{(i-1)}
 \end{aligned}$$

The initial state of the receiver, i.e., $m^{(i-1)}$, $k_1^{(i-1)}$, $k_2^{(i-1)}$, $y^{(i-1)}$, and the seeds have to be synchronized with those of the remote control.

¹³The purpose of the seed values is clarified in Sect. 8.6.3.

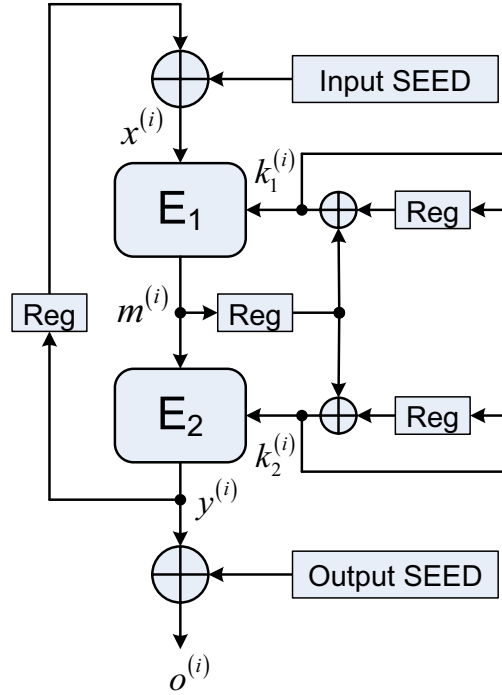


Figure 8.18: Structure of the basic **PRNG**, adapted from [PSP⁺08], where *Reg* denotes a register containing the value of the previous clock cycle.

Synchronizing the Receiver

In contrast to a rolling-code scheme, the code generator of the receiver is identical to that of a remote control. After receiving a dynamic code, the receiver basically repeats the steps performed by the remote control and compares the received code with the computed one. If they are identical, the code is accepted as valid. In practice, the states of the receiver and of the remote control may run out of synchronization, e.g., if the button of the remote control is pressed far away from the receiver, so that the sent dynamic code is not intercepted. Accordingly, a receiver must scan a sequence of generated codes, starting from the last valid initial state, to check the validity of a received dynamic code.

Thus, after receiving a complete message containing a serial number and a dynamic code, the receiver checks the serial number and accordingly initializes the **PRNG** with the corresponding last valid initial state. Then, the **PRNG** is clocked and its output is compared with the received dynamic code. The process is repeated until a consistency is detected or until a defined maximal number of rounds is reached. In the first case, the received code is considered as correct and the state of the **PRNG** is updated, while in the latter case the code is rejected.

Similar to a rolling-code scheme (see Sect. 2.3.2) two windows can be defined to prevent from authenticating with a randomly sent dynamic code¹⁴. The first window contains the next

¹⁴Since the generated sequence of dynamic codes depends uniquely on the secret keys, for a sufficiently large block size of the dynamic code, the probability for guessing a correct code is negligible. However, if a small block size is chosen by the system designer, implementing the windows can make sense.

16 codes from the last valid one. Any dynamic code within this window is directly accepted. The second window contains the subsequent codes until an upper limit of tested codes that is set depending on the speed of the implementation, as discussed in Sect. 8.6.4, is reached. A dynamic code in this window is only accepted, if two consecutive, valid dynamic codes are received.

Initialization and Learning Phase

Suppose that the initial values of the secret keys k_1 and k_2 are stored in both the remote control and the receiver. These two prominent secrets are proprietary values for each manufacturer and play a similar role to the manufacturer key in a rolling-code scheme, i.e., prevent conflicts in products of different companies.

For registering a remote control to a receiver, both participants are put into a learning mode, e.g., triggered by pressing a special button. Then, the remote control

- (1) sets the intermediate value m and the output y to zero,
- (2) sets the initial values for the secret keys k_1 and k_2 ,
- (3) generates its individual, random input and output seeds, stores them internally, and sends them to the receiver¹⁵,
- (4) runs the PRNG for one round, and
- (5) sends the output o concatenated with its serial number as dynamic code to the receiver.

Likewise, the receiver

- (1) sets the intermediate value m and the output y to zero,
- (2) sets initial values for the secret keys k_1 and k_2 ,
- (3) receives and stores the input and output seeds,
- (4) runs the PRNG for one round, and
- (5) registers the new transmitter, i.e., stores its seeds and the initial state together with the serial number, if (and only if) the computed output o and the received dynamic code are identical.

After this one-time learning procedure, the remote control and the receiver are ready to be used for their purpose, e.g., opening a door.

¹⁵It is assumed, that the one-time learning process is executed in a secure environment, i.e., where the adversary cannot intercept the seeds. Otherwise, the channel for transporting the seeds would have to be secured.

8.6.3 Security Considerations

Since the construction of our **RKE** scheme is based on the **PRNG** presented in [PSP⁺08] and the adversary can only observe the result of **XORing** the **PRNG** output y and a seed, the theoretical analyses given in [PSP⁺08] to evaluate the security of the **PRNG** hold also for our proposed **RKE** system. In short, supposing that the block ciphers E_1 and E_2 are ideal ciphers, the **PRNG** and hence the **RKE** is mathematically secure. No reasonable adversary can distinguish the uniform distribution of the internal state, y , from the visible uniform distribution at the output, o .

Improving the Security

As shown in [PSP⁺08], side-channel key recovery attacks which require a set of queries and measurements under the assumption that the secret key is fixed, cannot reveal the dynamic secret keys k_1 and k_2 . Indeed, the re-keying strategy, or in general using each session key only once, prevents most side-channel attacks, e.g., **DPA**. However, profiling attacks that are capable of revealing the secrets using a single measurement may challenge the **PRNG** and recover the secret keys used in one round.

Secret Seeds and Template Attacks One important characteristic of our **RKE** scheme and the originally proposed **PRNG**, that is important in the context of template attacks, are the seeds: For the **PRNG**, the input seed is public and the output of the **PRNG** is directly sent to the other party, without **XORing** an output seed. In contrast, in our **RKE** scheme the seeds have to be kept secret and the output of the remote control is generated by **XORing** the **PRNG** output with the additional output seed. If the seeds were known to a side-channel adversary, she could recover the secret keys k_1 and k_2 by profiling the target device and executing a template attack on the block ciphers E_1 and E_2 . In the following we discuss, if and how the secrecy of the seeds improves the protection of the implementation with respect to template and **DPA** attacks. We thereby assume, that either the output or the input of the attacked cipher must be known to mount a successful key recovery¹⁶.

A side-channel adversary executing a template attack targeting k_2 first has to profile the leakage of E_2 . The profiling step is performed for one fixed value of the output seed. The generated profiles thus only match for this particular output seed. Since, during the real attack, the output is always concealed with a different, secret (and thus unknown) output seed, the attacker cannot directly obtain a dynamic secret key k_2 with a template attack. However, the output seed is fixed in the proposed basic scheme, hence a template-based **DPA** attack may reveal it by considering the leakage of the **XOR** with the **PRNG** output y (the result of the **XOR** is the known dynamic code). Since all types of **DPA** attacks require a set of measurements with a fixed secret, such an attack can be prevented with a dynamic output seed that is different in each **PRNG** run.

The function of the input seed in our **RKE** scheme is to prevent overlaps between the streams generated by different remote controls. The remote control generates a random input seed, as illustrated in Sect. 8.6.2, and sends it (securely, if possible) to the receiver. If the input seed

¹⁶An extremely powerful adversary might even be able to derive the exact value of a key byte, when it is being loaded into the accumulator of the μC , by means of templates — in this case, none of the here proposed techniques counteracting template attacks would be effective.

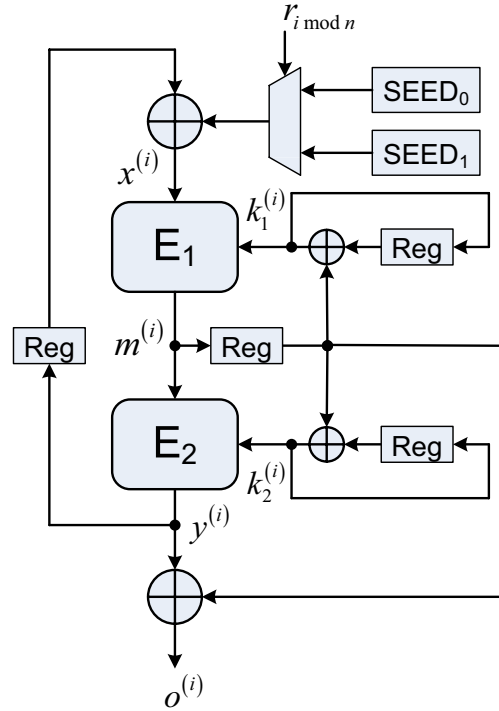


Figure 8.19: Final, improved structure of the **PRNG**-core of our **RKE** system

was public, challenging the **PRNG** by a template attack or even a template-based **DPA** attack would again become feasible.

Final Structure

Figure 8.19 shows the final structure of our proposed **RKE** scheme, resulting from an adaptation of the “secure initialization” of the **PRNG** presented in [PSP⁺08] and taking the previous considerations into account. When a remote control is in the learning mode, it now generates an n -bit random number r and runs the **PRNG** for n rounds without producing any output o . Then, it sends r (instead of the input and output seeds, cf. 8.6.2) and the last output o to the receiver. In contrast to the previous input seed, now r is a public value and can be distributed via an insecure channel. Still, both **SEED0** and **SEED1** have to be kept secret, e.g., they can be randomly generated during the manufacture.

In contrast to the secure initialization of the original **PRNG**, r is used here in every round during the execution of the **RKE** to choose between the two different input seeds **SEED0** and **SEED1**: Each time i a message is sent by the remote control, the $(i \bmod n)$ -th bit of r contributes to the computation, by selecting which of the two seeds is used. Furthermore, the intermediate value m now replaces the previous output seed. Since it changes in every round of the **PRNG**, the key-whitening step is not susceptible to (template-based) **DPA** anymore.

As pointed out in Sect. 8.5.3, the manufacturer of a **KEELOQ** rolling-code system is capable of circumventing it by monitoring a transmission of a genuine remote control. On the contrary, in our proposed **RKE** system, knowing the initial values of the secret keys k_1 and k_2 does not

allow the manufacturer to reproduce the dynamic codes sent by the remote control, as long as its random initialization value r remains unknown to him. The only possibility to get known to r is to monitor its transmission during the learning mode.

Remaining Limitations and Attack Vectors Since the construction of the transmitter and the receiver are the same in the proposed **RKE** system, their security properties are identical. A side-channel adversary who cannot break a transmitter is hence also not able to reveal secrets of a receiver.

A remaining open problem relates to the fact that a template attack may still challenge the system, exploiting the leakage of the **XOR** operation of the **PRNG** output y with the output seed m , on an 8-bit architecture: Suppose a **HW** leakage model for the target implementation and an adversary who previously profiled $\text{HW}(y)$ (or $\text{HW}(m)$) as an input of the aforementioned **XOR** operation, for all y (or m) values. Knowing a specific $\text{HW}(y^{(i)})$ (or $\text{HW}(m^{(i-1)})$) and the output $o^{(i)} = y^{(i)} \oplus m^{(i-1)}$, she obtains some information about y (or m) from a template attack.

As illustrated in [PSP⁺08], in a noise free situation the success rate of such an attack would be $S = b+1/2^b$ for a b -bit architecture, i.e., in our case for $b = 8$ the success rate of the adversary corresponds to $S = 0.035$. However, she can use an identity leakage function instead of the **HW** model. According to our practical experiments on an 8-bit Microchip PIC microcontroller, if the target leakage is caused by a linear function, e.g., an **XOR** operation, the success rate is about $S = 0.20$ using the identity leakage function, i.e., profiling $o = y \oplus m$ for all possible values of y and m ¹⁷. Interestingly, the success rate of recovering y (or m) and hence k_2 with a length of l bits is $S = (0.035)^{l/8}$ for the **HW** leakage model and $S = (0.20)^{l/8}$ for the identity leakage model. Supposing a typical 128-bit length for the key and block size of the block cipher, the resulting success rates for **HW** and identity leakage model are $S = 2^{-77}$ and $S = 2^{-37}$, respectively.

8.6.4 Implementation

For performance measurements and practical (security) analyses, we have implemented a complete system based on the illustrated **RKE** scheme on a μC of the Microchip PIC18F family [Micj]. Since commercial **RKE** systems often employ an 8-bit μC , and Rijndael (**AES**) is an efficient block cipher for such an architecture, we opted for **AES**-128 as the block cipher instantiating E_1 and E_2 . Consequently, the keys, the two input seeds, the intermediate value m , and the dynamic code o are 128-bit values¹⁸. Furthermore, we have selected the length of r as $n = 128$.

Speed

The performance of the implementation of the block cipher in the receiver is of great importance in practical scenarios. To achieve a sufficiently high throughput on an 8-bit μC , an efficient implementation of the **AES**-128 cipher is required. One of the fastest implementations for the chosen platform, needing 3137 clock cycles for an **AES**-128 encryption including an on-the-fly computation of the round keys, is proposed in [The]. Using a 20 MHz crystal, the algorithm is

¹⁷Note that the success rate in this case depends strongly on the noise of the measurement setup, as well as on the quality of the profiling.

¹⁸Note, that only a part of o could be sent in practice, e.g., to save energy during its transmission

executed in about $627\ \mu\text{s}$. Since two instances of the encryption must be executed in each round of the **PRNG**, a minimum speed of 512 rounds per second is achieved, including the computation overhead for adding the seed values and updating the initial state. The length of the second window, described in Sect. 8.6.2, was set to 512 codes in our implementation. Accordingly, if the button of a remote control is pressed approx. 500 times without being noticed by the receiver, it is still authenticated by sending two consecutive messages.

Energy Consumption

Another important issue when implementing **RKE** systems is the power consumption. Since the remote controls are usually battery operated, it is of crucial importance to have a low-power design for the transmitter. For an estimation, we consider only the two **AES** encryptions, constituting the biggest functions executed in a remote when sending a dynamic code. For generating a dynamic code, our implementation consumes on average 10 mA for 2 ms, while the **RF** communication part needs 50 mA for a duration of 450 ms. Using a standard 3 V/600 mAh battery, a transmitter can operate for more than 90 000 times. According to the measured figures, the energy consumption of the **RF** part exceeds that of the **AES** cipher by a factor of more than 1000 — the battery life-time could thus be greatly improved by sending shorter messages. Since the receiver is usually installed in an access point with a power supply, its energy consumption, e.g., running the **PRNG** for 512 times when receiving an invalid message, is not crucial.

8.6.5 Summary of the New RKE System

As a possible replacement for KEELOQ remote controls, this section presented an **RKE** application based on a secure **PRNG**, proposed in [PSP⁺08]. The **PRNG** has been designed to be secure against side-channel attacks, independent of the particular implementation of the cipher employed. The resulting **RKE** system is inherently secure against key recovery attacks by means of **SCA** and hence constitutes a remedy for the vulnerabilities of various commercial **RKE** systems, e.g., KEELOQ. The proposed **RKE** scheme also improves the privacy of the customers: in contrast to the KEELOQ rolling code scheme, the manufacturer of the **RKE** system is not able to access an object that is secured by one of his own products.

When realizing the originally proposed **PRNG** on an 8-bit μC , vulnerability to template attacks remains. Thus, its structure and the assumptions about its seed values have been adapted to improve the resistance against side-channel attacks including profiling. The resulting system is a practical example to show there are alternative choices to conventional countermeasures, e.g., masking, hiding, and those applied at layout level. These alternatives are often not taken into consideration by the manufacturers because of their supposed time, area, and power consumption overhead. This section demonstrates that designing new systems, based on secure ciphers and inherently counteracting side-channel attacks, is feasible and of high interest from the cryptographer's as well as from the industrial engineer's point-of-view.

8.7 Conclusion

This chapter presented very powerful **CPA** and **SPA** attacks targeting commercial KEELOQ implementations. The side-channel analyses, exploiting the power consumption or **EM** emana-

tions, can be applied to both **IFF** and code hopping devices. The success of the side-channel attacks is not related to the known mathematical vulnerabilities of the KEELOQ cipher, but can in principle be applied to an unprotected implementation of any cipher. We illustrated new power-analysis techniques to extract secret keys from hardware and software implementations of KEELOQ. After applying the presented key-recovery attacks, we demonstrated how to conduct a complete break of the KEELOQ code-hopping scheme.

Recovering the key of a remote control, K_{dev} , from **ASIC** hardware implementations takes only about ten power measurements and seconds of computations by means of the presented optimized **CPA**. Finding the 64-bit key of a remote control is hence possible by pressing the button of a remote control ten times while acquiring the power consumption with a sample rate as low as 20 MS/s. All investigated hardware implementations of KEELOQ show an almost perfect Hamming distance leakage and hence constitute an easy target for **SCA**. The extraction of the manufacturer key K_{man} from a software implementation of the cipher by **CPA** however requires thousands of power measurements, due to data-dependent time variations of the implementation.

After an in-depth analysis of a reference implementation of KEELOQ on an 8-bit microcontroller, we pinpointed a fatal key-dependent run-time of the algorithm. Exploiting it, we developed a very efficient key-recovery attack by means of **SPA** that requires no prior knowledge about neither the plaintext, nor the ciphertext. The illustrated **SPA** allows recovering K_{man} from only one single power measurement. The introduced new methods include a non-heuristic technique for automatically extracting the parameters required for the **SPA** from power traces, and thus avoid tedious visual inspection. Our attack neither requires a sophisticated measurement setup, nor any preprocessing steps to align or average measurements.

The knowledge of K_{man} enables taking over control of all **RKE** systems of that manufacturer, because the key K_{dev} of a remote control is computed from K_{man} . This **SPOF** in the design of the KEELOQ crypto-system violates the privacy and security of the customers, as the manufacturer of a KEELOQ code hopping system (or criminals gaining his manufacturer key illegally) can unnoticedly access any secured object where the system is installed.

Analyzing real-world applications of KEELOQ and taking all known key-derivation schemes into account, we developed several devastating attacks on real-world systems. As a result, two main implications arise when the secret master key K_{man} gets into the hands of an attacker. Firstly, the attacker can produce fake products that are compatible with those of that manufacturer — the monopoly of the manufacturer, e.g., him being the only supplier of spare remote controls, collapses. Secondly, a remote control of this manufacturer can be cloned from a distance, by monitoring one transmission, even without ever seeing the original remote control. With this powerful eavesdropping approach, even a low-skilled intruder can spoof a KEELOQ receiver with technical equipment for less than US\$ 50 and take over control of an **RKE** system, or deactivate an alarm system, without leaving any physical traces. Both attacks can be performed by a completely unskilled person, as the technically challenging part, including the key extraction by means of **SCA**, needs to be conducted only once for each manufacturer and can thus be outsourced to criminal cryptographers.

Implementing the key derivation of KEELOQ on the special-purpose hardware **COPACOBANA**, we furthermore quantified the effort required for a key-recovery in case that random seed values would be involved in the key derivation. Possessing K_{man} , after eavesdropping two hopping codes the exhaustive key-search takes 0.33s for a 32 bit seed, 5.9 hours for a 48 bit seed and

1011 days in case of a 60 bit seed on one **COPACOBANA**. Hence, a seed with maximum length can be regarded as sufficiently secure for medium-security purposes, e.g., accessing garage doors.

Summarizing, we were able to mount highly effective attacks with considerable implications on the security of KEELOQ code hopping systems that allow for

- recovering the secret key K_{dev} of a KEELOQ code hopping encoder with as few as ten power traces and only minutes of computation time,
- extracting the manufacturer key K_{man} used in a receiver from a single measurement, without knowing plaintexts or ciphertext a priori,
- cloning a remote control from the distance by eavesdropping a hopping code¹⁹, and
- taking over the control of a KEELOQ access control system, i.e., put it out-of-service for legitimate users while the attacker is still granted access.

The described complete break of the KEELOQ code hopping mechanism was performed with no previous knowledge about the implementations and is applicable to all KEELOQ key derivation schemes we are aware of. To our knowledge, and without naming any manufacturers, the described attacks can be applied to the vast majority of KEELOQ products in the field. Hence, it becomes practical for criminals to extract and collect master keys of many manufacturers, and perform devastating attacks on KEELOQ **RKE** systems. This contribution shows once more that widespread commercial applications, claiming to be highly secure, can be practically broken with modest cost and effort using **SCA** attacks. Thus, physical attacks must not be considered to be only relevant to the smartcard industry or to be a mere academic exercise. Rather, effective countermeasures against side-channel analysis need to be implemented not only in high-value systems such as smartcards, but also in other embedded security applications.

One example of an inherently power-analysis resistant **RKE** system was detailed at the end of this chapter. The scheme is based on a **PRNG** employing the **AES-128** cipher. The performance measurements of our implementation on an 8-bit μC demonstrate, that our highly secure scheme fits easily into the given boundaries (energy, speed, code size) of a practical **RKE** system.

Obviously, the key derivation is crucial for the security of any cryptographic system. Especially in the context of wireless applications, in which eavesdropping is straightforward, the key transport is non-trivial. This thesis has demonstrated for the commercial payment system in Chap. 6 and for the KEELOQ access control system in this chapter, that a **SPOF** in the key derivation can have dramatic impacts. The next wireless device analyzed in the final chapter of this thesis, i.e., the electronic passport, employs a key derivation without such a single breaking point.

¹⁹The manufacturer key is assumed to be known, e.g., from a previous attack

Chapter 9

Electronic Passport: Cracking Basic Access Control Keys with COPACOBANA

As demonstrated in the previous chapter, an attacker possessing the manufacturer key can decrypt eavesdropped transmissions of KEELQ RKE systems, due to a SPOF in the employed key derivation scheme. The Basic Access Control (BAC) scheme of the ePass analyzed in this chapter provides a special key management, i.e., the individual key of each ePass is derived from data that is printed on the passport. This scheme for protecting the private data transmitted by the ePass thereby avoids a single-point-of-failure.

Since the introduction of the Machine Readable Travel Document (MRTD) for identifying humans at the border control, debates have been raised about security and privacy concerns. Contactless smartcards, such as embedded in the passports, are vulnerable to eavesdropping and relay-attacks (see Sect. 2.2), hence electronic passports provide corresponding countermeasures. To hinder tracking at hand of the Unique Identifier (UID), which is required to initiate the communication according to the ISO 14443 standard (see Sect. 2.5), German passports generate a new random UID on every power-up. From there on, the communication with every passport is supposed to look technically indistinguishable, until a secure channel ensuring confidentiality is established on the basis of the protection scheme BAC. The scheme shall further ensure that decryption of the communication and accessing the content of the passport is only possible when the document is physically present: to derive the BAC keys, a passport needs to be optically inspected before its data is read out which renders relay attacks infeasible.

For breaking the BAC scheme SCA-attacks are not appropriate, since an attacker having physical access to an ePass (which is a prerequisite for SCA) can directly derive the secret keys by looking at the document. Rather, extracting the keys of an ePass from a distance, by means of eavesdropping encrypted messages, is considered a severe threat for the privacy and security of individuals. In this chapter, the first hardware implementations for cracking BAC keys of the e-passport issuing schemes in Germany and the Netherlands (and other countries) are presented. One of the presented approaches requires monitoring both the requests of a reader and the responses of the passport. A second, new approach enables a distant eavesdropper, who only intercepts the forward channel from reader to passport, to extract the secret keys. The developed implementations run on the reprogrammable key-search machine COPACOBANA and can be used to decrypt the communication. This enables tracking the movements of individuals, collecting their private data, and automatically detecting of the presence of a passport, e.g., to trigger an alarm, from a distance.

The presented attacks tackle the implementation of a security mechanism in the electronic passport as issued in Germany since November 2005 and are applicable to electronic passports of various other countries. After publicizing the findings of this chapter, which emerged from joint work with Kerstin Lemke-Rust and Yifei Liu [LKLRP07], a new version of the German electronic passport was released in November 2007 with an improved variant of the here attacked key derivation scheme.

9.1 Introduction and Related Work

The United States and several other countries have been engaged in the development of a new border control system that is based on biometric identification and **RFID** technologies. This technically and socially ambitious vision has already become reality. Many states, e.g., Germany, the Netherlands, and Belgium already started issuing electronic passports in 2005. Specifications for **MRTDs** that are also known as e-passports are defined by the **ICAO** [ICA03, Org05, Org04a, ICA04, MRT04, Org04b]. For the storage of biometric data, an **IC** with an **RF** interface is embedded in the passport document. A valuable overview on security and privacy threats in e-passports is provided in [JMW05a]. Further related work on security aspects of e-passports can be found in [KK05, HHJ⁺06].

Promoters of the **MRTD** system promise that by using ‘machine readable visas and/or passports as a source of reliable data, governments can build useful data bases that can serve as a uniform source of information in standardized format to speed the border control process [MRT]. Further benefits are said to lie in “the creation of data bases shared voluntarily, even across national boundaries, and between the public and private sectors. This will make it easier to identify people who are traveling with stolen documents, and people who have fraudulently obtained an otherwise valid passport based upon stolen citizenship document forms” [MRT].

This contribution focuses on the **BAC** establishing a secured channel between the **RFID** reader that is part of the inspection system and the e-passport, for providing both confidentiality and integrity of the data communication. The **BAC** employs symmetric cryptography and generates the corresponding encryption and authentication keys from data that is visible in the physical passport document. The scheme as implemented in dutch passports has already been compromised using offline dictionary attacks: the encrypted information can be revealed in three hours after intercepting the communication [Ris, Rob], because of weaknesses in the passport numbering scheme. Similar flaws have been reported for the passports issued in Germany [CLRPS] and Belgium [AKQ].

In 2010, an attack for tracking the movements of a particular passport without having to break the passport’s cryptographic key has been proposed in [CS10]. The authors show that the duration of computing a **MAC** during the authentication is key-dependent and hence allows to identify individual passports — without decrypting the communication. While this attack requires actively interrogating the passport from a maximal range of some centimeters, the findings presented in this chapter enable tracking of individuals from a distance of several meters by means of eavesdropping (see Sect. 2.7).

Cryptanalytical tools such as brute-force machines examine the soundness of security claims for cryptographic solutions and hence yield figures about the real effort needed for practical cryptanalysis. This knowledge may help in assessing and possibly avoiding privacy and security risks that are imposed on the individual.

With this background in mind, we feel that there is a public interest in determining of how efficient key-search algorithms on the **BAC** keys can be mounted in practice. For assessing the practical use of special purpose hardware we design a hardware architecture and implement it on the **FPGA**-based machine **COPACOBANA** [KPP⁺06b] introduced in Sect. 4.5.

This chapter is organized as follows: in Sect. 9.2 we explain the **BAC** protocol and the key derivation scheme. The underlying threat model of our attack is given in Sect. 9.3, and Sect. 9.4 provides concrete adversaries and settings to form applicable scenarios for the key search. Details about the practical implementations and results are given in Sect. 9.5. Finally, other implementation options are compared with our approach and further directions are considered in Sect. 9.6.

9.2 The Basic Access Control Protocol

Personalization of an e-passport includes printing a Machine Readable Zone (**MRZ**) on the paper document that can be optically scanned by an inspection system at the border control. As illustrated in Fig. 9.1, the **MRZ** consists of two lines containing, amongst others, personal data such as name, sex, date of birth, and the nationality of the owner. The particulars of the second line are of special importance for the e-passport as they are used for the derivation of the **BAC** keys. The relevant fields are

- the passport number (9 alphanumeric characters),
- the date of birth of the passport holder (6 characters), and
- the date of expiry of the passport (6 characters).

Each field additionally includes a numeric check digit.

P<D<<LASTNAME<<FIRSTNAME<<<<					
1220000016D<<6408125F1110078<					
passport number	check digit 1	nationality	date of birth	check digit 2	sex
				passport expiration date	check digit 3

Figure 9.1: An exemplary MRZ of the German E-Passport.

Before any personal information can be read from an e-passport via an **RFID** reader, e.g., at the border control, the **BAC** protocol needs to be carried out. In case of a successful mutual authentication, the parties agree on a session key that is used for the encryption of the subsequent exchange of information¹. The **BAC** mechanism relies on symmetric cryptography, hence both parties, i.e., reader and passport, need to know the same secret keys. The latter are stored in the embedded **IC** of the passport at production time and can be derived on the side of the reader after optically scanning the **MRZ**, according to the following scheme.

¹Note that a reading access to more sensitive data like digital fingerprints and iris scans may require a further authentication mechanism, e.g., in Germany the Extended Access Control (**EAC**) [FOFIS].

9.2.1 Key Derivation from the MRZ

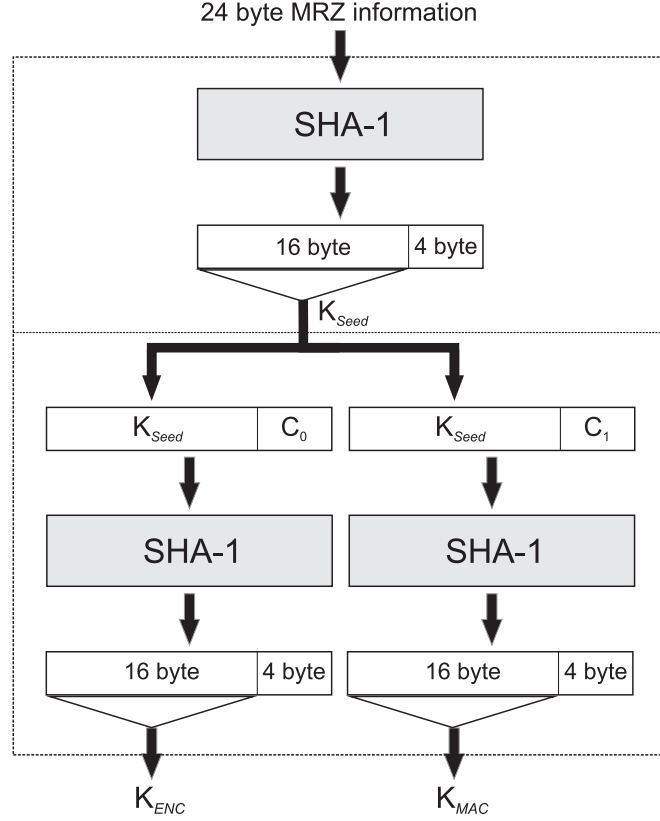


Figure 9.2: Derivation of keys for the Basic Access Control.

As illustrated in Fig. 9.2, first K_{Seed} is built from the most significant 16 bytes after applying a SHA-1 [oST] hash algorithm to the MRZ information. From K_{Seed} both an encryption key K_{ENC} and a key K_{MAC} for the MAC are computed. For their key derivation, two different constants are used: $C_0 = '00\ 00\ 00\ 01'$ for K_{ENC} and $C_1 = '00\ 00\ 00\ 02'$ for K_{MAC} . The most significant 16 bytes of the SHA-1 computation form the Triple-DES [Nat99] keys K_{ENC} and K_{MAC} :

$$K = msb_{16}(\text{SHA-1}(msb_{16}(\text{SHA-1}(MRZ)) \parallel C)),$$

where C is to be substituted with the constant C_0 or C_1 , depending on whether $K = K_{ENC}$ or $K = K_{MAC}$ is to be computed. Thus, for obtaining one key from the MRZ information, two subsequent rounds of SHA-1 have to be executed.

9.2.2 The Mutual Authentication Protocol

Based on the access keys K_{ENC} and K_{MAC} , session keys are established in the course of a three-pass authentication protocol. This protocol runs between the RFID reader (Interface Device (IFD)) that is part of the inspection system and the MRTD chip (Integrated Circuit Card (ICC)) as shown in Fig. 9.3 (see also [JMW05a, ICA04]).

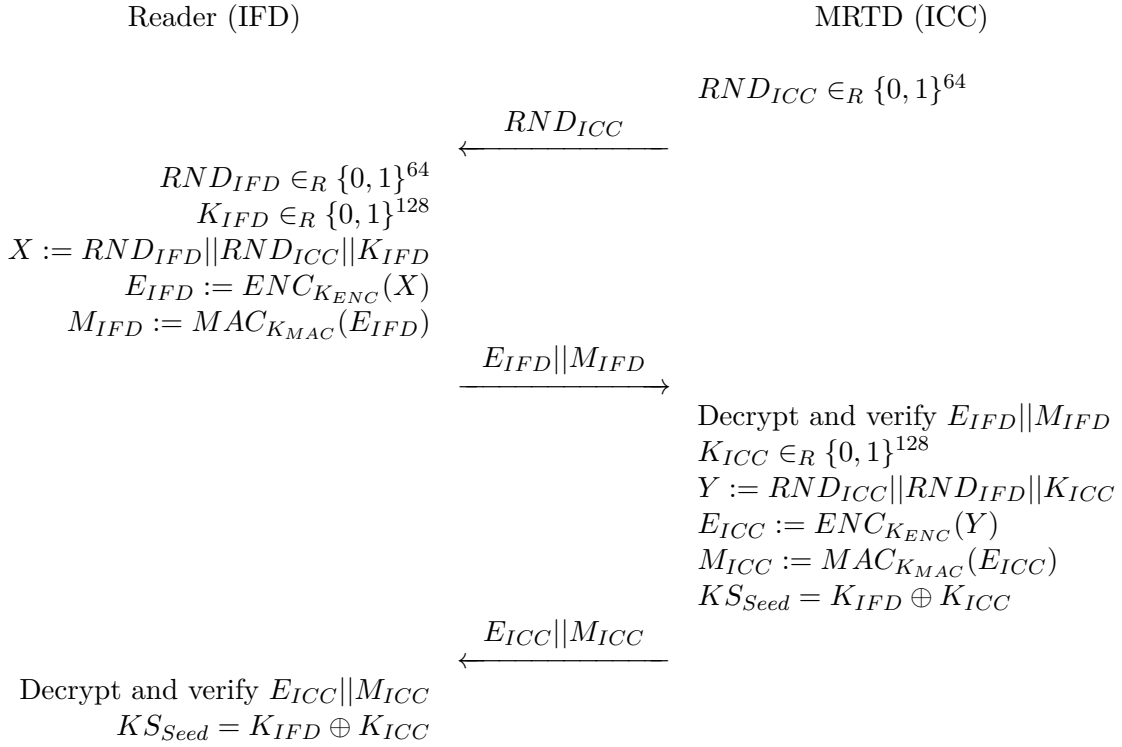


Figure 9.3: BAC protocol between the RFID reader (also referred to as IFD) and the MRTD chip (also referred to as ICC). ENC denotes Triple-DES encryption and MAC denotes a cryptographic checksum according to the ISO/IEC 9797-1 MAC Algorithm 3 [ICA04].

At the beginning of the BAC, the e-Pass generates a random number RND_{ICC} and sends it to the reader. The reader concatenates RND_{ICC} with further random bits RND_{IFD} and K_{IFD} , and encrypts the result with K_{ENC} to obtain $E_{IFD} = ENC_{K_{ENC}}(RND_{ICC} || RND_{IFD} || K_{IFD})$. In addition, a MAC $M_{IFD} = MAC_{K_{MAC}}(E_{IFD})$ is appended before $E_{IFD} || M_{IFD}$ is returned to the passport, where $MAC_{K_{MAC}}(\cdot)$ denotes the cryptographic checksum according to the ISO/IEC 9797-1 MAC Algorithm 3, as detailed in the annex of [ICA04]. After decrypting the data, the MRTD verifies the received RND_{ICC} against the original value and thereby assures that the reader possesses the correct secret keys. Finally, the e-Pass proves its knowledge of K_{ENC} to the reader in a similar fashion: RND_{ICC} is again concatenated with RND_{IFD} and some more random bits K_{ICC} before being encrypted to $E_{ICC} = ENC_{K_{ENC}}(RND_{ICC} || RND_{IFD} || K_{ICC})$. After generating the MAC $M_{ICC} = MAC_{K_{MAC}}(E_{ICC})$, the concatenation of the ciphertext with the MAC, i.e., $E_{ICC} || M_{ICC}$, is again broadcast via the RF link. The random values K_{IFD} and K_{ICC} generated by reader and passport are used for the derivation of a seed key $KS_{Seed} = K_{IFD} \oplus K_{ICC}$. The Triple-DES session key KS_{ENC} and the MAC session key KS_{MAC} are obtained from KS_{Seed} by applying the same key derivation scheme as depicted in the bottom of Fig. 9.2 for K_{ENC} and K_{MAC} . The subsequent communication transfers personal data records from the e-passport secured with KS_{ENC} and KS_{MAC} . For a fully

detailed description of the BAC and further security mechanisms integrated in the [MRTD](#) refer to [\[ICA04\]](#), [\[ICA03\]](#), [\[ICA05\]](#), and [\[FOFIS\]](#).

9.3 The Threat Model

Our threat model was initially introduced in [\[CLRPS\]](#) and is illustrated in Fig. 9.4. We propose

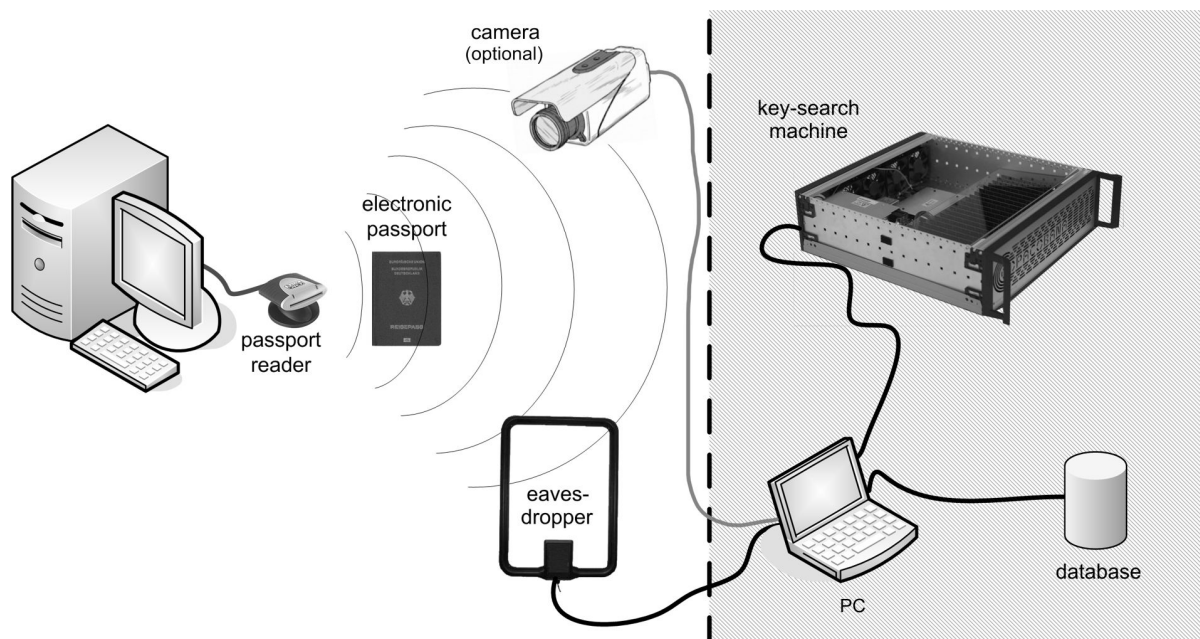


Figure 9.4: Architecture of the attack system

a hardware architecture that consists of two parts: The front-end is an [RF](#) eavesdropper that can continuously read and record the wireless communication at public places with a high e-passport density, e.g., placed nearby inspection systems at airports. Optionally, a surveillance camera may take pictures of the particular passport holder. The back-end is a cryptanalytic system that is connected to databases as well as to hardware or software modules for fast cryptanalysis of symmetric ciphers. Once eavesdropped data is stored in a database, a key search can be applied off-line. The reprogrammable machine [COPACOBANA](#) introduced in Sect. 4.5, which is optimized for computing cryptanalytical algorithms [\[KPP⁺06b, KPP⁺06c\]](#), is an integral part of the back-end. When [BAC](#) keys are compromised the revealed personal information such as name, sex, date of birth, nationality, passport number, date of expiry, and a facial image of the passport holder are inserted into databases.

Information in such databases is exploitable by criminals like terrorists or by detectives, data mining agencies, etc. The correctness of the private data is proven by a certificate of the issuing country and the digital photograph stored in the passport is optimized for automatic face recognition [\[JMV07\]](#). Ari Juels et al. [\[JMW05a\]](#) point out problems that are imposed on e-passport holders such as identity theft, tracking, and hotlisting. In the worst case scenario, an attacker may devise an RFID enabled bomb that is keyed to explode when reading a particular

individual's RF identifier [JMW05a]. The success of a BAC protocol that is initiated by a skimming device of a criminal may be used as such a triggering event.

For the RFID-communication, two different channels are used (compare with Sect. 2.5):

- RFID reader to ePass (forward channel): This channel supplies the ePass with energy and is used for transferring requests from the reader to the ePass.
- ePass to RFID reader (backward channel): This channel is used by the ePass to send its responses to the reader.

The signal from the reader to the e-passport is about 80 dB stronger [Fin02] than the so-called load modulation signal which is used for communication on the backward channel, in accordance with the ISO 14443 international standard [Int01] (compare with Sect. 2.5). Thus it is significantly more difficult to observe data on the backward channel than on the forward channel. However, as detailed in Sect. 2.7, eavesdropping the wireless communication from several meters poses a real threat: the two-way communication between an RFID reader and an ePass can be intercepted from 4 m, while eavesdropping the forward channel is practically feasible from a distance of up to a 25 m [Rob]. We show that even such a distant eavesdropper that is only intercepting the strong signal sent from the RFID reader to the MRTD, but not the channel back from the MRTD to the reader, suffices to identify a particular ePass following the approach detailed in Sect. 9.4.2.

9.4 The Key Search

As indicated, two different approaches can lead to success in determining the BAC keys. However, the data records of an e-passport can only be retrieved following the first approach, while the second approach is adequate to gain BAC keys and thus identify a certain passport from a great distance.

9.4.1 The First Approach based on Two-Channel Communication

The first attack option aims at recovering K_{ENC} . The plaintext for this case is the 64-bit number RND_{ICC} transmitted by the passport at the beginning of the BAC. The corresponding ciphertext $ENC_{K_{ENC}}(RND_{ICC})$ is part of the last message $E_{ICC}||M_{ICC}$ that is sent during the BAC. After eavesdropping RND_{ICC} , $E_{IFD}||M_{IFD}$, and $E_{ICC}||M_{ICC}$ of Fig. 9.3 (and the subsequent secured communication C) the adversary runs a key search on the MRZ information to find a match to the most significant eight bytes of E_{ICC} (see Fig. 9.3) during the protocol run. The encryption function $ENC_{K_{ENC}}(\cdot)$ is a Triple-DES in CBC mode (see Sect. 2.3.6) with the initialization vector being publicly known [ICA04]. Hence, the most significant eight bytes $msb_8(E_{ICC}||M_{ICC})$ can be decrypted with varying keys for comparison with RND_{ICC} , without knowledge of the remainder of the datagram. In case of a match, K_{ENC} and the related MRZ are found with a high probability.

The adversary computes $ENC^* = ENC_K(RND_{ICC})$ where K denotes possible candidates for K_{ENC} and ENC denotes the 3DES encryption. If

$$msb_8(ENC_{ICC}) \stackrel{?}{=} ENC^*,$$

the communication data C can be decrypted to reveal the data records of the **ePass**.

For each key candidate, this key search requires two computations of SHA-1 for the key derivation of K_{ENC} and one computation of **3DES**. By pre-computing the key derivation, two computations of SHA-1 can be saved at key-search time. The amount of data sent to the cryptanalytic module for performing the key search amounts to 16 bytes.

9.4.2 The Second Approach based on Forward-Channel Communication

There exists an alternative way of discovering the **BAC** keys, if a far-distance adversary does not succeed in eavesdropping the backward channel from the e-passport to the **RFID** reader: This time, the **MAC** key K_{MAC} is targeted and the interception of only one message of the reader, i.e., $E_{IFD}||M_{IFD}$, is sufficient for identifying a particular e-passport. Obviously, here E_{IFD} is the plaintext and its **MAC** M_{IFD} will match with $MAC_{K_{MAC}}(E_{Reader})$ in case of a correct key guess for K_{MAC} . Eavesdropping $E_{IFD}||M_{IFD}$ on the forward channel can be used for cracking **BAC** keys by checking

$$MAC_K(E_{IFD}) \stackrel{?}{=} M_{IFD},$$

where K is a key candidate for K_{MAC} . The knowledge of the **MAC** key can be exploited to identify a previously gathered e-passport from the database. Subsequently, if the adversary got close enough to an **MRTD** whose keys are already found, it could be activated and read out with a skimming device.

For each key candidate, a key search requires two computations of SHA-1 for the key derivation of K_{MAC} from the **MRZ** information. Furthermore, for the computation of the retail **MAC** with K_{MAC} according to ISO/IEC 9797-1 one needs to perform four single **DES** (as E_{IFD} is a ciphertext of 32 byte size) and one **3DES** for the last padded block. In terms of brute-force this approach requires four additional single **DES** if compared to the one in Sect. 9.4.1. Another drawback for a far-distance adversary is that neither the established session keys nor the transferred data records on the backward channel can be revealed. According to Sect. 9.4.1, if pre-computation is applicable this saves two computations of SHA-1 during the key search. For the second approach, the amount of data sent to the cryptanalytic module for performing the key search adds up to 40 bytes.

9.4.3 Complexity Analysis of the Key Space

The complexity of key spaces can be evaluated based on the Shannon entropy [Mac03]. Entropy is a common measure for evaluating the amount of information contained in some input, e.g., in the context of deriving secret keys.

Entropy

The Shannon entropy (also called information entropy) of an ensemble X is defined as the average Shannon information content of the outcome of a random experiment:

$$H(X) := - \sum_i^n P(x_i) \log_2 P(x_i),$$

where X denotes a random variable which takes on a finite set of values $(x_1, x_2, \dots, x_n)$ and $P(x_i)$ is the probability of x_i . For estimating the information contained in a complex system, consisting of several random variables, it is necessary to observe the joint entropy. The joint entropy of two variables (X, Y) is generally defined as

$$H(X, Y) := - \sum_i^n P(x_i, y_i) \log_2 P(x_i, y_i).$$

For two stochastically independent variables (X, Y) the joint entropy is equal to

$$H(X, Y) = H(X) + H(Y) \text{ iff } P(x, y) = P(x)P(y),$$

while for two stochastically dependent variables (X, Y) the joint entropy becomes

$$H(X, Y) \leq H(X) + H(Y).$$

In case of stochastically dependent variables (X, Y) it is also necessary to introduce the conditional entropy

$$H(X|Y) := - \sum_i^n P(x_i, y_i) \log_2 P(x_i|y_i).$$

The joint entropy and the conditional entropy are related by

$$H(X, Y) = H(X) + H(Y|X) = H(Y) + H(X|Y).$$

Further details about Shannon's theory and entropy gives Chap. 2 in Stinson [Sti05].

Real-World Scenarios and Adversaries

The complexity of the key space for **BAC** keys depends on the passport number issuing scheme that is controlled by the issuing state. In this contribution we focus on two European states issuing e-passports since 2005: Germany and the Netherlands. The information in Table 9.1 stems from [Rob] for the Netherlands and from [Pas, BKZ07, Hei, BDR07] for Germany².

The main flaw in the present passport numbering schemes is the low entropy of **BAC** keys. Low entropy is caused by

- (1) downsizing the key space of the passport number, i.e., instead of using nine alphanumeric characters for the passport number mainly numeric characters are used, some of which are even fixed or a check digit,
- (2) stochastic dependencies between the passport number and the expiry date, e.g., the passport numbers are assigned serially, and
- (3) a dependency of the key space on publicly available personal data, in particular the date of birth of the passport holder.

²Meanwhile, there have been modifications of the passport numbering scheme in both states. However, our complexity analysis remains valid for those e-passports that have already been issued.

Table 9.1: Special Parameters for Issuing Passports in Germany and the Netherlands.

Issuing State:	Germany	The Netherlands
Start of the System:	November 1, 2005	August 26, 2006
Validity of an E-Passport:	10 years	5 years
Passport Numbering:	4 numeric digits for local authority (BKZ) and a serial number of 5 numeric digits, e.g., for Berlin-Mitte with BKZ No. ‘2598’: ‘2598 12345’	1 fixed character ‘N’ and a serial number of 1 alphanumeric digit and 6 numerical digits followed by a 1 digit checksum, e.g., ‘NF3858053’
No. of known BKZs ³	295	
Individuals owning passports:	approx. 20 millions	approx. 9 millions
Issued passports per working day:	approx. 8000, i.e., $N_{day}^G = 8000$	approx. 7000, i.e., $N_{day}^{NL} = 7000$
Working days until June 1, 2007:	$T_{June,2007}^G \approx 365 \cdot 5/7 \cdot 19/12$, i.e., $T_{June,2007}^G \approx 413$	$T_{June,2007}^{NL} \approx 365 \cdot 5/7 \cdot 9/12$, i.e., $T_{June,2007}^{NL} \approx 196$

The complexity of the key search strongly depends on assumptions on the capabilities of the adversary. We consider three different adversaries $\mathcal{A}_1$, $\mathcal{A}_2$, and $\mathcal{A}_3$ as specified in Table 9.2. The transitions among them may be blurred as acquiring additional BAC keys as a result of a successful key search improves the knowledge on issued passports and thereby the configuration of key search algorithms in terms of efficiency.

Table 9.2: Capabilities of the Adversaries.

Adversary	Knowledge on the system
$\mathcal{A}_1$	only public knowledge
$\mathcal{A}_2$	stochastic dependency of passport number and date of expiry is known, i.e., incomplete database of BAC keys (in Germany: for each BKZ)
$\mathcal{A}_3$	complete database of BAC keys

³Note that the coverage of known local authorities in Germany (BKZs) among all BKZs in Germany is not publicly available. The number of known BKZs stems from [BKZ07].

Adversary $\mathcal{A}_1$ with the lowest capabilities knows the public parameters of the **ePass** issuing system (see Table 9.1) but does not know any passport numbers. $\mathcal{A}_2$ already owns a sparsely filled database of **BAC** keys that may be gained by collecting passport data from customers, e.g., at hotels or car rental companies. This previous knowledge allows $\mathcal{A}_2$ to predict some stochastic dependency between the passport number and the expiry date for the issuing state. $\mathcal{A}_3$ is the most powerful adversary: She has access to a complete database with **BAC** keys, e.g., as a result of social engineering attacks inside the infrastructure of the **ePass** system or by participating in databases shared by public and private sectors.

Table 9.3: Eavesdropping settings and known information for a cryptanalytical attack.

Setting	Knowledge on the Passport Holder	Note
$\mathcal{S}_1$	issuing state	
$\mathcal{S}_2$	issuing state, photo of passport holder	
$\mathcal{S}_3$	issuing state, date of birth	
$\mathcal{S}_4$	issuing state, site of eavesdropping	relevant only for Germany
$\mathcal{S}_5$	issuing state, site of eavesdropping, and photo of passport holder	relevant only for Germany

Another important factor for the cryptanalysis is the amount of information that is available as a result of eavesdropping during a **BAC** protocol instantiation. Here, we exemplify five settings (see Table 9.3). For all settings we assume that the issuing state of the passports is known, e.g., by observing special protocol information in the **ATS** (answer to select - compare with Sect. 2.5.6) response of the **ePass**. Setting $\mathcal{S}_1$ comprises solely information from the **RF** channel, whereas setting $\mathcal{S}_2$ assumes that additionally the age of the **MRTD** holder can be estimated by visual observation — either directly, or from a photo, e.g., taken in a video surveillance zone close to the inspection system. Setting $\mathcal{S}_3$ acts on the strong assumption that the exact date of birth of the passport holder is known. Settings $\mathcal{S}_4$ and $\mathcal{S}_5$ incorporate statistical properties that are specific for Germany, as for this country the passport numbering scheme also depends on the BKZ of the issuing authority and thus generally the town of residence of the passport owner. Based on the site of eavesdropping, assumptions can be made on the issuing authority.

Table 9.4 defines six attack scenarios, each combining an adversary from Table 9.2 with an eavesdropping setting from Table 9.3. Each scenario refers to a concrete time of the attack as the number of e-passports further increases. In our work, this concrete date is chosen to be June 1, 2007. Scenario 1 combines $\mathcal{A}_1$ and $\mathcal{S}_1$ leading to the highest complexity for both issuing states. The entropy of the date of birth denoted as H_{DB}^G and H_{DB}^{NL} in Scenario 1 was computed by using German demographic data [Deu], considering people from 18 to 80 years. In contrast, Scenario 6 combining the powerful adversary $\mathcal{A}_3$ with $\mathcal{S}_1$ needs the least effort

Table 9.4: Use cases and estimated entropy for Germany and the Netherlands.

Entropy for Germany	Entropy for the Netherlands
Scenario 1: $\mathcal{A}_1$ in $\mathcal{S}_1$ on June 1, 2007	
$H^G = H_{PN}^G + H_{DB}^G + H_{DE}^G$ $H_{PN}^G = \log_2(10^4) + \log_2(10^5) \approx 29.9$ $H_{DB}^G \approx 14.2$ $H_{DE}^G = \log_2(T_{June,2007}^G) \approx 8.7$ $H^G \approx 52.8$	$H^{NL} = H_{PN}^{NL} + H_{DB}^{NL} + H_{DE}^{NL}$ $H_{PN}^{NL} = \log_2(36 \cdot 10^6) \approx 25.1$ $H_{DB}^{NL} \approx 14.2$ $H_{DE}^{NL} = \log_2(T_{June,2007}^{NL}) \approx 7.6$ $H^{NL} \approx 46.9$
Scenario 2: $\mathcal{A}_1$ in $\mathcal{S}_2$ on June 1, 2007, range of 10 years for date of birth: $N_{Year} = 10$ for Germany: $N_{BKZ} = 295$	
$H^G = H_{PN}^G + H_{DB}^G + H_{DE}^G$ $H_{PN}^G = \log_2(N_{BKZ}) + \log_2(10^5) \approx 24.8$ $H_{DB}^G = \log_2(N_{Year} \cdot 365) \approx 11.8$ $H_{DE}^G = \log_2(T_{June,2007}^G) \approx 8.7$ $H^G \approx 45.3$	$H^{NL} = H_{PN}^{NL} + H_{DB}^{NL} + H_{DE}^{NL}$ $H_{PN}^{NL} = \log_2(36 \cdot 10^6) \approx 25.1$ $H_{DB}^{NL} = \log_2(N_{Year} \cdot 365) \approx 11.8$ $H_{DE}^{NL} = \log_2(T_{June,2007}^{NL}) \approx 7.6$ $H^{NL} \approx 44.5$
Scenario 3: $\mathcal{A}_1$ in $\mathcal{S}_5$ on June 1, 2007, range of 10 years for date of birth: $N_{Year} = 10$ for Germany: local area with 10 BKZ numbers: $N_{BKZ} = 10$	
$H^G = H_{PN}^G + H_{DB}^G + H_{DE}^G$ $H_{PN}^G = \log_2(N_{BKZ}) + \log_2(10^5) \approx 19.8$ $H_{DB}^G = \log_2(N_{Year} \cdot 365) \approx 11.8$ $H_{DE}^G = \log_2(T_{June,2007}^G) \approx 8.7$ $H^G \approx 40.3$	
Scenario 4: $\mathcal{A}_2$ in $\mathcal{S}_2$ on June 1, 2007, range of 10 years for date of birth: $N_{Year} = 10$ for Germany: $N_{BKZ} = 295$, each BKZ issues $N_P = 25$ passports per working day.	
$H^G = H_{PN}^G + H_{DB}^G + H_{DE}^G$ $H_{PN}^G = \log_2(T_{June,2007}^G \cdot N_P \cdot N_{BKZ}) \approx 21.5$ $H_{DB}^G = \log_2(N_{Year} \cdot 365) \approx 11.8$ $H_{DE}^G = \delta$ $H^G \approx 33.3 + \delta$	$H^{NL} = H_{PN}^{NL} + H_{DB}^{NL} + H_{DE}^{NL}$ $H_{PN}^{NL} = \log_2(T_{June,2007}^{NL} \cdot N_{day}^{NL}) \approx 20.4$ $H_{DB}^{NL} = \log_2(N_{Year} \cdot 365) \approx 11.8$ $H_{DE}^{NL} = \delta$ $H^{NL} \approx 32.2 + \delta$
Scenario 5: $\mathcal{A}_2$ in $\mathcal{S}_5$ on June 1, 2007, range of 10 years for date of birth: $N_{Year} = 10$ Germany: local area, $N_{BKZ} = 10$; $N_P = 60$ passports per BKZ and working day.	
$H^G = H_{PN}^G + H_{DB}^G + H_{DE}^G$ $H_{PN}^G = \log_2(T_{June,2007}^G \cdot N_P \cdot N_{BKZ}) \approx 14.6$ $H_{DB}^G = \log_2(N_{Year} \cdot 365) \approx 11.8$ $H_{DE}^G = \delta$ $H^G \approx 26.4 + \delta$	
Scenario 6: $\mathcal{A}_3$ in $\mathcal{S}_1$ on June 1, 2007	
$H^G = \log_2(N^G)$ $N^G \approx T_{June,2007}^G \cdot N_{day}^G \approx 3.3 \cdot 10^6$ $H^G \approx 21.7$	$H^{NL} = \log_2(N^{NL})$ $N^{NL} \approx T_{June,2007}^{NL} \cdot N_{day}^{NL} \approx 1.4 \cdot 10^6$ $H^{NL} \approx 20.4$

for the key search. Scenario 2 to Scenario 5 are of medium complexity acting on increasing assumptions on the capabilities and the information available for the attacker. We assume that the age of the passport holder can be guessed from a photograph with an accuracy of 10 years. Note that Scenario 2 to Scenario 5 typically have a probabilistic average success rate as the search algorithms focus on the most probable part of the entire key space. Therefore iterative runs with adapted assumptions might be necessary to find the **BAC** key. This affects especially Scenario 4 and Scenario 5 that exploit gathered stochastic properties of the passport issuing scheme. As some parameters are not precisely known, e.g., the number of issued passports per day may vary in practice, an uncertainty factor δ is added in Scenario 4 and Scenario 5, to take such deviations into account.

9.5 Practical Implementation on COPACOBANA

In the following, we first present the general idea of how we implement the key search on the parallel code-breaker COPACOBANA introduced in Sect. 4.5, and then detail the content of one single **FPGA** and the functional units it consists of. The execution speed and breaking the **BAC** is then evaluated with regard to some of the scenarios defined in Sect. 9.4.3.

9.5.1 Details of the Hardware Architecture

The architecture implemented in each **FPGA** is summarized in the left of Fig. 9.5. Each **FPGA** receives the same pair of plaintext and ciphertext from the database, i.e., the eavesdropped RND_{ICC} and the first 8 bytes of E_{ICC} for attacking K_{ENC} (see Sect. 9.4.1) or E_{IFD} and its **MAC** M_{IFD} for attacking K_{MAC} (see Sect. 9.4.2), and stores them in the corresponding registers.

The key search is accomplished by segmenting the contemplable key space into appropriate subspaces and processing them simultaneously. Since 120 **FPGAs** are available on **COPACOBANA**, the same number of subspaces needs to be assigned, according to the current attack scenario. The subspaces are allocated to the **FPGAs** by means of their *Part_of_MRZ* register which contains a portion of the **MRZ** that is fixed for each particular **FPGA**. The **MRZ** generator produces all remaining combinations of the **MRZ** assigned to that **FPGA** and supplies them to four engines that process the plaintext. Consequently, each unit works on a different fraction of the key space in parallel. If an **FPGA** is successful in finding the correct key, the respective **MRZ** information is output and can be stored in the database for further processing, i.e., decrypting the personal data.

Allocating the Key Space to the FPGAs

A very straightforward approach of distributing the **MRZ** information amongst the **FPGAs** would be to assign every single **MRZ** to be processed directly by the host **PC**. This would involve a significant amount of data transferred between the **PC** and the **COPACOBANA** and thus severely degrade the execution time of the key search: the communication with the **PC** would be the bottleneck of the implementation. Instead, each **FPGA** possesses a dedicated **MRZ** generator producing a new **MRZ** out of an assigned key space prior to each encryption in order to minimize the communication via the data buses.

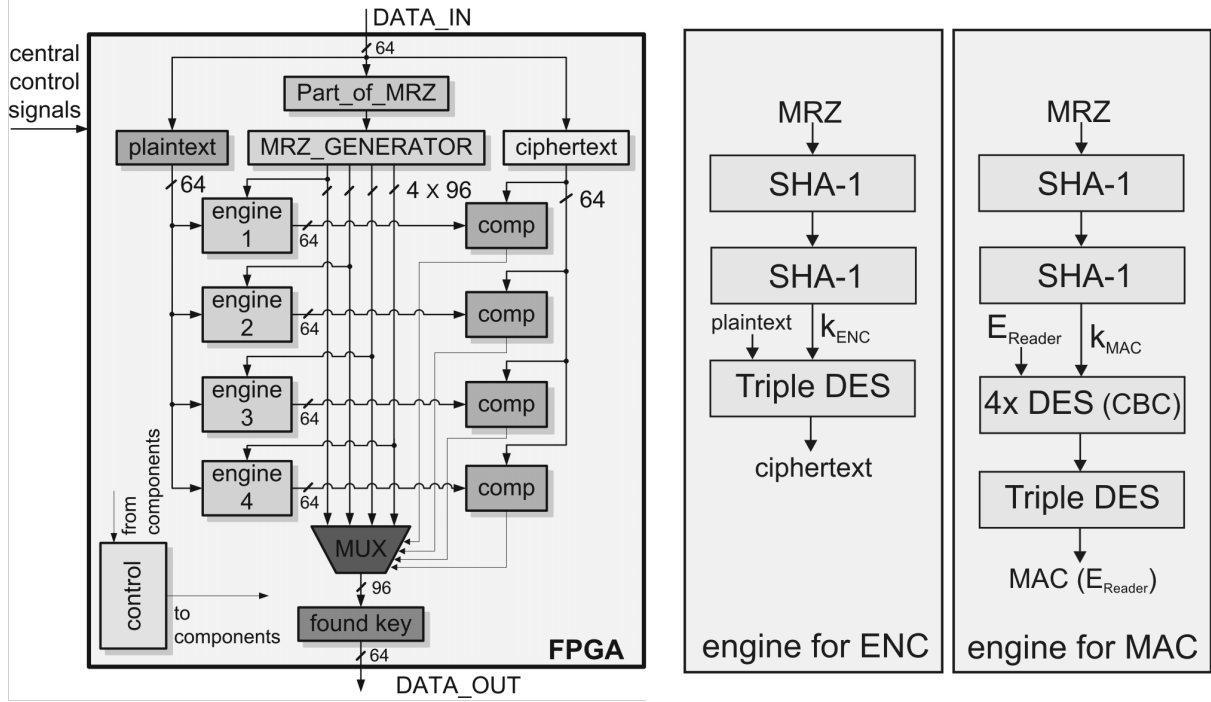


Figure 9.5: Structure of one FPGA (left) and details about the engines for attacking K_{ENC} (center) and K_{MAC} (right).

The architecture of this MRZ generator is crucial for the searching efficiency of each scenario; in particular the decision which part of the MRZ information will be fixed for each FPGA and stored in its Part_of_MRZ register is of great importance. The MRZ generator flexibly distributes the key space and enables adapting the key search according to the knowledge about the passport holder and the issuing state of the ePass. The latter property is extremely important for breaking BAC keys, as the entropy of the MRZ can be considerably reduced [Rob, LKLRP07] with an increasing knowledge of the adversary, as compiled in Sect. 9.4.3. The searching strategy can be updated for each scenario from the host PC via the USB port — the only information to be transferred to each FPGA for the adaption is the boundary of the assigned key space. After initializing the registers, each FPGA runs independently and no more communication is initiated, until a key is found. Some examples for partitioning the key space according to the associated scenario are given in Section 9.5.2.

Encryption Engines

The main components implemented in each FPGA are four encryption engines, whose outputs are fed into four comparators for detecting a match with the default ciphertext, as illustrated in the left of Fig. 9.5. The delivered ciphertexts are compared to the correct one stored in the ciphertext register. In case of a match, the sought-after MRZ is returned to the data bus.

Computing K_{ENC} As illustrated in Fig. 9.6, each encryption engine for attacking K_{ENC} consists of an access-key generator and a Triple-DES processor. The access-key generator basically

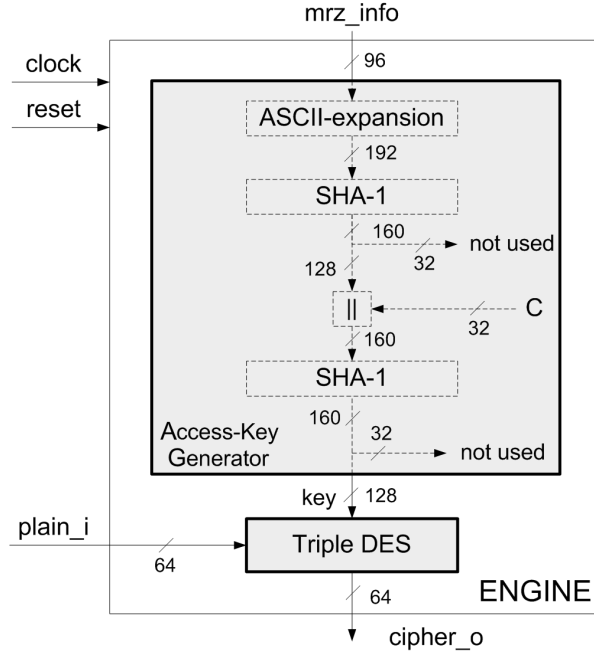


Figure 9.6: Details of the internal structure of an encryption engine for targeting K_{ENC} .

performs two SHA-1 algorithms with the appropriate constants to derive the keys for the **BAC** from the **MRZ** information, as detailed in Section 9.2. For reducing the data traffic on the buses of the **COPACOBANA**, the original 192 bits of **MRZ** information are compressed to only 96 bits before being sent to the **FPGAs**. It is the task of the **ASCII-expansion** unit to reconstruct the genuine **MRZ** information from the compressed data before the execution of the first SHA-1.

To further speed-up the calculation of the SHA-1, needing 80 clock cycles for one execution and therefore being the slowest part of the whole implementation, it is pipelined: when the first SHA-1 has processed its data, it hands over the output value to the second SHA-1 and instantaneously hashes the next **MRZ** information obtained from the **MRZ** generator, thus enabling simultaneous operation. When deriving K_{ENC} , pipelining is not advantageous for the Triple-**DES**, as its computation delivers a result after only 48 clock cycles and is hence faster compared to the SHA-1. The Triple-**DES** part of the **FPGA** is idle for $80 - 48 = 32$ clock cycles until the next output of the SHA-1 is determined, hence the pipelining of the SHA-1 constitutes the most efficient time-area tradeoff for our design.

Computing K_{MAC} For the second proposed attack targeting the **MAC**, four more **DES** rounds have to be executed in addition to the Triple-**DES**, as illustrated on the right side of Fig. 9.5. Hence, $7 \cdot 16 = 112$ clock cycles would be needed after each SHA-1 computation, if only one round of DES was implemented. This would have a bad impact on the overall performance, as $112 - 80 = 32$ clock cycles of idle time would occur each time a new hash value is delivered. Instead, our solution implements a second round of **DES** on each **FPGA**, for which sufficient area was available after some optimizations of the control logic. Now, the post-processing after the SHA-1 takes 56 clock cycles, and is thus again $80 - 56 = 24$ clock cycles faster than the SHA-1. Indeed, targeting K_{MAC} is as costly with respect to the execution time as aiming at

K_{ENC} , so that the practical results presented in the following apply for both implementation options.

9.5.2 Practical Results

To emphasize the practical relevance of our attack, we have implemented some of the scenarios proposed in Sect. 9.4.3 in the hardware description language **VHDL**. The code was first simulated and debugged with Xilinx edition of Mentor Graphics' Modelsim⁴ and then programmed into the **COPACOBANA**. The encrypted communication data was captured from several German e-passports using the customized **RFID** reader developed in Sect. 4.1 in our laboratory. The implementations for all detailed scenarios have been thoroughly tested and were able to find the correct **BAC** key for all passports tested.

As stated earlier, the time-critical component is the SHA-1, which requires 80 clock cycles to convert an **MRZ** into a Triple-**DES** key and further determines the critical path and thus the maximum clock frequency $f_{clk} = 40$ MHz. Processing of one key thus corresponds to $80 \cdot 25 \text{ ns} = 2 \mu\text{s}$. It follows that a single **FPGA**, containing four encryption engines working in parallel, can check four keys in $2 \mu\text{s}$, i.e., two million keys per second. For all 120 **FPGAs**, this results in $4 \cdot 120 = 480$ keys being tested every $2 \mu\text{s}$, i.e., a throughput of 240 million or $2^{27.84}$ keys per second. The variable part of the implementations is the **MRZ** generator which can be adapted

Table 9.5: Results for the practical implementation of some scenarios.

Issuing State:	Germany	The Netherlands
Scenario 2		
Total amount of MRZ candidates	$4.33 \cdot 10^{13}$	$2.49 \cdot 10^{13}$
Average time to find the MRZ	$\approx 9.02 \cdot 10^4 \text{ s} \approx 25 \text{ h}$	$\approx 5.18 \cdot 10^4 \text{ s} \approx 14 \text{ h}$
Scenario 3		
Total amount of MRZ candidates	$1.35 \cdot 10^{12}$	
Average time to find the MRZ	$\approx 2.82 \cdot 10^3 \text{ s} \approx 47 \text{ min}$	
Scenario 4		
Total amount of MRZ candidates	$1.06 \cdot 10^{10}$	$4.9 \cdot 10^9$
Average time to find the MRZ	$\approx 22 \text{ s}$	$\approx 10.3 \text{ s}$
Scenario 5		
Total amount of MRZ candidates	$8.85 \cdot 10^7$	
Average time to find the MRZ	$\approx 185 \text{ ms}$	

to the different scenarios such that every **FPGA** is occupied with key searching as long as possible. This minimizes the communication overhead and hence maximizes the throughput of the machine. We found that the optimal solution is opting for the date of birth of the passport holder as the fixed portion in an **MRZ** generator. This is especially convenient for Scenario 2 to Scenario 5 with regard to the partitioning, because there are exactly 120 months in 10 years

⁴<http://www.mentor.com/>

to be distributed to the 120 FPGAs. The running times for several scenarios are summarized in Table 9.5. For example, testing of 2^{33} keys, as required for Scenario 4, reveals the correct candidate on average in $2^{32.3}/2^{27.84} \approx 22$ seconds which suffices for a practical attack, compared to the duration of one inspection at the border control.

9.6 Other Implementation Options

For comparison, this section estimates the efficiency of implementing the presented key-search attack in software and gives hints about new hardware architectures for cryptanalysis.

9.6.1 Software Implementation

An alternative choice for the cryptanalysis of the BAC scheme is a software implementation. Fast implementations on the Pentium family require 837 cycles per SHA-1 operation and 928 cycles per Triple-DES operation [Bos]. Implementing a key search based on MRZ data needs two SHA-1 and one Triple-DES, i.e., 2602 cycles in total. If pre-computed BAC keys can be used, only one Triple-DES is required. Considering a Pentium clocked at 3.0 GHz, one can check about 1.15 million $= 2^{20.1}$ keys per second without pre-computation and 3.23 million $= 2^{21.6}$ keys with pre-computation. For the low-end scenarios involving the powerful adversary $\mathcal{A}_3$, software solutions are already appropriate and probably the method of choice for implementing tracking systems. However, testing 2^{35} key candidates requires 8.5 hours without pre-computation and 3 hours with pre-computation on a single Pentium. Clusters of standard computers can further speed-up the throughput.

9.6.2 New FPGA Key Search Machines

The main performance bottleneck of our implementation on the COPACOBANA is the SHA-1 computation that requires 80 clock cycles per key candidate. Furthermore, the SHA-1 sets the upper bound for the clock frequency as it determines the critical path of the overall implementation. However, as COPACOBANA was originally designed for a complete DES key search, sufficient memory for pre-computation is not available on this machine.

An improved follow-up of the COPACOBANA, termed RIVYERA⁵, relies on more powerful FPGAs, provides an improved communication bus and supports large amounts of on-board RAM. The machine is hence a promising platform to realize even faster implementations of exhaustive key-search, especially for realizing TMTO attacks.

9.7 Conclusion

This chapter tackles the security of probably the most powerful passive RFID chip available on the market, i.e., the electronic passport: the first hardware implementations for cracking the secret keys of the BAC protection scheme of these state-of-the-art contactless smartcards are presented. One attack requires to monitor both directions of the communication, while for a second, new attack, eavesdropping on the far-ranging requests of the reader is sufficient. The brute-force

⁵<http://www.sciengines.com>

attack is realized on the **COPACOBANA** that turned out to be a flexible platform for implementing probabilistic key-search scenarios. The achieved throughput is 240 million $\approx 2^{28}$ **BAC** keys per second. Testing 2^{35} key candidates requires 2 minutes and 23 seconds on **COPACOBANA**. This yields a factor of 214, if compared to a software implementation without pre-computation, and a factor of 74, if compared to a software implementation with pre-computation.

The findings presented in this chapter allow to circumvent the **BAC** security mechanism that aims at establishing confidentiality of the wirelessly transmitted, private data of the passport holder. Despite the cryptographic primitives of the **BAC** being adequately secure, flaws in the derivation of the secret keys imply dramatic consequences for the privacy and security of passport holders: After monitoring the wireless communication with a distant eavesdropper the secret keys for the **BAC** can be recovered in a short time — depending on the scenario in seconds — and the recorded transmissions can be decrypted.

The results of this chapter demonstrate that key-search machines constitute a real threat for the privacy and security of **ePass** holders. The practical realization of the **BAC** of the first generation of electronic passports in the Netherlands and in Germany should be regarded critically: An adversary can gather private information about passport holders, including a digitized photograph, from a distance. This is possible due to the low entropy of the **MRZ**. Besides collecting digitized private data, individuals can be unauthorizedly identified using our implementation. In consequence, their movements can be tracked and events related to their personal information could be automatically triggered.

Our results also show that, if the full entropy would have been used, an attack would be practically infeasible even with the special-purpose hardware currently at hand. Still, the key search performance can be significantly increased if fast on-board **RAM** for precomputations is available in future realizations of key-search machines.

The attacks are applicable to several variants of the electronic passport worldwide. After publicizing the findings presented in this chapter a new improved version of the German electronic passport, that amongst others introduces alphanumerical characters to increase the entropy of the **MRZ**, was released in November 2007. However, the passports issued in the previous two years remain vulnerable to the described attacks⁶. For the security and safety of passport holders, the locations in which the inspection systems operate should be properly shielded and other measures should be taken to prevent eavesdropping.

Acknowledgements

We would like to thank Tim Güneysu and Martin Novotný for their helpful and detailed explanation of how to use the key search machine **COPACOBANA**.

⁶The validity of a German passport expires after ten years

Chapter 10

Conclusions and Future Research

THIS thesis illustrated the security risk in many of today's wireless embedded devices. In the following, the main contributions of all chapters are reviewed, learnt lessons are pinpointed, and some ideas for future research are presented.

Tools for Security Analyses For practical analyses, a cost effective tool-set that is optimized for physical and protocol attacks on wireless tokens was presented, which can be extended to support virtually any type of embedded (cryptographic) device. The framework comprises functions for communication, automatic profiling, and security evaluation with respect to side-channel analysis (**SCA**), e.g., the analysis of **EM**, power, or timing measurements; fault injection, e.g., via the **EM** field; cryptanalytic attacks, e.g., brute-force or exploiting weak ciphers such as used in Mifare Classic cards; and protocol attacks, e.g., man-in-the-middle (**MITM**) or replay attacks. The most efficient practical key-recovery attack on Mifare Classic cards known to date has been realized using the developed low-cost **RFID** reader, which has been made available to the public as a **GPL** project [KCP07]. The implementation reveals a cryptographic key of a Mifare Classic card non-invasively in approx. 30 seconds.

Furthermore, tools for *emulating* different wireless devices have been developed. Amongst them is a microcontroller-based, freely programmable emulator for ISO 14443 compliant contactless smartcards, termed Chameleon. It enables simulating these cards, including their factory-programmed Unique Identifier (**UID**) and all cryptographic features, at a low cost (less than 20 €). The Chameleon has been successfully employed for various practical attacks in this thesis, e.g., to unauthorizedly open doors and to spoof a contactless payment system, and is maintained as a **GPL** project [KvMOP11].

Key-Recovery Attacks The process of profiling and analyzing a black-box device, with virtually no previous knowledge about its internals, was described, and implementation attacks were introduced. The concept of **SCA** was detailed in theory, and then applied to evaluate the security of **RFID** devices and actively-powered wireless tokens in the real world. As a result, powerful **SCA** attacks on commercial products were demonstrated in practice: The cryptographic keys of KEELOQ remote controls and the corresponding receivers of the Remote Keyless Entry (**RKE**) system were extracted from approx. 10 power measurements or one single power measurement, respectively. Similar attacks are also applicable to commercial cryptographic **RFID** devices, using the developed analog circuitry and filtering techniques: **SCA** of the **EM** field emanated by Mifare DESfire MF3IDC40 cards reveals the secret keys used by their **3DES** engine, however, with comparatively large effort. Note that no details about the cryptographic implementations of the targets had been known in advance, hence the devices had to be profiled in a tedious step-

by-step process to identify the appropriate leakage model and reverse-engineer their internal architecture.

A brute-force attack implemented on the codebreaker **COPACOBANA** assesses the security level provided by KEELOQ-based systems that utilize a seed-based key derivation. Another implementation of exhaustive key search targets the Basic Access Control (**BAC**) scheme securing electronic passports: In practical scenarios, **COPACOBANA** reveals the cryptographic keys, that protect private data, in seconds.

Impacts of Key Recovery in the Real World After extracting secret keys of commercial wireless products employing cryptography, the implications for the system as a whole were analyzed in each case. Various significant security vulnerabilities that can be exploited by a malicious opponent were pinpointed: A *contactless payment application* based on Mifare Classic cards is found to have the same secret keys programmed into all cards in the system. After extracting the keys from one card an attacker can — in 40 ms and from a range up to 30 cm — read out a card or write to it, increase or decrease its credit balance, or clone the card of a victim and impersonate him. Great financial damage can occur due to converting digital money into real cash or conducting a Denial-of-Service (**DoS**) attack.

Likewise, applications relying on the more secure Mifare DESfire MF3IDC40 cards are at risk. In the last approx. 10 years, these cards have been mostly deployed in very sensitive environments with high security demands, e.g., as **ID** cards, for access control, or for payments. Due to the presented key recovery by means of **EM** analysis, real-world attacks are likely to evolve in the near future.

Once the manufacturer key of a KEELOQ product is disclosed, an unskilled adversary can compute the cryptographic keys of corresponding remote controls and clone them by eavesdropping, from a distance of several hundred meters. She can arbitrarily access a secured object without leaving physical traces. Compared to a mechanical key, the security provided by a KEELOQ door opener thus comes off badly. In addition, the adversary can take over control of the **RKE** system and execute a **DoS** attack that locks out the legitimate user of the system, but still grants her access.

Circumventing the **BAC** of electronic passports and decrypting the personal information enables creating data bases and tracking of individuals. The weak **BAC** scheme implemented in the first generation of passports thus violates the privacy of many passport holders.

For the analyzed applications, countermeasures to improve the security of the systems were proposed that can raise the security of the existing systems to an acceptable level. In case of KEELOQ a secure, **SCA**-resistant scheme that can serve as a replacement is proposed.

A Main Cause for Weak Systems: Flawed Key Distribution Surprisingly, the analyses in this thesis reveal that an adequate distribution of the cryptographic keys seems to pose a major problem in commercial wireless systems, despite the elaborated, secure schemes known in the scientific community. Due to lack of a key derivation, the security of the analyzed contactless payment system completely collapses after extracting the keys of one card. As a consequence, wireless attacks from a distance become feasible — even if more secure cards would be employed. Implementing an appropriate key distribution mechanism would render most of the attacks impractical: Each payment card would possess an individual set of keys which would have to be extracted separately for each card in order to cheat on the system.

Compared to the above “fixed-key” realization, KEELOQ **RKE** systems provide a better key-derivation mechanism that assigns a unique secret key to each remote control. However, the secret manufacturer key for the key derivation poses a single point of failure (**SPOF**) and is contained in devices that are in the hands of potential attackers, i.e., in every KEELOQ receiver. Once an attacker recovers the manufacturer key, e.g., via **SCA**, she can compute the secret key of any remote control of the **RKE** system and thus open all doors.

In the key management of the **BAC** used by the electronic passport, the individual keys are derived from data printed on the document and are hence known to anyone having physical access to the passport. In consequence, **SCA** attacks are not expedient — why should an adversary who can visually read the secret key from the passport perform a complicated attack? However, knowing the key of one passport has no system-wide consequences, as the keys of distinct passports are not linked to each other. Unfortunately, the **BAC** scheme was also not realized adequately: The key space is so small that exhaustive key-search becomes practical. Likewise to the other investigated systems, again a weak key derivation allows an attacker to decrypt the communication of passports.

Effort for Practical Attacks and Inferences The developed tools and techniques set new lower bounds with respect to the cost and effort required for practical attacks. Cryptanalytical attacks and even state-of-the-art implementation attacks can be performed by anyone who has a sufficient know-how.

Performing the first successful **SCA** of an unknown target typically requires a highly skilled adversary, a lot of effort, and patience. However, after the first obstacles are taken, e.g., the cipher or other implementation details have been reverse-engineered, communication with the **DUT** has been established, and the encryption has been located in the power measurements, the efficiency of a key recovery and further attacks on the system level can develop rapidly. Once experience with a particular target has been gained, **SCA** attacks often become highly effective and analyzing similar targets or improved variants of the same target becomes straightforward.

In many cases, a key extraction can be automatized and in the following be performed by an unskilled adversary, who possesses the required hardware and software tools. Thus, depending on the actual system realization and the strength of the cryptographic components, the prerequisites with respect to the adversary vary. In case of a system relying on Mifare Classic, automatized tools for a key extraction are available, hence all steps to completely circumvent the security of the analyzed contactless payment system are feasible by an unskilled adversary. Extracting the manufacturer key of KEELOQ products or tackling the **3DES** encryption of a Mifare DESfire card by means of **SCA** is more complicated and thus only feasible for a skilled attacker — unless devices performing an automatized key recovery would become available. Once secret keys have been extracted, the realization on the system level determines, whether an unskilled adversary can cheat on the system, as in the case of KEELOQ rolling code applications. The brute-force hardware required for extracting keys of electronic passports usually relates to a skilled, well-funded adversary or organization. However, an attacker aiming at spying out private data on a large scale or at tracking individuals can often fulfill these requirements.

Altogether, the impact of a successful key recovery in the context of real-world systems turned out to be far more dramatic than expected: In all investigated commercial wireless applications, devastating flaws in the implementation on the system level that facilitate a complete system break and provoke fraud were disclosed. Reviewing the relevance of the various attack vectors,

and considering the low boundaries of their cost as determined in this thesis, performing solely implementation attacks would be — despite the low cost — not a practical threat for most systems, because the attacks typically require physical access to the targeted device. If each key in the system has to be recovered individually, e.g., by means of power-analysis, the ratio of the required effort and know-how to the benefits for a criminal can be considered too low for most systems. Note that only the keys of devices that are already in the possession of the adversary (at least sufficiently long to perform the attack) can be extracted. For example, smashing the window of a car or mechanically circumventing a door lock is more straightforward for gaining access to a protected object, compared to an **SCA** attack. Exceptions are contactless applications with very high security demands, e.g., access to military sites, in which extracting a single secret key already can have a very high impact.

Real practical threats arise mostly from those attacks that can be performed with low effort by an unskilled adversary, and in the context of wireless devices often from a distance: These attacks are often related to flaws in the system realization, e.g., the knowledge of one secret key results in failure of the security of the complete system, or a weak random number generator enables a straightforward attack. This type of “secure” system will hopefully disappear from the market soon, e.g., after the responsible designers have read this thesis. Other practical attacks remain that apply even to cryptographically secure systems and do not involve knowledge about cryptography, for instance accessing a car by jamming the frequency of its door opener or stealing it by means of a **MITM** attack are well-known techniques amongst car thieves.

Ethical Remarks What distinguishes scientists from criminals: Firstly, the former do not intend to harm other parties or to gain financial profit through illegal actions on the basis of their findings. Thus, researchers should act responsibly in order to minimize the damage provoked by their scientific results. Concretely, this implies that an affected enterprise or institution, e.g., producing or using a vulnerable product, should be informed about planned publications beforehand, in order to give the system integrators time to fix the security issues in the field.

In all contributions to this thesis, the affected manufacturers and system integrators were instantly notified of our findings and all papers were published only after previous communication. In most cases, the manufacturers were thankful for the valuable feedback and in consequence gave their permissions for a publication without problems. They understood that it is advantageous for them and their customers to know about security risks, and correspondingly be able to counteract them before criminals exploit the weaknesses and cause severe damage. Analyzing weaknesses of commercial products is even welcome by the security departments of many enterprises: They often know about certain weaknesses in the products, but cannot successfully enforce their elimination within the enterprise, often due to the higher cost of more secure variants. Allegedly, the budget for security measures is increased after the publication of vulnerabilities, at the latest when the image of the enterprise is at imminent danger.

10.1 Lessons Learnt

In short, the lessons learnt from this thesis and several years of experience with analyzing real world products are:

- Wireless systems are prone to special attacks and hence must be treated differently from contact-based solutions: Activating and communicating with wireless devices can happen imperceptible from a distance, thus extra security measures, e.g., to prevent identity theft or MITM attacks, are required.
- System design is important: Using secure ciphers and an appropriate key distribution is mandatory, a SPOF must be avoided, and mechanisms for detecting fraud need to be implemented.
- Implementation attacks, such as SCA, fault injection, or reverse engineering of program code, are often feasible with modest effort and are thus a crucial attack vector, even though physical access to the target is required: Choosing a secure hardware platform and considering a protection against SCA is compulsory. Regard the system as a whole!
- Attackers learn: Various security analyses are very complicated and costly at the beginning, but often rapidly evolve into highly efficient variants posing a practical threat once more details about the attacked system have been revealed. Even automatized SCA, e.g., for cloning remote controls or transponders, is conceivable. Don't educate the attacker with weak products!
- Team play: Security-analyzing real-world products is usually a big project requiring huge effort and great skills in various domains, e.g., reverse-engineering of code or circuits, establishing communication with the target, understanding protocols, acquiring and evaluating side-channel leakage, filtering and processing data, performing mathematical cryptanalysis, programming cryptanalytic hardware, ... — it is unlikely that a single person can perform all these tasks in a reasonable time and thus collaboration with security experts is advisable, likewise for designing secure products.

10.2 Future Directions

From the findings in this thesis, numerous other research topics emerge. Some ideas are listed next.

- The developed low-cost setup for implementation attacks can be augmented with respect to various types of fault injection and the promising *combination of side-channel analysis with fault injection*. The susceptibility of RFIDs with respect to inducing pulses via the EM field can be broader researched.
- The analog circuitry, filters, and SCA techniques for analyzing contactless smartcards can be improved further — more efficient attacks are likely to be feasible.
- The vulnerability of other contactless smartcards towards EM-based SCA attacks should be investigated, e.g., Sony Felica, Legic Advant, and various Java cards. Furthermore, electronic passports, governmental ID cards, or the Mifare DESfire EV1 (which at first glance appeared more secure than its predecessor DESfire) might be susceptible to improved attack variants.

- The achievable ranges for remotely extracting the keys of cryptographic devices by means of power analysis, i.e., *EM-based SCA attacks from a large distance*, should be broader researched.
- The presented attacks can be adapted to **RFIDs** operating at 125 kHz, such as Atmel CryptoRF used to counteract product piracy or the Hitag 2 and Megamos ciphers used in car immobilizers. Contrary to analyzing contactless smartcards, here low-pass filtering the signal is probably no option, since extremely little information leakage can be expected in the remaining frequency range below 125 kHz.
- Other channels and methods for eavesdropping on contactless smartcards can be tested: Most studies with respect to the achievable range to the knowledge of the author have been conducted with coil-shaped antennae that are inductively-coupled and hence only suitable for capturing the (magnetic) near-field component emanated by the **DUT**. However, the communication might be detectable from much greater distances in the far field, using directional far-field antennae. During our experiments, various other parasitic effects occurred, e.g., the **RF** signal of our **RFID** reader coupled via the **USB** cable into the controlling **PC** — the side-channel leakage hence was still measurable at the power socket of the **PC**.
- Emulating contactless smartcards can further be researched in two different ways: (i) an emulator for arbitrary cards (including the **UID** and all other content) should be realized on **NFC**-compliant smartphones. (ii) the feasibility of a long-range card emulator based on an active transmitter that sends the communication data at the respective frequencies instead of performing a load modulation should be investigated. Such a device would enable new real-world attacks, especially in contactless applications that assume the direct vicinity of the participating devices.
- Lightweight cryptography for **RFIDs** including **SCA** countermeasures should be further improved. The currently deployed devices for capacitive-coupled, long range **RFID**, i.e., **EPC** transponders [EPC10], cannot provide cryptographic schemes at all, because the amount of available energy is not sufficient. Other types of **RFID** devices do not provide **SCA** countermeasures due to the increased demand for power. Researching secured lightweight implementations for constrained devices should consider that the area of the chip is *not* relevant for the energy consumption. On the contrary, the amount of switching transistors determines the power characteristics: It might hence be an option to include relatively large circuits on an **RFID** chip, as long as only the parts required for the current fraction of the computations are switching (while the unused parts are temporarily not supplied with energy).
- Practical countermeasures against **MITM** attacks should be further researched.
- Secure schemes for key exchange and key distribution can be further researched and implemented in constricted wireless devices.

Appendix A

Statistical Tests for Timing Analysis

Section 3.2.1 demonstrates at hand of the SAM algorithm that a key-dependent runtime of a cryptographic algorithm leaks critical information, e.g., the secret key. Often, the dependency is not so evident, e.g., in case of the Square-and-Always-Multiply algorithm: This secured variant of SAM always performs a multiplication during the exponentiation, regardless of the processed bits of the exponent. The timing difference between $e_m = 0$ and $e_m = 1$ is thus very small, if there is any. In order to detect such small variations, a thorough statistical background is required, as given in this section which is based on the Master’s thesis of Andreas Hoheisel, advised by Thorsten Schütze and myself [Hoh09].

In the following, a random variable X with a Gaussian distribution denotes the runtime of the algorithm¹, i.e., $X \sim N(\mu_X, \sigma_X^2)$ with mean μ_X and variance σ_X^2 . For further details to statistics, e.g., probability density functions, statistical tests, etc., refer to any standard statistics book, for example [WG04, Bos98].

For a statistical analysis, runtime measurements with different plaintexts and different keys are performed. A measurement set consists of a fixed number N of measurements, each of which contains the plaintext, the key, and the runtime of an encryption. Due to the accuracy of the measurement process and platform-dependent variations of the runtime, fluctuations in the measured runtime occur. For example, a cache could be flushed during one execution of the algorithm, resulting in a different runtime for the corresponding measurement. The goal of the statistical tests introduced in the following is to verify, if these outliers or differences are systematic or not, i.e., to test, if different measurement sets can be distinguished at hand of their runtime.

A.1 Two Sample T-Test

The density function of a *T-distributed* random variable X with $m \in \mathbb{N}$ degrees of freedom is given by

$$f_{X,m}(x) = \frac{\Gamma(\frac{m+1}{2})}{\sqrt{\pi m} \Gamma(\frac{m}{2})} \left(1 + \frac{x^2}{m}\right)^{-\frac{m+1}{2}} \quad \text{for } x \in \mathbb{R}, \quad (\text{A.1})$$

where Γ denotes the Gamma function defined as

$$\Gamma(1) = 1, \quad \Gamma(n+1) = n! \quad \text{for } n \in \mathbb{N}.$$

¹More exactly, the runtime of an algorithm for a deterministic input is composed of a deterministic part and a random part which is caused by algorithmic noise, cache and pipelining behavior etc. — it is assumed that this random part is normally distributed.

Γ can be generalized to arbitrary real arguments $x \in \mathbb{R}$. For sufficiently large m , the density function of the T-distribution approximates the Gaussian normal distribution $N(0, 1)$, i.e., $f_{X,m} \xrightarrow{m \rightarrow \infty} N(0, 1)$.

If X is an $N(0, 1)$ -normal distributed random variable, Y is a chi-squared (χ^2) distributed random variable with m degrees of freedom, and both random variables are independent, then the random variable

$$Z = \frac{X}{\sqrt{\frac{Y}{m}}}$$

is *T-distributed* with m degrees of freedom.

Finally, the two sample T-test is defined: Let X_i ($i = 1, \dots, m$) and Y_j ($j = 1, \dots, n$) be independent and identically distributed (i.i.d) random variables according to

$$X_i \sim N(\mu_X, \sigma_X^2) \quad \text{and} \quad Y_j \sim N(\mu_Y, \sigma_Y^2),$$

with equal but unknown variances $\sigma_X^2 = \sigma_Y^2$. Under the hypothesis $H_0 : \mu_X = \mu_Y$, the test characteristic

$$T = \frac{\bar{X} - \bar{Y}}{S_P} \sqrt{\frac{mn}{m+n}} \quad (\text{A.2a})$$

with

$$S_P^2 = \frac{1}{m+n-2} \left((m-1)S_X^2 + (n-1)S_Y^2 \right), \quad (\text{A.2b})$$

$$S_X^2 = \frac{1}{m-1} \sum_{i=1}^m (X_i - \bar{X})^2, \quad (\text{A.2c})$$

$$S_Y^2 = \frac{1}{n-1} \sum_{j=1}^n (Y_j - \bar{Y})^2 \quad (\text{A.2d})$$

has a T-distribution with $m+n-2$ degrees of freedom.

For testing, if both random variables stem from the same population, two hypotheses are made:

$$H_0 : \mu_X = \mu_Y \quad \text{and} \quad H_1 : \mu_X \neq \mu_Y. \quad (\text{A.3})$$

With some significance level α , for example, $\alpha = 0.05$, the hypothesis H_0 can be rejected, if

$$|T| \geq t_{m+n-2; 1-\frac{\alpha}{2}}, \quad (\text{A.4})$$

where $t_{m+n-2; 1-\frac{\alpha}{2}}$ denotes the $1 - \alpha/2$ quantile of the T-distribution with $m+n-2$ degrees of freedom.

A.2 Fisher's F-Test

The T-test evaluates the mean values of measurements under the assumption that the unknown variances are identical. To statistically test the observed variances, Fisher's F-test can be used.

The F-test allows to verify statistical moments of order two, which are usually very small and difficult to detect. Assuming two normally distributed random variables the F-test indicates, if the variance of one random variable equals the variance of another random variable with unknown, but not necessarily equal mean values.

The density function of an *F-distributed* random variable X with (m, n) -degrees of freedom, $m, n \in \mathbb{N}$, is given by

$$f_X(x) = \begin{cases} \frac{\Gamma(\frac{m+n}{2})m^{\frac{m}{2}}n^{\frac{n}{2}}}{\Gamma(\frac{m}{2})\Gamma(\frac{n}{2})} \cdot \frac{x^{\frac{m}{2}-1}}{(n+mx)^{\frac{m+n}{2}}} & \text{for } x > 0, \\ 0 & \text{for } x \leq 0. \end{cases} \quad (\text{A.5})$$

If X and Y are two independent χ^2 -distributed random variables with m and n degrees of freedom, $X \sim \chi_m^2, Y \sim \chi_n^2$, then the random variable

$$Z = \frac{X/m}{Y/n}$$

is F-distributed with (m, n) -degrees of freedom.

Let $F_{m,n;\alpha}$ be the quantile with the order α . For large n the quantile of the F-distribution with (m, n) -degrees of freedom converges to the quantile of the χ^2 -distribution with m -degrees of freedom multiplied with the factor $1/m$:

$$F_{m,n;\alpha} \xrightarrow{n \rightarrow \infty} \frac{\chi_{m;\alpha}^2}{m}.$$

Now Fisher's F-test can be applied: Let X_i ($i = 1, \dots, m$) and Y_j ($j = 1, \dots, n$) be independent and normally distributed random variables,

$$X_i \sim N(\mu_X, \sigma_X^2) \quad \text{and} \quad Y_j \sim N(\mu_Y, \sigma_Y^2),$$

with μ_X and μ_Y unknown. Under the hypothesis $H_0 : \sigma_X^2 = \sigma_Y^2$, the test statistic

$$T = \frac{S_X^2}{S_Y^2} \quad (\text{A.6a})$$

with

$$S_X^2 = \frac{1}{m-1} \sum_{i=1}^m (X_i - \bar{X})^2, \quad (\text{A.6b})$$

$$S_Y^2 = \frac{1}{n-1} \sum_{j=1}^n (Y_j - \bar{Y})^2 \quad (\text{A.6c})$$

is F-distributed with $(m-1, n-1)$ degrees of freedom.

For applying the F-test, the hypotheses

$$H_0 : \sigma_X^2 = \sigma_Y^2 \quad \text{and} \quad H_1 : \sigma_X^2 \neq \sigma_Y^2 \quad (\text{A.7})$$

are computed.

The hypothesis H_0 can be rejected with a significance level α , for example, $\alpha = 0.05$, if

$$|T| \geq F_{m-1, n-1; 1-\frac{\alpha}{2}}. \quad (\text{A.8})$$

Appendix B

Side-Channel Resistance in the Context of AES

This section summarizes various practical countermeasures for securing cryptographic algorithms against **SCA** attacks and has emerged from the Master's thesis of Andreas Hoheisel, advised by Thorsten Schütze and myself [Hoh09]. The following discussion focuses on software implementations of the Advanced Encryption Standard (**AES**) [DR02], as a common example for a block cipher. Like all unprotected implementations of ciphers, **AES** is vulnerable to power-analysis attacks. We here consider common attacks, such as **SPA**, **DPA**, and **CPA**, but exclude highly sophisticated approaches, such as template attacks. The aim is to find a compromise between implementation overhead and gained protection, to establish a medium security level, i.e., at least resistance against first-order **DPA** attacks. The target platform is a common 32-bit μC as used, e.g., in automotive applications or in smartphones.

Next, power analysis in the context of **AES**-128 is introduced in Sect. B.1. Various masking and randomization countermeasures are illustrated in Sect. B.2 and Sect. B.3, respectively, and their implementation cost and security issues are discussed. Choosing a suitable Random Number Generator (**RNG**) for the countermeasures is described in Sect. B.4. Finally, Sect. B.5 compares the schemes and indicates how to find a compromise between security and implementation cost.

B.1 Introduction

We recall four different attack scenarios that are commonly used to determine, how powerful an attacker is:

known-plaintext attack the attacker knows pairs of ciphertexts and plaintexts

known-ciphertext attack attacker only knows ciphertexts

chosen-plaintext attack the attacker can specify the plaintexts

chosen-ciphertext attack the attacker can specify the ciphertexts

A main criterion for the feasibility of a power-analysis attack is, that a part of the targeted implementation must depend on a small amount of the key bits only (compare with Sect. 3.2.2). **AES** possesses such exploitable functions, e.g., the first and the last *AddRoundKey*. Since they process only eight bits at a time, these operations depend only on a small amount of key bits. **AES** provides another interesting characteristic which makes a power-analysis attack even more applicable, i.e., the nonlinear *SubBytes* transformation¹. Nonlinearity facilitates the testing of

¹Other ciphers with a diffusion step, for example **DES**, exhibit similar characteristics.

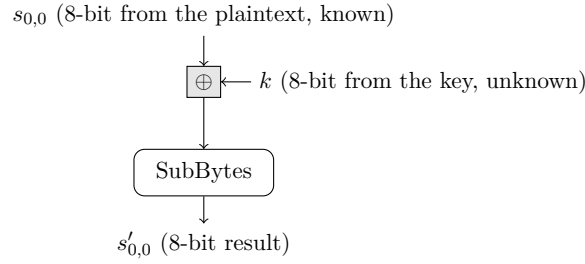


Figure B.1: [A promising target for power-analyzing the AES: Initial AddRoundKey, followed by the first SubBytes transformation in round one, for one byte of the State matrix.]

key hypotheses: If there is one bit wrong in the hypothesis, several bits of the S-Box output do not correspond to the hypothesized intermediate result. On average, half of the bits that were predicted by an incorrect key hypothesis differ from the actual intermediate result — independently of the number of incorrect bits in the key hypothesis. This enormously improves the process of finding the right key during an **SCA**. Figure B.1 depicts the initial *AddRoundKey* operation (XOR) before the first round and the first *SubBytes* transformation for the byte $s_{0,0}$ of the State matrix in round one. Hypothesizing the output of *SubBytes*, $s'_{0,0}$ in the figure, is promising for mounting a successful attack. Thus, an important task for realizing a side-channel resistant implementation of **AES** is protecting the *SubBytes* transformation.

B.2 The Masking Countermeasure

During a power analysis attack, the attacker hypothesizes the power consumption of the **DUT** performing the attacked operation. Our goal is, to make all intermediate values of the executed computations independent of the measured power consumption. This can be realized by masking the processed values.

Masking conceals every intermediate value with a random value m , the so-called mask. The attacker does not know the masking values and thus cannot compute meaningful hypothesis on the power consumption. As a result, the side-channel leakage of all intermediate, key-dependent values does not correlate with their hypothesized, unmasked counterparts.

At the latest before the ciphertext is output, the mask has to be removed. Therefore, the propagation of the mask throughout the **AES** has to be reproduced. Removing the mask turns out to be straightforward for the linear parts of the cipher. For the nonlinear part, namely the *SubBytes* transformation, considerable more effort is required, as will be shown in the following sections.

Next, various masking schemes that have been proposed in the literature until 2009 are illustrated and their security is discussed. Further, the cost for implementing the schemes on an 8-bit μC is estimated in terms of memory consumption and runtime (clock cycles) required for an encryption. For all masking schemes, the runtime of the key scheduling is excluded. The 176-byte **RAM** consumption for the expanded round key is included in the estimated **RAM** value.

We start with the simple additive masking in Sect. B.2.1, followed by the transformed masking method in Sect. B.2.2. Masking an **AES** inverter is detailed in Sect. B.2.3 and the approach of

combined masking in tower fields is introduced in Sect. B.2.4. Section B.2.5 then shows how to mask the MixColumns transformation of the AES.

B.2.1 Additive Masking

Additive masking conceals a value x with a random mask m by adding (XORing) them, to obtain the masked value $x_m = x \oplus m$. The mask can be removed by adding the mask value again, i.e., $x = x_m \oplus m$. For all linear operations, additive masking is a straightforward, but very effective masking scheme: If the mask m is random², the masked value x_m is not distinguishable from a random value.

However, the *SubBytes* transformation of the AES is a non-linear operation, which means $\text{SubBytes}(s_{i,j} \oplus m) \neq \text{SubBytes}(s_{i,j}) \oplus \text{SubBytes}(m)$. Thus, additive masking cannot be easily applied “out of the box”. The mask value m cannot be easily removed from the intermediate value $s'_{i,j}$ after the transformation without modifying the *SubBytes* transformation.

Additive Masking of the S-Box

For applying additive masking, the whole S-box has to be recomputed. The input mask m masks the index and the output mask mx conceals the result of the SubBytes call according to Alg. 3. This masking method was published first in [Mes00].

Algorithm 3 Computation of the additively masked AES SubBytes transformation, as proposed in [Mes00].

Input: m, mx

Output: $\text{MaskedSubBytes}(s_{i,j} \oplus mx) = \text{SubBytes}(s_{i,j}) \oplus m$

```

1: for  $i = 0$  TO 255 do
2:    $\text{MaskedSubBytes}(i \oplus mx) = \text{SubBytes}(i) \oplus m$ ;
3: end for
```

Implementation Considerations With one mask value pair m and mx for all intermediate values, additional masking requires 256 byte of RAM for the new masked S-box and two random bytes for the input mask m and the output mask mx . With the expanded key this makes 176 byte + 256 byte + 2 byte = 434 byte.

The precalculation needs 256 XOR-additions to mask the table index, 256 TLUs to read the unmasked S-box values from Read Only Memory (ROM), 256 XOR additions to mask the table entries and, finally, 256 write operations to store the masked table in RAM. This takes $256 + 2 \cdot 256 + 256 + 256 = 1280$ clock cycles for computing the new S-box.

With four pairs of masks, a whole column of the State matrix can be concealed. The advantage is, that during the *MixColumns* transformation (see Sect. B.2.5), which operates on a whole column (four bytes) of the State matrix, all intermediate values are masked with different masks. Therefore, we need eight random bytes and $4 \cdot 256$ byte = 1024 byte for the new lookup tables, which results (with the expanded key) in 176 byte + 8 byte + 1024 byte = 1208 byte of RAM. The precalculation overhead for four different masks leads to $4 \cdot 1280 = 5120$ clock cycles for

²When considering several subsequent operations, a new mask must be applied for each operation

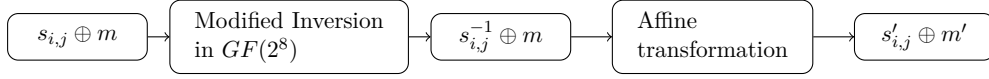


Figure B.2: SubBytes transformation with masking countermeasure

computing the new S-Boxes. During the encryption, we can assume the runtime of the normal *SubBytes* transformation, since we just have to index the correct masked table for the **TLU**.

In addition we have to mask the intermediate value from the State matrix with the input mask before and with the output mask after the transformation. That makes additional two clock cycles per byte of the State matrix and $2 \cdot 160 = 320$ clock cycles for the whole encryption. Summarizing, this scheme needs for all ten rounds $320 + 1280 = 1600$ clock cycles with one masked S-box and $320 + 5120 = 5440$ clock cycles with four masked S-boxes.

To mask a 32-bit T-tables implementation (refer to [DR02]), 32-bit mask values are required, since we now operate on whole rows of the State matrix instead of bytes. Masking the tables with one input and one output 32-bit mask requires eight random bytes for the masks and 8192 bytes **RAM** for the eight masked T-tables (four for the first nine rounds and four for the last round). In total this option requires (with the expanded key) 176 byte + 8 byte + 8192 byte = 8376 byte of memory. The precalculation needs 256 **XOR** additions to mask the 32-bit table index, 256 **TLUs** to read the unmask values from the **ROM** tables, 256 **XOR** additions to mask the table entries and finally 256 write operations to store the masked table 32-bit values in **RAM**. This costs $256 + 2 \cdot 256 + 256 + 256 = 1280$ clock cycles for the precalculation of the masks and in total $520 + 1280 = 1800$ clock cycles for the whole **AES**.

Masking all four columns of the State matrix with the T-tables implementation would require $8 \cdot 4$ bytes = 32 random bytes for the masks and $4 \cdot 8192$ bytes = 32768 bytes for the masked tables. In total that makes with the expanded key 176 bytes + 32 bytes + 32768 bytes = 32972 bytes **RAM**. The precalculation of the tables takes $4 \cdot 1280 = 5120$ clock cycles and in total $520 + 5120 = 5640$ clock cycles for the whole **AES**.

B.2.2 Transformed Masking Method

In [AG01], Akkar and Giraud do not use a lookup table for the *SubBytes* transformation. Instead, they compute its output by means of arithmetic and along the way mask the intermediate values $s_{i,j}$ multiplicatively during the inversion part of the *SubBytes* transformation.

Figure B.2 shows the respective masked version of the *SubBytes* transformation, in which a modified inversion function is used that conserves the mask. We will discuss the realization of this modified inversion in the following.

The idea of Transformed Multiplicative Masking (**TMM**) is to add a new multiplicative mask $Y_{i,j}$ to the intermediate value $s_{i,j}$ and remove the additive mask $M_{i,j}$. The intermediate value is then masked solely with the multiplicative mask. Figure B.3 illustrates on the left side, how the transition between additive and multiplicative masking is achieved. The advantage of the multiplicative mask is, that this mask can be removed after the inversion of the masked intermediate value, by inverting the multiplicative mask $Y_{i,j}$ separately and then multiplying the inverted mask $Y_{i,j}^{-1}$ with the inverted masked value. After the inversion, the additive mask is added again and the multiplicative mask is removed. As result, we obtain the inverse of the intermediate value $s_{i,j}$, additively masked with the mask $M_{i,j}$.

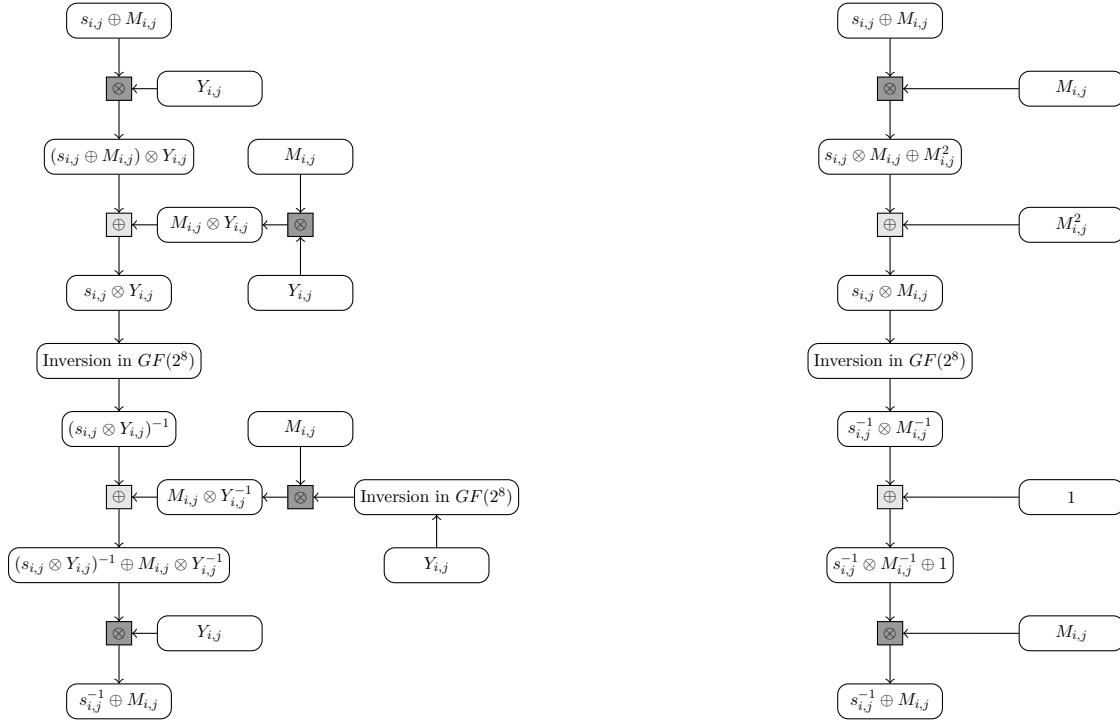


Figure B.3: Securing the AES: Multiplicative inversion in SubBytes (left) following [AG01] and Simplified Multiplicative inversion (right) as described in [TSG02].

Implementation Considerations This countermeasure is very costly since it contains an additional inversion and four multiplications in $GF(2^8)$. A multiplication is marked with a dark gray color in Fig. B.3. If we compute a multiplication with the *binary multiplication method*, as suggested in [FIP01], we need on average 16 shift operations and 8 XOR operations, which adds up to $16+8 = 24$ clock cycles.

Inverting one byte of the State matrix with this method, we need four multiplications, two XOR, and two inversions (TLU). This costs $4 \cdot 24 + 2 \cdot 1 + 2 \cdot 2 = 102$ clock cycles. To invert the $10 \cdot 16 \text{ bytes} = 160$ bytes for all ten rounds, $160 \cdot 102 = 16320$ clock cycles are required. Akkar and Giraud give a cycle count of 293500 clock cycles per encryption, which is about three times more than in the unprotected implementation, which requires 90500 clock cycles.

Simplified Transformed Masking and Implementation Considerations In [TSG02], Trichina et al. have published a simplified variant of the algorithm which is shown in Fig. B.3 on the right hand side. The scheme is equivalent to the original masking scheme, but the authors now use the same mask value for the multiplicative masking ($Y=M$).

For an inversion of one byte from the State matrix with this simplified method, we need only two multiplications, two XORs, and one inversion (TLU). This costs us $2 \cdot 24 + 2 \cdot 1 + 1 \cdot 2 = 52$ clock cycles. To invert the 160 bytes for all ten rounds, we need $160 \cdot 52 = 8320$ clock cycles. The authors in [TSG02] give no information about clock cycles or memory consumption in their paper, so we compare it to the Akkar/Giraud version: The simplified method saves half of the multiplications and inversions during one *SubBytes* transformation, which are the most

expensive and time consuming functions. We hence estimate, that this implementation required half as many clock cycles, as the **TMM** by Akkar/Giraud needs.

Embedded Multiplicative Masking

Both, the normal and the simplified transformed masking method mask only *non-zero values*. If a key byte equals a data byte, then the result of *AddRoundKey* is zero. This fact can be exploited in a special (first-order) differential side-channel attack, the *zero-value attack*. In [OMP04] it is shown, that only around 64 times more measurements are needed in a zero-value attack, compared to a standard **DPA** attack. Thus, zero-value attacks pose a serious practical threat and must be avoided.

One idea for this purpose, by [GT03], is to embed the finite field $GF(2^8)$ with the irreducible polynomial $P(x) = (x^8 + x^4 + x^3 + x + 1)$ into the larger ring $\mathbf{R} = GF(2)[x]/(P(x) \cdot Q(x))$, where $Q(x)$ has degree k (7 should not divide k), is irreducible over $GF(2^k)$, and coprime to $P(x)$. A random mapping ρ , mapping the field (with $P(x)$) to the ring $\mathbf{R}$, is defined as

$$\rho(U) = U + R \cdot P. \quad (\text{B.1})$$

The mask R is a random polynomial $\neq 0$ with a degree smaller than k . We further define a mapping $F : \mathbf{R} \rightarrow \mathbf{R}$ with $F(U) = U^{2^{54}}$, coinciding with the inversion³ in the field. Reducing $F(U)$ modulo $P(x)$ gives the inverse of U in the original field $GF(2^8)$. The additive masking can be restored before performing this final reduction step. If the polynomial R is random, the masking method does not reveal information even if $x = 0$: The zero value gets mapped to one of 2^k random values.

Implementation Considerations Embedded Multiplicative masking is a mathematically elegant way to bypass the zero value problem, but it leads to very costly implementations. According to [BGK04], at least 731 AND and 766 **XOR** operations are required, including the transformations to the new field representation and back. For our cycle count this means that we need at least $731 + 766 = 1497$ clock cycles for one byte of the State matrix and $160 \cdot 1497 = 239520$ clock cycles for all ten rounds.

B.2.3 Perfectly Masking AES Inverter Against First-Order Side-Channel Attacks

Another way to protect the *SubBytes* function is presented in [BGK04]. The authors propose a perfectly masked **AES** by securely computing the multiplicative inverse $s_{i,j}^{-1}$ of a State byte $s_{i,j}$ in $GF(2^8)$ according to

$$s_{i,j}^{-1} = \text{INV}(s_{i,j}) = \begin{cases} s_{i,j}^{-1}, & \text{if } s_{i,j} \in GF(2^8) \\ 0, & \text{if } s_{i,j} = 0 \end{cases} \quad (\text{B.2})$$

and masking all intermediate values. The computation of the inverse $\text{INV}(s_{i,j})$ is performed by means of exponentiation using the **SAM** algorithm, i.e., by exploiting Fermat's theorem: $s_{i,j}^{-1} = s_{i,j}^{2^{54}}$. The **SAM** algorithm and the resulting side-channel leakage are discussed in Sect. 3.2.

³Fermat's Little Theorem

The leakage of the State bytes $s_{i,j}$, the intermediate values during the calculation, and the resulting inverses is prevented as follows. Let r and r' be independently, uniformly distributed random masks, and let u denote a State byte, $u = s_{i,j}$. Starting with an additively masked value $x = u + r$, we want to calculate $y = \text{INV}(u) + r'$. Let us put the SAM algorithm into the given context with the exponent $e = 254$ and the given values:

Algorithm 4 Left-to-Right binary Square-and-Multiply Algorithm

Input: $x = (u + r), n = 255, e = \{e_p, \dots, e_0\}_2 = \{11111110\}_2$

Output: $y = x^e \bmod n = x^{-1} \bmod n$

```

1:  $y \leftarrow 1$ 
2: for  $k = p$  downto 0 do
3:    $y \leftarrow y^2 \bmod n$ 
4:   if  $e_k = 1$  then
5:      $y \leftarrow y \cdot x \bmod n$ 
6:   end if
7: end for

```

The algorithm consists of two main operations, i.e., a squaring in Step 3 followed by a conditional multiplication with a reduction of n in Step 5. These are the operations, which were modified by the authors of [BGK04] to ensure the side-channel resistance.

In the following two algorithms, the input $s_{i,j}$ becomes the additively masked input $u + r_{l,k}$, where r is the l^{th} mask byte in Step k of Alg. 4. Furthermore, the auxiliary values f, v, w and t are needed in the algorithm. The goal is, to invert u .

Algorithm 5 describes the Perfectly Masked Squaring (PMS) method. During the first step, the input $u^e + r_{1,k-1}$ is squared. In Step two and three, the old mask $r_{1,k-1}$ is transformed to the new mask $r_{1,k}$. The desired output is a squared input with a fresh mask $u^{2e} + r_{1,k}$.

Algorithm 5 Perfectly Masked Squaring (PMS), Source: [BGK04]

Input: $x = u^e + r_{1,k-1}, r_{1,k}$

Output: $t_k = u^{2e} + r_{1,k}$

```

1:  $f_k = x^2 \{f_1 = u^{2e} + r_{1,k-1}^2\}$ 
2:  $w_{1,k} = r_{1,k-1}^2 + r_{1,k} \{ \text{auxiliary term to correct } f_k \}$ 
3:  $t_k = f_k + w_{1,k} \{t_k = u^{2e} + r_{1,k}\}$ 

```

The Perfectly Masked Multiplication (PMM) method is illustrated in Alg. 6. The two input values x and x' correspond to the output of the previous step and a freshly masked value, respectively. In Step 1, the authors calculate the product of the two masked values. In Steps 2–5 the auxiliary terms are calculated. The remainder of the algorithm, Step 6 and 7, removes the disturbing parts of the masked product by simply adding up the two auxiliary terms $w_{1,k}$, $w_{2,k}$ and f_k .

Implementation Considerations To calculate the masked inverse of one element with the secured SAM algorithm, we need seven PMS with additional PMM, both including a modular reduction step, and at last one PMS with a modular reduction. Every intermediate value of

Algorithm 6 Perfectly Masked Multiplication (PMM), Source [BGK04]

Input: $x = u^e + r_{1,k-1}, x' = u + r_{2,k}, r_{1,k-1}, r_{1,k}, r_{2,k}$

Output: $t_k = u^{e+1} + r_{1,k}$

- 1: $f_k = x \cdot x' \{f_k = u^{e+1} + u^e \cdot r_{2,k} + u \cdot r_{1,k-1} + r_{1,k-1} \cdot r_{2,k}\}$
 - 2: $v_{1,k} = x' \cdot r_{1,k-1} \{v_{1,k} = u \cdot r_{1,k-1} + r_{1,k-1} \cdot r_{2,k}\}$
 - 3: $v_{2,k} = v_{1,k} + r_{1,k} \{v_{2,k} = u \cdot r_{1,k-1} + r_{1,k-1} \cdot r_{2,k} + r_{1,k}\}$
 - 4: $w_{1,k} = v_{2,k} + r_{1,k-1} \cdot r_{2,k} \{w_{1,k} = u \cdot r_{1,k-1} + r_{1,k}\}$
 - 5: $w_{2,k} = x \cdot r_{2,k} \{w_{2,k} = u^e \cdot r_{2,k} + r_{1,k-1} \cdot r_{2,k}\}$
 - 6: $t_{1,k} = f_k + w_{1,k} \{t_{1,k} = u^{e+1} + u^e \cdot r_{2,k} + r_{1,k-1} \cdot r_{2,k} + r_{1,k}\}$
 - 7: $t_k = t_{1,k} + w_{2,k} \{t_k = u^{e+1} + r_{1,k}\}$
-

the **PMS** and **PMM** is hidden by a fresh mask. So we need $(7 \cdot 3) + 1 = 22$ random masks for the inversion of one element of the State matrix. For a whole AES encryption or decryption we need $160 \cdot 22 = 3520$ random mask values each with the size of one byte. In the case of a first order side channel resistance, the authors point out in [BGK04, Section 5.4], that the number of random masks can be reduced to only three masks (r_1, r_2, r_3), by using the same masks in each step k .

To calculate the clock cycles for the inversion, we recall the clock cycle required for an addition (1 clock cycle), a multiplication (241 clock cycles), and a squaring (21 clock cycles). By counting the operations during the algorithm we get: $2 \cdot 2 + 2 = 6$ clock cycles (two squaring and two additions), for one **PMS**, and $4 \cdot 24 + 2 \cdot 1 = 98$ clock cycles (four multiplications and four additions) for one **PMM** operation. Hence, in sum the method needs

- $7 \cdot 6 = 42$ clock cycles for the **PMS** operation,
- $7 \cdot 98 = 686$ clock cycles for the **PMM** operation,
- 3 clock cycles for loading the random intermediate mask $r_{1,k}, r_{1,k-1}, r_{2,k}$,
- 1 clock cycle for loading the masked intermediate value from **RAM**, and
- 1 clock cycle for saving the result.

In total, $42 + 686 + 3 + 1 + 1 = 731$ clock cycles are needed for the inversion of one element of the State matrix. For a whole **AES** encryption or decryption we need $160 \cdot 731 = 117280$ clock cycles for the inversion part of the *SubBytes* transformation with three masks.

B.2.4 Combined Masking in Tower Fields

In [OMP04] and [OMPR05], Oswald et al. present a masking scheme called Combined Masking in Tower Fields (**CMTF**) which is based on composite field arithmetic. The idea is, that the inversion in $GF(2^2)$ is a linear transformation, which gives us the potential to track the changes which are applied to an additive mask m during an inversion.

To securely calculate the inversion of the current intermediate byte $s_{i,j}$ of the State matrix, we first mask it additively with m to obtain $a = s_{i,j} \oplus m$. Next we transform a to its composite

field representation: Every element of $GF(2^8)$ can be represented as a linear polynomial $a_h x + a_l$ with coefficients a_h and a_l out of $GF(2^4)$. The algorithm operates over the finite fields

$$GF(2^8) \simeq GF(2)[x]/(x^8+x^4+x^3+x+1) \text{ and} \quad (\text{B.3})$$

$$GF(2^4) \simeq GF(2)[x]/(x^4+x+1). \quad (\text{B.4})$$

Note that the operations $\oplus$ and $\otimes$ in the following denote operations with coefficients from $GF(2^4)$.

After the transformation, the masked value to be inverted is represented by $(a_h \oplus m_h)x + (a_l \oplus m_l)$ with $a_h, a_l, m_h, m_l \in GF(2^4)$. Both values representing a , i.e., a_h and a_l , are now additively masked with the masks m_h and m_l . As a preparation for understanding the masked inversion, we first recall the normal inversion in the composite field:

$$(a_h x \oplus a_l)^{-1} = a'_h x \oplus a'_l \quad (\text{B.5a})$$

$$a'_h = a_h \otimes d^{-1} \quad (\text{B.5b})$$

$$a'_l = (a_h \oplus a_l) \otimes d^{-1} \quad (\text{B.5c})$$

$$d = (a_h^2 \otimes \{E\}_{16}) \oplus (a_h \otimes a_l) \oplus a_l^2 \quad (\text{B.5d})$$

The masked inverse is computed following the steps of (B.5a) to (B.5d), but this time the operations are applied to the masked values:

$$((a_h \oplus m_h)x + (a_l \oplus m_l))^{-1} = (a'_h \oplus m'_h)x + (a'_l \oplus m'_l) \quad (\text{B.6a})$$

$$a'_h \oplus m'_h = a_h \otimes d^{-1} \oplus m'_h \quad (\text{B.6b})$$

$$a'_l \oplus m'_l = (a_h \oplus a_l) \otimes d^{-1} \oplus m'_l \quad (\text{B.6c})$$

$$d \oplus m_h = a_h^2 \otimes \{E\}_{16} \oplus a_h \otimes a_l \oplus a_l^2 \oplus m_h \quad (\text{B.6d})$$

Next, we need to securely calculate the inverse of $(d \oplus m_h)$, with $d, m_h \in GF(2^4)$. So far, we have only shifted the problem from the bigger field down to a smaller one. Because $GF(2^8)$ is a tower field over $GF(2^2)$, we can further shift the computation of the masked inversion for d down to $GF(2^2)$ as in (B.6a) to (B.6d), with the corresponding field operations. In $GF(2^2)$, the inversion is a linear operation. Hence, the additive mask can remain and can be easily removed after the inversion: The inversion operation preserves the mask, because $(d \oplus m)^{-1} = (d \oplus m)^2 = d^2 \oplus m^2$ in $GF(2^2)$.

This approach has been presented in [OMPR05] for tower fields that can be efficiently realized in hardware implementations, but are extremely slow in software, e.g., on a 32-bit processor: During the field transformations every individual bit of all bytes in the State matrix has to be processed — this is very costly on a μC . Instead of computing all steps according to the equations, all above transformations above can be precomputed and stored in tables. This method will be discussed in the next subsection.

Combined Masking in Tower Fields with Precomputed Tables

To compute the masked inversion in $GF(2^8)$, Schramm et al. presented a method in [OS05b] which is based on the inversion in composite fields, as discussed it in the last section. The

difference is, that the calculations of the intermediate values are mapped to a sequence of **TLUs**.

The authors first map the masked intermediate value $a = s_{i,j} \oplus m$ down to its field representation with coefficients from $GF(2^4)$ as explained before. Next, the masked intermediate values of (B.6b) to (B.5d) are calculated using the four tables

$$\begin{aligned} T_{d_1} : ((x \oplus m), m) &\mapsto x^2 \times \{\mathbf{E}\}_{16} \oplus m, \\ T_{d_2} : ((x \oplus m), (y \oplus m')) &\mapsto ((x \oplus m) \oplus (y \oplus m')) \times (y \oplus m'), \\ T_m : ((x \oplus m), (y \oplus m')) &\mapsto (x \oplus m) \times (y \oplus m'), \text{ and} \\ T_{inv} : ((x \oplus m), m) &\mapsto x^{-1} \oplus m, \end{aligned}$$

which can be precalculated and stored in **ROM**. Note, that all tables take two elements of $GF(2^4)$ as inputs and give an element of $GF(2^4)$ as output. With

$$\begin{aligned} d \oplus m_h &= T_{d_1}(a_h \oplus m_h, m_h) \oplus T_{d_2}((a_h \oplus m_h), (a_l \oplus m_l)) \\ &\quad \oplus T_m((a_h \oplus m_h), m_l) \oplus T_m((a_l \oplus m_l), m_h) \oplus T_m((m_h \oplus m_l), m_l) \end{aligned} \quad (\text{B.7})$$

we compute the masked inversion of d with the following table lookup:

$$d^{-1} \oplus m_h = T_{inv}(d \oplus m_h, m_h). \quad (\text{B.8})$$

Now we can compute $a'_h \oplus m'_h$ and $a'_l \oplus m'_l$ from (B.6b) and (B.6c) in the following way:

$$\begin{aligned} a'_h \oplus m'_h &= T_m(a_h \oplus m_h, d^{-1} \oplus m_l) \\ &\quad \oplus m_h \oplus T_m(d^{-1} \oplus m_l, m_h) \oplus T_m(a_h \oplus m_h, m_l) \oplus T_m(m_h, m_l), \end{aligned} \quad (\text{B.9})$$

and

$$\begin{aligned} a'_l \oplus m'_l &= T_m((a_l \oplus m_l), (d^{-1} \oplus m_h)) \\ &\quad \oplus m_l \oplus T_m(d^{-1} \oplus m_h, m_l) \oplus T_m(a_l \oplus m_l, m_h) \oplus f_{a_h} \oplus m_h \oplus T_m(m_h, m_l). \end{aligned} \quad (\text{B.10})$$

To map $GF(2^8)$ elements to $GF(2^4) \times GF(2^4)$ elements and vice versa, two additional tables with a size of 512 bytes are needed.

Implementation Considerations As the authors state (and as can be counted from the above equations), the total cost of performing a masked inversion this way corresponds to 14 **TLU** operations and 15 **XOR** operations for an inversion, resulting in $14 \cdot 2 + 15 = 43$ clock cycles. For a complete **AES** encryption this implies $160 \cdot 43 = 6880$ additional clock cycles for the new masked *SubBytes* transformations. To store the six tables we need $4 \cdot 256 \text{ bytes} + 2 \cdot 256 \text{ bytes} = 1536 \text{ bytes}$ of **ROM**.

B.2.5 Masking the MixColumns Transformation

The *MixColumns* transformation can leak timing information, if it is implemented using the *xtimes* operation. *Xtimes* contains a conditional reduction by $\{\mathbf{11B}\}_{16}$, if the result of the operation is larger than eight bits.

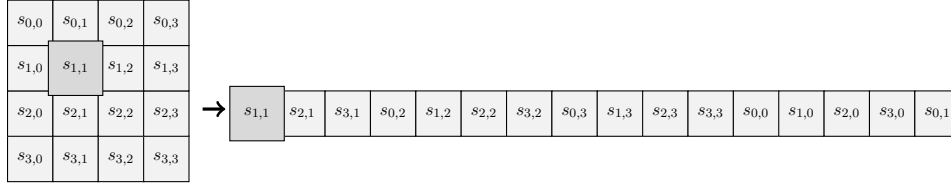


Figure B.4: Shuffled AES-State

To prevent both timing information leakage and power leakage, the State $\mathbf{S}$ has to be additively masked with the mask $\mathbf{M}$ before the transformation, to obtain $\mathbf{S}_{\mathbf{M}} = \mathbf{S} \oplus \mathbf{M}$, where $\mathbf{M}$ is a 4×4 matrix created of four random bytes $M_1, \dots, M_4$, see Eq. (B.11). After the transformation $\mathbf{S}'_{\mathbf{M}} = \text{MixColumns}(\mathbf{S}_{\mathbf{M}})$, the mask $\mathbf{M}$ can be removed from the transformed State $\mathbf{S}'_{\mathbf{M}}$ by adding the transformed mask $\mathbf{M}' = \text{MixColumns}(\mathbf{M})$, i.e., $\mathbf{S}' = \mathbf{S}'_{\mathbf{M}} \oplus \mathbf{M}'$.

$$\mathbf{M} = \begin{pmatrix} M_1 & M_2 & M_3 & M_4 \\ M_1 & M_2 & M_3 & M_4 \\ M_1 & M_2 & M_3 & M_4 \\ M_1 & M_2 & M_3 & M_4 \end{pmatrix} \quad (\text{B.11})$$

Masking the *MixColumns* transformation needs one additional *MixColumns* transformation (84 clock cycles) to obtain $\mathbf{M}'$, if the same mask $\mathbf{M}$ is used to mask all *MixColumns* transformations during a whole AES run. In addition, four random bytes are needed for the mask $\mathbf{M}$, and 16 bytes are needed for the transformed mask $\mathbf{M}' = \text{MixColumns}(\mathbf{M})$.

B.3 Countermeasure: Randomization

Randomizing the execution of the algorithm provides additional resistance against power analysis attacks. Herbst et al. suggest in [HOM06a] two ways to introduce randomness in the execution of the algorithm. They propose to shuffle the sequence of operations which work on the elements of the State matrix and to add dummy rounds (or parts of a round) during the normal ones.

B.3.1 Shuffling

For the shuffling approach, illustrated in Fig. B.4, we first randomly choose a starting column in the state and in that column a random element. Then, we proceed with the next element and all remaining elements of the state in the natural order. As we work on each intermediate value of the State matrix, this version of shuffling is applicable on 8-bit implementations. With shuffling, the probability that one specific intermediate value is processed at a specific point in time is $1/16$. Theoretically, we can shuffle the whole State matrix with $16!$ possibilities. However, this would not increase the security against a first-order DPA with the same factor, since for DPA-resistance it is important that the operations with the targeted intermediate values do not occur at the same point in time. For a successful side-channel attack, 16 times more traces are needed with this countermeasure than with an unprotected implementation.

d	d	$s_{0,0}$	$s_{1,0}$	$s_{2,0}$	$s_{3,0}$	$s_{0,1}$	$s_{1,1}$	$s_{2,1}$	$s_{3,1}$	$s_{0,2}$	$s_{1,2}$	$s_{2,2}$	$s_{3,2}$	$s_{0,3}$	$s_{1,3}$	$s_{2,3}$	$s_{3,3}$	d	d	d
-----	-----	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----	-----	-----

Figure B.5: AES-State with dummy values

$\dots$	d	$s_{1,1}$	$s_{2,1}$	$s_{3,1}$	$s_{0,1}$	$s_{2,2}$	$s_{3,2}$	$s_{0,2}$	$s_{1,2}$	$s_{0,3}$	$s_{1,3}$	$s_{2,3}$	$s_{3,3}$	$s_{2,0}$	$s_{3,0}$	$s_{0,0}$	$s_{1,0}$	d	d	$\dots$
---------	-----	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----------	-----	-----	---------

Figure B.6: AES-State with dummy values and shuffled AES-State

The randomization can be broken with windowing, published by Clavier et al. [CCD00b]: Windowing can be seen as performing multiple second-order DPA attacks in parallel and then combining the power traces of processing the mask with each of the points in time, where the targeted masked values occur.

B.3.2 Execution of Dummy Operations

Another way to increase the randomization is to add dummy intermediate values d to the (enlarged) State matrix. Those operations must not be easily distinguishable from real operations of the algorithm. For example, added wait states or added random No Operations (NOPs) can be detected and removed by analyzing a single power trace. The authors of [HOM06a] therefore suggest to insert dummy rounds or parts of a round in the form of dummy State bytes.

The number of added dummy operations D is a fixed value. A random number r , $0 \leq r \leq D$ partitions the D dummy operations into two parts. The dummy operations from 0 to $r - 1$ will be executed before and the dummy operations from r to $D - 1$ will be executed after the normal AES operations, as depicted in Fig. B.5 for $r = 2$ and $D = 5$.

B.3.3 Combining Shuffling and Dummy Operations

The combination of shuffling and dummy operations makes it more difficult for an attacker to locate the attack point in the first two and the last two rounds. These are the rounds requiring the strongest protection, because they are most vulnerable during a typical known (or chosen) plaintext/ciphertext attack. After these rounds, every value from the State has been subject to three *AddRoundKey* transformations and depends on a sufficient amount of key bytes to hinder a DPA attack [HOM06a]. The probability, that a certain intermediate value occurs at a specific point in time is $P = 1/(D + 1)$, where D is the number of additional dummy cycles. If we chose to insert 15 dummy operations, the probability is $P = 1/16$, which would increase the number of traces needed for a successful SCA by a factor of 16. Note that we have to see the normal consecutive State bytes as *one* value compared to the dummy bytes, to calculate this probability.

With D additional dummy rounds and shuffling, which is recommended by the authors, the number of required traces for a successful SCA attack increases by $(D + 1) \cdot 16$. With, e.g., $D = 5$ dummy rounds randomly inserted at the start and the end of the algorithm, 96 times more measurements would be required for a successful DPA attack.

Figure B.6 depicts the resulting shuffled execution of the State matrix, combined with added dummy bytes d . Table B.1 denotes how many random bits are required for the shuffling and the

dummy cycles countermeasures. A low-cost SCA-resistant AES implementation, which takes

Algorithm	No. random bits	Usage
shuffling	4	defines the entry point in State matrix
dummy operations	$\lceil \log_2(D) \rceil$	number of dummy operations before/after actual operation

Table B.1: Random values required for an implementation that is secured with dummy rounds and/or shuffling.

advantage of masking, shuffling, and execution of dummy cycles is presented in [THM07].

B.4 Generating Random Numbers for the Countermeasures

Generating high quality random numbers for the masks is very important for the security of all masking schemes, likewise time randomization countermeasures require reliable randomness. If the distribution of the random numbers has a bias with respect to the uniform distribution, a masked value leaks information [CGPR08] and a hiding countermeasure (randomization) becomes ineffective.

The German Federal Office for Information Security Bundesamt für Sicherheit in der Informationstechnik (BSI) describes four classes indicating the quality of deterministic RNGs in [BSI01]. From the “weak” class K1, the random numbers of which contain no identical consecutive elements with a high probability, to the “strong” class K4, where it is considered impossible for an attacker to calculate or guess from an inner state of the generator any previous numbers in the sequence of random numbers.

A K1 RNG is usually the appropriate choice for shuffling and for the dummy states, because the quality of the used random numbers in this case is not so critical. For masking intermediate values, the RNG should at least fulfill the requirements of a K2 RNG. This class ensures, that the random numbers pass statistical tests and are uniformly distributed. For generating those qualitative good random values, e.g., a well-seeded Grain-80 stream cipher [HJM05] suffices.

B.5 Comparison

The implementation costs of the previously discussed (low-cost) software countermeasures against power-analysis attacks are now summarized. The figures include 6560 clock cycles for the affine transformation, for those schemes which have to calculate it. We obtain the following runtime estimations for implementing the introduced schemes on a 32-bit μC :

- Additive Masking: $1732 + 1600 = 3332$ clock cycles
- Transformed Masking: $1732 + 6560 + 16320 = 24612$ clock cycles
- Simple Transformed Masking: $1732 + 6560 + 8320 = 16612$ clock cycles
- Embedded Masking: $1732 + 6560 + 239520 = 247812$ clock cycles
- Perfect Masking: $1732 + 6560 + 117280 = 125572$ clock cycles

■ **CMTF** with **TLUs**: $1732 + 6560 + 6880 = 15172$ clock cycles

Table B.2 contains the estimated memory consumption (**ROM** and **RAM**), the number of required random bits, and the runtime in clock cycles (cc) for the discussed masking methods. Note that we are only interested in a rough estimate to compare the countermeasures. The value on the left side denotes the estimated cost for using one random byte to mask all 16 elements of the State. The value on the right side is the estimated value using four random bytes to mask a whole column of the State matrix. All **RAM** estimates include the 176-byte expanded key array for the key scheduling.

With 3332 clock cycles and 434 byte of **RAM**, Additive Masking is efficient when used with one mask. With four different masks the **RAM** consumption of 1208 byte, which is mainly caused by the four masked S-boxes, becomes extremely big, while the runtime of $5440 + 1732 = 7172$ clock cycles is still acceptable. Both for an additively masked 32-bit T-tables implementation with one mask pair and for four mask pairs, the **RAM** consumption of 8376 byte and 32976 byte, respectively, is too big.

The Transformed Masking and the simplified Transformed Masking Method multiplicatively mask an element of the State matrix during the *SubBytes* transformation. The simplified variant is very interesting with respect to the **RAM** consumption of 180 byte. Its estimated runtime of 16612 clock cycles is still acceptable. An advantage of the **TMM** is its constant runtime, independent of the number of masks.

The Perfect Masking in line six needs 179 byte **RAM** (three for the masks and 176 for the expanded key). Since it calculates the inversion and the affine transformation on the fly, the scheme is very slow.

No.	Masking Method	ROM (bytes)	RAM (bytes)	Encryption (clock cycles)	No. of random bits
1	Unmasked	256	176 / 176	1732 / 1732	0 / 0
2	Additive Masking	256	434 / 1208	3332 / 7172	16 / 64
3	Transformed Masking	0	178 / 204	24612 / 24612	16 / 64
4	Simple Transf. Masking	0	177 / 180	16612 / 16612	8 / 32
5	Embedded Masking	0	177 / 180	247812 / 247812	8 / 32
6	Perfect Masking	0	179 / 179	125572 / 125572	3 / 12
7	CMTF with TL	1536	177 / 180	15172 / 15172	8 / 32
8	T-tables, unmasked	8192	176 / 176	520 / 520	64 / 64
9	T-tables, Add. Masking	8192	8376 / 32976	1800 / 5640	256 / 256

Table B.2: Estimated memory consumption and cycle count for different AES implementations with masking countermeasure for using one mask / four masks.

The same constant runtime characteristic as for **TMM** holds for the Combined Masking in Tower Fields (**CMTF**) in row seven. The big **ROM** consumption of 1536 bytes mainly results from a couple of tables, which are used for the transformation between the different representation systems and for calculating the masked inversion, see Sect. B.2.4.

The big part of the runtime from the Additive Masking Method arises from the creation of the masked S-boxes. If these masked S-boxes have been created, the implementation is even

as fast as the unmasked implementation since then, the transformation is only a table lookup. Transformed Masking and the improved versions of it would only pay off if a high number of random masks is used.

An open question that is not discussed here is the amount of required masks and how often a new mask needs to be computed, e.g., in between processing each individual byte of the State, in each round of the AES, after every three rounds, or after a full encryption. Note that using more masks does not automatically imply a practically more secure implementation: A higher order DPA works whenever the same mask values occur in two different points in time. Since all masks have to be created at a specific point in time and are used to mask an intermediate value at some point later, there are always two points in time that allow a higher order DPA attack.

Appendix C

125 kHz RFID: A Customized Reader and Details about Commercial Transponders

Immobilizers relying on **RFID** transponders operating at a frequency of 125 kHz are the dominant anti-theft solution in cars. For analyzing the security of these and other systems (access control, animal identification, ...), in the following a customized **RFID** reader to support basically all transponder protocols at 125 kHz and a corresponding firmware are developed. This chapter has emerged from the Bachelor's thesis of Vincent Immler, advised by myself and Christof Paar. The physical layer for communicating with 125 kHz transponders is introduced in Sect. C.1 and the customized reader realizing the protocols is detailed in Sect. C.2.

C.1 125 kHz Transponders

Transponders operating at 125 kHz are used for non-secured applications such as animal tracking, but are also widely employed in car immobilizers that realize cryptographic schemes. This section details the communication on the physical layer and the general capabilities of this type of tags.

C.1.1 Physical Layer

RFIDs at a frequency of $f_c = 125$ kHz operate similarly to those at $f_c = 13.56$ MHz as introduced in Sect. 2.5. The coils for the establishing the coupling at 125 kHz usually have more windings, i.e., typically more than 50 windings. Reader and tag can exchange data as described next.

Communication Transponder → Reader For reading from the tag, load modulation as detailed in Sect. 2.5.3 is employed. In contrast to contactless smartcards, all 125 kHz **RFID** protocols have in common that they do not use subcarriers. The two most common codes to transfer data from the transponder to the reader are the Manchester code described in Sect. 2.5.3 and the bi-phase code: In the bi-phase coding scheme, a logic '1' is encoded by transitions from either high to low or vice versa at the beginning of a bit period. A binary '0' is encoded by a transition of either type in the half bit period (compare with the Miller code described in Sect. 2.5.3).

Communication Reader → Transponder Depending on the actual system different types of modulation are used to transfer data from the reader to the transponder, e.g., **ASK**, Phase Shift Keying (**PSK**) or Frequency Shift Keying (**FSK**). A survey of several 125 kHz **RFID** protocols

testified, that the most commonly used modulation is 100 % **ASK**, i.e., the presence or absence of the **RF** field. A major problem of 100 % **ASK** is the tag suffering a power-loss every time the field is switched off. Thus, coding schemes providing a relatively continuous power supply to the transponder, such as Pulse Pause Coding (**PPC**), are usually employed.

In **PPC**, a binary one is represented by a short gap in the field of duration t_{gap} , followed by a field-on state of duration t_1 . A binary zero is represented similarly, that is a gap of duration t_{gap} followed by a different field-on state of duration t_0 . For demodulating the signal the durations of t_0 and t_1 between two gaps, where $t_{gap} \ll t_1$ and $t_{gap} \ll t_1$, need to be known. For implementing the **PPC**, the following timing parameters have been found by experimenting:

Table C.1: Values for the PPC.

PPC	value in μs
t_0	128
t_1	180
t_{gap}	44

C.1.2 Research on 125 kHz Devices

Cryptographic **RFID** systems operating at 125 kHz have not yet been widely researched compared to those operating at 13.56 MHz, hence only few information has been published so far. Most of the protocols and ciphers employed are proprietary and have not been disclosed to date. One exception is the HITAG 2 cipher and protocol by NXP that is widely used in car immobilizers. The proprietary, bit-oriented stream cipher is based (in its simplest variant) on a 48-bit **LFSR** and is well-suited for small hardware implementations.

Nicolas T. Courtois, Sean O’Neil and Jean-Jacques Quisquater are the authors of “Practical Algebraic Attacks on the HITAG2 Stream Cipher” [COQ09]. The paper deals with cryptanalyzing the HITAG2 cipher by converting a description of that cipher to many different equations and solving them by a SAT solver. The authors conclude that a key-recovery attack can be executed on a modern computer within a few hours. The cipher itself was firstly presented by O’Neil and Courtois in 2008 [CO08]. A more general approach in breaking **RFID** protocols is presented by Andrey Bogdanov and Christof Paar, who describe different types of protocol and dictionary attacks in their paper “On the Security and Efficiency of Real-World Lightweight Authentication Protocols” [BP08], with a focus on ciphers and protocols used in car immobilizers. Furthermore, Rajesh Bachani analyzes **TMTO** attacks on the HITAG2 stream cipher in his Master’s thesis [Bac08]. The author concludes that — even though the speed of breaking the cipher is considerably fast — there is a high uncertainty that enough keystream can be acquired for these attacks in a practical attack scenario. A detailed description of the most common protocols for 125 kHz, the HITAG 2 cipher, and various 125 kHz transponders employed for car immobilizers can be found in Sect. C.4.

C.2 A Customized Reader for 125 kHz

The module for communicating with RFID tags operating in the range of 100 kHz to 150 kHz is designed similar to the ISO 14443 module from Sect. 4.1. Providing an adequate trigger signal for starting measurements and giving full control over the protocol, timings, RF field, and debug output (in contrast to a commercial reader) the low-cost reader board easily integrates in the measurement environment and, e.g., allows to perform protocol attacks and assist side-channel analyses of 125 kHz transponders.

The core of the developed hardware consists of an Atmel U2270B IC realizing the RF interface (price: approx. 1 \$) and a standard Atmel ATmega32 8-bit μ C. The latter runs the firmware for controlling the RF chip, de- and encoding the data on the physical layer, and communicating with the PC via USB. In the following, the most relevant functional units of the reader are illustrated.

C.2.1 Outline of the reader

The RF reader IC Atmel U2270B provides the analogue front-end for communicating via the RF interface¹. It supports a carrier frequency f_c between 100 kHz and 150 kHz and a typical data rate of up to 5 kbaud at 125 kHz, while the received data may be encoded according to Manchester or Bi-phase modulation [Atmg]. The auxiliary circuitry on the developed PCB is based on the schematic given in Application 3 in the datasheet of the U2270B [Atm08].

For sending data, the Carrier Frequency Enable (CFE) pin turns the RF field on and off according to the input level at the pin (high=on, low=off). The μ C encodes the information to be sent with Non-Return-to-Zero (NRZ), Miller code, or PPC as described in Sect. C.1.1, and controls the CFE pin to transmit the bitstreams. The two independent output stages of the driver operate in differential mode, i.e., the voltages are in anti-phase, implying that the antenna coil is driven at a higher voltage compared to a synchronous mode. An oscillator control loop incorporating a phase detector [Atmf] adapts the oscillator frequency to the resonance of the reader's antenna and thereby improves the sensitivity and operating range of the reader. As a result, a high coil voltage of approx. 70 V (from a 5 V supply voltage) was measured using a moderately tuned antenna.

The antenna coil with an inductance $L = 1.6$ mH and a capacitor $C = 1$ nF that is connected in parallel form a resonant circuit with a frequency of $f = 1/2\pi\sqrt{L\cdot C} = 125.8$ kHz. For matching the circuit to different operating frequencies and/or antenna inductances, an additional (parallel) adjustable capacitor $C_t = 180$ pF allows to tune the resonance frequency by $\pm 20\%$.

The received signal is demodulated in a first step by means of an external envelope detector circuit that performs a standard amplitude demodulation by means of a diode and an RC-circuit. The preprocessed signal is fed via a high-pass capacitor with a value of 680 pF, which is according to [Atmf] optimal for a data rate of $f/32$, into the input pin of the U2270B IC. The latter features a fully integrated low-pass filter (4th-order Butterworth) with a cut-off frequency of 7 kHz that removes the remaining carrier signal and high-frequency noise. After passing an internal Schmitt Trigger the acquired logical state of the RF field, i.e., zero or one, is available for further processing by the microcontroller at the output pin of the RF chip.

¹This RF front-end is the counterpart to the EM 4094 chip used in the custom-built reader for contactless smartcards.

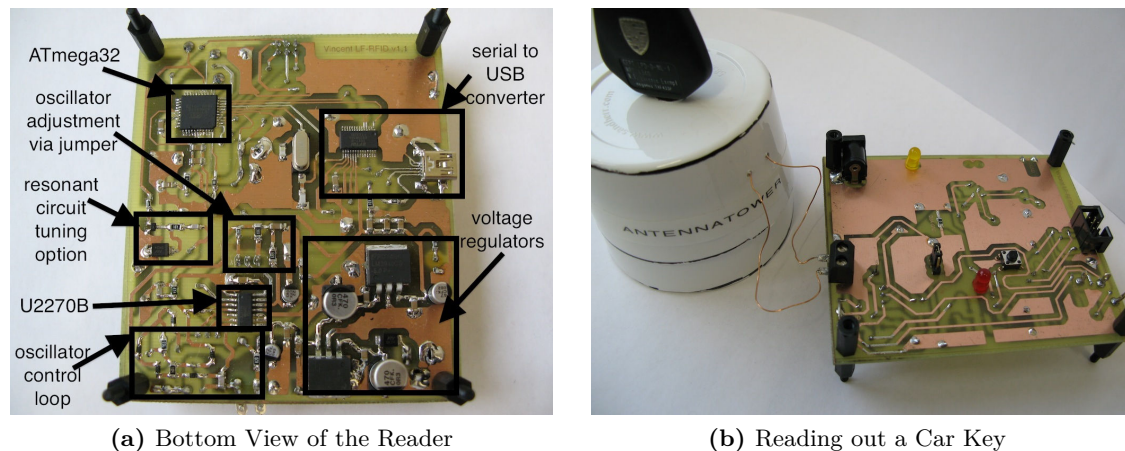


Figure C.1: The customized 125 kHz RFID reader with highlighted functional units (left), and with attached antenna and a car key in a fixed read position (right).

Figure C.1a illustrates the developed reader, the schematics of which can be found in Fig. J.3. The developed basic firmware implemented on the μC is similar to the generic reader mode described in Sect. G.2.6, i.e., all commands to be sent are prepared by the controlling PC, that also receives the response of the tag and, after performing the relevant computations, generates the next command to be sent. For testing purposes, the HITAG2 protocol and a protocol for read-only transponders has been realized on the reader. For implementing the PPC, setting the timing parameters $t_0 = 128 \mu\text{s}$, $t_1 = 180 \mu\text{s}$, and $t_{\text{gap}} = 44 \mu\text{s}$ allowed for successful communication with HITAG2 transponders, as employed in various car immobilizers. Since the reader is fully controlled by means of serial communication, it can be steered from any other program and hence easily integrates in the existing control framework.

C.3 Overview of Common 125 kHz Transponders

Even though the market of 125 kHz transponders seems crowded at first sight (because of many different manufacturers), using the developed reader it was possible to reduce the available solutions to only a few protocols. In the following, an overview of the most widespread solutions is given, with a special focus on the automotive industry.

RFIDs in the Automotive Industry

The automotive industry uses RFID transponders mainly in car immobilizers. At first, the immobilizer was a separate transponder that was included in each car key. Most of these were classic keys without the ability to open cars remotely. The next step were RKE remotes that in addition to the immobilizer allow to open cars remotely, and finally “Keyless Go” tokens that allow to open a car automatically when approaching it. These new remotes and tokens still incorporate the functions of the classic immobilizer and often use just one chip for both the RF and UHF communication.

The most important manufacturers that offer solutions in the area of immobilizers are:

- **NXP Semiconductors/Philips:** NXP was established in 2006 and is a former division of Philips. They offer a whole range of transponder products that are presented further below in detail. Any of their low frequency **RFID** products operate at 125 kHz. In 1994, the manufacturing of immobilizers was started [Gie06].
- **Texas Instruments (TI):** Texas Instruments also has a broad product portfolio in the **RFID** area. They started their first immobilizer product in 1993. Nowadays, there immobilizers can be found in many cars [TI].
- **EM Microelectronic/Sokymat Automotive:** EM Microelectronic is a semiconductor manufacturer specialized in the design and production of low power and low voltage integrated circuits. They also offer a broad range of **RFID** products. In 2003, they acquired the Sokymat IDent Component GmbH (SID) [Miel]. In 2008 this division of EM Microelectronic changed its name to Sokymat Automotive GmbH and was sold to the SMARTRAC Technology GmbH. They still offer immobilizer solutions and are together with NXP and TI, one of the three leading immobilizer **IC** suppliers [WO].
- **Delphi Automotive/Megamos:** Delphi is an automotive parts company that acquired Megamos (originally f+g Megamos) in 1997. Delphi offers immobilizer units, but still (partially) uses Megamos reader ICs and transponders.
- **Atmel/Temic:** Temic once offered immobilizer transponders, too. However they have been acquired by Atmel and it seems that no current car originally incorporates Atmel's **RFID** products — still the transponders can be used as replacements for the originals.

All car manufacturers charge high fees for ordering new keys. Because of this, there are manufacturers of mechanical keys that also offer key cloning devices. Two large manufacturers in this area are JMA and SILCA. Both of them also offer (blank) transponders to emulate the original transponders of the manufacturers mentioned above. Their product catalogues give a good overview of the market share of the different immobilizer solutions. Based on their catalogues [JMA, BBL], one can conclude that the market is shared between Philips/NXP, Texas Instruments, and Megamos/Sokymat, while other manufacturers only have a negligible role.

Identification of Immobilizer Transponders

When it comes to immobilizer transponders, one can be easily confused by many different naming schemes. This section intends to provide the reader with a good understanding of these schemes. The most common and distinct naming scheme seems to be based on SILCA's RW4 Plus transponder copy machine [SIL]. This device assigns **IDs** to the different protocols. However, one very often encounters different names for the same product. As an example, we have a look at two very common transponders types.

HITAG 2 transponders: HITAG 2 is the name of the protocol used by the transponder and is identified as ID46 by SILCA's RW4. The original transponder from NXP is of the type PCF7936 (or same product family, see Sect. C.3 for more details). Nevertheless, SILCA and

JMA offer compatible transponder types, too. In case of SILCA, this transponder is called T14 and in case of JMA it is called TP12. Still, one will in most cases find the more generic name “Philips 2nd Generation Crypto Code” for this type of transponder.

Sokymat Magic transponders: This transponder is identified as ID48 by SILCA’s RW4 — here the original transponder type is not known. In case of SILCA, the compatible transponder offered is T6 and in case of JMA it is called TP08. Most of the time, one will find the more generic name “Megamos Crypto” for this transponder, even though it is (to the author’s best knowledge) probably identical to Sokymat’s Magic transponder. We will see later, that the protocol is in fact the same as that used by EM4170 of EM Microelectronic.

Overview of the PCF Transponder Family

As NXP offers a huge variety of transponders, this section intends to organize their transponder product family, as information on this is very hard to find (most likely because of security by obscurity reasons).

At first, one has to distinguish protocols from transponders because many transponders that exist share the same protocol but are distinguishable by other features (e.g. form factor, additional functions, ...). By evaluating NXP’s portfolio and former Philip’s products of 125 kHz transponders, one is able to identify two main product branches:

- Logistics and animal identification:

- HITAG 1
- HITAG S
- HITAG μ

- Automotive:

- Immobilization:
 - * PIT
 - * SECT
 - * HITAG 2
 - * HITAG Pro

The different names represent both the product name and the protocol involved. A major difference between logistics products and any automotive product is, that all of the automotive products lack anti-collision (a technique to read multiple transponders at the same time), as this feature is not necessary for immobilization applications.

According to the classification that has been presented in Sect. 2.3, the different transponders are assigned to the different generations and types of protocols. All of the immobilizer transponders can be found in real cars, as the analysis of various car keys has shown, see Fig. C.16.

PIT Protocol-based Transponders

PCF7931 This 1st generation type transponder implements the read capabilities of the PIT protocol. As this transponder is **OTP**, there is no option of using a synchronization code scheme.

PCF7930 This transponder is a 2nd generation type transponder and uses the PIT protocol. It is compatible to the 1st generation protocol and offers the option of using a synchronization code scheme. Because of this, almost all car manufacturers have implemented their own scheme [NXPb].

SECT Protocol-based Transponders

PCF7935 This transponder is a 4th generation type transponder and is compatible to the PCF7931 and PCF7930. It implements the SECT protocol which is a simple challenge response protocol, using a not specified cipher [NXPd].

HITAG2 Protocol-based Transponders

HITAG2 is a 5th generation protocol that can be found in many transponders. Even though many variants of this type exist, they are distinguishable by their UHF capabilities as required for RKE and keyless-go applications.

PCF7936 This is the first HITAG2 transponder and does not have any UHF capabilities. Hence, it is used solely for immobilizers [NXPg].

PCF7941 This transponder includes UHF capabilities and a RISC Controller with a 4kB ROM that is used to program keyless entry features. Transponders of this type are also termed “STARC lite” (Security Transponder and RISC Controller).

PCF7942/43/44 This transponder includes UHF capabilities and a RISC Controller with an 8kB ROM for programming keyless entry features. Transponders of this type are also called “STARC” (Security Transponder and RISC Controller) [NXPe].

PCF7945 This transponder uses an updated protocol version (in comparison with PCF7941/42) to communicate via UHF for keyless entry purposes. The immobilization features are the same as above.

PCF7946/47 In contrast to the STARC based transponders, this transponder does not have a RISC controller and instead features a built-in rolling code generator for keyless entry [NXPf].

PCF7952 This transponder is the first for keyless-go applications and again offers a RISC controller [NXPc].

All HITAG 2 based transponders share a 32-bit UID, formatted according to XX XX XX UX (hex), whereby X represents a hex number and U the product identifier. Analyzing different car keys it was possible to identify the following scheme for the UIDs:

- PCF7936 – XX XX XX 1X
- PCF7942 – XX XX XX 4X

- PCF7946 – XX XX XX 2X
- PCF7952 – XX XX XX 7X

This information can be used to adapt a reading device to the different options of the transponders.

Future Products According to an NXP statement [NXP09b] there is a next generation immobilizer available, which is called “Hitag Pro” and uses AES encryption. The corresponding transponder is most probably the PCF7939PA. A transponder development kit is available, but to our best knowledge this type of transponder has not yet been deployed in cars [NXP08g].

C.3.1 125 kHz Protocols

In the following, (unencrypted) communication with common 125 kHz transponders is described. Some Hitag 2 transponders can emulate these simple transponders, by putting them into special operating modes termed “public modes”. In Sect. C.4, the Hitag 2 cipher and the corresponding encrypted protocol is detailed.

Public Mode A (EM 4102)

The EM4102 protocol is a read-only protocol by EM Microelectronic that cyclically transmits 64 bits of data [Mic05]. At first, a header consisting of 9 bits (all set to ‘1’) is sent, afterwards 40 bit of the transponder memory are being sent as rows each consisting of four data bits + one parity bit. The last five bits are four column parity bits and one stop bit set to 0’. The data transmitted by the transponder is also explained in Table C.2.

bit	function	value
$b_0 \cdots b_8$	header	all set to ‘1’
$b_9 \cdots b_{13}$	data + row parity	$b_{13} = b_9 \oplus b_{10} \oplus b_{11} \oplus b_{12}$
$b_{14} \cdots b_{18}$	data + row parity	$b_{18} = b_{14} \oplus b_{15} \oplus b_{16} \oplus b_{17}$
$\vdots$	$\vdots$	$\vdots$
$b_{54} \cdots b_{58}$	data + row parity	$b_{58} = b_{54} \oplus b_{55} \oplus b_{56} \oplus b_{57}$
b_{59}	column parity 1	$b_{59} = b_9 \oplus b_{14} \oplus \cdots \oplus b_{54}$
b_{60}, b_{61}, b_{62}	column parity 2, 3, 4	comparable to column parity 1
b_{63}	stop bit	‘0’

Table C.2: Encoding of the data link layer.

The bit period during the reading is usually 64 periods ($f/64$) of the RF field, but 32 ($f/32$) and 16 ($f/16$) period versions are also available from EM Microelectronic. The protocol supports bi-phase and Manchester encoding. Note, that this protocol is the same as in Sokymat’s Titan and is also implemented as a read-only capability in Sokymat’s Q5 protocol (which also offers write options).

When a HITAG 2 transponder is configured to operate in the Public Mode A, the data contained in Block 4 and Block 5 is transmitted when entering an RF field. In this most common

mode for this protocol mode, Manchester code with a period of 64 RF clocks is available. The data contained in Block 4 and Block 5 has to be set appropriately to reflect that data format of the EM4102 protocol, e.g., FF8FC002 (Block 4) and 523EBDE2 (Block 5) to transmit the ID 1F00441E77, whereas FF8 in Block 4 represents the 9-bit header. Even though the tag is configured to operate in Public Mode A, it is still possible to change the configuration byte. Figure C.2 illustrates the communication from transponder to base station.

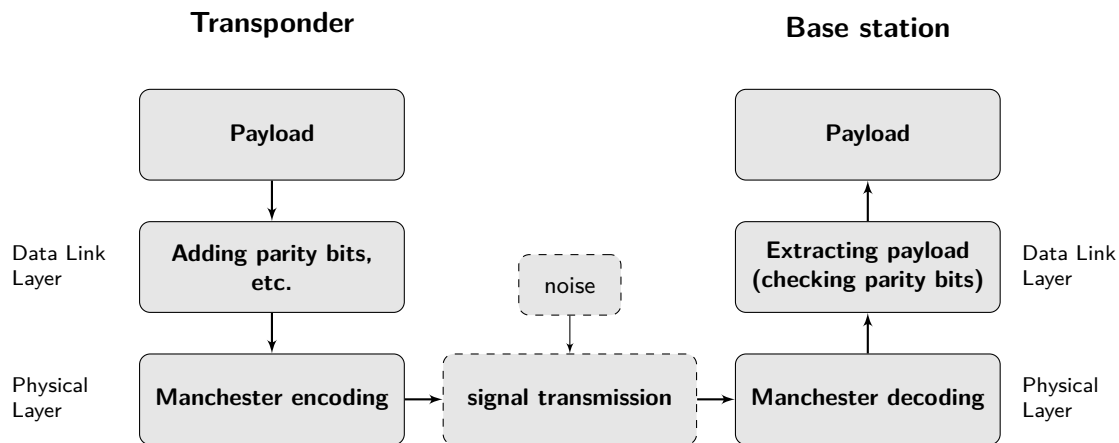


Figure C.2: Data transmission from transponder to base station (“reading”).

PIT Protocol

The Programmable Identification Transponder (PIT) protocol is supported, e.g., by PCF7930 and PCF7931 transponders. In case of the re-writable PCF7930, a synchronization code can be implemented (see Sect.2.3), contrary to the PCF7931 which is one-time-programmable. PCF7931 tags can be emulated by a HITAG 2 transponder set to operate in public mode C, in which the contents of Pages 4 to 7 are cyclically transmitted (128 bits in total). By default, bi-phase coding is used.

C.4 The HITAG 2 Stream Cipher and Transponders

This section gives a detailed description of the HITAG 2 protocol, as used in many car immobilizers. Other protocols (SECT, HITAG 1, HITAG S, ...) are not covered here, because their protocol and encryption schemes are confidential and not yet publicized. Even though the protocol is confidential, an official protocol description is available from the Proxmark website [NXP_a].

C.4.1 Modes of Hitag 2

The protocol has different modes of operation. The *normal* mode incorporates a password and a crypto authentication mode, and three different *public modes* enable compatibility to other protocols. The main difference between the normal mode and the public mode is, that the

public modes are operated in a **TTF** fashion, while the normal mode operates in a Reader Talks First (**RTF**) fashion. Figure C.3 illustrates the different options offered by the protocol.

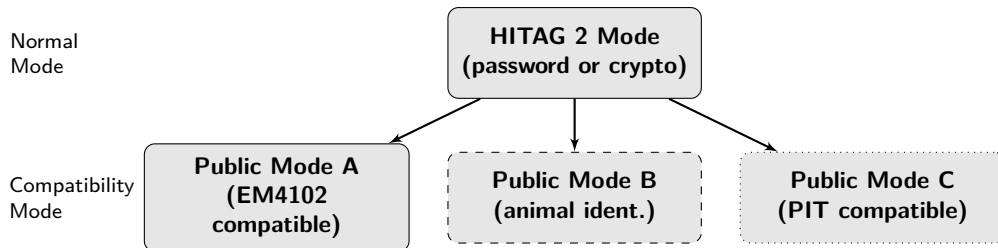


Figure C.3: Overview of the different modes of operation of the HITAG 2 protocol.

Before the protocols are presented, an overview of the memory map of a HITAG 2 transponder is given. Afterwards, the stream cipher used in the protocol is explained in detail.

C.4.2 Memory Map of HITAG 2 Transponders

A HITAG 2 transponder has 256 bits **EEPROM** memory that is organized in 8 pages with 32 bits each. The first page is the factory programmed serial number (SN). Page 1 is considered to contain the password of the read-write-device (RWD) in password mode (for authentication), or the 32 low bits of the 48-bit symmetric key used in the crypto mode.

While Page 2 is not used in password mode, it contains the 16 high bits of the key in crypto mode. Page 3 is a special page for the configuration of the transponder that is explained in Sect. C.4.2. The other pages all serve as read/write pages and contain arbitrary user programmable data. Table C.3 illustrates the memory map of the HITAG 2 transponder.

Table C.3: Memory maps for password or crypto mode of a HITAG 2 transponder.

Page	Content (Password Mode)	Content (Crypto Mode)
0	Serial Number (SN)	Serial Number (SN)
1	Password RWD	32 bit “KEY LOW”
2	reserved	16 bit “KEY HIGH”, 16 bit reserved
3	8 bit Configuration, 24 bit Tag Password	8 bit Configuration, 24 bit Tag Password
4	read/write page	read/write page
5	read/write page	read/write page
6	read/write page	read/write page
7	read/write page	read/write page

Configuration Byte

All functions of the transponder are controlled via a configuration byte contained in the most significant 8 bits of Page 3. The lower 24 bits are a tag password that is *not* relevant for the authentication. The use of this tag password is up to the user.

The functions of the most significant 8 bits are described in Table C.4. Bits 6 and 7 are **OTP**. Furthermore, when Bit 7 is set to '1', Page 1 and 2 are both neither readable nor writeable in crypto mode. Thus, even if one could authenticate to the tag via a replay attack it would be impossible to read out the key. In password mode, Page 1 is also set to no read and no write, but it is still possible to read Page 2.

Table C.4: The configuration byte of HITAG 2 transponders.

Bit	Function
0	Manchester Code (0) (default), Biphase Code (1)
1 and 2	see Table C.5
3	Password mode (0) or Crypto mode (1)
4	Page 6 and 7 read/write (0), read only (1)
5	Page 4 and 5 read/write (0), read only (1)
6	Page 3 read/write (0), read only(1) [OTP]
7	Page 1 and 2: read/write (0), special (see text) (1) [OTP]

The operation mode of the tag is determined by Bits 1 and 2 of the configuration byte, as illustrated in Table C.5. The most common settings of this byte are:

- 0x6: Password Mode
- 0xE: Crypto Mode

Table C.5: Bit 1 and 2 of the configuration byte determine the operation mode and physical layer of the tag.

Bit 2	Bit 1	Version	Coding
0	0	Public Mode B	bi-phase
0	1	Public Mode A	Manchester
1	0	Public Mode C	bi-phase
1	1	HITAG 2	depending on bit 0

Transponders termed “HITAG S” guarantee interoperability with HITAG 2 transponders [Phi02]. However, it is not known, if the HITAG S protocol itself uses the same cipher or not.

C.4.3 HITAG 2 Stream Cipher

HITAG is a trademark of NXP Semiconductors that relates to a whole range of **RFID** transponders and protocols. Besides that, it is also the name for a family of stream ciphers which are used for mutual authentication and confidentiality between transponders and the read/write device. Devices that make use of this cipher are, e.g., **RKE** systems and immobilizers for cars. The ciphers as well as the protocols are confidential, but the cipher that is used in the HITAG 2 protocol was either reverse engineered or leaked in January 2008, according to an NXP statement [NXP09b]. The implementation in C, including test vectors, and a visualization of the cipher are available online [man07].

Outline

The HITAG 2 cipher is a bit-oriented stream cipher that is well suited for a hardware implementation. Some important identifiers are outlined below:

- 48-bit secret key
- 48-bit **LFSR**
- a non-linear output function with 20 inputs bits and 1 output bit per clock

The computation of the keystream involves also the following values that are sent in plain during a protocol run:

- 32 bit serial number (SN)/identifier (IDE) of the tag
- 32 bit random **IV**

Output Function The main part of the cipher is the non-linear output function, that can be represented in several ways.

- (1) *Representation using multiplexors:* The output function can be seen as two levels of multiplexors, as shown in Fig. C.4, that work as follows:

- The input bits are used as address bits
- The output bit is the addressed bit of the contained data bits

The address bits are either chosen from the **LFSR** or from previous multiplexors. In the first level, 20 bits of the **LFSR** are fed as address bits to five four-bit multiplexors, resulting in a total of five bits of output (one from each multiplexor). These five bits are then used as address bits to a single five-bit multiplexor in the second level. The output of this larger multiplexor is the output of the cipher itself. Table C.6 summarizes the multiplexors used and the data bits contained.

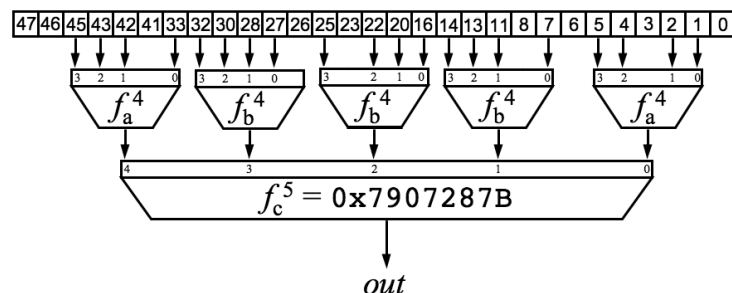


Figure C.4: HITAG 2 non-linear output function (Based on [Bac08])

Table C.6: Multiplexors making the output function in HITAG 2. Source: [Bac08].

Multiplexor	Function	Data bits	Address bits	Output bit
Level 1 Multiplexors				
MUX 1	f_a^4	0x2C79	1, 2, 4, 5	o_1
MUX 2	f_b^4	0x6671	7, 11, 13, 14	o_2
MUX 3	f_b^4	0x6671	16, 20, 22, 25	o_3
MUX 4	f_b^4	0x6671	27, 28, 30, 32	o_4
MUX 5	f_a^4	0x2C79	33, 42, 43, 45	o_5
Level 2 Multiplexor				
MUX 6	f_c^5	0x7907287B	o_1, o_2, o_3, o_4, o_5	out

Table C.7: HITAG2 function f_a^4 represented as a truth table.

d	c	b	a	f_a^4
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	0

- (2) *Representation in Algebraic Normal Form (ANF)*: It is obvious, that these multiplexors can be expressed using simple S-Boxes or truth tables, as shown in Table C.7 for function f_a^4 . It is then easily possible to deduce the ANF, as shown in Table C.8. For simplicity reasons, we omit the representation of the functions f_b^4 and f_c^5 .

Using the method shown in Table C.8 for all three functions yields:

$$f_a^4 = 0x2C79 = abc + ac + ad + bc + a + b + d + 1 \quad (C.1)$$

$$f_b^4 = 0x6671 = abd + acd + bcd + ab + ac + bc + a + b + d + 1 \quad (C.2)$$

$$f_c^5 = 0x7907287B = cde + abde + ade + de + abce + bce + ce + be + bcd + acd + bd + d + bc + ab + b + 1 \quad (C.3)$$

The method itself is described in the lecture notes “Kryptographie I” of Gregor Leander [Lea].

Table C.8: Example on how to calculate the ANF of function f_a^4 .

f_a^4	0	1	2	3	translated
1	1	1	1	1	1
0	1	1	1	1	a
0	0	1	1	1	b
1	1	0	0	0	–
1	1	1	0	0	–
1	0	0	1	1	ac
1	1	0	1	1	bc
0	1	1	1	1	abc
0	0	0	0	1	d
0	0	0	0	1	ad
1	1	1	1	0	–
1	0	0	0	0	–
0	0	0	0	0	–
1	1	1	1	0	–
0	0	0	1	0	–
0	0	1	1	0	–

C.4.4 Keystream Generation

The entire setup of the keystream generator is done in two stages. The first stage is the initialization of the **LFSR** that sets the internal state to a non-zero value using the key and the serial number of the tag. The second stage clocks the **LFSR** and randomizes its state. To do so, the key and **IV** bits are processed together with the first 32 output bits of the non-linear function. During the initial processing, the **LFSR** feedback is not being used and the keystream output is disabled. Once the feedback is activated, in the third stage the keystream is generated from the internal state. The three stages are now described in detail.

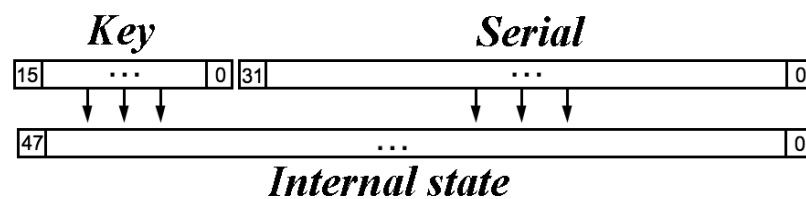


Figure C.5: HITAG 2 LFSR initialization during the initial setup. Source: [Bac08]

1. LFSR Initialization:

The state of the **LFSR** is initialized with the serial number and the secret key. The serial number is stored to bits 0..31 of the **LFSR** (where index 0 indicates the **LSB**) and the key bits 0..15 are stored to the bits with index 32..47 of the **LFSR**, as shown in Fig. C.5.

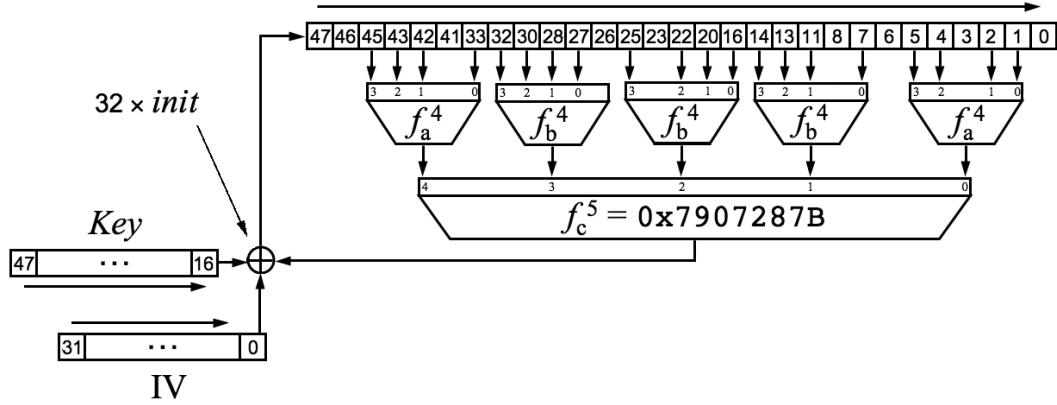


Figure C.6: HITAG 2 state randomization during the initial setup. Based on: [Bac08]

2. State Randomization:

To randomize the state of the **LFSR**, the keystream generator is clocked 32 times without the feedback function of the **LFSR**. In each clock cycle, the **LFSR** is shifted to the right and a new value is stored in the leftmost bit. This new value is calculated as follows: one key bit (index 16 through 47) **XOR** one bit of the IV (index 0 through 31) **XOR** the output bit of the non-linear function. The state randomization is shown in Fig. C.6. Please note, that after 32 clock cycles, the internal state is prepared for the generation of keystream.

3. Keystream Generation:

To generate a keystream, the feedback function (which is an **XOR** of all taps) of the **LFSR** is now in enabled, in addition to the output function. All other parts of the cipher remain inactive. In each clock cycle, the **LFSR** is shifted to the right, while the **MSB** is filled with the output of the feedback function. Once the internal state is updated, the output bit is recomputed. This process is repeated for producing a keystream of arbitrary length.

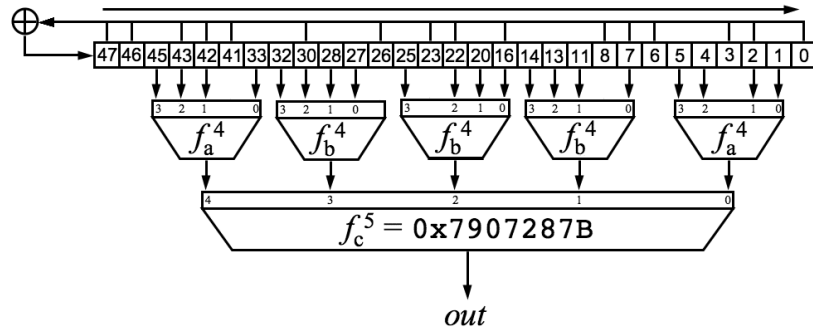


Figure C.7: HITAG 2 keystream generation. Source: [Bac08]

Enhanced Mode Some of the transponders that make use of the HITAG 2 cipher also offer an enhanced mode of operating HITAG 2 [NXPc]. This mode employs a 64 bit LFSR and a 96-bit secret key. No further details about the enhanced mode are known at the time of writing.

C.4.5 HITAG 2 Protocol

The HITAG 2 protocol offers two modes of operation: the plain mode and the crypto mode. In either case, an authentication command needs to be sent to read and write data. The authentication sequences are different, but reading and writing commands are the same, except that in crypto mode the complete command sequence is encrypted.

Authentication in Password Mode

In password mode, an instruction called `START_AUTH` from the read/write device causes all transponders in the field to respond with a start sequence of 5 bits all set to '1' followed by their 32-bit serial number. After the transponder has sent its serial number the base station provides a 32-bit password to the transponder. In case this password sent equals the password stored on the transponder its Page 3 is sent back to the base station. After authentication it is possible to read and write data to the transponder using the commands `READ_PAGE` and `WRITE_PAGE`.

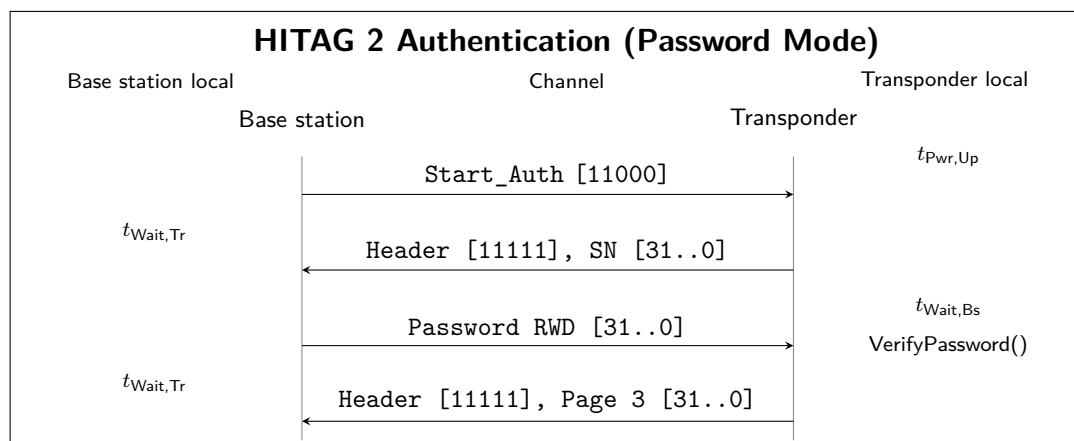


Figure C.8: Mutual authentication of the HITAG 2 protocol (Password Mode).

Authentication in Crypto Mode

In crypto mode, the same command `START_AUTH` is issued. The transponder responds as before, but this time the base station responds with an IV and an authenticator. This authenticator consists of the inverted first 32 output bits of the HITAG 2 stream cipher. Upon reception of the IV and authenticator the transponder computes the same authenticator and checks if the received authenticator matches the one which was computed. In case they match, the transponder responds with a header and the encrypted content of Page 3.

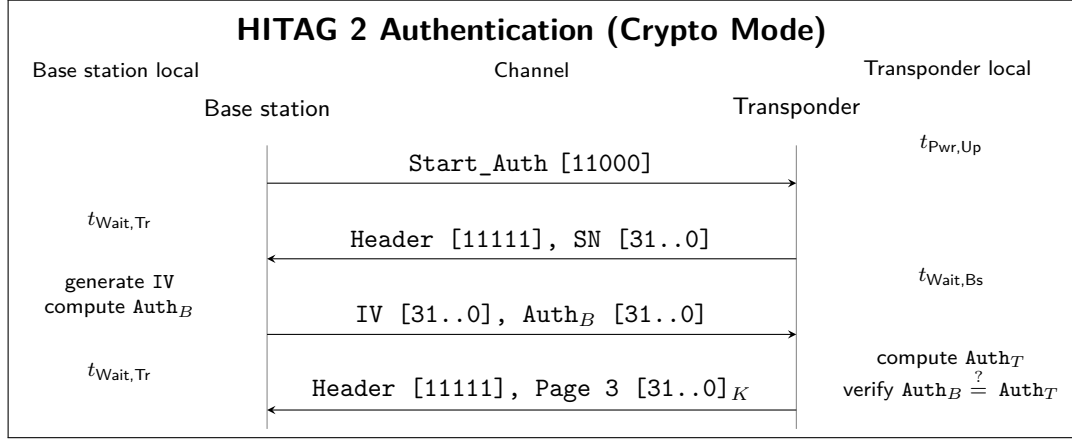


Figure C.9: Mutual authentication of the HITAG 2 protocol (Crypto Mode).

Read Page The read page command is built up as follows: $11XXX00\bar{X}\bar{X}\bar{X}$, where XXX are the address bits to address the desired page of the transponder and $\bar{X}\bar{X}\bar{X}$ denote the bit-wise inverted address. This sequence of bits is transmitted and a 32-bit response is sent back. In crypto mode, the complete command sequence and the response of the transponder are encrypted, except of the five-bit header.

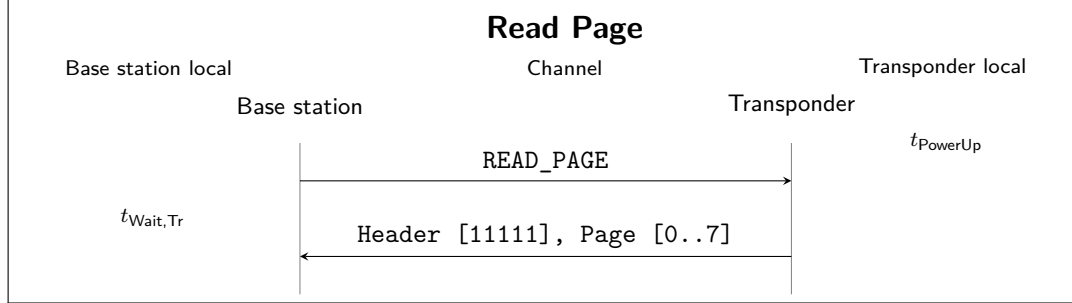


Figure C.10: Illustration of the READ_PAGE command.

Write Page The write page command is built up according to $10XXX01\bar{X}\bar{X}\bar{X}$, where again XXX are the address bits and $\bar{X}\bar{X}\bar{X}$ the bit-wise inverted address. To ensure data integrity, the command is resent to the base station for verifying the address. After successful verification the base station sends the new content of the page.

Two additional commands of the protocol, **HALT** and **READ_PAGE_INVERTED** are used to force a transponder into a wait state and to read the inverted data of the transponder, respectively. More details of the Hitag 2 protocol can be found in [NXPa].

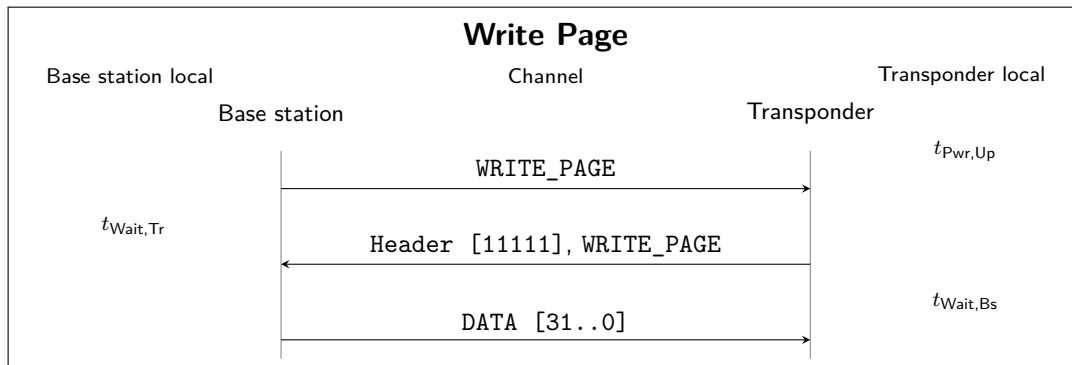


Figure C.11: Illustration of the WRITE_PAGE command.

C.5 Practical Results for 125 kHz RFIDs

The developed reader supports both Manchester and bi-phase coding and allows the implementation of various **LF RFID** protocols. A setup as shown in Fig. C.1b enables a reading distance of approx. 3cm using a car key and 6 to 8cm using other transponders (depending on the geometry of the transponder). After implementing the relevant protocols on the reader, various 125kHz transponders were analyzed. As a result, some reverse-engineered parts of the proprietary, unpublished protocol used by Megamos and Sokymat transponders is given in Sect. C.5.2.

C.5.1 An Exemplary 125 kHz Communication

Figure C.12 is a measurement of a communication between a HITAG 2 transponder and the **RFID** reader and illustrates the communication. The upper trace depicts the **RF** field generated by the reader and the signal in the center corresponds to the output pin of the U2270B **IC** on the reader. At the bottom, a debug signal output by a pin of the microcontroller indicates a successful reading of data from the transponder.

- (1) The field is turned off to reset the transponder and then switched on to supply the transponder with energy.
- (2) The reader sends the **START_AUTH** command 11000_2 (binary representation).
- (3) The transponder responds with a header, 11111_2 ,
- (4) followed by a 32-bit serial number.
- (5) The debug pin is used to verify the correct detection of the edges: the first peak indicates that the acquisition of the first bit is completed.
- (6) After the transmission, the output pin of the U2270B **IC** still outputs a signal. This has to be considered in the signal detection routine.

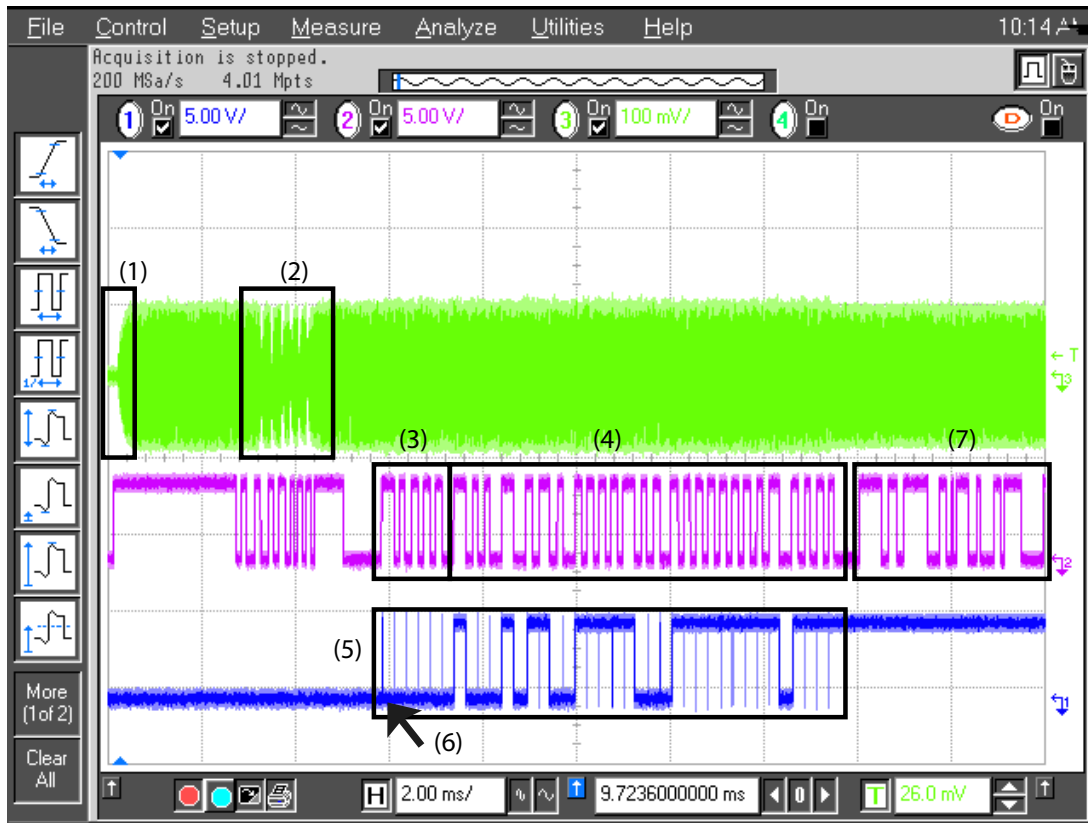


Figure C.12: A recorded sample communication between the RFID reader and a HITAG 2 transponder.

C.5.2 Reverse-Engineering the Sokymat Magic Protocol

According to JMA's and SILCA's catalogues, Sokymat's Magic protocol is also very widespread in immobilizers. No data sheet is available for this protocol and almost no information. At the time of writing, neither the protocol nor the encryption of the Magic protocol is published. Hence, to ease further research, this protocol and the corresponding commands are analyzed in the following.

Analysis of the Protocol

The protocol was analyzed using the developed 125 kHz reader. An oscilloscope helped to manually verify the responses of the tags. A first step was to ensure that the test samples were indeed a Sokymat Magic or Megamos Crypto transponder. According to all information that was publicly available, these transponders share the same ID (48) and therefore the same protocol characteristics and encryption.

A first test was to expose these transponders to an RF field without sending any commands. One could clearly identify a modulated signal (repetitive pattern) at the output of the reader IC. This proves the statement in [EP] that this type of transponder uses a Listening Window (LIW). An LIW is a special status message of the transponder indicating the time instant when the

transponder is ready to accept a command. However, the received LIW illustrated in Fig. C.13 was different from that described in [EP].

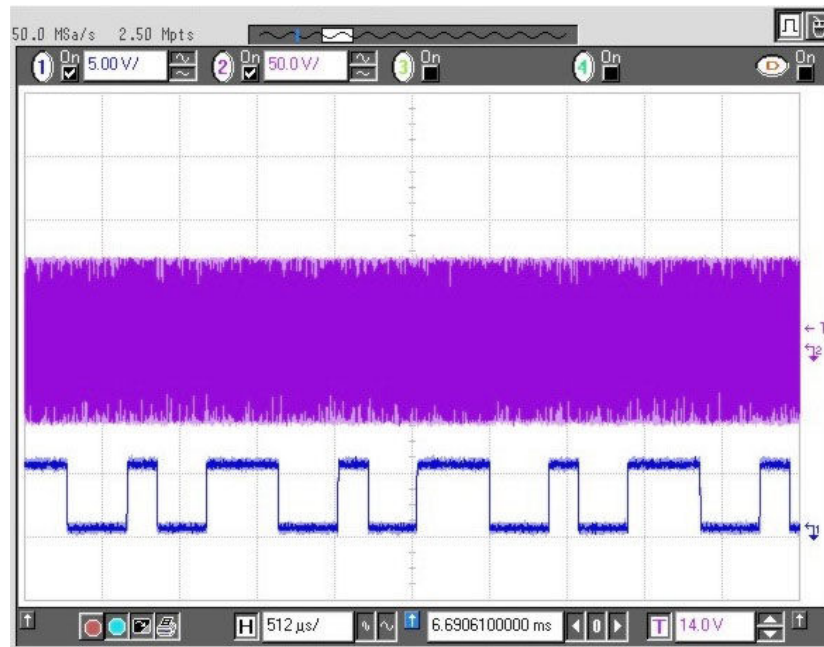
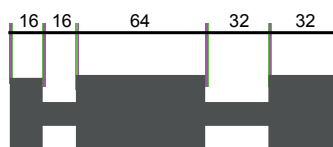
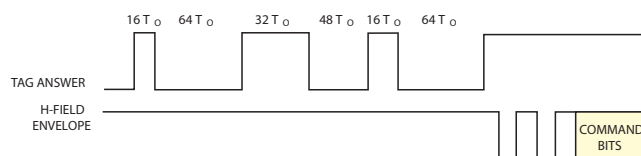


Figure C.13: Listening Window (LIW) of Megamos/Sokymat Magic Transponders as received by the developed reader. It neither matches the LIW of the EM4170 protocol nor the LIW as shown in [EP].

Since Sokymat had been acquired by EM Microelectronics, as the next step all known data sheets of that company to were studied in order to find hints about the Magic protocol. In different protocols of this company, the same principle of an LIW is used. By trial and error, i.e., adapting the timing parameters in the customized reader, a time slot that switched the transponder into receive mode when sending the pattern '00' was identified. To fix this point in time, the field is switched off to reset the transponder and then turned on. Finally, the response of the tag could be matched to the one described in the EM4170 protocol [Mick]. Yet again, the LIW described in the EM4170 data sheet differs from that received by the transponder — the originally expected LIWs are illustrated in Fig. C.14.



(a) LIW of the EM4170 protocol.



(b) LIW of Sokymat Magic as per [EP].

Figure C.14: Different LIWs that were expected, but not received by the reader. The different numbers represent the periods of the RF field during a state, whereas T_0 is $8 \mu\text{s}$.

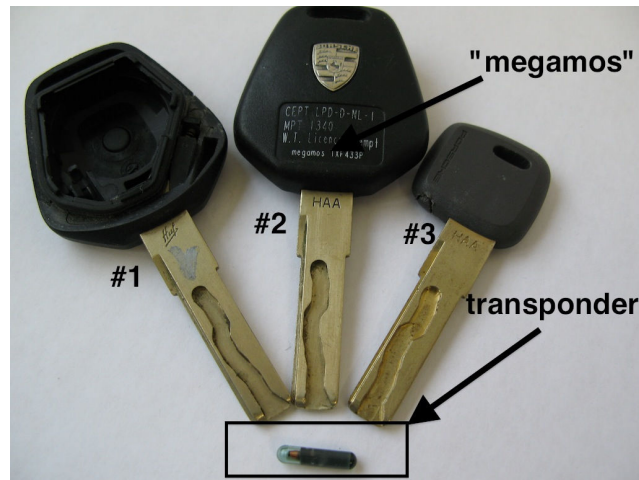


Figure C.15: A set of Porsche keys labeled with “Megamos” showing the contained immobilizer transponder.

Further investigations, i.e., exhaustive search of the command space, revealed that the transponder (from now on termed 'Magic 1') responds at least to the following commands:

- ID-MODE [0001]
- UM-MODE-1 [0101]

In addition to these two commands, a different set of transponders (from now on called 'Magic 2') also responds to the following command:

- UM-MODE-2 [1111]

The other known commands of the EM4170 protocol have not yet been reverse-engineered, as an encryption engine or the knowledge of a secret (a pin code) would have been necessary for that. Comparing the transponders and car keys to that one which can be found in SILCA's and JMA's catalogues, one is able to identify [SIL]:

- TP08 Magic 1 (ID 48)
- TP22 Magic 2 (SEAT) (turned out to be ID A3)
- TP23 Magic 2 (VOLKSWAGEN) (turned out to be ID A1)
- TP24 Magic 2 (SKODA) (turned out to be ID A4)
- TP25 Magic 2 (AUDI) (turned out to be ID A2)

Result

It has been shown, that a popular class of immobilizer transponders use the same protocol. However, due to lack of public documentation it is not known if they use different encryption routines. A sample data set contained in the car keys shown in Fig. C.15 is provided below:

- Key #1: ID: 008538A3 - Word 0: 8765 - Word 1: 4000
- Key #2: ID: 00855507 - Word 0: 8765 - Word 1: 4000
- Key #3: ID: 7274C268 - Word 0: 876D - Word 1: 4000

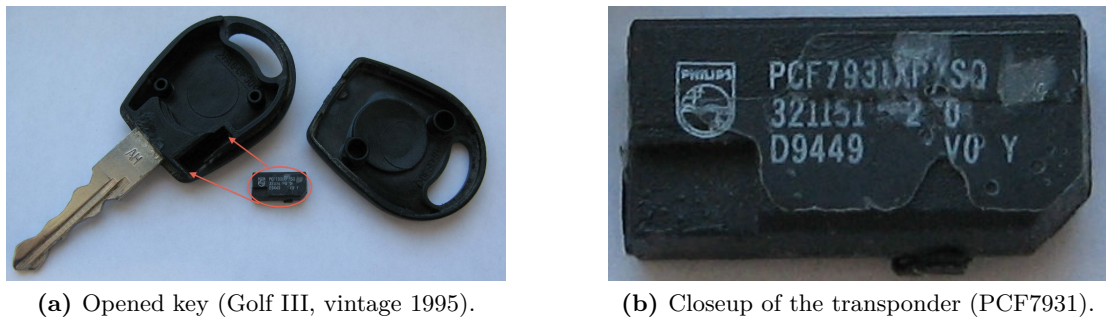


Figure C.16: Volkswagen Golf III key and the built-in transponder.

C.5.3 Analyzing Car Keys

Of a multitude of car keys that has been analyzed, an example is shown in Fig. C.16. The majority of keys was quite easy to open mechanically, since a means of exchanging the battery needs to be provided. Some keys however are sealed so that only destructive force allowed to open them (especially keys with no RKE option).

C.6 Future Research for 125 kHz Transponders

On the basis of the developed reader, more research can be carried out on commercial products in the future. For instance, the effectiveness of dictionary attacks can be verified, and side-channel analyses of this type of transponders can be performed. Furthermore, emulating transponders is promising for real-world attacks: Many access control systems in practice do not employ cryptography, but only rely on the serial number of the transponder.

Appendix D

Extensions for Implementation Attacks on Smartcards

For extending the measurement setup, detailed in Sect. 4, in order to analyze contact-based smartcards, e.g., power analysis of Java cards [VWG07], a modified chipcard reader and a chipcard adaptor have been developed in the course of this thesis, as detailed in the following.

D.1 A Modified Chipcard Reader for Power Analysis

The smartcard reader depicted in Fig. D.1 has been developed as a tool for conducting power analyses. A commercial ISO-7816-compatible “SCR335 USB-Chipcard reader”¹, that can be bought for less than 20\$, has been modified and extended with a special PCB. The reader is very widespread and hence drivers are included on most operating systems (or can be downloaded from the website of the manufacturer).

To enable power measurements of the smartcard under test, the ground path from the smartcard socket to the voltage supply is cut and a variable resistor, that can be adjusted between 0 and 200 Ohm, is inserted. By connecting an oscilloscope in parallel to this resistor, the power consumption of the smartcard can be digitized. A special PCB mounted on top of the reader allows for convenient access to all relevant signals. Furthermore, the VCC connection to the smartcard is modified, to support powering of the card from an external supply. When no plug is inserted in the power socket, the smartcard will be powered from the control PC via the USB cord (standard power mode). If a plug is inserted, a mechanical switch built into the plug socket will automatically connect the power pins of the smartcard with an external power supply and at the same time disconnect the USB supply (external power mode).

Finally, a trigger signal is required for starting the measurements. Hence, the modified chipcard reader provides two more connectors that may be used for triggering: the DATA pin is connected to the I/O signal of the smartcard (Pin 7 of ISO 7816 [Int04a]) and the TRIG pin is connected to an undefined “N.C.” Pin 4 of ISO 7816. For some smartcards, it is possible to access the latter pin and generate a trigger signal on it. The developed modified smartcard reader has meanwhile become an essential part of a power-analysis lab course at the Ruhr-University Bochum and is used in various projects.

¹<http://www.scmmicro.com>

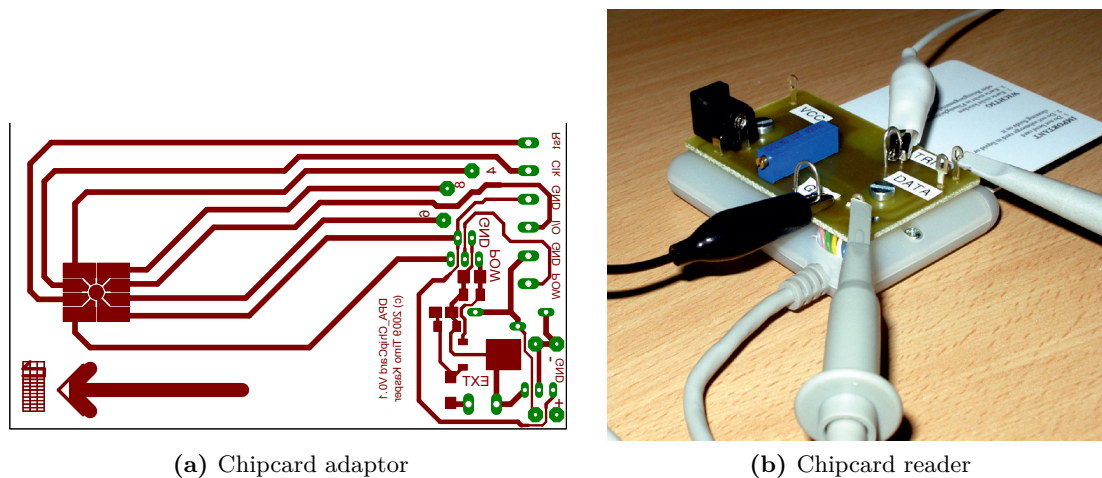


Figure D.1: Adaptor for emulating smartcards (left) and modified smartcard reader for power analysis (right)

D.2 Chipcard Adaptor for Emulating and Side-Channel Analysis

The chipcard adaptor depicted in the left of Fig. D.1 allows for emulating contact-based smartcards. It can be used for performing relay attacks, for monitoring purposes, and for assisting side-channel analysis of contact-based smartcards according to ISO 7816 [Int04a]. The adaptor has the appropriate dimensions and the specified contact interface to fit into any commercial smartcard reader. On the other side of the adaptor, a socket for smartcards allows any ISO 7816 card to be plugged in.

The data and power wires are tapped and rewired, such that the bitstreams can be relayed from and to a standard reader or a μC , e.g., for the analysis of communication protocols. The power lines allow to connect an external stable power supply (or a module for inducing power faults) while a variable resistor is inserted in series with the ground pin of the smartcard for performing power analyses. Similar to the tools for contactless cards and **RFIDs** a smartcard can be emulated, a relay attack can be conducted, and the communication data can be monitored. Simultaneously, faults can be injected or measurements for side-channel analysis can be recorded. We have successfully tested the adaptor in combination with our measurement setup by performing a power-analysis of an 8-Bit smartcard by Atmel containing an **AES** implementation in software: The correct 128-Bit key was revealed in minutes from approx. 100 measurements.

Appendix E

Extensions for Fault Injection

In [KOP10], a cost-efficient **FPGA**-based module has been developed that is compatible to the setup and allows for injecting faults with a very high precision. Faults can be induced by varying the voltage and generating clock glitches, as detailed in the paper, and furthermore optical faults and **EM** pulses are supported by the submodules illustrated in this section.

E.1 Optical Fault Injection

A modified electronic flash of a photo-camera serves as the basis for a low-cost module for optical fault injections as described, e.g., in [Sch08]. A small **PCB** that is connected to the flash of the camera serves for this purpose. It mainly consists of a High Voltage Generator (**HVG**) that produces up to 400 V DC out of the 3 V battery supply of the camera, and a 220 pF capacitor C_1 that is charged with electrons by the **HVG**. Figure E.1 illustrates the principle of the module.

A pin of the **PCB** ('Ready') that indicates when C_1 is fully charged, i.e., a flash is ready to be triggered, is connected to an input pin of an **FPGA**. The switch S_1 that had been mechanically switched when pressing the button of the camera, more precisely, when the lens opens to take a picture, was replaced by a transistor and can now also be controlled by the **FPGA**. Turning it on instantly discharges C_1 into the flash and thereby releases emanations that can cause an optical fault. The **FPGA** is programmed such that it busy-waits until the flash is ready, then starts the interaction with the **DUT**, and finally triggers an optical fault at the desired time instant. If a coil is connected instead of the flash, it is reported [MM06] that the resulting strong magnetic field can inject permanent faults into **RFID**s such as electronic passports, i.e., the device can be forever deactivated.

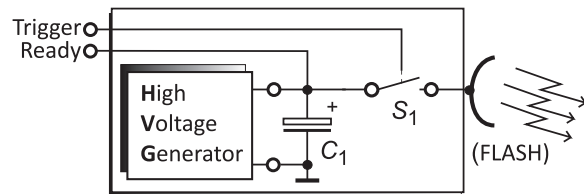


Figure E.1: Optical fault injection module.

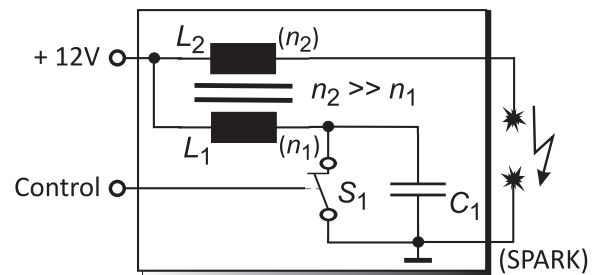


Figure E.2: Injecting faults with sparks.

E.2 Electro-Magnetic Fault Injection with Sparks

It is well-known in electronics that a sudden change in electric current generates an **EM** field — the higher the amplitude and the faster the alteration, the stronger will the resulting **EM** field be. The module for the injection of faults by generating sparks, as illustrated in Fig. E.2, creates strong, short **EM** pulses. The basic components of the module are an ignition coil and a common spark plug for cars.

The ignition coil consists of two inductances L_1 and L_2 with different numbers of turns n_1 and n_2 , that are coupled to form an electric transformer. The voltages V_1 and V_2 over the coils L_1 and L_2 follow the equation $V_1/V_2 = n_1/n_2$. The turn ratio is such that $n_2 \gg n_1$, hence any voltage occurring on the side of L_1 will be amplified orders of magnitude higher on the side of L_2 . To ignite an arc, a large over-voltage pulse is required: 250 V to 300 V on the side of L_1 , corresponding to approx. 20 kV on the side of L_2 , are sufficient to generate a spark [Mel99]. An STP10NK50Z Insulated Gate Bipolar Transistor (**IGBT**) [ST 05] employed as the switch S_1 withstands high voltages up to 500 V and can be turned on with a gate voltage of approx. 3 V. This allows for directly switching the 'Control' signal and thereby triggering the switch S_1 in Fig. E.2 by means of the controlling **FPGA**.

The generation of a spark consists of two phases, controlled by the time during which S_1 is switched on: During the first phase, the coil needs to be charged for a minimum amount of time. Then, in the second phase, the spark is released by opening the switch. While the switch S_1 is turned on, a **DC** current flows through the coil L_1 , and hence charges L_1 with energy, until S_1 opens the connection: The sudden interruption of the electric current flow through L_1 implies that the magnetic field collapses rapidly, inducing a high voltage on the side of L_1 , the amplitude of which depends on the amount of energy that has been stored during the charging phase. The corresponding very high voltage induced in L_2 then ignites the spark at the spark plug. The capacitor C_1 limits the overshooting of the voltage to protect the switch from damage.

As a power source a 12 V car battery was employed, as it supplies very large currents. If a standard power supply is used, a very large capacitor ($> 10\,000\,\mu\text{F}$) should be connected in parallel to the power source, to provide a sufficient amount of electric current. The strength of the induced fault can be steered by varying the amount of time during which the ignition coil can store energy. During our tests, a charge phase of 5 ms was sufficient to produce reliable, strong sparks. Furthermore, the positioning of the spark plug is of importance: in the near field, the field strength decreases rapidly as a function of the distance r to the **DUT**, proportional to $1/r^3$, hence small changes in r have a strong effect on the outcome of the faults. Optionally, a coil could be connected instead of the spark plug to generate a purely magnetic field.

Appendix F

Toolchain for Realizing Software Implementations

This section details on the development environments, editors, compilers, and a programmer used for developing the embedded and desktop applications, and introduces other software used in the course of this thesis.

F.1 Atmel AVR Studio

AVR Studio [Atmd] version 4.16 from Atmel shown in Fig. F.1 was used for developing embedded software for Atmel μ Cs. AVR Studio is an Integrated Development Environment (IDE) facilitating the development of applications for Atmel microcontrollers. It includes a set of tools allowing for fast and comfortable writing and debugging of software: An integrated project management tool, a source file editor and a simulator for various Atmel devices. AVR Studio can also interact with in-circuit emulators, such as AVR Dragon (See Sect. F.2), and development boards available for the AVR 8-bit RISC family of microcontrollers. Prior to compiling the software project the oscillation frequency of the μ C needs to be configured in the corresponding Project Options tab, e.g., in the case for the ATMega32 μ C of the RFID tool the value must be set to 13,560,000 Hz.

The simulation tool of AVR Studio is particularly handy, since it allows to perform time measurements of critical code segments without the need to actually program the μ C. This feature was used to provide performance characteristics, e.g., for the implemented cryptographic mechanisms as detailed in Appendix G.3.4.

F.1.1 WinAVR

While AVR Studio provides support for C, Pascal, BASIC and Assembly languages, the software developed for μ Cs in this thesis is written mostly in C, with embedded Assembly parts where timing is crucial. WinAVR [O'F], a suite of executable, open-source software development tools, was used in conjunction with AVR Studio to compile and debug the embedded C code. WinAVR includes the avr-gcc compiler for C and C++, the programmer avrdude and the debugger avr-gdb.

F.2 AVR Dragon Programmer

For programming of Atmel microcontrollers, a programming device from Atmel termed AVR Dragon [Atmc] illustrated in Figure F.2 is employed. The AVR Dragon is a low-cost development

Appendix F. Toolchain for Realizing Software Implementations

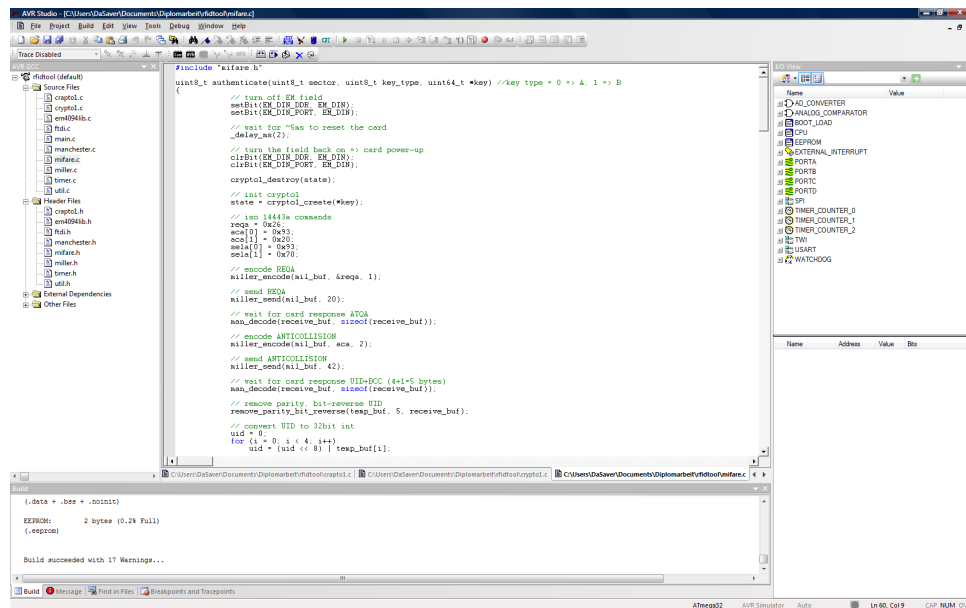


Figure F.1: Main screen of Atmel AVR Studio 4

tool that supports all programming modes for the AVR device family, including High Voltage Serial Programming (**HVSP**), Parallel Programming, the **ISP** required for the **ATMega32** of the **RFID** tool (Sect. 4.1) and Joint Test Action Group (**JTAG**) programming as required for the **ATxmega** of the Chameleon (Chap. 5). It also includes complete emulation and debugging support for devices with 32 kB or less Flash memory.

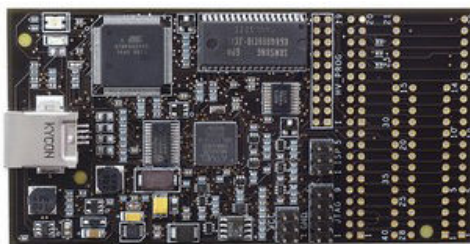


Figure F.2: Atmel AVR Dragon programming device, Source: [Atmc].

The AVR Dragon is powered via **USB** and is also capable of powering an external target. For our purposes the AVR Dragon was connected via **USB** to a host PC and through a customized 6-pin cable to the **ISP** header mounted on the **PCB** of the **RFID** tool. The pinout of the standardized **ISP** header of the AVR Dragon is described in [Atmc] and the corresponding pins on the **PCB** of the **RFID** tool are presented in Table F.1.

Table F.1: Pinout match of the AVR Dragon ISP interface and the programming header on the RFID tool.

AVR Dragon	↔	RFID Tool	Description
PIN-1	↔	PROGRAM-4	MISO
PIN-2	↔	PROGRAM-3	VTG/VCC
PIN-3	↔	PROGRAM-6	SCK
PIN-4	↔	PROGRAM-2	MOSI
PIN-5	↔	PROGRAM-1	RESET
PIN-6	↔	PROGRAM-5	GND

F.3 Microsoft Visual Studio

Some PC-side software code was written in C using Microsoft Visual Studio 2008 Professional [Micm] and can be executed on any Windows platform. The code also compiles successfully with the freely available Microsoft Visual C++ 2008 Express. In order to run the compiled code on a machine that does not have MS Visual Studio or MS Visual C++ Express, the additional Microsoft Visual C++ 2008 Redistributable package¹ must be installed.

F.4 HTerm

HTerm [Ham] illustrated in Figure F.3 is a serial terminal program for Windows with a GUI front-end and has been helpful tool for testing and debugging the communication with the various devices developed in this thesis. It can be used to send and receive data bytes in ASCII, hex, decimal and binary formats, e.g., for communicating via a VCP with the FTDI FT245R USB chip of the RFID tool. Optionally, the data may be retrieved from or written to a text log file.

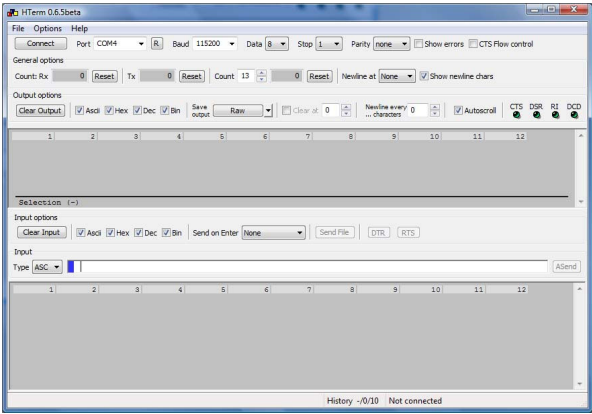


Figure F.3: Main screen of HTerm v0.6.5(beta).

¹<http://www.microsoft.com/downloads/details.aspx?FamilyID=9b2da534-3e03-4391-8a4d-074b9f2bc1bf>

Appendix G

RFID-Tool Software: Detailed Insights

A summary of the operating system and other functions implemented on the **RFID** tool (see Sect. 4.1.1) is given in the following. While this thesis so far has only very briefly covered the implemented functions in Sect. 4.1.3 and Sect. 6.5.1, the next sections explain all relevant details about the remaining developed software and can serve as a manual for the usage of the tool.

Section G.1 illustrates the low-level functions for communication with the **PC** and handling the physical interface, i.e., encoding and decoding of data according to ISO 14443. Next, the “operating system” of the reader and various other operating modes, e.g., for relay attacks, are detailed in Sect. G.2. Finally, Sect. G.3 describes the operating modes for attacking Mifare Classic cards and the related desktop software.

G.1 Low-Level Functions

Several libraries with low-level functions have been developed to provide an **API** on the μC for directly accessing the components of the hardware.

EM 4094 library initializes the **RF** frontend according to the desired standard (ISO 14443 Type A or B), sets filter functions accordingly, and provides functions for controlling the **EM** field.

Coding libraries provide the de- and encoding of the communication data on the physical layer according to the required **NRZ**, Miller, or Manchester codes.

USB library initializes and controls the communication via the FT245 **USB** chip.

Utility libraries establish an interface to the timers of the μC and care for on- μC computations of **CRC**-checksums, parity bits, or reversing the bit-order of messages.

The details of the developed functions are given next.

G.1.1 EM 4094 Library

The source file `em4094lib.c` contains three low-level functions for the initialization of the **RF** chip and sending Miller or Manchester-encoded data:

```
void EM_Init(void);
void miller_send(uint8_t *data, uint16_t length);
void man_send(uint8_t *data, uint16_t length);
```


The initialization routine `EM_Init()` prepares the EM 4094 for signal modulation and demodulation according to ISO 14443 by setting the appropriate chip parameters according to the datasheet[EM 03]. Furthermore, the three timer/counters of the μC are initialized as required for transmitting and receiving data via the RF interface.

The function `miller_send()` used to send commands from reader to card receives two arguments: a pointer `*data` referring to an array containing Miller-coded data, and the length of the latter length. The ISR of the 8-bit Timer/Counter 0 of the μC is used to establish precise timing during data transmission. The ISR is called after each half-bit period, i.e., 64 clock cycles of the μC , and can trigger the `EM_DIN_PULSE` pin, e.g., for creating short ($\approx 2.5 \mu\text{s}$) pauses in the RF field. While sending data, a special register flag `ISR_PAUSE` is set in the `miller_send()` function depending on the current value in the Miller array. The pause is then created by the ISR in the right instant, if and only if `ISR_PAUSE` is set.

The function `man_send()` transmits data by means of the Fake Tag and receives two arguments: an array with Manchester-coded data, pointed to by `*data`, and its length `length`. The operation is very similar to the one described above for `miller_send()` with two differences:

- The ISR is now used in combination with the 8-bit Timer/Counter 2.
- The ISR controls the `EM_MAN_OUT` pin for transmission of Manchester encoded data.

Note that the least-significant byte of the respective array, corresponding to one bit of information, is transmitted first. `miller_send()` and `man_send()` call the following functions for processing the communication data.

Miller Functions

Two routines for Miller encoding

```
uint16_t miller_encode(uint8_t *result, uint8_t *data, uint8_t
    length);
uint16_t miller_encode_fake_parity(uint8_t *result, uint8_t *data,
    uint8_t length, uint32_t fake_parity);
```

and one routine for Miller decoding

```
uint8_t miller_decode(uint8_t *result, uint8_t max_length);
```

are provided in the source file `miller.c`. While the decoding routine operates on the physical layer and directly decodes Miller data received from the corresponding pin of the EM 4094, both encoding routines do *not* send data. Instead they prepare the arrays as required for subsequently sending bitstreams via the `miller_send()` command described above.

The `miller_encode()` routine encodes raw data of a reader command according to the Miller code — the least-significant bit of the least-significant byte is transmitted first — and stores the result in an internal array of maximum length $2^{16} = 65536$ bytes, which can then be passed to the `miller_send()` function. The routine receives three arguments: The array with raw data to be encoded pointed to by `*data`, its length in bytes `length`, and the pointer to an array to store the encoded data `*result`. The length of the latter is the return value of the function. A time-memory tradeoff is utilized for the encoding: one bit of Miller-encoded data is stored in one byte of the array. This step is necessary for the timing during the subsequent call to

`miller_send()`: For an 8-bit μC architecture it is more efficient to test a whole byte register — accessing each individual bit in that register would take too many clock cycles in the `ISR`.

An additional parameter `fake_parity` is passed to the `miller_encode_fake_parity()` function. In contrast to `miller_encode()` where correct parity bits are calculated automatically, the `miller_encode_fake_parity()` function allows for encoding of encrypted data according to the Mifare Classic protocol, where parity bits are calculated from the plaintext instead of the ciphertext that is transmitted (compare with Fig. 6.4). The precomputed parity bits are passed in `fake_parity` where the **LSB** equals the parity bit of the first transmitted data byte.

Two arguments are passed to the `miller_decode()` routine: A pointer to an array storing the decoded data `*result`, and the maximum length of the array `max_length` which can be up to $2^8 = 256$ bytes. The function returns the actual number of bytes decoded. Upon calling the function, it waits infinitely on a transition on input pin `EM_MIL_IN` indicating an incoming command from a reader. The function can be either terminated by pressing the user button or upon successfully decoding the received command, i.e., receiving an End Of File (**EOF**). Timer/Counter 0 is utilized to distinguish the time periods between two transitions on the `EM_MIL_IN` pin, and thereby detects the logic value of the current bit transmitted. The

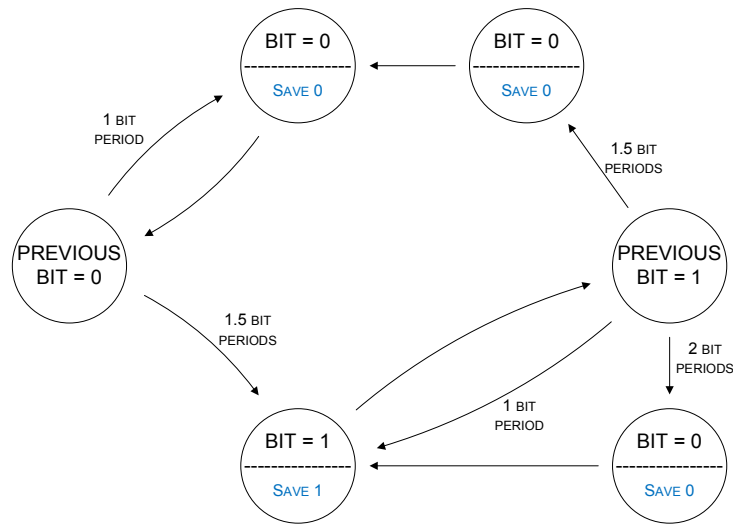


Figure G.1: Decoding of Miller-coded data visualized as a finite-state machine.

Finite-State Machine (**FSM**) describing the decoding process is presented in Fig. G.1.

Manchester Functions

The source file `manchester.c` contains two routines for Manchester encoding and one for Manchester decoding:

```

uint16_t man_encode(uint8_t *result, uint8_t *data, uint8_t length);
uint16_t man_encode_fake_parity(uint8_t *result, uint8_t *data,
    uint8_t length, uint32_t fake_parity);
uint8_t man_decode(uint8_t *result, uint8_t max_length);

```

The Manchester functions are very similar to the respective Miller functions described above, with the difference that the `man_decode()` routine does not wait infinitely for incoming data from a smartcard (or the Fake Tag). A timeout using the 8-bit Timer/Counter 2 is utilized to detect the end of a transmission, if no more data is received for more than one bit period.

In pseudo-code, the operation of the decoding routine `manchester_decode()` can be summarized as follows:

```
initialize();
wait_for_rising_edge();
// no response within 1 sec
if(timed_out) return;

start_timer2();

// main sample and decode loop
while(1) {
    wait_for_sample_point();
    edge_found = wait_for_edge();

    if(edge_found) {
        if(rising_edge)
            store_result(0);
        else
            store_result(1);
    }
    else {
        break;
    }
}

stop_timer2();
```

In the algorithm, `wait_for_sample_point()` polls a flag (Bit 2 in `UBRRL`) that is reset by the Timer 2 compare match **ISR** after clock cycle 127 has been reached, indicating the next sample point. Therefore, the complete loop must execute in less than 127 cycles, which requires careful programming especially in terms of saving **SRAM** accesses. Therefore, most data is kept in registers — in particular the result, which is stored bit-wise and only written to memory after eight bits have been collected. The second parameter of `manchester_decode()` specifies the maximum number of bytes to record. In the current firmware version, this value is set to 32. Note that the pseudocode gives an outline of the routine, but hides details like bitshifts necessary for the last 8-bit-buffer (in case the number of received bits is not a multiple of eight) to maintain the format of the result data.

G.1.2 FT245R Library

Three functions

```

void ftdi_init(void);
void ftdi_send(const uint8_t * data, const uint8_t length);
uint8_t ftdi_receive(uint8_t * data, const uint8_t max_length, const
    unsigned int timeout);

```

are provided for initializing the FT245R chip from FTDI, as well as for sending and receiving data via **USB**.

The byte array pointed to by **data* and its length *length* are passed to `ftdi_send()` which transmits the data by pulling the WR pin of the FT245R high and low after the current byte to be sent is output at the appropriate port of the **μC**. The receive function `ftdi_receive()` obtains three arguments: The byte array to store the received data **data*, the maximum number of bytes to be received *max_length* and a timeout value *timeout* specifying the maximum time to wait (in **CPU** cycles — one cycle is $\approx 73.7\text{ ns}$ at 13.56 MHz) for the arrival of the next byte. The function returns the number of bytes actually read. In the current version the timeout is set to 14,000 clock cycles which equals approx. 1 ms at $f = 13.56\text{ MHz}$. Each incoming byte is obtained by pulling the RD pin high and low and reading the data from the appropriate port of the **μC**.

G.1.3 Timers and Utilities

In addition to simple functions allowing for controlling the timers of the ATmega32, the **LED**, and the user button, three additional routines have been implemented:

```

void compute_crc(uint8_t *data, uint8_t length, uint8_t *first,
    uint8_t *second);
void remove_parity_bit_reverse(uint8_t *result, uint8_t length,
    uint8_t *raw);
uint8_t bit_reverse(uint8_t byte);

```

The function `compute_crc()` generates **CRC** bytes that need to be appended to transmitted data according to ISO/IEC 14443. The code is based on the sample function from the standard [Int01]. The function accepts the pointer **data* to the data buffer of length *length*, and two pointers to memory locations **first* and **second* where the resulting **CRC** bytes are to be stored.

In order to obtain meaningful commands, the bitstreams extracted from the **RF** field have to be converted, i.e., the parity bits have to be truncated and the order of the bits has to be reversed for each byte. For commands in the standard-frame format the function `remove_parity_bit_reverse()` is provided, which truncates the parity bits and reverses the bit order for all bytes in the array of length *length* pointed to by **raw*, and stores the result in the buffer pointed to by **result*. As the short-frame commands are only one byte long and do not contain parity bits, the extra function `bit_reverse()` just reverses the byte order of the byte supplied as argument and then returns the result.

G.2 Operating System and Modes

The `main()` function acts as the simplest form of an Operating System (**OS**). After initializing utility functions and both hardware chips EM 4094 and FT245R, the **OS** outputs a “Hello”

message on the **USB** interface and enters an infinite loop in the menu mode. In the main loop of the firmware, the **USB** interface is periodically polled for arriving data. A specific mode is selected by sending a 1-byte command via **USB** from a **PC**. The selected mode is stored into the **EEPROM** of the μC . After a re-boot or power-off, the stored mode is read from the **EEPROM** during the startup and automatically activated. The subroutine of an operating mode terminates either automatically upon completion of its task, or through user interaction, and then returns to the menu mode. In general, every mode can be terminated manually by pressing the user button. In the following, a brief description of various modes is presented that completes the modes introduced in Sect. 6.5.1.

G.2.1 Menu Mode

Command (hex : ascii) = 0x6D : 'm'

This is the default entry point for the **OS**. The μC waits for user input to enter a specific mode. Once the command indicating the desired mode is received via **USB**, it is saved in an internal variable and to the **EEPROM**. In case of a reset of the μC , the selected mode is read from the **EEPROM** and directly entered, which is of importance for stand-alone operation.

G.2.2 Passive MITM (Relay) Mode

Command (hex : ascii) = 0x72 : 'r'

The relay mode is used to conduct a passive **MITM** attack, as detailed in Sect. 2.2.4. Once entered, the green **LED** lights up and the data exchanged between a reader and a card is relayed in real time. At the same time the data is saved in a temporary buffer in the **SRAM**. If the user button is pressed the relay subroutine terminates and sends the saved data via **USB** to a **PC**. The temporary buffer is needed for performance reasons, as the data must be relayed and decoded “on-the-fly”.

G.2.3 Stand-Alone Passive MITM Mode

Command (hex : ascii) = 0x73 : 's'

In contrast to the normal relay mode the data is not sent to a **PC**, but first stored into the **SRAM**, and saved to the **EEPROM** after pressing the user button. The process of writing data to the **EEPROM** is signaled by the red **LED**. Note that accessing a byte in the **EEPROM** takes approx. 623 μs and is hence way too slow for storing decoded data in it “on-the-fly”.

G.2.4 Active MITM Mode

Command (hex : ascii) = 0x61 : 'a'

The details of the functions for active **MITM** attacks are given in the main part of this thesis in Sect. 4.1.3.

G.2.5 Relay Output Mode

Command (hex : ascii) = 0x6F : 'o'

This mode is closely linked to the stand-alone relay mode. The communication data stored in the **EEPROM** during a relay attack is read out and sent to a **PC** by entering this mode and pressing the user button.

G.2.6 Generic Reader Mode

Command (hex : ascii) = 0x65 : 'e'

A flexible reader functionality for arbitrary communication with a card and for switching the field on and off is provided. Besides controlling the reader from a program on the side of the **PC**, this mode can also be completely operated from a serial terminal, e.g., HTerm described in Sect. F.4.

Communication is always initiated by the **PC**, i.e., the reader does not send data without being queried. All commands start with an identifier character, followed by some parameter bytes. The command set for the reader mode is defined by:

's' $C\ Y_1 \dots Y_C$ The first parameter byte C determines the number of actual data bytes $Y_1 \dots Y_C$ to be sent. The reader then transmits these bytes via the **RFID** interface and waits for the response of the card. If an answer has been received within a timeout period (≈ 1 s), it is forwarded to the **PC**, where the first response byte again indicates the number of data bytes following. Only Miller-encoding and Manchester-decoding is performed on the reader — all other functionality, e.g., **CRC** checksum generation or parity checking, is performed on the controlling **PC**.

'w' M If the parameter byte M is **ASCII** '1', the **RF** interface is turned on, i.e., a card in the field is powered up. Any other parameter value switches off the generation of the carrier. For this command, no confirmation is returned by the microcontroller.

Note that **'s'** and **'w'** denote the **ASCII** characters **'s'** and **'w'**, respectively, whereas the values C and $Y_1 \dots Y_C$ are hex values. Note further that the timing in this mode is not deterministic, since **USB** communication (via Windows) induces an unpredictable delay.

Example: A typical communication sequence to initialize and enumerate a **RFID** device might start like this (values in apostrophes are **ASCII** bytes):

```
PC → Reader:   'w1'
PC → Reader:   's' 0x01 0x52
Reader → PC:   ATQA
PC → Reader:   's' 0x02 0x93 0x20
:
```

G.2.7 Generic Card Mode

Command (hex : ascii) = 0x64 : 'd'

This mode demonstrates a proof-of-concept of the card emulation capabilities of the **RFID** tool, allowing for emulation of an ISO/IEC 14443 compliant contactless smartcard with the Fake Tag introduced in Sect. 4.1.2. The current implementation supports smartcards with a double-size

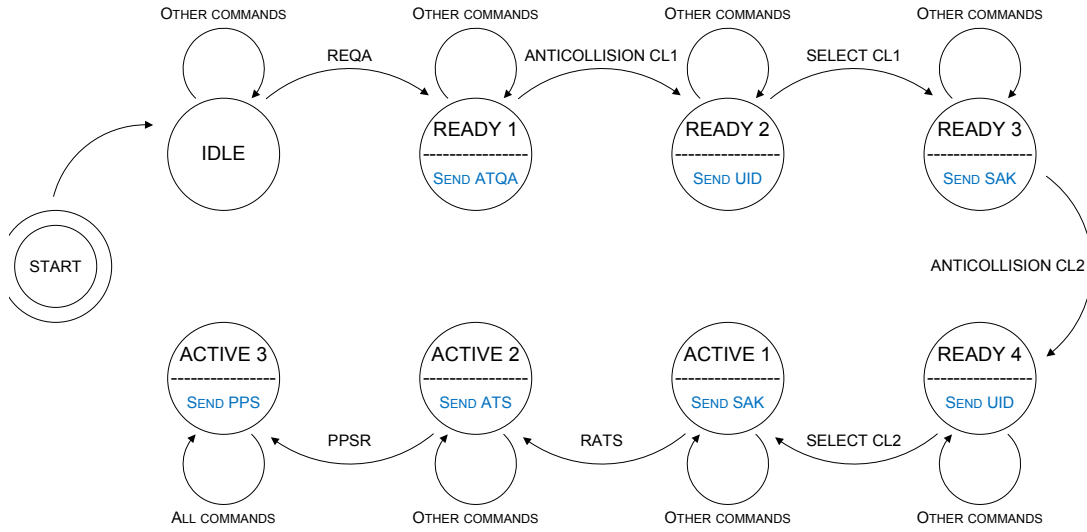


Figure G.2: Finite-state machine defining state transitions in the card emulation mode.

UID, e.g., Mifare DESfire. The operation of the card emulation mode is illustrated in the **FSM** illustrated in Fig. G.2. An infinite loop polls for the incoming commands from a reader and performs a state transition based on the current state stored in the variable `picc_state` and the received command.

There are strict timing requirements for the response of the card specified in Part 3 of ISO/IEC 14443, i.e., the **FDT** and the bit grid have to be considered, as detailed in Sect. 2.5.5. For this purpose Timer 0 of the μC is utilized to establish the exact timing required for reliable operation. Note that all Manchester-encoded card responses are precomputed before entering the **FSM** loop for simplicity of the timing control. However, this can be changed to allow for encoding of arbitrary commands “online” if the timer values for **FDT** are modified accordingly. The provided timing values have been determined experimentally at hand of a commercial **RFID** reader and may vary for different reader implementations.

G.2.8 Mifare Classic Card Mode

Command (hex : ascii) = 0x63 : 'c'

As an extension of the Generic Card Mode, this mode provides basic support for the Mifare Classic protocol, i.e., authenticating to a sector and reading Block 0. Every parameter transmitted by the Fake Tag, including the **UID** of the card and the card nonce n_C , can be set.

The **FSM** shown in Fig. G.3 is similar to the one of the Generic Card Mode, with two differences:

- The **UID** of Mifare Classic cards is 4 byte long, i.e., single-size, thus only one **CL** is supported
- Mifare Classic does not comply to Part 4 of ISO/IEC 14443, thus the **ACTIVE** states are different

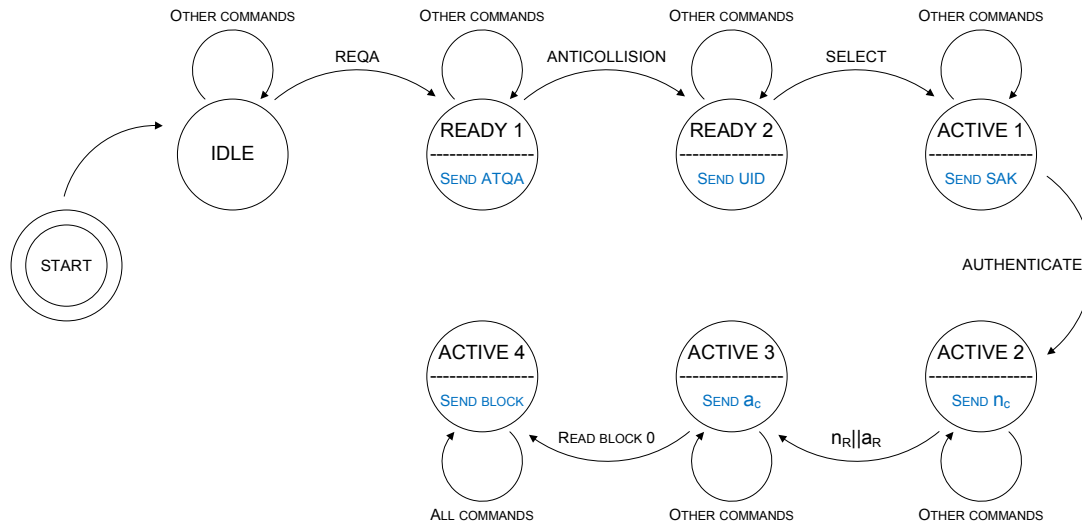


Figure G.3: Finite-state machine defining state transitions for emulating a Mifare Classic.

G.2.9 Timing Mode

Command (hex : ascii) = 0x74 : 't'

The timing mode was developed for testing of compliance to the ISO14443 standard of **RFID** readers with respect to the strict timing requirements for the response of a card (compare with Sect.2.5.5). The operation of this mode is very straightforward: First, an infinite loop is entered that waits on the **REQA** command from the reader and starts a timer with a pre-specified delay value for **FDT** if a **REQA** is received. Once the delay time elapses, an **ATQA** is sent to the reader that either accepts the response or not. The indicator for a success is the subsequent **ANTICOLLISION** command issued by the reader. The tested delay value is sent to the **PC** and is subsequently increased for the next iteration.

G.3 Software for Manipulating Mifare Classic

This section extends Sect. 6.5.1 and details on various implemented functions for modifying Mifare Classic cards.

G.3.1 Embedded Software

The embedded software for manipulating Mifare Classic cards running on the Atmel ATMega32 **μC** is organized in multiple C source and header files. The source files are summarized in Table G.1 along with a brief description of the contained functionality. Each source file, except the `main.c`, has a corresponding header file where all function prototypes are defined.

CRYPTO1 Functions

The source files `crypto1.c` and `crapto1.c` contain the software implementation of the proprietary stream cipher CRYPTO1 used for authentication and data encryption in Mifare Classic

Table G.1: List of source files of the project with a brief description of their content.

File	Functional Description
main.c	Main file of the project containing the <code>main()</code> function that is called upon reset of the μC .
em4094lib.c	Library of low-level functions for initialization of the EM 4094 RF-transceiver and sending of Manchester and Miller encoded data.
ftdi.c	Library of low-level functions for the FTDI FT245R USB chip.
miller.c	Contains routines for encoding and decoding of Miller data.
manchester.c	Contains routines for encoding and decoding of Manchester data.
mifare.c	Provides API for accessing a Mifare Classic card, including authentication, reading and writing routines.
crypto.c	Implementation of the CRYPTO1 encryption algorithm.
crapto.c	Contains two routines for rolling back the LFSR of the CRYPTO1.
timer.c	Library of low-level functions for setting-up and accessing the Timer/-Counters of the μC .
util.c	Auxiliary routines for parity, CRC calculation, etc.

smartcards. The code is based on an open-source project [Cra08]. It has been adapted and optimized for the 8-bit runtime environment of the μC and can be viewed as a “light-weight” version of the referenced implementation. The listing of CRYPTO1 related data structures and functions is given in the following.

```

struct Crypto1State {uint32_t odd, even;};

struct Crypto1State* crypto1_create(uint64_t key);
void crypto1_destroy(struct Crypto1State* s);
void crypto1_get_lfsr(struct Crypto1State* s, uint64_t* lfsr);
uint8_t crypto1_bit(struct Crypto1State* s, uint8_t in, int fb);
uint8_t crypto1_byte(struct Crypto1State* s, uint8_t in, int fb);
uint32_t crypto1_word(struct Crypto1State* s, uint32_t in, int fb);
uint32_t prng_successor(uint32_t x, uint32_t n);
void lfsr_rollback_bit(struct Crypto1State* s, uint32_t in, int fb);

```

The data structure `Crypto1State` holds the current state of the LFSR. The cipher is initialized with the 48-bit key passed as an argument to `crypto1_create()` which returns a pointer to the cipher state on completion. This pointer is used later on to clock the LFSR and obtain the keystream output by the cipher.

The functions `crypto1_word()`, `crypto1_byte()` and `crypto1_bit()` return 32, 8 and 1 bit of keystream, respectively, i.e., in each subsequent function call new keystream bits are generated and returned. During the initialization of the cipher (cf. Section 6.3.3) the values $UID \oplus n_C$ and n_R are input to the cipher. These values can be optionally provided through the `in` parameter of the respective function. This parameter can be omitted by providing 0 for an autonomous feedback operation of the cipher without any input. The parameter `fb` indicates whether the

input in is encrypted, e.g., the encrypted reader nonce n_R . If the input is encrypted, it is automatically decrypted and input to the cipher.

The **PRNG** successor function `prng_successor()` clocks the **LFSR** of the **PRNG**, initialized with x , for n times. Note that clocking in this case refers to the internal clock of a Mifare Classic card and not to the clock cycles of the μC . This function is amongst others used to compute both responses to the card nonce a_R and a_C .

The function `lfsr_rollback_bit()` rolls back the state of the **LFSR** by one bit. This function is used in the context of encrypting parity bits: After encrypting the parity bit the cipher is rolled back so that the first bit of the next data byte will be encrypted using the same keystream bit as the parity.

It is worthwhile noting that the values supplied to all CRYPTO1 functions must be in the big-endian format, i.e., the most-significant byte of the variable is the least-significant byte in the algorithm, while the bit order in each byte is in little-endian format, i.e., the least-significant bit in the variable is the least-significant bit in the algorithm.

Mifare Classic Protocol Functions

The library of Mifare Classic functions can be found in the source file `mifare.c`. It provides basic functionality for sector authentication, as well as accessing the memory of a Mifare Classic card, and contains the following functions:

```
uint8_t authenticate(uint8_t sector, uint8_t key_type, uint64_t *key
);
uint8_t read_block(uint8_t block, uint8_t *result);
uint8_t write_block(uint8_t block, uint8_t *data);
uint8_t inc_dec_value_block(uint8_t block, uint8_t *value, uint8_t
    is_dec);
uint8_t transfer(uint8_t block);
```

On success, all of the above functions return a positive value, while in case of an error a zero value is returned.

The function `authenticate()` accepts three arguments: Number of the sector (`sector`) to authenticate to, type of the key to use (`key_type`) which is $= 0$ for key A and $\neq 0$ for key B, and a pointer to the 48-bit key (`*key`).

The functions `read_block()` and `write_block()` accept as arguments the block number (`block`) in the range 0..63, and a pointer to a data array in the μC where the data should be stored in or retrieved from. The value block operations are supported by the `inc_dec_value_block()` function, which receives the pointer `*value` to a value to increase or decrease the value block by, and the flag `is_dec` indicating what type of operation should be carried out (1 for decrease, 0 for increase). After all value block operations are performed and the result is stored in a temporary byte register of the card (cf. Section 6.3.2), the function `transfer()` must be called in order to save the result permanently in the **EEPROM** of the Mifare card.

Resetting the PRNG

The **EM** field is switched off and thereby the **PRNG** is reset by driving the DIN pin of the EM 4094: setting it to a high level through

```
setBit (EM_DIN_DDR, EM_DIN) ;
setBit (EM_DIN_PORT, EM_DIN) ;
```

switches off the field and setting it to a low level with

```
clrBit (EM_DIN_DDR, EM_DIN) ;
clrBit (EM_DIN_PORT, EM_DIN) ;
```

switches the field back on. While the field is switched off the computations that do not require direct interaction with the card are performed, e.g., calculating the new reader nonce.

G.3.2 Notes to the Modes for Manipulating Mifare Classic Cards

For the key-recovery attack as described in Sect. 6.5.1, the following additional notes and considerations should be taken into account.

- The first 35 nonces (can be adjusted for each individual card) are skipped for the differential attack due to transient effects leading to some (undesired) variation of the card nonces generated.
- A constant runtime is required for the critical code sequence in the inner while-loop of the differential attack. Otherwise, it is not possible to recreate the same card nonce in multiple authentication attempts.
- The randomness involved in the differential attack is provided by two library functions `srand()` and `rand()`. The first one receives an integer value (“seed”) as an argument. The seed is increased in every for-loop iteration. Depending on the seed value a different sequence of random numbers is output by the `rand()` function, which is used to generate random values for the attack.
- Some newer models of Mifare Classic cards are not susceptible to the differential attack, as they have a new **PRNG** that does not allow to fix the card nonce. These newer cards can still be cracked with our attack, however, due to the issue of fixing the nonce the time required to perform the attack may grow considerably.

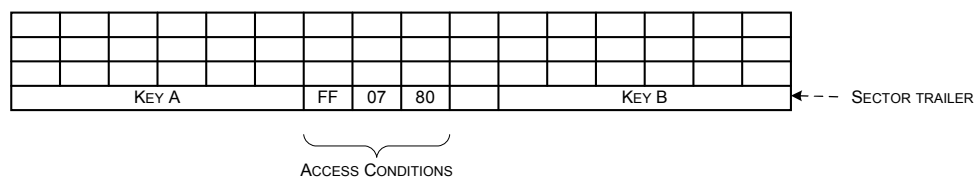


Figure G.4: Default configuration of the access condition bits permitting rewriting of a Mifare Classic.

For the dumping and writing modes the following should be considered:

- In the dump mode, a for-loop iterates over all 16 sectors of a Mifare Classic 1K. For each sector a complete authentication with key A stored in the Flash memory of the μ C is carried out with subsequent reading accesses to all four blocks of the respective sector. It

Listing G.1: Usage of the command-line tool

```

readercom.exe port [speed] [mode] [dumpfilename/value]
    port ::= com1 | com2 | com3 | com4 | com5 | com6
    speed ::= 300 | 4800 | 9600 | 19200 | 38400 | 57600 | 115200 | 230400
              ~~~~~
    mode ::= read [r] | write [w] | increase [i] | decrease [d] | crackkeys [c]
    dumpfilename ::= <anyname> (modes r, w and c only)
    value ::= <any 32-bit value> (modes i and d only)

Example: readercom.exe com1 115200 read dump.mfd
    open the terminal on port com1 with 115200 bit/s,
    read card content to file dump.mfd
Example: readercom.exe com1 115200 write dump.mfd
    open the terminal on port com1 with 115200 bit/s,
    write dump file content from dump.mfd to blank card
Example: readercom.exe com2 19200 increase 1000
    open the terminal on port com2 with 19200 bit/s,
    increase balance on card by 1000 c = 10 eur
Example: readercom.exe com2 19200 decrease 200
    open the terminal on port com2 with 19200 bit/s,
    decrease balance on card by 200 c = 2 eur
Example: readercom.exe com4 115200 crackkeys sectorkeys.log
    open the terminal on port com4 with 115200 bit/s,
    crack Mifare secret keys and save them to sectorkeys.log

```

must be noted that secret keys stored in each sector trailer are not readable and return logical zeroes when read. Therefore, in order to generate a complete dump of the card, the secret keys A and B stored in the Flash memory are inserted in each sector trailer in a separate step by the μ C and then sent to the PC for saving to dump file.

- A critical issue with writing blank Mifare Classic card is making sure that the configuration of the card will not become frozen afterwards, i.e., rewriting of the card remains possible in the future. If no extra measures are taken, writing of the unchanged dump of an ID Card to a blank Mifare Classic card will lock it for rewriting through the changed access condition bits. Hence, the access condition bits must be modified accordingly prior to writing the card to leave the card open for future rewrites. This is done through setting the access condition bits to default values as shown in Figure G.4.

G.3.3 Details of the Desktop Software

When executed without any parameters a short help is displayed on the screen explaining the usage of the developed command line program. The output of the program is presented in Listing G.1.

Details of Recovering Mifare Classic Keys with the Combined Attack

The set of data required for the off-line key search on the PC by means of the *differential attack* [Cou09] is summarized in Table G.2, where x_{enc} denote encrypted values and $i \in \{0 \dots 7\}$.

There are two steps involved in the process of finding the correct key:

Table G.2: Set of data required for the differential attack on the Mifare Classic sector key.

Value	Size(Bytes)	Description
u	4	UID of the card
n_C	4	Card nonce fixed by the μC
$n_{R,enc}^0$	4	The first encrypted reader nonce with a fixed 29-bit prefix
$a_{R,enc}^i$	$4 \cdot 8 = 32$	Eight encrypted reader responses to the card nonce
P_{enc}^i	$1 \cdot 8 = 8$	Eight encrypted parity bytes with P^i holding parity bits for $n_R^i a_R^i$
N_{enc}^i	$1 \cdot 8 = 8$	Eight encrypted 4-bit card responses (NACKs) N_{enc}^i corresponding to $n_{R,enc}^i a_{R,enc}^i$

- (1) Find key candidates: With the 32 bit of keystream calculated from the encrypted NACK N_{enc}^i with a known plaintext ($N^i = 0x5 \forall i$), the key space is reduced by 32 bit, i.e., on average $2^{48-32} = 2^{16}$ key candidates remain.
- (2) Check the key candidates and find the correct Crypto1 key: This is done by checking whether the parity bits in $\tilde{P}^i$ calculated over $n_R^i || a_R^i$ (decrypted values) match the (decrypted) parity bits in P^i .

The success probability of the algorithm is approx. 75%. If the correct key has been found, the software proceeds with the nested authentication attack, otherwise the μC collects data for the same sector once again.

The set of data required for the *nested authentication* attack is summarized in Table G.3, where x_{enc} denote encrypted values and $i \in \{3 \dots 18\}$.

Table G.3: Set of data required for the nested authentication attack.

Value	Size (in Bytes)	Description
u	4	UID of the card
n_C^1	4	Plaintext card nonce from the 1 st authentication
$n_{C,enc}^2$	4	Encrypted card nonce from the 2 nd authentication
P_{enc}^2	1	4 encrypted parity bits for $n_{C,enc}^2$
$n_{C,enc}^i$	$4 \cdot 16 = 64$	16 encrypted card nonces from the i^{th} authentication
P_{enc}^i	$1 \cdot 16 = 16$	64 encrypted parity bits for $n_{C,enc}^i$

There are three steps needed for extracting the key using the nested authentication attack:

- (1) Compute a list of possible nonces that correspond to the encrypted value $n_{C,enc}^2$. In the current implementation the list contains on average 75 nonces.
- (2) Calculate 32 bits of keystream by XORing $n_{C,enc}^2$ with the candidate of the decrypted nonce. The key space is again reduced by 32 bit, 2^{16} key candidates are expected.
- (3) Proceed according to Step (2) of the differential attack.

Processing the Key Candidates Two functions

```
uint32_t gen_odd_candidates(uint32_t *list, uint8_t *nack_enc);
uint32_t gen_even_candidates(uint32_t *list, uint8_t *nack_enc);
```

have been developed that accept an array of eight encrypted 4-bit card responses N_{enc}^i , pointed to by `*nack_enc`, as an argument. Both `gen_odd_candidates()` and `gen_even_candidates()` compute each 2^8 candidates on average for the odd and even bits of the internal state of the **LFSR**, respectively. In total $2^8 \cdot 2^8 = 2^{16}$ candidates for the internal state of the **LFSR** are obtained, sorted by odd and even bits. Two lists of possible states with respect to odd and even bits are saved in the previously allocated memory segments, pointed to by `*list`, and the length of each list is returned.

Subsequently, both lists with the key candidates and their respective lengths are passed to the

```
int find_key(uint64_t *key, uint32_t *odd, uint32_t odd_size,
            uint32_t *even, uint32_t even_size, uint32_t uid, uint32_t nt,
            uint32_t nr_enc, uint32_t *ar_enc, uint8_t *parity_enc);
```

function, together with the rest of the data collected from the card (cf. Table G.2), which is required to reject the wrong key candidates. The total number of key candidates is $\sigma = \text{odd_size} \cdot \text{even_size}$. Please note that although only $n_{R,enc}^0$ is passed to the function, the other values $n_{R,enc}^i$ for $i \in [1, 7]$ can be easily obtained by binary incrementing the three last bits of $n_{R,enc}^0$. The Algorithm 7 is executed inside the function `find_key()` to find the correct key, where K_O and K_E denote the lists of odd and even key candidates computed in the previous step. The success probability of the algorithm is $\approx 75\%$. The return value of the `find_key()` function is forwarded to the μC to indicate whether it should proceed with the nested authentication attack or attempt to collect data for the same sector once again.

The helper function

```
uint32_t compute_nonce_candidates(uint32_t *filtered_nonce_cands,
                                uint32_t nt, uint32_t nt_new_enc, uint8_t nt_new_par_enc, int
                                median, int tolerance);
```

is used to compute nonce candidates corresponding to the encrypted value `nt_new_enc`. For this purpose the successor nonces to `nt` in the range from `median-tolerance` to `median+tolerance` are calculated which are subsequently sieved using the information from the three parity bits in `nt_new_par_enc`. The resulting possible nonces are stored in the array pointed to by `*filtered_nonce_cands` and the size of the array ς is returned.

The computed nonce candidates are passed to

```
int find_key_nested(uint64_t *key, uint32_t nonce_size, uint32_t *
                  filtered_nonce_cands, uint32_t nt_new_enc, uint32_t uid, uint32_t
                  *enc_nonces, uint8_t *enc_paritys);
```

where for each possible card nonce in the list `*filtered_nonce_cands` 2^{16} (on average) possible **LFSR** states are obtained. These state candidates are checked against 64 parity bits stored in the array pointed to by `*enc_paritys` for 16 encrypted card nonces obtained from the card, stored in the array pointed to by `*enc_nonces`. If the correct key is found, it is saved to `*key` and the function returns a non-negative status value. Otherwise, the state candidates for the next

Algorithm 7 Find the secret key K from a set of candidate keys

Input: $K_O, K_E, \sigma, u, n_C, n_{R,enc}^0, a_{R,enc}^i, P_{enc}^i$ **Output:** Correct secret key K or FAIL

```

1: Derive  $n_{R,enc}^i$  for  $i \in [1, 7]$  from  $n_{R,enc}^0$ 
2: for  $j = 0$  TO  $\sigma - 1$  do
3:    $i \leftarrow 0$ 
4:    $wrong\_key \leftarrow 0$ 
5:   while  $wrong\_key = 0$  AND  $i < 8$  do
6:      $P^i \leftarrow \text{Decrypt}(P_{enc}^i, K^j)$ 
7:      $n_R^i \leftarrow \text{Decrypt}(n_{R,enc}^i, K^j)$ 
8:      $a_R^i \leftarrow \text{Decrypt}(a_{R,enc}^i, K^j)$ 
9:     Compute  $\tilde{P}^i$  for  $n_R^i || a_R^i$ 
10:    if  $P^i \neq \tilde{P}^i$  then
11:       $wrong\_key \leftarrow 1$ 
12:    end if
13:     $i \leftarrow i + 1$ 
14:  end while
15:  if  $wrong\_key = 0$  then
16:    Save  $K$ 
17:    return 1
18:  end if
19: end for
20: return 0

```

possible nonce are computed and searched for the correct key. Failure to find a correct key in the end of the routine indicates a too small range of tolerance used in `compute_nonce_candidates()`; in this case the function returns a zero value.

The internal operation of the function is defined by the Algorithm 8, where N denotes the list of possible card nonces. The return value of the `find_key_nested()` function is forwarded to the μC to indicate whether it should proceed with the next sector or collect data for the same sector once again. In case the correct key is found, it is both displayed on the screen and saved to the log file. Listing G.2 shows the typical content of the log file after successfully recovering all 16 sector keys.

Making Card Dumps

In the current implementation, in order to make a complete dump of a Mifare Classic card the keys for memory access to every sector must be pre-stored in the Flash memory of the μC . For instance, the command

```
C:\>readercom.exe com4 115200 r carddump.mfd
```

would result in accessing the VCP COM4 at a baud rate of 115200 for reading out all 16 sectors of a Mifare Classic card and storing it in the hex dump file `carddump.mfd`. The data is sent *block-wise* from μC to PC, i.e., 16 byte at a time are written to the file. Once the

Algorithm 8 Find the secret key K from a list of possible card nonces

Input: $\varsigma, N, u, n_{C,enc}^2, n_{C,enc}^i, P_{enc}^i$ **Output:** Correct secret key K or FAIL

```

1: for  $j = 0$  TO  $\varsigma - 1$  do
2:   Compute 32 bit of keystream  $ks_j = n_{C,enc}^2 \oplus n_j$  with  $n_j \in N$ 
3:   Compute list of key candidates  $K_C$  from  $ks_j$ 
4:   for all  $k_C \in K_C$  do
5:     wrong_key  $\leftarrow 0$ 
6:     for  $i = 3$  TO 18 do
7:        $P^i \leftarrow \text{Decrypt}(P_{enc}^i, k_C)$ 
8:        $n_C^i \leftarrow \text{Decrypt}(n_{C,enc}^i, k_C)$ 
9:       Compute  $\tilde{P}^i$  for  $n_C^i$ 
10:      if  $P^i \neq \tilde{P}^i$  then
11:        wrong_key  $\leftarrow 1$ 
12:      end if
13:    end for
14:    if wrong_key = 0 then
15:      Save  $K$ 
16:      return 1
17:    end if
18:  end for
19: end for
20: return 0

```

Listing G.2: Log file containing the recovered sector keys.

```

SECTOR KEY 0 = 44253d5a24f0
SECTOR KEY 1 = 6c2f620354ff
SECTOR KEY 2 = 5bfb60a2ad04
SECTOR KEY 3 = c8eedfbe5482
SECTOR KEY 4 = 2b18dbf03927
SECTOR KEY 5 = a04b0e84a40f
SECTOR KEY 6 = 7d8c140284e4
SECTOR KEY 7 = 6ad71821f0ed
SECTOR KEY 8 = 91029bc139ad
SECTOR KEY 9 = 94730c690b4b
SECTOR KEY 10 = 4d989f41da27
SECTOR KEY 11 = b669cb162971
SECTOR KEY 12 = e0a5bd8059ee
SECTOR KEY 13 = b8afcd7f83b
SECTOR KEY 14 = 15b2922a8b24
SECTOR KEY 15 = 32213e273b48

```


data is saved to the file, it can be opened with any free or commercial hex editor for data manipulations. Dumping of the entire memory of a Mifare Classic card takes approx. 1.2s in the current implementation.

Writing Card Dumps to a Mifare Classic

Any Mifare Classic smartcard that has not been locked through modified access condition bits (cf. “Mifare Classic Reader Mode (Clone Card)” in Section G.2) can be written, e.g., with the command

```
C:\>readercom.exe com4 115200 w carddump.mfd
```

which is semantically analog to the read command, with the data being *extracted* from the dump file `carddump.mfd` and sent to the μC . The data is transmitted to the μC *sector-wise*, i.e., 64 byte at a time, until the whole file has been processed. Writing the entire **EEPROM** of the card takes approx. 1.5 s in the current implementation.

Increasing & Decreasing Value Blocks

The mode allows for increasing and decreasing of value blocks of an arbitrary Mifare Classic card. However, the current implementation of this mode is tailored to suit the scenario of the payment system analyzed in this thesis and the particular memory configuration of the ID Card. The sample commands to increase and decrease the credit balance stored on the ID Card by 5€ are:

```
C:\>readercom.exe com4 115200 i 500
C:\>readercom.exe com4 115200 d 500
```

The specified value is transmitted to the μC which then accesses the memory of the card to modify two value blocks according to the supplied value.

G.3.4 Timing of a Straightforward CRYPTO1 Implementation

There is a limitation to the straightforward Mifare Classic Card Mode for 8-bit μC s described in Sect. G.2.8: it only works for reader implementations that allow for adjusting the timeout value, i.e., the maximum time the reader waits on the response of the card. The operation has been successfully tested with the HF Multi ISO RFID Reader [id06] from ACG which has a default timeout value of 4.8 ms but can be adjusted up to 76.5 ms. The reason for this limitation is the long run time of specific library functions pertinent to the encryption and transmission of card responses.

While the timing has been improved to fully meet the requirements of the ISO 14443 standard (end hence the problem is solved) in the optimized implementation presented in Sect. 5.4.2, for reference in the following the relevant execution times of the slow version are included. The long durations of the functions give an impression about the high level of optimization achieved in the improved program code. Critical function calls together with the approximate runtimes measured on the ATmega32 μC operated at 13.56 MHz are presented in Table G.4.

The critical code segment while reading a data block from memory is

Table G.4: Runtimes of critical code segments with functions required for encryption and data transmission in the Mifare Classic Card Mode.

Code	Runtime
<code>state = crypto1_create(key);</code>	1.7 ms
<code>ks32 = crypto1_word(state, 0, 0);</code>	2.5 ms
<code>ks8 = crypto1_byte(state, 0, 0);</code>	0.5 ms
<code>ks1 = crypto1_bit(state, 0, 0);</code>	94.4 μ s
<code>lfsr_rollback_bit(state, 0, 0);</code>	33 μ s
<code>man_encode_fake_parity(mil, cmd, 4, par);</code>	132 μ s

```

for (i = 0; i < 18; i++)
{
    block0[i] ^= crypto1_byte(state, 0, 0); // encrypt data
    par_b1 ^= (uint32_t)crypto1_bit(state, 0, 0) << i; // encrypt parity bit
    lfsr_rollback_bit(state, 0, 0); // roll back LFSR after parity
}
man_encode_fake_parity(block0man, block0, 18, par_b1); // encode with Manchester

```

which is responsible for encrypting 18 bytes of block data as well as the corresponding parity bits, and encoding the result with the Manchester code. The execution of this code segment takes approx. 11.7 ms. For instance, for a Mifare reader that has a 4 ms¹ timeout the oscillation frequency of the μ C needs to be increased by at least fourfold, i.e., it has to be set to 54.24 MHz, when using the straightforward, slow implementation.

¹As recommended in Part 4 of ISO 14443

Appendix H

Details of Attacking ID-Cards

This chapter explains which data is stored on the ID-cards analyzed in Chap. 6 and gives details about the practical tests.

H.1 Data stored on cards

We revealed that only the bytes in Tab. H.1 are affected by monetary transactions, while most parts of the memory remain unused. The *credit value* is stored in plain, twice in the value

Table H.1: Payment management data stored on an ID Card.

Sector	Block	Block Type	Byte	Data
0	2	read/write	7-9	Card number
			15	Check byte, calculated as XOR over all bytes of blocks 1 and 2
1	4,5	value	0-15	Value of the current credit balance
2	8	read/write	2-5	Time stamp of the last charging
			10-13	Credit amount charged last time
			15	ID of the last used terminal
	9	read/write	2-5	Time stamp of the last payment
			14	Transaction sequence number
			15	Check byte, calculated as XOR over all bytes of blocks 8 and 9

Blocks 4 and 5 on the card — probably for the sake of data integrity. Note that, as specified in [NXP08a], the content of each value block is already stored once inverted and again non-inverted for exactly that purpose, i.e., establishing data integrity. Block 2 of Sector 0 contains the *card number* that is also displayed on the charging terminals when charging an ID Card or ascertaining its current credit value. Note the difference between the card number, which is chosen and stored in the rewritable memory of the card by the card issuer, and the **UID** which is read-only and fixed in the **ROM** of the card. We noticed that for different cards, in addition to the varying card number, Byte 15 of the same block differs. Following a trial-and-error approach we found that this byte is simply computed by means of XORing all bytes of Blocks 1 and 2. Thus, when altering the card number, this byte needs to be recomputed accordingly. The same principle applies for Byte 15 of Block 9 when altering the content of Block 8 or Block 9.

H.2 Details of Practical System Level Tests

The exact financial transactions carried out with ID-cards during the analysis are detailed in the following.

H.2.1 Cards Used

The cards with their corresponding **UIDs** that were used during the analyses of the ID-card system are presented in Table H.2.

Table H.2: Cards used for the practical system-level tests.

Card	UID
G: Genuine ID Card, Card# 1677489	B410DF9E
A: Anonymous payment card, Card# 12266269	E5F11915
1: Blank Mifare Classic 1k card	15C6203F
2: Blank Mifare Classic 1k card	95AD763F
3: Blank Mifare Classic 1k card	956C713F
4: Blank Mifare Classic 1k card	55EF343F
5: Blank Mifare Classic 1k card	5510713F

H.2.2 Cloned Cards

The first test with cloned cards took place on Friday, 02.10.2009. All monetary transactions with the cards are summarized in Tab. H.3.

Table H.3: Monetary transactions associated with the test “Payments with Cloned Cards”.

Card	Date	Card #	Available	Amount Withdrawn	Rest
1	02.10.2009	1677489	4.85 €	3.10 €	1.75 €
2	02.10.2009	1677489	4.85 €	4.10 €	0.75 €
3	02.10.2009	1677489	4.85 €	4.60 €	0.25 €
G	05.10.2009	1677489	4.85 €	2.70 €	2.15 €

A total amount of 11.80 € was withdrawn from the associated shadow account using all three clones, which is 6.95 € more than the initially available credit value of 4.85 €. A payment with the genuine card three days after the shadow account had been overdrawn proves that no automatic blacklisting of cards is implemented.

Fraudulently Charged Card

Further payments were carried out after writing a credit value of 100 € to another clone of the original ID-card.

Table H.4: Monetary transactions associated with “Tampering with the Credit Balance”.

Card	Date	Card #	Amount		
			Available	Withdrawn	Rest
5	05.10.2009	1677489	100 €	18.80 €	81.20 €

All monetary transactions with the cards are summarized in Tab. H.4. The shadow account – if existing – for the card number 1677489 should now contain $-(9.65 € + 18.80 €) = -28.45 €$.

H.2.3 Generating New Cards

Several fake cards were generated and further payments were carried out as summarized in Table H.5.

Table H.5: Monetary transactions associated with the test “Generation of New Cards”.

Card	Date	Card #	Amount		
			Available	Withdrawn	Rest
1	06.10.2009	1677491	13.85 €	8.20 €	5.65 €
2	06.10.2009	12120412	10.25 €	4.70 €	5.55 €
3	06.10.2009	1677490	9.85 €	3.50 €	6.35 €
4	06.10.2009	10395848	14.85 €	7.60 €	7.25 €
1	07.10.2009	1677491	5.65 €	4.00 €	1.65 €
2	07.10.2009	12120412	5.55 €	4.50 €	1.05 €
3	07.10.2009	1677490	6.35 €	3.50 €	2.85 €
4	07.10.2009	10395848	7.25 €	3.20 €	4.05 €

H.2.4 Turning bits into Cash

The credit balance stored on the anonymous pay card M was fraudulently increased from 4.00 € to 44.00 €. The balance and the deposit were paid out in cash (see Table H.6).

Table H.6: Monetary transactions associated with the test “Cashback”.

Card	Date	Card #	Cashback
M	06.10.2009	12266269	54.00 € incl. 10.00 € deposit fee

H.2.5 Using a Clone After Cashback

A clone of the anonymous card that had been used for the cashback attack and that had already been returned to the issuing company, could still be used for further payments.

The total sum of the monetary transactions for the test is presented in Tab. H.7. The shadow account for the card number 12266269 now contains $-(40.00 € + 38.12 €) = -78.12 €$.

Table H.7: Monetary transactions associated with the test “Using a Clone After Cashback”.

Card	Date	Card #	Available	Amount Withdrawn	Rest
5	07.10.2009	12266269	44.00 €	38.12 €	5.88 €

H.2.6 Blacklisted ID Card

Table H.8: Recharges of the genuine ID Card.

Card	Date	Time	Card #	Avail.	Amount Charged	Result
G	07.10.2009	12:13	1677489	2.15 €	8.00 €	10.15 €
G	07.10.2009	13:10	1677489	0.35 €	10.00 €	10.35 €
G	07.10.2009	14:40	1677489	1.55 €	10.00 €	11.55 €

The credit balance of the genuine ID-card G was modified three times, as specified in Tab. H.8, in order to carry out further payment tests with the card (see Table H.9).

Table H.9: Monetary transactions associated with the test “Blacklisted ID Card”.

Card	Date #	Card #	Available	Amount Withdrawn	Rest
S	07.10.2009	1677489	10.15 €	9.80 €	0.35 €
S	07.10.2009	1677489	10.35 €	8.80 €	1.55 €
S	07.10.2009	1677489	11.55 €	10.75 €	0.80 €

Appendix I

Details of Tools for SCA Attacks on Contactless Smartcards

This chapter contains additional details about the software and hardware realized in the context of SCA-attacking contactless smartcards. It includes measurements and practical results for some of the tools realized in Chap. 7; a manual for using the developed CPA software in Sect. I.2; as well as schematics, transfer functions, and performance measurements of analogue filters, and other circuits built for removing the carrier signal of the reader, in Sect. I.3.

I.1 Measurements and Practical Results

Sect. I.1.1 illustrates the performance of the carrier subtracter developed in Sect. 7.3.3. Measured differences in the response times of a Mifare DESfire MF3ICD40 card when repeatedly sending the same plaintext are detailed in Sect. I.1.2. The usage of a utility for performing a DOM test, described in Sect. 3.3.2, is presented in Sect. I.1.3.

I.1.1 Performance of the Carrier Subtractor

The waveforms in Fig. I.1 demonstrate the effect of the subtraction approach: Plot 1 shows the probe signal (blue) and the reference signal measured at the crystal oscillator of the reader (red). The second measurement displays the sum of the signals (blue), amplified by a factor of 10, with the small amplitude variations being more clearly visible.

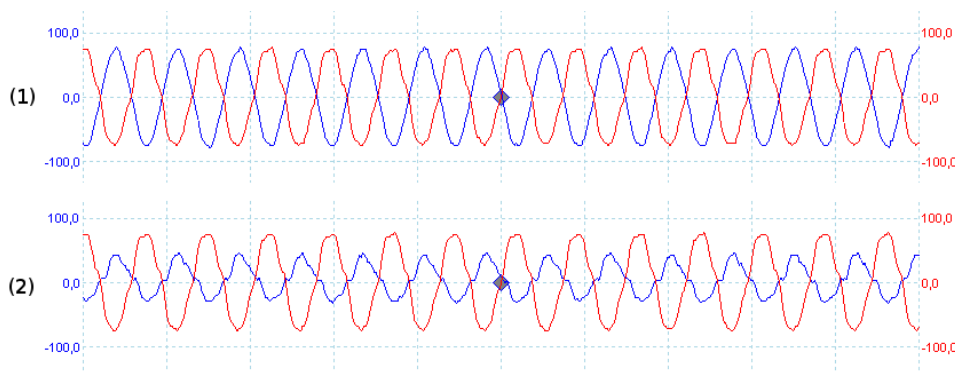


Figure I.1: Signals for difference amplifier circuit

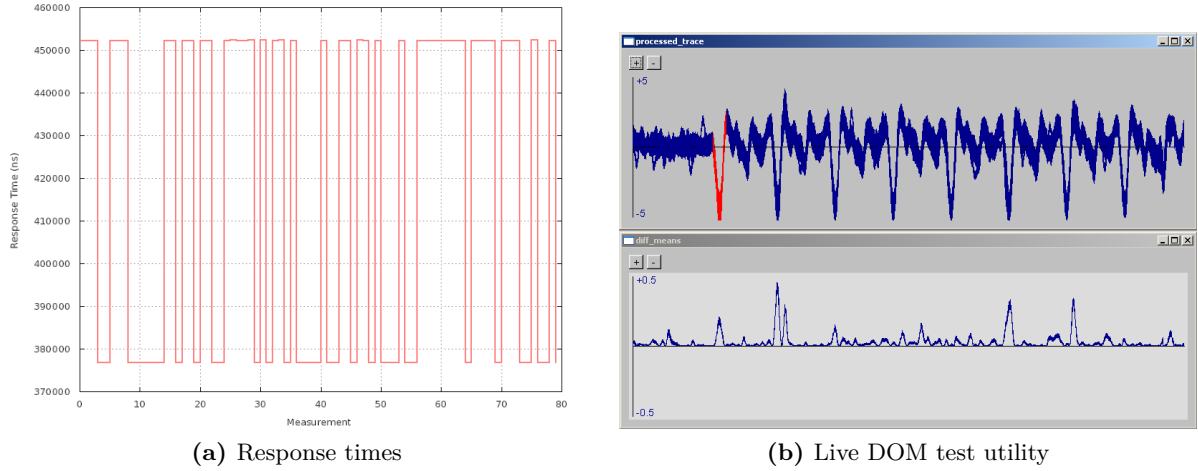


Figure I.2: Response times during a DESfire authentication and a screenshot of the Live DOM test utility.

I.1.2 Strange Response Times of a Mifare DESfire MF3ICD40

For repeatedly sending the same plaintext and the key set to all zeroes, two almost discrete response times can be identified: $376 \mu\text{s}$ and $452 \mu\text{s}$, with all measured values being very close to one of these, as depicted in Fig. I.2a.

I.1.3 DOM Test

Figure I.2b depicts a screenshot of the Live-DOM utility developed in Sect. 7.4.2, where the upper window displays the processed and aligned traces with the alignment pattern highlighted, while the lower window shows the squared DOM of S_1 and S_2 , i.e., $\left(\delta^{(1-2)}(k)\right)^2$.

I.2 Usage of the CPA Software

The actual CPA part includes code for power-analysis via the correlation method, both with and without the optimizations described in Sect. 7.3.1. The traditional variant is carried out by `dpa_correlation()`, implemented in `dpa.h/cpp`, for one predicted value of the S-Box output of the first round. For a given key candidate, a lookup-table of these predictions can be generated via `dpa_candidate()`.

The optimized CPA is encapsulated in a C++ class named `dpa_corr_recursive`. Upon instantiation, the trace length K and the S-Box to attack have to be specified. Next, key candidates K_s are attached with `add()`. Then, new traces can be added using `update()`, passing a reference to the trace data and the corresponding challenge at the S-Box input. To obtain the correlation for one K_s , `get_corr()` has to be invoked. Besides, timeseries of mean and variance of the measurements can be retrieved by calling `get_mean()` and `get_var()`.

For the online CPA, a so-called state file can optionally be specified. The file `state.txt` is created at the end of a CPA run for this purpose and contains parameters and intermediate values for mean, variance and all key candidate objects. From this file, it is possible to resume the

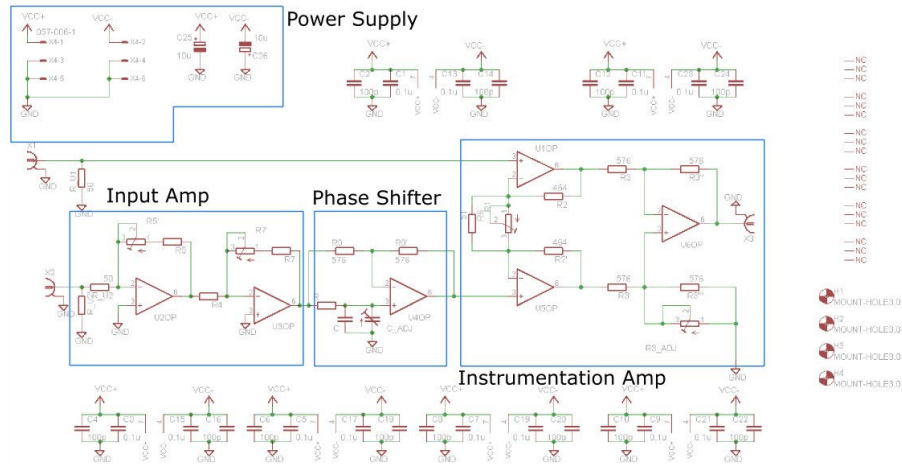


Figure I.3: Wiring diagram of the instrumentation amplifier circuit

processing in case of an interruption, using the information gathered in previous measurements, i.e., to proceed with the next step of the recursion.

I.3 Further Developed Circuits

An instrumentation amplifier is detailed in Sect. I.3.1, while Sect. I.3.2 presents an exemplary transfer function and schematics of two more analog filters.

I.3.1 Instrumentation Amplifier

In contrast to the implementation of the carrier subtracter illustrated in Sect. 7.3.3, i.e., a standard adder circuit, next an alternative approach based on an instrumentation amplifier is put into practice. The instrumentation amplifier consists of two input buffers, followed by an **opamp** in differential configuration, yielding the amplified difference of the input signals at the output. The main advantage of the circuit illustrated in Fig. I.3 is the higher Common Mode Rejection Ratio (**CMRR**), i.e., the attenuation of in-phase components present in both inputs.

Besides, due to the differential architecture, the offset voltages of the **opamps** are suppressed, which otherwise are included as a **DC** offset in the difference signal. Because of these advantages, an instrumentation amplifier was created on a **PCB**. However, its implementation is not straightforward: The high-bandwidth **opamps** employed (Texas Instruments THS3201, 1.8 GHz) have a current feedback structure that poses strict requirements on the resistor values for optimum frequency response and stability. Furthermore, the resistors around the central **opamp** have to be closely matched to reach optimum performance.

Finally, in our tests this realization of an instrumentation amplifier turned out to be unreliable and unstable, as the operational amplifiers often saturated during its operation. Still, using instrumentation amplifiers — in a more stable version — is generally promising due to the above described advantages. For future applications, completely integrated circuits are promising, e.g., a device such as the AD8129 providing a bandwidth of 200 MHz at a gain of 10 should

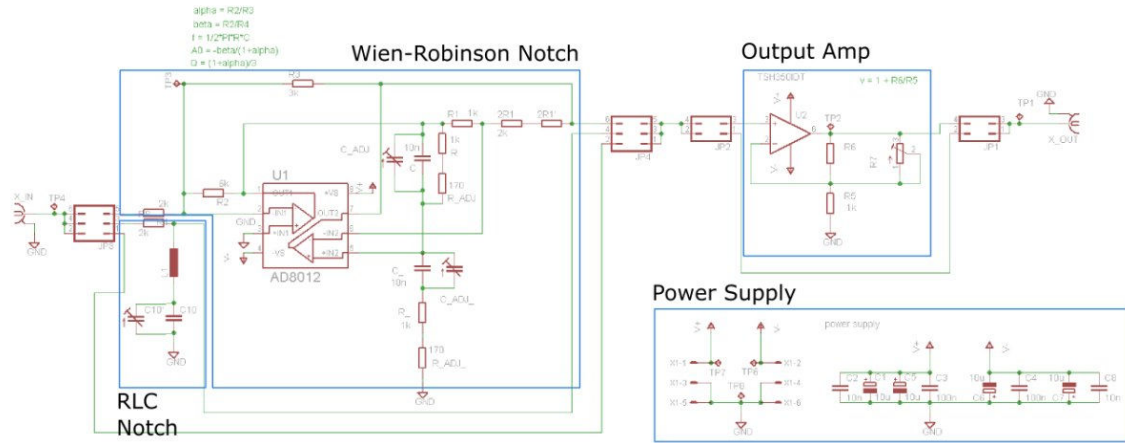


Figure I.4: Circuit diagram of the RLC and Wien-Robinson notch filter circuits

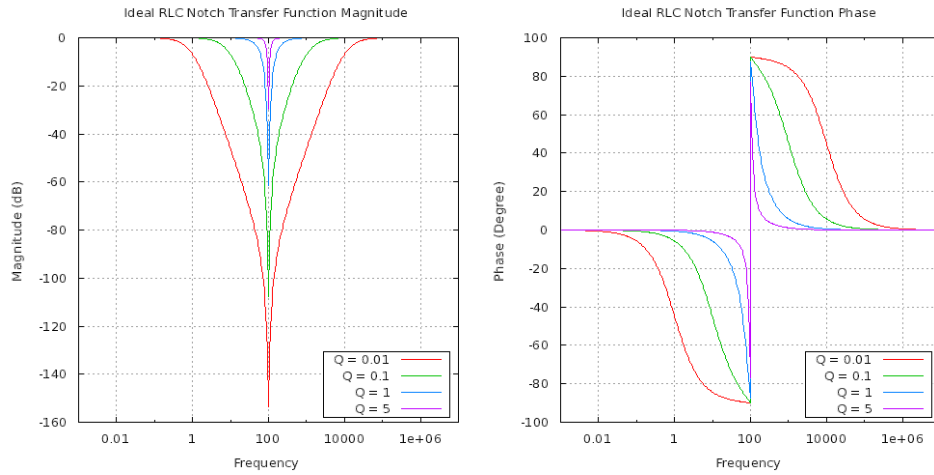


Figure I.5: Simulated frequency response of an ideal RLC notch filter

be considered, as it avoids the problems that occur when building an instrumentation amplifier from discrete components.

I.3.2 Analog Filters

The schematics of the passive second-order RLC notch and the active Wien-Robinson filter for removing the carrier frequency is shown in Fig. I.4.

The theoretical transfer function of the RLC notch filter is plotted for different values of Q and $\omega_0 = 2\pi 100$ Hz in Figure I.5.

Appendix J

Example Code and Wiring Diagrams for RFID Readers and Emulators

Last, but not least, Sect. J.1 illustrates an example code and Sect. J.2 the wiring diagram for the Chameleon from Chap. 5 ; in Sect. J.3 the circuit diagram of the 125 kHz **RFID** reader from Sect. C is given ; and Sect. J.4 depicts the schematic diagram and layout of the reader for contactless smartcards from Sect. 4.1.1.

J.1 Example Code for the Chameleon

The DESfire authentication is emulated on Chameleon as shown in Listing J.1. After receiving an authentication command (0x0A) from a reader this code is executed following Prot. 2 (see Sect. 7.2.1).

```
if(buf[1] == 0x0A)
{
    key_no = buf[2];
    // 8 byte random or fixed number
    uint8_t rndb[8] = {0xF0,0xF1,0xF2,0xF3,0xF4,0xF5,0xF6,0xF7};
    // encrypt rndb
    TripleDES_CBC_Encrypt(rndb, enc_data, keys, init, 1);
    // PCB
    uint8_t pcb_rndb [12];
    pcb_rndb [0] = 0x02;
    pcb_rndb [1] = 0xaf;
    // copy encrypted rndb
    for(i=0;i<8; i++)
        pcb_rndb[i+2] = enc_data[i];
    // append crc
    compute_crc(pcb_rndb,10,&pcb_rndb[10],&pcb_rndb[11]);
    // encode & send
    uint8_t rndb_desman[12*18+4];
    man_encode(rndb_desman, pcb_rndb, 12);
    man_send(rndb_desman,12*18+4);
    // receive PCD answer
    memset(receive_buf, 0x00, sizeof(receive_buf));
    length = miller_decode(receive_buf, sizeof(receive_buf));
    remove_parity_bit_reverse_asm(buf, receive_buf);
    // received dec(rnda+rndb') so encrypt and and extract rnda
    uint8_t rnda[8];
    TripleDES_CBC_Encrypt(buf+2, enc_data, keys, init, 2);
    // copy rnda
    for(i=0;i<8;i++)
        rnda[i] = enc_data[i];
```

```

// build rnda' by rotating rnda
uint8_t rnda_[8];
rnda_[0] = rnda[1];
rnda_[1] = rnda[2];
rnda_[2] = rnda[3];
rnda_[3] = rnda[4];
rnda_[4] = rnda[5];
rnda_[5] = rnda[6];
rnda_[6] = rnda[7];
rnda_[7] = rnda[0];
// encrypt rnda'
TripleDES_CBC_Encrypt(rnda_, enc_data, keys, init, 1);
// PCB
uint8_t pcb_rnda[12];
pcb_rnda[0] = 0x03;
pcb_rnda[1] = 0x00;
for(i=0;i<8; i++)
    pcb_rnda[i+2] = enc_data[i];
// append crc
compute_crc(pcb_rnda,10,&pcb_rnda[10],&pcb_rnda[11]);
// encode & send
uint8_t rnda_desman[12*18+4];
man_encode(rnda_desman, pcb_rnda, 12);
man_send(rnda_desman,12*18+4);
// create session key
uint8_t session_key[16];
session_key[0] = rnda[0];
session_key[1] = rnda[1];
session_key[2] = rnda[2];
session_key[3] = rnda[3];
session_key[4] = rndb[0];
session_key[5] = rndb[1];
session_key[6] = rndb[2];
session_key[7] = rndb[3];
session_key[8] = rnda[4];
session_key[9] = rnda[5];
session_key[10] = rnda[6];
session_key[11] = rnda[7];
session_key[12] = rndb[4];
session_key[13] = rndb[5];
session_key[14] = rndb[6];
session_key[15] = rndb[7];
break;
}
    
```

Listing J.1: "DESfire authentication source code."

J.2 Wiring Diagram of the Chameleon

The wiring of the microcontroller and the circuit diagram for the signal processing is given in Fig. J.1 and Fig. J.2, respectively. Furthermore, the partlist for building the emulator for contactless smartcards, developed in Chap. 5, is presented in Sect. J.2.1.



J.2.1 Components of Chameleon

The electrical components required for building one Chameleon are summarized in the table below.

Amount	Component	Name
1	ATxmega192A3	IC6
1	FTDI FT245RL	U2
1	IRFD110	T1
1	LM311D	IC3
1	LM1086 TO263	IC4
1	74ALVC08	IC5
1	74HC74	IC1
1	74HCT393D	IC2
1	820 Ω	R11
1	1.8 k Ω	R5
2	1 k Ω	R3, R6
1	4,7 k Ω	R9
1	8.2 k Ω	R4
1	10 Ω	R7
1	10 k Ω	R10
1	22 k Ω	R8
1	220 Ω	R1
1	1 k Ω	R2
2	10 μ F	C2, C8
1	10 nF	C32
2	32 pF	C11, C12
1	100 nF	C28
4	100 nF	C4, C5, C6, C7
1	150 pF	C9
1	220 nF	C3
1	Variable capacitor	C10
1	Variable capacitor (pF)	C1
3	Schottky	D1, D2, D3
2	4.7 V Zener	D4, D5
1	13.56 MHz crystal	Q1
1	Ferrit	FERRITE1
1	LED	LED1
1	MINI-USB-32005-201	X1

Table J.1: Components for building a Chameleon

J.3 Wiring Diagram of the 125 kHz Reader

Figure J.3 depicts the circuit diagram of the 125 kHz **RFID** reader from Sect. C.

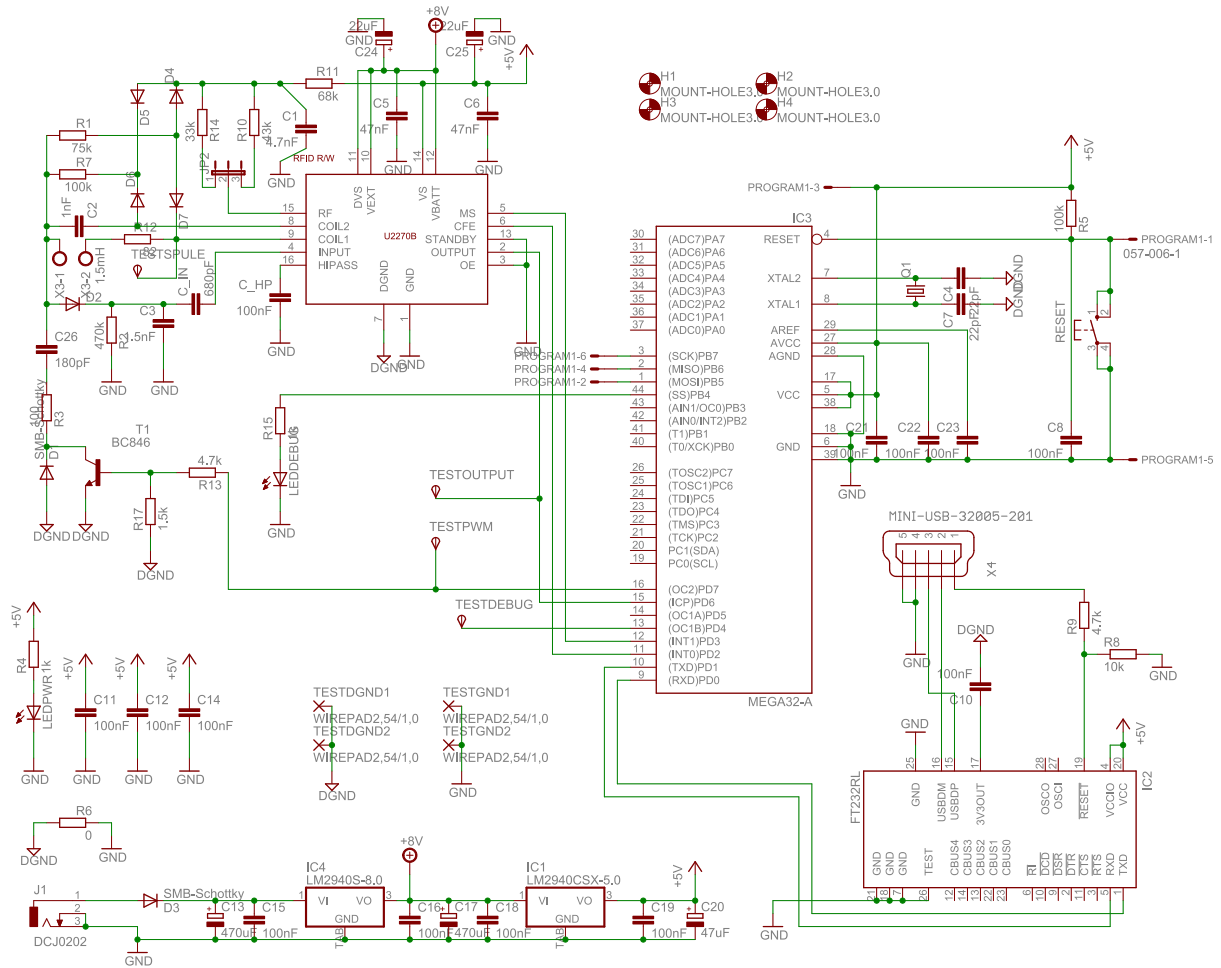


Figure J.3: Schematic of the developed 125 kHz reader

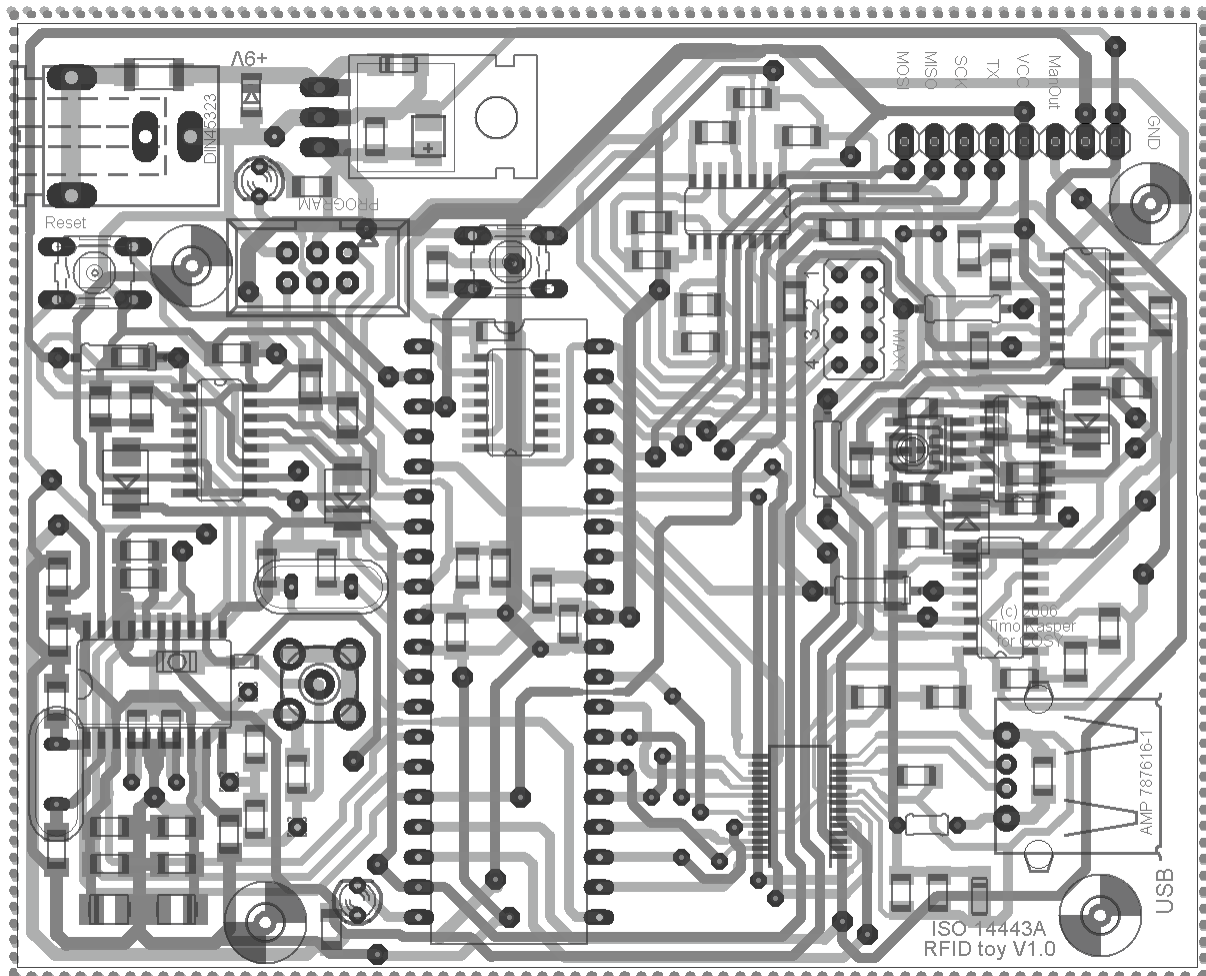


Figure J.5: Layout of the RFID Tool (before creating the ground planes).

Bibliography

- [AARR03] Dakshi Agrawal, Bruce Archambeault, Josyula Rao, and Pankaj Rohatgi. The EM Side-Channel(s). In Burton Kaliski, Çetin Koç, and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems (CHES 2002)*, volume 2523 of *Lecture Notes in Computer Science*, pages 29–45. Springer, 2003.
- [ABDM⁺10] Wim Aerts, Eli Biham, Dieter De Moitié, Elke De Mulder, Orr Dunkelman, Sebastiaan Indesteege, Nathan Keller, Bart Preneel, Guy Vandenbosch, and Ingrid Verbauwhede. A Practical Attack on KeeLoq. pages 1–22. Springer New York, 2010.
- [ADDS91] D. G. Abraham, G. M. Dolan, G. P. Double, and J. V. Stevens. Transaction security system. *IBM Systems Journal - Special issue on cryptology*, 30:206–229, March 1991.
- [AG01] M. L. Akkar and Chr. Giraud. An Implementation of DES and AES, Secure against Some Attacks. In C. K. Koç, D. Naccache, and Chr. Paar, editors, *Proceedings of CHES 2001*, volume 2162 of *Lecture Notes in Computer Science*, pages 309–318. Springer-Verlag, 2001.
- [Agi06] Agilent. *User's Guide Agilent 1160 Series Miniature Passive Oscilloscope Probes*, November 2006. <http://cp.literature.agilent.com/litweb/pdf/01160-92009.pdf>.
- [AK98] Ross J. Anderson and Markus G. Kuhn. Low Cost Attacks on Tamper Resistant Devices. In *Proceedings of the 5th International Workshop on Security Protocols*, pages 125–136, London, UK, 1998. Springer-Verlag.
- [AKQ] Gildas Avoine, Kassem Kalach, and Jean-Jacques Quisquater. Belgian Biometric Passport does not get a pass... Your personal data are in danger! <http://www.dice.ucl.ac.be/crypto/passport/index.html>.
- [AKS07] Onur Aciğmez, Çetin Kaya Koç, and Jean-Pierre Seifert. On the Power of Simple Branch Prediction Analysis. In *ASIACCS 2007: Proceedings of the 2nd ACM symposium on Information, computer and communications security*, pages 312–320, New York, NY, USA, 2007. ACM.
- [Ana03] Analog Devices, Inc. *AD8058 Dual, High Performance Voltage Feedback, 325 MHz Amplifier Datasheet*, 2003. http://www.analog.com/static/imported-files/data_sheets/AD8057_8058.pdf.
- [APSQ06] Cédric Archambeau, Eric Peeters, François-Xavier Standaert, and Jean-Jacques Quisquater. Template Attacks in Principal Subspaces. In Louis Goubin and

- Mitsuru Matsui, editors, *Proceedings of CHES 2006*, volume 4249 of *Lecture Notes in Computer Science*, pages 1–14. Springer-Verlag, 2006.
- [Atma] Atmel. ATMega32 data sheet. http://www.atmel.com/dyn/resources/prod_documents/doc2503.pdf.
- [Atmb] Atmel. ATxmega192A3 Data Sheet. http://www.atmel.com/dyn/resources/prod_documents/doc8068.pdf.
- [Atmc] Atmel. AVR Dragon online-help. <http://support.atmel.no/knowledgebase/avrstudiohelp/mergedProjects/AVRDragon/AVRDragon.htm>.
- [Atmd] Atmel. AVR Studio 4. <http://www.atmel.com/avrstudio>.
- [Atme] Atmel. AVR XMEGA A Manual. http://www.atmel.com/dyn/resources/prod_documents/doc8077.pdf.
- [Atmf] Atmel. *Electronic Immobilizers for the Automotive Industry*. http://www.atmel.com/dyn/resources/prod_documents/doc3a78163c4000a.pdf.
- [Atmg] Atmel. *U2270B Read/Write Basestation*. http://www.atmel.com/dyn/resources/prod_documents/doc4684.pdf.
- [Atm08] Atmel. Datasheet of Read/Write Base Station U2270B, 2008. www.atmel.com.
- [Bac08] R. Bachani. Time-memory-data cryptanalysis of the HiTag2 stream cipher. Master's thesis, Technical University of Denmark, 2008. <http://code.google.com/p/cryptanalysis-of-hitag2/>.
- [BBD⁺08] Holger Bock, Michael Braun, Markus Dichtl, Erwin Hess, Johann Heyszl, Walter Kargl, Helmut Koroschetz, Bernd Meyer, and Hermann Seuschek. A milestone towards rfid products offering asymmetric authentication based on elliptic curve cryptography. In *Workshop on RFID Security (RFIDSec'08)*, July 2008.
- [BBKK07] Alex Biryukov, Andrey Bogdanov, Dmitry Khovratovich, and Timo Kasper. Collision Attacks on AES-Based MAC: Alpha-MAC. In *CHES*, volume 4727 of *Lecture Notes in Computer Science*, pages 166–180. Springer, 2007.
- [BBL] BBL. Bbl stands. Page 100 contains overview of SILCA transponders, <http://www.bbblocks.co.za/PDF/catalogueDownload.pdf>.
- [BC93] Stefan Brands and David Chaum. Distance-Bounding Protocols (Extended Abstract). In *EUROCRYPT*, pages 344–359, 1993.
- [BCO04] Eric Brier, Christophe Clavier, and Francis Olivier. Correlation Power Analysis with a Leakage Model. In Marc Joye and Jean-Jacques Quisquater, editors, *Cryptographic Hardware and Embedded Systems - CHES 2004*, volume 3156 of *Lecture Notes in Computer Science*, pages 16–29. Springer, 2004.

-
- [BDL97] Dan Boneh, Richard A. DeMillo, and Richard J. Lipton. On the Importance of Checking Cryptographic Protocols for Faults (Extended Abstract). In *EUROCRYPT*, volume 1233 of *Lecture Notes in Computer Science*, pages 37–51. Springer, 1997.
- [BDR07] 3-millionster deutscher ePass ausgeliefert, 2007. http://www.bundesdruckerei.de/de/presse/pressemeldungen/pm_2007_04_02.html.
- [Ber05a] Daniel J. Bernstein. Cache-timing Attacks on AES, 2005. <http://cr.yp.to/antiforgery/cachetiming-20050414.pdf>.
- [Ber05b] Daniel J. Bernstein. The Poly1305-AES Message-Authentication Code. In *FSE*, volume 3557 of *Lecture Notes in Computer Science*, pages 32–49. Springer, 2005. <http://www.iacr.org/cryptodb/archive/2005/FSE/2992/2992.pdf>.
- [BFK09] Jens Bender, Marc Fischlin, and Dennis Kügler. Security Analysis of the PACE Key-Agreement Protocol. In Pierangela Samarati, Moti Yung, Fabio Martinelli, and Claudio Ardagna, editors, *Information Security*, volume 5735 of *Lecture Notes in Computer Science*, pages 33–48. Springer, 2009.
- [BGK04] Johannes Blömer, Jorge Guajardo, and Volker Krummel. Provably Secure Masking of AES. In H. Handschuh and M. Anwar Hasan, editors, *Selected Areas in Cryptography (SAC 2004)*, number 3357 in *Lecture Notes in Computer Science*, pages 69–83. Springer-Verlag, 2004.
- [BGP⁺11] Lejla Batina, Benedikt Gierlichs, Emmanuel Prouff, Matthieu Rivain, François-Xavier Standaert, and Nicolas Veyrat-Charvillon. Mutual Information Analysis: a Comprehensive Study. *Journal of Cryptology*, 24(2):269–291, 2011.
- [BGS⁺05] Stephen C. Bono, Matthew Green, Adam Stubblefield, Ari Juels, Aviel D. Rubin, and Michael Szydlo. Security analysis of a cryptographically-enabled RFID device. In *Proceedings of the 14th conference on USENIX Security Symposium - Volume 14*. USENIX Association, 2005. <http://www.usenix.org/events/sec05/tech/bono/bono.pdf>.
- [BHM08] Michael Braun, Erwin Hess, and Bernd Meyer. Using Elliptic Curves on RFID Tags. In *International Journal of Computer Science and Network Security*, volume 8, February 2008.
- [BKR11] Andrey Bogdanov, Dmitry Khovratovich, and Christian Rechberger. Biclique Cryptanalysis of the Full AES. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, volume 7073 of *Lecture Notes in Computer Science*, pages 344–371. Springer, 2011.
- [BKZ07] Behördenkennzahlen für deutsche Personalausweise und Reisepässe, 2007. <http://www.pruefziffernberechnung.de/Begleitdokumente/BKZ.pdf>.
- [BMS11] William J. Broad, John Markoff, and David E. Sanger. Israeli Test on Worm Called Crucial in Iran Nuclear Delay. Technical report, New York Times, January

2011. <http://www.nytimes.com/2011/01/16/world/middleeast/16stuxnet.html>.
- [Bog07] Andrey Bogdanov. Attacks on the KeeLoq Block Cipher and Authentication Systems. In *Workshop on RFID Security (RFIDSec'08)*, 2007. rfidsec07.etsit.uma.es/slides/papers/paper-22.pdf.
- [Bos] Antoon Bosselaers. Fast Implementations on the Pentium. <http://homes.esat.kuleuven.be/~bosselae/fast.html>.
- [Bos98] Karl Bosch. *Statistik-Taschenbuch*. Oldenbourg, January 1998.
- [Bou00] Brian T. Boulter. Digital Filter Design - Writing Difference Equations For Digital Filters. *Applied Industrial Control Solutions case studies*, 2000. http://www.apicsllc.com/apics/Sr_3/Sr_3.htm.
- [BP08] Andrey Bogdanov and Christof Paar. On the security and efficiency of real-world lightweight authentication protocols, 2008.
- [BPT10] Alessandro Barenghi, Gerardo Pelosi, and Yannick Tégli. Improving first order differential power attacks through digital signal processing. In *ACM-SIGSAC International Conference on Security of Information and Networks*, pages 124–133. ACM, 2010.
- [BS90] Eli Biham and Adi Shamir. Differential Cryptanalysis of DES-like Cryptosystems. In *CRYPTO*, volume 537 of *Lecture Notes in Computer Science*, pages 2–21. Springer, 1990.
- [BS97] Eli Biham and Adi Shamir. Differential Fault Analysis of Secret Key Cryptosystems. In *CRYPTO*, volume 1294 of *Lecture Notes in Computer Science*, pages 513–525. Springer, 1997.
- [BSI] BSI - German Ministry of Security. ePass - Der Reisepass mit biometrischen Merkmalen. https://www.bsi.bund.de/cln_156/DE/Themen/ElektronischeAusweise/TRundSchutzprofile/trundschutzprofile_node.html.
- [BSI01] Bundesamt für Sicherheit in der Informationstechnik BSI. AIS 20: Funktionalitätsklassen und Evaluationsmethodologie für deterministische Zufallszahlengeneratoren. Anwendungshinweise und Interpretationen zum Schema (AIS) Version 1, 2.12.1999, BSI, 2001. Available at <http://www.bsi.bund.de/zertifiz/zert/interpr/ais20.pdf>.
- [BSI08] BSI. Messung der Abstrahleigenschaften von RFID-Systemen, Version 2.05. Technical report, Bundesamt für Sicherheit in der Informationstechnik, 2008. <https://www.bsi.bund.de/ContentBSI/Themen/Elekausweise/rfid/MarsStudie/marsstudie.html>.

-
- [Bun11] Bundeskriminalamt. Anzahl der Diebstähle von Kraftwagen in Deutschland von 1997 bis 2010, 2011. <http://de.statista.com/statistik/daten/studie/1261/umfrage/kfz-diebstaehle-in-deutschland/>.
- [BW99] Alex Biryukov and David Wagner. Slide Attacks. In *FSE*, volume 1636 of *Lecture Notes in Computer Science*, pages 245–259. Springer, 1999.
- [Car05] Dario Carluccio. Electromagnetic Side Channel Analysis for Embedded Crypto Devices. Master’s thesis, Chair for Communication Security at the Ruhr University Bochum, 2005. Diploma thesis.
- [Car06] Bruce Carter. High-speed notch filters. *Analog Applications Journal*, 2006. <http://focus.tij.co.jp/jp/lit/an/slyt235/slyt235.pdf>.
- [CBB08] Nicolas T. Courtois, Gregory V. Bard, and Andrey Bogdanov. Periodic ciphers with small blocks and cryptanalysis of keeloq. In *Tatra Mountains Mathematical Publications*, 2008.
- [CBW08] Nicolas T. Courtois, Gregory V. Bard, and David Wagner. Algebraic and Slide Attacks on KeeLoq. In *FSE 2008*, volume 5086 of *Lecture Notes in Computer Science*, pages 97–115. Springer, 2008.
- [CCD00a] Christophe Clavier, Jean-Sébastien Coron, and Nora Dabbous. Differential Power Analysis in the Presence of Hardware Countermeasures. In *CHES 2000*, volume 1965 of *Lecture Notes in Computer Science*, pages 252–263. Springer, 2000.
- [CCD00b] Christophe Clavier, Jean-Sébastien Coron, and Nora Dabbous. Differential Power Analysis in the Presence of Hardware Countermeasures. In Christof Paar and Çetin Kaya Koç, editors, *Cryptographic Hardware and Embedded Systems - CHES 2000*, volume 1965 of *Lecture Notes in Computer Science*, pages 252–263. Springer-Verlag, 2000.
- [CGPR08] Jean-Sébastien Coron, Christophe Giraud, Emmanuel Prouff, and Matthieu Rivain. Attack and Improvement of a Secure S-Box Calculation Based on the Fourier Transform. In *Cryptographic Hardware and Embedded Systems - CHES 2008*, volume 5154 of *Lecture Notes in Computer Science*, pages 1–14. Springer-Verlag, 2008.
- [Cha82] David Chaum. Blinding Signatures for Untraceable Payments. In David Chaum, Ronald L. Rivest, and Alan T. Sherman, editors, *Advances in Cryptology - CRYPTO 1982*, pages 199–203, 1982.
- [CLP05] Dario Carluccio, Kerstin Lemke, and Christof Paar. Electromagnetic Side Channel Analysis of a Contactless Smart Card: First Results. In *Workshop on RFID and Lightweight Crypto (RFIDSec’05)*, Graz, Austria, July 2005. Ecrypt. <http://events.iaik.tugraz.at/RFIDandLightweightCrypto05/RFID-SlidesandProceedings/Carluccio-EMSideChannel.pdf>.

- [CLRPS] Dario Carluccio, Kerstin Lemke-Rust, Christof Paar, and Ahmad-Reza Sadeghi. E-Passport: The Global Traceability or How to Feel Like an UPS Package. In *Workshop on Information Security Applications (WISA 2006)*, volume 4298 of *Lecture Notes in Computer Science*, pages 391–404. Springer.
- [CNO08] Nicolas T. Courtois, Karsten Nohl, and Sean O’Neil. Algebraic Attacks on the Crypto-1 Stream Cipher in MiFare Classic and Oyster Cards. Cryptology ePrint Archive, Report 2008/166, 2008.
- [CO08] Nicolas T. Courtois and Sean O’Neil. FSE Rump Session – HITAG2 Cipher, 2008. <http://fse2008rump.cr.yp.to/00564f75b2f39604dc204d838da01e7a.pdf>.
- [COQ09] Nicolas T. Courtois, Sean O’Neil, and Jean-Jacques Quisquater. Practical Algebraic Attacks on the Hitag2 Stream Cipher. In *12th International Conference on Information Security (ISC ’09)*, pages 167–176, Berlin, Heidelberg, 2009. Springer-Verlag.
- [Cor99] Jean-Sébastien Coron. Resistance against Differential Power Analysis for Elliptic Curve Cryptosystems. In *CHES*, volume 1717 of *Lecture Notes in Computer Science*, pages 292–302. Springer, 1999.
- [Cou09] Nicolas Courtois. The Dark Side of Security by Obscurity - and Cloning MiFare Classic Rail and Building Passes, Anywhere, Anytime. In *SECRYPT*, pages 331–338. INSTICC, 2009.
- [Cra08] Crpto1. Open Implementation of CRYPTO1. 2008.
- [CRR02] Suresh Chari, Josyula R. Rao, and Pankaj Rohatgi. Template Attacks. In *CHES 2002*, volume 2523 of *Lecture Notes in Computer Science*, pages 13–28, London, UK, August 2002. Springer-Verlag.
- [CS10] Tom Chothia and Vitaliy Smirnov. A traceability attack against e-passports. In *Financial Cryptography and Data Security*, volume 6052 of *Lecture Notes in Computer Science*, pages 20–34. Springer, 2010.
- [Deu] Statistisches Bundesamt Deutschland. GENESIS-Online - Das statistische Informationssystem. <https://www-genesis.destatis.de/genesis/online/logon>.
- [dKGGH08] Gerhard de Koning Gans, Jaap-Henk Hoepman, and Flavio D. Garcia. A Practical Attack on the MIFARE Classic. In *CARDIS*, volume 5189 of *Lecture Notes in Computer Science*, pages 267–282. Springer, 2008.
- [Dor82] Dorothy Elizabeth Robling Denning. Cryptography and Data Security. In *ACM Classic Books Series*. Addison-Wesley, 1982.
- [DR02] Joan Daemen and Vincent Rijmen. *The Design of Rijndael, AES — The Advanced Encryption Standard*. Springer-Verlag, 2002.

- [DS09] Itai Dinur and Adi Shamir. Cube Attacks on Tweakable Black Box Polynomials. In *EUROCRYPT*, volume 5479 of *Lecture Notes in Computer Science*, pages 278–299. Springer, 2009.
- [DSB] Defense Science Board. <http://www.acq.osd.mil/dsb/>.
- [ECM] Standards ECMA-340 and ECMA-352 for the Near Field Communication Interface and Protocol. <http://www.ecma-international.org>.
- [EKM⁺08] Thomas Eisenbarth, Timo Kasper, Amir Moradi, Christof Paar, Mahmoud Salmasizadeh, and Mohammad T. Manzuri Shalmani. On the Power of Power Analysis in the Real World: A Complete Break of the KeeLoq Code Hopping Scheme. In *Advances in Cryptology - CRYPTO 2008*, volume 5157 of *Lecture Notes in Computer Science*, pages 203–220. Springer, 2008.
- [EKPI11] Thomas Eisenbarth, Timo Kasper, Christof Paar, and Sebastian Indestege. KeeLoq. In Henk C. A. van Tilborg and Sushil Jajodia, editors, *Encyclopedia of Cryptography and Security*. Springer, September 2011.
- [EM 03] EM Microelectronic. EM4094 fact sheet, 2003. http://www.emmicroelectronics.com/webfiles/product/rfid/ds/EM4094_fs.pdf.
- [EM 05] EM Microelectronic. EM4102 Data Sheet, 2005. www.hedam.de/PDF/EM4102.pdf.
- [EP] Eric Perraud. RF cryptotransponder reader increases vehicle- and building-access security. <http://www.edn.com/file/22897-45988.pdf?force=true>, <http://www.edn.com/file/22897-45988.pdf>.
- [EPC10] EPCGlobal GS1. EPC Tag Data Standard 1.5, August 2010. <http://www.gs1.org/gsmp/kc/epcglobal/tds>.
- [Eri07] Eric Peeters and François-Xavier Standaert and Jean-Jacques Quisquater. Power and Electromagnetic Analysis: Improved Model, Consequences and Comparisons. volume 40, pages 52–60. Elsevier, 2007.
- [eu9] Richtlinie 95/56/EG: Sicherungseinrichtung gegen unbefugte Benutzung von Kraftfahrzeugen. <http://eur-lex.europa.eu/LexUriServ/LexUriServ.do?uri=CONSLEG:1995L0056:19951219:DE:PDF>.
- [Eur06] European Commission. EU - Passport Specification. http://ec.europa.eu/justice_home/doc_centre/freetravel/documents/doc/c_2006_2909_en.pdf, June 2006.
- [Fai] Fairchild Semiconductors. Application note 313: DC electrical characteristics of MM74HC high speed logic. <http://www.fairchildsemi.com/an/AN/AN-313.pdf>.
- [FDv10] Aurelien Francillon, Boris Danev, and Srdjan Čapkun. Relay Attacks on Passive Keyless Entry and Start Systems in Modern Cars. Cryptology ePrint Archive, Report 2010/332, 2010. <http://eprint.iacr.org/>.

- [Fin02] Klaus Finkenzeller. *RFID-Handbuch*. Hanser Fachbuchverlag, Third edition, October 2002.
- [Fin03] Klaus Finkenzeller. *RFID Handbook: Fundamentals and Applications in Contactless Smart Cards and Identification*. John Wiley and Sons, 2nd edition, 2003.
- [FIP01] FIPS-197. *Advanced Encryption Standard (AES)*. Federal Information Processing Standards Publication 197, November 2001. Available at <http://csrc.nist.gov/>.
- [FJ07] Matteo Frigo and Steven G. Johnson. FFTW: Fast C routines for computing the Discrete Fourier Transform. Web resource, 2007. <http://www.fftw.org/>.
- [FK] Thomas Finke and Harald Kelter. Radio Frequency Identification - Abhörmöglichkeiten der Kommunikation zwischen Lesegerät und Transponder am Beispiel eines ISO14443-Systems. http://www.bsi.de/fachthem/rfid/Abh_RFID.pdf. BSI - Bundesamt für Sicherheit in der Informationstechnik <https://www.bsi.bund.de/ContentBSI/Themen/Eldekaeweise/rfid/Whitepaper/whitepaper.html>.
- [FL93] Walter Fumy and Peter Landrock. Principles of Key Management. *IEEE Journal on Selected Areas in Communications*, 11(5):785–793, 1993.
- [FOfIS] Germany Federal Office for Information Security. Advanced Security Mechanisms for Machine Readable Travel Documents – Extended Access Control. http://www.bsi.de/fachthem/epass/EACTR03110_v110.pdf.
- [FTD] FTDI. FT245 USB chip data sheet. http://www.ftdichip.com/Documents/DataSheets/DS_FT245R_v105.pdf.
- [Fut] Future Technology Devices International Ltd.. FT245R Datasheet. http://www.ftdichip.com/Support/Documents/DataSheets/ICs/DS_FT245R.pdf.
- [Gam83] Taher El Gamal. A subexponential-time algorithm for computing discrete logarithms over $gf(p^2)$. In *CRYPTO*, pages 275–292, 1983.
- [GdKGM⁺08] Flavio D. Garcia, Gerhard de Koning Gans, Ruben Muijers, Peter van Rossum, Roel Verdult, Ronny Wichers Schreur, and Bart Jacobs. Dismantling MIFARE Classic. In *ESORICS*, volume 5283 of *Lecture Notes in Computer Science*, pages 97–114. Springer, 2008.
- [Gie06] Thomas Giesler. State of the Art Car Access Security Systems, march 2006. The First International Workshop on IT-Solutions for Physical Security. http://www.sics.se/phys-sec/2006/phys-sec_giesler.pdf.
- [GKN⁺08] Tim Güneysu, Timo Kasper, Martin Novotný, Christof Paar, and Andy Rupp. Cryptanalysis with COPACOBANA. *IEEE Transactions on Computers*, 57(11):1498–1513, 2008.

- [GLRP06] Benedikt Gierlichs, Kerstin Lemke-Rust, and Christof Paar. Templates vs. Stochastic Methods. In Louis Goubin and Mitsuru Matsui, editors, *Proceedings of CHES 2006*, volume 4249 of *Lecture Notes in Computer Science*, pages 15–29. Springer-Verlag, 2006.
- [GT03] Jovan Dj. Golic and Christophe Tymen. Multiplicative Masking and Power Analysis of AES. In Burton S. Kaliski Jr., Çetin Kaya Koç, and Christof Paar, editors, *Proceedings of CHES 2002*, volume 2523 of *Lecture Notes in Computer Science*, pages 198–212. Springer-Verlag, 2003.
- [GvRVS09] Flavio D. Garcia, Peter van Rossum, Roel Verdult, and Ronny Wichers Schreur. Wirelessly Pickpocketing a Mifare Classic Card. In *IEEE Symposium on Security and Privacy*, pages 3–15. IEEE, 2009.
- [Ham] Tobias Hammer. Hterm.
<http://www.der-hammer.info/terminal>.
- [Han04] Helena Handschuh. Contactless Technology Security Issues. Information security bulletin, volume 9, 2004. <http://www.chi-publishing.com/samples/ISBN0903HH.pdf>.
- [Han05] Gerhard Hancke. A practical relay attack on ISO 14443 proximity cards, 2005. <http://www.cl.cam.ac.uk/~gh275/relay.pdf>.
- [Han06] Gerhard P. Hancke. Practical Attacks on Proximity Identification Systems (Short Paper). In *IEEE Symposium on Security and Privacy 2006*, 2006. <http://www.cl.cam.ac.uk/~gh275/SPPPractical.pdf>.
- [Han08] G.P. Hancke. Eavesdropping Attacks on High-Frequency RFID Tokens. In *Workshop on RFID Security*, 2008.
- [Hay83] Simon Haykin. *Communications Systems*, chapter 8. Wiley, 2nd edition, 1983.
- [HB] Ernst Haselsteiner and Klemens Breitfuß. Security in near field communication. In *Workshop on RFID Security (RFIDSec'06)*.
- [Hei] Bundestag verabschiedet Novelle des Passgesetzes. <http://www.heise.de/newsticker/meldung/90202>.
- [Hel80] M. Hellman. A Cryptanalytic Time-Memory Tradeoff. *IEEE Transactions on Information Theory*, 26(4):401–406, 1980.
- [HHBR⁺08] Daniel Halperin, Thomas S. Heydt-Benjamin, Benjamin Ransford, Shane S. Clark, Benessa Defend, Will Morgan, Kevin Fu, Tadayoshi Kohno, and William H. Maisel. Pacemakers and Implantable Cardiac Defibrillators: Software Radio Attacks and Zero-Power Defenses. In *Proceedings of the 2008 IEEE Symposium on Security and Privacy*, pages 129–142. IEEE Computer Society, 2008.

- [HHJ⁺06] Jaap-Henk Hoepman, Engelbert Hubbers, Bart Jacobs, Martijn Oostdijk, and Ronny Wichers Schreur. Crossing Borders: Security and Privacy Issues of the European e-Passport. In Hiroshi Yoshiura, Kouichi Sakurai, Kai Rannenberg, Yuko Murayama, and Shin ichi Kawamura, editors, *First International Workshop on Security (IWSEC 2006)*, volume 4266 of *Lecture Notes in Computer Science*, pages 152–167. Springer, 2006.
- [HID02] HID Global. 200X iCLASS Card Data Sheet. http://www.hidglobal.com/documents/iclass_card_ds_en.pdf, 2002.
- [HJM05] Martin Hell, Thomas Johansson, and Willi Meier. Grain - A Stream Cipher for Constrained Environments. eSTREAM, ECRYPT Stream Cipher. Technical report, 2005/010, ECRYPT (European Network of Excellence for Cryptology), 2005.
- [HK05] Gerhard P. Hancke and Markus G. Kuhn. An RFID Distance Bounding Protocol. In *Proceedings of IEEE/Create-Net SecureComm 2005*, pages 67–73. IEEE Computer Society Press, 2005.
- [HMF07] Michael Hutter, Stefan Mangard, and Martin Feldhofer. Power and EM Attacks on Passive 13.56 MHz RFID Devices. In Pascal Paillier and Ingrid Verbauwhede, editors, *Cryptographic Hardware and Embedded Systems - CHES 2007*, LNCS 4727, pages 320 – 330. Springer, 2007.
- [HMS00] Erwin Hess, Bernd Meyer, and Torsten Schütze. Information Leakage Attacks Against Smart Card Implementations of Cryptographic Algorithms and Countermeasures – A Survey. In *EUROSMART Security Conference*, pages 55–64, 2000.
- [Hoh09] Andreas Hoheisel. Side-Channel Analysis Resistant Implementation of AES on Automotive Processors. Master’s thesis, Chair for Embedded Security at the Ruhr University Bochum, 2009. Master thesis, http://emsec.rub.de/media/crypto/attachments/files/2010/04/ms_hoheisel.pdf.
- [HOM06a] Christoph Herbst, Elisabeth Oswald, and Stefan Mangard. An AES Smart Card Implementation Resistant to Power Analysis Attacks. In Jianying Zhou, Moti Yung, and Feng Bao, editors, *Proceedings of ACNS 2006*, volume 3989 of *Lecture Notes in Computer Science*, pages 239–255. Springer-Verlag, 2006.
- [HOM06b] Christoph Herbst, Elisabeth Oswald, and Stefan Mangard. An AES Smart Card Implementation Resistant to Power Analysis Attacks. In *ACNS 2006*, volume 3989 of *Lecture Notes in Computer Science*, pages 239–252. Springer, 2006.
- [HPS99] Helena Handschuh, Pascal Paillier, and Jacques Stern. Probing Attacks on Tamper-Resistant Devices. In *CHES*, pages 303–315, 1999.
- [IAI] IAIK Graz. HF Demo Tag. http://www.iaik.tugraz.at/content/research/rfid/tag_emulators.

- [ICA03] ICAO. Machine Readable Travel Documents, Doc 9303, Part 1 Machine Readable Passports, Fifth Edition, 2003.
- [ICA04] ICAO. Machine Readable Travel Documents, PKI for Machine Readable Travel Documents offering ICC Read-Only Access, Technical Report, 2004. <http://www.mrtd.icao.int>.
- [ICA05] ICAO. Machine Readable Travel Documents, Supplement to Doc9303-part1-sixth edition, 2005.
- [id06] ACG id. HF Multi ISO RFID Reader User Manual, 2006. http://www.rfid-webshop.com/shop/download/Reader/HF%2013.56%20MHz/ACG/Multi%20ISO/TAGnology_acg_contactless_flyer_neu_ACG_1356_HFMultiManual.pdf.
- [IKD⁺08] Sebastiaan Indesteege, Nathan Keller, Orr Dunkelman, Eli Biham, and Bart Preneel. A Practical Attack on KeeLoq. In *EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 1–18. Springer, 2008.
- [Inn05] Innovatron. Calypso Functional Card Application v 1.3. http://www.calypsotechnology.net/index.php?option=com_docman&task=doc_download&gid=3&Itemid=40, 2005.
- [Inta] International Organization for Standardization (ISO). ISO/IEC 15693: Contactless integrated circuit cards - Vicinity cards.
- [Intb] International Rectifier. Data sheet for IRFD110 N-channel MOSFET. <http://www.irf.com/product-info/datasheets/data/irfd110.pdf>.
- [Int01] International Organization for Standardization (ISO). ISO/IEC 14443: Identification Cards - Contactless Integrated Circuit(s) Cards - Proximity Cards - Part 1-4, 2001. www.iso.ch.
- [Int04a] International Organization for Standardization (ISO). ISO 7816 Identification Cards - Integrated Circuit Cards with Contacts, 2004. www.iso.ch.
- [Int04b] International Organization for Standardization (ISO). ISO/IEC 18092: Near Field Communication - Interface and Protocol (NFCIP-1), 2004. www.iso.ch.
- [JMA] JMA. Transponder 2010. http://www.jma-argentina.com.ar/archivos/JMA_TRANSPONDER_v_37.pdf.
- [JMV07] S. Vaudenay J. Monnerat and M. Vuagnoux. About Machine-Readable Travel Documents. In *Workshop on RFID Security (RFIDSec'07)*, pages 15–28, 2007.
- [JMW05a] A. Juels, D. Molnar, and D. Wagner. Security and Privacy Issues in E-passports. Cryptology ePrint Archive, Report 2005/095, 2005. <http://eprint.iacr.org/2005/095.pdf>.

- [JMW05b] Ari Juels, David Molnar, and David Wagner. Security and privacy issues in e-passports. In *Security and Privacy for Emerging Areas in Communications Networks, 2005. SecureComm 2005.*, pages 74–88. IEEE, September 2005.
- [JPS05] Marc Joye, Pascal Paillier, and Berry Schoenmakers. On Second-Order Differential Power Analysis. In *CHES 2005: Revised Papers from the 7th International Workshop on Cryptographic Hardware and Embedded Systems*, volume 3659 of *Lecture Notes in Computer Science*, pages 293–308. Springer-Verlag, 2005.
- [JR94] Burton S. Kaliski Jr. and Matthew J. B. Robshaw. Linear cryptanalysis using multiple approximations. In *CRYPTO*, volume 839 of *Lecture Notes in Computer Science*, pages 26–39. Springer, 1994.
- [JRS03] Ari Juels, Ronald L. Rivest, and Michael Szydlo. The blocker tag: selective blocking of RFID tags for consumer privacy. In *Proceedings of the 10th ACM Conference on Computer and Communications Security (CCS)*, pages 103–111, 2003.
- [Kas06] Timo Kasper. Embedded Security Analysis of RFID Devices. Master’s thesis, Ruhr Universität Bochum, July 2006. http://www.emsec.rub.de/media/crypto/attachments/files/2010/04/timo_kasper__embedded_security_analysis_of_rfid_devices.pdf.
- [KCP07] Timo Kasper, Dario Carluccio, and Christof Paar. An Embedded System for Practical Security Analysis of Contactless Smartcards. In *Workshop in Information Security Theory and Practice (WISTP 2007)*, volume 4462 of *Lecture Notes in Computer Science*, pages 150–160. Springer, 2007. Customized RFID Reader — GNU General Public License project <http://sourceforge.net/projects/reader14443>.
- [Ker83] Auguste Kerckhoffs. La cryptographie militaire. In *Journal des sciences militaires*, volume IX, pages 5–83, Jan. 1883.
- [KJJ99] P. Kocher, J. Jaffe, and B. Jun. Differential Power Analysis. In M. J. Wiener, editor, *Proceedings of CRYPTO 1999*, volume 1666 of *Lecture Notes in Computer Science*, pages 388–397. Springer, 1999.
- [KK05] G.S. Kc and P.A. Karger. Security and Privacy Issues in Machine Readable Travel Documents (MRTDs). RC 23575, IBM T. J. Watson Research Labs, April 2005.
- [KK06] Karl-Dirk Kammeyer and Kristian Kroschel. *Digitale Signalverarbeitung*. Teubner Verlag, 6th edition, 2006.
- [KKMP09] Markus Kasper, Timo Kasper, Amir Moradi, and Christof Paar. Breaking KeeLoq in a Flash: On Extracting Keys at Lightning Speed. In *Progress in Cryptology - AFRICACRYPT 2009*, volume 5580 of *Lecture Notes in Computer Science*, pages 403–420. Springer, 2009.

- [KNS⁺08] Dennis Kuegler, Heike Neumann, Sebastian Stappert, Markus Ullmann, and Matthias Voegeler. Password Authenticated Key Agreement for Contactless Smart Cards. In *Workshop on RFID Security (RFIDSec'08)*, Budapest, Hungary, July 2008.
- [Knu98] Donald E. Knuth. *The Art of Computer Programming*, volume 2: Seminumerical Algorithms. Addison-Wesley, Boston, 3rd edition, 1998.
- [Koc96] P. Kocher. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems. In N. Kobitz, editor, *Proceedings of CRYPTO 1996*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer-Verlag, 1996.
- [KOP09] Timo Kasper, David Oswald, and Christof Paar. EM Side-Channel Attacks on Commercial Contactless Smartcards using Low-Cost Equipment. In *WISA 2009*, volume 5932 of *Lecture Notes in Computer Science*, pages 79–93. Springer, 2009.
- [KOP10] Timo Kasper, David Oswald, and Christof Paar. A Versatile Framework for Implementation Attacks on Cryptographic RFIDs and Embedded Devices. In Marina L. Gavrilova, Chih Jeng Kenneth Tan, and Edward D. Moreno, editors, *Transactions on Computational Science X - Special Issue on Security in Computing*, volume 6340 of *Lecture Notes in Computer Science*, pages 100–130. Springer, 2010.
- [KOP11a] Timo Kasper, David Oswald, and Christof Paar. Security of Wireless Embedded Devices in the Real World. In Norbert Pohlmann, Helmut Reimer, and Wolfgang Schneider, editors, *Information Security Solutions (ISSE) 2011 - Securing Electronic Business Processes*, pages 174–189. Vieweg+Teubner, 2011. <http://www.viewegteubner.de/Buch/978-3-8348-1911-6/ISSE-2011-Securing-Electronic-Business-Processes.html>.
- [KOP11b] Timo Kasper, David Oswald, and Christof Paar. Side-Channel Analysis of Cryptographic RFIDs with Analog Demodulation. In Ari Juels and Christof Paar, editors, *7th Workshop on RFID Security (RFIDsec 2011)*, volume 7055 of *Lecture Notes in Computer Science*, pages 61–77. Springer, 2011.
- [KOP11c] Timo Kasper, David Oswald, and Christof Paar. Wireless security threats: Eavesdropping and detecting of active RFIDs and remote controls in the wild. In *19th International Conference on Software, Telecommunications and Computer Networks (SoftCOM)*. IEEE, 2011. <http://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6064450&isnumber=6064354>.
- [KPP⁺06a] S. Kumar, C. Paar, J. Pelzl, G. Pfeiffer, A. Rupp, and M. Schimmler. How to break DES for 8,980. In *International Workshop on Special-Purpose Hardware for Attacking Cryptographic Systems — SHARCS'06, Cologne, Germany*, April 2006.
- [KPP⁺06b] Sandeep Kumar, Christof Paar, Jan Pelzl, Gerd Pfeiffer, Andy Rupp, and Manfred Schimmler. How to Break DES for € 8,980. In *Special-purpose Hardware*

- for Attacking Cryptographic Systems (SHARCS'06)*, pages 17–35, 2006. http://www.hyperelliptic.org/tanja/SHARCS/talks06/copa_sharcs.pdf.
- [KPP⁺06c] Sandeep Kumar, Christof Paar, Jan Pelzl, Gerd Pfeiffer, and Manfred Schimmler. Breaking Ciphers with COPACOBANA - A Cost-Optimized Parallel Code Breaker. In Louis Goubin and Mitsuru Matsui, editors, *Cryptographic Hardware and Embedded Systems (CHES 2006)*, volume 4249 of *Lecture Notes in Computer Science*, pages 101–118. Springer, 2006.
- [KSP10] Timo Kasper, Michael Silbermann, and Christof Paar. All You Can Eat or Breaking a Real-World Contactless Payment System. In *Financial Cryptography 2010*, volume 6052 of *Lecture Notes in Computer Science*, pages 343–350. Springer, 2010.
- [Kug03] Thomas Kugelstadt. *Op Amps for Everyone*, chapter 16 - Active Filter Design Techniques. Texas Instruments, 2nd edition, 2003.
- [KvMOP11] Timo Kasper, Ingo von Maurich, David Oswald, and Christof Paar. Chameleon: A Versatile Emulator for Contactless Smartcards. In *ICISC 2010, Seoul, Korea*, volume 6829 of *Lecture Notes in Computer Science*, pages 189–206. Springer, 2011. Chameleon — GNU General Public License project <http://sourceforge.net/projects/chameleon14443>.
- [KW05] Ziv Kfir and Avishai Wool. Picking virtual pockets using relay attacks on contactless smartcard systems. Cryptology ePrint Archive, Report 2005/052, 2005. <http://eprint.iacr.org>.
- [KW06] Ilan Kirschenbaum and Avishai Wool. How to Build a Low-Cost, Extended-Range RFID Skimmer. In *USENIX Security Symposium*. USENIX Association, 2006.
- [Lan] Langer EMV-Technik. Details of Near Field Probe Set RF 2. web resource. <http://www.langer-emv.de/en/products/disturbance-emission/near-field-probes/rf-2/>.
- [LC01] Jeremy Landt and Barbara Catlin. Shrouds of Time: The history of RFID, 2001. http://www.transcore.com/pdf/AIM%20shrouds_of_time.pdf.
- [Lea] Gregor Leander. Kryptographie I (Lecture Notes). <http://caccioppoli.mac.rub.de/website/teachingmaterial/k1-ws0607/SkriptKryptoI.pdf>.
- [LEG] LEGIC Identsystems. LEGIC advant Crypto Transponder Chips. http://www.legic.com/en/legic_advant/transponder_chips.html.
- [Len87] H. Lenstra. Factoring Integers with Elliptic Curves. *Annals Math.*, 126:649–673, 1987.
- [Lin] LinearCorp. MegaCode and MultiCode Systems. http://www.linearcorp.com/radio_megacode.html.

- [LKG⁺09] Lang Lin, Markus Kasper, Tim Güneysu, Christof Paar, and Wayne Burleson. Trojan Side-Channels: Lightweight Hardware Trojans through Side-Channel Engineering. In Christophe Clavier and Kris Gaj, editors, *CHES*, volume 5747 of *Lecture Notes in Computer Science*, pages 382–395. Springer, 2009.
- [LKLRP07] Y. Liu, T. Kasper, K. Lemke-Rust, and C. Paar. E-Passport: Cracking Basic Access Control Keys. In *Proc. of OTM’07, Part II*, volume 4804 of *Lecture Notes in Computer Science*, pages 1531–1547. Springer, 2007.
- [LRP07] Kerstin Lemke-Rust and Christof Paar. Gaussian mixture models for higher-order side channel analysis. In *Cryptographic Hardware and Embedded Systems (CHES’07)*, volume 4727 of *Lecture Notes in Computer Science*, pages 14–27. Springer, 2007.
- [LSR06] Tobias Lohmann, Matthias Schneider, and Christoph Ruland. Analysis of power constraints for cryptographic algorithms in mid-cost RFID tags. In Josep Domingo-Ferrer, Joachim Posegga, and Daniel Schreckling, editors, *Smart Card Research and Advanced Applications*, volume 3928 of *Lecture Notes in Computer Science*, pages 278–288. Springer, 2006.
- [Mac03] David J.C. MacKay. *Information Theory, Inference and Learning Algorithms*. Cambridge University Press, September 2003.
- [man07] *Hitag2 stream cipher C implementation and graphical description*, 2006-2007. <http://cryptolib.com/ciphers/hitag2/>, as of June, 14th 2010.
- [Mas06] Massachusetts Bay Transportation Authorit. The Charlie Card Reusable Ticket System, 2006. http://www.mbta.com/fares_and_passes/charlie.
- [Mat93] Mitsuru Matsui. Linear Cryptanalysis Method for DES Cipher. In *EURO-CRYPT*, volume 765 of *Lecture Notes in Computer Science*, pages 386–397. Springer, 1993.
- [Mat94] Mitsuru Matsui. The First Experimental Cryptanalysis of the Data Encryption Standard. In *CRYPTO*, volume 839 of *Lecture Notes in Computer Science*, pages 1–11. Springer, 1994.
- [MBKP11] Amir Moradi, Alessandro Barenghi, Timo Kasper, and Christof Paar. On the Vulnerability of FPGA Bitstream Encryption against Power Analysis Attacks: Extracting keys from Xilinx Virtex-II FPGAs. In Yan Chen, George Danezis, and Vitaly Shmatikov, editors, *ACM Conference on Computer and Communications Security (CCS 2011)*, pages 111–124, 2011.
- [MDS99a] Thomas S. Messerges, Ezzy A. Dabbish, and Robert H. Sloan. Investigations of Power Analysis Attacks on Smartcards. In *Proceedings of the USENIX Workshop on Smartcard Technology*, pages 151–162. USENIX Association, 1999.
- [MDS99b] Thomas S. Messerges, Ezzy A. Dabbish, and Robert H. Sloan. Power Analysis Attacks of Modular Exponentiation in Smartcards. In *CHES 1999*, volume 1717 of *Lecture Notes in Computer Science*, pages 144–157. Springer, 1999.

- [Mel] Melexis. Application note: A power booster for the MLX90121. http://www.melexis.com/prodfiles/0003881_AN90121_4_1.pdf.
- [Mel99] M. Melito. Application Note 484/1293, Car Ignition with IGBTs, 1999.
- [Mes00] Thomas Messerges. Securing the AES Finalists Against Power Analysis Attacks. In Bruce Schneier, editor, *Proceedings of FSE 2000*, volume 1978 of *Lecture Notes in Computer Science*, pages 150–164. Springer-Verlag, 2000.
- [Mica] Microchip. An Introduction to KeeLoq Code Hopping. <http://ww1.microchip.com/downloads/en/AppNotes/91002a.pdf>.
- [Micb] Microchip. AN642: Code Hopping Decoder using a PIC16C56. <http://www.keeloq.boom.ru/decryption.pdf>.
- [Micc] Microchip. HCS200, KEELOQ Code Hopping Encoder. <http://ww1.microchip.com/downloads/en/DeviceDoc/40138c.pdf>.
- [Micd] Microchip. HCS201, KEELOQ Code Hopping Encoder. <http://ww1.microchip.com/downloads/en/DeviceDoc/41098c.pdf>.
- [Mice] Microchip. HCS300, KEELOQ Code Hopping Encoder. <http://ww1.microchip.com/downloads/en/DeviceDoc/21137f.pdf>.
- [Micf] Microchip. HCS301, KEELOQ Code Hopping Encoder. <http://ww1.microchip.com/downloads/en/DeviceDoc/21143b.pdf>.
- [Micg] Microchip. HCS410, KEELOQ Code Hopping Encoder and Transponder Data Sheet. <http://ww1.microchip.com/downloads/en/DeviceDoc/40158e.pdf>.
- [Mich] Microchip. HCS410/WM, KEELOQ Crypto Read/Write Transponder Module. <http://ww1.microchip.com/downloads/en/DeviceDoc/41116b.pdf>.
- [Mici] Microchip. PIC16C5X Data Sheet. <http://ww1.microchip.com/downloads/en/DeviceDoc/30453d.pdf>.
- [Micj] Microchip. PIC18FXX2 Data Sheet. <http://ww1.microchip.com/downloads/en/DeviceDoc/39564c.pdf>.
- [Mick] EM Microelectronic. *125kHz CRYPTO READ/WRITE Contactless Identification Device*. <http://pdf1.alldatasheet.com/datasheet-pdf/view/154686/EMMICRO/EM4170/+044798VwLLpETS+IyCYGv+/datasheet.pdf>.
- [Micl] EM Microelectronic. Swatch Group, parent of EM, acquires Sokymat SID GmbH. <http://www.emmicroelectronic.com/DetailNews.asp?IdNews=45>.
- [Micm] Microsoft. Visual Studio 2008 Professional. <http://www.microsoft.com/visualstudio>.
- [Mic05] EM Microelectronic. *EM4102 – Read Only Contactless Identification Device*, February 2005. http://www.emmicroelectronic.com/webfiles/Product/RFID/DS/EM4102_DS.pdf.

- [MK09] Amir Moradi and Timo Kasper. A New Remote Keyless Entry System Resistant to Power Analysis Attacks. In *7th International Conference on Information, Communications and Signal Processing (ICICS)*, pages 1062–1067. IEEE Press, 2009.
- [MLKS08] Gerald Madlmayr, Josef Langer, Christian Kantner, and Josef Scharinger. NFC Devices: Security and Privacy. In *ARES*, pages 642–647. IEEE Computer Society, 2008.
- [MM06] MiniMe and Mahajivana. The RFID-Zapper. In *22C3 – 22nd Chaos Communication Congress*, 2006. http://events.ccc.de/congress/2005/static/r/f/i/RFID-Zapper%28EN%29_77f3.html.
- [MM09] A. Theodore Marketos and Simon W. Moore. The Frequency Injection Attack on Ring-Oscillator-Based True Random Number Generators. In *Cryptographic Hardware and Embedded Systems - CHES 2009, 11th International Workshop, Lausanne, Switzerland, September 6-9, 2009, Proceedings*, volume 5747 of *Lecture Notes in Computer Science*, pages 317–331. Springer, 2009.
- [MOP07] Stefan Mangard, Elisabeth Oswald, and Thomas Popp. *Power Analysis Attacks: Revealing the Secrets of Smart Cards*. Springer, 2007. <http://www.dpabook.org>.
- [MPO05] Stefan Mangard, Norbert Pramstaller, and Elisabeth Oswald. Successfully Attacking Masked AES Hardware Implementations. In *Cryptographic Hardware and Embedded Systems - CHES 2005*, volume 3659 of *Lecture Notes in Computer Science*, pages 157–171. Springer, 2005.
- [MRT] Benefits of MRTD. <http://mrt.d.icao.int/content/view/28/203/>.
- [MRT04] ICAO TAG MRTD/NTWG. Biometrics Deployment of Machine Readable Travel Documents, Technical Report, 2004.
- [Mul09] Collin Mulliner. Vulnerability Analysis and Attacks on NFC-Enabled Mobile Phones. In *ARES*, pages 695–700, 2009.
- [MvOV97] Alfred Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography (HAC)*. CRC Press, Boca Raton, Florida, USA, 1997. <http://www.cacr.math.uwaterloo.ca/hac/>.
- [Nata] National Institute of Standards and Technology (NIST). Block Cipher Modes of Operation. <http://csrc.nist.gov/groups/ST/toolkit/BCM/index.html>.
- [Natb] National Security Agency (NSA). TEMPEST: A Signal Problem. Declassified September 2007. http://www.nsa.gov/public_info/_files/cryptologic_spectrum/tempest.pdf.
- [Natc] National Semiconductor. Datasheet for LM311 voltage comparator. <http://www.national.com/pf/LM/LM311.html#Datasheet>.

- [Nat99] National Institute of Standards and Technology (NIST). FIPS 46-3 Data Encryption Standard (DES), 1999. <http://csrc.nist.gov/publications/fips/fips46-3/fips46-3.pdf>.
- [NESP08] Karsten Nohl, David Evans, Starbug, and Henryk Plötz. Reverse-Engineering a Cryptographic RFID Tag. In Paul C. van Oorschot, editor, *USENIX Security Symposium*, pages 185–194, 2008. http://www.usenix.org/events/sec08/tech/full_papers/nohl/nohl.pdf.
- [NK09] Martin Novotný and Timo Kasper. Cryptanalysis of KeeLoq with COPA-COBANA. In *Workshop on Special Purpose Hardware for Attacking Cryptographic Systems (SHARCS'09)*, 2009.
- [NS06] Michael Neve and Jean-Pierre Seifert. Advances on Access-Driven Cache Attacks on AES. In *Selected Areas in Cryptography*, pages 147–162, 2006.
- [NXPa] NXP. *HT2 Transponder Family Communication Protocol – Revision 2.1*. <http://www.proxmark.org/files/index.php?dir=Documents%2F125+kHz+-+Hitag%2F&download=HT2protocol.pdf>.
- [NX Pb] NXP. *PCF7930 Programmable Identification Transponder (PIT)*. http://www.tvsat.com.pl/pdf/P/pcf7930_ph.pdf.
- [NX Pc] NXP. *Preliminary Specification PCF7952 Active Tag IC and Processor*. <http://hitag2.land.ru/f7952att.pdf>.
- [NX Pd] NXP. *Product Profile PCF7935AS*. http://www.nxp.com/acrobat_download2/other/identification/pcf7935-pp.pdf.
- [NX Pe] NXP. *Product Profile PCF7942*. http://www.nxp.com/acrobat_download2/other/identification/pcf7942-pp.pdf.
- [NX Pf] NXP. *Product Profile PCF7946*. http://www.nxp.com/acrobat_download2/other/identification/pcf7946-pp.pdf.
- [NX Pg] NXP. *Product Specification PCF7936 Security Transponder (HITAG2)*.
- [NX P95] NXP. Mifare Classic. <http://www.nxp.com>, 1995.
- [NX P97] NXP. Hitag2 Data Sheet, 1997. www.nxp.com/acrobat_download2/expired_datasheets/2MOA3-01.pdf.
- [NX P01] NXP. Hitag Data Sheet, 2001. www.nxp.com/documents/data_sheet/HTRC11001T.pdf.
- [NX P03] NXP. Mifare Ultralight MF0ICU1 Functional specification contactless single-trip ticket IC. http://www.nxp.com/acrobat/other/identification/M028634_MF0ICU1_Functional_Spec_V3.4.pdf, 2003.
- [NX P04a] NXP. *Mifare DESFire Short Form Specification MF3 IC D40*, 2004. http://www.nxp.com/acrobat_download/other/identification/SFS075530.pdf.

-
- [NXP04b] NXP. Short Form Specification SmartMX P5CC072, Secure PKI Smart Card Controller, 2004. http://www.classic.nxp.com/acrobat_download2/other/identification/096210.pdf.
- [NXP07] NXP. AN200701: ISO/IEC 14443 Eavesdropping and Activation Distance. Technical report, 2007.
- [NXP08a] NXP. Mifare Classic 1K MF1 IC S50 Functional Specification. http://www.nxp.com/acrobat_download/other/identification/M001053_MF1ICS50_rev5_3.pdf, 2008.
- [NXP08b] NXP. *Mifare Classic 4K MF1 IC S70 Functional Specification*. NXP, 2008. http://www.nxp.com/acrobat/other/identification/M043541_MF1ICS70_Fspec_rev4_1.pdf.
- [NXP08c] NXP. *Mifare Classic Mini MF1 IC S20 Functional Specification*. NXP, 2008. http://www.nxp.com/acrobat/other/identification/M132211_MF1ICS20_rev1_1.pdf.
- [NXP08d] NXP. Short Data Sheet Mifare DESFire EV1 MF3 IC D41. http://www.nxp.com/acrobat_download/datasheets/MF3ICD21_41_81_SDS_2.pdf, 2008.
- [NXP08e] NXP. *Short Data Sheet Mifare Plus*. NXP, 2008. <http://www.nxp.com/acrobat/literature/9397/75016624.pdf>.
- [NXP08f] NXP. *Short Data Sheet Mifare Ultralight C*. NXP, 2008. http://www.nxp.com/acrobat_download/literature/9397/75016620.pdf.
- [NXP08g] NXP Semiconductors. *Transponder Evaluation and Development Kit, 2nd Generation*, april 2008. <http://www1.futureelectronics.com/doc/NXP/OM6716,599.pdf>, as of November 5, 2012.
- [NXP09a] NXP. MIFARE Type Identification Procedure, 2009. http://www.nxp.com/acrobat_download/applicationnotes/AN10833_31.pdf.
- [NXP09b] NXP Semiconductors. Holding Statement – Hitag Attack, april 2009. http://www.nxp.com/acrobat_download2/other/identification/HITAG_HoldingStatement_QAs_010409_v1.pdf.
- [NXP10a] NXP. Mifare Ultralight Data Sheet. http://www.mifare.net/index.php/download_file/view/5/, 2010.
- [NXP10b] NXP Semiconductors. The success of MIFARE, February 2010. <http://www.mifare.net/>.
- [Oct97] Octopus. Octopus Card in Hong Kong, 1997. <http://www.octopuscards.com/consumer/products/en/index.jsp>.
- [Oec03] P. Oechslin. Making a Faster Cryptanalytic Time-Memory Trade-Off. In *Proc. of CRYPTO'03*, volume 2729 of *Lecture Notes in Computer Science*, pages 617–630. Springer, 2003.

- [O'F] Colin O'Flynn. Downloading, Installing and Configuring WinAVR. http://winavr.sourceforge.net/install_config_WinAVR.pdf.
- [OKA08] OKAWA Electric Design. RLC Band-Stop Filter Design Tool. Web resource, 2008. <http://sim.okawa-denshi.jp/en/RLCbekeisan.htm>.
- [OMP04] Elisabeth Oswald, Stefan Mangard, and Norbert Pramstaller. Secure and Efficient Masking of AES – a Mission Impossible? Cryptology ePrint Archive, Report 2004/134, 2004. <http://eprint.iacr.org/>.
- [OMPR05] Elisabeth Oswald, Stefan Mangard, Norbert Pramstaller, and Vincent Rijmen. A Side-Channel Analysis Resistant Description of the AES S-Box. In Henri Gilbert and Helena Handschuh, editors, *Fast Software Encryption (FSE 2005)*, volume 3557 of *Lecture Notes in Computer Science*, pages 413–423. Springer, 2005.
- [ÖOP03] Siddika Berna Örs, Elisabeth Oswald, and Bart Preneel. Power-Analysis Attacks on an FPGA - First Experimental Results. In *Cryptographic Hardware and Embedded Systems (CHES'03)*, volume 2779 of *Lecture Notes in Computer Science*, pages 35–50. Springer, 2003.
- [Ope] OpenPICC. Programmable RFID-tag. <http://www.openpcd.org>.
- [Org04a] International Civil Aviation Organization. Annex I, Use of Contactless Integrated Circuits In Machine Readable Travel Documents, 2004. <http://www.icao.int/mrtd>.
- [Org04b] International Civil Aviation Organization. Machine Readable Travel Documents, Technical Report, Development of a Logical Data Structure - LDS For Optional Capacity Expansion Technologies, 2004. <http://www.icao.int/mrtd>.
- [Org05] International Civil Aviation Organization. Machine Readable Travel Documents, Supplement to Doc9303-part1-sixth edition, 2005. <http://www.icao.int/mrtd>.
- [OS] Yossi Oren and Adi Shamir. Power Analysis of RFID Tags. <http://www.wisdom.weizmann.ac.il/~yossio/rfid/>.
- [OS05a] Elisabeth Oswald and Kai Schramm. An Efficient Masking Scheme for AES Software Implementations. In JooSeok Song, Taekyoung Kwon, and Moti Yung, editors, *Proceedings of WISA 2005*, number 3786 in *Lecture Notes in Computer Science*, pages 292–305. Springer-Verlag, 2005.
- [OS05b] Elisabeth Oswald and Kai Schramm. An Efficient Masking Scheme for AES Software Implementations. In JooSeok Song, Taekyoung Kwon, and Moti Yung, editors, *Proceedings of WISA 2005*, number 3786 in *Lecture Notes in Computer Science*, pages 292–305. Springer-Verlag, 2005.
- [OS07] Yossef Oren and Adi Shamir. Remote Password Extraction from RFID Tags. volume 56, pages 1292–1296, Washington, DC, USA, 2007. IEEE Computer Society. <http://iss.oy.ne.ro/RemotePowerAnalysisOfRFIDTags>.

-
- [oST] National Institute of Standards and Technology. FIPS 180-3 Secure Hash Standard (Draft). <http://www.csrc.nist.gov/publications/PubsFIPS.html>.
- [OST06] Dag Arne Osvik, Adi Shamir, and Eran Tromer. Cache Attacks and Countermeasures: The Case of AES. In David Pointcheval, editor, *Proceedings of CT-RSA 2006*, volume 3860 of *Lecture Notes in Computer Science*, pages 1–20. Springer-Verlag, 2006.
- [OV-05] OV-chipkaart. All about the OV-chipcard, 2005. <http://www.ov-chipkaart.nl/allesoverdeov-chipkaart>.
- [Pas] Paßgesetz PaßG. http://www.gesetze-im-internet.de/bundesrecht/pa_g_1986/gesamt.pdf.
- [Phia] Philips. Data sheet for 4 bit binary ripple counter 74393. <http://www.semiconductors.philips.com/pip/74HC393D#datasheet>.
- [Phib] Philips. Data sheet for 7408 AND gate. <http://www.semiconductors.philips.com/pip/74HC08N>.
- [Phic] Philips. Data sheet for D type flip-flop 7474. <http://www.semiconductors.philips.com/pip/74F74.html#datasheet>.
- [Phid] Philips. Data sheet for monostable multivibrator 74HC/HCT123. <http://www.semiconductors.philips.com/pip/74HCT123D#datasheet>.
- [Phi02] Philips Semiconductors. 3rd generation HITAG family, september 2002. <http://www.silone.com/assets/pdfs/HITAG%203.pdf>.
- [Phi06] Philips. Philips scores in German stadiums. *On the move*, page 3, Mar 2006.
- [Pic07] Pico Technology. *PicoScope 5000 Series User's Guide*, November 2007.
- [Pic08] Pico Technology. PicoScope 5200 USB PC Oscilloscopes, 2008. <http://www.picotech.com/picoscope5200-specifications.html>.
- [Plö08] Henryk Plötz. Mifare Classic - Eine Analyse der Implementierung. Master's thesis, Humboldt-Universität zu Berlin, 2008.
- [Plo08] Thomas Plos. Susceptibility of UHF RFID Tags to Electromagnetic Analysis. In Tal Malkin, editor, *Topics in Cryptology - CT-RSA 2008*, volume 4964 of *Lecture Notes in Computer Science*, pages 288–300. Springer Berlin / Heidelberg, 2008.
- [PM05] Thomas Popp and Stefan Mangard. Masked Dual-Rail Pre-charge Logic: DPA-Resistance Without Routing Constraints. In *CHES 2005*, volume 3659 of *Lecture Notes in Computer Science*, pages 172–186. Springer, 2005.
- [PN09] Henrik Plötz and Karsten Nohl. Legic Prime: Obscurity in Depth, 2009. http://events.ccc.de/congress/2009/Fahrplan/attachments/1506_legic-slides.pdf.

- [PP09] Christof Paar and Jan Pelzl. *"Understanding Cryptography: A Textbook for Students and Practitioners"*. Springer, 2009. <http://www.crypto-textbook.com/>.
- [Pro] Proxmark III. A Radio Frequency IDentification Tool. <http://www.proxmark.org/>.
- [PSP⁺08] Christophe Petit, François-Xavier Standaert, Olivier Pereira, Tal G. Malkin, and Moti Yung. A Block Cipher based Pseudo Random Number Generator Secure Against Side-Channel Key Recovery. In *ASIACCS 2008*, pages 56–65. ACM, 2008.
- [Qui08] Jean-Jacques Quisquater. Why the Bellcore attack is not working against an ENIGMA machine Parts 1 and 2, 2008. <http://rump2008.cr.yp.to/875b25bf93b0e9f98e82cc3971f7c7bb.pdf>.
- [RCT05] Melanie R. Rieback, Bruno Crispo, and Andrew S. Tanenbaum. RFID Guardian: A Battery-Powered Mobile Device for RFID Privacy Management. In *Information Security and Privacy, 10th Australasian Conference (ACISP)*, volume 3574 of *Lecture Notes in Computer Science*, pages 184–194, 2005.
- [RCT06a] Melanie R. Rieback, Bruno Crispo, and Andrew S. Tanenbaum. Is Your Cat Infected with a Computer Virus? In *Conference on Pervasive Computing and Communications (PerCom)*, pages 169–179. IEEE Computer Society, 2006.
- [RCT06b] Melanie R. Rieback, Bruno Crispo, and Andrew S. Tanenbaum. RFID Malware: Truth vs. Myth. *IEEE Security and Privacy*, 4:70–72, 2006.
- [RCT06c] Melanie R. Rieback, Bruno Crispo, and Andrew S. Tanenbaum. The Evolution of RFID Security. volume 5, pages 62–69, Jan-Mar 2006.
- [Ris] Privacy issues with new digital passport. <http://www.riscure.com/news/passport.html>.
- [Rob] Harko Robbroch. ePassport Privacy Attack, Presentation at Cards Asia Singapore, April 26, 2006. <http://www.riscure.com>.
- [Rv10] Kasper Bonne Rasmussen and Srdjan Čapkun. Realization of RF Distance Bounding. In *19th USENIX Security Symposium*, pages 389–402. USENIX Association, 2010. http://www.usenix.org/events/sec10/tech/full_papers/Rasmussen.pdf.
- [SA02] Sergei P. Skorobogatov and Ross J. Anderson. Optical Fault Induction Attacks. In *CHES*, volume 2523 of *Lecture Notes in Computer Science*, pages 2–12. Springer, 2002.
- [Sav10] Savi Technology, Inc. Data Sheets of Savi ST-6XX Active RFID Tags, 2010. <http://www.savi.com>.

-
- [Sch01] Albrecht Schmidt. Serialterm v1.1, 2001. <http://www.comp.lancs.ac.uk/~albrecht>.
- [Sch08] Jörn-Marc Schmidt. Differential Fault Analysis - Final Report. Technical report, TU Graz, June 2008.
- [Sha06] K. Sam Shanmugam. *Digital & Analog Communication Systems*, chapter 8.3.2. Wiley-India, 2006.
- [Sie04a] Siemens. CerPass, 2004. www.cee.siemens.com/web/austria/de/industry/bt/systeme/gebaeudesicherheit/zutritts/cerpass/Documents/scuploadfilecerpass1359337.pdf.
- [Sie04b] Siemens. SIPOrt, 2004. www.cee.siemens.com/web/austria/de/industry/bt/systeme/gebaeudesicherheit/zutritts/siport/Documents/scuploadfilesiport1359491.pdf.
- [Sie07] Siemens. SiPass. www.cee.siemens.com/web/austria/de/industry/bt/produkte/gebaeudesicherheit/zutrittskontrolle/Documents/scuploadfileBRSiPassintegratedde1447274.pdf, 2007.
- [SIL] SILCA. Vehicle guide legend. <http://www.silca.de/media/171934/v1/File/nr-05-2010-vehicle-guide-legend.pdf>.
- [SLFP04] Kai Schramm, Gregor Leander, Patrick Felke, and Christof Paar. A Collision-Attack on AES: Combining Side Channel- and Differential-Attack. In *Cryptographic Hardware and Embedded Systems - CHES 2004*, volume 3156 of *Lecture Notes in Computer Science*, pages 163–175. Springer, 2004.
- [sLL] scorpio-lk Ltd. Tango programmer. http://www.scorpio-lk.com/eng/tango_main_eng.html.
- [Smi97a] Ph.D. Steven W. Smith. *The Scientist and Engineer's Guide to Digital Signal Processing*. California Technical Publishing, 1st edition, 1997. <http://www.dspguide.com/pdfbook.htm>.
- [Smi97b] Ph.D. Steven W. Smith. *The Scientist and Engineer's Guide to Digital Signal Processing*, chapter 20: Chebyshev Filters. California Technical Publishing, 1st edition, 1997.
- [Smi97c] Ph.D. Steven W. Smith. *The Scientist and Engineer's Guide to Digital Signal Processing*, chapter 16: Windowed-Sinc Filters. California Technical Publishing, 1st edition, 1997.
- [Spe] Spectra Security. Cotag Website. www.cotag.co.uk.
- [ST 05] ST Microelectronics. Data Sheet for STP10NK50Z, N-Channel Zener-Protected MOSFET, 2005.
- [Sta05] W. Stallings. *Cryptography and Network Security: Principles and Practice*. Prentice Hall, 4th edition, 2005.

- [STA⁺10] Jörn-Marc Schmidt, Michael Tunstall, Roberto Maria Avanzi, Ilya Kizhvatov, Timo Kasper, and David Oswald. Combined implementation attack resistant exponentiation. In *LATINCRYPT*, volume 6212 of *Lecture Notes in Computer Science*, pages 305–322. Springer, 2010.
- [Sti05] Douglas R. Stinson. *Cryptography: Theory and Practice*. CRC press, 3rd edition, 2005.
- [SvdS08] Pieter Siekerman and Maurits van der Schee. Security Evaluation of the disposable OV-chipkaart v1.7. Technical report, University of Amsterdam, 2008. <http://staff.science.uva.nl/~delaat/sne-2006-2007/p41/Report.pdf>.
- [SWP03] Kai Schramm, Thomas J. Wollinger, and Christof Paar. A new class of collision attacks and its application to DES. In *Proceedings of FSE'03*, volume 2887 of *Lecture Notes in Computer Science*, pages 206–222. Springer, 2003.
- [TAV02] Kris Tiri, Moonmoon Akmal, and Ingrid Verbauwhede. A Dynamic and Differential CMOS Logic with Signal Independent Power Consumption to Withstand Differential Power Analysis on Smart Cards. In *ESSCIRC 2002*, pages 403–406, 2002.
- [Tee08] Wouter Teepe. Making the Best of Mifare Classic Update. <http://www.sos.cs.ru.nl/applications/rfid/2008-thebest-updated.pdf>, December 2008.
- [Tex] Texas Instruments. Texas Instruments to deliver RFID solution for MasterCard PayPass. http://www.ti.com/rfid/docs/news/news_releases/2005/rel01-17-05a.shtml.
- [Tex04] Texas Instruments. HF Antenna Cookbook Technical Application Report, 2004. <http://www.ti.com/rfid/docs/manuals/appNotes/HFAntennaCookbook.pdf>.
- [The] The Hardware Side of Cryptography. Fast AES Implementation on PIC18F4550. <http://edipermadi.wordpress.com/2008/06/17/fast-aes-implementation-on-pic18f4550>.
- [THM07] Stefan Tillich, Christoph Herbst, and Stefan Mangard. Protecting AES Software Implementations on 32-Bit Processors Against Power Analysis. In Jonathan Katz and Moti Yung, editors, *Proceedings of ACNS 2007*, volume 4521 of *Lecture Notes in Computer Science*, pages 141–157. Springer-Verlag, 2007.
- [TI] Texas Instruments. Timeline of RFID products. <http://www.ti.com/rfid/timeline/timeline.shtml>.
- [Tou] Touchatag. Touchatag RFID Reader. <http://www.touchatag.com/>.
- [Tra03] Transport for London. What is Oyster? <http://www.tfl.gov.uk/tickets/oysteronline/2732.aspx>, 2003.

-
- [TS01] U. Tietze and Ch. Schenk. *Halbleiter-Schaltungstechnik*. Springer, eleventh edition, 2001.
- [TSG02] Elena Trichina, Domenico De Seta, and Lucia Germani. Simplified Adaptive Multiplicative Masking for AES. In Burton S. Kaliski Jr., Çetin Kaya Koç, and Christof Paar, editors, *Proceedings of CHES 2002*, volume 2523 of *Lecture Notes in Computer Science*, pages 187–197. Springer-Verlag, 2002.
- [Ver] R. Verdult. Proof of concept, cloning the ov-chip card. <http://www.sos.cs.ru.nl/applications/rfid/2008-concept.pdf>.
- [Voc04] Ing. Josef Vochyan. Notch Filter on Higher Frequency - Structure Wien-Robinson. 2004. http://www.feec.vutbr.cz/EEICT/2004/sbornik/03-Doktorske_projekty/01-Elektronika/35-xvochy00.pdf.
- [VWG07] Dennis Vermoen, Marc F. Witteman, and Georgi Gaydadjiev. Reverse Engineering Java Card Applets Using Power Analysis. In *WISTP*, 2007.
- [Web] Webpage, last visited Juli 2011. Program Code for KeeLoq Decryption. <http://www.pic16.com/bbs/dispbbs.asp?boardID=27&ID=19437>.
- [WG04] Guido Walz and Barbara Grabowski, editors. *Lexikon der Statistik: mit ausführlichem Anwendungsteil (German Edition)*. Spektrum Akademischer Verlag, first edition, June 2004. <http://amazon.com/o/ASIN/3827414237/>.
- [WMMT99] Hideo Watanabe, Manabu Matsubara, Akira Matsuuba, and Norio Tsuruta. Development for Vehicle Anti-Theft System. In *Fujitsu Ten Technical Journal*, volume 12, 1999. <http://www.fujitsu-ten.com/business/technicaljournal/pdf/12-4E.pdf>.
- [WO] 10th September 2009 Wallstreet Online. Sokymat automotive changes name to SMARTRAC TECHNOLOGY GERMANY GmbH. <http://www.wallstreet-online.de>.
- [WW04] Jason Waddle and David Wagner. Towards Efficient Second-Order Power Analysis. In *CHES 2004*, volume 3156 of *Lecture Notes in Computer Science*, pages 1–15. Springer, 2004.
- [Xil05] Xilinx. Spartan-3 FPGA Family: Complete Data Sheet, DS099, January 2005. <http://www.xilinx.com>.
- [YWV⁺05] Shengqi Yang, Wayne Wolf, Narayanan Vijaykrishnan, Dimitrios N. Serpanos, and Yuan Xie. Power Attack Resistant Cryptosystem Design: A Dynamic Voltage and Frequency Switching Approach. In *DATE 2005*, pages 64–69. IEEE Computer Society, 2005.

List of Tables

5.1	Memory structure of Mifare Ultralight according to [NXP10a].	87
5.2	Data manipulation commands of Mifare Ultralight.	87
5.3	Messages exchanged during an authentication protocol run between a reader R and a Mifare DESfire EV1 card C.	89
5.4	Execution times of crucial Crypto1 functions.	92
5.5	Durations of 3DES and AES-128 encryption and decryption functions implemented on the Chameleon.	93
6.1	Proprietary command set of a Mifare Classic reader.	105
6.2	A typical authentication protocol run between a reader R and a Mifare Classic C.	108
7.1	Messages exchanged during an authentication protocol run between a reader R and a Mifare DESfire card C.	138
8.1	Counting clock cycles required by selected conditional branches, as a function of their input parameters.	179
8.2	Worst case times for the brute force attack on KEELOQ	191
9.1	Special Parameters for Issuing Passports in Germany and the Netherlands.	212
9.2	Capabilities of the Adversaries.	212
9.3	Eavesdropping settings and known information for a cryptanalytical attack.	213
9.4	Use cases and estimated entropy for Germany and the Netherlands.	214
9.5	Results for the practical implementation of some scenarios.	218
B.1	Random values required for an implementation that is secured with dummy rounds and/or shuffling.	243
B.2	Estimated memory consumption and cycle count for different AES implementations with masking countermeasure for using one mask / four masks.	244
C.1	Values for the PPC.	248
C.2	Encoding of the data link layer.	254
C.3	Memory maps for password or crypto mode of a HITAG 2 transponder.	256
C.4	The configuration byte of HITAG 2 transponders.	257
C.5	Bit 1 and 2 of the configuration byte determine the operation mode and physical layer of the tag.	257
C.6	Multiplexors making the output function in HITAG 2	259
C.7	HITAG2 function f_a^4 represented as a truth table.	259
C.8	Example on how to calculate the ANF of function f_a^4	260

F.1	Pinout for programming the RFID tool with the AVR Dragon ISP interface . . .	275
G.1	List of source files of the project with a brief description of their content. . . .	286
G.2	Set of data required for the differential attack on the Mifare Classic sector key. .	290
G.3	Set of data required for the nested authentication attack.	290
G.4	Runtimes of critical code segments	295
H.1	Payment management data stored on an ID Card.	297
H.2	Cards used for the practical system-level tests.	298
H.3	Monetary transactions associated with the test “Payments with Cloned Cards”. .	298
H.4	Monetary transactions associated with “Tampering with the Credit Balance”. . .	299
H.5	Monetary transactions associated with the test “Generation of New Cards”. . . .	299
H.6	Monetary transactions associated with the test “Cashback”.	299
H.7	Monetary transactions associated with the test “Using a Clone After Cashback”. .	300
H.8	Recharges of the genuine ID Card.	300
H.9	Monetary transactions associated with the test “Blacklisted ID Card”.	300
J.1	Components for building a Chameleon	308

List of Figures

1.1	Car theft rate in Germany	2
1.2	Differences in the computational power and the security level of wireless devices	3
2.1	Fixed Code Scheme	26
2.2	Rolling Code Scheme	27
2.3	Details of a Rolling Code Scheme in a transponder	27
2.4	Details of a rolling-code scheme in a base station.	28
2.5	Synchronization Code Protocol	29
2.6	Challenge-Response Protocol	30
2.7	Mutual Authentication Protocol	31
2.8	CBC Mode	32
2.9	Operation principle of passive RFID	35
2.10	A sample data sequence encoded with a Miller code.	38
2.11	ISO 14443: Measured waveforms of the modulations used to transmit data	38
2.12	Sample Manchester-encoded data sequence transmitted from card to reader.	39
2.13	Short frame format for transmitting short commands with a length of 7 bits.	39
2.14	Standard frame format with (odd) parity for transmitting longer data blocks.	40
3.1	Overview of the field of cryptanalysis	50
3.2	Simple power analysis of an implementation of RSA.	56
4.1	System Overview: RFID Reader and Fake Tag.	69
4.2	Operation Principle of the RFID-Reader	70
4.3	Operation Principle of the Fake-Tag	72
4.4	Manchester-encoded card responses with a faked ATS command	75
4.5	USB scope for digitizing data (left) and near-field probes for acquiring EM emanations (right).	76
4.6	Architecture of COPACOBANA	81
5.1	The stand-alone RFID emulation device “Chameleon”.	85
5.2	Operating Principle of the Chameleon.	86
5.3	Illustration of the CBC -modes used in DESfire encryptions and decryptions.	91
5.4	Tests with the RFID reader installed in the access control system	96
6.1	Memory structure of a Mifare Classic 1K card.	103
6.2	Top-level structure of the CRYPTO1 stream cipher.	106
6.3	Structure of the internal Mifare Classic PRNG	106
6.4	Principle of the encryption mechanism of Mifare Classic.	110
6.5	Setup for card-only analyses of Mifare Classic.	113

6.6	Combining the Differential Attack with Nested Authentication	117
6.7	UID emulation with Chameleon.	121
7.1	Frequency spectrum of the remote power analysis model	141
7.2	Frequency spectrum of the additive power model	142
7.3	Measurement setup for analyzing contactless smartcards.	142
7.4	Measured frequency response of the RLC notch filter	144
7.5	Principle of the carrier subtracter	145
7.6	Phase response of a first-order allpass filter	146
7.7	Phase shift and addition circuit	147
7.8	Generating special DES input patterns	149
7.9	Simplified excerpt of the authentication protocol, as required for an EM attack. .	150
7.10	Difference-of-Means test.	152
7.11	Optimal positioning of the EM near field probe.	152
7.12	Block diagram of incoherent digital amplitude demodulator.	153
7.13	Raw traces and demodulated traces, with and without carrier subtracter	154
7.14	Correlation coefficients (Hamming weight) of the plaintext and ciphertext bytes. .	155
7.15	Overview of the order of the internal operations of a Mifare DESfire.	155
7.16	Part of a demodulated trace of a 3DES encryption.	156
7.17	First attempt of a bin-wise CPA with peak extraction.	157
7.18	Correlation coefficients for a full-key recovery by means of CPA	158
8.1	Block diagram of the KeeLoq encryption (a) and decryption (b).	164
8.2	Generation of KEELOQ hopping codes	166
8.3	Key derivation schemes	167
8.4	Steps taken for a key recovery by means of DPA.	169
8.5	Power consumption of a HCS IC contained in a remote control	172
8.6	Results: Power analysis of KeeLoq in hardware	173
8.7	Number of measurements required for revealing the secret key of a HCS201 IC. .	174
8.8	Correlation coefficient of the correct key in a CPA attack on the software imple- mentation of the KEELOQ decryption	176
8.9	A source code for a KEELOQ decryption as proposed by Microchip	177
8.10	Visual inspection of power traces of the KEELOQ cipher.	178
8.11	Number of cycles required for the execution of an exemplary implementation of the KEELOQ decryption, depending on the conditional branches.	180
8.12	Extracting peaks of the power consumption trace of a PIC microcontroller exe- cuting KeeLoq.	181
8.13	An example for the correlation coefficients in vector $\mathcal{C}$	182
8.14	Two exemplary reference patterns resulting in different correlation vectors.	183
8.15	Screenshot of a GUI for a KeeLoq eavesdropping attack.	187
8.16	Principle of the hardware architecture for an exhaustive key search on the basis of two intercepted KEELOQ hopping codes.	190
8.17	Generating Dynamic Codes with the proposed PRNG-based RKE system.	193
8.18	Structure of the basic PRNG.	194
8.19	Final, improved structure of the PRNG-core of our RKE system	197

9.1	An exemplary MRZ of the German E-Passport.	205
9.2	Derivation of keys for the Basic Access Control.	206
9.3	Basic Access Control Protocol	207
9.4	Architecture of the attack system	208
9.5	Structure of one FPGA (left) and details about the engines for attacking K_{ENC} (center) and K_{MAC} (right).	216
9.6	Details of the internal structure of an encryption engine for targeting K_{ENC} . . .	217
B.1	A promising target for the power analysis of AES.	232
B.2	SubBytes transformation with masking countermeasure	234
B.3	Securing the AES with Multiplicative Inversion	235
B.4	Shuffled AES-State	241
B.5	AES-State with dummy values	242
B.6	AES-State with dummy values and shuffled AES-State	242
C.1	The customized 125 kHz RFID reader	250
C.2	Data transmission from transponder to base station (“reading”).	255
C.3	Overview of the different modes of operation of the HITAG 2 protocol.	256
C.4	HITAG 2 non-linear output function	258
C.5	HITAG 2 LFSR initialization during the initial setup	260
C.6	HITAG 2 state randomization during the initial setup	261
C.7	HITAG 2 keystream generation	261
C.8	Mutual authentication of the HITAG 2 protocol (Password Mode).	262
C.9	Mutual authentication of the HITAG 2 protocol (Crypto Mode).	263
C.10	Illustration of the READ_PAGE command.	263
C.11	Illustration of the WRITE_PAGE command.	264
C.12	A recorded sample communication with a HITAG 2 transponder	265
C.13	Listening Window (LIW) of Megamos/Sokymat Magic Transponders	266
C.14	Differences in LIWs	266
C.15	Porsche keys	267
C.16	Volkswagen Golf III key and the built-in transponder.	268
D.1	Adaptor for emulating smartcards (left) and modified smartcard reader for power analysis (right)	270
E.1	Optical fault injection module.	271
E.2	Injecting faults with sparks.	271
F.1	Main screen of Atmel AVR Studio 4	274
F.2	Atmel AVR Dragon programming device, Source: [Atmc].	274
F.3	Main screen of HTerm v0.6.5(beta).	275
G.1	Decoding of Miller-coded data visualized as a finite-state machine.	279
G.2	Finite-state machine defining state transitions in the card emulation mode. . . .	284
G.3	Finite-state machine defining state transitions for emulating a Mifare Classic. . .	285
G.4	Default configuration of the access condition bits permitting rewriting of a Mifare Classic.	288

List of Figures

I.1	Signals for difference amplifier circuit	301
I.2	Response times during a DESfire authentication and a screenshot of the Live DOM test utility.	302
I.3	Wiring diagram of the instrumentation amplifier circuit	303
I.4	Circuit diagram of the RLC and Wien-Robinson notch filter circuits	304
I.5	Simulated frequency response of an ideal RLC notch filter	304
J.1	Schematics of the microcontroller and the USB interface.	307
J.2	Circuit diagrams of the power supply and the (de)modulation circuitry of the Chameleon	307
J.3	Schematic of the developed 125 kHz reader	309
J.4	Wiring diagram of the RFID Tool	310
J.5	Layout of the RFID Tool (before creating the ground planes).	311

Chair for Embedded Security (EMSEC)
Faculty of Electrical Engineering and Information Technology
Ruhr-University Bochum
ID 2/621, Universitätsstraße 150
44780 Bochum, Germany

E-Mail: Timo.Kasper@rub.de



Timo Kasper

Personal Details

- Date of birth: 22.02.1979
- Place of birth: Bochum, Germany
- Researcher at the Horst Görtz Institute for IT Security (HGI)

Research and Teaching

Research Activities

Since October 2006: Research assistant at the Chair for Embedded Security (Prof. Dr.-Ing. Christof Paar)

Fields of Expertise:

- Practical security analyses of embedded cryptographic systems
- Implementation attacks (power analysis, fault injection, reverse engineering)
- Security of RFID and wireless systems
- Implementation of countermeasures
- Reviewer for various workshops, conferences, journals, and books, e.g., CHES, Eurocrypt, RFIDSec, Elsevier Journal of Systems and Software, Springer Transactions on Computational Science
- Member of program committee for [Workshop on RFID Security 2008](#) and [2013](#)
- Publications: see list of publications

Teaching Experience

Faculty of Information Technology, Czech Technical University (CVUT), Prague, Czech Republic:

- *Lecture* “MI-SCA – Mikroprocesory vestavěné do čipových karet: Efektivní implementace a útoky postranními kanály” (held in English), *Contents*: Implementations of ciphers on smartcards, measuring with an oscilloscope, extracting secret keys via side-channel analysis. Summer term 2012.

Bonn-Rhine-Sieg University of Applied Sciences:

- *Lecture* “Informationssicherheit”, *Contents*: Applied cryptography, stream ciphers, block ciphers, hash functions, signature schemes, and security of operating systems. Summer term 2010.

Ruhr-Universität Bochum (since 2006):

- *Lab course* “Embedded Smartcard Microcontrollers”, *Contents*: Efficient implementations of symmetric cryptography on embedded microcontrollers.
- *Lab course* “FPGAs in IT Security”, *Contents*: Introduction to VHDL and practical realization of secure, optimized implementations of cryptographic algorithms on FPGAs.
- *Lab course* “Side-Channel Attacks”, *Contents*: Side-channel analysis (SPA, DPA, CPA), power models, measuring information leakage, key-recovery attacks, and implementing countermeasures.
- *Assistant for lectures* “Implementation of Cryptographic Schemes” (Parts I and II) by Prof. Paar, *Contents*: Efficient arithmetic with long numbers, programming of cryptographic algorithms in C, implementation attacks, efficient implementations of block ciphers in software, realizing finite field arithmetic in software and hardware.
- *Supervision of students*: Diploma / Master’s / Bachelor’s theses, seminars, and study projects.

International School of IT Security AG ([ISITS](#)), Bochum (since 2008):

- *E-learning courses* “Implementation of Cryptographic Schemes” I and II. *Contents*: see above.

RFID Security Workshop (RFIDSec) 2011, Amherst (MA), USA:

- *Tutorial* “Hands-on Side Channel Attacks against Smart Cards and Other Tokens”,
Contents: Side-channel analysis in theory and practice. June 2011.

25th Chaos Communication Congress, Berlin:

- DPA (Differential Power Analysis) *workshop* (with Thomas Eisenbarth). December 2008.

Education and Employment

First Place Award

- Diploma thesis “Embedded Security Analysis of RFID Devices”: 1st prize of the IT-Security Research Award 2006 at the Competence Center for Applied Security Technology (CAST), Darmstadt

Studies

- September 2011:
PhD defense and reception of doctor's degree, due to this thesis
- January 2007:
Acceptance at the cross-faculty Research School of the Ruhr-University Bochum
- October 1999 - July 2006, Ruhr-University Bochum, Germany and University of Sheffield, UK:
Studies of Electrical Engineering and Information Technology, Dipl.-Ing. (Master's) degree in July 2006

Work Experience

- 2012, Bochum: Foundation of [Kasper & Oswald GmbH](#)
- July 2005 - August 2006, [ELMOS](#) Semiconductor AG, Dortmund:
Testing of ADC and DAC, evaluation, documentation, measurement setup, software development
- November 2004 - March 2005, Institute for Electrical Engineering and Plasma Technology, RUB:
Development of software for controlling the process sequence of a plasma reactor (with Labview), acquisition and analysis of acquired reactor data with a Langmuir probe
- June - December 2003, Lehrstuhl für Mess- und Schaltungstechnik, RUB:
Development of a circuit for driving a piezo actuator, analyses of measurements of piezo actuators.
- November 2001 - January 2004, Electronic Workshop (EWS):
Realization of electronic-projects for students. Tasks: design of circuit diagrams; simulation with PSpice; creation of the layout; manufacture, assembly and testing of printed circuit boards; and documentation. Projects, e.g., MP3-player, digital audio storage, experimental microcontroller boards (AVR / 8051).
- 1994 - 1998, Schmitt Elektronik / [Elektronik Wunderland](#), Bochum:
Sales and repairs of electrical devices, computer administration, advertisement.

Timo Kasper – Publications, Talks and other Research Activity

1) Peer-Reviewed Articles in Books and Journals

- Chapter in book “High-Performance Computing using FPGAs”, Springer, to appear 2013
- “A Versatile Framework for Implementation Attacks on Cryptographic RFIDs and Embedded Devices”, T. Kasper, D. Oswald, and C. Paar; Transactions on Computational Science 10, Springer, 2010
- “Cryptanalysis with COPACOBANA”, T. Güneysu, T. Kasper, M. Novotny, C. Paar and A. Rupp; Transactions on Computers, Volume 57, Issue 11, IEEE, 2008

2) Peer-Reviewed Papers in Conference Proceedings

- “Security of Wireless Embedded Devices in the Real World”, T. Kasper, D. Oswald, and C. Paar; Information Security Solutions (ISSE), Prague, Czech Republic, 2011
- “Wireless Security Threats: Eavesdropping and Detecting of Active RFIDs and Remote Controls in the Wild”, T. Kasper, D. Oswald, and C. Paar; SoftCOM, Croatia, 2011
- “On the Vulnerability of FPGA Bitstream Encryption against Power Analysis Attacks - Extracting Keys from Xilinx Virtex-II FPGAs”, A. Moradi, A. Barengi, T. Kasper, and C. Paar; ACM CCS, Chicago, USA, 2011
- “Side-Channel Analysis of Cryptographic RFIDs with Analog Demodulation”, D. Oswald, T. Kasper, and C. Paar; RFIDSec, Amherst, USA, Springer LNCS, 2011
- “Chameleon: A Versatile Emulator for Contactless Smartcards”, T. Kasper, I. von Maurich, D. Oswald, and C. Paar; ICISC, Seoul, Korea, Springer LNCS, 2010
- “Cloning Cryptographic RFID Cards for 25\$”, T. Kasper, I. von Maurich, D. Oswald, and C. Paar; WISSec, Nijmegen, the Netherlands, 2010
- “Combined Implementation Attack Resistant Exponentiation”, J.-M. Schmidt, M. Tunstall, R. M. Avanzi, I. Kizhvatov, T. Kasper, and D. Oswald; LATINCRYPT, Springer LNCS, 2010
- “All You Can Eat or Breaking a Real-World Contactless Payment System”, T. Kasper, M. Silbermann, and C. Paar; Financial Cryptography and Data Security, Tenerife, Spain, Springer LNCS, 2010
- “A New Remote Keyless Entry System Resistant to Power Analysis Attacks”, A. Moradi, T. Kasper; ICICS, Macau, China, IEEE, 2009
- “Cryptanalysis of KeeLoq with COPACOBANA”, M. Novotný and T. Kasper; SHARCS, Lausanne, Switzerland, 2009
- “KeeLoq and Side-Channel Analysis - Evolution of an Attack”, C. Paar, T. Eisenbarth, M. Kasper, T. Kasper, and A. Moradi; FDTC, Lausanne, Switzerland, Springer LNCS, 2009
- “EM Side-Channel Attacks on Commercial Contactless Smartcards using Low-Cost Equipment”, T. Kasper, D. Oswald, and C. Paar; WISA, S. Korea, Springer LNCS, 2009
- “New Methods for Cost-Effective Side-Channel Attacks on Cryptographic RFIDs”, T. Kasper, D. Oswald, and C. Paar; RFIDSec, Leuven, Belgium, 2009
- “Breaking KeeLoq in a Flash: On Extracting Keys at Lightning Speed”, M. Kasper, T. Kasper, A. Moradi, and C. Paar; AFRICACRYPT, Gammarth, Tunisia, Springer LNCS, 2009
- “On the Power of Power Analysis in the Real World: A Complete Break of the KeeLoq Code Hopping Scheme”, T. Eisenbarth, T. Kasper, A. Moradi, C. Paar, M. Salmasizadeh, and M.T.M. Shalmani; CRYPTO, Santa Barbara, USA, Springer LNCS, 2008
- “Susceptibility of eSTREAM Candidates towards Side-Channel Analysis”, B. Gierlichs, L. Batina, C. Clavier, T. Eisenbarth, A. Gouget, H. Handschuh, T. Kasper, K. Lemke-Rust, S. Mangard, A. Moradi, and E. Oswald; State of the Art of Stream Ciphers (SASC), Lausanne, Switzerland, 2008
- “E-Passport: Cracking Basic Access Control Keys”, Y. Liu, T. Kasper, K. Lemke-Rust, and C. Paar; OTM (On the Move), Vilamoura, Portugal, Springer LNCS, 2007
- “Collision Attacks on AES-based MAC: Alpha-MAC”, A. Biryukov, A. Bogdanov, D. Khovratovich, and T. Kasper; CHES, Vienna, Austria, Springer LNCS, 2007
- “A Lightweight Hardware Implementation of the Stream Cipher VEST-4”, T. Gendrullis, T. Kasper, and C. Paar; WeWORC, Bochum, Germany, 2007
- “An Embedded System for Practical Security Analysis of Contactless Smartcards”, T. Kasper, D. Carluccio, and C. Paar; WISTP, Crete, Greece, Springer LNCS, 2007

3) Other Publications

- „Praktische Angriffe auf die Bitstromverschlüsselung von Xilinx FPGAs“, M. Kasper, T. Kasper, A. Moradi, and C. Paar; Datenschutz und Datensicherheit - DuD, Issue 11/2011
- „Türöffnen, Identifizieren und Bezahlen per Funk? Ein Rückblick über Angriffe auf kommerzielle kontaktlose Systeme“, T. Kasper, D. Oswald, and C. Paar; Tagungsband Innosecure, 2012
- „Seitenkanalanalyse kontaktloser Smart-Cards“, T. Kasper, D. Oswald, and C. Paar; Datenschutz und Datensicherheit - DuD, Issue 11/2011
- “KeeLoq”, T. Eisenbarth, T. Kasper, C. Paar, and S. Indesteege; Encyclopedia of Cryptography and Security, Second Edition, Springer, 2011
- „Sicherheit moderner Funktüröffnersysteme“, T. Eisenbarth, T. Kasper, and C. Paar; Datenschutz und Datensicherheit - DuD, Issue 8/2008
- „Torschlusspanik - Sicherheitslücken durch Seitenkanalattacken“, T. Eisenbarth, T. Kasper, and C. Paar; Magazin für professionelle Informationstechnik (iX), 9/2008

Selected Contributions in other Media

- „Die Funkchips und der Datenschutz“, Westpol, 28.06.2009, WDR Television
- „Der moderne Einbrecher kommt per Funk - Forscher knacken den Code von Funktüröffner-Systemen“, Leonardo, 17.07.2008, WDR5 Radio
http://www.wdr5.de/fileadmin/user_upload/Sendungen/Leonardo/2008/07/Manuskripte/ms080717_Funktueroeffner_Schwarzer__1_.pdf
- „Funkschlüssel: Code geknackt“, Markt, 07.04.2008, WDR Television
- „Forscher knacken den Code von Auto- und Garagentüren“, 05.04.2008, 3Sat NANO
http://www.emsec.rub.de/media/crypto/attachments/files/2011/04/keeloq_nano.mp4

4) Invited Talks

- “Security Analysis of Wireless Systems in the Real World”
TWISC Annual Workshop – Taiwan Information Security Center, November 2012, Taichung (Taiwan)
- “Physikalische Angriffe gegen Bezahlssysteme”
DuD 2012 Fachkonferenz für Datenschutz und Datensicherheit, Juni 2012, Berlin (D)
- “Best Practices: Lessons Learned from Attacking Commercial Wireless Tokens”
IT-Defense, February 2012, München (D)
- “Security Issues of Cashless Payments with Contactless Smartcards”
SAFE-NL Workshop RFID Security and Privacy, 31.05.2010, Nijmegen (NL)
- “How (In)secure are Real-World Contactless Payment Systems”
Faculty of Information Technology, Czech Technical University, 12.05.2010, Prague (CZ)
- “RFID Security”
CAST-Workshop Embedded Security, 23.04.2009, Darmstadt (D)
- “Messing Around with Garage Doors”
25th Chaos Communication Congress, 27.12.2008, Berlin (D) ([Video](#))
http://media.ccc.de/browse/congress/2008/25c3-3030-en-messing_around_with_garage_doors.html
- “On the Power of Power Analysis in the Real World”
Workshop on Elliptic Curve Cryptography, 23.09.2009, Eindhoven (NL)
- “Praktische Sicherheitsanalyse von Funktüröffnern”
Workshop RFID im Fokus - Sicherheit und Datenschutz, 01.07.2008, Potsdam (D)
- “Schwachstellen und Angriffe - Die Evolution der Sicherheitsmechanismen im ePass”
HGI/PTB-Workshop „E-Sicherheit im E-Government“, 31.03.2008, Bochum (D)
- “Angriffe gegen kontaktlose Protokolle (Theorie und Praxis)”
Praxistag RFID-Sicherheit, 12.12.2007, Oberhausen (D)
- Various other talks in the industry (e.g., Bosch, NXP, Siemens, ELMOS) and in academic institutions, e.g., Bergische Universität Wuppertal (D), Université de Rennes (F)

5) Participation in Selected Workshops and Conferences

- CHES 2012 (Leuven, Belgium)
- Escar 2011 (Dresden, Germany)
- RFIDSec 2011 (Amherst, USA)
- Cryptarchi 2011 (Bochum, Germany)
- Chaos Communication Camp 2011 (Berlin, Germany)
- CRYPTO 2010 (Santa Barbara, USA)
- CHES 2010 (Santa Barbara, USA)
- FDTC 2010 (Santa Barbara, USA)
- Financial Cryptography 2010 (Tenerife, Spain)
- WISSec 2010 (Nijmegen, the Netherlands)
- European Cryptography Day 2010 (Leuven, Belgium)
- Eurocrypt 2009 (Cologne, Germany)
- WISA 2009 (Busan, Korea)
- RFIDSec 2009 (Leuven, Belgium)
- FDTC 2009 (Leuven, Belgium)
- ECC 2009 - Summer School on Elliptic and Hyperelliptic Curve Cryptography 2009 (Eindhoven, the Netherlands)
- SHARCS 2009 (Lausanne, Switzerland)
- CHES 2009 (Lausanne, Switzerland)
- CRYPTO 2008 (Santa Barbara, USA)
- CHES 2008 (Washington DC, USA)
- RFIDSec 2008 (Budapest, Hungary)
- 25th Chaos Communication Congress 2008 (Berlin, Germany)
- CAST Workshops 2007 (Darmstadt, Germany)
- CHES 2007 (Vienna, Austria)
- SHARCS 2007 (Vienna, Austria)
- RFIDSec 2007 (Malaga, Spain)
- „Rückverfolgbarkeit und Fälschungssicherheit von Produkten und Lebensmitteln mit RFID“ 2007 (Bruxelles, Belgium)
- WISTP 2007 (Heraklion, Greece)
- SASC 2007 (Bochum, Germany)
- On The Move 2007 (Vilamoura, Portugal)
- Chaos Communication Camp 2007 (Berlin, Germany)
- Deutscher IT-Sicherheitspreis 2006 (Bochum, Germany)
- RFIDSec Workshop 2006 (Graz, Austria)

A

JE

TO!